

BRICS Astronomy: Data Analysis

Prof. Najam Hasan (Maulana Azad National Urdu University, India)

Dr. Priya Hasan (Maulana Azad National Urdu University, India)

Mr. Daniel Egbo (South Africa Astronomical Observatory, South Africa)

Prof. Reinaldo Rosa (INPE - National Institute for Space Research, Brazil)

Mrs. Narusha Isaacs-Klein (The Inter-university Institute for Data Intensive Astronomy, South A

Dr. Aleksey Dianchenko (Ural Federal University, Russia)

Dr. Nikita Rawat (South Africa Astronomical Observatory, South Africa)

Dr. Jyoti Prakash (The Inter-University Centre for Astronomy and Astrophysics, India)

Mr. Moleboge Lekoloane (South African Astronomical Observatory, South Africa)

Ms. Duduzile Kubheka (South African Astronomical Observatory, South Africa)

Contents

BRICS Astronomy: Data Analysis	13
Authors	15
1 Preface	17
2 Pre-Orientation: Data Analytics in Astronomy	19
2.1 1. Course Introduction	19
2.2 2. Target Audience & Prerequisites	20
2.3 3. Course Objectives	20
2.4 4. Installation Guidelines: Setting Up Your Environment	21
2.5 6. Pre-Course Preparation (Optional)	22
2.6 7. Contact & Support (Pre-Course)	22
3 Get Started	23
3.1 Using Google Colab	23
3.2 Running Locally	23
3.3 How to run codes in Jupyter Notebook	24
4 Variables and Data Types	25
4.1 What is a Variable?	25
4.2 Naming Variables	26
4.3 Data Types	26
4.4 Escape Characters	27

4.5	Comments	28
4.6	Operators	28
4.7	Summary	34
4.8	Further Reading	34
4.9	Exercises	35
5	Data Structures	37
5.1	Tuples	37
5.2	Lists	38
5.3	Sets	41
5.4	Dictionaries	42
5.5	Exercises	44
5.6	Summary	45
6	String Operations in Python for Astronomy	47
6.1	1. Creating and Manipulating Strings	47
6.2	2. Concatenating Strings	47
6.3	Using f-strings for better readability	48
6.4	3. Repeating Strings Using * Operator	48
6.5	4. String Methods in Astronomy	48
6.6	5. Finding the Position of a Substring	49
6.7	6. Counting Occurrences of a Word in a String	49
6.8	7. Formatting Strings	49
6.9	8. Splitting and Extracting Data from Strings	50
6.10	9. Joining Strings Using <code>join()</code>	50
6.11	10. Parsing Coordinates from a String	50
6.12	11. Checking for Substrings	51
6.13	12. Using Strings in SQL Queries for Astronomical Data	51
6.14	Exercise	51
6.15	Summary	52

<i>CONTENTS</i>	5
7 Python Loops	53
7.1 What are Loops?	53
7.2 for Loops: Repeating Actions a Specific Number of Times	54
7.3 while Loops: Repeating Actions Until a Condition is Met	57
7.4 Loop Control: break and continue	59
7.5 Looping Through Lists and Strings	60
7.6 Nested Loops: Loops within Loops	61
7.7 More Practical Astronomy Examples	62
7.8 Exercises	63
7.9 Summary	63
8 Python Functions	65
8.1 Functions: Encapsulation and Reusability	65
8.2 Benefits of Functions:	66
8.3 Function Arguments (Inputs)	67
8.4 Return Values (Outputs)	68
8.5 Docstrings: Function Documentation	69
8.6 Scope: Variable Visibility	70
8.7 Advanced Examples: Applying Functions in Astronomy	71
8.8 Exercises	72
8.9 Summary	73
9 Introduction to Python Classes	75
9.1 Understanding Attributes and Methods	76
9.2 The <code>__init__</code> Method: Initializing Objects	78
9.3 Exercises	86
9.4 Summary	86
10 Files and Error Handling in Python	87
10.1 Opening Files in Python	87
10.2 Reading Data from Files	90
10.3 Writing Data to Files	91

10.4 Error Handling: The <code>try...except</code> Block	92
10.5 Handling Different Exception Types in Python:	93
10.6 The <code>finally</code> Block	95
10.7 Raising Custom Exceptions	96
10.8 Exercises	97
10.9 Summary	97
10.10 Additional Resources	97
11 Modules and Packages in Python	99
11.1 Importing Specific Items from a Module: <code>from...import</code>	99
11.2 Importing with Aliases: <code>as</code>	100
11.3 Exploring Local Modules: <code>string</code> , <code>array</code> , <code>time</code>	100
11.4 Installing External Packages with <code>pip</code>	102
11.5 Using External Packages: <code>numpy</code> , <code>pandas</code>	102
11.6 Managing Packages: <code>pip list</code> , <code>pip show</code> , <code>pip uninstall</code>	104
11.7 Namespaces and Avoiding Naming Conflicts	104
11.8 Exercises	105
11.9 Summary	105
12 NumPy: Handling Numerical Data	107
12.1 Why Numpy	107
12.2 The <code>ndarray</code> : NumPy's Core Object	108
12.3 Array Attributes: What an Array Knows About Itself	110
12.4 Array Indexing and Slicing: Accessing Elements	111
12.5 Basic Operations on Arrays	113
12.6 Putting it Together: Astronomy Examples with NumPy	116
12.7 Saving and Loading NumPy Arrays	119
12.8 Exercises	123
12.9 Summary	124

13 Pandas: Analysing Tabular Data	125
13.1 What is Pandas and the DataFrame	125
13.2 Creating DataFrames	126
13.3 Accessing and Selecting Data	127
13.4 Filtering DataFrames	131
13.5 Basic Operations and Adding New Columns	133
13.6 Reading Data from Files (CSV)	135
13.7 Reading Data from Online Sources (URLs)	136
13.8 Handling Missing Data	138
13.9 Exercises	140
13.10 Summary	141
14 Statistical Analysis	143
14.1 Technical Terms Used	149
14.2 What does the output mean?	151
14.3 Technical Terms	152
14.4 Tips	152
15 Descriptive Statistics	153
15.1 Definitions of descriptive statistics	153
15.2 Data-based estimates of descriptive statistics	158
16 Visualisation: An Introduction to Matplotlib and Seaborn	167
16.1 Matplotlib Basics: Figure and Axes	168
16.2 Common Plot Types with Matplotlib	169
16.3 Line Plots	171
16.4 Bar Plots	173
16.5 Customizing Plots: Labels, Titles, Legends, and Appearance . .	175
16.6 Introducing Seaborn	177
16.7 Exercises	184
16.8 Summary	184

17 Visualising Gaia Data with Matplotlib and Seaborn	187
17.1 Hertzsprung-Russell (HR) Diagram	196
18 Essential Statistics Tutorial	199
18.1 1. Required Libraries	199
18.2 2. Descriptive Statistics	200
18.3 3. Data Visualization	200
18.4 4. Probability Basics	201
18.5 5. Normal Distribution	202
18.6 6. Correlation	203
18.7 7. Hypothesis Testing (t-test)	204
18.8 8. Confidence Intervals	205
18.9 9. Simple Linear Regression	206
18.10 10. Practice Exercise	206
19 Statistics for Astronomers	209
19.1 1. Introduction	209
19.2 2. Probability Distributions in Astronomy	210
19.3 3. Bayesian Inference for Astronomical Problems	210
19.4 4. Survival Analysis for Censored Data	213
19.5 5. Time Series Analysis	215
19.6 6. Multivariate Statistics and Dimensionality Reduction	216
19.7 7. Machine Learning for Astronomical Classification	216
19.8 8. Spatial Statistics	218
19.9 9. Resources for Further Learning	218
19.10 10. Exercises for Practice	219
20 Matplotlib is used for plotting	221
20.1 Matplotlib extra	221
20.2 The first plot	221
20.3 Exercise 1: Use the numpy cheat sheet to calculate the value of the function $7x+2$ between 0 and 10	222
20.4 You can plot two plots in the same graph by calling the plot function twice	223

21 Exercise 2: Create a graph for two functions	227
21.1 To create a new figure, use <code>plt.figure()</code>	227
21.2 Another way to plot is to use the axes object that is got by the <code>plt.subplots</code> function	229
21.3 This becomes useful when you have a lot of graphs to do	230
22 Seaborn.	233
22.1 Seaborn Tutorial	233
23 astropy Constants (SI Units)	267
23.1 Units, Quantities, and Constants in astropy	268
23.2 astropy Units List	271
23.3 Constants	277
23.4 Example: Luminosity of a star-like object using the Stefan– Boltzmann law	279
23.5 Working with <code>astropy.units</code> and NumPy Arrays	282
23.6 Summary: astropy Units, Quantities, and Constants	285
23.7 Assignment: Determining Stellar Distances from Parallax	286
23.8 Part II: Interstellar travel time with Voyager-like speed	288
24 Summary: Core Concepts of <code>astropy.coordinates</code>	289
24.1 Introduction to <code>astropy.coordinates</code>	289
25 Introduction to <code>astropy.table</code>	299
25.1 Summary of <code>astropy.table</code>	304
26 CDS Services	307
26.1 Start Aladin Lite	308
26.2 Find and download tables from VizieR	309
26.3 Bonus: MOCs and Python	313
27 Introduction to Image Analysis and Processing	325
28 FITS	327

29 Scaling or Stretch function	343
30 RGB Composite image	347
30.1 Sloan Digital Sky Survey	347
31 Skyview	349
32 RGB Image using astropy	361
33 Puzzle I	363
34 Saving FITS image	365
35 Time Python	367
35.1 The Concept of Time in Python	367
36 The Time Domain in Astronomy	375
36.1 The Concept of Time in Astronomical Observations	375
36.2 Handling Astronomical Time with Astropy	376
36.3 The Light Curve	378
36.4 Common Light Curve Examples	379
36.5 Sgr A*	381
36.6 Radio light curve	382
36.7 Summary	384
36.8 Additional Resources	385
37 Time-series Analysis for Astronomical Data	387
38 1. What is Time-Series Data?	389
39 2. What Data Products are Available?	391
40 3. Searching for Light Curves	393
40.1 3.1 Downloading a single light curve	395
41 4. Finding periodicity in the light curve	399

<i>CONTENTS</i>	11
42 5. References	403
43 6. Exercise	405
43.1 Additional Resources	405
44 Machine Learning for Astronomical Data Analysis: A Conceptual Introduction	407
44.1 Types of Machine Learning	407
44.2 Common Use Cases in Astronomy	409
44.3 Summary and Next Steps	409
45 Introduction to Scikit-Learn	411
45.1 Loading an example dataset	411
45.2 Shape of the data arrays	412
45.3 Learning and Predicting	414
45.4 Saving the model	421
46 Regression: Photometric Redshifts of Galaxies	423
46.1 Regression Evaluation	428
46.2 Feature Scaling	440
46.3 Summary	444
46.4 Additional Resources	445
47 Hyperparameter Tuning: Optimizing Model Performance	447
47.1 Cross-Validation with KFold and Grid Search	449
47.2 Random Forest	455
47.3 Summary	463
48 Machine Learning Basics for Astronomy	465
49 2. Preprocessing	487
49.1 1. Setup	489
49.2 1.2 Load and Explore Data	490
50 Vizier Data Query	501

51 Classification: Learning Labels of Astronomical Sources	503
52 Astroml Dimensionality Reduction	523
52.1 Dimensionality reduction	523
53 Deep Learning: Classifying Astronomical Images	543
53.1 Neural Network Frameworks	550
53.2 Creating a Neural Network with Keras	552
54 Principal Component Analysis (PCA) - BRICS Tutorial	571
55 Download an example SDSS spectrum FITS file	573
56 Open the FITS file and extract wavelength and flux	575
57 Capstone Project Guidelines – BRICS Astronomy/IDIA Data Analytics	583
57.1 Objective	583
57.2 Project Requirements	583
57.3 Timeline	584
57.4 Suggested Data Sources	584
57.5 Grading Rubric (100 points total)	585
57.6 Submission	586

BRICS Astronomy: Data Analysis

Open Source Course Book

Authors

Prof. Najam Hasan (Maulana Azad National Urdu University, India)
Dr. Priya Hasan (Maulana Azad National Urdu University, India)
Mr. Daniel Egbo (South Africa Astronomical Observatory, South Africa)
Prof. Reinaldo Rosa (INPE - National Institute for Space Research, Brazil)
Mrs. Narusha Isaacs-Klein (The Inter-university Institute for Data Intensive Astronomy, South Africa)
Dr. Aleksey Dianchenko (Ural Federal University, Russia)
Dr. Nikita Rawat (South Africa Astronomical Observatory, South Africa)
Dr. Jyoti Prakash (The Inter-University Centre for Astronomy and Astrophysics, India)
Mr. Moleboge Lekoloane (South African Astronomical Observatory, South Africa)
Ms. Duduzile Kubheka (South African Astronomical Observatory, South Africa)

Chapter 1

Preface

This open-source book compiles the BRICS Data Analytics in Astronomy training materials, organized by week and project guidance. The course is designed for beginners and progresses from foundational programming concepts to practical data analysis and machine learning applications in astronomy.

Chapter 2

Pre-Orientation: Data Analytics in Astronomy

Welcome, Astronomers! We're thrilled you'll be joining our **Data Analytics in Astronomy** training. This short guide provides essential information to help you prepare for the course and ensure you have a smooth start.

2.1 1. Course Introduction

This training is designed to equip you with the foundational Python programming skills necessary to tackle common tasks in astronomy. We'll move from the absolute basics of programming to applying Python concepts to analyze astronomical data, automate calculations, and understand how code can be a powerful tool in your scientific journey.

Our focus will be on practical application, using relatable examples drawn from astronomy wherever possible. We aim to create a supportive learning environment where questions are encouraged. By the end of this course, you'll have a solid base to build upon and the confidence to start using Python in your own astronomical projects.

2.2 2. Target Audience & Prerequisites

- **Primary Audience:** Undergraduate students, researchers, or enthusiasts in astronomy (or related fields) who are **new to programming** or have very limited prior experience with Python.
 - **Programming Level: Beginner.** No prior programming knowledge is required. We will start from the ground up.
 - **Astronomy Level:** A basic interest or background in astronomy is helpful for understanding the examples, but advanced astronomical knowledge is **not** required.
 - **Technical Requirements:** Access to a computer (Windows, macOS, or Linux) with internet access and the ability to install software (see Section 4).
-

2.3 3. Course Objectives

Upon successful completion of this training, you will be able to:

- Understand fundamental programming concepts (variables, data types, operators).
 - Write and execute basic Python scripts.
 - Implement control flow using loops (**for**, **while**) and conditional statements (**if**, **else**).
 - Define and use functions to create reusable and modular code.
 - Read data from and write data to files.
 - Implement basic error handling using **try...except** blocks.
 - Import and utilize standard Python modules (like **math**, **time**).
 - Understand the concept of packages and install external packages (like **numpy**, **pandas**, **astropy** and other packages) using **pip** or **conda**.
 - Apply these Python concepts to solve simple problems relevant to astronomy.
-

2.4 4. Installation Guidelines: Setting Up Your Environment

To participate fully in the hands-on exercises, you must install the necessary software before the first session. We strongly recommend using the **Anaconda Distribution**, as it simplifies the installation of Python and many essential scientific packages.

Steps:

1. Download Anaconda:

- Go to the official Anaconda Distribution download page: <https://www.anaconda.com/download/success>
- Download the installer appropriate for your operating system (Windows, macOS, or Linux). Choose the Python 3.x version (the latest is recommended).

2. Install Anaconda:

- Locate the downloaded installer file.
- Double-click the installer and follow the on-screen instructions.
- **Recommendation:** Unless you have a specific reason not to, we recommend accepting the default settings during installation. This usually includes adding Anaconda to your system's PATH environment variable (though the installer might advise against it for Windows, it often simplifies things for beginners – follow the installer's recommendation if unsure).

3. Verify Installation:

- **Option A (Anaconda Navigator):** After installation, search for and open “Anaconda Navigator”. This is a graphical interface. From Navigator, you should be able to launch “Jupyter Notebook”. If Jupyter Notebook launches in your web browser, your installation is likely successful.
- **Option B (Terminal/Command Prompt):**
 - Open your Terminal (macOS/Linux) or Command Prompt/Anaconda Prompt (Windows).
 - Type `python --version` and press Enter. You should see a Python version corresponding to the Anaconda installation (e.g., `Python 3.9.7`).
 - Type `conda list anaconda$` and press Enter. This checks if the core Anaconda package is installed.
 - Type `jupyter lab` and press Enter. This should launch Jupyter Notebook in your web browser.

4. Troubleshooting:

- Installation issues are common, especially with path configurations.
 - Refer to the official Anaconda documentation for detailed installation guides and troubleshooting tips: <https://docs.anaconda.com/anaconda/install/>
 - If you encounter persistent problems, please reach out using the contact information provided in Section 6 before the first training session.
-

2.5 6. Pre-Course Preparation (Optional)

While not mandatory, you might find it helpful to:

- Familiarize yourself with your computer's file system (how to create folders, find files).
 - Briefly browse a very basic Python tutorial online (e.g., the first few sections of the official Python tutorial at <https://docs.python.org/3/tutorial/>) just to get a feel for the syntax, but don't worry if it doesn't all make sense – we will cover it!
-

2.6 7. Contact & Support (Pre-Course)

If you face difficulties with the software installation or have urgent questions before the training begins, please contact us at through the Slack channel.

Please try to resolve installation issues *before* the first session so we can dive right into the material.

We are looking forward to exploring the universe of Python with you! Get ready for an exciting journey into astronomical programming.

Chapter 3

Get Started

3.1 Using Google Colab

Colab is a free online jupyter notebook service offered by Google that allows users to run Jupyter notebooks in Google Cloud. The service is available at colab.research.google.com.

To run a Jupyter notebook from this course in Google Colab, click the **Open in Colab** button at the top of the notebook to redirect you to a new web page where you can access the notebook in Google Colab.

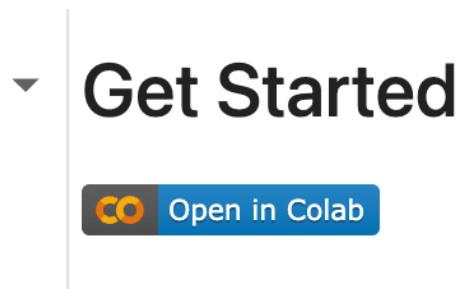


Figure 3.1: Colab Workflow

3.2 Running Locally

To run Jupyter notebooks locally on your machine, you can follow the installations provided in Section 4 of the precourse materials: <https://github.com/>

Danselem/brics__astro/blob/main/precourse.md to open JupyterLab in your default web browser, where you can navigate to the notebook you want to run.

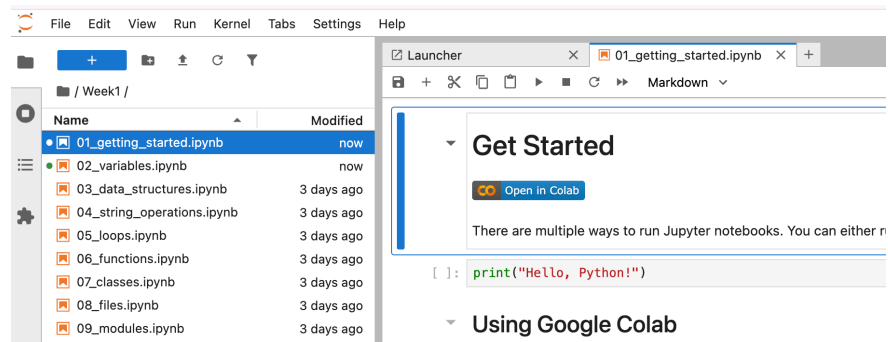


Figure 3.2: image

3.3 How to run codes in Jupyter Notebook

Jupyter Notebooks provide cells where codes can be written.

To run a code, click on the cell to select it, then press **SHIFT+ENTER** or press the play button above.

Run the next cell by selecting it and pressing **SHIFT+ENTER**.

```
print("Hello, Python!")
```

```
Hello, Python!
```

For more on how to run Jupyter notebooks, check out <https://www.codecademy.com/article/how-to-use-jupyter-notebooks>.

Chapter 4

Variables and Data Types

4.1 What is a Variable?

Variable is a common terminology in programming for storing information. In Python, a variable is a symbolic name that references or points to an object. Once an object is assigned to a variable, you can refer to that object by the variable name.

Below is an example of creating a variable that represents the name of a star in astronomy.

```
star_name = "Alpha Centauri"
```

This variable `star_name` now holds the string value Alpha Centauri.

To view the value of the variable, we can use the `print()` function.

```
print(star_name)
```

```
Alpha Centauri
```

Alternatively, we can simply type the variable name in a code cell and run the cell to display the value of the variable.

```
star_name
```

```
'Alpha Centauri'
```

4.2 Naming Variables

When naming variables, you should follow these rules:

- Variable names must start with a letter or an underscore, such as `_`.
- The remainder of the variable name can consist of letters, numbers, and underscores.
- Variable names are case-sensitive, so `star_name` and `Star_Name` are different variables.
- Variable names should be descriptive and meaningful, such as `star_name` instead of `sn`.
- Avoid using Python keywords and built-in functions as variable names, such as `print`, `sum`, `list`, `dict`, `str`, `int`, `float`, `bool`, `set`, `tuple`, `range`, `type`, `object`, `None`, `True`, `False`, `and`, `or`, `not`, `if`, `else`, `elif`, `for`, `while`, `break`, `continue`, `pass`, `def`, `return`, `lambda`, `class`, `import`, `from`, `as`, `with`, `try`, `except`, `finally`, `raise`, `assert`, `del`, `in`, `is`, `global`, `nonlocal`, `yield`, `async`, `await`.

4.3 Data Types

Data types are different ways a data can be represented. Python supports various data types, which are essential to understand before working with astronomy data. The most common data types include:

a) Integers (int): These data representing whole numbers, e.g., 1, 150, -4. It can be positive or negative but do not have a part of a fraction (decimal points).

```
num_features = 20 # Represents the number of features in a catalog
```

b) Floating-point numbers (float): These are data representing numbers with a decimal point, e.g., 3.14, -0.1, 100.5 etc. You can write multiple lines of code in a single code cell. The output will be displayed for the last line of code.

```
RA = 219.9021 # Represents the Right Ascension of Alpha Centauri
Dec = -60.8340 # Represents the Declination of Alpha Centauri
```

c) Strings (str): Strings are representation of characters, e.g., “Hello”, “Astronomy”, “Star”, “Galaxy”.

```
coordinate_system = "Equatorial" # Represents a commonly used coordinate system in As
```

Strings can be enclosed in single quotes (`'`) or double quotes (`"`). You can also use triple quotes (`'''` or `"""`) for multiline strings.

d) Booleans (bool): Booleans represent one of two values: `True` or `False`

```
is_galaxy = True # Represents whether a dataset is a galaxy or not.
```

e) **Lists:** Lists are ordered collections of items, and can be of any data type.

```
coordinates = [  
    219.9021,  
    -60.8340,  
] # A list representing Right ascension and declination of Alpha Centauri
```

f) **Dictionaries (dict):** Dictionaries are collections of key-value pairs.

```
feature_attributes = {  
    "name": "Alpha Centauri A",  
    "B_mag": 0.4,  
    "type": "Star",  
    "coordinates": [219.9021, -60.8340,]  
}
```

4.4 Escape Characters

Escape characters are used to insert characters that are illegal in a string. For example, you can use the escape character `\n` to insert a new line in a string.

```
print("Hello World!\nThis is a Python script.")
```

```
Hello World!  
This is a Python script.
```

Another common escape character is `\t`, which inserts a tab in a string.

```
print("This is the first line.\n\tThis is the second line. It is indented.")
```

```
This is the first line.  
    This is the second line. It is indented.
```

If you want to include a single quote in a string, you can wrap the string in double quotes. Alternatively, you can use the escape character `\'` to include a single quote in a string.

```
print("What's your name?")
```

```
What's your name?
```

4.5 Comments

Comments are used to explain a code and make it readable. In Python, comments start with the hash # symbol. Everything after the # symbol on a line is ignored by the Python interpreter.

```
# This is a comment  
num_points = 120 # This is an inline comment
```

4.6 Operators

Operators are special symbols or keywords in Python that perform operations on values (called operands). Think of them as the mathematical symbols (+, -, *, /) and logical connectors (and, or, not) you use in calculations and decision-making, but adapted for programming. Understanding operators is fundamental to performing calculations, comparing values, and controlling the flow of your scientific problems.

4.6.1 Arithmetic Operators

These are used for standard mathematical operations.

- + : Addition (Adds operands)
- - : Subtraction (Subtracts the right operand from the left)
- * : Multiplication (Multiplies operands)
- / : Division (Divides the left operand by the right, result is always a float)
- // : Floor Division (Divides and rounds down to the nearest whole number)
- % : Modulus (Returns the remainder of the division)
- ** : Exponent (Raises the left operand to the power of the right)

4.6.2 Arithmetic Operators Examples


```
# Distances in Astronomical Units (AU)
distance_earth_sun = 1.0
distance_mars_sun = 1.52
distance_jupiter_sun = 5.2

# Addition: Total distance from Earth to Jupiter via Sun (simplified)
total_distance = distance_earth_sun + distance_jupiter_sun
print(f"Approximate distance Earth to Jupiter via Sun: {total_distance} AU")
```

```
Approximate distance Earth to Jupiter via Sun: 6.2 AU
```

```
# Subtraction: Difference in distance from Sun between Mars and Earth
distance_difference = distance_mars_sun - distance_earth_sun
print(f"Difference in distance (Mars - Earth): {distance_difference:.2f} AU") # Format to 2 decimal places
```

```
Difference in distance (Mars - Earth): 0.52 AU
```

```
# Multiplication: Convert Jupiter's distance to kilometers (1 AU approx 150 million km)
distance_jupiter_km = distance_jupiter_sun * 150e6 # 150e6 means 150 * 10^6
print(f"Jupiter's distance from Sun: {distance_jupiter_km:.2e} km") # Scientific notation
```

```
Jupiter's distance from Sun: 7.80e+08 km
```

```
# Division: How many times farther is Jupiter than Mars from the Sun?
jupiter_mars_ratio = distance_jupiter_sun / distance_mars_sun
print(f"Jupiter is {jupiter_mars_ratio:.2f} times farther than Mars from the Sun")
```

```
Jupiter is 3.42 times farther than Mars from the Sun
```

```
# Floor Division: If a probe travels 0.5 AU per year, how many full years to reach Jupiter?
travel_speed_au_year = 0.5
full_years_to_jupiter = distance_jupiter_sun // travel_speed_au_year
print(f"Full years to reach Jupiter at {travel_speed_au_year} AU/year: {full_years_to_jupiter} years")
```

```
Full years to reach Jupiter at 0.5 AU/year: 10.0 years
```

```
# Modulus: Remainder distance after those full years of travel
remainder_distance = distance_jupiter_sun % travel_speed_au_year
print(f"Remaining distance after {full_years_to_jupiter} years: {remainder_distance} AU")
```

```
Remaining distance after 10.0 years: 0.200000000000000018 AU
```

```
# Exponent: If brightness decreases with the square of the distance (simplified)
brightness_factor_jupiter = 1 / (distance_jupiter_sun ** 2)
brightness_factor_mars = 1 / (distance_mars_sun ** 2)
print(f"Relative brightness factor at Jupiter's distance: {brightness_factor_jupiter:.2f}")
print(f"Relative brightness factor at Mars' distance: {brightness_factor_mars:.3f}")
```

```
Relative brightness factor at Jupiter's distance: 0.037
Relative brightness factor at Mars' distance: 0.433
```

4.6.3 Comparison Operators

These operators compare two values and return a Boolean result (True or False).

- ==: Equal to (Checks if two values are equal)
- !=: Not equal to (Checks if two values are not equal)
- >: Greater than
- <: Less than
- >=: Greater than or equal to
- <=: Less than or equal to

4.6.4 Comparison Operators Examples

```
sun_temp_kelvin = 5778
sirius_temp_kelvin = 9940
earth_mass_relative = 1.0
jupiter_mass_relative = 317.8
```

```
# Equal to: Is the Sun's temperature exactly 5800 K?
is_sun_5800k = (sun_temp_kelvin == 5800)
print(f"Is Sun's temperature 5800 K? {is_sun_5800k}")
```

```
Is Sun's temperature 5800 K? False
```

```
# Not equal to: Is Jupiter's mass not equal to Earth's mass?
masses_not_equal = (jupiter_mass_relative != earth_mass_relative)
print(f"Are Jupiter's and Earth's masses different? {masses_not_equal}")
```

```
Are Jupiter's and Earth's masses different? True
```

```
# Greater than: Is Sirius hotter than the Sun?
is_sirius_hotter = (sirius_temp_kelvin > sun_temp_kelvin)
print(f"Is Sirius hotter than the Sun? {is_sirius_hotter}")
```

```
Is Sirius hotter than the Sun? True
```

```
# Less than: Is Earth less massive than Jupiter?
is_earth_less_massive = (earth_mass_relative < jupiter_mass_relative)
print(f"Is Earth less massive than Jupiter? {is_earth_less_massive}")
```

```
Is Earth less massive than Jupiter? True
```

```
# Greater than or equal to: Is the Sun's temperature >= 5500 K?
is_sun_temp_ge_5500 = (sun_temp_kelvin >= 5500)
print(f"Is Sun's temperature >= 5500 K? {is_sun_temp_ge_5500}")
```

```
Is Sun's temperature >= 5500 K? True
```

```
# Less than or equal to: Is Earth's mass <= 1?
is_earth_mass_le_1 = (earth_mass_relative <= 1.0)
print(f"Is Earth's mass <= 1? {is_earth_mass_le_1}")
```

```
Is Earth's mass <= 1? True
```

4.6.5 Logical Operators

These operators combine multiple conditional statements.

- **and**: Returns True if both statements are true.
- **or** : Returns True if at least one statement is true.
- **not**: Reverses the result (returns False if the statement is true, True if it's false).

4.6.6 Logical Operators Examples

```
planet_distance_au = 1.0 # Like Earth
planet_has_liquid_water = True
```

```
planet_is_rocky = True
planet_is_gas_giant = False
```

```
# 'and': Is the planet potentially habitable (rocky AND has liquid water)?
is_potentially_habitable = planet_is_rocky and planet_has_liquid_water
print(f"Is the planet potentially habitable? {is_potentially_habitable}")
```

```
Is the planet potentially habitable? True
```

```
# 'or': Is the planet far away (> 5 AU) OR a gas giant?
is_far_or_gas = (planet_distance_au > 5.0) or planet_is_gas_giant
print(f"Is the planet far away OR a gas giant? {is_far_or_gas}")
```

```
Is the planet far away OR a gas giant? False
```

```
# 'not': Is the planet NOT a gas giant?
is_not_gas_giant = not planet_is_gas_giant
print(f"Is the planet NOT a gas giant? {is_not_gas_giant}")
```

```
Is the planet NOT a gas giant? True
```

```
# Combining: Is the planet habitable AND not too far (< 1.5 AU)?
habitable_and_close = is_potentially_habitable and (planet_distance_au < 1.5)
print(f"Is the planet habitable AND relatively close? {habitable_and_close}")
```

```
Is the planet habitable AND relatively close? True
```

4.6.7 Assignment Operators

These operators are used to assign values to variables. The basic one is `=`, but there are shorthand operators combined with arithmetic operators.

- `=` : Assigns value from right side to left side operand.
- `+=` : Adds right operand to the left operand and assigns the result to the left (e.g., `x += 1` is same as `x = x + 1`).
- `-=` : Subtracts right operand from the left and assigns the result to the left.
- `*=` : Multiplies left operand with the right and assigns the result to the left.
- `/=` : Divides left operand by the right and assigns the result to the left.
- `//=` : Performs floor division and assigns the result.
- `%=` : Performs modulus and assigns the result.
- `**=` : Performs exponentiation and assigns the result.

4.6.8 Assignment Operators Examples

```
# Let's say we are counting the number of planets in our Solar System
number_of_planets = 8
print("There are", number_of_planets, "planets in the Solar System.")

# += : Adds right operand to the left
# Suppose we discover 2 new stars in a star cluster
stars_in_cluster = 5
stars_in_cluster += 2
print("Total stars in the cluster now:", stars_in_cluster)
```

```
There are 8 planets in the Solar System.
Total stars in the cluster now: 7
```

```
# -= : Subtracts right operand from the left
# A galaxy loses 1 satellite galaxy due to merging
satellite_galaxies = 4
satellite_galaxies -= 1
print("Remaining satellite galaxies:", satellite_galaxies)
```

```
Remaining satellite galaxies: 3
```

```
# *= : Multiplies left operand with the right
# Distance to a nearby star is tripled due to better measurements
distance_to_star = 4 # in light-years
distance_to_star *= 3
print("Updated distance to the star:", distance_to_star, "light-years")
```

```
Updated distance to the star: 12 light-years
```

```
# /= : Divides left operand by the right
# Total mass of a galaxy is spread across sectors
total_mass = 300 # in billion solar masses
sectors = 3
mass_per_sector = total_mass
mass_per_sector /= sectors
print("Mass per sector:", mass_per_sector, "billion solar masses")
```

```
Mass per sector: 100.0 billion solar masses
```

```
# //= : Performs floor division  
# Distributing exoplanets evenly among telescopes  
exoplanets = 22  
telescopes = 5  
exoplanets_per_telescope = exoplanets  
exoplanets_per_telescope //= telescopes  
print("Each telescope observes", exoplanets_per_telescope, "exoplanets")
```

Each telescope observes 4 exoplanets

```
# %= : Performs modulus  
# A planet orbits its star in 365 days. What day of the week does it return to the same  
orbit_day = 365  
orbit_day %= 7 # days of the week  
print("The planet returns to the same position on day:", orbit_day)
```

The planet returns to the same position on day: 1

```
# **= : Performs exponentiation  
# Light intensity decreases with the square of the distance  
distance = 2 # in light-years  
intensity_factor = distance  
intensity_factor **= 2 # square the distance  
print("Light intensity drops by a factor of", intensity_factor)
```

Light intensity drops by a factor of 4

4.7 Summary

Understanding Python variables and data types is crucial in data science programming. As you proceed with more complex analyses, these concepts will serve as the foundation for your work. Continue practicing by experimenting with different data types and operations in an astronomy context.

Happy coding!

4.8 Further Reading

For more information on variables and data types in Python, check out the **Basics** section of the A Byte of Python book: <https://python.swaroopch.com/basics.html>, https://python.swaroopch.com/op_exp.html or

the Python Tutorial Introduction page <https://docs.python.org/3/tutorial/introduction.html>.

4.9 Exercises

Variables

1. How would you create a variable named `planet_name` and store the text "Jupiter" in it?
2. How would you create a variable named `distance_to_sun` and store the number `149.6` (representing million kilometers) in it?
3. If you have a variable `star_temp = 5500`, how would you change its value to `6000`?
4. How would you create a variable named `is_gas_giant` and set it to `True` (representing that Jupiter is a gas giant)?
5. If you have the variable `moon_count = 2`, how would you print a message like: "Mars has 2 moons." using this variable?
6. Is `Number Of Stars` a good variable name in Python? Why or why not?
7. How would you create a variable called `galaxy_type` and store the text "Spiral" in it?

Operators

1. Subtraction: Earth is 1.0 AU from the Sun, Mars is 1.52 AU. How would you calculate the difference between Mars's distance and Earth's distance using the subtraction operator (`-`)?
2. Multiplication: Jupiter's orbit takes 11.86 Earth years. An Earth year is 365.25 days. How would you calculate Jupiter's orbital period in days using the multiplication operator (`*`)?
3. Assignment (`+=`): If a variable `jupiter_moons` currently holds the value 79, how would you use the `+=` operator to add 3 more moons to it?
4. Comparison (`>`): Venus's temperature is 462 degrees, Earth's is 15. How would you check if Venus's temperature is greater than Earth's using the `>` operator? (What is the result?)
5. Comparison (`==`): If you have a variable `galaxy_type = "Spiral"`, how would you check if its value is exactly equal to "Elliptical" using the `==` operator? (What is the result?)
6. Logical (`and`): A planet needs `has_water = True` AND `atmosphere_ok = False` to be habitable (simple rule). How would you use the `and` operator to see if *both* conditions are met? (What is the result?)
7. Logical (`or`): Observing is possible if `is_night = True` OR `is_solar_telescope = False`. How would you use the `or` operator to see if *at least one* of these conditions is met? (What is the result?)

8. Multiple Arithmetic: Proxima Centauri is 4.24 light-years away (`distance_ly`). One light-year is `9.461e12` km (`km_per_ly`). How would you first calculate the total distance in km (`distance_km`) and then print it?

Chapter 5

Data Structures

5.1 Tuples

Tuples are immutable sequences, meaning that once a tuple is created, its elements cannot be changed. Tuples are useful for storing fixed collections of items.

For example, a tuple can be used to store the coordinates of an astronomical source:

```
source_coords = (  
    219.9021,  
    -60.8340  
) # Tuple representing the equatorial coordinates (Right Ascension, Declination) of Alpha Centauri
```

In Python, tuple indices start from 0, just like lists. You can access elements by index:

```
right_ascension = source_coords[0]  
declination = source_coords[1]  
print(f"RA: {right_ascension}, Declination: {declination}")
```

```
RA: 219.9021, Declination: -60.834
```

Common Tuple Methods

Although tuples are immutable and support fewer methods than lists, the following two methods are available:

Method	Description	Example
<code>count(value)</code>	Returns the number of times <code>value</code> appears in the tuple	<code>(1, 2, 2, 3).count(2) → 2</code>
<code>index(value, start[, stop])</code>	Returns the first index of <code>value</code> ; optional <code>start</code> and <code>stop</code> limits	<code>(1, 2, 3).index(2) → 1</code>

5.2 Lists

Lists are ordered, mutable sequences, meaning you can change, add, or remove elements after the list has been created. Lists are very flexible and can store multiple types of data, making them useful for various astronomy tasks.

For example, you can store a list of coordinates representing an astronomical object(s).

```
star_coordinates = [
    (219.9021, -60.8340),
    (101.2875, -16.7161),
    (95.9879, -52.6957),
    (213.9153, 19.1824)] # List of tuples representing the coordinates of bright star.
```

Common List Methods

Lists in Python are **mutable**, which means you can change, add, or remove elements after the list is created. Python provides many built-in methods to manipulate lists.

Method	Description	Example
<code>append(x)</code>	Adds an item <code>x</code> to the end of the list	<code>[1, 2].append(3) → [1, 2, 3]</code>
<code>extend(iterable)</code>	Extends the list by appending all elements from the iterable	<code>[1, 2].extend([3, 4]) → [1, 2, 3, 4]</code>
<code>insert(i, x)</code>	Inserts item <code>x</code> at index <code>i</code>	<code>[1, 3].insert(1, 2) → [1, 2, 3]</code>
<code>remove(x)</code>	Removes the first occurrence of item <code>x</code>	<code>[1, 2, 3].remove(2) → [1, 3]</code>
<code>pop([i])</code>	Removes and returns item at index <code>i</code> (default is the last item)	<code>[1, 2, 3].pop() → returns 3</code>

Method	Description	Example
<code>clear()</code>	Removes all items from the list	<code>[1, 2, 3].clear()</code> → <code>[]</code>
<code>index(x[, start[, end]])</code>	Returns index of the first occurrence of item <code>x</code>	<code>[1, 2, 3].index(2)</code> → 1
<code>count(x)</code>	Returns the number of times <code>x</code> appears in the list	<code>[1, 2, 2, 3].count(2)</code> → 2
<code>sort()</code>	Sorts the list in place (ascending by default)	<code>[3, 1, 2].sort()</code> → <code>[1, 2, 3]</code>
<code>reverse()</code>	Reverses the list in place	<code>[1, 2, 3].reverse()</code> → <code>[3, 2, 1]</code>
<code>copy()</code>	Returns a shallow copy of the list	<code>a = [1, 2, 3]; b = a.copy()</code>

You can add a new point to the path:

```
# 1. append() - Add a new star coordinate
```

```
star_coordinates.append((279.2347, 38.7837)) # Adding another star coordinates to the list
print("Updated coordinates:", star_coordinates)
```

```
Updated coordinates: [(219.9021, -60.834), (101.2875, -16.7161), (95.9879, -52.6957), (213.9153,
```

```
# 2. extend() - Add multiple new star coordinates
```

```
star_coordinates.extend([(192.9246, 27.5407), (233.6104, -9.1834)])
print("After extend:", star_coordinates)
```

```
After extend: [(219.9021, -60.834), (101.2875, -16.7161), (95.9879, -52.6957), (213.9153, 19.1824,
```

```
# 3. insert() - Insert a star at index 5
```

```
star_coordinates.insert(5, (250.3932, -17.7421))
print("After insert at index 5:", star_coordinates)
```

```
After insert at index 5: [(219.9021, -60.834), (101.2875, -16.7161), (95.9879, -52.6957), (213.9153,
```

```
# 4. remove() - Remove a specific star coordinate
```

```
star_coordinates.remove((279.2347, 38.7837))
print("After remove:", star_coordinates)
```

After remove: [(219.9021, -60.834), (101.2875, -16.7161), (95.9879, -52.6957), (213.9153, 19.1824)]

```
# 5. pop() - Remove and return the star at index 7
removed = star_coordinates.pop(7)
print("Popped element:", removed)
print("After pop:", star_coordinates)
```

Popped element: (233.6104, -9.1834)

After pop: [(219.9021, -60.834), (101.2875, -16.7161), (95.9879, -52.6957), (213.9153, 19.1824)]

```
# 6. index() - Find the index of a coordinate
index = star_coordinates.index((213.9153, 19.1824))
print("Index of (213.9153, 19.1824):", index)
```

Index of (213.9153, 19.1824): 3

```
# 7. count() - Count how many times a coordinate appears
count = star_coordinates.count((219.9021, -60.8340))
print("Count of (219.9021, -60.8340):", count)
```

Count of (219.9021, -60.8340): 1

```
# 8. copy() - Make a shallow copy of the list
copy_coords = star_coordinates.copy()
print("Copy of coordinates:", copy_coords)
```

Copy of coordinates: [(219.9021, -60.834), (101.2875, -16.7161), (95.9879, -52.6957), (213.9153, 19.1824)]

```
# 9. reverse() - Reverse the list in-place
star_coordinates.reverse()
print("After reverse:", star_coordinates)
```

After reverse: [(192.9246, 27.5407), (250.3932, -17.7421), (250.3932, -17.7421), (213.9153, 19.1824)]

```
# 10. clear() - Remove all elements from the list
star_coordinates.clear()
print("After clear:", star_coordinates)
```

After clear: []

5.3 Sets

Sets are unordered collections of unique elements. They are useful when you need to store a collection of items and also get rid of duplicates.

For example, you might want to store a set of unique coordinates of stars.

```
star_names = {"Sirius", "Canopus", "Vega"} # Set of stars
print(star_names)
```

```
{'Canopus', 'Sirius', 'Vega'}
```

Common Set Methods

Method	Description
<code>add(elem)</code>	Adds an element to the set
<code>update(iterable)</code>	Adds multiple elements from an iterable
<code>remove(elem)</code>	Removes the specified element (raises <code>KeyError</code> if not found)
<code>discard(elem)</code>	Removes the element if present (no error if not found)
<code>pop()</code>	Removes and returns an arbitrary element
<code>clear()</code>	Removes all elements from the set
<code>union(other)</code>	Returns a new set with elements from both sets
<code>intersection(other)</code>	Returns common elements from both sets
<code>difference(other)</code>	Returns elements in the set but not in the other
<code>symmetric_difference(other)</code>	Returns elements in either set, but not both
<code>copy()</code>	Returns a shallow copy of the set

```
# 1. Add a single star name
star_names.add("Eta Carinae")
print("Updated stars:", star_names)
```

```
Updated stars: {'Eta Carinae', 'Canopus', 'Sirius', 'Vega'}
```

```
# 2. Add multiple star names
star_names.update(["Mira Ceti", "Beta Lyrae"])
print("Updated stars:", star_names)
```

```
Updated stars: {'Sirius', 'Vega', 'Canopus', 'Mira Ceti', 'Beta Lyrae', 'Eta Carinae'}
```

```
# 3. Remove a specific star name (raises KeyError if not present)
star_names.remove("Canopus")
print("Updated stars:", star_names)
```

```
Updated stars: {'Sirius', 'Vega', 'Mira Ceti', 'Beta Lyrae', 'Eta Carinae'}
```

```
# 4. Remove an arbitrary star name  
print("Popped:", star_names.pop())
```

```
Popped: Sirius
```

```
# 5. Clear all star names  
star_names.clear()
```

```
# Set operations  
a = {1, 2, 3}  
b = {3, 4, 5}  
  
print("Union:", a.union(b))  
print("Intersection:", a.intersection(b))  
print("Difference:", a.difference(b))  
print("Symmetric difference:", a.symmetric_difference(b))  
print("Copy of a:", a.copy())
```

```
Union: {1, 2, 3, 4, 5}  
Intersection: {3}  
Difference: {1, 2}  
Symmetric difference: {1, 2, 4, 5}  
Copy of a: {1, 2, 3}
```

5.4 Dictionaries

Dictionaries are collections of key-value pairs, where each key is unique. Dictionaries are extremely useful for storing data that is associated with specific identifiers, such as attribute data for geographic features.

For example, you can use a dictionary to store attributes of an astronomy feature, such as a city.

```
star_attributes = {  
    "name": "Alpha Centauri A",  
    "B_mag": 0.4,  
    "type": "Star",  
    "coordinates": [219.9021, -60.8340,]  
}
```

Common dict methods

Method	Description
<code>get(key[, default])</code>	Returns the value for the key; returns <code>default</code> if key is not found
<code>keys()</code>	Returns a view of all keys in the dictionary
<code>values()</code>	Returns a view of all values in the dictionary
<code>items()</code>	Returns a view of all key-value pairs as tuples
<code>update([other])</code>	Updates the dictionary with key-value pairs from another dictionary
<code>pop(key[, default])</code>	Removes specified key and returns the value; returns <code>default</code> if missing
<code>popitem()</code>	Removes and returns the last inserted key-value pair
<code>setdefault(key[, default])</code>	Returns the value of key if it exists; sets it if not
<code>clear()</code>	Removes all items from the dictionary
<code>copy()</code>	Returns a shallow copy of the dictionary

```
# 1. Get value with default
print(star_attributes.get("distance", "Unknown")) # Output: Unknown
```

```
Unknown
```

```
# 2. View keys
print(star_attributes.keys()) # dict_keys(['name', 'B_mag', 'type', 'coordinates'])
```

```
dict_keys(['name', 'B_mag', 'type', 'coordinates'])
```

```
# 3. View values
print(star_attributes.values())
```

```
dict_values(['Alpha Centauri A', 0.4, 'Star', [219.9021, -60.834]])
```

```
# 4. View items
print(star_attributes.items())
```

```
dict_items([('name', 'Alpha Centauri A'), ('B_mag', 0.4), ('type', 'Star'), ('coordinates', [219.
```

```
# 5. Update dictionary
star_attributes.update({"distance": 4.37}) # 4.37 light years
print(star_attributes)
```

```
{'name': 'Alpha Centauri A', 'B_mag': 0.4, 'type': 'Star', 'coordinates': [219.9021, -60.834]}
```

```
# 6. Pop a key
b_mag = star_attributes.pop("B_mag")
print("Removed B_mag:", b_mag)
```

```
Removed B_mag: 0.4
```

```
# 7. Pop last item
last_item = star_attributes.popitem()
print("Popped last item:", last_item)
```

```
Popped last item: ('distance', 4.37)
```

```
# 8. Set default if not present
star_attributes.setdefault("luminosity", "1.519 L ")
print(star_attributes)
```

```
{'name': 'Alpha Centauri A', 'type': 'Star', 'coordinates': [219.9021, -60.834], 'luminosity': '1.519 L '}
```

```
# 9. Copy dictionary
star_copy = star_attributes.copy()
print("Copy:", star_copy)
```

```
Copy: {'name': 'Alpha Centauri A', 'type': 'Star', 'coordinates': [219.9021, -60.834], 'luminosity': '1.519 L '}
```

```
# 10. Clear all entries
star_attributes.clear()
print("Cleared:", star_attributes)
```

```
Cleared: {}
```

5.5 Exercises

Lists

1. How would you make a Python list holding the names: “Mercury”, “Venus”, “Earth”?
2. If you have `planets = ["Mercury", "Venus", "Earth"]`, how do you get only “Venus”?

3. How would you add “Mars” to the end of the list `planets = ["Mercury", "Venus", "Earth"]`?
4. How do you find out how many items are in the list `moons = ["Moon", "Phobos", "Deimos"]`?

Dictionaries

5. How would you create a Python dictionary storing the fact: the Sun’s name is “Sun”?
6. If you have `star_info = {'name': "Sirius", 'color': "Blue-White"}`, how do you get only the color “Blue-White”?
7. How would you add a new fact to `star_info = {'name': "Sirius"}`? Add that its distance is 8.6 light-years (using the key 'distance').
8. How can you see all the keys used in the dictionary `galaxy_info = {'name': "Andromeda", 'type': "Spiral"}`?

Tuples

9. How would you store the year (1961) and name (“Gagarin”) together so they *cannot* be changed later?
10. If you have `moon_info = (1969, "Armstrong")`, how do you get only the year 1969?
11. Can you change the year in `moon_info = (1969, "Armstrong")` to 1970 after you create it? (Yes or No)

Sets

12. How would you create a Python set containing these gas planets: “Jupiter”, “Saturn”, “Uranus”, “Neptune”?
13. If you have `gas_giants = {"Jupiter", "Saturn", "Uranus", "Neptune"}`, how would you check if “Earth” is in this set?
14. If you add “Jupiter” again to the set `gas_giants = {"Jupiter", "Saturn"}`, what will the set look like afterwards?
15. How would you use a set to get only the unique names from this list: `observed = ["Moon", "Mars", "Moon", "Jupiter", "Mars"]`?

5.6 Summary

Understanding and utilizing Python’s data structures such as tuples, lists, sets, and dictionaries are fundamental skills in astronomy programming. These structures provide the flexibility and functionality required to manage and manipulate spatial data effectively.

Continue exploring these data structures by applying them to your astronomy projects and analyses.

Additional Resources

For more on python data structures, see <https://docs.python.org/3/tutorial/datastructures.html>.

Chapter 6

String Operations in Python for Astronomy

6.1 1. Creating and Manipulating Strings

Strings can be created using single, double, or triple quotes.

```
object_name = "Betelgeuse"
catalog_entry = 'HD 39801'
multiline_string = """Betelgeuse is a red supergiant star in the Orion constellation."""
print(object_name, catalog_entry)
```

```
Betelgeuse HD 39801
```

6.2 2. Concatenating Strings

We often need to combine multiple pieces of textual information. The character `+` is used for string concatenation.

```
constellation = "Orion"
description = "is home to the star " + object_name + " and other bright stars."
print("The constellation " + constellation + " " + description)
```

```
The constellation Orion is home to the star Betelgeuse and other bright stars.
```

6.3 Using f-strings for better readability

```
concatenated_string = f"The star {object_name} is part of the {constellation} constell:
print(concatenated_string)
```

```
The star Betelgeuse is part of the Orion constellation.
```

6.4 3. Repeating Strings Using * Operator

The * operator allows repetition of strings

```
separator = "-" * 30
print(separator)
message = ("Important Astronomical Event! " * 3).strip()
print(message)
```

```
-----
Important Astronomical Event! Important Astronomical Event! Important Astronomical Eve
```

6.5 4. String Methods in Astronomy

Let's explore some useful string methods.

```
star_name = "  Sirius A  " # A common stellar designation with extra spaces
print(star_name.strip()) # Removes extra spaces
print(star_name.lstrip()) # Removes leading spaces
print(star_name.rstrip()) # Removes trailing spaces
```

```
Sirius A
Sirius A
  Sirius A
```

```
star_type = "Main Sequence"
print(star_type.lower()) # Converts to lowercase
print(star_type.upper()) # Converts to uppercase
```

```
main sequence
MAIN SEQUENCE
```

```
# Replacing parts of a string
star_description = "Vega is a white dwarf."
corrected_description = star_description.replace("white dwarf", "main sequence star")
print(corrected_description)
```

```
Vega is a main sequence star.
```

6.6 5. Finding the Position of a Substring

This can help locate specific information in star descriptions.

```
galaxy_description = "Andromeda is the nearest spiral galaxy."
position = galaxy_description.find("spiral")
print("Position of 'spiral' in the description:", position)
```

```
Position of 'spiral' in the description: 25
```

6.7 6. Counting Occurrences of a Word in a String

This is useful for analyzing astronomical text data.

```
text_data = "The Milky Way is a spiral galaxy. The Andromeda Galaxy is also a spiral galaxy."
count_spiral = text_data.count("spiral")
print("Occurrences of 'spiral':", count_spiral)
```

```
Occurrences of 'spiral': 2
```

6.8 7. Formatting Strings

In astronomy, we often need to format strings for readability.

```
declination = -16.7161
right_ascension = "06h 45m 08.9s"
formatted_string = f"Sirius has RA {right_ascension} and Dec {declination} degrees."
print(formatted_string)
```

```
Sirius has RA 06h 45m 08.9s and Dec -16.7161 degrees.
```

6.9 8. Splitting and Extracting Data from Strings

We often need to extract specific information from catalog entries.

```
catalog_entry = "NGC 224 - Andromeda Galaxy"
parts = catalog_entry.split(" - ") # Splitting by the separator " - "
print("Object ID:", parts[0]) # Extracts "NGC 224"
print("Object Name:", parts[1]) # Extracts "Andromeda Galaxy"
```

```
Object ID: NGC 224
Object Name: Andromeda Galaxy
```

6.10 9. Joining Strings Using join()

We can join a list of strings into a single string using a delimiter.

```
word_list = ["The", "Milky", "Way", "is", "a", "galaxy"]
joined_string = " ".join(word_list)
print(joined_string)
```

```
The Milky Way is a galaxy
```

6.11 10. Parsing Coordinates from a String

```
coordinates = """RA: 10h 56m 29s, Dec: +07° 00' 52" """ # Sample coordinates
ra_dec_split = coordinates.split(", ")
ra = ra_dec_split[0].replace("RA: ", "")
dec = ra_dec_split[1].replace("Dec: ", "")
print("Right Ascension:", ra)
print("Declination:", dec)
```

```
Right Ascension: 10h 56m 29s
Declination: +07° 00' 52"
```

6.12 11. Checking for Substrings

Sometimes, we need to check if a string contains a specific keyword.

```
object_info = "Messier 31 - Spiral Galaxy"
if "Spiral Galaxy" in object_info:
    print("This object is a spiral galaxy!")
```

```
This object is a spiral galaxy!
```

6.13 12. Using Strings in SQL Queries for Astronomical Data

Suppose we have a database of celestial objects and need to generate SQL queries dynamically.

```
table_name = "galaxies"
query = f"SELECT * FROM {table_name} WHERE type = 'Spiral Galaxy';"
print("Generated SQL Query:", query)

object_name = "Andromeda"
sql_condition = f"SELECT * FROM {table_name} WHERE name = '{object_name}';"
print("Search Query for a Specific Galaxy:", sql_condition)
```

```
Generated SQL Query: SELECT * FROM galaxies WHERE type = 'Spiral Galaxy';
Search Query for a Specific Galaxy: SELECT * FROM galaxies WHERE name = 'Andromeda';
```

6.14 Exercise

1. Create a string containing the name and type of any celestial object.
2. Convert the name to uppercase and print it.
3. Extract and print the type of object from your string.
4. Format a string that includes the object's name and a short description.
5. Concatenate two strings to describe an astronomical feature.
6. Extract right ascension and declination from a given coordinate string.
7. Use the * operator to repeat a warning message about a cosmic event.
8. Use join() to combine words into a complete sentence.
9. Find the position of a specific word in an astronomical description.
10. Count the occurrences of a word in a given string.

11. Write a small SQL query as a string to search for an astronomical object in a database. Example: `object_info = "Vega - Main Sequence Star"` (Perform the above tasks on this or another example of your choice).

Submit your responses as a Python script or Jupyter Notebook cell outputs.

6.15 Summary

String operations are essential in astronomy for handling and processing textual data related to celestial objects, coordinates, and astronomical catalogs. Whether formatting star names, extracting right ascension and declination, or generating SQL queries for astronomical databases, mastering string operations allows for efficient data manipulation and analysis. These skills are invaluable when working with observational data, automating research workflows, and ensuring clarity in astronomical documentation.

Chapter 7

Python Loops

7.1 What are Loops?

Imagine you want to print the numbers from 1 to 5. You could do this:

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

```
1
2
3
4
5
```

But what if you wanted to print the numbers from 1 to 100? Writing 100 lines of code would be tedious!

That's where loops come in. A loop allows us to repeat a block of code multiple times, automatically.

There are two main types of loops in Python: 'for' loops and 'while' loops. We'll explore each of them.

7.2 for Loops: Repeating Actions a Specific Number of Times

A `for` loop is used when you know *in advance* how many times you want to repeat a block of code.

It's perfect for iterating over a sequence of things (like numbers, stars in a list, or letters in a word).

The basic syntax is:

```
# for variable in sequence:
#     # Code to be executed repeatedly
```

- variable: A temporary variable that takes on the value of each item `in` the sequence,
 - sequence: A collection of items (e.g., a range of numbers, a list of star names, a string)

The code inside the loop (indented!) `is` executed once `for` each item `in` the sequence.

```
```python
Example 1: Counting Stars

Let's print a countdown for a rocket launch!

for i in range(10, 0, -1): # range(start, stop, step) generates a sequence of numbers
 print(i) #print the current number 'i'
print("BLAST OFF!")
```

```
10
9
8
7
6
5
4
3
2
1
BLAST OFF!
```

In this example above:

- `i` takes on the values 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 in that order.
- `range(10, 0, -1)` creates a sequence of numbers starting at 10, going down to (but not including) 0, decreasing by 1 each time.
- The `print(i)` statement is executed for each value of `i`.

### 7.2.1 Understanding the `range()` Function

The `range()` function is incredibly useful for creating sequences of numbers to use in `for` loops. It can take one, two, or three arguments:

- `range(stop)`: Generates numbers from 0 up to (but not including) `stop`.
- `range(start, stop)`: Generates numbers from `start` up to (but not including) `stop`.
- `range(start, stop, step)`: Generates numbers from `start` up to (but not including) `stop`, incrementing by `step`.

Let's see some examples:

```
Example 2: Different uses of range()
```

```
Numbers from 0 to 9
```

```
for i in range(10):
 print(i)
```

```
print("----")
```

```
Numbers from 2 to 5
```

```
for j in range(2, 6):
 print(j)
```

```
print("----")
```

```
Even numbers from 0 to 10
```

```
for k in range(0, 11, 2):
 print(k)
```

```
0
1
2
3
4
5
6
7
8
9

2
3
4
```

```
5

0
2
4
6
8
10
```

### 7.2.2 Astronomy Example: Simulating Planet Orbits

Let's simulate the orbit of a planet around a star. We'll print the planet's position at different points in its orbit.

(This is a simplified example. Real planet orbits are more complex!)

```
Example 3: Planet Orbit Simulation (simplified)

star_name = "Sun"
planet_name = "Earth"
orbital_period = 365 # Days

Let's print the planet's position every 30 days
for day in range(0, orbital_period, 30):
 position = day / orbital_period * 360 # Calculate the planet's angular position (
 print(f"Day {day}: {planet_name} is at {position:.2f} degrees around the {star_name}")
```

```
Day 0: Earth is at 0.00 degrees around the Sun.
Day 30: Earth is at 29.59 degrees around the Sun.
Day 60: Earth is at 59.18 degrees around the Sun.
Day 90: Earth is at 88.77 degrees around the Sun.
Day 120: Earth is at 118.36 degrees around the Sun.
Day 150: Earth is at 147.95 degrees around the Sun.
Day 180: Earth is at 177.53 degrees around the Sun.
Day 210: Earth is at 207.12 degrees around the Sun.
Day 240: Earth is at 236.71 degrees around the Sun.
Day 270: Earth is at 266.30 degrees around the Sun.
Day 300: Earth is at 295.89 degrees around the Sun.
Day 330: Earth is at 325.48 degrees around the Sun.
Day 360: Earth is at 355.07 degrees around the Sun.
```

## 7.3 while Loops: Repeating Actions Until a Condition is Met

A **while** loop is used when you want to repeat a block of code *as long as* a certain condition is true. You don't necessarily know in advance how many times the loop will run.

The basic syntax is:

```
while condition:
Code to be executed repeatedly (as long as the condition is True)
```

where: \* condition: An expression that evaluates to either True or False.

The code inside the loop (indented!) is executed repeatedly as long as the condition remains True.

Important: Make sure the condition eventually becomes False, or your loop will run forever (an infinite loop!).

```
Example 4: Counting Down with a while loop

countdown = 10

while countdown > 0:
 print(countdown)
 countdown = countdown - 1 # Crucial: Decrement the counter to eventually make the condition
print("BLAST OFF!")
```

```
10
9
8
7
6
5
4
3
2
1
BLAST OFF!
```

### 7.3.1 Astronomy Example: Searching for Habitable Planets

Imagine we're searching for planets within the habitable zone of a star. The habitable zone is the region around a star where liquid water could exist on a

planet's surface (a key ingredient for life as we know it!). For more on habitable zone, see the NASA Article.

We'll simulate measuring the temperature of planets at increasing distances from a star until we find one that's potentially habitable.

*# Example 5: Habitable Planet Search Simulation*

```

star_name = "Kepler-186"
distance = 0.5 # Starting distance from the star (in Astronomical Units, AU)
temperature = 200 # Starting temperature (in Kelvin)
habitable = False #set a variable to stop the loop
We'll assume a planet is habitable if its temperature is between 273K (0°C) and 303K
habitable_min_temp = 273
habitable_max_temp = 303

while not habitable:
 print(f"Checking planet at distance {distance:.2f} AU from {star_name}. Temperature: {temperature:.2f} K")

 if habitable_min_temp <= temperature <= habitable_max_temp:
 print(f"Potential habitable planet found at {distance:.2f} AU! Temperature is {temperature:.2f} K")
 habitable = True
 else:
 distance = distance + 0.1 # Move to the next planet further out
 temperature = temperature - 15 #Assume temperature decreases as distance increases
 if temperature <= 0:
 print("no habitable planet found")
 break

```

```

Checking planet at distance 0.50 AU from Kepler-186. Temperature: 200.00 K
Checking planet at distance 0.60 AU from Kepler-186. Temperature: 185.00 K
Checking planet at distance 0.70 AU from Kepler-186. Temperature: 170.00 K
Checking planet at distance 0.80 AU from Kepler-186. Temperature: 155.00 K
Checking planet at distance 0.90 AU from Kepler-186. Temperature: 140.00 K
Checking planet at distance 1.00 AU from Kepler-186. Temperature: 125.00 K
Checking planet at distance 1.10 AU from Kepler-186. Temperature: 110.00 K
Checking planet at distance 1.20 AU from Kepler-186. Temperature: 95.00 K
Checking planet at distance 1.30 AU from Kepler-186. Temperature: 80.00 K
Checking planet at distance 1.40 AU from Kepler-186. Temperature: 65.00 K
Checking planet at distance 1.50 AU from Kepler-186. Temperature: 50.00 K
Checking planet at distance 1.60 AU from Kepler-186. Temperature: 35.00 K
Checking planet at distance 1.70 AU from Kepler-186. Temperature: 20.00 K
Checking planet at distance 1.80 AU from Kepler-186. Temperature: 5.00 K
no habitable planet found

```

In a real simulation, we'd use a more accurate model to calculate temperature

based on distance, star type, and planetary albedo (reflectivity).

## 7.4 Loop Control: break and continue

Sometimes you need more control over how a loop executes. That's where **break** and **continue** come in.

- **break**: Immediately exits the loop (even if the condition is still True).
- **continue**: Skips the rest of the current iteration of the loop and goes to the next iteration.

```
Example 6: Using break and continue

Let's search for a specific star in a list of stars.

stars = ["Alpha Centauri", "Sirius", "Betelgeuse", "Vega", "Polaris"]
search_star = "Betelgeuse"

for star in stars:
 if star == search_star:
 print(f"Found {search_star}!")
 break # Exit the loop once we find the star
 else:
 print(f"Checking star: {star}")
```

```
Checking star: Alpha Centauri
Checking star: Sirius
Found Betelgeuse!
```

```
#Using Break to avoid a never ending loop.

i = 0
while True:
 print(i)
 i = i + 1
 if i > 10:
 break # Will stop the loop once i is greater than 10.
```

```
0
1
2
3
4
```

```
5
6
7
8
9
10
```

## 7.5 Looping Through Lists and Strings

Loops are especially powerful when working with lists and strings.

- **Lists:** A list is an ordered collection of items. We can use a `for` loop to access each item in the list.
- **Strings:** A string is a sequence of characters. We can use a `for` loop to access each character in the string.

```
Example 7: Looping through a list of constellations
```

```
constellations = ["Orion", "Ursa Major", "Gemini", "Taurus"]

print("Constellations:")
for constellation in constellations:
 print(constellation)

print("---")
```

```
Example 8: Looping through a string (star name)
```

```
star_name = "Antares"

print("Characters in", star_name + ":")
for char in star_name:
 print(char)
```

```
Constellations:
Orion
Ursa Major
Gemini
Taurus

Characters in Antares:
A
n
```



t  
a  
r  
e  
s

## 7.6 Nested Loops: Loops within Loops

A nested loop is a loop inside another loop. They are useful for working with multi-dimensional data, like tables or grids. Imagine analyzing a grid of pixels in an image from the James Webb Space Telescope or MeerKAT radio image.

*# Example 9: Nested Loops (Creating a Multiplication Table)*

```
for i in range(1, 4): # Outer loop
 for j in range(1, 4): # Inner loop
 print(f"{i} * {j} = {i*j}")
 print("----") # Separator between rows
```

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9

```

*# Nested loop example for the stars*

```
star_systems = ["Alpha Centauri A", "Alpha Centauri B", "Proxima Centauri"], ["Sirius A", "Sirius B"]
```

*# Outer loop iterates through each star system*

```
for system in star_systems:
 print(f"Star System: {system}")
 #Inner loop iterates through the stars in the current system
 for star in system:
 print(f"Star: {star}")
 print("----")
```

```

Star System: ['Alpha Centauri A', 'Alpha Centauri B', 'Proxima Centauri']
Star: Alpha Centauri A
Star: Alpha Centauri B
Star: Proxima Centauri

Star System: ['Sirius A', 'Sirius B']
Star: Sirius A
Star: Sirius B

```

## 7.7 More Practical Astronomy Examples

Here are some more complex examples for further illustration of loops in astronomical.

*#Example 10: Calculating total luminosity*

```

luminosities = [0.5, 1.2, 0.8, 5.3, 0.9] # Luminosity values of stars relative to the .
total_luminosity = 0 # Initialize the total luminosity

#Iterate through the luminosities and add each value to the total
for luminosity in luminosities:
 total_luminosity = total_luminosity + luminosity
#Print the total luminosity
print(f"The total luminosity of the stars is: {total_luminosity}")

```

The total luminosity of the stars is: 8.7

*#Example 11: Filter list of exoplanets*

```

exoplanets = [
 {"name": "Kepler-186f", "radius": 1.2, "habitable": True},
 {"name": "HD 209458 b", "radius": 1.38, "habitable": False},
 {"name": "TRAPPIST-1e", "radius": 0.91, "habitable": True},
 {"name": "51 Pegasi b", "radius": 1.50, "habitable": False},
]

habitable_exoplanets = [] # List to store habitable exoplanets

#Iterate through the exoplanets and add the habitable ones to the new list
for exoplanet in exoplanets:
 if exoplanet["habitable"]:

```

```
habitable_exoplanets.append(exoplanet["name"])

#Print the list of habitable exoplanets
print("Habitable exoplanets:", habitable_exoplanets)
```

```
Habitable exoplanets: ['Kepler-186f', 'TRAPPIST-1e']
```

## 7.8 Exercises

1. Star Catalog: You have a list of star names: `stars = ["Sirius", "Canopus", "Rigel", "Proxima Centauri", "Vega"]`. Use a `for` loop to print each star name with its index in the list (starting from 1). For example: "1. Sirius", "2. Canopus", etc.
2. Temperature Conversion: You have a list of temperatures in Celsius: `celsius_temps = [ -10, 0, 25, 100, 500 ]`. Use a `for` loop to convert each temperature to Fahrenheit and print the result. The formula is: `Fahrenheit = (Celsius * 9/5) + 32`.
3. Exoplanet Distance: Simulate the distance of an exoplanet from its star, increasing by 0.1 AU in each step. Use a `while` loop to stop the simulation when the distance reaches 2.0 AU. Print the distance at each step.
4. Prime Number Checker: Write a code that checks if a number is Prime.
5. Fibonacci Sequence: Write a code that Prints the Fibonacci Sequence until a limit. For more on Fibonacci, see [this article](#).

## 7.9 Summary

Congratulations! You've completed your journey into the world of Python loops. You've learned how to:

- Use `for` loops to repeat actions a specific number of times.
- Use `while` loops to repeat actions until a condition is met.
- Control loops with `break` and `continue`.
- Loop through lists and strings.
- Use nested loops for multi-dimensional data.

Loops are essential tools for programmers and scientist. Keep practicing, and you'll be using them to analyze data, whether for astronomy or otherwise.

### Additional Resources

To learn more about loops, kindly the Python Control Flow documentation [here](#).

## Chapter 8

# Python Functions

### 8.1 Functions: Encapsulation and Reusability

Consider the task of calculating the distance to a star using its parallax angle.

$$d = \frac{1}{p}$$

where  $d$  is the distance in parsecs and  $p$  is the parallax in arcseconds.

For more about astronomy parallax and distance, see the articles by Les Cumbres Observatory: [parallax](#) and [distance](#).

```
parallax_angle_star1 = 0.77233 # arcseconds
distance_star1_parsecs = 1 / parallax_angle_star1
distance_star1_lightyears = distance_star1_parsecs * 3.26
print(f"Distance to Star 1: {distance_star1_lightyears:.2f} light-years")

parallax_angle_star2 = 0.00406 # arcseconds
distance_star2_parsecs = 1 / parallax_angle_star2
distance_star2_lightyears = distance_star2_parsecs * 3.26
print(f"Distance to Star 2: {distance_star2_lightyears:.2f} light-years")
```

```
Distance to Star 1: 4.22 light-years
Distance to Star 2: 802.96 light-years
```

Repeating this code for numerous stars is inefficient. Functions provide a solution by encapsulating this calculation.

## 8.2 Benefits of Functions:

- Reusability: Avoid redundant code by packaging logic into reusable units.
- Modularity: Break down complex problems into smaller, manageable components.
- Readability: Improve code clarity by assigning meaningful names to code blocks.

### 8.2.1 Defining Functions with the def Keyword

Functions are defined using the `def` keyword, followed by the function name, parentheses for arguments (if any), and a colon. The function's code block is indented.

```
def function_name(argument1, argument2, ...):
Function body (code to be executed)
return value # Optional: Return a result
```

- `def`: Keyword for function definition.
- `function_name`: A descriptive name (e.g., `calculate_distance`).
- `arguments`: Input values the function accepts (optional).
- `return`: Specifies the value the function returns (optional; defaults to `None`).

```
Example 1: Calculating Stellar Distance (Function)

def calculate_distance(parallax_angle_arcsec):
 """Calculates the distance to a star in light-years based on its parallax."""
 distance_parsecs = 1 / parallax_angle_arcsec
 distance_lightyears = distance_parsecs * 3.26
 return distance_lightyears

star1_parallax = 0.77233
star1_distance = calculate_distance(star1_parallax)
print(f"Distance to Star 1: {star1_distance:.2f} light-years")

star2_parallax = 0.00406
star2_distance = calculate_distance(star2_parallax)
print(f"Distance to Star 2: {star2_distance:.2f} light-years")

The function encapsulates the distance calculation, making the code more concise.
```

```
Distance to Star 1: 4.22 light-years
Distance to Star 2: 802.96 light-years
```

## 8.3 Function Arguments (Inputs)

Functions can accept arguments, providing them with data to operate on. Multiple arguments can be defined, each with a name.

```
def function_name(argument1, argument2, argument3):
Use argument1, argument2, and argument3 within the function
pass
```

Default values can be assigned to arguments. If a caller omits a value for an argument with a default, the default value is used.

```
def function_name(argument1, argument2=default_value):
argument2 will use default_value if not provided by the caller
pass
```

Below is an example for writing a function for gravitational force:

$$F = G \frac{m_1 m_2}{r^2}$$

Where:

- $F$  is the gravitational force (in newtons, N)
- $G$  is the gravitational constant ( $6.674 \times 10^{-11} \text{ N} \cdot \text{m}^2 / \text{kg}^2$ )
- $m_1$  and  $m_2$  are the masses of the two objects (in kilograms, kg)
- $r$  is the distance between the centers of the two masses (in meters, m)

```
Example 2: Calculating Gravitational Force
```

```
def calculate_gravitational_force(mass1, mass2, distance):
 """Calculates the gravitational force between two objects.

 Args:
 mass1 (float): Mass of the first object (kg).
 mass2 (float): Mass of the second object (kg).
```

```

 distance (float): Distance between the objects (m).

 Returns:
 float: The gravitational force (N).
 """
 G = 6.674e-11 # Gravitational constant (N m^2 / kg^2)
 force = (G * mass1 * mass2) / (distance ** 2)
 return force

earth_mass = 5.972e24 # kg
moon_mass = 7.348e22 # kg
earth_moon_distance = 3.844e8 # m

gravitational_force = calculate_gravitational_force(earth_mass, moon_mass, earth_moon_distance)
print(f"Gravitational force between Earth and Moon: {gravitational_force:.2e} N")

```

```
Gravitational force between Earth and Moon: 1.98e+20 N
```

## 8.4 Return Values (Outputs)

A function can return a value using the `return` statement. The value can be of any data type.

If no `return` statement is present, the function implicitly returns `None`.

The example below is a demonstration of stellar (star) spectral classification based on temperature. See this article for more details.

```

Example 3: Classifying Stars by Temperature

def classify_star(temperature_kelvin):
 """Classifies a star based on its surface temperature (Kelvin).

 Returns:
 str: Spectral type ("O", "B", "A", "F", "G", "K", "M", or "Unknown").
 """
 if temperature_kelvin >= 30000:
 return "O"
 elif 10000 <= temperature_kelvin < 30000:
 return "B"
 elif 7500 <= temperature_kelvin < 10000:
 return "A"
 elif 6000 <= temperature_kelvin < 7500:
 return "F"

```



```

elif 5200 <= temperature_kelvin < 6000:
 return "G"
elif 3700 <= temperature_kelvin < 5200:
 return "K"
elif temperature_kelvin < 3700:
 return "M"
else:
 return "Unknown"

star1_temperature = 25000
star1_spectral_type = classify_star(star1_temperature)
print(f"Star with temperature {star1_temperature} K: Spectral type {star1_spectral_type}")

star2_temperature = 4500
star2_spectral_type = classify_star(star2_temperature)
print(f"Star with temperature {star2_temperature} K: Spectral type {star2_spectral_type}")

```

```

Star with temperature 25000 K: Spectral type B
Star with temperature 4500 K: Spectral type K

```

## 8.5 Docstrings: Function Documentation

A docstring (documentation string) is a multi-line string used to document the purpose, arguments, and return value of a function. It's crucial for code maintainability.

Docstrings are enclosed in triple quotes (`"""Docstring goes here"""`).

To access docstrings, use the `help` function e.g. `help(function_name)` or `function_name.__doc__`.

```

Example 4: Function with Docstring

def calculate_escape_velocity(mass, radius):
 """Calculates the escape velocity of a celestial body.

 Args:
 mass (float): Mass of the celestial body (kg).
 radius (float): Radius of the celestial body (m).

 Returns:
 float: Escape velocity (m/s).
 """
 G = 6.67430e-11 # Gravitational constant

```

```

 escape_velocity = (2 * G * mass / radius)**0.5
 return escape_velocity

help(calculate_escape_velocity) # Display the docstring
print(calculate_escape_velocity.__doc__) #alternative way to access the docstring

```

Help on function calculate\_escape\_velocity in module \_\_main\_\_:

```

calculate_escape_velocity(mass, radius)
 Calculates the escape velocity of a celestial body.

 Args:
 mass (float): Mass of the celestial body (kg).
 radius (float): Radius of the celestial body (m).

 Returns:
 float: Escape velocity (m/s).

Calculates the escape velocity of a celestial body.

 Args:
 mass (float): Mass of the celestial body (kg).
 radius (float): Radius of the celestial body (m).

 Returns:
 float: Escape velocity (m/s).

```

## 8.6 Scope: Variable Visibility

Scope determines where a variable can be accessed. Variables defined inside a function have *local scope* (accessible only within the function).

Variables defined outside functions have *global scope* (accessible everywhere).

It is advisable to minimize global variable usage for better code organization and to avoid potential conflicts.

```

Example 5: Variable Scope

global_variable = "Accessible Globally"

def my_function():
 local_variable = "Accessible Locally"
 print("Inside the function:")

```

## 8.7. ADVANCED EXAMPLES: APPLYING FUNCTIONS IN ASTRONOMY71

```
print(global_variable) # Accessing global is allowed
print(local_variable)

my_function()

print("Outside the function:")
print(global_variable) # Accessing global is allowed
print(local_variable) # Error: local_variable is not defined outside my_function
```

```
Inside the function:
Accessible Globally
Accessible Locally
Outside the function:
Accessible Globally
```

## 8.7 Advanced Examples: Applying Functions in Astronomy

These examples demonstrate the use of functions in more complex astronomical calculations.

```
Example 6: Blackbody Radiation Calculation

from math import exp, pi

def calculate_blackbody_radiation(temperature, wavelength):
 """Calculates blackbody spectral radiance at a given temperature and wavelength.

 Args:
 temperature (float): Temperature (Kelvin).
 wavelength (float): Wavelength (meters).

 Returns:
 float: Blackbody spectral radiance (W / m^2 / sr / m).
 """
 c = 2.998e8 # Speed of light (m/s)
 h = 6.626e-34 # Planck's constant (J s)
 k = 1.381e-23 # Boltzmann constant (J/K)

 intensity = (2 * h * c**2 / wavelength**5) / (exp((h * c) / (wavelength * k * temperature)) - 1)
 return intensity
```

```
Example usage:
temperature = 5778 # Sun's surface temperature (K)
wavelength = 500e-9 # Wavelength of green light (m)
radiance = calculate_blackbody_radiation(temperature, wavelength)
print(f"Blackbody radiance at {wavelength*1e9:.1f} nm and {temperature} K: {radiance:.1f}
```

```
Blackbody radiance at 500.0 nm and 5778 K: 2.64e+13 W/m^2/sr/m
```

```
Example 7: Radians to Degrees Conversion

def radians_to_degrees(radians):
 """Converts an angle from radians to degrees.

 Args:
 radians (float): Angle in radians.

 Returns:
 float: Angle in degrees.
 """
 degrees = radians * 180 / pi
 return degrees

Example usage:
angle_radians = pi / 2 # 90 degrees
angle_degrees = radians_to_degrees(angle_radians)
print(f"{angle_radians:.2f} radians is equal to {angle_degrees:.2f} degrees")
```

```
1.57 radians is equal to 90.00 degrees
```

## 8.8 Exercises

Apply your knowledge of functions to solve these problems:

1. Magnitude Difference: Create a function to calculate the magnitude difference between two stars (magnitude1 - magnitude2).
2. Distance Modulus: Create a function to calculate the distance modulus (apparent magnitude - absolute magnitude).
3. Wien's Law: Create a function to calculate the peak wavelength of blackbody radiation using Wien's Law (peak wavelength = Wien's constant / temperature). Wien's constant =  $2.898 \times 10^{-3} \text{ m} \cdot \text{K}$ .

4. Sphere Area: Create a function to calculate the surface area of a sphere given its radius.
5. Leap Year: Create a function that determines whether a given year is a leap year.

## 8.9 Summary

You have now learned to define, use, and document Python functions. These skills are crucial for writing well-structured and maintainable code in any programming context, and especially valuable in data analysis and research.

### **Additional Resources**

Built in functions: Check out the built in or default python functions: <https://docs.python.org/3/library/functions.html>.

Functions: <https://realpython.com/defining-your-own-python-function/>.



## Chapter 9

# Introduction to Python Classes

```
Example 1: Defining a Simple Planet Class

Let's define a class called 'Planet' to represent a planet in our solar system.

class Planet:
 """Represents a planet with a name and mass."""

 def __init__(self, name, mass): # This is a special method
 """Initializes a Planet object.

 Args:
 name (str): The name of the planet (e.g., "Earth", "Mars").
 mass (float): The mass of the planet (in Earth masses).
 """

 # 'self' refers to the instance of the class (the specific Planet object)
 # We assign the values passed to the __init__ method to the object's attributes:
 self.name = name # The planet's name
 self.mass = mass # The planet's mass

 def describe(self):
 """Prints a description of the planet."""
 print(f"Planet: {self.name}")
 print(f"Mass: {self.mass} Earth masses")

Creating an object (instance) of the Planet class:
earth = Planet("Earth", 1.0) #Name and Mass
```

```
mars = Planet("Mars", 0.107) #Name and Mass
```

`earth` and `mars` are now objects of the `Planet` class. Both has attributes `name` and `mass` with the assigned values.

Now we can call the method

```
earth.describe()
mars.describe()
```

```
Planet: Earth
Mass: 1.0 Earth masses
Planet: Mars
Mass: 0.107 Earth masses
```

## 9.1 Understanding Attributes and Methods

- **Attributes:** These are the variables that store data related to the object. In the `Planet` class example, `name` and `mass` are attributes. They hold specific values for each planet object.
- **Methods:** These are functions that operate on the object's data. They define the object's behavior. In the `Planet` class example, `describe` is a method. It prints information about the planet.

To access an attribute or call a method, use the dot notation (`object.attribute` or `object.method()`).

```
Example 2: Adding More Attributes to a Star Class

class Star:
 """Represents a star with name, mass, radius, and temperature."""

 def __init__(self, name, mass, radius, temperature):
 """Initializes a Star object."""
 self.name = name
 self.mass = mass
 self.radius = radius
 self.temperature = temperature

 def describe(self):
 """Prints a description of the star."""
 print(f"Star: {self.name}")
```



```

 print(f" Mass: {self.mass} Solar masses")
 print(f" Radius: {self.radius} Solar radii")
 print(f" Temperature: {self.temperature} K")

Creating Star objects:
sun = Star("Sun", 1.0, 1.0, 5778)
sirius = Star("Sirius", 2.02, 1.71, 9940)

sun.describe()
sirius.describe()

```

```

Star: Sun
 Mass: 1.0 Solar masses
 Radius: 1.0 Solar radii
 Temperature: 5778 K
Star: Sirius
 Mass: 2.02 Solar masses
 Radius: 1.71 Solar radii
 Temperature: 9940 K

```

### 9.1.1 Adding Methods - Calculating Circumference of a Planet

Methods are defined inside the class and can do various operations on the class attributes. They must always have `self` as its first argument.

```

Example 3: Adding a method

class Planet:
 """Represents a planet with name, mass and radius."""

 def __init__(self, name, mass, radius):
 """Initializes a Planet object."""
 self.name = name
 self.mass = mass
 self.radius = radius

 def describe(self):
 """Prints a description of the star."""
 print(f"Planet: {self.name}")
 print(f" Mass: {self.mass} Earth Masses")
 print(f" Radius: {self.radius} Earth Radii")

 def calculate_circumference(self):

```

```

 circumference = 2 * 3.14159 * self.radius
 return circumference

Creating Planet objects:
earth = Planet("Earth", 1.0, 1.0)
mars = Planet("Mars", 0.1, 0.5)

earth.describe()
print(f"Earth Circumference: {earth.calculate_circumference()}")
mars.describe()
print(f"Mars Circumference: {mars.calculate_circumference()}")

```

```

Planet: Earth
 Mass: 1.0 Earth Masses
 Radius: 1.0 Earth Radii
Earth Circumference: 6.28318
Planet: Mars
 Mass: 0.1 Earth Masses
 Radius: 0.5 Earth Radii
Mars Circumference: 3.14159

```

## 9.2 The `__init__` Method: Initializing Objects

The `__init__` method (pronounced “dunder init,” short for “double underscore init”) is a special method in Python classes. It is automatically called when a new object (instance) of the class is created. Its primary purpose is to initialize the object’s attributes with the initial values.

- **Purpose:** Sets up the initial state of the object.
- **Arguments:** Takes `self` as the first argument (referring to the instance being created) and any other arguments needed to initialize the object’s attributes.
- **Automatic Call:** It’s called automatically when you create an instance of the class (e.g., `earth = Planet("Earth", 1.0)`).

```

Example 4: The power of Init - Creates a class of stars

class Star:
 """Represents a star with a name, right ascension, and declination.

 Attributes:
 name (str): The name of the star.
 ra (float): The right ascension of the star in degrees.

```

```

 dec (float): The declination of the star in degrees.
 """

 def __init__(self, name, ra, dec):
 """Initializes a Star object.

 Args:
 name (str): The name of the star.
 ra (float): The right ascension of the star in degrees.
 dec (float): The declination of the star in degrees.
 """
 self.name = name
 self.ra = ra
 self.dec = dec

Creating Star objects
sun = Star("Sun", 0, 0)
sirius = Star("Sirius", 101.2872, -16.7161) #RA = 101.2872 degrees, DEC = -16.7161 degrees
proxima_centauri = Star("Proxima Centauri", 217.4219, -62.6795)

Printing the attributes of the Star objects
print(f"The RA for {sun.name} is: {sun.ra}")
print(f"The DEC for {sun.name} is: {sun.dec}")

print(f"The RA for {sirius.name} is: {sirius.ra}")
print(f"The DEC for {sirius.name} is: {sirius.dec}")

print(f"The RA for {proxima_centauri.name} is: {proxima_centauri.ra}")
print(f"The DEC for {proxima_centauri.name} is: {proxima_centauri.dec}")

The RA for Sun is: 0
The DEC for Sun is: 0
The RA for Sirius is: 101.2872
The DEC for Sirius is: -16.7161
The RA for Proxima Centauri is: 217.4219
The DEC for Proxima Centauri is: -62.6795

```

### 9.2.1 Class with boolean attributes:

Python has boolean attributes that can be used inside a class.

### 9.2.2 The if statement in classes

Inside classes, we can create methods that utilizes if statements.

```

Example 5: if statements and boolean in classes

class Planet:
 """Represents an exoplanet with name, radius and habitability

 Attributes:
 name (str): Name of the exoplanet
 radius (float): Radius of the exoplanet
 isHabitable (bool): Boolean that determines if the planet is habitable
 """

 def __init__(self, name, radius, isHabitable):
 """Initializes a Planet object

 Args:
 name (str): Name of the exoplanet
 radius (float): Radius of the exoplanet
 isHabitable (bool): Boolean that determines if the planet is habitable
 """
 self.name = name
 self.radius = radius
 self.isHabitable = isHabitable

 def checkHabitability(self):
 """Checks if the exoplanet is habitable or not"""

 if self.isHabitable:
 print(f"{self.name} is a habitable exoplanet.")
 else:
 print(f"{self.name} is not a habitable exoplanet.")

Creating Planet objects
kepler186f = Planet("Kepler-186f", 1.11, True) #Radius = 1.11 Earth radii. Habitable =
kepler1649b = Planet("Kepler-1649b", 1.06, False) #Radius = 1.06 Earth radii. Habitable =

Printing the attributes of the Star objects
kepler186f.checkHabitability()
kepler1649b.checkHabitability()

```

```

Kepler-186f is a habitable exoplanet.
Kepler-1649b is not a habitable exoplanet.

```

### 9.2.3 Combining Classes

Classes can be combined with each other, in this way a planet can be attribute of the class Star.

```
Example 6: Combining classes

class Planet:
 """Represents a planet with name and radius

 Attributes:
 name (str): Name of the exoplanet
 radius (float): Radius of the exoplanet
 """

 def __init__(self, name, radius):
 """Initializes a Planet object

 Args:
 name (str): Name of the exoplanet
 radius (float): Radius of the exoplanet
 """
 self.name = name
 self.radius = radius

 def describe(self):
 """Prints a description of the planet"""

 print(f"{self.name} has a radius of {self.radius}")

class Star:
 """Represents a star with name, temp and list of planets

 Attributes:
 name (str): Name of the star
 temp (str): Surface temperature of the star
 planets (str): list of planet objects.
 """

 def __init__(self, name, temp, planets):
 """Initializes a Star object

 Args:
 name (str): Name of the star
 temp (str): Surface temperature of the star
 planets (str): list of planet objects.
```

```

 """
 self.name = name
 self.temp = temp
 self.planets = planets

 def describe(self):
 """Prints a description of the planet"""

 print(f"{self.name} has a surface temperature of {self.temp} and these planets")

 for planet in self.planets:
 print(planet.name)

Creating Planet objects
kepler186f = Planet("Kepler-186f", 1.11) #Radius = 1.11 Earth radii.
kepler1649b = Planet("Kepler-1649b", 1.06) #Radius = 1.06 Earth radii.
earth = Planet("Earth", 1) #Radius = 1.00 Earth radii.

Adding planets to a list
planets = [kepler186f, kepler1649b, earth]

Creating a Star objects
kepler186 = Star("Kepler-186", 3755, planets) #Surface Temp = 3755K

Printing the attributes of the Star objects
kepler186.describe()

Kepler-186 has a surface temperature of 3755 and these planets:
Kepler-186f
Kepler-1649b
Earth

```

### 9.2.4 Class to create rockets

We can also create classes to classify stars, adding attributes such as name, spectral type and luminosity class and methods such as evolve.

```

Example 7: Creating a class for Stars.

class Star:
 """Represents a Star with spectral and luminosity classification.

 Attributes:
 name (str): Name of the star.
 """

```

```

 spectral_type (str): Spectral type (O, B, A, F, G, K, M).
 luminosity_class (str): Luminosity class (I, II, III, IV, V).
 """

 def __init__(self, name, spectral_type, luminosity_class):
 """Initializes a Star object.

 Args:
 name (str): Name of the star.
 spectral_type (str): Spectral type of the star (e.g., 'G').
 luminosity_class (str): Luminosity class (e.g., 'V' for main sequence).
 """
 self.name = name
 self.spectral_type = spectral_type.upper()
 self.luminosity_class = luminosity_class.upper()

 def classify(self):
 """Returns the full stellar classification."""

 classification = f"{self.spectral_type}{self.luminosity_class}"
 print(f"{self.name} is classified as a {classification} star.")

 if classification == "G2V":
 print("This is a main-sequence star like the Sun.")
 elif self.luminosity_class == "I":
 print("This is a supergiant star.")
 elif self.luminosity_class == "III":
 print("This is a giant star.")
 else:
 print("This is a star of type", classification)

```

```

Example 1: A Sun-like main sequence star
star1 = Star("Alpha Centauri A", "G2", "V") # Sun-like star
star1.classify()

Example 2: A blue supergiant
star2 = Star("Rigel", "B8", "I") # Blue supergiant
star2.classify()

Example 3: A red giant
star3 = Star("Aldebaran", "K5", "III") # Red giant
star3.classify()

Example 4: A white main-sequence star
star4 = Star("Sirius A", "A1", "V") # Bright white main-sequence star

```

```

star4.classify()

Example 5: A red dwarf
star5 = Star("Proxima Centauri", "M5", "V") # Small red main-sequence star
star5.classify()

Example 6: A yellow subgiant
star6 = Star("Eta Bootis", "G0", "IV") # Subgiant transitioning from main sequence
star6.classify()

```

```

Alpha Centauri A is classified as a G2V star.
This is a main-sequence star like the Sun.
Rigel is classified as a B8I star.
This is a supergiant star.
Aldebaran is classified as a K5III star.
This is a giant star.
Sirius A is classified as a A1V star.
This is a star of type A1V
Proxima Centauri is classified as a M5V star.
This is a star of type M5V
Eta Bootis is classified as a G0IV star.
This is a star of type G0IV

```

### 9.2.5 Class for Compact Object

We can create classes for Compact Object with a collapse method.

```

Example 8: Creating a class for Compact Objects.

class CompactObject:
 """Represents a Compact Object (Black Hole, Neutron Star, White Dwarf)."""

 def __init__(self, name, object_type, mass, radius):
 self.name = name
 self.object_type = object_type.lower() # Store the type in lowercase
 self.mass = mass
 self.radius = radius

 def state(self):
 if self.object_type == "black hole":
 print(f"{self.name} is a black hole.")
 elif self.object_type == "neutron star":
 if self.mass > 1.4:
 print(f"{self.name} is a neutron star at the critical mass limit.")

```



```

 else:
 print(f"{self.name} is a stable neutron star.")
 elif self.object_type == "white dwarf":
 if self.mass > 1.4:
 print(f"{self.name} will collapse into a neutron star or black hole.")
 else:
 print(f"{self.name} is a stable white dwarf.")
 else:
 print(f"Unknown compact object type: {self.object_type}.")

def collapse(self):
 if self.mass > 3:
 print(f"{self.name} collapsed into a black hole.")
 self.object_type = "black hole"
 self.radius = 0
 elif self.mass > 1.4:
 print(f"{self.name} collapsed into a neutron star.")
 self.object_type = "neutron star"
 else:
 print(f"{self.name} remains a white dwarf.")

```

```

Example compact object objects
compact_obj1 = CompactObject("Cygnus X-1", "Black Hole", 14.8, 40) # Black hole example
compact_obj2 = CompactObject("PSR B1919+21", "Neutron Star", 1.44, 10) # Neutron star at critical mass
compact_obj3 = CompactObject("Sirius B", "White Dwarf", 1.0, 0.008) # White dwarf example

Checking the state of the objects
compact_obj1.state()
compact_obj2.state()
compact_obj3.state()

Simulating collapse if mass exceeds limit
compact_obj3.collapse()
compact_obj2.collapse()

```

```

Cygnus X-1 is a black hole.
PSR B1919+21 is a neutron star at the critical mass limit.
Sirius B is a stable white dwarf.
Sirius B remains a white dwarf.
PSR B1919+21 collapsed into a neutron star.

```

### 9.3 Exercises

1. Telescope Class: Create a `Telescope` class with attributes like `aperture` (diameter of the lens), `location` (latitude, longitude), and methods to `point_at` (take a target RA and Dec) and `take_image`.
2. Comet Class: Create a `Comet` class with attributes like `name`, `orbital_period`, and `closest_approach_date`. Add a method to `calculate_distance_from_sun` given a date.
3. Solar System: Create a Solar System class that contains the Sun object, and can include a method to describe the Sun and its Planet objects.
4. Define a class for Galaxies. The galaxy needs attributes such as num of stars, galaxy type and the presence or absense of a black hole.
5. Define a class for pulsars. The pulsars need attributes such as period, magnetic field. It also needs a method to calculate EM emmisions.

### 9.4 Summary

This notebook introduced the fundamental concepts of classes in Python. You learned how to define classes, create objects, define attributes and methods, and use the `__init__` method to initialize objects. Classes provide a powerful way to organize code and model real-world concepts, enabling you to create more complex simulations and analysis.

#### Additional Resources

Here is a python class documentation <https://docs.python.org/3/tutorial/classes.html>.

Read about Object Oriented Programming from Real Python.

## Chapter 10

# Files and Error Handling in Python

### 10.1 Opening Files in Python

In Python, working with files is a fundamental task. Before you can read from or write to a file, you must first *open* it. Opening a file creates a connection between your Python program and the file on your computer's file system.

#### The `open()` Function:

The key to opening files is the `open()` function. It takes two main arguments:

1. **filename:** The path to the file you want to open. This can be a relative path (relative to the current working directory) or an absolute path. For example:
  - `"my_data.txt"` (relative path: in the same folder as your Python script)
  - `"data/star_catalog.csv"` (relative path: in a subfolder named "data")
  - `"/home/user/documents/observational_data.fits"` (absolute path: specifies the exact location on the system; *nix based systems example*)
  - `"C:\\Users\\MyUser\\Documents\\data.txt"` (absolute path on Windows).
2. **mode:** A string that specifies how you want to use the file. The most common modes are:

- **"r": Read mode.** Opens the file for reading. This is the default mode if you don't specify one. You can only read data from the file. If the file doesn't exist, a `FileNotFoundError` will be raised.
- **"w": Write mode.** Opens the file for writing. If the file exists, its contents will be *overwritten* (all existing data will be lost!). If the file doesn't exist, a new file will be created. Use this mode with caution.
- **"a": Append mode.** Opens the file for writing. If the file exists, new data will be added to the *end* of the file. If the file doesn't exist, a new file will be created. This mode is useful for adding data to an existing file without erasing its contents.
- **"x": Exclusive Creation mode.** Creates a new file for writing. If the file already exists, it will raise a `FileExistsError`.
- **"b": Binary mode.** Opens the file in binary mode.
- **"t": Text mode.** Opens the file in text mode (default).
- **"+": Updating mode.** Opens the file for updating (reading and writing).

Example:

```
try:
 file_handle = open("star_catalog.txt", "r") # Open the file in read mode

 # Perform operations on the file (e.g., read data)
 # ...

 file_handle.close() # Important: Close the file when you're finished!

except FileNotFoundError:
 print("Error: The file 'star_catalog.txt' was not found.")

```python
# Example 1: Opening and Closing Files

# To work with a file, you first need to open it using the `open()` function. This cre
# which is a reference to the open file.

# Opening a file for reading:
try:
    file_handle = open("star_catalog.txt", "r") # "r" mode opens the file for reading
    print("File opened successfully!")

    # Perform operations on the file here (e.g., read data)

    file_handle.close() # Always close the file when you're finished with it. This re
    print("File closed.")
```

```
except FileNotFoundError:
    print("Error: File 'star_catalog.txt' not found.")
```

```
Error: File 'star_catalog.txt' not found.
```

Alternative (and recommended) approach using `with open()`.

```
#The with statement automatically closes the file for you, even if errors are encountered, making
try:
    with open("star_catalog.txt", "r") as file_handle:
        print("File opened successfully (using 'with'!)")
        # The file is automatically closed when the 'with' block ends.

except FileNotFoundError:
    print("Error: File 'star_catalog.txt' not found (using 'with').")
```

```
Error: File 'star_catalog.txt' not found (using 'with').
```

There is no `star_catalog.txt` hence why the code prints `Error: File 'star_catalog.txt' not found.`

To create the file and write into the file, run the next cell:

```
try:
    file_handle = open("star_catalog.txt", "r") # Try to open for reading
    print("File opened successfully!")

    # Read contents (optional)
    contents = file_handle.read()
    print("Contents:\n", contents)

    file_handle.close()
    print("File closed.")

except FileNotFoundError:
    print("File not found. Creating 'star_catalog.txt'...")

    # Create the file and write some info
    file_handle = open("star_catalog.txt", "w") # "w" mode creates the file if it doesn't exist
    file_handle.write("Star Name, Spectral Type, Magnitude\n")
    file_handle.write("Alpha Centauri A, G2V, 0.01\n")
    file_handle.write("Sirius A, A1V, -1.46\n")
    file_handle.close()
    print("File created and data written.")
```

```
File not found. Creating 'star_catalog.txt'...
File created and data written.
```

10.2 Reading Data from Files

Once a file is open for reading, you can use various methods to retrieve the data:

- `file_handle.read()`: Reads the entire file content as a single string.
- `file_handle.readline()`: Reads a single line from the file, including the newline character (`\n`).
- `file_handle.readlines()`: Reads all lines from the file and returns them as a list of strings.

For large files, it's more efficient to read the file line by line to avoid loading the entire content into memory at once.

Example 2: Reading Star Catalog from a File

```
try:
    with open("star_catalog.txt", "r") as file_handle:
        header = file_handle.readline().strip() # Read and skip the header

        for line in file_handle:
            line = line.strip()
            if line:
                star_data = line.split(",")
                if len(star_data) == 3:
                    star_name = star_data[0].strip()
                    spectral_type = star_data[1].strip()
                    try:
                        magnitude = float(star_data[2])
                        print(f"Star: {star_name}, Type: {spectral_type}, Magnitude: {magnitude}")
                    except ValueError:
                        print(f"Error: Invalid magnitude value for star {star_name}")
                else:
                    print(f"Error: Invalid format in line: {line}")
except FileNotFoundError:
    print("Error: File 'star_catalog.txt' not found.")
```

```
Star: Alpha Centauri A, Type: G2V, Magnitude: 0.01
Star: Sirius A, Type: A1V, Magnitude: -1.46
```

10.3 Writing Data to Files

To write data to a file, you need to open it in either "w" (write) or "a" (append) mode.

- `file_handle.write(string)`: Writes the given string to the file. Note that `write()` does not automatically add a newline character; you must include it explicitly (`\n`).
- `file_handle.writelines(list_of_strings)`: Writes a list of strings to the file.

```
# Example 3: Writing Exoplanet Data to a File

# Let's create a file to store exoplanet data:
# Exoplanet Name, Radius (Earth radii), Orbital Period (days)

exoplanets = [
    ("Kepler-186f", 1.11, 129.9),
    ("Kepler-1649b", 1.06, 8.68),
    ("TRAPPIST-1e", 0.91, 6.10),
]

try:
    with open("exoplanet_catalog.txt", "w") as file_handle:
        file_handle.write("Exoplanet Name,Radius (Earth radii),Orbital Period (days)\n") # Write header

        for name, radius, period in exoplanets:
            line = f"{name},{radius:.2f},{period:.2f}\n" # Format the data
            file_handle.write(line) # Write the line to the file

        print("Exoplanet data written to 'exoplanet_catalog.txt'.")

except IOError as e:
    print(f"Error writing to file: {e}")

#Appending:
try:
    with open("exoplanet_catalog.txt", "a") as file_handle:
        file_handle.write("kepler111,1.22,23.44\n")

    print("Appended succesfully")

except FileNotFoundError:
    print("Error: File 'exoplanet_catalog.txt' not found.")
```

```
except IOError as e:
    print(f"Error writing to file: {e}")
```

```
Exoplanet data written to 'exoplanet_catalog.txt'.
Appended succesfully
```

10.4 Error Handling: The try...except Block

Errors are inevitable when working with files and data. Python's try...except block allows you to handle exceptions gracefully, preventing your program from crashing.

The basic syntax is:

```
# try:
#     # Code that might raise an exception
# except ExceptionType1:
#     # Code to handle ExceptionType1
# except ExceptionType2:
#     # Code to handle ExceptionType2
# ...
# else:
#     # Code that executes if no exception was raised
# finally:
#     # Code that always executes (cleanup operations)
```

- **try:** The block of code that might raise an exception.
- **except:** One or more blocks of code that handle specific exception types. If an exception occurs within the try block, Python searches for a matching except block.
- **else:** An optional block of code that is executed only if no exception was raised in the try block.
- **finally:** An optional block of code that is always executed, regardless of whether an exception was raised or not. This is typically used for cleanup operations (e.g., closing files, releasing resources).

Example 4: Handling FileNotFoundError

```
try:
    with open("nonexistent_file.txt", "r") as file_handle:
        # This code will not be executed because the file doesn't exist
        content = file_handle.read()
    print(content)
```



```

except FileNotFoundError:
    print("Error: The specified file was not found.")

except Exception as e:
    print(f"An unexpected error occurred: {e}")

finally:
    print("Attempted to open the file.") # This will always be printed

```

```

Error: The specified file was not found.
Attempted to open the file.

```

10.5 Handling Different Exception Types in Python:

When working with files, data, or any external resource, errors are unavoidable. Python's exception handling mechanism allows you to gracefully manage these errors, preventing your program from crashing and providing informative feedback to the user. A key part of effective error handling is understanding and handling different types of exceptions.

Understanding Exception Types:

Exceptions are events that disrupt the normal flow of program execution. Each exception has a specific *type*, indicating the nature of the error. Recognizing common exception types is crucial for writing targeted error handling code. Here are some relevant exception types:

- **FileNotFoundError:** Raised when you try to open a file that doesn't exist.
- **IOError:** A general exception for input/output operations. Catches most file operations.
- **ValueError:** Raised when a function receives an argument of the correct type but an inappropriate value (e.g., trying to convert the string "abc" to an integer).
- **TypeError:** Raised when an operation or function is applied to an object of an inappropriate type (e.g., trying to add a string and an integer).
- **IndexError:** Raised when trying to access an index in a list that is out of bounds.

- **KeyError:** Raised when trying to access a key in a dictionary that doesn't exist.
- **ZeroDivisionError:** Raised when dividing by zero.

The try...except Block with Specific Exception Types:

The try...except block allows you to attempt an operation that might raise an exception and handle the exception if it occurs. You can specify multiple except blocks to handle different exception types differently.

```
try:
    # Code that might raise an exception
    result = 10 / 0 # This will raise a ZeroDivisionError
except ZeroDivisionError:
    # Code to handle the ZeroDivisionError
    print("Error: Division by zero is not allowed.")
except TypeError:
    print("Type Error")
except ValueError:
    print("Value error")
except Exception as e: #Generic Exception
    # Code to handle any other exception
    print(f"An unexpected error occurred: {e}")

```python
Example 5: Handling Different Exception Types - RA/Dec Conversion

def convert_ra_dec(ra_str, dec_str):
 """Converts right ascension (RA) and declination (Dec) strings to floats.

 Args:
 ra_str (str): Right ascension string (e.g., "12h34m56.7s").
 dec_str (str): Declination string (e.g., "+34d56m7.8s").

 Returns:
 tuple: A tuple containing the RA and Dec as floats in degrees, or None if an error occurs.
 """
 try:
 ra = float(ra_str) #Dummy value to simulate an error
 dec = float(dec_str) #Dummy value to simulate an error

 return ra, dec

 except ValueError:
 print("Error: Invalid format in RA or Dec value.")
```

```

 return None

 except TypeError:
 print("Error: RA and Dec must be strings")
 return None

Test cases
star_ra = "Invalid RA"
star_dec = "+34d56m7.8s"

coords = convert_ra_dec(star_ra, star_dec)

if coords:
 ra, dec = coords
 print(f"Converted RA: {ra:.4f}, Dec: {dec:.4f}")

```

```
Error: Invalid format in RA or Dec value.
```

## 10.6 The finally Block

The `finally` block is an optional part of the `try...except` structure.

The code inside the `finally` block is *always* executed, regardless of whether an exception was raised or not.

This is extremely useful for cleanup operations, such as closing files, releasing resources, or ensuring that certain actions are always performed.

```

Example 6: Using `finally` to Ensure File Closure

file_handle = None # Initialize the file handle to None

try:
 file_handle = open("exoplanet_catalog.txt", "r")
 # Perform operations on the file
 content = file_handle.read()
 print(content)
except FileNotFoundError:
 print("Error: File 'messier_objects.txt' not found.")
except Exception as e:
 print(f"An unexpected error occurred: {e}")
finally:
 if file_handle: # Check if the file was opened
 file_handle.close()

```

```

 print("File closed (in 'finally' block).")
 else:
 print("File was not opened.")

```

```

Exoplanet Name,Radius (Earth radii),Orbital Period (days)
Kepler-186f,1.11,129.90
Kepler-1649b,1.06,8.68
TRAPPIST-1e,0.91,6.10
kepler111,1.22,23.44

```

```

File closed (in 'finally' block).

```

## 10.7 Raising Custom Exceptions

Sometimes, you need to signal specific error conditions that are not covered by standard Python exceptions.

You can raise custom exceptions by creating new exception classes (inheriting from the base `Exception` class) and using the `raise` statement.

This allows you to provide more informative error messages and handle specific situations more effectively.

*# Example 7: Raising a Custom Exception - MassValue Error*

```

class InvalidMassError(Exception):
 """Custom exception raised when a star's mass is invalid."""
 pass

def validate_star_mass(mass):
 """Validates that a star's mass is within a reasonable range.

 Args:
 mass (float): The mass of the star in solar masses.

 Raises:
 InvalidMassError: If the mass is outside the valid range (0.08 to 100 solar ma.

 Returns:
 float: The valid mass.
 """
 if not 0.08 <= mass <= 100:
 raise InvalidMassError("Error: Star mass must be between 0.08 and 100 solar ma
 return mass

```

```
Usage
try:
 star_mass = 200.0
 validated_mass = validate_star_mass(star_mass)
 print(f"Valid star mass: {validated_mass} solar masses")
except InvalidMassError as e:
 print(e)
```

```
Error: Star mass must be between 0.08 and 100 solar masses.
```

## 10.8 Exercises

1. **Data Validation:** Write a program that reads star data (name, RA, Dec, magnitude) from a file and validates that the RA and Dec values are within a reasonable range (0-360 for RA, -90 to +90 for Dec). Raise a custom exception if the values are invalid.
2. **File Conversion:** Write a program that reads star data from a file in one format (e.g., CSV) and converts it to another format (e.g., a fixed-width format). Handle potential errors such as missing data or invalid data types.
3. **Log File Analysis:** Write a program that reads a telescope log file and extracts information about observing sessions (start time, end time, target object). Handle potential errors in the log file format.

## 10.9 Summary

This notebook covered the essentials of file and error handling in Python, providing you with the tools to read, write, and process data from files and manage potential errors gracefully. By mastering these concepts, you can write more reliable code, capable of handling real-world data and unexpected situations.

## 10.10 Additional Resources

On working with files, see <https://realpython.com/working-with-files-in-python/>.

For error handling and exceptions, see <https://docs.python.org/3/tutorial/errors.html>, <https://realpython.com/python-exceptions/>.



## Chapter 11

# Modules and Packages in Python

*# Example 1: Importing a Module*

```
import math # Imports the 'math' module, which provides mathematical functions.
```

*# Using functions from the 'math' module:*

```
square_root = math.sqrt(16) # Calculate the square root of 16
print(f"The square root of 16 is: {square_root}")
```

```
pi_value = math.pi # Access the value of pi
print(f"The value of pi is: {pi_value}")
```

```
The square root of 16 is: 4.0
```

```
The value of pi is: 3.141592653589793
```

### 11.1 Importing Specific Items from a Module: `from...import`

Instead of importing the entire module, you can import specific functions, classes, or variables using the `from...import` statement.

This can make your code more concise and easier to read.

*# Example 2: Importing Specific Items*

```
from math import sqrt, pi #Importing only sqrt and pi

Now you can use 'sqrt' and 'pi' directly, without the 'math.' prefix:
square_root = sqrt(25) #sqrt is not math.sqrt anymore
print(f"The square root of 25 is: {square_root}")

print(f"The value of pi is: {pi}")
```

```
The square root of 25 is: 5.0
The value of pi is: 3.141592653589793
```

## 11.2 Importing with Aliases: `as`

You can assign a shorter or more descriptive name to a module or item using the `as` keyword.

This is useful for avoiding naming conflicts or making your code more readable.

*# Example 3: Importing with Aliases*

```
import math as m #Alias = m

Now you can use 'm' as a shorthand for 'math':
square_root = m.sqrt(36) #Use m
print(f"The square root of 36 is: {square_root}")
```

```
The square root of 36 is: 6.0
```

## 11.3 Exploring Local Modules: `string`, `array`, `time`

Python comes with a variety of built-in modules that provide useful functionality for common tasks. Let's explore a few of these modules:

- **string:** Provides constants and classes for working with strings.
- **array:** Provides an array object that is more efficient for storing numerical data than lists.
- **time:** Provides functions for working with time-related tasks (e.g., getting the current time, pausing execution).



For more on python built-in libraries, see <https://docs.python.org/3/library/index.html>.

```
Example 4: Using the 'string' Module

import string #import the string module

Accessing string constants:
uppercase_letters = string.ascii_uppercase
print(f"Uppercase letters: {uppercase_letters}") #Access the alphabet

Using the built-in replace function.
text = "This is a test to be replaced."
print(text.replace("test", "success"))
```

```
Uppercase letters: ABCDEFGHIJKLMNOPQRSTUVWXYZ
This is a success to be replaced.
```

```
Example 5: Using the 'array' Module

import array #import the array module

Creating an array of integers:
my_array = array.array('i', [1, 2, 3, 4, 5]) #Array type and values

print(f"My array: {my_array}")
```

```
My array: array('i', [1, 2, 3, 4, 5])
```

```
Example 6: Using the 'time' Module

import time #import the time module

Getting the current time:
current_time = time.ctime()
print(f"Current time: {current_time}")

Pausing execution for 2 seconds:
print("Waiting for 2 seconds...")
time.sleep(2)
print("Done waiting.")
```

```
Current time: Tue May 6 18:40:41 2025
Waiting for 2 seconds...
Done waiting.
```

## 11.4 Installing External Packages with pip

The real power of Python comes from its large ecosystem of external packages available on PyPI (Python Package Index).

To install these packages, you'll use `pip`, the package installer for Python.

To install a package, open your terminal or command prompt and run the following command:

```
pip install package_name
```

Alternatively, you can install from a jupyter notebook with same command as above with an exclamation mark (!). See the next cell.

```
!pip install numpy
```

```
Requirement already satisfied: numpy in /Users/user1/Data/brics_astro/.venv/lib/python:
```

```
[notice] A new release of pip is available: 23.0.1 -> 25.1.1
[notice] To update, run: pip install --upgrade pip
```

## 11.5 Using External Packages: `numpy`, `pandas`

Let's explore two popular external packages:

- **numpy:** A fundamental package for numerical computing in Python. It provides support for arrays, matrices, and mathematical functions optimized for numerical operations.
- **pandas:** A powerful package for data analysis and manipulation. It provides data structures like DataFrames that make it easy to work with structured data.

```
Example 7: Using 'numpy' for Astronomical Calculations
```

```
import numpy as np #Standard convention
```

```

Creating a numpy array of stellar magnitudes:
magnitudes = np.array([5.2, 4.8, 6.1, 3.9, 7.0]) #Create an array

Calculating the mean magnitude:
mean_magnitude = np.mean(magnitudes) #Calculate the mean

print(f"Mean magnitude: {mean_magnitude:.2f}")

Converting magnitudes to flux (relative to a reference magnitude):
fluxes = 10**(-0.4 * magnitudes) #Math operation with numpy
print(f"Fluxes: {fluxes}")

totalFlux = np.sum(fluxes) #Sum of all fluxes
print(f"Total Fluxes: {totalFlux}")

```

```

Mean magnitude: 5.40
Fluxes: [0.00831764 0.01202264 0.00363078 0.02754229 0.00158489]
Total Fluxes: 0.05309824283074463

```

```

Example 8: Using 'pandas' to Read Exoplanet Data from a CSV File

import pandas as pd #Standard convention

Assume we have a CSV file named "exoplanet_data.csv" with the following format:
Name,Radius,OrbitalPeriod
Kepler-186f,1.11,129.9
Kepler-1649b,1.06,8.68
TRAPPIST-1e,0.91,6.10

Reading the CSV file into a pandas DataFrame:
exoplanet_data = pd.read_csv("exoplanet_catalog.txt")

Displaying the first few rows of the DataFrame:
print(exoplanet_data.head()) #Prints first lines of the csv

Calculating the mean radius of the exoplanets:
mean_radius = exoplanet_data["Radius (Earth radii)"].mean() #Calculating the mean
print(f"Mean radius: {mean_radius:.2f}")

Filtering exoplanets with an orbital period less than 10 days:
short_period_exoplanets = exoplanet_data[exoplanet_data["Orbital Period (days)"] < 10] #Condition
print(f"\nExoplanets with orbital period < 10 days:\n{short_period_exoplanets}")

```

	Exoplanet Name	Radius (Earth radii)	Orbital Period (days)
0	Kepler-186f	1.11	129.90
1	Kepler-1649b	1.06	8.68
2	TRAPPIST-1e	0.91	6.10
3	kepler111	1.22	23.44
Mean radius: 1.07			
Exoplanets with orbital period < 10 days:			
	Exoplanet Name	Radius (Earth radii)	Orbital Period (days)
1	Kepler-1649b	1.06	8.68
2	TRAPPIST-1e	0.91	6.10

## 11.6 Managing Packages: `pip list`, `pip show`, `pip uninstall`

`pip` provides several commands for managing your installed packages:

- `pip list`: Lists all installed packages.
- `pip show package_name`: Displays information about a specific package (e.g., version, location, dependencies).
- `pip uninstall package_name`: Uninstalls a package.

You can run these commands in your terminal or command prompt.

## 11.7 Namespaces and Avoiding Naming Conflicts

Modules create their own namespaces, which helps avoid naming conflicts. This means that you can have variables or functions with the same name in different modules without them interfering with each other.

When you import a module, you access its contents through its namespace (e.g., `math.sqrt`). This ensures that you're using the correct `sqrt` function, even if you have another variable or function named `sqrt` in your own code.

This isolation of code is helpful for creating large programs where different parts may have variables with the same name.

## 11.8 Exercises

1. Circular Area Calculation: Create a module called `circle_utils.py` that contains a function to calculate the area of a circle given its radius. Import this module and use it to calculate the area of a circular telescope mirror.
2. Temperature Conversion: Use the `math` module to implement functions for converting temperatures between Celsius and Fahrenheit. Create a module called `temp_converter.py` and import those methods.
3. String Formatting: Use the `string` module to create a function that generates formatted strings for astronomical coordinates (RA and Dec) with specified precision.
4. Array Statistics: Use the `numpy` module to calculate the mean, median, and standard deviation of a dataset of stellar magnitudes.
5. Exoplanet Data Analysis: Use the `pandas` module to read an exoplanet dataset from a CSV file and perform some basic analysis (e.g., find the exoplanet with the largest radius, calculate the average orbital period).

## 11.9 Summary

This notebook provided a comprehensive introduction to modules and packages in Python. You learned how to import modules, use their contents, install external packages with `pip`, and manage your package environment. By leveraging the power of modules and packages, you can significantly enhance your Python programming capabilities and tackle a wide range of astronomical challenges. Remember to explore the PyPI repository to discover new and exciting packages that can help you in your research and projects.



## Chapter 12

# NumPy: Handling Numerical Data

```
!pip install numpy pandas matplotlib
```

### 12.1 Why Numpy

NumPy is Essential for Numerical Tasks.

Imagine you have a list of star brightness measurements defined as `star_brightnesses`.

```
star_brightnesses = [10.5, 12.1, 9.8, 11.5, 13.0]
```

If you wanted to multiply each brightness by 2 (maybe converting units), with lists you might do:

```
doubled_brightnesses_list = []
for brightness in star_brightnesses:
 doubled_brightnesses_list.append(brightness * 2)

print("Using Python list and loop:", doubled_brightnesses_list)
```

```
Using Python list and loop: [21.0, 24.2, 19.6, 23.0, 26.0]
```

This works, but for very large datasets (like millions of pixels in an image), this loop can be slow.

NumPy allows us to do this much more simply and efficiently:

```
import numpy as np # Standard way to import NumPy

Convert the list to a NumPy array
star_brightnesses_np = np.array(star_brightnesses)

Multiply the *entire array* by 2
doubled_brightnesses_np = star_brightnesses_np * 2

print("Using NumPy:", doubled_brightnesses_np)
```

```
Using NumPy: [21. 24.2 19.6 23. 26.]
```

This ability to perform operations on whole arrays at once is called **vectorisation**, and it is one reason NumPy is efficient for numerical calculations.

## 12.2 The ndarray: NumPy's Core Object

The central feature of NumPy is the `ndarray` object. An `ndarray` (N-dimensional array) is a grid of values, all of the same data type. You can think of it like a table (2 dimensions), or even a cube of numbers (3 dimensions), or more!

- **Homogeneous:** All elements in a NumPy array must be of the same data type (e.g., all integers, all floating-point numbers). This is a key difference from Python lists and helps with efficiency.
- **N-dimensional:** Can have any number of dimensions (1D for a list of values, 2D for a table, 3D for image cubes, etc.).

### 12.2.1 Example 1: Creating Arrays

```
1. From a Python list or tuple:
A 1-dimensional array (like a list)
star_magnitudes = [2.5, 1.8, 0.1, -1.4]
magnitude_array = np.array(star_magnitudes)
print("1D Array from list:", magnitude_array)

A 2-dimensional array (like a table)
Representing [RA, Dec, Magnitude] for two stars
```



```

star_data = [
 [101.28, -16.71, -1.46], # Sirius
 [37.95, 89.26, -2.08] # Polaris
]
star_array_2d = np.array(star_data)
print("\n2D Array from list of lists:")
print(star_array_2d)

```

```
1D Array from list: [2.5 1.8 0.1 -1.4]
```

```
2D Array from list of lists:
```

```
[[101.28 -16.71 -1.46]
 [37.95 89.26 -2.08]]
```

```

2. Using built-in NumPy functions:
An array of zeros (useful for initializing data)
zeros_array = np.zeros(5) # A 1D array of 5 zeros
print("\nArray of zeros:", zeros_array)

A 2x3 array of ones
ones_array = np.ones((2, 3)) # Shape is given as a tuple (rows, columns)
print("\nArray of ones (2x3):")
print(ones_array)

An array with values from 0 up to (but not including) 10
arange_array = np.arange(10)
print("\nArray using arange(10):", arange_array)

An array of 5 evenly spaced values between 0 and 1 (inclusive)
linspace_array = np.linspace(0, 1, 5)
print("\nArray using linspace(0, 1, 5):", linspace_array)

```

```
Array of zeros: [0. 0. 0. 0. 0.]
```

```
Array of ones (2x3):
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

```
Array using arange(10): [0 1 2 3 4 5 6 7 8 9]
```

```
Array using linspace(0, 1, 5): [0. 0.25 0.5 0.75 1.]
```

## 12.3 Array Attributes: What an Array Knows About Itself

NumPy arrays have several useful attributes that tell you about their structure and contents.

- `.shape`: A tuple indicating the size of each dimension (rows, columns, etc.).
- `.size`: The total number of elements in the array.
- `.ndim`: The number of dimensions (e.g., 1 for a 1D array, 2 for a 2D array).
- `.dtype`: The data type of the elements in the array (e.g., `int64`, `float64`).

### 12.3.1 Example 2: Array Attributes

Let us use the arrays we created earlier:

```
magnitude_array = np.array([2.5, 1.8, 0.1, -1.4])
star_array_2d = np.array([[101.28, -16.71, -1.46], [37.95, 89.26, -2.08]])

print("--- Attributes of 1D Array ---")
print("Array:", magnitude_array)
print("Shape:", magnitude_array.shape) # (4,) means 1 dimension with 4 elements
print("Size:", magnitude_array.size) # Total elements
print("Number of dimensions:", magnitude_array.ndim)
print("Data type:", magnitude_array.dtype)

print("\n--- Attributes of 2D Array ---")
print("Array:")
print(star_array_2d)
print("Shape:", star_array_2d.shape) # (2, 3) means 2 rows, 3 columns
print("Size:", star_array_2d.size)
print("Number of dimensions:", star_array_2d.ndim)
print("Data type:", star_array_2d.dtype) # Notice NumPy often chooses float by default

--- Attributes of 1D Array ---
Array: [2.5 1.8 0.1 -1.4]
Shape: (4,)
Size: 4
Number of dimensions: 1
Data type: float64

--- Attributes of 2D Array ---
Array:
```

```
[[101.28 -16.71 -1.46]
 [37.95 89.26 -2.08]]
Shape: (2, 3)
Size: 6
Number of dimensions: 2
Data type: float64
```

## 12.4 Array Indexing and Slicing: Accessing Elements

Just like with Python lists, you can access individual elements or portions of a NumPy array using indexing and slicing.

- **Indexing:** Accessing a single element using its position (index). Remember Python is 0-indexed!
- **Slicing:** Accessing a range of elements.

For 2D arrays, you use a comma to separate the row index/slice from the column index/slice: `array[row_index_or_slice, column_index_or_slice]`.

### 12.4.1 Example 3: Indexing and Slicing

```
star_data = np.array([
 [101.28, -16.71, -1.46], # Row 0 (Sirius)
 [37.95, 89.26, -2.08], # Row 1 (Polaris)
 [217.42, -62.68, 0.13] # Row 2 (Proxima Centauri)
])
print("Original Array:")
print(star_data)

print("\n--- Indexing ---")
Get the value at row 1, column 2 (Polaris's magnitude)
polaris_magnitude = star_data[1, 2]
print("Polaris's magnitude:", polaris_magnitude)
```

```
Original Array:
[[1.0128e+02 -1.6710e+01 -1.4600e+00]
 [3.7950e+01 8.9260e+01 -2.0800e+00]
 [2.1742e+02 -6.2680e+01 1.3000e-01]]
```

```
--- Indexing ---
```

```
Polaris's magnitude: -2.08
```

```
Get the value at row 0, column 0 (Sirius's RA)
```

```
sirius_ra = star_data[0, 0]
```

```
print("Sirius's RA:", sirius_ra)
```

```
Get an entire row (Row 2 - Proxima Centauri data)
```

```
proxima_row = star_data[2, :] # ':' means select all columns in this row
```

```
print("\nProxima Centauri's data (row 2):", proxima_row)
```

```
Get an entire column (Column 1 - Declination for all stars)
```

```
declination_column = star_data[:, 1] # ':' means select all rows in this column
```

```
print("Declinations of all stars (column 1):", declination_column)
```

```
print("\n--- Slicing ---")
```

```
Get the first two rows
```

```
first_two_rows = star_data[0:2, :] # From row 0 up to (not including) row 2, all columns
```

```
print("First two rows:")
```

```
print(first_two_rows)
```

```
Sirius's RA: 101.28
```

```
Proxima Centauri's data (row 2): [2.1742e+02 -6.2680e+01 1.3000e-01]
```

```
Declinations of all stars (column 1): [-16.71 89.26 -62.68]
```

```
--- Slicing ---
```

```
First two rows:
```

```
[[101.28 -16.71 -1.46]
```

```
 [37.95 89.26 -2.08]]
```

```
Get columns 1 and 2 (Dec and Magnitude) for all stars
```

```
dec_mag_columns = star_data[:, 1:3] # All rows, from column 1 up to (not including) column 3
```

```
print("Declination and Magnitude columns:")
```

```
print(dec_mag_columns)
```

```
Get the RA and Dec for the first two stars
```

```
first_two_stars_ra_dec = star_data[0:2, 0:2]
```

```
print("RA and Dec for first two stars:")
```

```
print(first_two_stars_ra_dec)
```

```
Declination and Magnitude columns:
```

```
[[-16.71 -1.46]
```

```
[89.26 -2.08]
[-62.68 0.13]]
RA and Dec for first two stars:
[[101.28 -16.71]
 [37.95 89.26]]
```

```
Modifying an element using indexing
star_data[1, 2] = -1.80 # Update Polaris's magnitude
print("\nArray after updating Polaris's magnitude:")
print(star_data)
```

```
Array after updating Polaris's magnitude:
[[1.0128e+02 -1.6710e+01 -1.4600e+00]
 [3.7950e+01 8.9260e+01 -1.8000e+00]
 [2.1742e+02 -6.2680e+01 1.3000e-01]]
```

## 12.5 Basic Operations on Arrays

You can perform mathematical operations directly on entire arrays. NumPy applies the operation element-wise (to each element) or uses broadcasting when operating between arrays of different shapes or between an array and a single number.

### 12.5.1 Example 4: Array Arithmetic

```
magnitudes = np.array([2.5, 1.8, 0.1, -1.4])
offsets = np.array([0.1, 0.1, 0.1, 0.1]) # An array of same shape

Addition: Add an offset to all magnitudes
shifted_magnitudes = magnitudes + offsets
print("Magnitudes + offset array:", shifted_magnitudes)

Subtraction:
subtracted_values = magnitudes - offsets
print("Magnitudes - offset array:", subtracted_values)
```

```
Magnitudes + offset array: [2.6 1.9 0.2 -1.3]
Magnitudes - offset array: [2.4 1.7 0. -1.5]
```

```

Multiplication: Double the magnitudes
doubled_magnitudes = magnitudes * 2 # Multiplying by a single number (broadcasting)
print("Magnitudes * 2:", doubled_magnitudes)

Division: Divide magnitudes by 4
divided_magnitudes = magnitudes / 4.0
print("Magnitudes / 4:", divided_magnitudes)

Exponentiation: Calculate 10 raised to the power of -0.4 * magnitude
This is how you convert magnitudes back to relative flux/brightness
relative_fluxes = 10**(-0.4 * magnitudes)
print("Magnitudes converted to relative fluxes:", relative_fluxes)

```

```

Magnitudes * 2: [5. 3.6 0.2 -2.8]
Magnitudes / 4: [0.625 0.45 0.025 -0.35]
Magnitudes converted to relative fluxes: [0.1 0.19054607 0.91201084 3.63078055]

```

## 12.5.2 Example 5: Array Comparisons

```

magnitudes = np.array([2.5, 1.8, 0.1, -1.4, 3.0, 0.5])

Find stars brighter than magnitude 1.0 (remember smaller magnitude = brighter)
is_brighter_than_1 = magnitudes < 1.0
print("Is magnitude < 1.0?", is_brighter_than_1) # Returns a boolean array

```

```

Is magnitude < 1.0? [False False True True False True]

```

```

Use the boolean array for filtering (Fancy Indexing)
bright_stars = magnitudes[is_brighter_than_1]
print("Magnitudes of stars brighter than 1.0:", bright_stars)

```

```

Magnitudes of stars brighter than 1.0: [0.1 -1.4 0.5]

```

```

Find stars with magnitude exactly equal to 0.1
is_mag_0_1 = magnitudes == 0.1
print("\nIs magnitude == 0.1?", is_mag_0_1)

```

```

Is magnitude == 0.1? [False False True False False False]

```

```
Find stars with magnitude greater than 0 AND less than 2
Need parentheses for each comparison
is_between_0_and_2 = (magnitudes > 0) & (magnitudes < 2) # Use '&' for element-wise AND
print("\nIs magnitude between 0 and 2?", is_between_0_and_2)
print("Magnitudes between 0 and 2:", magnitudes[is_between_0_and_2])
```

```
Is magnitude between 0 and 2? [False True True False False True]
Magnitudes between 0 and 2: [1.8 0.1 0.5]
```

### 12.5.3 Example 6: Common NumPy Functions

```
stellar_masses = np.array([0.8, 1.2, 0.5, 2.0, 0.9]) # Masses in solar masses

Sum: Calculate the total mass of these stars
total_mass = np.sum(stellar_masses)
print("Total mass:", total_mass)

Mean: Calculate the average mass
average_mass = np.mean(stellar_masses)
print("Average mass:", average_mass)
```

```
Total mass: 5.4
Average mass: 1.08
```

```
Max: Find the maximum mass
max_mass = np.max(stellar_masses)
print("Maximum mass:", max_mass)

Min: Find the minimum mass
min_mass = np.min(stellar_masses)
print("Minimum mass:", min_mass)
```

```
Maximum mass: 2.0
Minimum mass: 0.5
```

```
Other useful functions (element-wise):
orbital_phases_radians = np.array([0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi]) # Angles in radians
print("\nOrbital phases (radians):", orbital_phases_radians)
print("Cosine of phases:", np.cos(orbital_phases_radians)) # Cosine applied to each element
print("Sine of phases:", np.sin(orbital_phases_radians)) # Sine applied to each element
```

```
Orbital phases (radians): [0. 1.57079633 3.14159265 4.71238898 6.28318531]
Cosine of phases: [1.0000000e+00 6.1232340e-17 -1.0000000e+00 -1.8369702e-16
 1.0000000e+00]
Sine of phases: [0.0000000e+00 1.0000000e+00 1.2246468e-16 -1.0000000e+00
-2.4492936e-16]
```

## 12.6 Putting it Together: Astronomy Examples with NumPy

Let's look at some more complete examples applying NumPy to astronomical problems.

### 12.6.1 Example 7: Analyzing a Simple Star Catalog Array

Imagine this array represents [Temperature (K), Luminosity (Solar Lum), Distance (pc)] for several stars

```
star_catalog_data = np.array([
 [5778, 1.0, 4.85], # Sun
 [9940, 83.0, 2.64], # Sirius A
 [4500, 0.3, 10.0], # Alpha Centauri B
 [25000, 10000.0, 100.0] # O-type star example
])
print(star_catalog_data)
```

```
[[5.778e+03 1.000e+00 4.850e+00]
 [9.940e+03 8.300e+01 2.640e+00]
 [4.500e+03 3.000e-01 1.000e+01]
 [2.500e+04 1.000e+04 1.000e+02]]
```

```
Select the Temperature column
temperatures = star_catalog_data[:, 0]
print("Temperatures:", temperatures)

Select the Luminosity column
luminosities = star_catalog_data[:, 1]
print("Luminosities:", luminosities)
```

```
Temperatures: [5778. 9940. 4500. 25000.]
Luminosities: [1.0e+00 8.3e+01 3.0e-01 1.0e+04]
```



## 12.6. PUTTING IT TOGETHER: ASTRONOMY EXAMPLES WITH NUMPY 117

```
Select the Distance column
distances_pc = star_catalog_data[:, 2]
print("Distances (pc):", distances_pc)

Calculate the average temperature of these stars
average_temp = np.mean(temperatures)
print(f"\nAverage temperature: {average_temp:.1f} K")
```

```
Distances (pc): [4.85 2.64 10. 100.]
```

```
Average temperature: 11304.5 K
```

```
Calculate the total luminosity
total_luminosity = np.sum(luminosities)
print(f"Total luminosity: {total_luminosity:.1f} Solar Lum")

Convert distances from parsecs (pc) to light-years (ly) (1 pc = 3.26 ly)
distances_ly = distances_pc * 3.26
print(f"Distances (ly): {distances_ly}")
```

```
Total luminosity: 10084.3 Solar Lum
```

```
Distances (ly): [15.811 8.6064 32.6 326.]
```

### 12.6.2 Example 8: Simulating Brightness Decrease with Distance

Assume initial brightness at a reference distance is 100 (arbitrary units)

```
initial_brightness = 100.0

Create an array of increasing distances
distances = np.array([1, 2, 3, 4, 5, 10]) # Distances in some unit

Brightness decreases with the square of the distance (Brightness = Initial / Distance^2)
brightness_at_distance = initial_brightness / (distances ** 2)

print(f"Brightness at distances {distances}: \n {brightness_at_distance}")
```

```
Brightness at distances [1 2 3 4 5 10]:
[100. 25. 11.11111111 6.25 4.
 1.]
```

```
Find distances where brightness is below a threshold (e.g., 5)
threshold = 5.0
is_below_threshold = brightness_at_distance < threshold
print(f"\nIs brightness below {threshold}? {is_below_threshold}")
print(f"Distances where brightness is below {threshold}: {distances[is_below_threshold]}
```

```
Is brightness below 5.0? [False False False False True True]
Distances where brightness is below 5.0: [5 10]
```

### 12.6.3 Example 9: Simple 2D Image Data (Grayscale)

Imagine a small 3x3 pixel image represented as a 2D array of pixel intensity values

```
image_data = np.array([
 [10, 20, 15],
 [30, 25, 35],
 [5, 15, 10]
])
print("Image Data (3x3 array):")
print(image_data)
print("Shape of image:", image_data.shape)
```

```
Image Data (3x3 array):
[[10 20 15]
 [30 25 35]
 [5 15 10]]
Shape of image: (3, 3)
```

```
Accessing a specific pixel (e.g., the center pixel)
center_pixel = image_data[1, 1]
print("\nCenter pixel value:", center_pixel)
```

```
Center pixel value: 25
```

```
Get a row of pixels (e.g., the middle row)
middle_row = image_data[1, :]
print("Middle row of pixels:", middle_row)
```

```
Middle row of pixels: [30 25 35]
```

```
Get a column of pixels (e.g., the first column)
first_column = image_data[:, 0]
print("First column of pixels:", first_column)
```

```
First column of pixels: [10 30 5]
```

```
Calculate the average pixel intensity in the image
average_intensity = np.mean(image_data)
print(f"\nAverage pixel intensity: {average_intensity:.2f}")
```

```
Average pixel intensity: 18.33
```

```
Find pixels with intensity greater than 20
bright_pixels_mask = image_data > 20
print("Mask for bright pixels (>20):\n", bright_pixels_mask)
```

```
Mask for bright pixels (>20):
[[False False False]
 [True True True]
 [False False False]]
```

```
Get the values of bright pixels
bright_pixel_values = image_data[bright_pixels_mask]
print("Values of bright pixels:", bright_pixel_values)
```

```
Values of bright pixels: [30 25 35]
```

## 12.7 Saving and Loading NumPy Arrays

In astronomy, you'll often need to save arrays you've created or manipulated to a file so you can use them later or share them with others. You'll also need to load arrays that someone else has saved. NumPy provides convenient functions for this.

There are two main approaches for beginners:

1. **NumPy's binary format (.npz):** This is the most efficient way to save and load NumPy arrays while preserving their exact data type and shape. The files are not human-readable.

2. **Text format (.txt, .csv, etc.):** This saves the data in a human-readable format, often with values separated by commas (CSV) or spaces. These files can be opened by other programs (like spreadsheets) but might not preserve all NumPy-specific information.

Let's look at how to use `np.save()`, `np.load()`, `np.savetxt()`, and `np.loadtxt()`.

### 12.7.1 Example 10: Saving and Loading with .npy format

Imagine this is an array of calculated star distances in light-years

```
star_distances_ly = np.array([4.37, 8.60, 10.0, 14.5, 36.0])
```

### 12.7.2 Saving the array to a .npy file

`np.save()` takes the filename (without extension, `.npy` is added automatically) and the array you want to save.

```
np.save("my_star_distances", star_distances_ly) # This will create a file named "my_star_distances.npy"
print("Array saved to 'my_star_distances.npy'")
```

```
Array saved to 'my_star_distances.npy'
```

### 12.7.3 Loading the array from the .npy file

`np.load()` takes the filename (including the `.npy` extension)

```
loaded_distances = np.load("my_star_distances.npy")
print("\nArray loaded from 'my_star_distances.npy':")
print(loaded_distances)
```

```
Array loaded from 'my_star_distances.npy':
[4.37 8.6 10. 14.5 36.]
```

```
Verify it's a NumPy array and has the same data
print("Type of loaded data:", type(loaded_distances))
print("Loaded data matches original:", np.array_equal(star_distances_ly, loaded_distances))
```

```
Type of loaded data: <class 'numpy.ndarray'>
Loaded data matches original: True
```

### 12.7.4 Example 11: Saving and Loading with Text Files (.txt or .csv)

Imagine this 2D array is data for a few exoplanets: [Mass (Earth), Radius (Earth), Orbital Period (days)]

```
exoplanet_table = np.array([
 [1.0, 1.0, 365.25], # Earth
 [0.11, 0.53, 687.0], # Mars
 [317.8, 11.2, 4331.0] # Jupiter
])
print("Exoplanet data to save:")
print(exoplanet_table)
```

```
Exoplanet data to save:
[[1.0000e+00 1.0000e+00 3.6525e+02]
 [1.1000e-01 5.3000e-01 6.8700e+02]
 [3.1780e+02 1.1200e+01 4.3310e+03]]
```

### 12.7.5 Saving the array to a text file (e.g., CSV)

`np.savetxt()` is used for text files.

It takes the filename, the array, and optional arguments like ‘delimiter’

```
np.savetxt("exoplanet_data.csv", exoplanet_table, delimiter=",") # Use comma as separator
print("\nArray saved to 'exoplanet_data.csv' (CSV format)")
```

```
Array saved to 'exoplanet_data.csv' (CSV format)
```

### 12.7.6 Loading the array from the text file

`np.loadtxt()` is used for loading text files.

It takes the filename and optional arguments like ‘delimiter’.

```
loaded_exoplanet_data = np.loadtxt("exoplanet_data.csv", delimiter=",")
print("\nArray loaded from 'exoplanet_data.csv':")
print(loaded_exoplanet_data)

Verify it's a NumPy array and has the same data
print("Type of loaded data:", type(loaded_exoplanet_data))
print("Loaded data matches original:\n", np.array_equal(exoplanet_table, loaded_exoplanet_data))

You can open 'exoplanet_data.csv' in a text editor or spreadsheet program to see the
```

```
Array loaded from 'exoplanet_data.csv':
[[1.0000e+00 1.0000e+00 3.6525e+02]
 [1.1000e-01 5.3000e-01 6.8700e+02]
 [3.1780e+02 1.1200e+01 4.3310e+03]]
Type of loaded data: <class 'numpy.ndarray'>
Loaded data matches original:
True
```

### 12.7.7 Example 12: Loading Text File with Headers/Comments

Sometimes, astronomical data files have header lines or comments. `np.genfromtxt()` can skip these.

Imagine a file ‘star\_observations.txt’ like this:

```
Observation Data - 2023-10-27
Star Name, Flux, Error
Alpha, 123.45, 1.2
Beta, 56.78, 0.8
Gamma, 234.56, 2.1

Create a dummy file for this example:
file_content = """# Observation Data - 2023-10-27
Star Name, Flux, Error
Alpha, 123.45, 1.2
Beta, 56.78, 0.8
Gamma, 234.56, 2.1
"""

with open("star_observations.txt", "w") as f:
 f.write(file_content)
print("Created dummy file 'star_observations.txt'")
```

Created dummy file 'star\_observations.txt'

```
--- Loading data, skipping the first 2 lines (header/comments) ---
use skiprows parameter
loaded_obs_data = np.genfromtxt(
 "star_observations.txt",
 delimiter=",",
 skip_header=2, # same effect as skiprows
 dtype=None, # auto-detect data types
 encoding='utf-8' # handle string decoding
)
print("\nData loaded from 'star_observations.txt' (skipping 2 rows):")
print(loaded_obs_data)

Note: loadtxt is best for purely numerical data. For mixed data (like star names and numbers),
pandas read_csv (which uses numpy internally) is often more convenient.
```

```
Data loaded from 'star_observations.txt' (skipping 2 rows):
[('Alpha', 123.45, 1.2) ('Beta', 56.78, 0.8) ('Gamma', 234.56, 2.1)]
```

## 12.8 Exercises

Practice makes perfect! Try these exercises to solidify your understanding of NumPy basics:

1. Create a 1D NumPy array named `planet_radii` containing the approximate radii of the inner planets in Earth radii: `[0.38, 0.95, 1.0, 0.53]`.
2. For the `planet_radii` array you created, print its `.shape`, `.size`, and `.ndim`.
3. From the `planet_radii` array, access and print only the radius of Earth (which is the third value in the sequence).
4. From the `planet_radii` array, access and print only the radius of Mars (the last value).
5. Create a new NumPy array named `planet_diameters` by multiplying the `planet_radii` array by 2. Print the `planet_diameters` array.
6. Using the `planet_radii` array, create a boolean array that is `True` for planets with a radius less than 0.6 Earth radii. Use this boolean array to print the radii of only the smaller planets.
7. Calculate and print the average radius from the `planet_radii` array using a NumPy function.
8. Create a 2D NumPy array named `star_distances` with shape `(3, 2)`. Fill it with hypothetical data where each row is a star, the first column

- is distance in parsecs (pc), and the second column is distance in light-years (ly). You can use values like `[[5.0, 16.3], [10.0, 32.6], [2.5, 8.15]]`.
9. From the `star_distances` array, print only the column containing distances in light-years.
  10. From the `star_distances` array, find and print the maximum distance in the parsecs column.
  11. Save the `planet_radii` array you created earlier (`[0.38, 0.95, 1.0, 0.53]`) to a `.npy` file named `inner_planet_radii.npy`. Then, load this file into a new variable and print it to verify.
  12. Save the `star_distances` 2D array you created earlier (`[[5.0, 16.3], [10.0, 32.6], [2.5, 8.15]]`) to a CSV file named `star_distances.csv` using a comma as the delimiter.
  13. Load the `star_distances.csv` file back into a NumPy array and print it.

## 12.9 Summary

Congratulations! You've taken your first steps into using NumPy, a powerful library for numerical operations in Python. You've learned about:

- The importance of NumPy for efficient numerical calculations, especially with arrays.
- The core `ndarray` object.
- Creating 1D and 2D arrays.
- Checking array attributes (`.shape`, `.size`, `.ndim`, `.dtype`).
- Accessing and modifying array elements using indexing and slicing.
- Performing element-wise arithmetic and comparison operations on arrays.
- Using common NumPy functions like `sum()`, `mean()`, `max()`, and `min()`.
- Saving and loading arrays using `np.save()`, `np.load()`, `np.savetxt()`, `np.loadtxt()` and `np.genfromtxt()`.

NumPy is a foundational tool in the scientific Python ecosystem. As you continue your journey in astronomy and programming, you'll find it indispensable for analysing data, running simulations, visualising results and saving and loading arrays. Keep practising these fundamental concepts!

### Additional Resources

Numpy user guide: <https://numpy.org/doc/stable/user/index.html>

Numpy for beginners: [https://numpy.org/doc/stable/user/absolute\\_beginners.html](https://numpy.org/doc/stable/user/absolute_beginners.html)

Numpy arrays: <https://www.datacamp.com/tutorial/python-numpy-tutorial>

Vectors and matrices: <https://www.kaggle.com/code/jhossain/introduction-to-numpy-vectors-and-matrices>



## Chapter 13

# Pandas: Analysing Tabular Data

```
!pip install numpy pandas matplotlib
```

### 13.1 What is Pandas and the DataFrame

Imagine you have data like this:

Name	Type	Distance (light-years)
Andromeda	Spiral	2.537 million
Milky Way	Spiral	0 (our galaxy)
Triangulum	Spiral	3.0 million
Large Magellanic Cloud	Irregular	0.16 million

In Python lists, you might store this as lists within lists:

```
galaxies_list = [
 ['Andromeda', 'Spiral', 2.537],
 ['Milky Way', 'Spiral', 0.0],
 ['Triangulum', 'Spiral', 3.0],
 ['Large Magellanic Cloud', 'Irregular', 0.16]
]
```

*# Or maybe a list of dictionaries:*

```
galaxies_list_of_dicts = [
 {'Name': 'Andromeda', 'Type': 'Spiral', 'Distance_mly': 2.537},
 {'Name': 'Milky Way', 'Type': 'Spiral', 'Distance_mly': 0.0},
 {'Name': 'Triangulum', 'Type': 'Spiral', 'Distance_mly': 3.0},
 {'Name': 'Large Magellanic Cloud', 'Type': 'Irregular', 'Distance_mly': 0.16}
]
```

These work, but performing operations (like finding the average distance of only Spiral galaxies) can become complex with nested loops and conditional logic.

Pandas provides the DataFrame, which is much better suited for this:

```
import pandas as pd # Standard way to import Pandas

Creating a DataFrame from our list of dictionaries:
galaxies_df = pd.DataFrame(galaxies_list_of_dicts)

Printing the DataFrame:
print("Our Galaxies DataFrame:")
galaxies_df
```

Our Galaxies DataFrame:

	Name	Type	Distance_mly
0	Andromeda	Spiral	2.537
1	Milky Way	Spiral	0.000
2	Triangulum	Spiral	3.000
3	Large Magellanic Cloud	Irregular	0.160

Notice how it's structured with column names and row numbers (the index). This makes working with the data much easier!

## 13.2 Creating DataFrames

The most common ways to create a DataFrame are:

1. **From a Dictionary:** Where keys are column names and values are lists or NumPy arrays of data for those columns. This is often the easiest way to start with small datasets you define in your code.
2. **From a List of Dictionaries:** Each dictionary represents a row.
3. **From Files:** Reading data from CSV, Excel, JSON files, etc.
4. **From Online Sources:** Reading directly from URLs.

We'll focus on the dictionary method first, then move to files and URLs.

### 13.2.1 Example 1: Creating a DataFrame from a Dictionary

Data for some bright stars: Name, Spectral Type, Apparent Magnitude

```
star_data_dict = {
 'Name': ['Sirius', 'Canopus', 'Alpha Centauri', 'Arcturus'],
 'Spectral_Type': ['A1V', 'F0', 'G2V', 'K1.5III'],
 'Apparent_Magnitude': [-1.46, -0.72, -0.27, -0.04]
}

Create the DataFrame
bright_stars_df = pd.DataFrame(star_data_dict)

print("DataFrame created from dictionary:")
bright_stars_df
```

DataFrame created from dictionary:

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72
2	Alpha Centauri	G2V	-0.27
3	Arcturus	K1.5III	-0.04

Each key in the dictionary becomes a column name. The lists become the data in those columns. Pandas automatically adds a numerical index (0, 1, 2, 3) for the rows.

## 13.3 Accessing and Selecting Data

Once you have a DataFrame, you need to be able to select specific columns, rows, or even individual values.

- **Selecting Columns:** You can select one or more columns using their names.
- **Selecting Rows:** You can select rows using their index label (`.loc`) or their integer position (`.iloc`).
- **Selecting Specific Cells:** Combine row and column selection.

### 13.3.1 Example 2: Selecting Columns

Using our `bright_stars_df` from Example 1

```
print("Original DataFrame:")
bright_stars_df
```

Original DataFrame:

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72
2	Alpha Centauri	G2V	-0.27
3	Arcturus	K1.5III	-0.04

```
Select a single column:
Result is a Pandas Series (a 1D labeled array)
star_names = bright_stars_df['Name'] # Use column name like a dictionary key
print("\nSelecting the 'Name' column:")
print(star_names)
print("Type of selected column:", type(star_names)) # It's a Series
```

```
Selecting the 'Name' column:
0 Sirius
1 Canopus
2 Alpha Centauri
3 Arcturus
Name: Name, dtype: object
Type of selected column: <class 'pandas.core.series.Series'>
```

```
Select multiple columns:
Pass a list of column names. Result is a DataFrame.
names_and_magnitudes = bright_stars_df[['Name', 'Apparent_Magnitude']] # Use a LIST of
print("\nSelecting 'Name' and 'Apparent_Magnitude' columns:")
print(names_and_magnitudes)
print("Type of selected columns:", type(names_and_magnitudes)) # It's a DataFrame
```

```
Selecting 'Name' and 'Apparent_Magnitude' columns:
 Name Apparent_Magnitude
0 Sirius -1.46
1 Canopus -0.72
2 Alpha Centauri -0.27
3 Arcturus -0.04
Type of selected columns: <class 'pandas.core.frame.DataFrame'>
```

### 13.3.2 Example 3: Selecting Rows with .loc and .iloc

Using our `bright_stars_df`

```
print("Original DataFrame:")
bright_stars_df
```

Original DataFrame:

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72
2	Alpha Centauri	G2V	-0.27
3	Arcturus	K1.5III	-0.04

```
Select a row by its label using .loc
The default labels are the integer index (0, 1, 2, ...)
sirius_row = bright_stars_df.loc[0] # Get the row with index label 0
print("\nSelecting row with label 0 (Sirius) using .loc:")
print(sirius_row)
print("Type of selected row:", type(sirius_row)) # It's a Series
```

Selecting row with label 0 (Sirius) using .loc:

```
Name Sirius
Spectral_Type A1V
Apparent_Magnitude -1.46
Name: 0, dtype: object
Type of selected row: <class 'pandas.core.series.Series'>
```

```
Select a row by its integer position using .iloc
This is just like list/NumPy array indexing
canopus_row = bright_stars_df.iloc[1] # Get the row at integer position 1
print("\nSelecting row at position 1 (Canopus) using .iloc:")
canopus_row
```

Selecting row at position 1 (Canopus) using .iloc:

```
Name Canopus
Spectral_Type F0
Apparent_Magnitude -0.72
Name: 1, dtype: object
```

```
Select multiple rows using slicing with .loc (uses labels)
first_two_stars_loc = bright_stars_df.loc[0:1] # Includes label 1! (Unlike Python list
print("\nSelecting rows with labels 0 to 1 (Sirius and Canopus) using .loc[0:1]:")
print(first_two_stars_loc)
```

```
Selecting rows with labels 0 to 1 (Sirius and Canopus) using .loc[0:1]:
```

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72

```
Select multiple rows using slicing with .iloc (uses positions)
first_two_stars_iloc = bright_stars_df.iloc[0:2] # Excludes position 2 (Like Python li
print("\nSelecting rows at positions 0 and 1 (Sirius and Canopus) using .iloc[0:2]:")
print(first_two_stars_iloc)
```

```
Selecting rows at positions 0 and 1 (Sirius and Canopus) using .iloc[0:2]:
```

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72

### 13.3.3 Example 4: Selecting Specific Cells

```
Using our bright_stars_df
print("Original DataFrame:")
print(bright_stars_df)

Select a specific value (Sirius's magnitude) using .loc[row_label, column_label]
sirius_mag = bright_stars_df.loc[0, 'Apparent_Magnitude']
print("\nSirius's magnitude using .loc[0, 'Apparent_Magnitude']:", sirius_mag)
```

```
Original DataFrame:
```

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72
2	Alpha Centauri	G2V	-0.27
3	Arcturus	K1.5III	-0.04

```
Sirius's magnitude using .loc[0, 'Apparent_Magnitude']: -1.46
```

```
Select a specific value (Canopus's Spectral Type) using .iloc[row_position, column_position]
canopus_spectral_type = bright_stars_df.iloc[1, 1]
print("Canopus's Spectral Type using .iloc[1, 1]:", canopus_spectral_type)

Selecting a slice of rows and a slice of columns
some_data = bright_stars_df.loc[1:2, ['Name', 'Apparent_Magnitude']]
print("\nSelecting subset of rows and columns:")
some_data
```

```
Canopus's Spectral Type using .iloc[1, 1]: F0
```

```
Selecting subset of rows and columns:
```

	Name	Apparent_Magnitude
1	Canopus	-0.72
2	Alpha Centauri	-0.27

## 13.4 Filtering DataFrames

A very common task is to select only the rows that meet certain criteria. Pandas makes this easy using boolean indexing, similar to NumPy. You create a condition that results in a Series of `True/False` values, and then use this Series to select the rows where the condition is `True`.

### 13.4.1 Example 5: Filtering Rows

```
Using our bright_stars_df
print("Original DataFrame:")
print(bright_stars_df)

Condition 1: Find stars that are brighter than magnitude 0.0 (smaller number = brighter)
is_brighter_than_0 = bright_stars_df['Apparent_Magnitude'] < 0.0
print("\nBoolean Series for stars brighter than 0.0:")
print(is_brighter_than_0)
```

```
Original DataFrame:
```

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72
2	Alpha Centauri	G2V	-0.27

```
3 Arcturus K1.5III -0.04
```

```
Boolean Series for stars brighter than 0.0:
```

```
0 True
1 True
2 True
3 True
```

```
Name: Apparent_Magnitude, dtype: bool
```

```
Use the boolean Series to filter the DataFrame
```

```
very_bright_stars_df = bright_stars_df[is_brighter_than_0]
print("\nDataFrame filtered for stars brighter than 0.0:")
print(very_bright_stars_df)
```

```
Condition 2: Find stars that are Spectral Type 'G2V'
```

```
is_spectral_type_G2V = bright_stars_df['Spectral_Type'] == 'G2V'
print("\nBoolean Series for stars with Spectral Type 'G2V':")
print(is_spectral_type_G2V)
```

```
DataFrame filtered for stars brighter than 0.0:
```

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72
2	Alpha Centauri	G2V	-0.27
3	Arcturus	K1.5III	-0.04

```
Boolean Series for stars with Spectral Type 'G2V':
```

```
0 False
1 False
2 True
3 False
```

```
Name: Spectral_Type, dtype: bool
```

```
Filter using the condition directly (common shorthand)
```

```
G2V_stars_df = bright_stars_df[bright_stars_df['Spectral_Type'] == 'G2V']
print("\nDataFrame filtered for Spectral Type 'G2V':")
print(G2V_stars_df)
```

```
DataFrame filtered for Spectral Type 'G2V':
```

	Name	Spectral_Type	Apparent_Magnitude
2	Alpha Centauri	G2V	-0.27



You can combine multiple conditions using logical operators:

```
& (AND)
| (OR)
~ (NOT)
```

**IMPORTANT:** Each condition must be enclosed in parentheses ()

### 13.4.2 Example 6: Filtering with Multiple Conditions

```
Find stars that are brighter than 0.0 AND have Spectral Type 'A1V'
bright_and_A1V = (bright_stars_df['Apparent_Magnitude'] < 0.0) & (bright_stars_df['Spectral_Type'] == 'A1V')
filtered_df = bright_stars_df[bright_and_A1V]
print("Stars brighter than 0.0 AND Spectral Type 'A1V':")
filtered_df
```

Stars brighter than 0.0 AND Spectral Type 'A1V':

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46

```
Find stars that are Spectral Type 'F0' OR Spectral Type 'K1.5III'
F0_or_K_stars = (bright_stars_df['Spectral_Type'] == 'F0') | (bright_stars_df['Spectral_Type'] == 'K1.5III')
filtered_df_or = bright_stars_df[F0_or_K_stars]
print("\nStars with Spectral Type 'F0' OR 'K1.5III':")
filtered_df_or
```

Stars with Spectral Type 'F0' OR 'K1.5III':

	Name	Spectral_Type	Apparent_Magnitude
1	Canopus	F0	-0.72
3	Arcturus	K1.5III	-0.04

## 13.5 Basic Operations and Adding New Columns

You can perform mathematical operations on entire columns (Series) directly. You can also easily add new columns to your DataFrame, often based on calculations from existing columns.

### 13.5.1 Example 7: Operations and Adding Columns

```
Using our bright_stars_df
print("Original DataFrame:")
bright_stars_df
```

Original DataFrame:

	Name	Spectral_Type	Apparent_Magnitude
0	Sirius	A1V	-1.46
1	Canopus	F0	-0.72
2	Alpha Centauri	G2V	-0.27
3	Arcturus	K1.5III	-0.04

Add a new column for Absolute Magnitude (simplified calculation)

Absolute Magnitude (M) = Apparent Magnitude (m) - 5 \* log10(distance in parsecs / 10)

Let's assume some hypothetical distances in parsecs for these stars

```
distances_pc = pd.Series([1.34, 99.0, 1.34, 11.0]) # Create a Series with distances
bright_stars_df['Distance_pc'] = distances_pc # Add this Series as a new column
```

```
Now calculate Absolute Magnitude
Need numpy for log10
import numpy as np
```

```
bright_stars_df['Absolute_Magnitude'] = bright_stars_df['Apparent_Magnitude'] - 5 * np
print("\nDataFrame after adding 'Distance_pc' and 'Absolute_Magnitude' columns:")
bright_stars_df
```

DataFrame after adding 'Distance\_pc' and 'Absolute\_Magnitude' columns:

	Name	Spectral_Type	Apparent_Magnitude	Distance_pc	\
0	Sirius	A1V	-1.46	1.34	
1	Canopus	F0	-0.72	99.00	
2	Alpha Centauri	G2V	-0.27	1.34	
3	Arcturus	K1.5III	-0.04	11.00	
	Absolute_Magnitude				
0			2.904476		

```
1 -5.698176
2 4.094476
3 -0.246963
```

```
Calculate the average Apparent Magnitude
average_apparent_mag = bright_stars_df['Apparent_Magnitude'].mean() # Use .mean() method on the Series
print(f"\nAverage Apparent Magnitude: {average_apparent_mag:.2f}")

Calculate the maximum Absolute Magnitude
max_absolute_mag = bright_stars_df['Absolute_Magnitude'].max()
print(f"Maximum Absolute Magnitude: {max_absolute_mag:.2f}")
```

```
Average Apparent Magnitude: -0.62
Maximum Absolute Magnitude: 4.09
```

## 13.6 Reading Data from Files (CSV)

Real astronomical data often comes in files, especially CSV (Comma-Separated Values) files. Pandas provides the `pd.read_csv()` function, which is extremely useful for loading these files directly into a DataFrame.

### 13.6.1 Example 8: Reading from a Local CSV File

First, let's create a simple dummy CSV file named `exoplanet_data.csv` with some hypothetical data. You can create this file manually or run this code:

```
Example 8: Reading from a Local CSV File

First, let's create a simple dummy CSV file named 'exoplanet_data.csv'
with some hypothetical data. You can create this file manually or run this code:
csv_content = """Name,Radius_Earth,OrbitalPeriod_days,Habitable
Kepler-186f,1.11,129.9,True
Kepler-1649b,1.06,8.68,False
TRAPPIST-1e,0.91,6.10,True
51 Pegasi b,1.50,4.23,False
"""

with open("exoplanet_data.csv", "w") as f:
 f.write(csv_content)

print("Created dummy file 'exoplanet_data.csv'")
```

```
Created dummy file 'exoplanet_data.csv'
```

```
Now, read the CSV file into a DataFrame:
exoplanets_df = pd.read_csv("exoplanet_data.csv")

print("\nDataFrame read from 'exoplanet_data.csv':")
print(exoplanets_df)

Check the data types Pandas inferred
print("\nInfo about the DataFrame:")
exoplanets_df.info()
```

```
DataFrame read from 'exoplanet_data.csv':
```

	Name	Radius_Earth	OrbitalPeriod_days	Habitable
0	Kepler-186f	1.11	129.90	True
1	Kepler-1649b	1.06	8.68	False
2	TRAPPIST-1e	0.91	6.10	True
3	51 Pegasi b	1.50	4.23	False

```

Info about the DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
Column Non-Null Count Dtype
--- -
0 Name 4 non-null object
1 Radius_Earth 4 non-null float64
2 OrbitalPeriod_days 4 non-null float64
3 Habitable 4 non-null bool
dtypes: bool(1), float64(2), object(1)
memory usage: 232.0+ bytes

```

## 13.7 Reading Data from Online Sources (URLs)

One of the most powerful features of `pd.read_csv()` is its ability to read directly from a URL! If a CSV file is available online, you can pass its web address to `read_csv()`, and Pandas will download and load it for you. This is incredibly useful for accessing publicly available astronomical datasets.

### 13.7.1 Example 9: Reading from an Online CSV File (Exoplanet Archive Simplified Data)

This is a URL pointing to a simplified CSV file often used in tutorials, potentially representing a small subset of data from the NASA Exoplanet Archive or similar. Note: Real URLs might change, and data providers might have APIs instead of static files. This is a demonstration of the capability.

```
exoplanet_url = "https://raw.githubusercontent.com/astroryan97/Exploring-Exoplanet-Data/main/oper
try:
 # Read the data directly from the URL
 online_exoplanets_df = pd.read_csv(exoplanet_url)

 print("DataFrame read from URL:")
 print(online_exoplanets_df.head()) # Print only the first few rows (head)
 print(f"\nTotal rows loaded: {len(online_exoplanets_df)}")

 # Let's do a quick check - how many rows have 'KOI' in the 'NAME' column?
 koi_planets = online_exoplanets_df[online_exoplanets_df['name'].str.contains('KOI', na=False)]
 print(f"\nNumber of KOI planets found: {len(koi_planets)}")

except Exception as e:
 print(f"Could not load data from the URL. Make sure the URL is correct and accessible.")
 print(f"Error details: {e}")
```

```
DataFrame read from URL:
 name binaryflag mass radius period semimajoraxis \
0 KOI-1843.03 0 0.0014 0.054 0.176891 0.0048
1 Kepler-974 b 0 NaN 0.140 4.194497 0.0390
2 KOI-1843.02 0 NaN 0.071 6.356006 0.0520
3 Kepler-9 b 0 0.2500 0.840 19.224180 0.1430
4 Kepler-9 c 0 0.1700 0.820 39.031060 0.2290

 eccentricity periastron longitude ascendingnode ... lastupdate \
0 NaN NaN NaN NaN ... 13/07/15
1 NaN NaN NaN NaN ... 17/11/28
2 NaN NaN NaN NaN ... NaN
3 0.0626 NaN NaN NaN ... 15/12/03
4 0.0684 NaN NaN NaN ... 15/12/03

 system_rightascension system_declination system_distance hoststar_mass \
0 19 00 03.1450 +40 13 14.7985 134.833 0.52
1 19 00 03.1450 +40 13 14.7985 134.833 0.52
```

```

2 19 00 03.1450 +40 13 14.7985 134.833 0.52
3 19 02 17.7544 +38 24 03.1772 650.000 1.07
4 19 02 17.7544 +38 24 03.1772 650.000 1.07

 hoststar_radius hoststar_metallicity hoststar_temperature hoststar_age \
0 0.50 0.07 3687.0 NaN
1 0.50 0.07 3687.0 NaN
2 0.50 0.07 3687.0 NaN
3 1.02 0.12 5777.0 NaN
4 1.02 0.12 5777.0 NaN

 list
0 Controversial
1 Confirmed planets
2 Controversial
3 Confirmed planets
4 Confirmed planets

[5 rows x 25 columns]

Total rows loaded: 4493

Number of KOI planets found: 72

```

## 13.8 Handling Missing Data

Real-world data often has gaps or missing values. Pandas represents missing numerical values as `NaN` (Not a Number). You'll need to be able to detect and handle these missing values, either by removing rows/columns with missing data or filling them in.

### 13.8.1 Example 10: Handling Missing Data

```

Let's create a DataFrame with some missing values
data_with_missing = {
 'Object': ['Star A', 'Star B', 'Galaxy C', 'Star D'],
 'Magnitude': [2.1, 3.5, np.nan, 4.8], # np.nan represents a missing value
 'Redshift': [0.001, 0.002, 0.015, np.nan]
}
missing_df = pd.DataFrame(data_with_missing)

```

```
print("DataFrame with missing data:")
print(missing_df)
```

```
DataFrame with missing data:
 Object Magnitude Redshift
0 Star A 2.1 0.001
1 Star B 3.5 0.002
2 Galaxy C NaN 0.015
3 Star D 4.8 NaN
```

```
Check which values are missing
print("\nChecking for missing values (isnull()):")
print(missing_df.isnull()) # Returns True where data is missing

Get a summary of missing values per column
print("\nMissing values per column:")
print(missing_df.isnull().sum())
```

```
Checking for missing values (isnull()):
 Object Magnitude Redshift
0 False False False
1 False False False
2 False True False
3 False False True

Missing values per column:
Object 0
Magnitude 1
Redshift 1
dtype: int64
```

```
Option 1: Drop rows with *any* missing values
cleaned_df_dropped_rows = missing_df.dropna()
print("\nDataFrame after dropping rows with missing data:")
print(cleaned_df_dropped_rows)
```

```
DataFrame after dropping rows with missing data:
 Object Magnitude Redshift
0 Star A 2.1 0.001
1 Star B 3.5 0.002
```

```
Option 2: Drop columns with *any* missing values
cleaned_df_dropped_cols = missing_df.dropna(axis=1) # axis=1 means columns
print("\nDataFrame after dropping columns with missing data:")
print(cleaned_df_dropped_cols)
```

DataFrame after dropping columns with missing data:

	Object
0	Star A
1	Star B
2	Galaxy C
3	Star D

```
Option 3: Fill missing values (e.g., with a placeholder or mean)
filled_df = missing_df.fillna(value=0) # Fill NaN with 0
print("\nDataFrame after filling missing values with 0:")
print(filled_df)
```

DataFrame after filling missing values with 0:

	Object	Magnitude	Redshift
0	Star A	2.1	0.001
1	Star B	3.5	0.002
2	Galaxy C	0.0	0.015
3	Star D	4.8	0.000

```
Fill missing magnitude with the mean magnitude (excluding NaN)
mean() ignores NaN by default
mean_magnitude = missing_df['Magnitude'].mean()
filled_df_mean_mag = missing_df.fillna({'Magnitude': mean_magnitude})
print("\nDataFrame after filling missing Magnitude with mean:")
print(filled_df_mean_mag)
```

DataFrame after filling missing Magnitude with mean:

	Object	Magnitude	Redshift
0	Star A	2.100000	0.001
1	Star B	3.500000	0.002
2	Galaxy C	3.466667	0.015
3	Star D	4.800000	NaN

## 13.9 Exercises

Time to practice your Pandas skills with astronomical data!



1. **Create a Small Catalog:** Create a DataFrame for three hypothetical stars with columns 'Name', 'Mass\_Solar', and 'Radius\_Solar'.
2. **Select Mass Column:** From your star DataFrame, select and print only the 'Mass\_Solar' column.
3. **Select First Star:** Select and print the data for the first star using `.loc[0]`.
4. **Select Name and Radius for Second Star:** Select and print *only* the name and radius for the second star using a combination of row and column selection.
5. **Filter by Mass:** From your star DataFrame, filter and print only the stars with a mass greater than 1.0 Solar Mass.
6. **Add Density Column (Bonus):** Add a new column named 'Density\_Solar' (relative to the Sun). Density is Mass / Volume, and Volume is proportional to Radius<sup>3</sup>. So, `Density_Solar = Mass_Solar / (Radius_Solar ** 3)`. Calculate this for each star and add it as a new column. Print the updated DataFrame.
7. **Read Your Exoplanet CSV:** Load the 'exoplanet\_data.csv' file you created in Example 8 back into a DataFrame.
8. **Filter Habitable Planets:** From the 'exoplanet\_data.csv' DataFrame, filter and print only the rows where the 'Habitable' column is `True`.
9. **Calculate Average Radius:** Calculate and print the average 'Radius\_Earth' for all exoplanets in the DataFrame loaded from 'exoplanet\_data.csv'.
10. **Read Online Data:** (If Example 9 worked) Load the online exoplanet data again from the provided URL. Calculate the average 'RADIUS' for all planets in that online dataset.

## 13.10 Summary

You've now gained fundamental skills in using the Pandas library for handling and analysing tabular data! You've learned:

- The DataFrame is the core structure for labelled, mixed-type data.
- How to create DataFrames.
- How to select columns, rows, and cells using labels (`.loc`) and positions (`.iloc`).
- How to filter DataFrames based on single or multiple conditions.
- How to perform operations and add new columns.
- Crucially, how to read data into DataFrames from both local CSV files and online URLs.
- How to identify and perform basic handling of missing data.

Pandas is an essential tool in the data science toolkit and is widely used in astronomy. With these basics, you are well-equipped to start exploring, cleaning,

and analysing real datasets. Keep practising!

### **Additional Resources**

Here are some resources for understanding **Pandas**

Getting started with Pandas: [https://pandas.pydata.org/docs/getting\\_started/index.html#getting-started](https://pandas.pydata.org/docs/getting_started/index.html#getting-started)

Pandas user guide: [https://pandas.pydata.org/docs/user\\_guide/index.html#user-guide](https://pandas.pydata.org/docs/user_guide/index.html#user-guide)

Groupby with Pandas: <https://realpython.com/pandas-groupby/>

Handling big data with pandas: <https://www.scaler.com/topics/pandas/handling-large-datasets-in-pandas/>

How to handle parquet data files: <https://www.datacamp.com/tutorial/apache-parquet>

Introduction to Polars: <https://www.datacamp.com/blog/an-introduction-to-polars-python-s-tool-for-large-scale-data-analysis>

## Chapter 14

# Statistical Analysis

```
!pip install numpy pandas matplotlib scipy statsmodels
```

### 14.0.1 1. Setup: Import Libraries and Load Data

```
1. Importing Libraries
import pandas as pd # 'pd' is short for pandas, the library that gives us DataFrames (our data
import numpy as np # 'np' is short for numpy, which helps with math and arrays (lists of numbers
from scipy import stats # For advanced statistics.
import statsmodels.api as sm # For even more statistics and models.

2. Load Gaia Data
df = pd.read_csv('data/gaia-hrd.csv')

If you are using Google Colab, uncomment the next line:
df = pd.read_csv("https://raw.githubusercontent.com/Danselem/brics_astro/main/Week2/data/gaia-hrd.csv")

Show the first 5 rows of the DataFrame.
(Imagine df as a big table of stars: each row is a star, each column is a property like brightness)
df.head()
```

	source_id	ra	dec	pmra	pmdec	\
0	5542245319386060288	128.082874	-36.997840	-31.597539	55.733054	
1	5542245319397657472	128.084596	-36.998033	-32.540762	58.041920	
2	5542415846778481920	128.564266	-35.815318	-3.461656	-37.194745	
3	5542415846778484608	128.561657	-35.815391	-3.528435	-38.697425	
4	5542770336201711488	127.943541	-34.926597	-133.068824	-18.808747	

	parallax	parallax_over_error	phot_g_mean_mag	phot_bp_mean_mag	\
0	10.911685	71.94558	18.333282	18.427774	
1	10.848546	602.55927	14.110725	15.709354	
2	14.024835	1255.10290	13.113724	14.119819	
3	14.022714	359.34033	16.154750	17.736862	
4	14.156612	452.39978	15.625710	17.195644	

	phot_rp_mean_mag	bp_rp	mg
0	17.885862	0.541912	13.522742
1	12.885056	2.824299	9.287583
2	12.119724	2.000094	8.848212
3	14.934289	2.802573	11.888911
4	14.410464	2.785180	11.380506

### 14.0.2 2. Data Cleaning: Remove Problematic Values

Let's clean the data by removing stars with: - Negative or zero parallax (distance can't be negative) - Missing (NaN) values

```
Remove stars with parallax 0]

Remove any stars with missing values (NaN means "empty box" or missing data).
df = df.dropna()

print(f"Rows after cleaning: {len(df)}")
(Now our DataFrame only has stars where we know all the important information.)
```

Rows after cleaning: 12366

### 14.0.3 3. Data Filtering: Focus on Bright, Nearby Stars

Suppose we want only stars that are brighter than 5 (G magnitude < 5) and parallax greater than 2:

```
filtered_df = df[(df['phot_g_mean_mag'] < 5) & (df['parallax'] > 2)]
print(filtered_df.shape)
filtered_df.head()

(We make a new DataFrame called filtered_df. It only keeps stars that are bright (G magnitude < 5) and close to us (parallax > 2). This is called "filtering" the data.)
```

(41, 13)

	source_id	ra	dec	pmra	pmdec	\
249	4281199284413726208	284.055154	4.203745	46.474018	31.312731	
250	4281199318773462016	284.061227	4.202254	50.043812	28.363343	
938	5338833263117260672	163.374239	-58.852998	78.168773	38.800125	
1068	5877059048308526720	229.377706	-58.801812	-97.182091	-136.055432	
1145	2027458764012840576	300.275493	27.753591	57.816973	3.994316	
	parallax	parallax_over_error	phot_g_mean_mag	phot_bp_mean_mag		\
249	24.269567	138.41821	4.568889	4.661161		
250	24.531043	226.54736	4.925873	5.039706		
938	33.522466	358.92407	3.533434	4.077420		
1068	33.820454	134.45961	4.060981	4.088622		
1145	13.442694	128.05238	4.627144	4.705686		
	phot_rp_mean_mag	bp_rp	mg	distance_pc		
249	4.404667	0.256494	1.494199	41.203866		
250	4.730397	0.309309	1.874453	40.764676		
938	2.888392	1.189028	1.160114	29.830741		
1068	3.951576	0.137046	1.706878	29.567906		
1145	4.435495	0.270190	0.269575	74.389853		

#### 14.0.4 4. Calculate New Columns

Let's add a **distance** column (in parsecs):

```
parallax is how much a star seems to move, measured in tiny angles.
(We can figure out how far away a star is by dividing 1000 by its parallax number.
This gives us distance in parsecs, which is a way to measure how far stars are.)

df['distance_pc'] = 1000 / df['parallax']

We add a new column to our DataFrame called 'distance_pc'.
This uses a formula to turn parallax into distance in parsecs (a unit for star distances).
df.head()
```

	source_id	ra	dec	pmra	pmdec	\
0	5542245319386060288	128.082874	-36.997840	-31.597539	55.733054	
1	5542245319397657472	128.084596	-36.998033	-32.540762	58.041920	
2	5542415846778481920	128.564266	-35.815318	-3.461656	-37.194745	
3	5542415846778484608	128.561657	-35.815391	-3.528435	-38.697425	
4	5542770336201711488	127.943541	-34.926597	-133.068824	-18.808747	

	parallax	parallax_over_error	phot_g_mean_mag	phot_bp_mean_mag	\
0	10.911685	71.94558	18.333282	18.427774	
1	10.848546	602.55927	14.110725	15.709354	
2	14.024835	1255.10290	13.113724	14.119819	
3	14.022714	359.34033	16.154750	17.736862	
4	14.156612	452.39978	15.625710	17.195644	

	phot_rp_mean_mag	bp_rp	mg	distance_pc
0	17.885862	0.541912	13.522742	91.644870
1	12.885056	2.824299	9.287583	92.178247
2	12.119724	2.000094	8.848212	71.302089
3	14.934289	2.802573	11.888911	71.312873
4	14.410464	2.785180	11.380506	70.638369

## 14.0.5 5. Basic Statistics

### 14.0.5.1 A. Summary Statistics

```
print("Mean G Magnitude:", df['phot_g_mean_mag'].mean())
Mean is the average value in the column 'phot_g_mean_mag' - how bright, on average,

print("Median Parallax:", df['parallax'].median())
Median is the middle value if we line up all the parallax numbers from smallest to b

print("Standard Deviation of pmra:", df['pmra'].std())
Standard deviation tells us how much the pmra values are spread out from the average
```

```
Mean G Magnitude: 13.906374901811418
Median Parallax: 12.668849430955
Standard Deviation of pmra: 118.70508046746373
```

### 14.0.5.2 B. Correlation

```
df[['parallax', 'phot_g_mean_mag', 'pmra', 'pmdec']].corr()
Correlation tells us if two things move together.
For example, if stars are closer (higher parallax), are they also brighter (lower ma,
```

	parallax	phot_g_mean_mag	pmra	pmdec
parallax	1.000000	-0.158439	-0.111996	-0.102405
phot_g_mean_mag	-0.158439	1.000000	0.001472	0.032965
pmra	-0.111996	0.001472	1.000000	-0.064362
pmdec	-0.102405	0.032965	-0.064362	1.000000

## 14.0.6 6. Distributions: Plotting with Pandas/Matplotlib

```
import matplotlib.pyplot as plt

We can create plots to show how our data is spread out.
For example, a histogram of star distances or brightness.

Histogram of G Magnitude
df['phot_g_mean_mag'].hist(bins=20)
plt.xlabel('G Magnitude')
plt.ylabel('Number of Stars')
plt.title('Distribution of G Magnitude')
plt.show()
```

<Figure size 640x480 with 1 Axes>

## 14.0.7 7. Hypothesis Testing with SciPy

**Example:** Do bright stars ( $G < 15$ ) have different average proper motion [`pmra`] than fainter stars ( $G \geq 15$ )?

```
bright = df[df['phot_g_mean_mag'] < 15]['pmra']
faint = df[df['phot_g_mean_mag'] >= 15]['pmra']

t_stat, p_value = stats.ttest_ind(bright, faint)
print(f"T-statistic: {t_stat:.2f}, p-value: {p_value:.4f}")

if p_value < 0.05:
 print("There is a statistically significant difference!")
else:
 print("No significant difference found.")
```

T-statistic: 0.26, p-value: 0.7983  
No significant difference found.

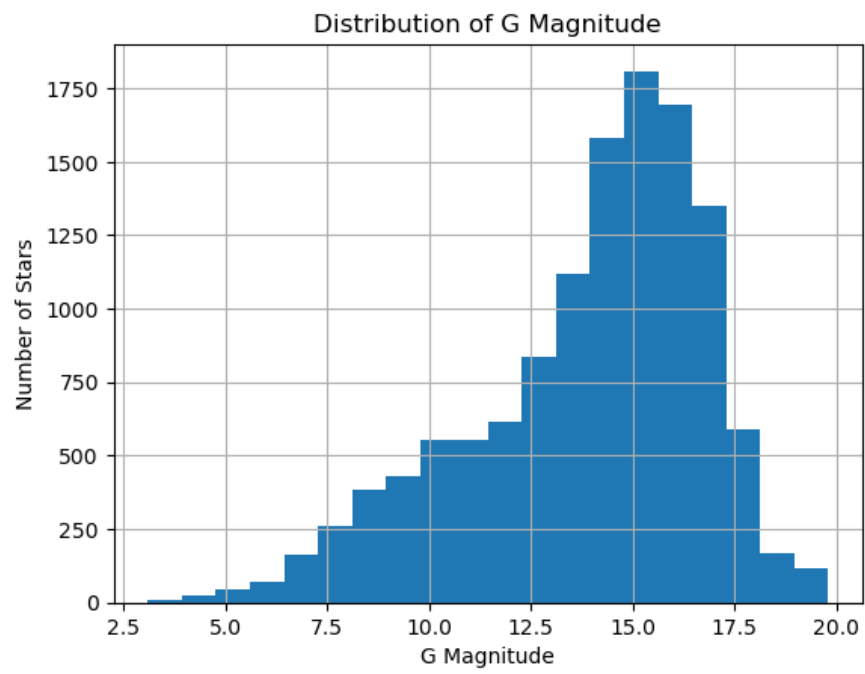


Figure 14.1: output



### 14.0.8 Statistical Test: t-test

- `stats.ttest_ind(bright, faint)` runs an **independent two-sample t-test**.
  - This test checks if the **average pmra** (proper motion in RA) is different between bright and faint stars.
  - **T-statistic**: Measures how different the two group means are, relative to the variation in the data.
  - **p-value**: Tells us the probability of seeing such a difference just by chance.
- 

### 14.0.9 Interpreting the Result

- If `p_value >= 0.05`, we say **no significant difference** is found.

## 14.1 Technical Terms Used

- **Filtering**: Selecting only the rows that meet certain criteria (e.g., bright or faint stars).
- **t-test**: A statistical test to compare the means of two groups.
- **T-statistic**: A number that tells us how different the means are, taking into account the spread of the data.
- **p-value**: The probability that the observed difference happened by random chance (lower is more significant).
- **Statistically significant**: Means the difference is probably real, not just random.

### 14.1.1 8. Simple Regression with Statsmodels

**Example:** Predict G magnitude using distance.

Performing a **simple linear regression** using the statsmodels library to see how the distance to a star (in parsecs) relates to its brightness (G-band magnitude).

**14.1.1.1** `x = sm.add_constant(df['distance_pc'])`

- **What is `df['distance_pc']`?**
  - This is a column in your DataFrame that gives the distance to each star in parsecs.

- What does `sm.add_constant()` do?

- It adds a column of 1s to `X`. This is called the **intercept** or **constant term** in regression. It allows the line to be shifted up or down on the graph, instead of always passing through the origin (0,0).

14.1.1.2 `y = df['phot_g_mean_mag']`

- What is `y`?

- This is the column with the G-band mean magnitude (brightness) for each star. In astronomy, a lower magnitude means a brighter star.

14.1.1.3 `model = sm.OLS(y, X).fit()`

- What is `sm.OLS`?

- OLS stands for **Ordinary Least Squares**, a method for fitting a straight line to data.
- Here, you are telling statsmodels to fit a regression model where `y` (brightness) is predicted by `X` (distance).

- What does `.fit()` do?

- It calculates the best-fitting line for your data.

14.1.1.4 `print(model.summary())`

- What does this do?

- It prints a detailed summary of the regression results, including:
  - \* The coefficients (slope and intercept)
  - \* R-squared (how well the line fits the data)
  - \* p-values (whether the relationship is statistically significant)
  - \* Standard errors, confidence intervals, and more

```
X = sm.add_constant(df['distance_pc']) # Add intercept
y = df['phot_g_mean_mag']

model = sm.OLS(y, X).fit()
print(model.summary())
```

OLS Regression Results			
=====			
Dep. Variable:	phot_g_mean_mag	R-squared:	0.020
Model:	OLS	Adj. R-squared:	0.020

Method:	Least Squares		F-statistic:	58.28		
Date:	Fri, 16 May 2025		Prob (F-statistic):	3.08e-14		
Time:	12:58:52		Log-Likelihood:	-5720.3		
No. Observations:	2861		AIC:	1.144e+04		
Df Residuals:	2859		BIC:	1.146e+04		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]
-----						
const	15.8898	0.064	247.180	0.000	15.764	16.016
distance_pc	-0.0005	7.02e-05	-7.634	0.000	-0.001	-0.000
=====						
Omnibus:	535.338		Durbin-Watson:	2.063		
Prob(Omnibus):	0.000		Jarque-Bera (JB):	1131.336		
Skew:	-1.088		Prob(JB):	2.16e-246		
Kurtosis:	5.181		Cond. No.	1.76e+03		
=====						
Notes:						
[1] Standard Errors assume that the covariance matrix of the errors is correctly						
[2] The condition number is large, 1.76e+03. This might indicate that there are						
strong multicollinearity or other numerical problems.						

## 14.2 What does the output mean?

- **coef (coefficients):**
  - The intercept (constant) and the slope for distance.
  - The slope tells you how much the G magnitude (brightness) changes for each parsec farther away.
- **R-squared:**
  - A number between 0 and 1 that says how much of the change in brightness can be explained by distance. Closer to 1 means a better fit.
- **p-value:**
  - If it's less than 0.05, the relationship is considered statistically significant.
- **std err:**
  - Standard error - tells you how much the estimated coefficient might vary.

## 14.3 Technical Terms

- **Regression:** A way to model the relationship between two or more variables.
- **Intercept:** The value where the regression line crosses the y-axis (when distance is zero).
- **Slope:** How much the y-value (brightness) changes for each unit increase in x (distance).
- **OLS (Ordinary Least Squares):** The most common method for fitting a straight line to data.
- **p-value:** Probability that the observed relationship is due to random chance.
- **R-squared:** Proportion of variance in the dependent variable explained by the independent variable.

### 14.3.1 Homework Questions

1. **Data Cleaning:** Remove all stars with a parallax error greater than 20%. (Hint: If you don't have a 'parallax\_error' column, assume it is 0.1 for all stars.)
2. **Filtering:** Make a new DataFrame of stars with `distance_pc` less than 50 parsecs and `phot_g_mean_mag` less than 14.
3. **Statistics:** What is the mean and median proper motion in declination (`pmdec`) for stars with G magnitude brighter than 13?
4. **Hypothesis Testing:** Use a t-test to check if the mean parallax of stars with `pmra > 0` is different from those with `pmra < 0`.
5. **Regression:** Use `statsmodels` to predict `phot_g_mean_mag` using both `distance_pc` and `parallax`. (Which variable is a stronger predictor?)

## 14.4 Tips

- Use `df.describe()` for quick stats.
- Use `df[(condition)]` for filtering.
- For more plotting, try `df['column'].plot.hist()`.

### Additional Resources

Exploratory Data Analysis with Pandas [https://mlcourse.ai/book/topic01/topic01\\_pandas\\_data\\_analysis.html](https://mlcourse.ai/book/topic01/topic01_pandas_data_analysis.html)

## Chapter 15

# Descriptive Statistics

### 15.1 Definitions of descriptive statistics

**Arithmetic mean (expectation value):** a location parameter that quantifies where most of the probability density is located.

$$\mu = E(x) = \int_{-\infty}^{\infty} xh(x) dx$$

**Variance:** a scale parameter that quantifies how wide a distribution is, calculated by the average of the squared differences from the mean.

$$V = \int_{-\infty}^{\infty} (x - \mu)^2 h(x) dx$$

**Standard deviation:** the square root of variance; it's a scale sometimes preferred over variance since it's units are the same as that of the mean rather than units<sup>2</sup>.

$$\sigma = \sqrt{V}$$

**Skewness:** a shape parameter that quantifies how asymmetric a distribution is. Symmetric distributions, such as uniform, Gaussian, and Laplace distributions, have no skewness.

$$\Sigma = \int_{-\infty}^{\infty} \left( \frac{x - \mu}{\sigma} \right)^3 h(x) dx$$

**Kurtosis:** a shape parameter that quantifies the “tailedness” (the weight of a distribution’s tails relative to the center) of a probability distribution. Highly peaked (“leptokurtic”) distributions have positive kurtosis, and flat-topped (“platykurtic”) distributions have negative kurtosis. The Gaussian distribution is “mesokurtic”, meaning it has zero kurtosis.

$$K = \int_{-\infty}^{\infty} \left( \frac{x - \mu}{\sigma} \right)^4 h(x) dx - 3$$

*Note: the definition that the textbook uses is sometimes called “excess kurtosis” in other sources.*

**Absolute deviation about d:** the average distance between each data point and the mean. When taken about the median, the absolute deviation is minimized.

$$\delta = \int_{-\infty}^{\infty} |x - d| h(x) dx$$

**Mode:** a location parameter that corresponds to the most probable values, which occur where the derivative is zero (i.e. peaks). However, this isn’t the most reliable location parameter; it can be misleading distributions with multiple sharp but narrow peaks. It can also be hard to estimate from samples.

$$\left( \frac{dh(x)}{dx} \right)_{x_m} = 0$$

**Percent quantiles:** segments dividing the range of a probability distribution into continuous intervals with equal probabilities. The most often used quantiles are the median,  $q_{50}$ , and the first and third quartile,  $q_{25}$  and  $q_{75}$ . The difference between the third and the first quartiles is called the interquartile range.

$$\frac{p}{100} = \int_{-\infty}^{q_p} h(x) dx$$

---

*Note: The higher a moment is, the harder it is to estimate it with small samples. Furthermore, there is more sensitivity to outliers (less robustness); for these reasons, higher-order moments, such as skewness and kurtosis, should be used with caution when samples are small.*

*Additionally, some distributions, such as the Cauchy distribution, do not have finite variance. So, when the distribution’s variance is infinite (i.e., the tails of  $h(x)$  do not decrease faster than  $x^{-3}$  for large  $|x|$ ), the skewness and kurtosis will diverge as well.*

In the example below, we'll show distributions with different skewness (top panel) and kurtosis (bottom panel). In the top panel, we'll plot a Gaussian, a modified Gaussian, and a log-normal distribution with  $\sigma = 1.2$ . The modified Gaussian is a normal distribution multiplied by a Gram-Charlier series  $h(x) = N(x, 1) \sum_{k=0}^{\infty} a_k H_k(z)$  with  $a_0 = 2$ ,  $a_1 = 1$ , and  $a_2 = 0.5$ . For the kurtosis panel, we'll plot a uniform, Laplace, cosine, and Gaussian distribution.

We can find the values for skewness and kurtosis for each distribution by importing the desired distribution from `scipy.stats`. For example, we can call `from scipy.stats import uniform` and then call `uniform.stats(moments='sk')` to get the skewness and kurtosis for a uniform distribution. For the modified Gaussian however, we will hard-code  $\Sigma = -0.36$ .

```
from scipy.stats import uniform, norm, laplace, cosine, lognorm

uni = float(uniform.stats(moments = 'k'))
lap = int(laplace.stats(moments = 'k'))
cos = float(cosine.stats(moments = 'k'))
log = float(lognorm.stats(1.2, moments = 's'))

gauss = norm.stats(moments = 'sk')
skew_gauss = int(gauss[0])
kurt_gauss = int(gauss[1])
```

Next, we'll plot the distributions.

```
import numpy as np
from scipy import stats
from matplotlib import pyplot as plt

fig = plt.figure(figsize=(7.5, 10))
fig.subplots_adjust(right=0.95, hspace=0.05, bottom=0.07, top=0.95)

First show distributions with different skew
ax = fig.add_subplot(211)
x = np.linspace(-8, 8, 1000)
N = stats.norm(0, 1)

l1, = ax.plot(x, N.pdf(x), '-k',
 label='Gaussian, $\Sigma={}$ '.format(skew_gauss))
l2, = ax.plot(x, 0.5 * N.pdf(x) * (2 + x + 0.5 * (x * x - 1)),
 '--k', label='mod. Gauss, $\Sigma=-0.36$ ')
l3, = ax.plot(x[499:], stats.lognorm(1.2).pdf(x[499:]), '-.k',
```

```

 label='log normal, $\Sigma={}$ '.format(round(log,1))

ax.set_xlim(-5, 5)
ax.set_ylim(0, 0.7001)
ax.set_ylabel('$p(x)$', fontsize = 12)
ax.xaxis.set_major_formatter(plt.NullFormatter())

trick to show multiple legends
leg1 = ax.legend([l1], [l1.get_label()], loc=1, fontsize = 12)
leg2 = ax.legend([l2, l3], (l2.get_label(), l3.get_label()), loc=2, fontsize = 12)
ax.add_artist(leg1)
ax.set_title('Skew Σ and Kurtosis K ', fontsize = 16)

next show distributions with different kurtosis
ax = fig.add_subplot(212)
x = np.linspace(-5, 5, 1000)
l1, = ax.plot(x, stats.laplace(0, 1).pdf(x), '--k',
 label='Laplace, $K={}$ '.format(lap))
l2, = ax.plot(x, stats.norm(0, 1).pdf(x), '-k',
 label='Gaussian $K={}$ '.format(kurt_gauss))
l3, = ax.plot(x, stats.cosine(0, 1).pdf(x), '-.k',
 label='Cosine, $K=-0.59$ '.format(round(cos,2)))
l4, = ax.plot(x, stats.uniform(-2, 4).pdf(x), ':k',
 label='Uniform, $K=-1.2$ '.format(uni))

ax.set_xlim(-5, 5)
ax.set_ylim(0, 0.55)
ax.set_xlabel('x', fontsize = 12)
ax.set_ylabel('$p(x)$', fontsize = 12)

trick to show multiple legends
leg1 = ax.legend((l1, l2), (l1.get_label(), l2.get_label()), loc=2, fontsize = 12)
leg2 = ax.legend((l3, l4), (l3.get_label(), l4.get_label()), loc=1, fontsize = 12)
ax.add_artist(leg1)

plt.show()

```

<Figure size 750x1000 with 2 Axes>

Note that the highly peaked Laplace distribution has positive kurtosis while the flat-topped uniform distribution has negative kurtosis. As for skew, the modified distribution with a longer tail on the left has a negative skew, whereas the log-normal distribution with a longer tail on the right has a positive skew.



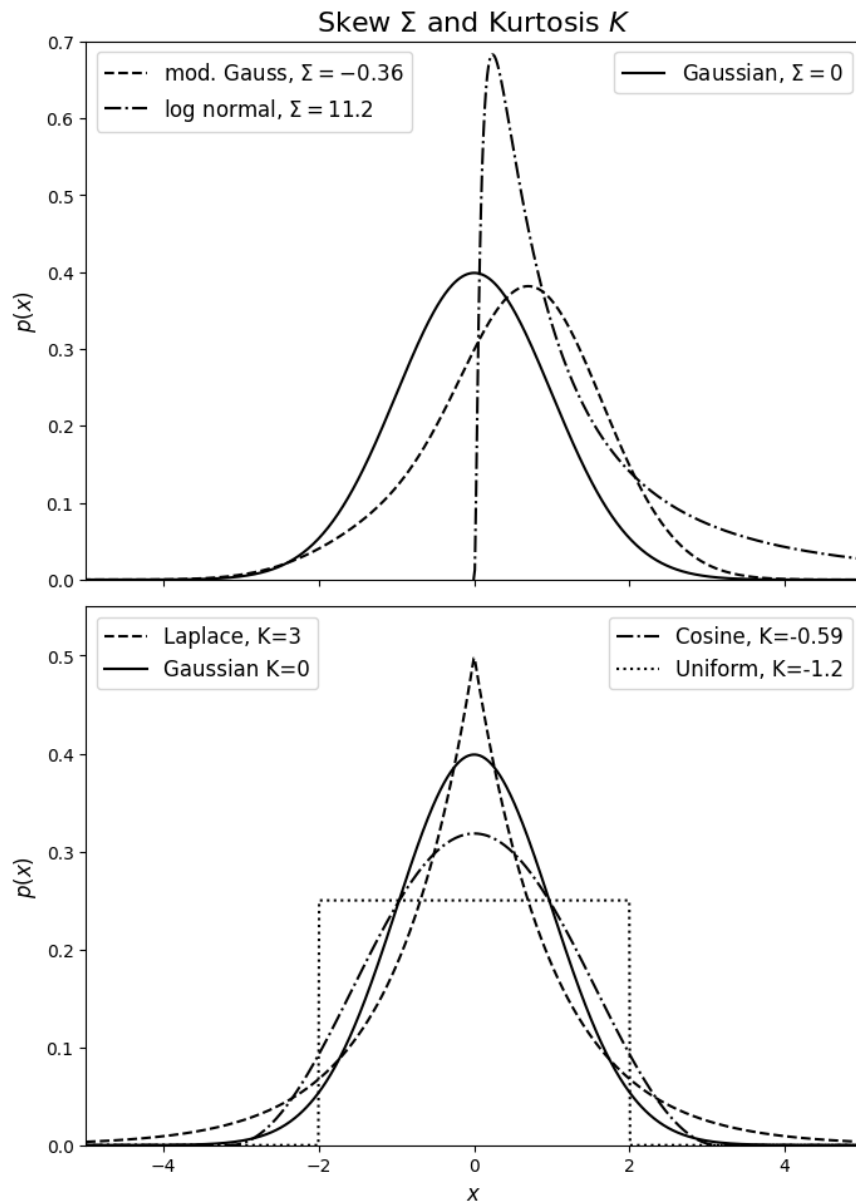


Figure 15.1: output

### 15.1.1 Useful NumPy and SciPy functions

The cell below computes multiple statistical functions on a one-dimensional array  $\mathbf{x}$ .

```
import numpy as np
import scipy.stats
x = np.random.random(100) # 100 random numbers

q25, q50, q75 = np.percentile(x, [25, 50, 75])
mean = np.mean(x)
median = np.median(x)
variance = np.var(x)
standard_deviation = np.std(x)
skew = scipy.stats.skew(x)
kurtosis = scipy.stats.kurtosis(x)
mode = scipy.stats.mode(x)
```

## 15.2 Data-based estimates of descriptive statistics

Any of these quantities can be estimated directly from data, in which case they are called sample statistics (instead of population statistics). Assume we are given  $N$  measurements,  $x_i$ , where  $i$  ranges from 1 to  $N$ . In general, when estimating the above quantities for a sample of  $N$  measurements, the integrals of the form  $\int_{-\infty}^{\infty} g(x)h(x)dx$  become  $\frac{1}{N} \sum_{i=1}^N g(x_i)$ .

For example, the equation for the arithmetic mean,  $\mu = \int_{-\infty}^{\infty} xh(x) dx$  becomes

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i,$$

and the equation for standard deviation,  $\sigma = \sqrt{V} = \sqrt{\int_{-\infty}^{\infty} (x - \mu)^2 h(x) dx}$  becomes

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

when working with sample statistics. The reason for the  $(N-1)$  term instead of  $N$  is related to the fact that  $\bar{x}$  also comes from data. By replacing  $N$  with  $N-1$ , the sample variance  $s^2$  becomes unbiased (and the sample standard deviation becomes less biased) – this is known as Bessel's correction.

When we say bias, we mean that if we drew a large number of samples and calculated the uncorrected sample standard deviation, we would notice that each sample would underestimate the population variance. The amount that it's underestimated depends on the size of  $N$ : 20% for  $N = 2$ , 3% for  $N = 10$ , and less than 1% for  $N > 30$ . As  $N \rightarrow \infty$ , the percentage goes to zero.

We use different symbols in the above two equations ( $\bar{x}$  and  $s$ ) rather than  $\mu$  and  $\sigma$  because the latter represents the “truth” (they are definitions based on the true  $h(x)$ , whatever it may be); the former are simply estimators of that truth based on a finite-size sample ( $\hat{x}$  is often used instead of  $x$ ). These estimators have a variance and a bias, and often they are judged by comparing their mean squared errors,

$$\text{MSE} = V + \text{bias}^2$$

where  $V$  is the variance, and the **bias** is defined as **the expectation value of the difference between the estimator and its true (population) value**. Estimators whose variance and bias vanish as the sample size goes to infinity are called consistent estimators.

We can demonstrate Bessel's correction by generating a bunch of  $N = 5$  samples from a standard Gaussian and calculating the uncorrected sample standard deviation for each set of 5. A histogram of the uncorrected sample standard deviations should have a mean noticeably below 1, while the corrected sample standard deviation should have a mean closer to 1. To calculate the corrected and uncorrected samples, we can use `np.std`; for the corrected sample, we can set `ddof = 1` whereas for the uncorrected sample, `numpy` defaults `ddof = 0`. The divisor used by `numpy` is  $N - \text{ddof}$ , where  $N$  represents the number of elements.

```
uncorrected = []
corrected = []
for x in range(1,5000):
 samples = np.random.normal(loc=5.0, scale=1.0, size = 5)
 uncorrected.append(np.std(samples, ddof=0))
 corrected.append(np.std(samples, ddof=1))

fig, ax = plt.subplots(1,2)
fig.set_size_inches(15,6)
ax[0].hist(uncorrected);
ax[1].hist(corrected);
ax[0].text(1.65, 1200, f'mean = {np.mean(uncorrected):.2}',
 bbox={'facecolor':'white', 'alpha':0.5, 'pad':1, 'boxstyle':"round"})
ax[1].text(1.85, 1200, f'mean = {np.mean(corrected):.2}',
 bbox={'facecolor':'white', 'alpha':0.5, 'pad':1, 'boxstyle':"round"})
```

```
ax[0].set_title("Uncorrected Standard Deviations");
ax[1].set_title("Corrected Standard Deviations");
```

<Figure size 1500x600 with 2 Axes>

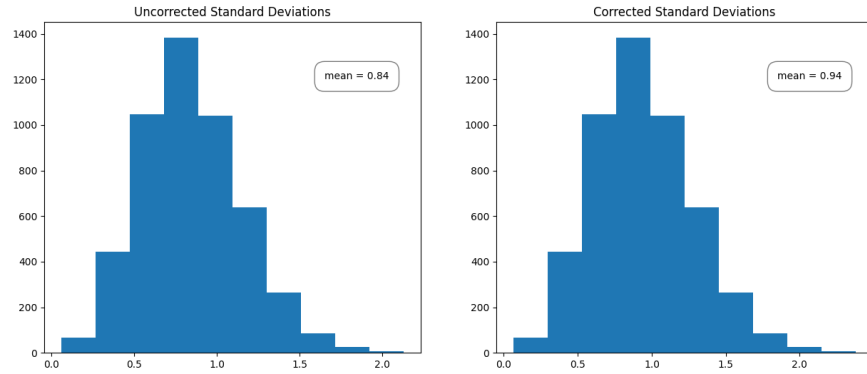


Figure 15.2: output

We expect the biased sample to underestimate the population variance, which we can see in the graph on the left. Note that the shape of the histogram is the same, just shifted slightly left.

### 15.2.1 Uncertainty in our estimators $\bar{x}$ and $s$

When  $N$  is large (at least ten or so), and if the variance of  $h(x)$  is finite, we expect from the central limit theorem (in chapter 3.4) that  $x$  and  $s$  will be distributed around their values given by eqs. 1 and 2 according to Gaussian distributions with the widths (standard errors) equal to

$$\sigma_{\bar{x}} = \frac{s}{\sqrt{N}},$$

which is called *the standard error of the mean*, and

$$\sigma_s = \frac{s}{\sqrt{2(N-1)}} = \frac{1}{\sqrt{2}} \sqrt{\frac{N}{N-1}} \sigma_{\bar{x}},$$

which is the *error of the standard deviation*.

Note that for large  $N$ , the uncertainty of the location parameter is about 40% larger than the uncertainty of the scale parameter ( $\sigma_{\bar{x}} \sim \sqrt{2}\sigma_s$ ). Note also that for small  $N$ ,  $\sigma_s$  is not much smaller than  $s$  itself.

### 15.2.2 Example of standard deviation vs. standard error

**Standard deviation:** Imagine we flip a coin 16 times; this will be one trial. As the number of trials goes to infinity, the number of heads flipped in one trial will yield a normal distribution with mean  $\mu = 8$  and std  $\sigma = 2$ . No matter how many measurements we perform, the standard deviation will not reduce as its a property intrinsic to the nature of the coin.

**Standard error:** The average number of heads in a 16 flip coin toss is eight since each side has a 50/50 chance of being landed on. With enough measurements, the error of that estimate of the mean number of heads can become arbitrarily small.

Below is the distribution of a 16-flip coin toss for  $N = 5000$  and  $N = 15000$

```
import numpy as np
from matplotlib import pyplot as plt
fig, ax = plt.subplots(1,2)
fig.set_size_inches(10,4)
np.random.seed(42)
N = [5000, 15000]

for m,k in enumerate(N):
 y = np.random.binomial(n=16, p=0.5, size=k) # N trials of 16 flip toss
 values = []
 x = np.linspace(1,16, num = 16)
 for i in range(1,17):
 values.append(np.count_nonzero(y == i))
 ax[m].bar(x,values)
 print(f'Mean = {np.mean(y):.2}, Std = {np.std(y):.2}')
```

```
Mean = 8.0, Std = 2.0
```

```
Mean = 8.0, Std = 2.0
```

```
<Figure size 1000x400 with 2 Axes>
```

Thus we can see that increasing  $N$  *does not* decrease the standard deviation.

The standard error of the mean, on the other hand, *does* decrease with increasing  $N$ . We'll import `sem` from `scipy.stats`, which computes the standard error. Then we'll use `np.random.binomial` again to simulate a 16-flip coin toss and increase the size of  $N$  on each iteration.

```
from scipy.stats import sem
j = []
for i in [3,30,100,1000]:
```

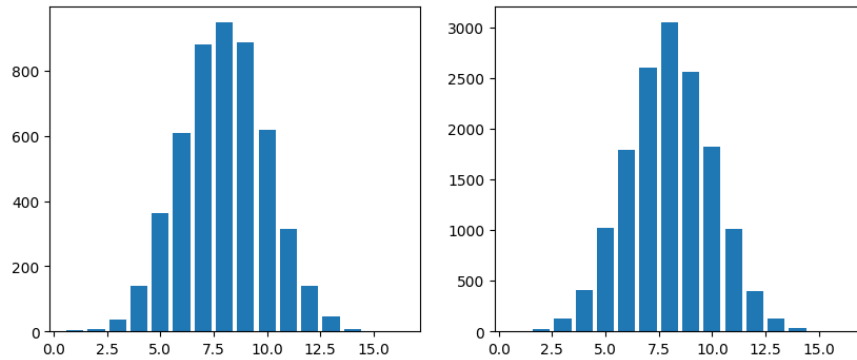


Figure 15.3: output

```
sample = np.random.binomial(n=16, p=0.5, size= i)
sample = round(sem(sample),2)
j.append(sample)
print(j)
```

```
[np.float64(0.88), np.float64(0.35), np.float64(0.17), np.float64(0.06)]
```

### 15.2.3 Estimator efficiency

Estimators can be compared in terms of their **efficiency**, which measures how large a sample is required to obtain a given accuracy.

For example, the median determined from data drawn from a Gaussian distribution shows a scatter around the true location parameter,  $\mu$ , larger by a factor of  $\pi/2 \sim 1.253$  than the scatter of the mean value (see eq. 3 below). Since the scatter decreases with  $1/\sqrt{N}$ , the efficiency of the mean is  $\pi/2$  times larger than the efficiency of the median.

An estimator is asymptotically normal if its distribution around the true value approaches a Gaussian distribution for large sample size, with variance decreasing proportionally to  $1/N$ . The smallest attainable variance for an unbiased estimator is called the **minimum variance bound (MVB)** and such an estimator is called the **minimum variance unbiased estimator (MVUE)**.

### 15.2.4 Robust descriptive statistics

For the case of real data, which can have “outliers,” quantiles offer a more robust method for determining location and scale parameters than the mean

and standard deviation. \* The median is a much more robust estimator of the location than the mean \* The interquartile range ( $q_{75} - q_{25}$ ) is a more robust estimator of the scale parameter than the standard deviation.

In other words, the median and interquartile range are much less affected by the presence of outliers than the mean and standard deviation.

For example, if you take 25% of your measurements that are larger than  $q_{75}$  and arbitrarily modify them by adding a large number to all of them (or multiply them all by a large number/different large numbers), both the mean and the standard deviation will be severely affected. However, the median and the interquartile range will remain constant.

Furthermore, even in the absence of outliers, for some distributions that do not have finite variance, such as the Cauchy distribution, the median and the interquartile range are the best choices for estimating location and scale parameters. We often see the interquartile range renormalized so that the width estimator,  $\sigma_G$ , becomes an unbiased estimator of  $\sigma$  for a Gaussian distribution,

$$\sigma_G = 0.7413(q_{75} - q_{25})$$

```
import numpy as np

np.random.seed(0)
x = np.random.normal(size=1000) # 1000 normally distributed points
```

There are downsides to robustness; as we have already discussed, the efficiency of the median as a location parameter is worse than that of the mean in the case of a Gaussian distribution. Additionally, it's much easier to compute the mean than the median for large samples. However, to avoid the possibility of outliers severely affecting results, one might be willing to pay the price of  $\sim 25\%$  larger errors.

To compute the standard error,  $\sigma_{q_p}$  for an arbitrary quantile  $q_p$ , we can use

$$\sigma_{q_p} = \frac{1}{h_p} \sqrt{\frac{p(1-p)}{N}}$$

where  $h_p$  is the value of the probability distribution function at the  $p$ th percentile (e.g., for the median,  $p = 0.5$ ). For a Gaussian, the standard error for the median is

$$\sigma_{q^{50}} = s \sqrt{\frac{\pi}{2N}}$$

with  $h_{50} = 1/(s\sqrt{2\pi})$  and  $s \sim \sigma$  in the limit of large  $N$ .

To demonstrate robust statistics, we'll sample a Gaussian with and without outliers. We expect the mean and standard deviation to be heavily affected by the outliers, but the median and  $\sigma_G$  to be less affected. To represent the Gaussian with outliers, we'll put together a sample where we'll draw 95% of the data from a normal Gaussian distribution and 5% from a Cauchy distribution. We'll draw the Gaussian samples using `np.random.normal`. For the 5% outliers, we'll import `Cauchy` from `scipy.stats` and call `cauchy.rvs` to draw five samples from a Cauchy distribution. Then we'll concatenate the 95 normal samples with five outlier data points to create a Gaussian with outliers. We'll set the location parameter  $\mu$  to eight to prevent our graph from going negative for a more readable graph.

```
import numpy as np
import scipy.stats as stats
from matplotlib import pyplot as plt
from scipy.stats import cauchy

normal = np.random.normal(loc=8.0, scale=1.0, size=100) # 100 samples from a Gaussian

a = np.random.normal(loc=8.0, scale=1.0, size=95) #95 samples from a Gaussian
b = cauchy.rvs(loc=8.0, scale=20, size=5) #5 samples from a Cauchy
normal_with_outliers = np.concatenate([a, b]) # combine to create Gaussian with outliers
```

Next, we'll plot two bar graphs: one that shows the mean and standard deviations and one with the median and  $\sigma_G$  for both the normal distribution and the normal with outliers.

```
labels = ['no outliers', 'with outliers']
means = [np.mean(normal), np.mean(normal_with_outliers)]
standard_deviations = [np.std(normal), np.std(normal_with_outliers)]
medians = [np.median(normal), np.median(normal_with_outliers)]

x = np.arange(len(labels)) # the label locations
width = 0.35 # the width of the bars

fig, ax = plt.subplots(1,2)
fig.set_size_inches(14,6)
rects1 = ax[0].bar(x - width/2, means, width, label='mean', color = 'steelblue')
rects2 = ax[0].bar(x + width/2, standard_deviations, width, label='std', color = 'darkk')
rects3 = ax[1].bar(x - width/2, medians, width, label='median', color = 'olivedrab')

titles = ["means and std's", "medians"]
rected = [(1,2), (3,4)]
```



```

for i in [0,1]:
 ax[i].set_ylabel('value')
 ax[i].set_xticks(x);
 ax[i].set_xticklabels(labels)
 ax[i].legend()
 ax[i].set_title(titles[i])

combined = means + standard_deviations + medians
ax[1].set_ylim([0, np.max(combined)+0.5])

ax[0].bar_label(rects1, padding=3)
ax[0].bar_label(rects2, padding=3)
ax[1].bar_label(rects3, padding=3)

fig.tight_layout()
plt.show()

```

<Figure size 1400x600 with 2 Axes>

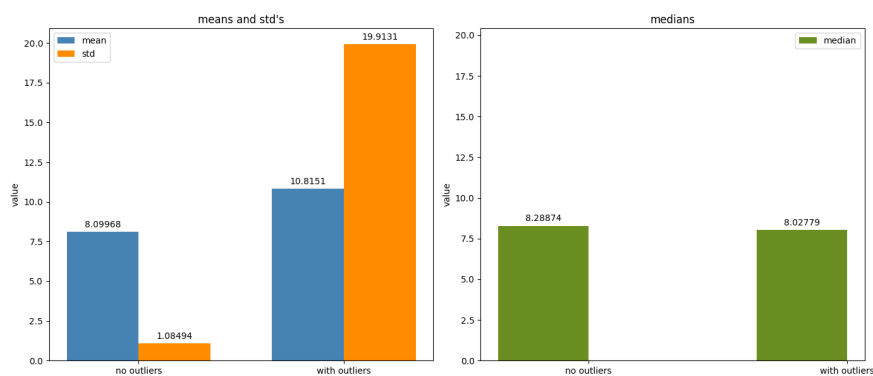


Figure 15.4: output

We can see that the median and  $\sigma_G$  are mostly unaffected by the presence of outliers, whereas the mean and standard deviation are more affected.



## Chapter 16

# Visualisation: An Introduction to Matplotlib and Seaborn

```
Import necessary libraries
import matplotlib.pyplot as plt # The standard plotting interface
import seaborn as sns # For statistical graphics
import numpy as np # For numerical operations and data generation
import pandas as pd # For working with tabular data (DataFrames)

--- Data Simulation Code ---
Scenario 1: Star Mass vs. Luminosity (Scatter Plot Example)
num_stars = 100
star_masses = np.random.lognormal(mean=0., sigma=0.5, size=num_stars) * 0.5 + 0.1
star_luminosities = star_masses**3.5 * np.random.normal(1.0, 0.3, size=num_stars)

Scenario 2: Variable Star Brightness Over Time (Line Plot Example)
num_time_points = 50
time_points = np.arange(num_time_points)
brightness_signal = 5 + 2 * np.sin(time_points / 5 * np.pi)
brightness_noise = np.random.normal(0, 0.5, size=num_time_points)
variable_star_brightness = brightness_signal + brightness_noise

Scenario 3: Distribution of Galaxy Redshifts (Histogram Example)
num_galaxies = 500
galaxy_redshifts = np.random.gamma(shape=2.0, scale=0.1, size=num_galaxies)
```

```

Scenario 4: Count of Different Object Types (Bar Plot Example)
object_types = ['Star', 'Galaxy', 'Quasar', 'Nebula', 'Star Cluster']
object_counts = np.array([150, 75, 20, 40, 35])

Scenario 5: Exoplanet Properties (DataFrame for Seaborn Examples)
exoplanet_data_dict = {
 'Planet Name': [f'Exo-{i:03d}' for i in range(20)],
 'Type': np.random.choice(['Gas Giant', 'Super-Earth', 'Neptune-like'], size=20),
 'Orbital_Period_Days': np.random.lognormal(mean=1.5, sigma=1.0, size=20) + 1,
 'Radius_Earth': np.random.lognormal(mean=0., sigma=0.4, size=20) * 5 + 0.5
}
exoplanets_df = pd.DataFrame(exoplanet_data_dict)
exoplanets_df['Type'] = exoplanets_df['Type'].astype('category')

exoplanets_df.head()

```

	Planet Name	Type	Orbital_Period_Days	Radius_Earth
0	Exo-000	Neptune-like	10.570501	5.866134
1	Exo-001	Neptune-like	9.431618	2.631607
2	Exo-002	Super-Earth	17.788973	3.168798
3	Exo-003	Neptune-like	4.080576	3.127169
4	Exo-004	Neptune-like	3.326556	5.397618

## 16.1 Matplotlib Basics: Figure and Axes

In Matplotlib, every plot resides within a **Figure**. Think of the Figure as the overall window or canvas where your visualization will appear. Inside the Figure, you create one or more **Axes** (also often called subplots). The Axes is the actual plotting area where your data points, lines, bars, etc., are drawn, complete with x and y axes, tick marks, and labels.

The most common way to start a plot is by using the `plt.subplots()` function. This function conveniently creates both a **Figure** and a single **Axes** object for you:

```
fig, ax = plt.subplots()
```

Once you have the `ax` (Axes) object, you call various plotting methods on it, such as `ax.scatter()`, `ax.plot()`, `ax.hist()`, `ax.bar()`, passing your data as arguments.

## 16.2 Common Plot Types with Matplotlib

Let's generate some basic plot types using the simulated data and Matplotlib's `ax` object methods.

### 16.2.1 Scatter Plots

**Purpose:** To show the relationship or correlation between two numerical variables. Each data point is represented by a marker.

**Method:** `ax.scatter(x_values, y_values)`

**Example 1:** Simple Scatter Plot - Star Mass vs. Luminosity)

```
Use the simulated star_masses and star_luminosities

1. Create a Figure and Axes
fig, ax = plt.subplots()

2. Plot the data on the Axes using the scatter method
ax.scatter(star_masses, star_luminosities)

3. Add labels and a title for clarity
ax.set_xlabel("Star Mass (Solar Masses)")
ax.set_ylabel("Star Luminosity (Solar Luminosities)")
ax.set_title("Simulated Stellar Mass vs. Luminosity")

4. Display the plot
plt.show()
```

<Figure size 640x480 with 1 Axes>

### 16.2.2 Scatter Plot - Exoplanet Period vs. Radius

Let's apply a scatter plot to our exoplanet data, which is stored in a Pandas DataFrame. We'll plot the orbital period against the planet's radius.

**Example 2:** Scatter Plot - Exoplanet Period vs. Radius)

```
Use data from the exoplanets_df DataFrame
periods = exoplanets_df['Orbital_Period_Days']
radii = exoplanets_df['Radius_Earth']

fig, ax = plt.subplots(figsize=(8, 5)) # Optional: Set figure size for better readability
```

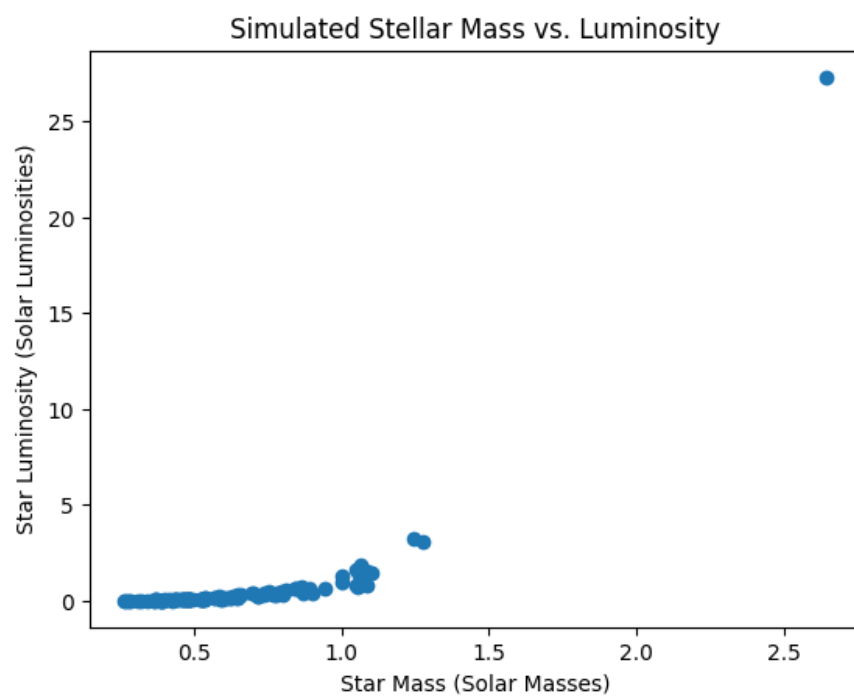


Figure 16.1: output

```
ax.scatter(periods, radii)

ax.set_xlabel("Orbital Period (Days)")
ax.set_ylabel("Radius (Earth Radii)")
ax.set_title("Simulated Exoplanet Orbital Period vs. Radius")

Add a grid for easier reading of values
ax.grid(True, linestyle='--', alpha=0.6)

plt.show()
```

<Figure size 800x500 with 1 Axes>

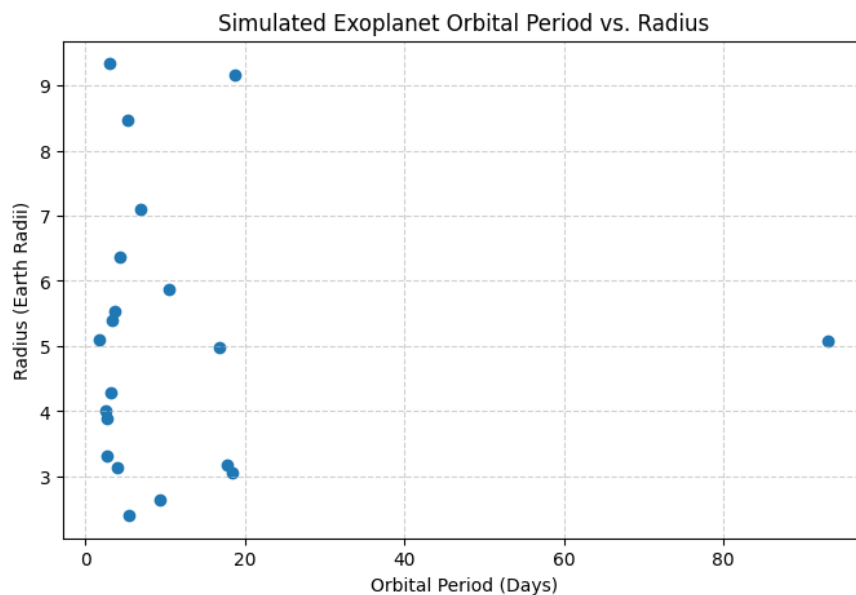


Figure 16.2: output

## 16.3 Line Plots

**Purpose:** To show a trend or change in a numerical variable over a continuous sequence, like time or position along a path.

**Method:** `ax.plot(x_values, y_values)`

**Example 3:** Line Plot - Variable Star Light Curve)

```
Use the simulated time_points and variable_star_brightness

fig, ax = plt.subplots(figsize=(10, 5)) # Make it wider to see the time trend

ax.plot(time_points, variable_star_brightness)

ax.set_xlabel("Time (Arbitrary Units)")
ax.set_ylabel("Brightness")
ax.set_title("Simulated Variable Star Light Curve")

plt.show()
```

<Figure size 1000x500 with 1 Axes>

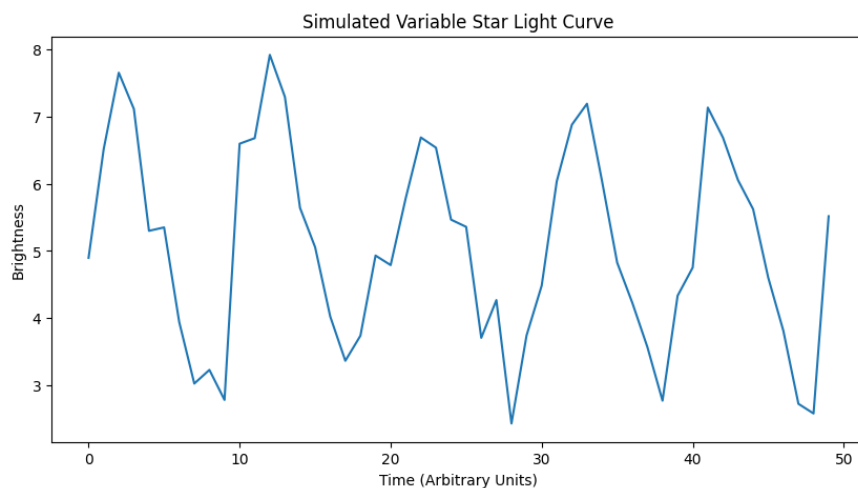


Figure 16.3: output

### 16.3.1 Histograms

**Purpose:** To show the distribution of a single numerical variable. It divides the range of data into “bins” and counts how many data points fall into each bin, displayed as bars.

**Method:** `ax.hist(data, bins=number_of_bins)`

**Example 4:** Histogram - Galaxy Redshift Distribution)



```
Use the simulated galaxy_redshifts

fig, ax = plt.subplots(figsize=(7, 5))

ax.hist(galaxy_redshifts, bins=20) # We choose to divide the data into 20 bins

ax.set_xlabel("Galaxy Redshift")
ax.set_ylabel("Number of Galaxies (Count)")
ax.set_title("Simulated Distribution of Galaxy Redshifts")

plt.show()
```

<Figure size 700x500 with 1 Axes>

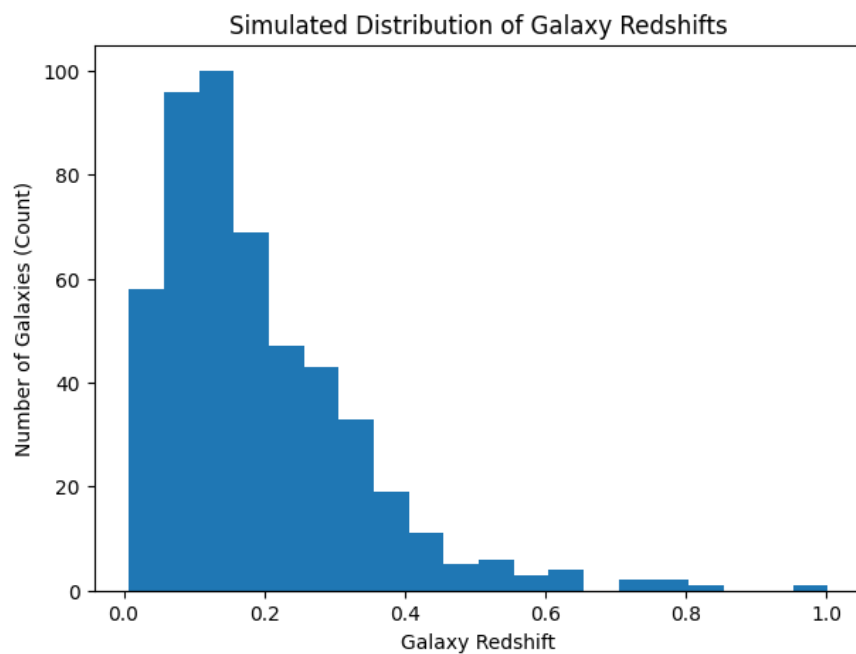


Figure 16.4: output

## 16.4 Bar Plots

**Purpose:** To compare discrete values across different categories. Each category has a bar whose length represents the value.

**Method:** `ax.bar(category_labels, category_values)`

**Example 5:** Bar Plot - Count of Object Types)

```
Use the simulated object_types (our categories) and object_counts (our values)

fig, ax = plt.subplots(figsize=(8, 5))

ax.bar(object_types, object_counts)

ax.set_xlabel("Object Type")
ax.set_ylabel("Count")
ax.set_title("Simulated Count of Astronomical Object Types")

Rotate x-axis labels if they are long and overlap
plt.xticks(rotation=45, ha='right') # Rotate labels by 45 degrees, align text to the r

plt.tight_layout() # Automatically adjust plot parameters for a tight layout
plt.show()
```

<Figure size 800x500 with 1 Axes>

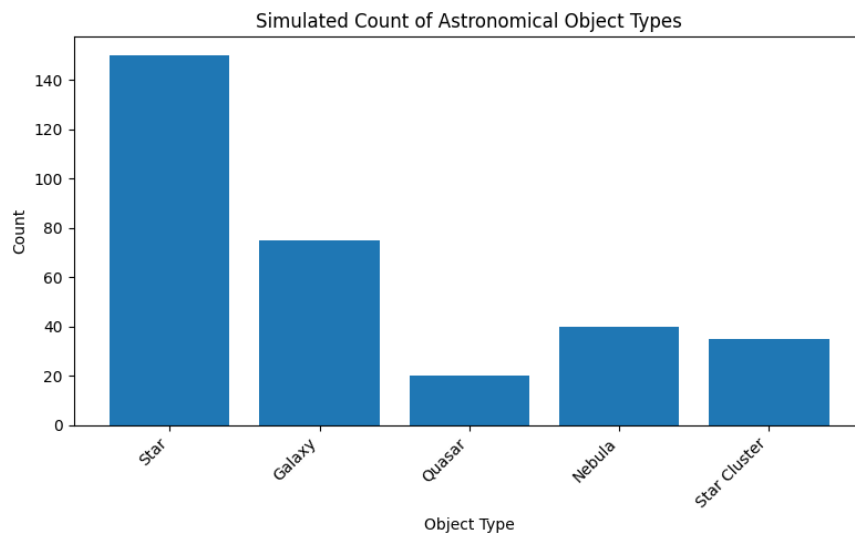


Figure 16.5: output

## 16.5 Customizing Plots: Labels, Titles, Legends, and Appearance

Adding informative text and adjusting the appearance are key to making your plots understandable and visually appealing. You can control many aspects of the plot through the `ax` object and the arguments of the plotting methods.

- Adding Labels: `ax.set_xlabel("Your Label")`, `ax.set_ylabel("Your Label")`.
- Adding Titles: `ax.set_title("Your Title")` for the Axes title, `fig.suptitle("Overall Title")` for a title covering the whole Figure (useful with multiple subplots).
- Adding Legends: If you add a `label='...'` argument to your plotting calls (like `ax.plot()` or `ax.scatter()`), you can then call `ax.legend()` to display these labels in a legend box.
- Customizing Appearance: Arguments like `color`, `linestyle`, `marker`, `s` (size for scatter points), `alpha` (transparency), etc., can be passed to the plotting methods.

**Example 6:** Line Plot with Color and Label)

```
Use the simulated time_points and variable_star_brightness

fig, ax = plt.subplots(figsize=(10, 6))

Plot the data with specific color, line style, and a label for the legend
ax.plot(time_points, variable_star_brightness, color='purple', linestyle='--', label='Observed Bri

ax.set_xlabel("Time (Arbitrary Units)")
ax.set_ylabel("Brightness")
ax.set_title("Simulated Variable Star Light Curve with Style")

Add a legend to show the label we provided in ax.plot()
ax.legend()

plt.show()
```

<Figure size 1000x600 with 1 Axes>

**Example 7:** Scatter Plot with Colors based on Type)

```
Use exoplanets_df. We want to plot different 'Type' categories with different colors.
```

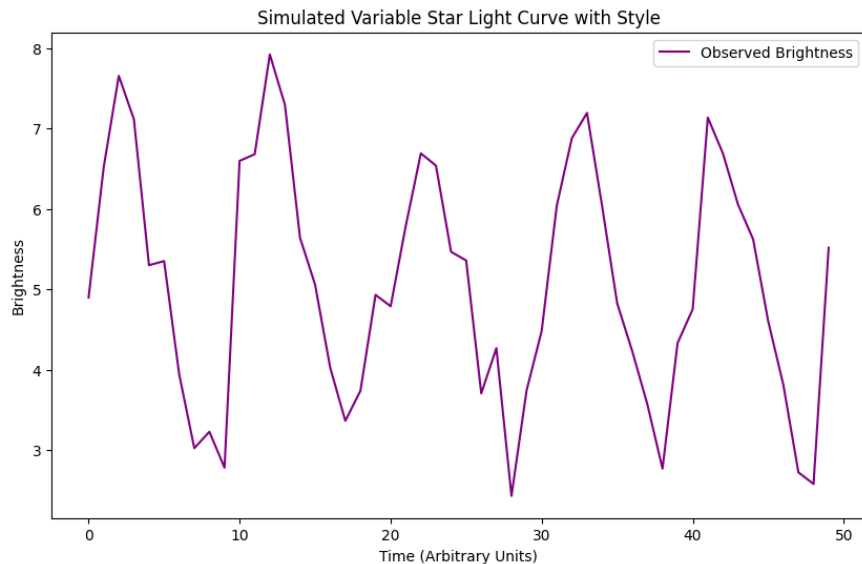


Figure 16.6: output

```
fig, ax = plt.subplots(figsize=(10, 6))

Get the unique planet types from the DataFrame column
unique_types = exoplanets_df['Type'].unique()

Define a list of colors, one for each type (make sure you have enough colors!)
colors = ['blue', 'green', 'red']

Loop through each unique planet type
for i, planet_type in enumerate(unique_types):
 # Filter the DataFrame to get only the rows for the current planet type
 subset_df = exoplanets_df[exoplanets_df['Type'] == planet_type]
 # Plot this subset of data
 ax.scatter(subset_df['Orbital_Period_Days'], subset_df['Radius_Earth'],
 color=colors[i], # Use the color corresponding to this type
 label=planet_type, # Use the type name as the label for the legend
 alpha=0.7, # Make points slightly transparent to see overlaps
 s=50) # Set point size

Add labels and title
ax.set_xlabel("Orbital Period (Days)")
ax.set_ylabel("Radius (Earth Radii)")
```

```

ax.set_title("Simulated Exoplanet Orbital Period vs. Radius by Type")

It's common for orbital periods to be plotted on a logarithmic scale because they vary greatly
ax.set_xscale('log')
ax.grid(True, which='both', linestyle='--', alpha=0.5) # Add a grid to help read values

Add a legend to identify the different colored points
ax.legend(title="Planet Type")

plt.show()

```

<Figure size 1000x600 with 1 Axes>

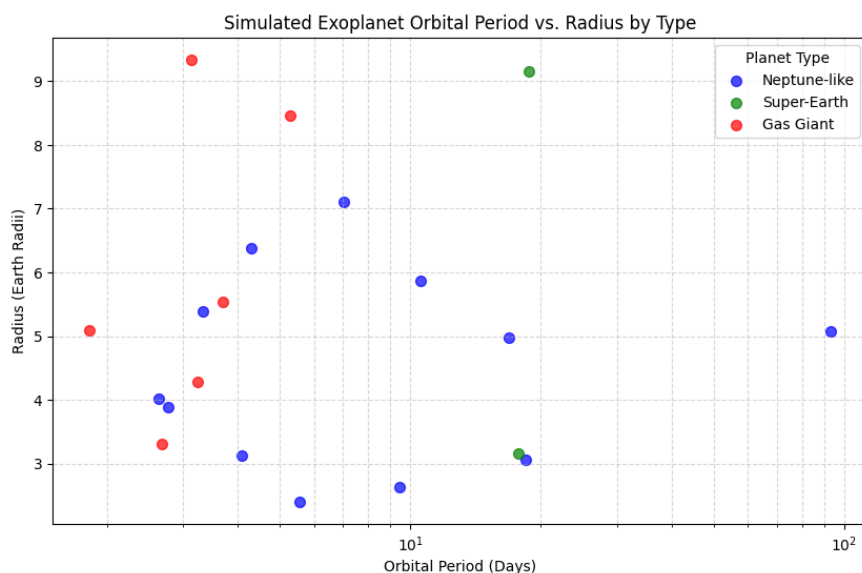


Figure 16.7: output

## 16.6 Introducing Seaborn

Seaborn is a library specifically designed to make creating statistical graphics in Python easier and more visually appealing. It is built on top of Matplotlib, meaning you can often use Matplotlib functions to further customize plots created with Seaborn.

Seaborn works particularly well with structured data like Pandas DataFrames. Many Seaborn plotting functions can take the DataFrame itself as the `data` argument, and then you just specify the column names for the `x`, `y`, `hue`, etc., arguments. This often reduces the amount of code needed compared to pure Matplotlib, especially when you want to color or separate data based on categories.

### 16.6.1 Seaborn Scatter Plots

**Method:** `sns.scatterplot(data=dataframe, x='column_x', y='column_y', ...)`

Seaborn's `scatterplot` has nice default styling and integrates easily with DataFrames. The `hue` argument is especially powerful for coloring points by category.

**Example 8:** Seaborn Scatter Plot - Exoplanet Period vs. Radius (Basic)

```
Use the exoplanets_df DataFrame

fig, ax = plt.subplots(figsize=(10, 6))

Use sns.scatterplot, specifying the dataframe and column names for x and y
sns.scatterplot(data=exoplanets_df, x='Orbital_Period_Days', y='Radius_Earth', ax=ax)

Add labels and title using the Matplotlib axes object
ax.set_xlabel("Orbital Period (Days)")
ax.set_ylabel("Radius (Earth Radii)")
ax.set_title("Simulated Exoplanet Orbital Period vs. Radius (Seaborn)")
ax.set_xscale('log') # Still use log scale for x-axis

plt.show()
```

<Figure size 1000x600 with 1 Axes>

**Example 9:** Seaborn Scatter Plot - Period vs. Radius with `hue` by Type

The seaborn's `hue` argument makes it easy to colour points based on a categorical column.

```
fig, ax = plt.subplots(figsize=(10, 6))

Use 'hue' and pass the column name 'Type'
sns.scatterplot(data=exoplanets_df, x='Orbital_Period_Days', y='Radius_Earth',
 hue='Type', # Color points based on the 'Type' column
```

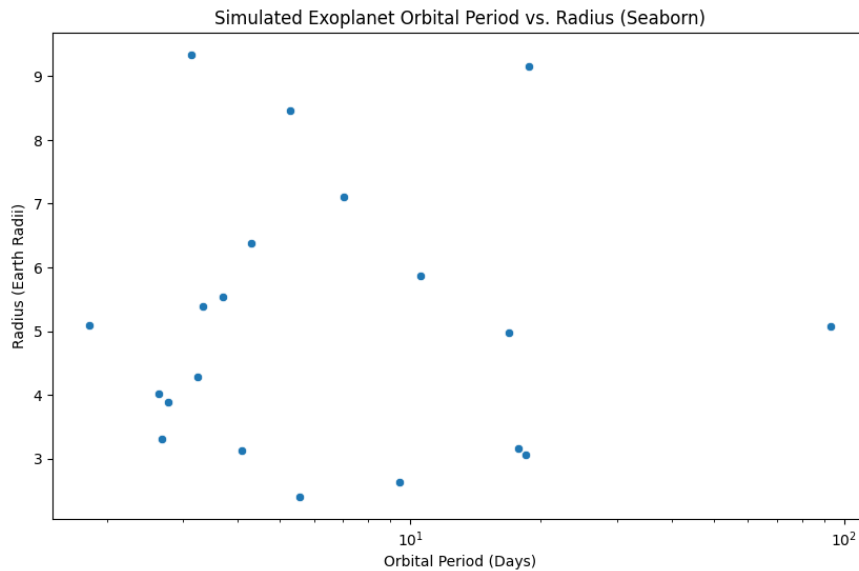


Figure 16.8: output

```
ax=ax)

ax.set_xlabel("Orbital Period (Days)")
ax.set_ylabel("Radius (Earth Radii)")
ax.set_title("Simulated Exoplanet Period vs. Radius by Type (Seaborn Hue)")
ax.set_xscale('log')
ax.grid(True, which='both', linestyle='--', alpha=0.5)

plt.show()
```

<Figure size 1000x600 with 1 Axes>

## 16.6.2 Seaborn Histograms

**Method:** `sns.histplot(data=data_array_or_series, bins=..., ax=...)` or `sns.histplot(data=dataframe, x='column_name', bins=..., ax=...)`

Seaborn's `histplot` is a versatile histogram function with several useful options.

**Example 10:** Seaborn Histogram - Galaxy Redshift Distribution

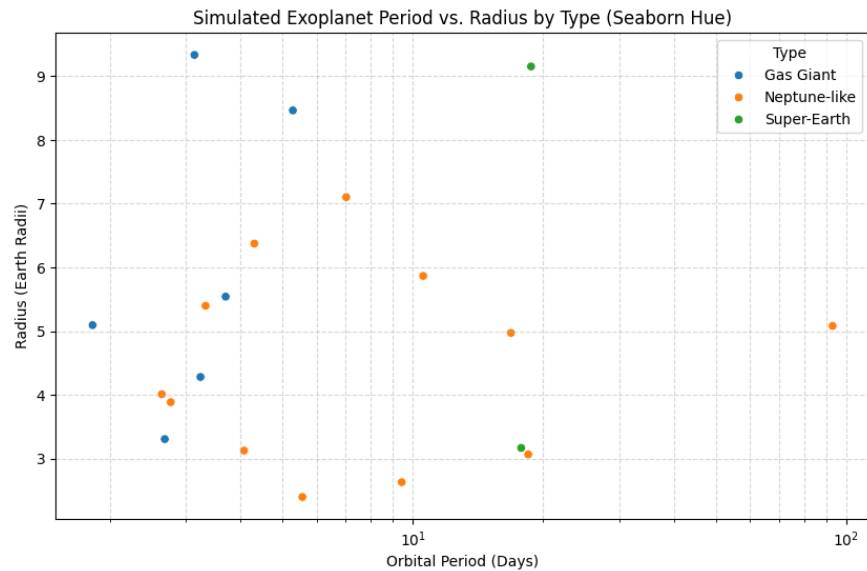


Figure 16.9: output

```
Use the simulated galaxy_redshifts array

fig, ax = plt.subplots(figsize=(7, 5))

sns.histplot can take a NumPy array directly
sns.histplot(galaxy_redshifts, bins=20, ax=ax)

ax.set_xlabel("Galaxy Redshift")
ax.set_ylabel("Count")
ax.set_title("Simulated Distribution of Galaxy Redshifts (Seaborn)")

plt.show()
```

<Figure size 700x500 with 1 Axes>

### 16.6.3 Seaborn Bar Plots

**Method:** `sns.barplot(x=category_labels, y=category_values, ax=...)`  
 or `sns.countplot(data=dataframe, x='category_column', ax=...)`



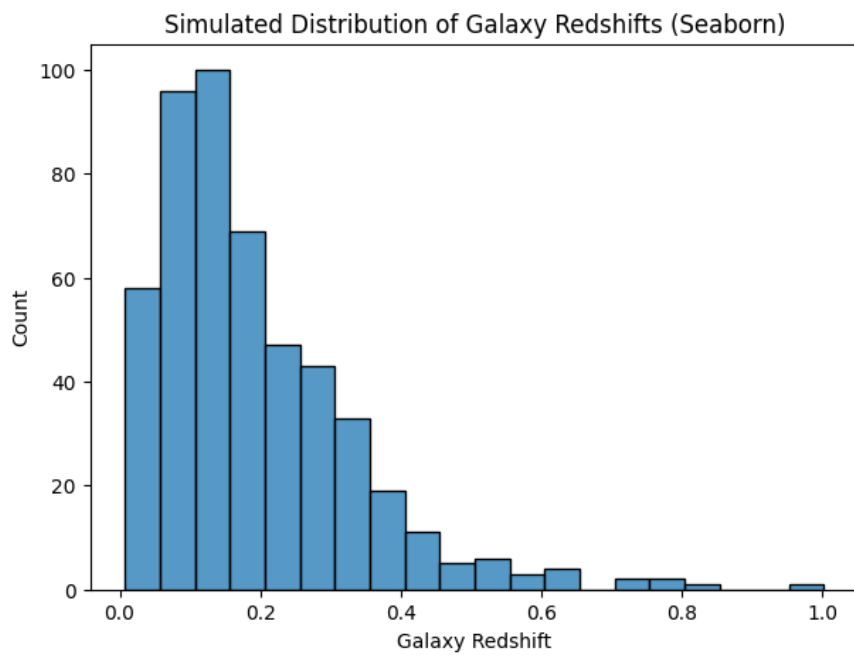


Figure 16.10: output

Seaborn offers `barplot` (for plotting a value per category) and `countplot` (specifically for showing the number of observations in each category). `countplot` is often easier when you just have a list or Series of category names and want to count them.

**Example 11:** Seaborn Bar Plot - Count of Object Types

```
Use the simulated object_types and object_counts

fig, ax = plt.subplots(figsize=(8, 5))

sns.barplot(x=object_types, y=object_counts, ax=ax)

ax.set_xlabel("Object Type")
ax.set_ylabel("Count")
ax.set_title("Simulated Count of Astronomical Object Types (Seaborn Barplot)")

plt.xticks(rotation=45, ha='right') # Rotate labels
plt.tight_layout() # Adjust layout

plt.show()
```

<Figure size 800x500 with 1 Axes>

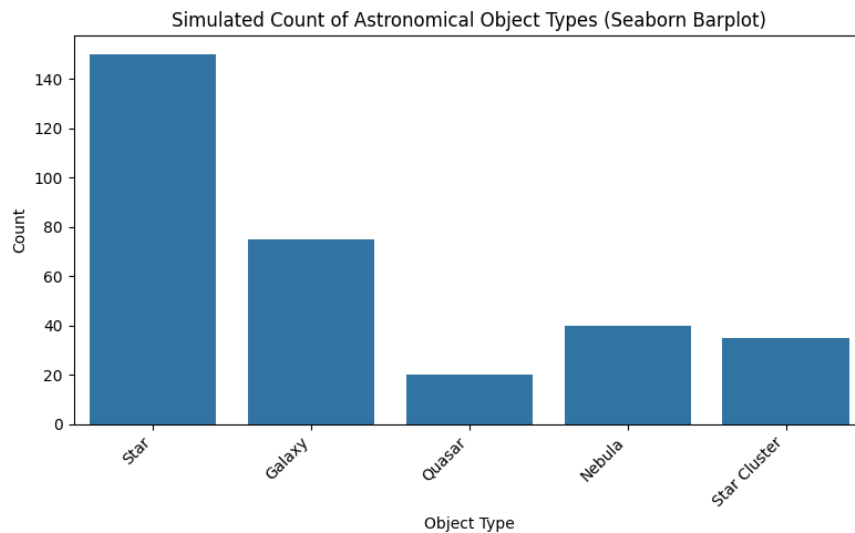


Figure 16.11: output

**Example 12:** Seaborn Countplot - Number of Each Exoplanet Type

`sns.countplot` will automatically count the occurrences of each category.

```
fig, ax = plt.subplots(figsize=(7, 5))

Use sns.countplot, specifying the dataframe and the column with categories
sns.countplot(data=exoplanets_df, x='Type', ax=ax)

ax.set_xlabel("Exoplanet Type")
ax.set_ylabel("Number of Planets")
ax.set_title("Simulated Count of Exoplanet Types (Seaborn Countplot)")

plt.show()
```

<Figure size 700x500 with 1 Axes>

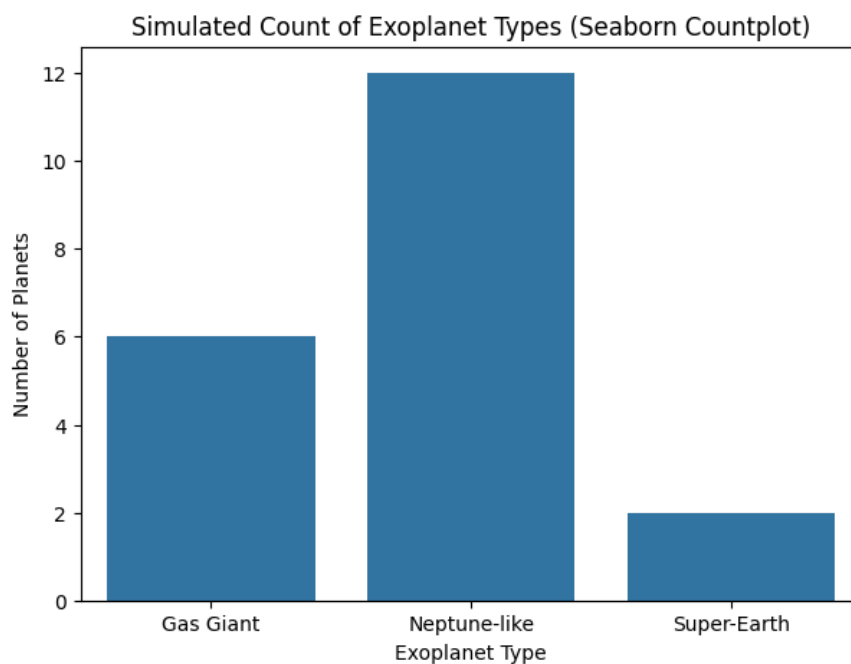


Figure 16.12: output

## 16.7 Exercises

Use Matplotlib and/or Seaborn to create plots based on the simulated data or your own variations.

1. Recreate the Star Mass vs. Luminosity scatter plot from Example 1, but this time use Seaborn (`sns.scatterplot`) and the `star_masses` and `star_luminosities` NumPy arrays directly (Seaborn can often handle arrays too).
2. Recreate the Variable Star Light Curve line plot from Example 3. Choose a different color (e.g., 'darkgreen') and a dashed line style (`linestyle='--'`).
3. Recreate the Galaxy Redshift Histogram from Example 4. Try using `bins=30` and add an edge color to the bars (`edgecolor='black'`).
4. Recreate the Object Type Bar Plot from Example 5, but make it a horizontal bar plot instead using `ax.barh()`. You'll need to swap the x and y arguments accordingly (`ax.barh(category_labels, category_values)`).
5. Recreate the Exoplanet Period vs. Radius scatter plot from Example 2, but swap the axes so Radius is on the x-axis and Period is on the y-axis. Use Matplotlib.
6. Using the `exoplanets_df`, create a scatter plot of 'Orbital\_Period\_Days' vs. 'Radius\_Earth'. Use the `size` argument in `sns.scatterplot` to make the size of the points proportional to the 'Radius\_Earth'. Set the x-axis to logarithmic scale.
7. Recreate the Exoplanet Type countplot from Example 12 as a horizontal bar plot using `sns.countplot(data=dataframe, y='category_column', ax=...)`.
8. Create a Matplotlib Figure with two Axes side-by-side (e.g., `fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))`). Plot the Star Mass vs. Luminosity scatter on `ax1` and the Galaxy Redshift histogram on `ax2`. Add titles and labels to both axes. Ensure `plt.tight_layout()` is called before `plt.show()`.

## 16.8 Summary

You've taken important steps in visualising data using Python's powerful libraries, Matplotlib and Seaborn! You were introduced to:

- The critical role of visualisation in understanding data.
- The distinct but complementary roles of **Matplotlib** (foundational plotting) and **Seaborn** (statistical graphics).
- The basic components of a Matplotlib plot: the **Figure** (canvas) and **Axes** (plotting area).

- Creating fundamental plot types: **Scatter Plots** (relationships), **Line Plots** (trends), **Histograms** (distributions), and **Bar Plots** (comparisons).
- Adding essential plot elements for clarity: **labels**, **titles**, and **legends**.
- Using **Seaborn** to simplify the creation of common statistical plots, leverage better default aesthetics, and integrate seamlessly with DataFrames, including using the **hue** parameter to color points by category.
- Applying these visualization techniques to **simulated astronomical data**.

Visualisation is an important skill in any data-driven field, including astronomy. These tools will empower you to explore your datasets, identify patterns, and communicate your findings effectively.

**Additional Resources** The hands-on examples provided in this notebook are not exhaustive. Hence, we have provided links to resources you can explore to learn how to make beautiful plots in your Data Science journey.

- Check out the Using Matplotlib page: <https://matplotlib.org/stable/users/index.html>
- Matplotlib Example Gallery page: <https://matplotlib.org/stable/gallery/>
- Seaborn Example Gallery: <https://seaborn.pydata.org/examples/index.html>



## Chapter 17

# Visualising Gaia Data with Matplotlib and Seaborn

```
!pip install numpy pandas matplotlib scipy statsmodels
```

### 17.0.1 Step 1: Import Libraries

First, we need to import the necessary Python libraries.

```
importing python packages
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

### 17.0.2 Step 2: Load the Data

Load the `gaia.csv` data into a Pandas DataFrame.

```
df = pd.read_csv('data/gaia.csv')

If you are using Google Colab, uncomment the next line:
df = pd.read_csv("https://raw.githubusercontent.com/Danselem/brics_astro/main/Week3/data/gaia.csv")
```

Let's inspect the first few rows of the DataFrame to understand the data.

```
df.head()
```

	source_id	ra	dec	pmra	pmdec	parallax	\
0	3586362911238437248	171.809877	-12.086810	80.773084	31.038119	7.191948	
1	6084196726724370560	202.890673	-46.281027	12.731613	2.493211	2.821713	
2	2499641245098603648	37.350369	-1.647166	-5.752432	-30.534618	2.098506	
3	5214925720023958912	117.438720	-74.058326	-12.103186	5.926152	2.347004	
4	6511230039178478848	335.331638	-51.731239	-0.236398	-2.922207	2.620485	
	parallax_over_error	phot_g_mean_mag	phot_bp_mean_mag	phot_rp_mean_mag	\		
0	319.736180	13.987773	14.809981	12.990531			
1	30.065174	11.581702	11.823022	11.032214			
2	82.557220	14.687146	15.298927	13.930087			
3	27.595547	17.950108	19.436530	16.751095			
4	25.010994	17.788868	19.338089	16.609950			
	bp_rp	mg					
0	1.819450	8.272006					
1	0.790808	3.834267					
2	1.368840	6.296697					
3	2.685434	9.802677					
4	2.728140	9.880776					

### 17.0.3 Step 3: Basic Scatter Plot (RA vs. Dec)

Let's create a scatter plot of Right Ascension (RA) vs. Declination (Dec) to visualise the distribution of stars on the sky.

```
plt.figure(figsize=(10, 8))
plt.scatter(df['ra'], df['dec'], s=5, alpha=0.5) # s controls marker size, alpha contr
plt.xlabel('Right Ascension (degrees)', fontsize=18)
plt.ylabel('Declination (degrees)', fontsize=18)
plt.title('Star Positions from Gaia', fontsize=20)
plt.tick_params(axis='both', labelsize=16)
plt.gca().invert_xaxis() # RA increases to the left
plt.show()
```

<Figure size 1000x800 with 1 Axes>

### 17.0.4 Step 4: Scatter Plot with Magnitude as Color

We can add more information to the scatter plot by using the `phot_g_mean_mag` column to color the points. This shows the brightness of the stars.



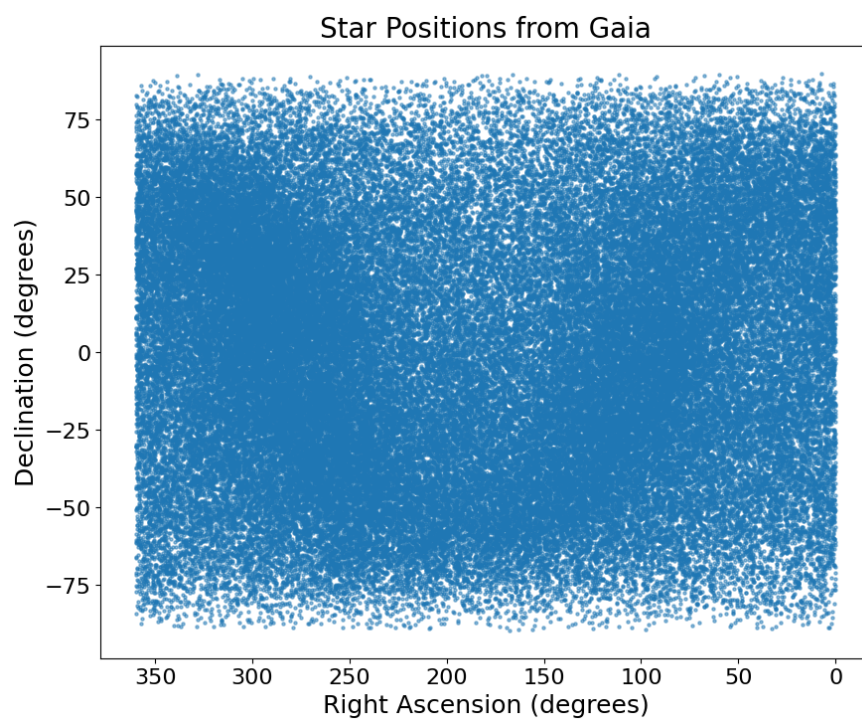


Figure 17.1: output

```
plt.figure(figsize=(11, 8))
sc = plt.scatter(df['ra'], df['dec'], c=df['phot_g_mean_mag'], cmap='viridis', s=5, al)

plt.xlabel('Right Ascension (degrees)', fontsize=18)
plt.ylabel('Declination (degrees)', fontsize=18)
plt.title('Star Positions Colored by Magnitude', fontsize=20)
plt.tick_params(axis='both', labelsize=16)
plt.gca().invert_xaxis()

cbar = plt.colorbar(sc)
cbar.set_label('G Magnitude', fontsize=18) # Explicit label and size
cbar.ax.tick_params(labelsize=20) # Tick label size

plt.show()
```

<Figure size 1100x800 with 2 Axes>

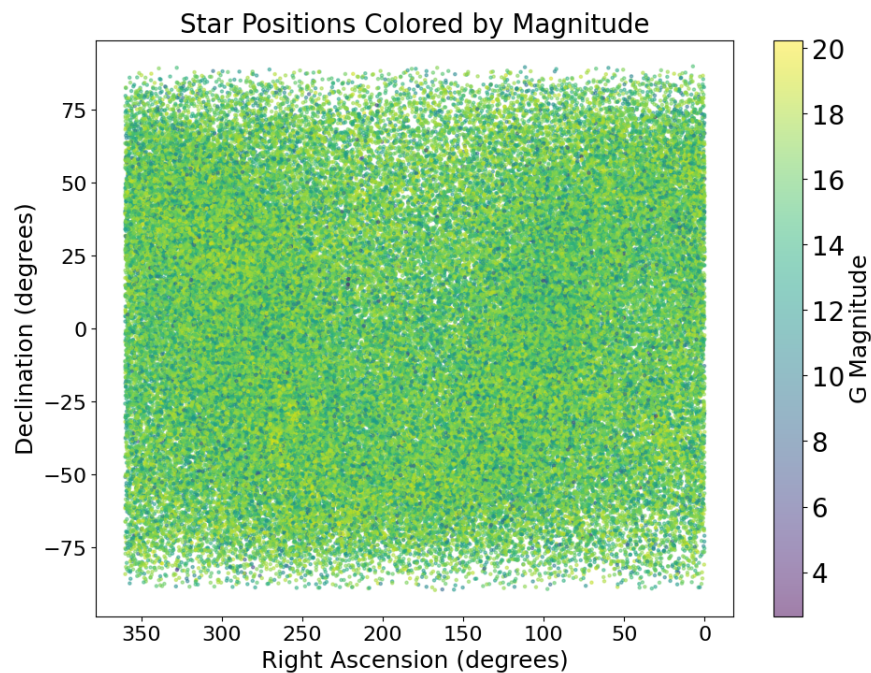


Figure 17.2: output

### 17.0.5 Step 5: Histogram of Magnitudes

A histogram shows the distribution of a single variable. Let's plot a histogram of the G magnitudes.

```
plt.figure(figsize=(10, 6))
plt.hist(df['phot_g_mean_mag'], bins=30, color='skyblue', edgecolor='black')
plt.xlabel('G Magnitude', fontsize=18)
plt.ylabel('Number of Stars', fontsize=18)
plt.title('Distribution of G Magnitudes', fontsize=20)
plt.tick_params(axis='both', labelsize=16)
plt.show()
```

<Figure size 1000x600 with 1 Axes>

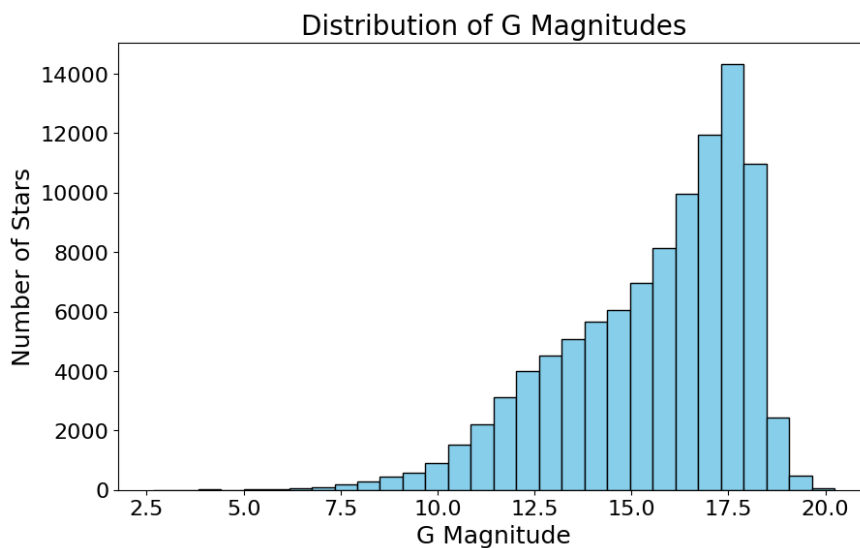


Figure 17.3: output

We can compute the tangential velocity (how fast the star is moving across the sky) from proper motion and distance (derived from parallax).

```
Constants
pc_to_km = 3.086e13 # parsec in km
yr_to_sec = 3.154e7 # year in seconds
au_to_km = 1.496e8 # au in km
```

```

#Calculate distance in parsecs
df['distance'] = 1/df['parallax']

Convert proper motions from mas/yr to rad/sec
df['pmra_rad_sec'] = df['pmra'] / (1000 * 3600 * 180 / np.pi * yr_to_sec)
df['pmdec_rad_sec'] = df['pmdec'] / (1000 * 3600 * 180 / np.pi * yr_to_sec)

Calculate tangential velocities in km/s
df['vt_ra'] = df['distance'] * pc_to_km * df['pmra_rad_sec'] / au_to_km
df['vt_dec'] = df['distance'] * pc_to_km * df['pmdec_rad_sec'] / au_to_km

#Total tangential velocity
df['vt'] = np.sqrt(df['vt_ra']**2 + df['vt_dec']**2)

Plot tangential velocity distribution
plt.figure(figsize=(10, 6))
plt.hist(df['vt'], bins=50, color='mediumseagreen', edgecolor='black')
plt.xlabel('Tangential Velocity (km/s)', fontsize=18)
plt.ylabel('Number of Stars', fontsize=18)
plt.title('Distribution of Tangential Velocities', fontsize=20)
plt.tick_params(axis='both', labelsize=16)
#plt.xlim(0,100)
plt.show()

```

<Figure size 1000x600 with 1 Axes>

This gives you a sense of the distribution of stellar speeds. Very high tangential velocities can indicate halo stars or other populations with different kinematics.

### 17.0.6 Step 6: Using Seaborn for a Scatter Plot

Seaborn can make more visually appealing plots with less code. Let's recreate the RA vs. Dec scatter plot using Seaborn.

```

plt.figure(figsize=(10, 8))
sns.scatterplot(x='ra', y='dec', data=df, s=5, alpha=0.5)
plt.xlabel('Right Ascension (degrees)', fontsize=18)
plt.ylabel('Declination (degrees)', fontsize=18)
plt.title('Star Positions from Gaia (Seaborn)', fontsize=20)
plt.tick_params(axis='both', labelsize=16)
plt.gca().invert_xaxis()
plt.show()

```

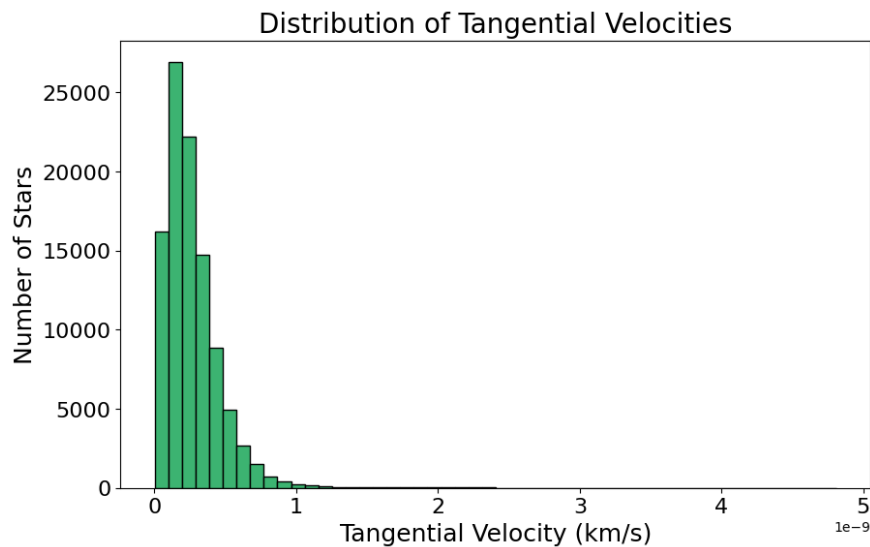


Figure 17.4: output

<Figure size 1000x800 with 1 Axes>

### 17.0.7 Step 7: Seaborn with Magnitude as Hue

We can use the `hue` parameter in Seaborn to color the points by magnitude, similar to what we did with Matplotlib.

```
plt.figure(figsize=(12, 10))
sns.scatterplot(x='ra', y='dec', hue='phot_g_mean_mag', data=df, s=10, alpha=0.5, palette='viridi')
plt.xlabel('Right Ascension (degrees)', fontsize=18)
plt.ylabel('Declination (degrees)', fontsize=18)
plt.title('Star Positions Colored by Magnitude (Seaborn)', fontsize=20)
plt.tick_params(axis='both', labelsize=16)
plt.gca().invert_xaxis()
plt.show()
```

<Figure size 1200x1000 with 1 Axes>

This plot shows arrows indicating the direction and magnitude of the proper motions of the stars.

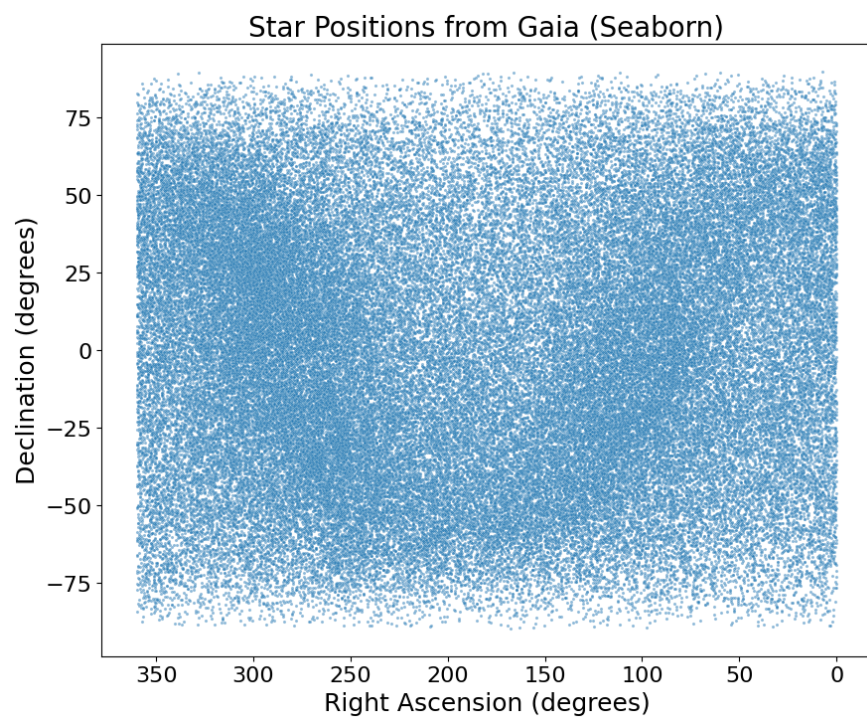


Figure 17.5: output



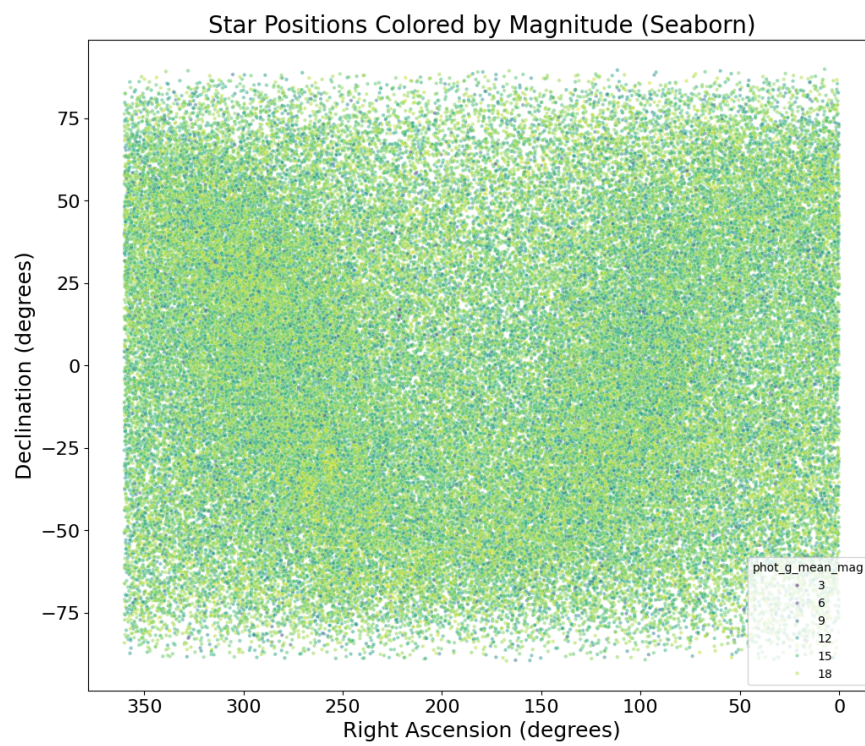


Figure 17.6: output

## 17.1 Hertzsprung-Russell (HR) Diagram

The HR diagram is a fundamental tool in astronomy. We'll approximate it by plotting absolute magnitude vs. color. First, we need to calculate absolute magnitude. This requires the parallax.

```
Remove stars with parallax <= 0
df = df[df['parallax'] > 0]

Calculate absolute magnitude
df['abs_mag'] = df['phot_g_mean_mag'] + 5 * np.log10(df['parallax']/100)

Create a color index (approximation, needs better photometry for real science)
df['color_index'] = df['bp_rp']

Scatter plot of HR diagram
plt.figure(figsize=(10, 8))
plt.scatter(df['color_index'], df['abs_mag'], s=1, alpha=0.5)
plt.xlabel(r'Color Index, $G_{BP} - G_{RP}$ (mag)', fontsize=18)
plt.ylabel(r'Absolute Magnitude, M_G ', fontsize=18)
plt.title('Hertzsprung-Russell Diagram (Approximate)', fontsize=20)
plt.tick_params(axis='both', labelsize=16)
plt.ylim(20, -5) # Invert y axis to match convention
plt.show()
```

<Figure size 1000x800 with 1 Axes>

A proper HR diagram needs a good color index. With better photometric data (like BP and RP magnitudes from Gaia), you could compute BP-RP and use that as the color. Absolute magnitude is calculated using the distance modulus formula, which relies on the parallax.

### 17.1.1 Conclusion

This tutorial provides a basic introduction to visualising astronomical data using Matplotlib and Seaborn with real Gaia data. Remember to explore the documentation for both libraries to learn more about their capabilities and customisation options.

### 17.1.2 Exercises

#### 1. Star Position Analysis

Create a scatter plot of Right Ascension (RA) vs. Declination (Dec) for



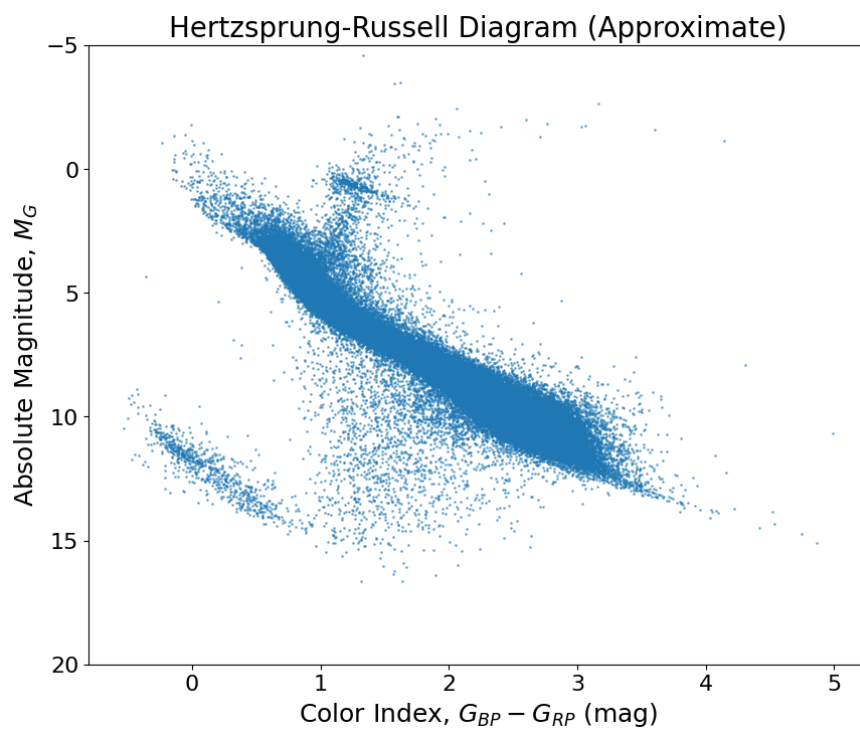


Figure 17.7: output

all stars, colored by their `phot_g_mean_mag`. Add a colorbar and ensure the RA axis is inverted.

*Deliverable:* Plot + 1-sentence explanation of why brighter stars (lower magnitudes) might cluster in certain regions.

## 2. Magnitude Distribution

Plot a histogram of `phot_g_mean_mag` with 20 bins.

*Deliverable:* Plot + Answer: “How does the distribution reflect observational limitations of telescopes?”

## 3. Tangential Velocity

Calculate tangential velocities using:

$$v_t \text{ (km/s)} = 4.74 \times \text{distance (pc)} \times \sqrt{\mu_\alpha^2 + \mu_\delta^2}$$

where

$$\mu_\alpha = \text{pmra}/1000$$

and

$$\mu_\delta = \text{pmdec}/1000$$

(mas/yr to arcsec/yr).

*Deliverable:* Code snippet + Histogram of calculated velocities.

## 4. Hertzsprung-Russell Diagram

Plot `phot_g_mean_mag` vs. `parallax`.

*Deliverable:* Plot + Identify which quadrant likely contains giant stars and why.

## 5. Proper Motion Analysis

Create a vector field plot for 200 randomly selected stars, scaling `pmra` and `pmdec` by 20x for visibility.

*Deliverable:* Plot + Note any patterns in motion directions.

## Chapter 18

# Essential Statistics Tutorial

### 18.1 1. Required Libraries

#### 18.1.1 1.1 Install Required Packages (if you haven't already):

#### 18.1.2 pip install numpy pandas matplotlib seaborn scipy jupyter

**Note** If you are running this jupyter notebook from Colab, then run the next cell by pressing SHIFT+ENTER to install the required packages for this notebook. Otherwise, skip the next cell.

```
!pip install numpy pandas matplotlib seaborn scipy
```

#### 18.1.3 1.2 Import the required libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

Set style for plots
sns.set_style('whitegrid')
%matplotlib inline
```

## 18.2 2. Descriptive Statistics

```
Create a sample dataset
data = [23, 45, 56, 78, 23, 45, 67, 89, 12, 34, 56, 78, 90, 12, 34, 56, 78]

Basic descriptive statistics
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Mode:", stats.mode(data))
print("Standard Deviation:", np.std(data))
print("Variance:", np.var(data))
print("Minimum:", np.min(data))
print("Maximum:", np.max(data))
print("25th Percentile:", np.percentile(data, 25))
print("75th Percentile:", np.percentile(data, 75))
```

```
Mean: 51.529411764705884
Median: 56.0
Mode: ModeResult(mode=np.int64(56), count=np.int64(3))
Standard Deviation: 25.132864931280302
Variance: 631.6608996539792
Minimum: 12
Maximum: 90
25th Percentile: 34.0
75th Percentile: 78.0
```

## 18.3 3. Data Visualization

```
Histogram
plt.figure(figsize=(8, 5))
plt.hist(data, bins=5, color='skyblue', edgecolor='black')
plt.title('Histogram of Sample Data')
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.show()
```

```
<Figure size 800x500 with 1 Axes>
```

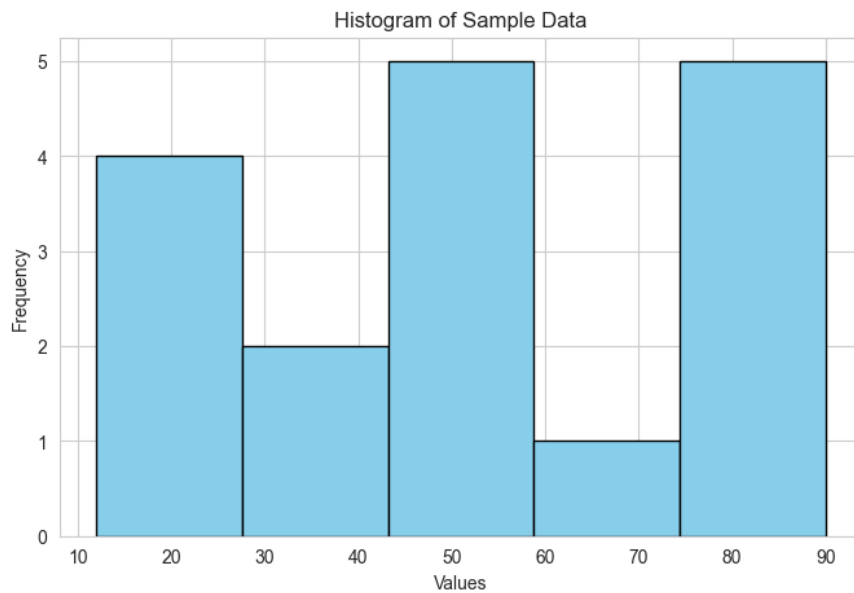


Figure 18.1: output

```
Box plot
plt.figure(figsize=(8, 5))
sns.boxplot(data)
plt.title('Box Plot of Sample Data')
plt.show()
```

<Figure size 800x500 with 1 Axes>

## 18.4 4. Probability Basics

```
Probability of a fair coin toss
outcomes = ['Heads', 'Tails']
probabilities = [0.5, 0.5]

plt.figure(figsize=(6, 4))
plt.bar(outcomes, probabilities, color=['blue', 'orange'])
plt.title('Probability Distribution of a Fair Coin')
plt.ylabel('Probability')
```

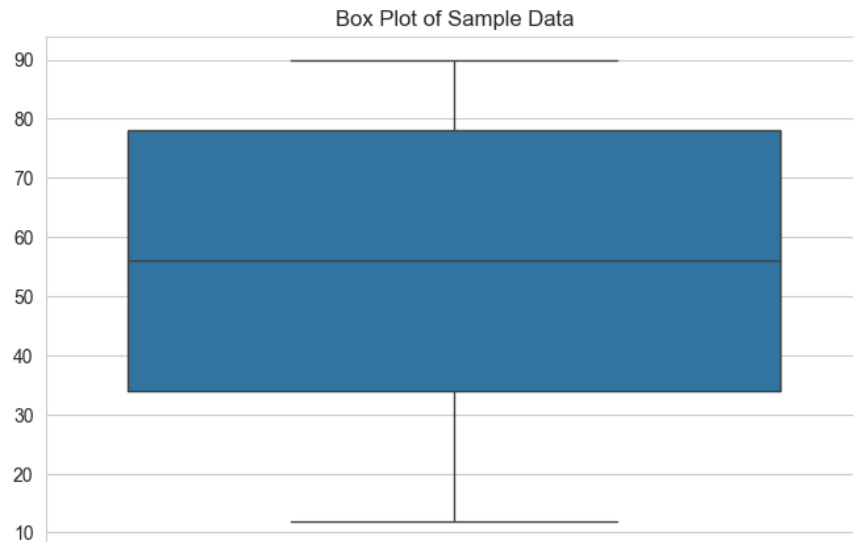


Figure 18.2: output

```
plt.ylim(0, 1)
plt.show()
```

<Figure size 600x400 with 1 Axes>

## 18.5 5. Normal Distribution

```
Generate normal distribution data
mu, sigma = 0, 1 # mean and standard deviation
normal_data = np.random.normal(mu, sigma, 1000)

Plot
plt.figure(figsize=(8, 5))
sns.histplot(normal_data, kde=True, color='green')
plt.title('Normal Distribution (=0, =1)')
plt.xlabel('Value')
plt.ylabel('Density')
plt.show()
```

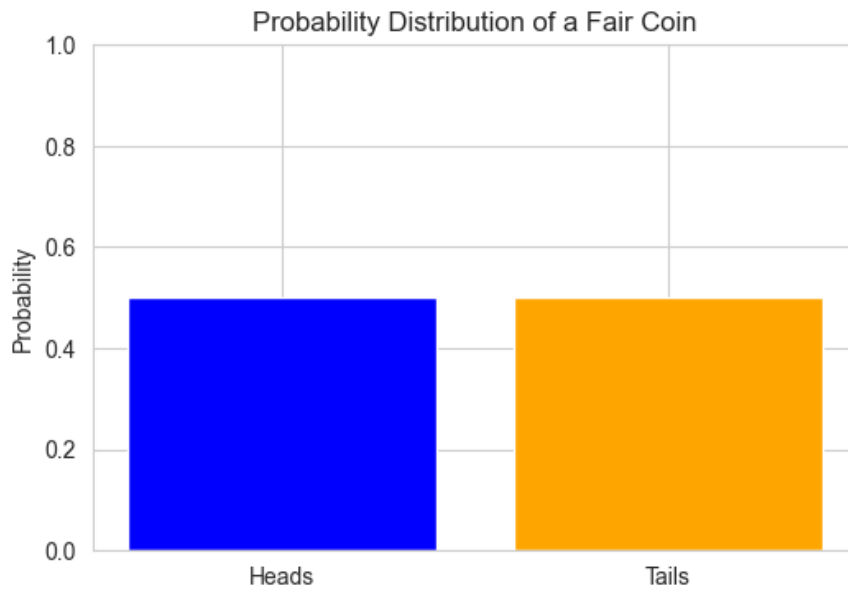


Figure 18.3: output

<Figure size 800x500 with 1 Axes>

## 18.6 6. Correlation

```
Create correlated data
np.random.seed(42)
x = np.random.rand(50)
y = x * 2 + np.random.normal(0, 0.1, 50)

Calculate correlation coefficient
correlation = np.corrcoef(x, y)[0, 1]
print(f"Pearson Correlation Coefficient: {correlation:.2f}")

Scatter plot
plt.figure(figsize=(8, 5))
plt.scatter(x, y, color='purple')
plt.title(f'Scatter Plot (Correlation: {correlation:.2f})')
plt.xlabel('X')
```

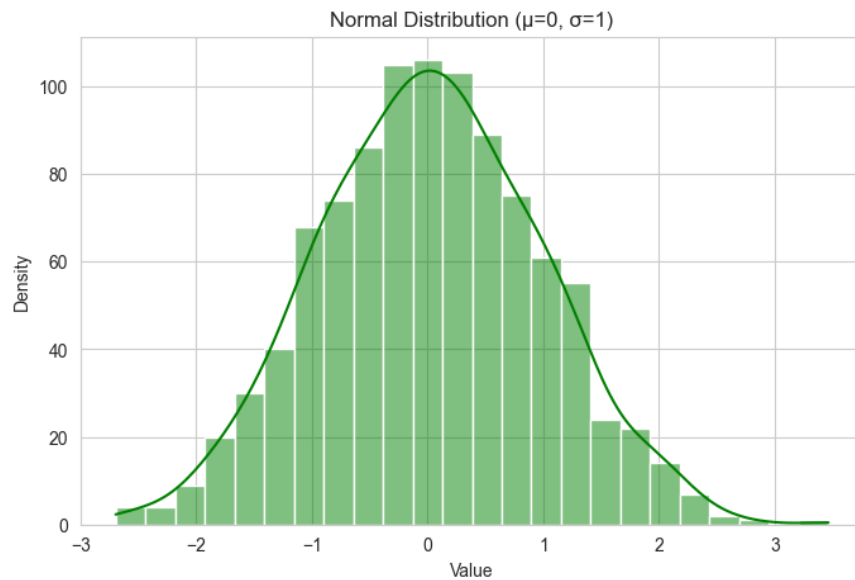


Figure 18.4: output

```
plt.ylabel('Y')
plt.show()
```

Pearson Correlation Coefficient: 0.99

<Figure size 800x500 with 1 Axes>

## 18.7 7. Hypothesis Testing (t-test)

```
Create two sample groups
group1 = np.random.normal(5, 1, 30)
group2 = np.random.normal(6, 1, 30)

Perform independent t-test
t_stat, p_value = stats.ttest_ind(group1, group2)

print(f"T-statistic: {t_stat:.2f}")
print(f"P-value: {p_value:.4f}")
```



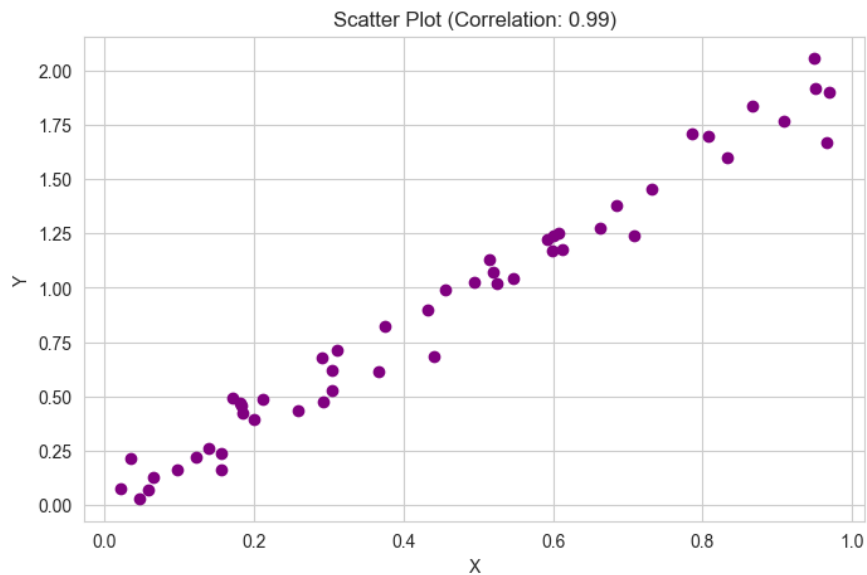


Figure 18.5: output

```
Interpretation
alpha = 0.05
if p_value < alpha:
 print("Reject the null hypothesis (significant difference between groups)")
else:
 print("Fail to reject the null hypothesis (no significant difference)")
```

T-statistic: -3.70

P-value: 0.0005

Reject the null hypothesis (significant difference between groups)

## 18.8 8. Confidence Intervals

```
Calculate 95% confidence interval for the mean
confidence_level = 0.95
degrees_freedom = len(data) - 1
sample_mean = np.mean(data)
sample_standard_error = stats.sem(data)
```

```
confidence_interval = stats.t.interval(confidence_level, degrees_freedom,
 sample_mean, sample_standard_error)

print(f"Mean: {sample_mean:.2f}")
print(f"95% Confidence Interval: [{confidence_interval[0]:.2f}, {confidence_interval[1]:.2f}]")
```

```
Mean: 51.53
95% Confidence Interval: [38.21, 64.85]
```

## 18.9 9. Simple Linear Regression

```
Create data
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
y = np.array([2, 4, 5, 7, 8, 10, 11, 14, 15, 17])

Perform linear regression
slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

print(f"Slope: {slope:.2f}")
print(f"Intercept: {intercept:.2f}")
print(f"R-squared: {r_value**2:.2f}")

Plot regression line
plt.figure(figsize=(8, 5))
plt.scatter(x, y, color='red', label='Data points')
plt.plot(x, intercept + slope*x, 'b-', label='Regression line')
plt.title('Simple Linear Regression')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()
```

```
Slope: 1.64
Intercept: 0.27
R-squared: 0.99
```

```
<Figure size 800x500 with 1 Axes>
```

## 18.10 10. Practice Exercise

Try analyzing this sample dataset:

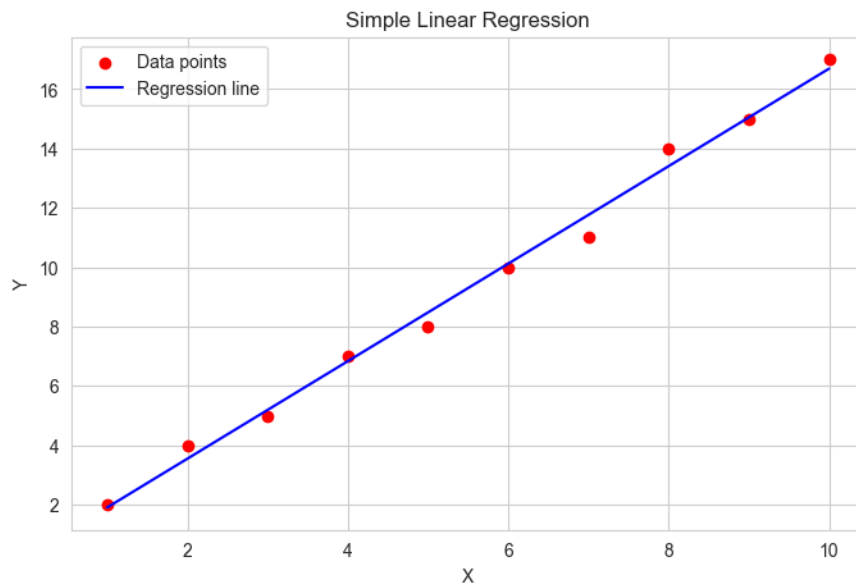


Figure 18.6: output

```
Sample dataset
exam_scores = {
 'Students': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi'],
 'Math': [85, 72, 90, 68, 88, 75, 92, 80],
 'Science': [78, 85, 80, 72, 90, 82, 88, 75],
 'Hours_Studied': [10, 8, 12, 6, 11, 9, 13, 10]
}

df = pd.DataFrame(exam_scores)
df
```

	Students	Math	Science	Hours_Studied
0	Alice	85	78	10
1	Bob	72	85	8
2	Charlie	90	80	12
3	David	68	72	6
4	Eve	88	90	11
5	Frank	75	82	9
6	Grace	92	88	13
7	Heidi	80	75	10

**Tasks:** 1. Calculate descriptive statistics for each subject 2. Visualize the

distribution of Math scores 3. Check if there's a correlation between study hours and Math scores 4. Perform a t-test to compare Math and Science scores 5. Create a linear regression model predicting Math scores from study hours

## Chapter 19

# Statistics for Astronomers

```
!pip install numpy pandas matplotlib seaborn scipy emcee corner lifelines
```

### 19.1 1. Introduction

Astronomy is fundamentally a statistical science - we often deal with: - Incomplete data (censoring, truncation) - Measurement errors - Low signal-to-noise observations - High-dimensional datasets - Complex underlying models

This tutorial will focus on practical implementation of statistical methods relevant to modern astronomy research.

```
Import standard libraries
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
import astropy.stats as astrostats
from astropy.modeling import models, fitting
import emcee
import corner
import pandas as pd

%matplotlib inline
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12
```

## 19.2 2. Probability Distributions in Astronomy

Common distributions in astronomical research:

```
Demonstrate common distributions
x = np.linspace(0, 10, 1000)

Poisson (for counting statistics)
plt.plot(x, stats.poisson.pmf(np.round(x), mu=2), label='Poisson (=2)')

Gaussian
plt.plot(x, stats.norm.pdf(x, loc=5, scale=1), label='Gaussian (=5, =1)')

Power-law
plt.plot(x, stats.powerlaw.pdf(x, 1.5, loc=0, scale=10), label='Power-law (=1.5)')

plt.xlabel('x')
plt.ylabel('PDF')
plt.legend()
plt.title('Common Probability Distributions in Astronomy')
plt.show()
```

<Figure size 1000x600 with 1 Axes>

## 19.3 3. Bayesian Inference for Astronomical Problems

Bayesian methods are particularly valuable in astronomy where we often have:

- Prior information from previous studies
- Small sample sizes
- Complex parameter spaces

### 19.3.1 3.1 MCMC Sampling with emcee

Example: Fitting a line to data with heteroscedastic errors

```
Generate synthetic data
np.random.seed(42)
true_slope = 2.5
true_intercept = -1.0
x = np.linspace(0, 10, 25)
y_err = 0.5 + 0.5 * np.random.rand(len(x))
```

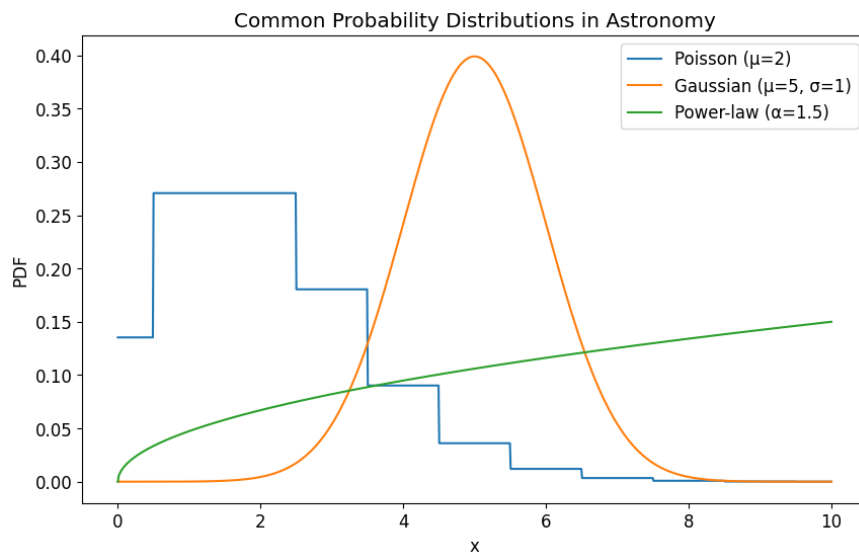


Figure 19.1: output

```

y = true_slope * x + true_intercept + y_err * np.random.randn(len(x))

plt.errorbar(x, y, yerr=y_err, fmt='o')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Synthetic Data with Errors')
plt.show()

```

<Figure size 1000x600 with 1 Axes>

```

Define log likelihood and prior
def log_likelihood(theta, x, y, yerr):
 slope, intercept = theta
 model = slope * x + intercept
 return -0.5 * np.sum((y - model) / yerr)**2)

def log_prior(theta):
 slope, intercept = theta
 if -5.0 < slope < 5.0 and -10.0 < intercept < 10.0:
 return 0.0
 return -np.inf

```

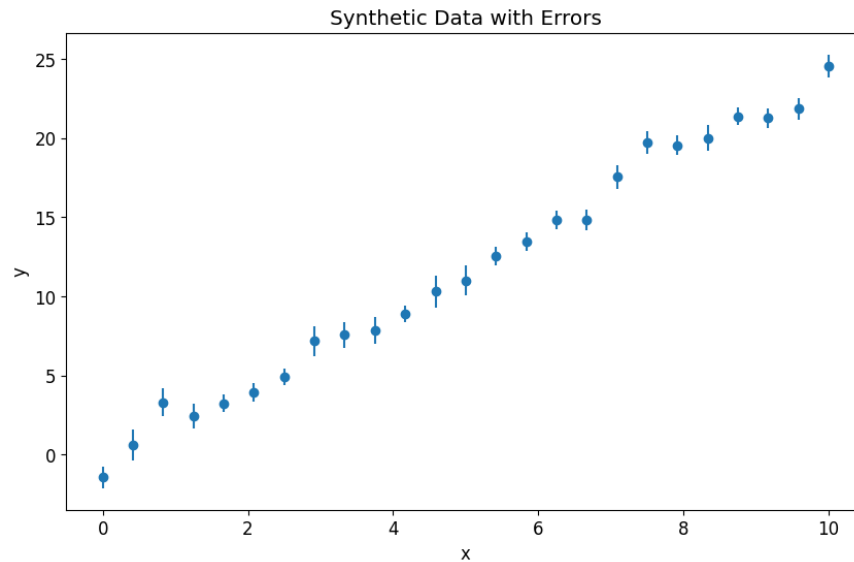


Figure 19.2: output

```
def log_probability(theta, x, y, yerr):
 lp = log_prior(theta)
 if not np.isfinite(lp):
 return -np.inf
 return lp + log_likelihood(theta, x, y, yerr)

Run MCMC
pos = np.array([true_slope, true_intercept]) + 0.1 * np.random.randn(32, 2)
nwalkers, ndim = pos.shape

sampler = emcee.EnsembleSampler(nwalkers, ndim, log_probability, args=(x, y, y_err))
sampler.run_mcmc(pos, 5000, progress=True);

100%| | 5000/5000 [00:01<00:00, 3479.70it/s]

Plot results
samples = sampler.get_chain(discard=100, thin=15, flat=True)

fig = corner.corner(samples, labels=['slope', 'intercept'],
 truths=[true_slope, true_intercept])
plt.show()
```



<Figure size 550x550 with 4 Axes>

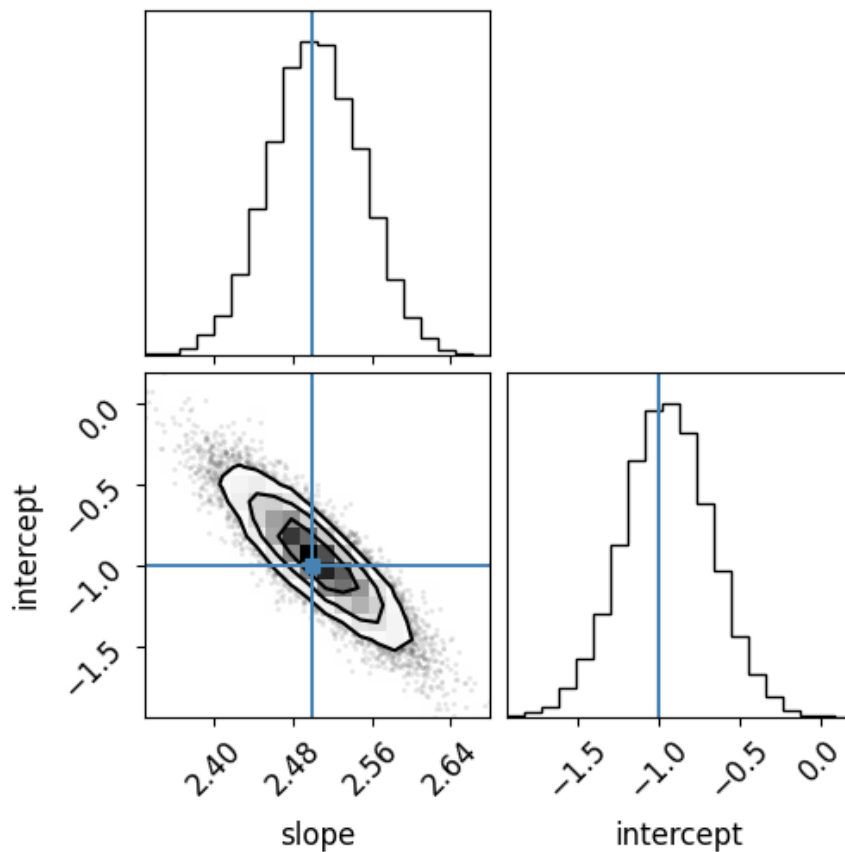


Figure 19.3: output

## 19.4 4. Survival Analysis for Censored Data

Astronomical data often contains upper limits (censored data). We'll use the `lifelines` package for survival analysis.

```
Install if needed
!pip install lifelines

from lifelines import KaplanMeierFitter
```

```

Example: Detections and upper limits in flux measurements
flux = np.concatenate([np.random.normal(5, 1, 50),
 np.random.normal(0, 0.5, 30)]) # Some "non-detections"
detected = flux > 1.0 # Simple detection threshold
upper_limits = 1.0 * np.ones(sum(~detected))

kmf = KaplanMeierFitter()
kmf.fit(flux[detected], event_observed=np.ones(sum(detected)),
 label='Detections')
kmf.fit(upper_limits, event_observed=np.zeros(sum(~detected)),
 label='Upper Limits')

kmf.plot_survival_function()
plt.xlabel('Flux')
plt.ylabel('Probability of Detection')
plt.title('Survival Analysis for Censored Astronomical Data')
plt.show()

```

<Figure size 1000x600 with 1 Axes>

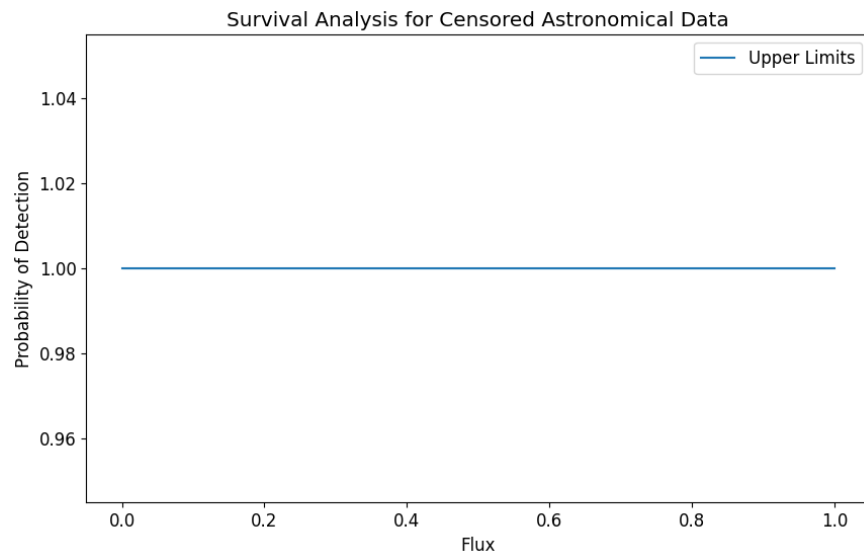


Figure 19.4: output

## 19.5 5. Time Series Analysis

Astronomical time series present unique challenges: - Irregular sampling - Multiple periodicities - Non-sinusoidal signals

```
from astropy.timeseries import LombScargle

Generate synthetic time series
t = np.sort(np.random.uniform(0, 100, 200))
y = 2.0 * np.sin(2 * np.pi * t / 10.0) + 0.5 * np.random.randn(len(t))

Compute Lomb-Scargle periodogram
frequency, power = LombScargle(t, y).autopower(minimum_frequency=0.01,
 maximum_frequency=1.0)

plt.plot(frequency, power)
plt.xlabel('Frequency (1/day)')
plt.ylabel('Power')
plt.title('Lomb-Scargle Periodogram')
plt.show()
```

<Figure size 1000x600 with 1 Axes>

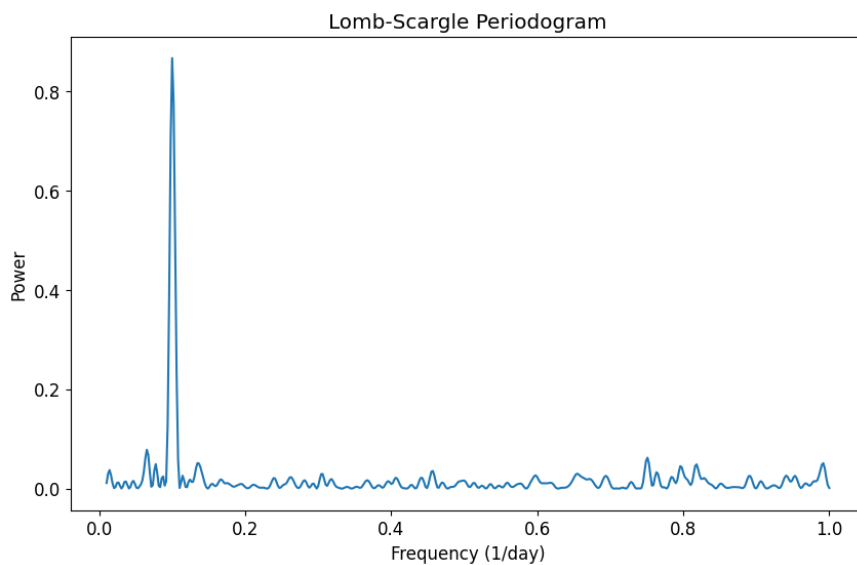


Figure 19.5: output

## 19.6 6. Multivariate Statistics and Dimensionality Reduction

Modern surveys produce high-dimensional data. We'll explore PCA and t-SNE.

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler

Example: Simulated galaxy spectra with 100 wavelength bins
np.random.seed(42)
n_objects = 200
n_features = 100

Create three classes of spectra
class1 = np.random.normal(1, 0.2, (n_objects//2, n_features))
class2 = np.random.normal(2, 0.3, (n_objects//4, n_features))
class3 = np.random.normal(3, 0.4, (n_objects//4, n_features))
X = np.vstack([class1, class2, class3])
y = np.array(['Class1']*len(class1) + ['Class2']*len(class2) + ['Class3']*len(class3))

Standardize and apply PCA
X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=pd.factorize(y)[0], alpha=0.7)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('PCA of Simulated Galaxy Spectra')
plt.colorbar(label='Class')
plt.show()
```

<Figure size 1000x600 with 2 Axes>

## 19.7 7. Machine Learning for Astronomical Classification

Brief example using random forests for classification.

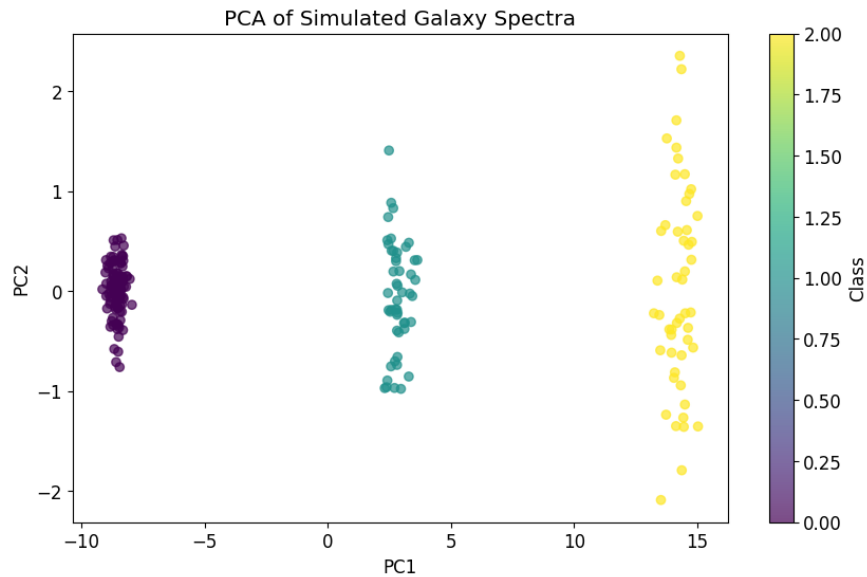


Figure 19.6: output

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

Train classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

Evaluate
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
Class1	1.00	1.00	1.00	31
Class2	1.00	1.00	1.00	15
Class3	1.00	1.00	1.00	14
accuracy			1.00	60

macro avg	1.00	1.00	1.00	60
weighted avg	1.00	1.00	1.00	60

## 19.8 8. Spatial Statistics

Analyzing the distribution of objects on the sky.

```
import numpy as np
import matplotlib.pyplot as plt
from astropy.stats import RipleysKEstimator

Generate clustered and random point patterns
np.random.seed(12345)
cluster1 = np.random.normal(loc=0, scale=0.1, size=(50, 2))
cluster2 = np.random.normal(loc=1, scale=0.1, size=(50, 2))
random_points = np.random.uniform(low=0, high=1, size=(100, 2))

Calculate K-function
Kest = RipleysKEstimator(area=1, x_max=1, y_max=1, x_min=0, y_min=0)
r = np.linspace(0, 0.5, 50)

Calculate K values separately
K_clustered = Kest(data=np.vstack([cluster1, cluster2]), radii=r)
K_random = Kest(data=random_points, radii=r)

Plot results
plt.plot(r, Kest.poisson(r), 'k-', label='Poisson')
plt.plot(r, K_clustered, 'r-', label='Clustered')
plt.plot(r, K_random, 'b-', label='Random')

plt.xlabel('Radius r')
plt.ylabel('K(r)')
plt.legend()
plt.title('Ripley\'s K-function Analysis')
plt.show()
```

<Figure size 1000x600 with 1 Axes>

## 19.9 9. Resources for Further Learning

- Feigelson & Babu: “Modern Statistical Methods for Astronomy”

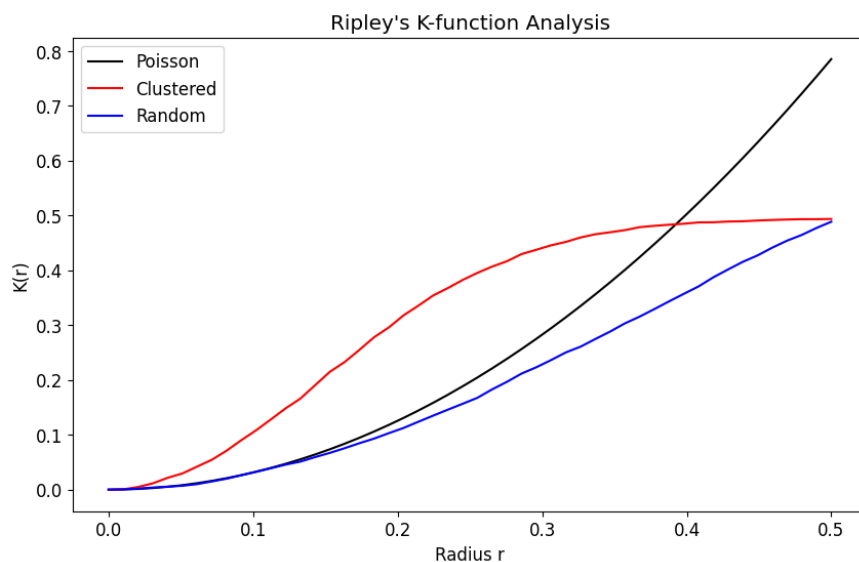


Figure 19.7: output

- Ivezić et al.: “Statistics, Data Mining, and Machine Learning in Astronomy”
- VanderPlas: “Python Data Science Handbook” (free online)
- Astropy documentation on statistical functions
- ArXiv astro-ph.IM for latest methodological developments

## 19.10 Exercises for Practice

1. Modify the MCMC example to include intrinsic scatter in the linear model
2. Apply the survival analysis methods to real astronomical data with upper limits
3. Compare PCA and t-SNE on SDSS galaxy spectra
4. Implement a Bayesian model selection approach for comparing power-law and exponential models
5. Analyze the spatial distribution of galaxies from a survey catalog





## Chapter 20

# Matplotlib is used for plotting

### 20.1 Matplotlib extra

#### ▼ Get Started



```
import matplotlib.pyplot as plt
import numpy as np
```

```
x=np.linspace(0,10,100)
```

```
y=np.exp(x)
```

### 20.2 The first plot

```
plt.plot(x,y)
```

```
[<matplotlib.lines.Line2D at 0x11ecce630>]
```

```
<Figure size 432x288 with 1 Axes>
```

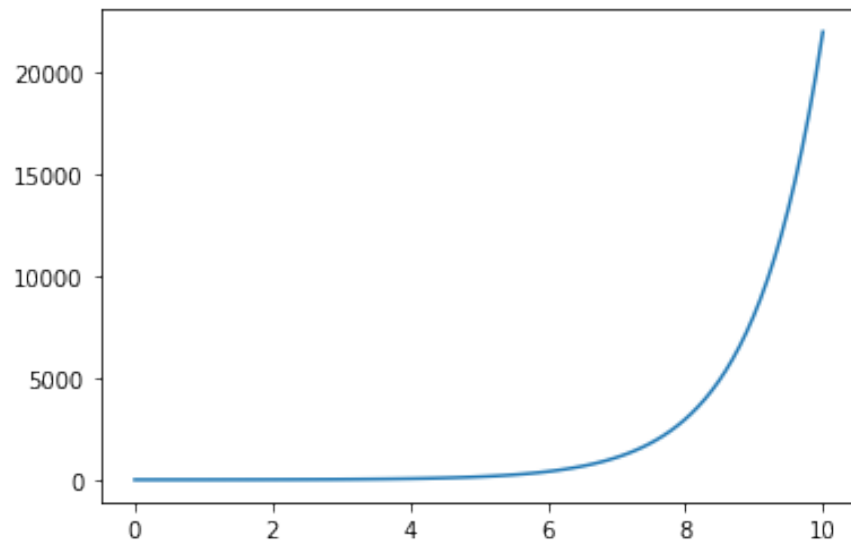


Figure 20.1: output

### 20.3 Exercise 1: Use the numpy cheat sheet to calculate the value of the function $7x+2$ between 0 and 10

- Define a linspace from zero to 10 as shown above and save as x.
- Calculate  $y=7x+2$ .
- Make the plot

*#Write code here!*

```
y_1=np.cos(x)
y_2=np.sin(x)
```

```
y_1
```

```
array([1. , 0.99490282, 0.97966323, 0.95443659, 0.91948007,
 0.87515004, 0.8218984 , 0.76026803, 0.69088721, 0.61446323,
 0.53177518, 0.44366602, 0.35103397, 0.25482335, 0.15601496,
 0.0556161 , -0.04534973, -0.14585325, -0.24486989, -0.34139023,
 -0.43443032, -0.52304166, -0.60632092, -0.68341913, -0.75355031,
 -0.81599952, -0.87013012, -0.91539031, -0.95131866, -0.97754893,
 -0.9938137 , -0.99994717, -0.9958868 , -0.981674 , -0.95745366,
 -0.92347268, -0.88007748, -0.82771044, -0.76690542, -0.69828229,
 -0.6225406 , -0.54045251, -0.45285485, -0.36064061, -0.26474988,
 -0.16616018, -0.06587659, 0.03507857, 0.13567613, 0.23489055,
 0.33171042, 0.4251487 , 0.51425287, 0.59811455, 0.67587883,
 0.74675295, 0.8100144 , 0.86501827, 0.91120382, 0.94810022,
 0.97533134, 0.99261957, 0.99978867, 0.99676556, 0.98358105,
 0.96036956, 0.9273677 , 0.88491192, 0.83343502, 0.77346177,
 0.70560358, 0.63055219, 0.54907273, 0.46199582, 0.37020915,
 0.27464844, 0.17628785, 0.07613012, -0.0248037 , -0.12548467,
 -0.2248864 , -0.32199555, -0.41582217, -0.50540974, -0.58984498,
 -0.66826712, -0.7398767 , -0.8039437 , -0.859815 , -0.90692104,
 -0.94478159, -0.97301068, -0.99132055, -0.99952453, -0.99753899,
 -0.98538417, -0.96318398, -0.93116473, -0.88965286, -0.83907153])
```

## 20.4 You can plot two plots in the same graph by calling the plot function twice

```
plt.plot(x,y_1)
plt.plot(x,y_2)
```

```
[<matplotlib.lines.Line2D at 0x122648dd8>]
```

```
<Figure size 432x288 with 1 Axes>
```

```
plt.style.use('ggplot')
```

```
plt.plot(x,y_1)
plt.plot(x,y_2)
```

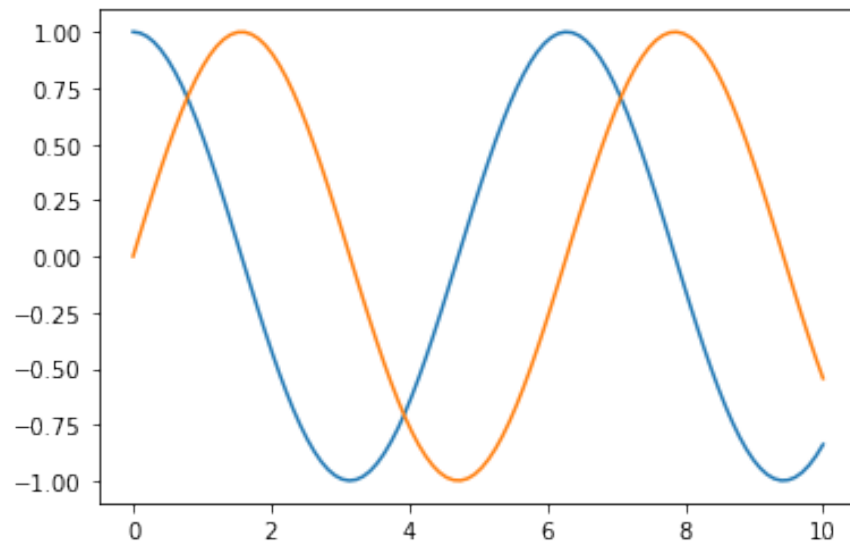


Figure 20.2: output

```
[<matplotlib.lines.Line2D at 0x122761f60>]
```

```
<Figure size 432x288 with 1 Axes>
```

```
plt.plot(x,y_1,marker='o',linestyle='None',label='Cosine')
plt.plot(x,y_2,linestyle='--',label='Sine')
plt.title('Sine and Cos')
plt.xlabel('Input (in Radians)')
plt.ylabel('Value')
plt.legend()
```

```
<matplotlib.legend.Legend at 0x122a92860>
```

```
<Figure size 432x288 with 1 Axes>
```

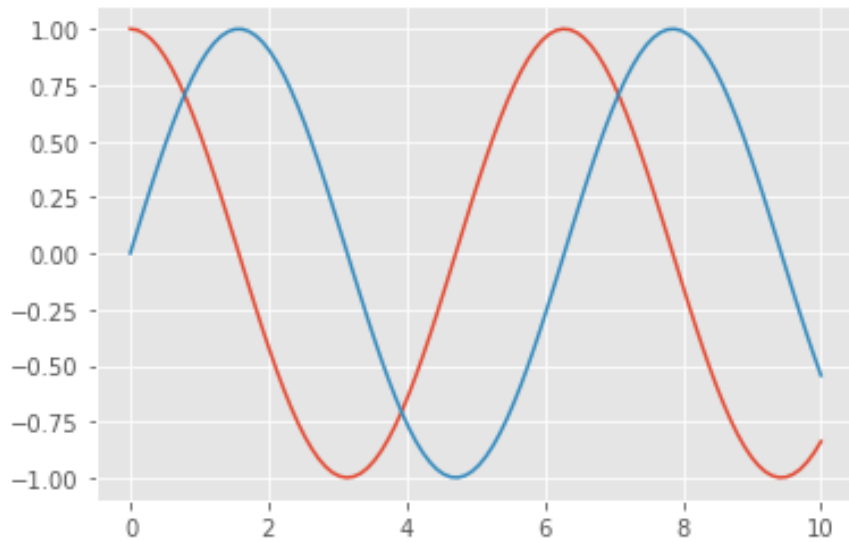


Figure 20.3: output

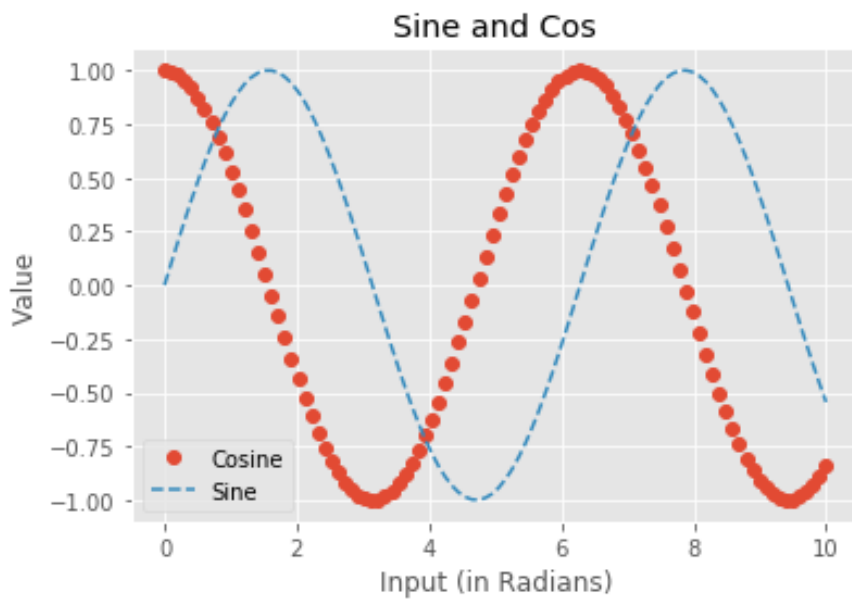


Figure 20.4: output



## Chapter 21

### Exercise 2: Create a graph for two functions

- $f_1(x)=\sin(x)$  and  $f_2(x)=2\sin(x)\cos(x)$  and plot it on the same graph.
- Make  $f_1$  dashed and make  $f_2$  thicker (using linewidth)
- Label Nicely

*#Write Code Here!*

#### 21.1 To create a new figure, use `plt.figure()`

```
plt.plot(x,y_1)
plt.figure()
plt.plot(x,y_2)
```

```
[<matplotlib.lines.Line2D at 0x122c66198>]
```

```
<Figure size 432x288 with 1 Axes>
```

```
<Figure size 432x288 with 1 Axes>
```

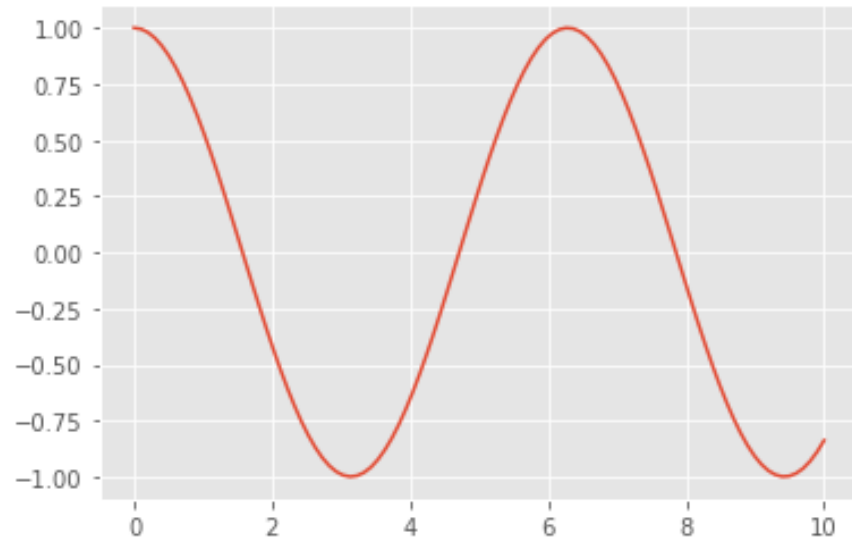


Figure 21.1: output

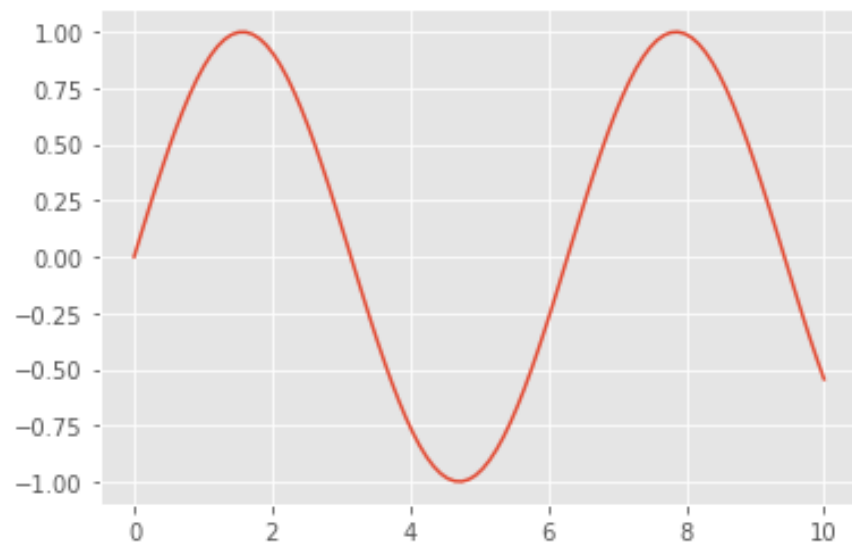


Figure 21.2: output



21.2. ANOTHER WAY TO PLOT IS TO USE THE AXES OBJECT THAT IS GOT BY THE `PLT.SUBPLOTS` FUNCTION

## 21.2 Another way to plot is to use the axes object that is got by the `plt.subplots` function

```
fig,ax=plt.subplots()
```

<Figure size 432x288 with 1 Axes>

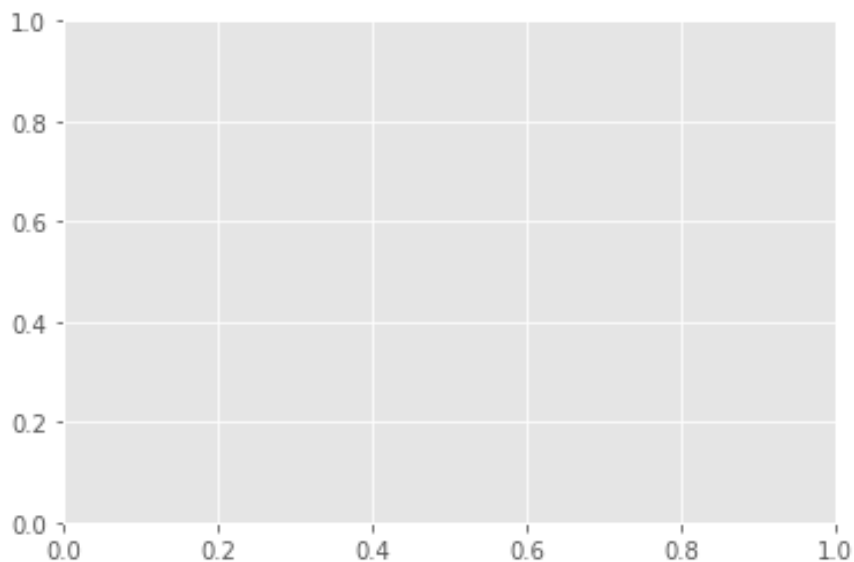


Figure 21.3: output

```
fig,ax=plt.subplots()
ax.plot(x,y_1)
ax.legend(labels = ['Cosine'], loc = 'lower right') # legend placed at lower right
ax.set_title("Cosine Function") #Notice that it is set_title and not title
ax.set_xlabel('Input') #Some input
ax.set_ylabel('Value') #Some Output
plt.show()
```

<Figure size 432x288 with 1 Axes>

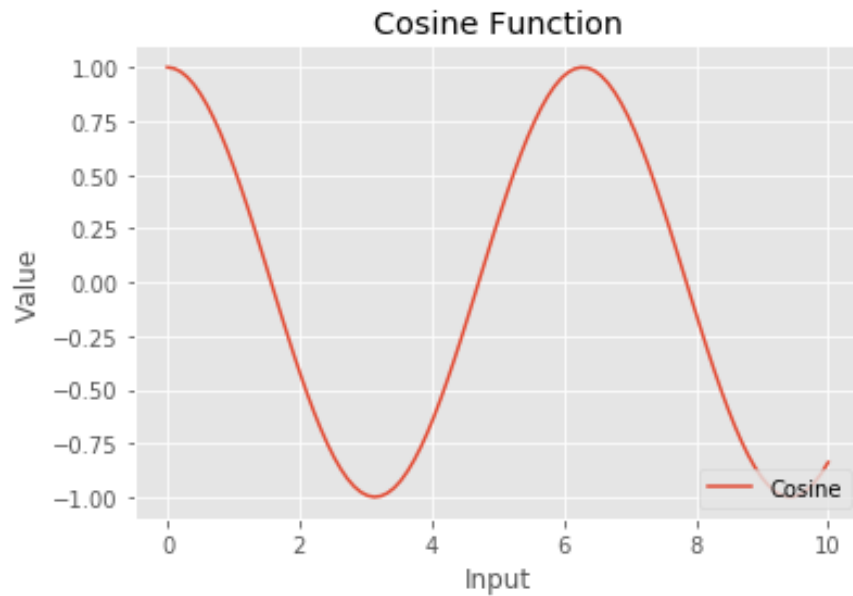


Figure 21.4: output

### 21.3 This becomes useful when you have a lot of graphs to do

```
fig, axes = plt.subplots(2, 2)
axes[0, 0].plot(x, y, label='Exponent')
axes[0, 0].legend()
axes[1, 1].plot(x, y_1, label='Sine')
axes[1, 1].legend()
axes[0, 1].plot(x, y_2, label='Cos')
axes[0, 1].legend()
axes[1, 0].plot(x, x**2, label='Squared')
axes[1, 0].legend()
```

```
<matplotlib.legend.Legend at 0x1240360f0>
```

```
<Figure size 432x288 with 4 Axes>
```

### 21.3. THIS BECOMES USEFUL WHEN YOU HAVE A LOT OF GRAPHS TO DO 231

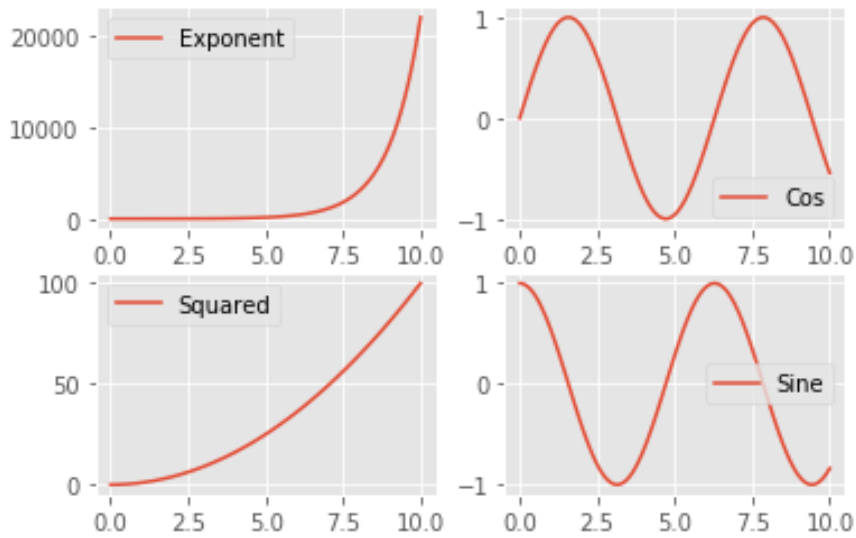


Figure 21.5: output

```
fig, axes = plt.subplots(2, 2)
axes[0, 0].plot(x, y, label='Exponent')
axes[1, 1].plot(x, y_1, label='Sine')
axes[0, 1].plot(x, y_2, label='Cos')
axes[1, 0].plot(x, x**2, label='Squared')
plt.legend() # Compare it to this and see how its different
```

<matplotlib.legend.Legend at 0x123a14588>

<Figure size 432x288 with 4 Axes>

```
#Define a 100 equally spaced points using linspace. And then another 100 random points. And then
#Add x and y together and save in z
#Plot x and z and stylize.
#Do this once using the basic interface and the second time using plt.subplots
```

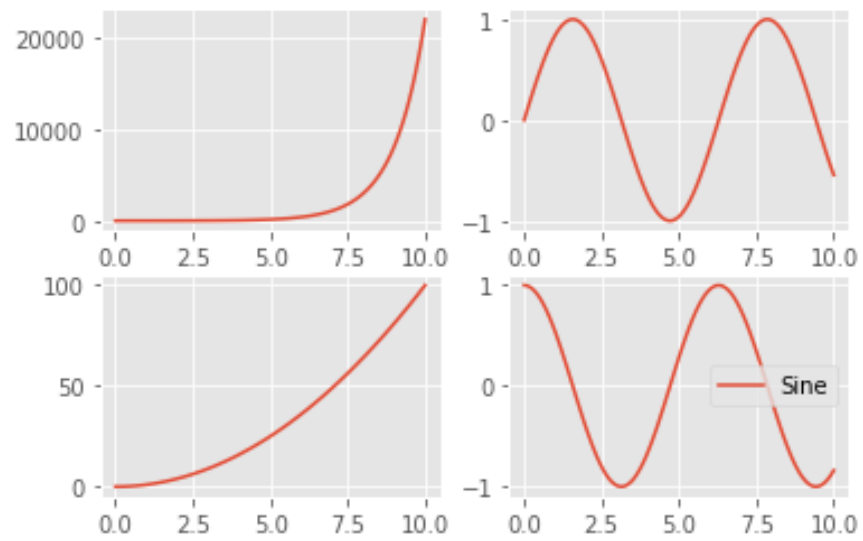


Figure 21.6: output

## Chapter 22

# Seaborn.

### 22.1 Seaborn Tutorial

#### ▼ Get Started



Seaborn is a data visualization library built on top of Matplotlib. It is often used because it makes attractive visualizations and works easily with Pandas. While in Matplotlib you often had to write multiple lines of code to create a plot Seaborn makes assumptions on what you want which often translates into getting the same plot with 1 line of code.

You can install it using the Anaconda Environment tab, or by executing the following in your terminal `pip install seaborn` or `conda install seaborn`.

Shift + Tab after attribute to see options

Adapted from <https://github.com/derekbanas/seaborn>

#### 22.1.1 import

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

Auto reloads notebook when changes are made
%reload_ext autoreload
%autoreload 2
```

### 22.1.2 Import Data

```
You can import custom data
cs_df = pd.read_csv('data/m3.csv')

If you are using Google Colab, uncomment the next line:
cs_df = pd.read_csv("https://raw.githubusercontent.com/Danselem/brics_astro/main/Wee

Seaborn provides built in datasets
#print(sns.get_dataset_names())

Load a built in dataset based on US State car crash percentages
#crash_df = sns.load_dataset('car_crashes')
```

### 22.1.3 Distribution Plots

#### 22.1.4 Distribution Plot

```
Provides a way to look at a univariate distribution. A
univariate distribution provides a distribution for one variable
Kernel Density Estimation with a Histogram is provided
kde=False removes the KDE
Bins define how many buckets to divide the data up into between intervals
For example put all profits between $10 and $20 in this bucket
sns.histplot(cs_df['Dec'], kde=True, bins=25)
```

```
<Axes: xlabel='Dec', ylabel='Count'>
```

```
<Figure size 640x480 with 1 Axes>
```

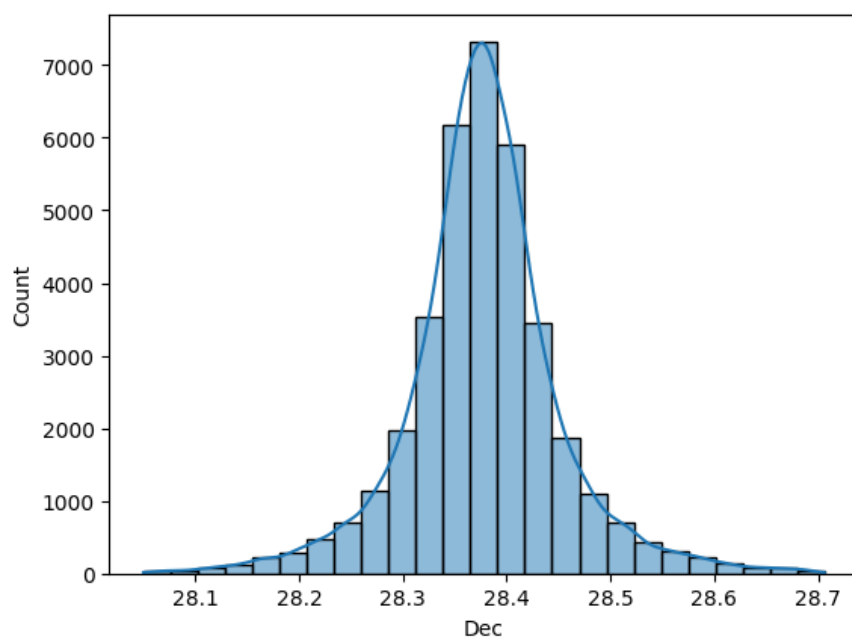


Figure 22.1: output

### 22.1.5 Joint Plot

```
Jointplot compares 2 distributions and plots a scatter plot by default
As we can see as people tend to speed they also tend to drink & drive
With kind you can create a regression line with kind='reg'
You can create a 2D KDE with kind='kde'
Kernal Density Estimation estimates the distribution of data
You can create a hexagon distribution with kind='hex'
sns.jointplot(x='RA', y='Dec', data=cs_df, kind='reg') #kde, hex
```

```
<seaborn.axisgrid.JointGrid at 0x105d53940>
```

```
<Figure size 600x600 with 3 Axes>
```

### 22.1.6 KDE Plot

```
Get just the KDE plot
sns.kdeplot(cs_df['RA'])
```

```
<Axes: xlabel='RA', ylabel='Density'>
```

```
<Figure size 640x480 with 1 Axes>
```

```
sns.kdeplot(cs_df['Dec'])
```

```
<Axes: xlabel='Dec', ylabel='Density'>
```

```
<Figure size 640x480 with 1 Axes>
```

### 22.1.7 Pair Plots

```
Pair Plot plots relationships across the entire data frames numerical values
sns.pairplot(cs_df)
```



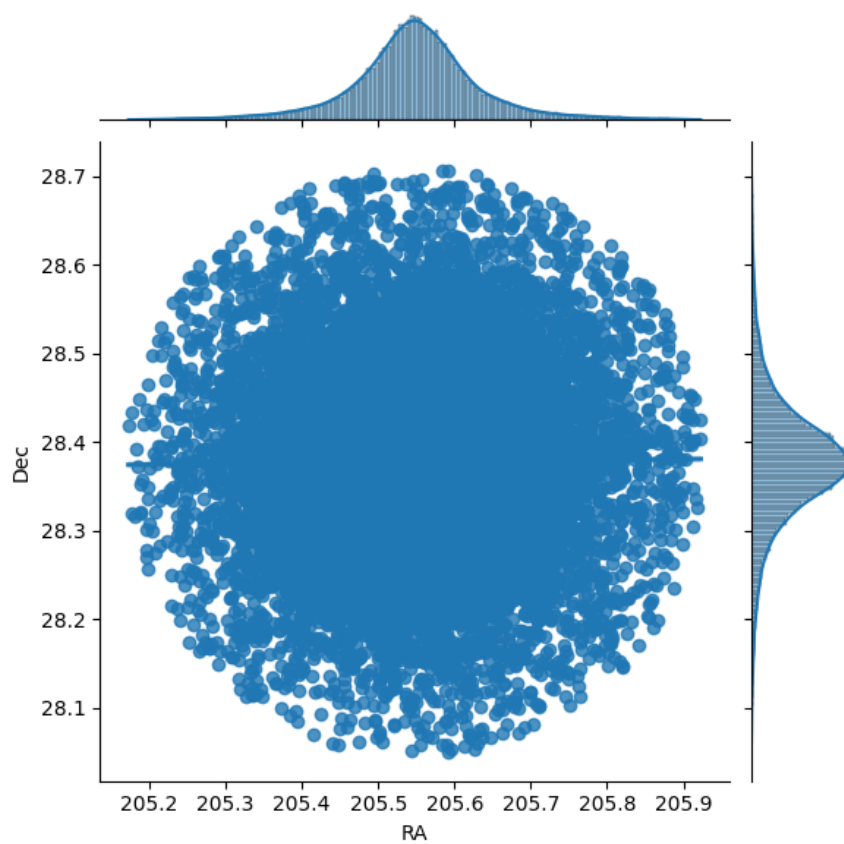


Figure 22.2: output

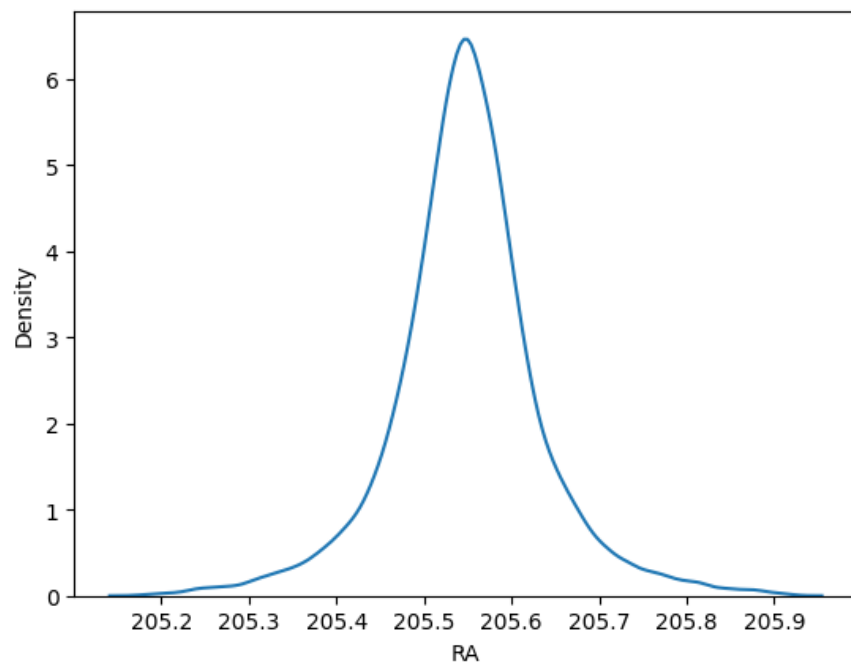


Figure 22.3: output

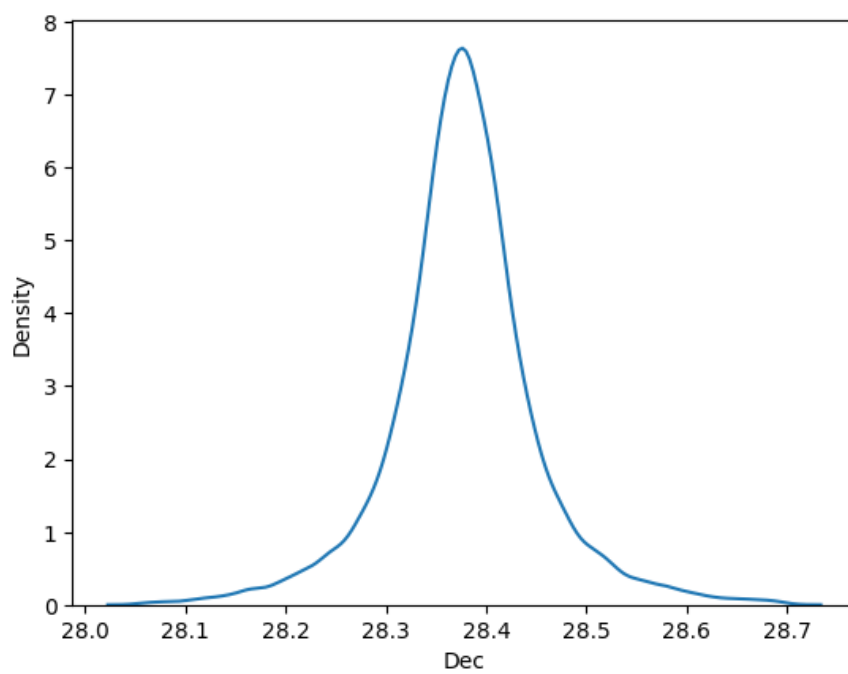


Figure 22.4: output

```
<seaborn.axisgrid.PairGrid at 0x11fc27e50>
```

```
<Figure size 1250x1250 with 30 Axes>
```

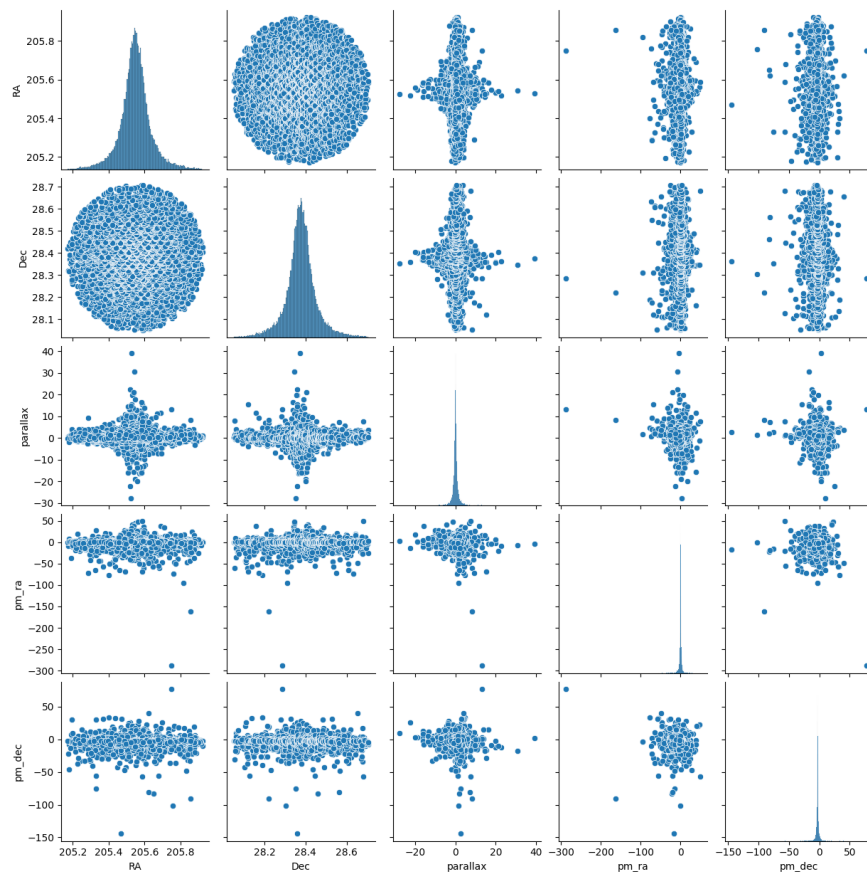


Figure 22.5: output

```
Load data on tips
tips_df = sns.load_dataset('tips')

With hue you can pass in a categorical column and the charts will be colored
You can use color maps from Matplotlib to define what colors to use
sns.pairplot(tips_df, hue='sex', palette='Blues')
```

```
tips_df
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
..	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

```
[244 rows x 7 columns]
```

```
sns.pairplot(tips_df, hue='sex', palette='Blues')
```

```
<seaborn.axisgrid.PairGrid at 0x11fc0f640>
```

```
<Figure size 850.25x750 with 12 Axes>
```

### 22.1.8 Rug Plots

```
Plots a single column of datapoints in an array as sticks on an axis
With a rug plot you'll see a more dense number of lines where the amount is
most common. This is like how a histogram is taller where values are more common
sns.rugplot(tips_df['tip'])
```

```
<Axes: xlabel='tip'>
```

```
<Figure size 640x480 with 1 Axes>
```

```
sns.rugplot(cs_df['parallax'])
```

```
<Axes: xlabel='parallax'>
```

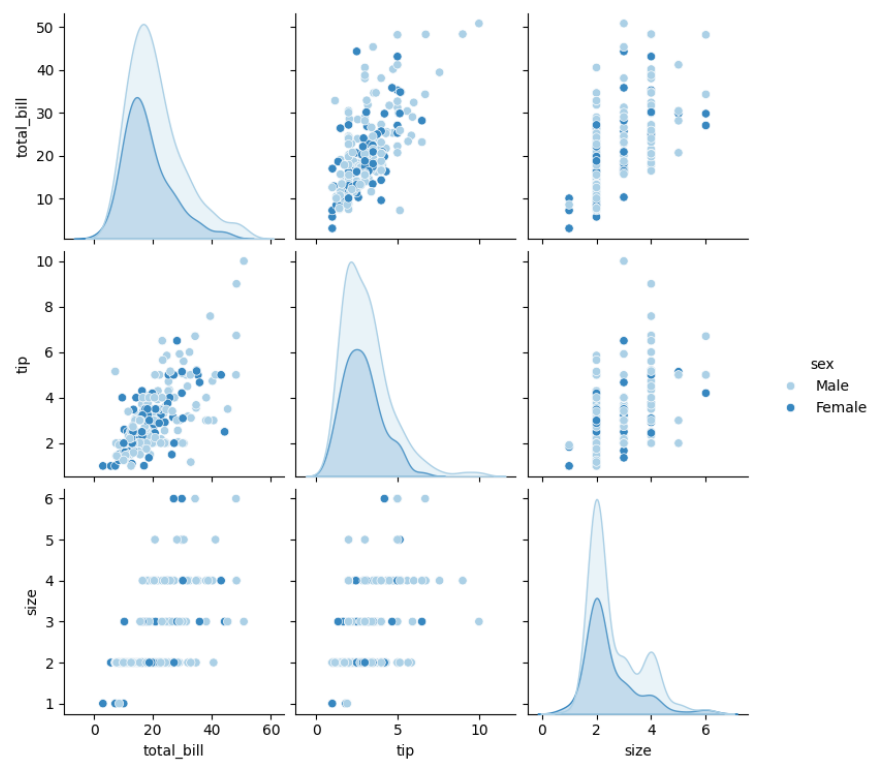


Figure 22.6: output

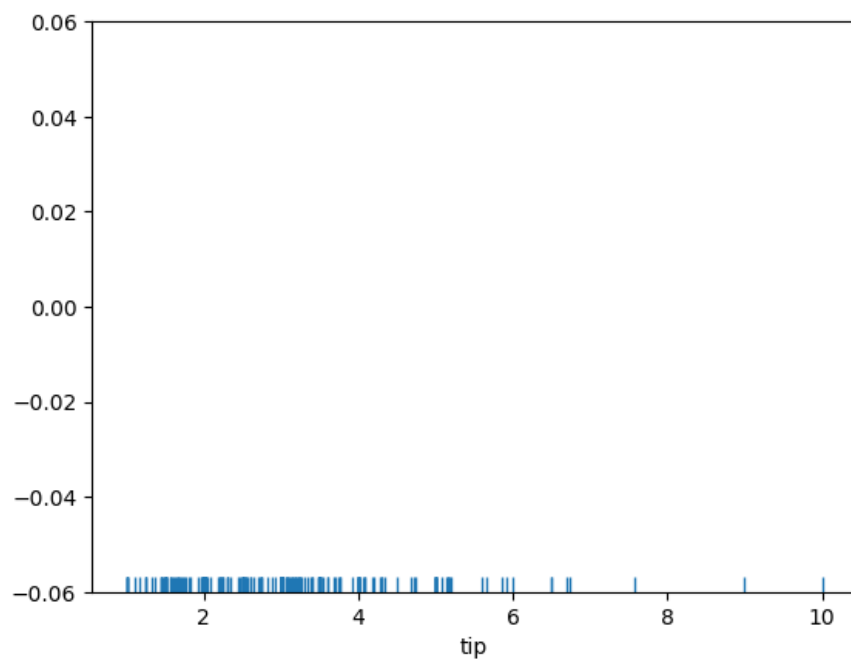


Figure 22.7: output

<Figure size 640x480 with 1 Axes>

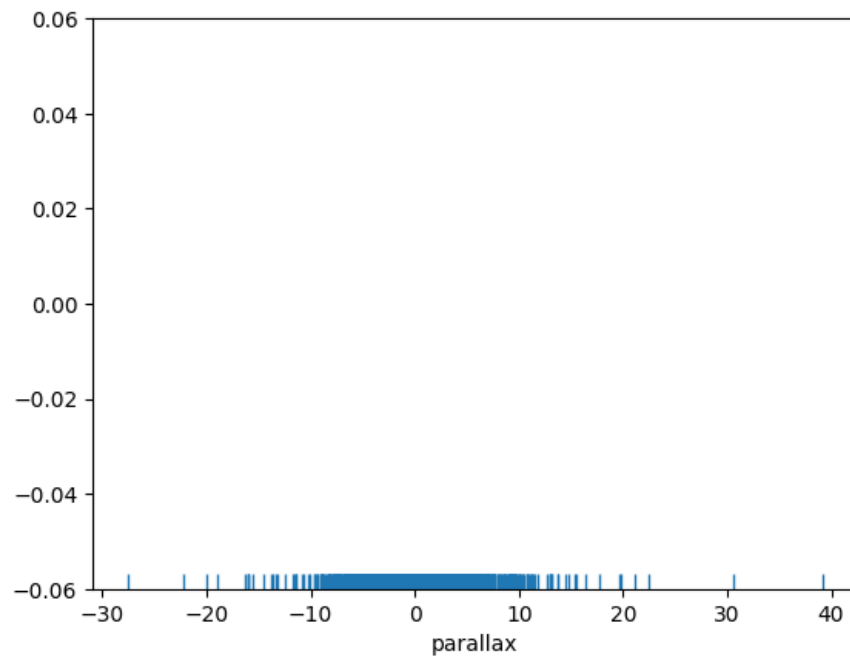


Figure 22.8: output

### 22.1.9 Styling

```
You can set styling for your axes and grids
white, darkgrid, whitegrid, dark, ticks
sns.set_style('white')

You can use figure sizing from Matplotlib
plt.figure(figsize=(8,4))

Change size of lables, lines and other elements to best fit
how you will present your data (paper, talk, poster)
sns.set_context('paper', font_scale=1.4)

sns.jointplot(x='RA', y='Dec', data=cs_df, kind='reg')
```



```
Get rid of spines
You can turn of specific spines with right=True, left=True
bottom=True, top=True
sns.despine(left=False, bottom=False)
```

<Figure size 800x400 with 0 Axes>

<Figure size 600x600 with 3 Axes>

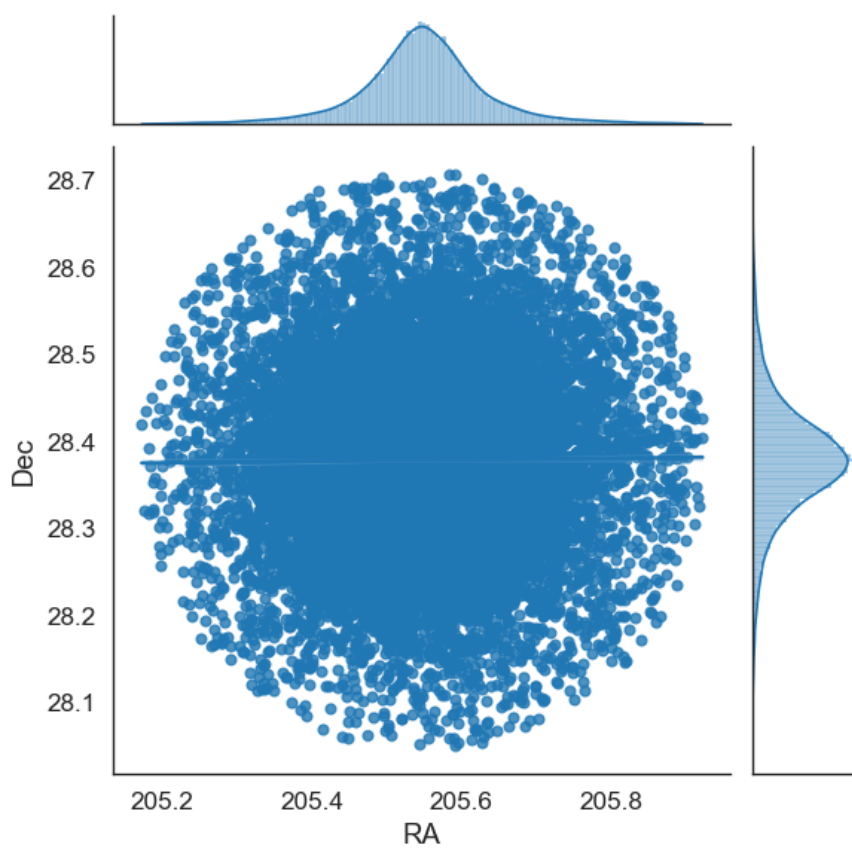


Figure 22.9: output

### 22.1.10 Categorical Plots

#### 22.1.11 Bar Plots

```
Focus on distributions using categorical data in reference to one of the numerical
columns
```

```
Aggregate categorical data based on a function (mean is the default)
Estimate total bill amount based on sex
With estimator you can define functions to use other than the mean like those
provided by NumPy : median, std, var, cov or make your own functions
#sns.barplot(x='pm_ra',y='pm_dec',data=cs_df, estimator=np.median)
```

```
sns.barplot(x='sex',y='total_bill',data=tips_df, estimator=np.median)
```

```
<Axes: xlabel='sex', ylabel='total_bill'>
```

```
<Figure size 640x480 with 1 Axes>
```

#### 22.1.12 Count Plot

```
A count plot is like a bar plot, but the estimator is counting
the number of occurrences
sns.countplot(x='parallax',data=cs_df)
```

```
<Axes: xlabel='parallax', ylabel='count'>
```

```
<Figure size 640x480 with 1 Axes>
```

```
sns.countplot(x='sex',data=tips_df)
```

```
<Axes: xlabel='sex', ylabel='count'>
```

```
<Figure size 640x480 with 1 Axes>
```

#### 22.1.13 Box Plot

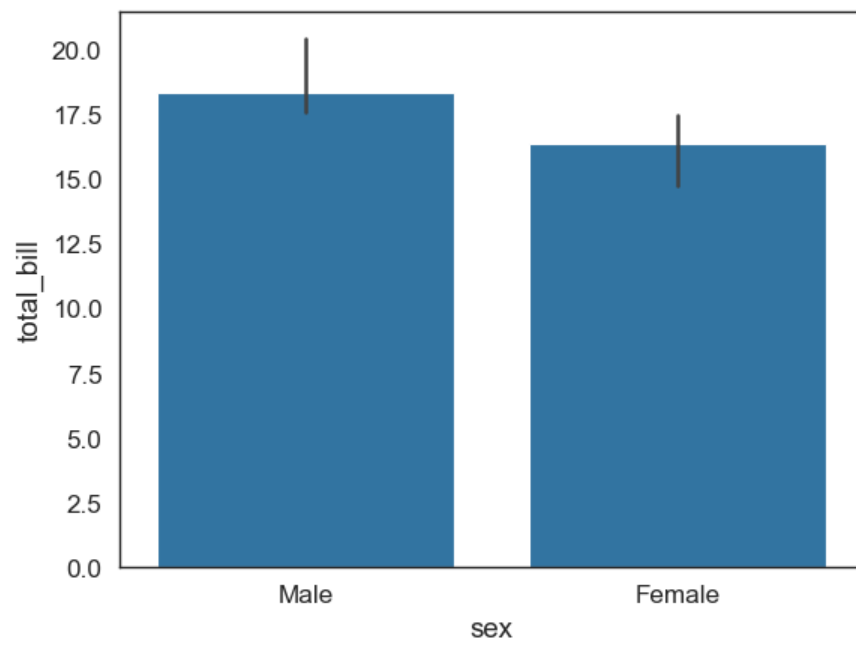


Figure 22.10: output

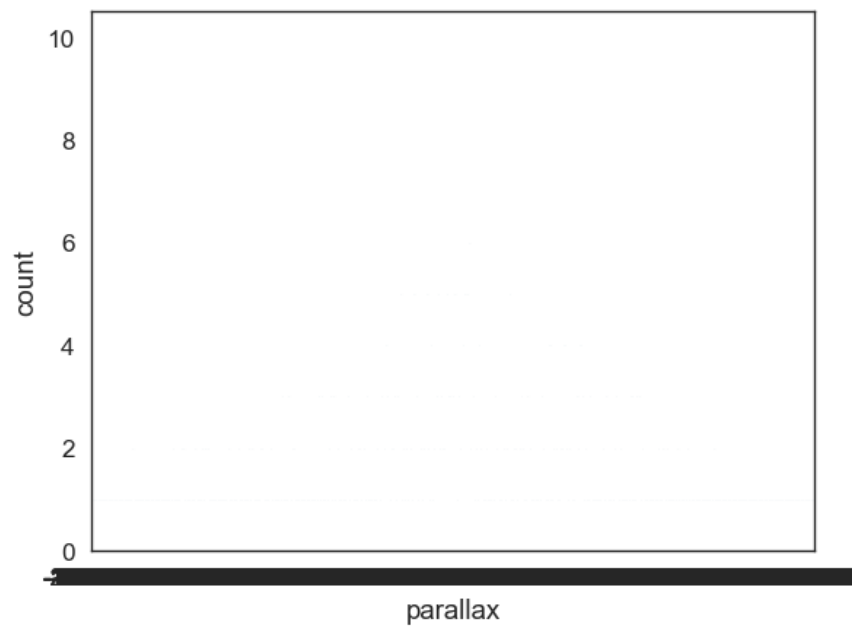


Figure 22.11: output

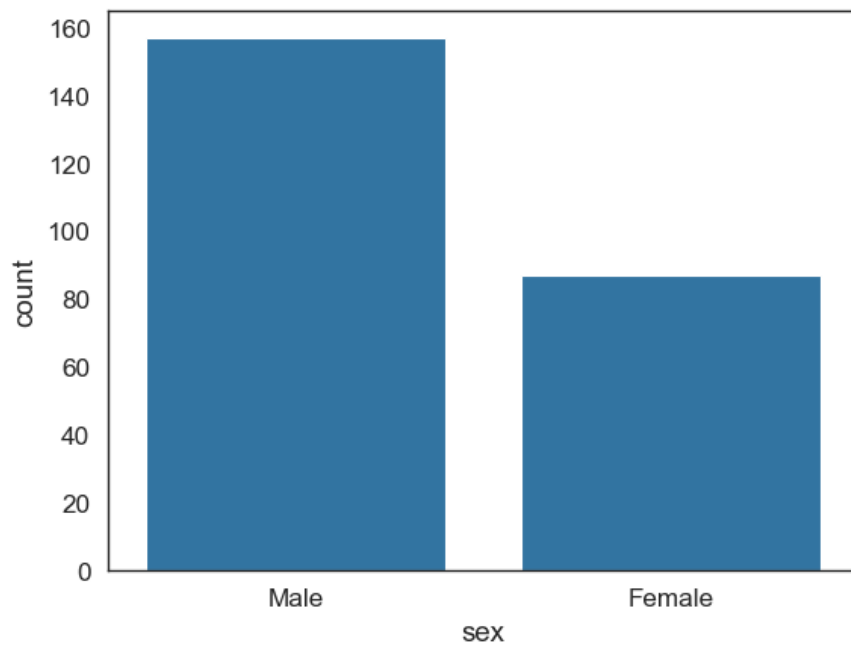


Figure 22.12: output

```
plt.figure(figsize=(14,9))
sns.set_style('darkgrid')

A box plot allows you to compare different variables
The box shows the quartiles of the data. The bar in the middle is the median and
the box extends 1 standard deviation from the median
The whiskers extend to all the other data aside from the points that are considered
to be outliers
Hue can add another category being sex
We see men spend way more on Friday versus less than women on Saturday
sns.boxplot(x='day',y='total_bill',data=tips_df, hue='sex')

Moves legend to the best position
plt.legend(loc=0)
```

```
<matplotlib.legend.Legend at 0x30ed5d630>
```

```
<Figure size 1400x900 with 1 Axes>
```

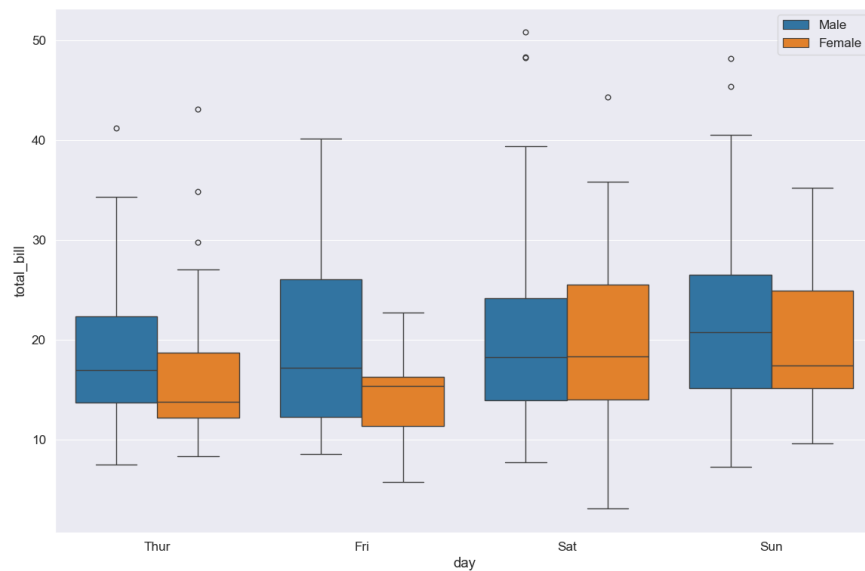


Figure 22.13: output

```
sns.boxplot(x='parallax',y='pm_ra',data=cs_df)
```

```
<Axes: xlabel='parallax', ylabel='pm_ra'>
```

```
<Figure size 640x480 with 1 Axes>
```

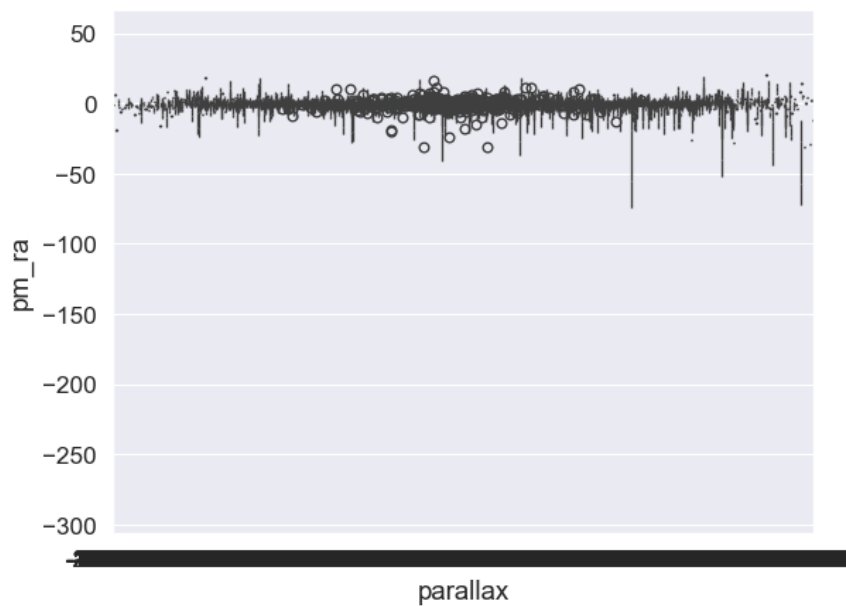


Figure 22.14: output

### 22.1.14 Violin Plot

```
Violin Plot is a combination of the boxplot and KDE
While a box plot corresponds to data points, the violin plot uses the KDE estimation
of the data points
Split allows you to compare how the categories compare to each other
sns.violinplot(x='day',y='total_bill',data=tips_df, hue='sex',split=True)
```

```
<Axes: xlabel='day', ylabel='total_bill'>
```

<Figure size 640x480 with 1 Axes>

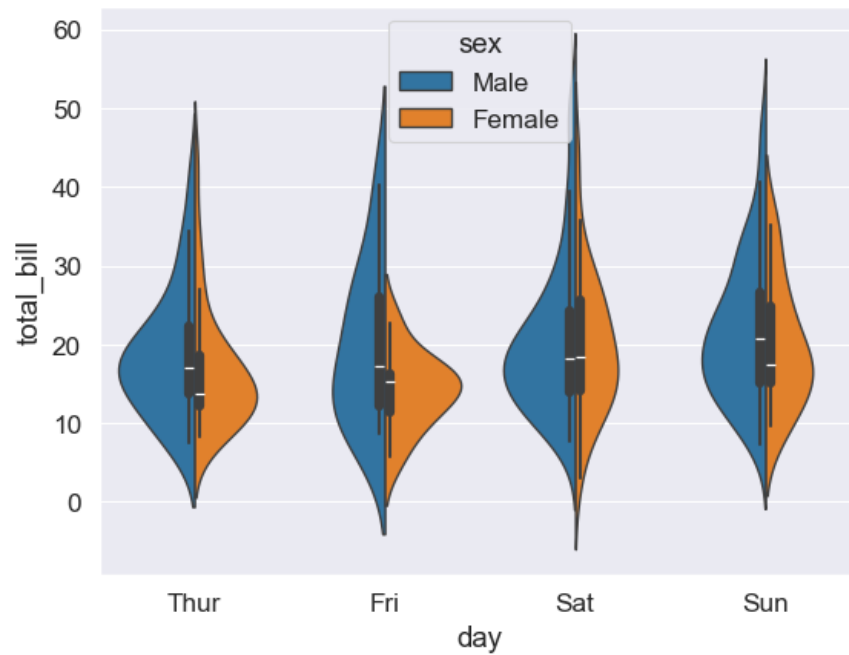


Figure 22.15: output

### 22.1.15 Strip Plot

```
plt.figure(figsize=(8,5))

The strip plot draws a scatter plot representing all data points where one
variable is categorical. It is often used to show all observations with
a box plot that represents the average distribution
Jitter spreads data points out so that they aren't stacked on top of each other
Hue breaks data into men and women
Dodge separates the men and women data
sns.stripplot(x='day', y='total_bill', data=tips_df, jitter=True,
 hue='sex', dodge=True)
```



```
<Axes: xlabel='day', ylabel='total_bill'>
```

```
<Figure size 800x500 with 1 Axes>
```

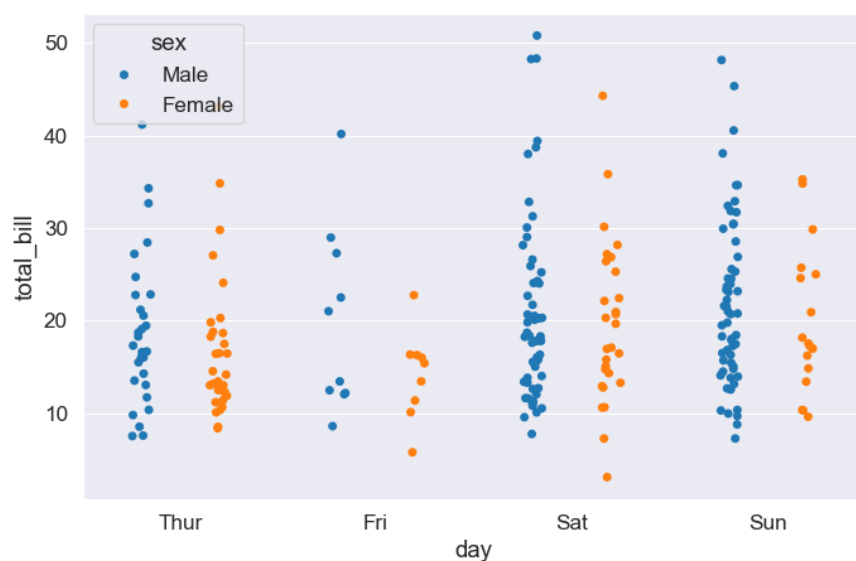


Figure 22.16: output

```
#sns.stripplot(x='RA',y='Dec',data=cs_df, jitter=True, dodge=True)
```

### 22.1.16 Swarm Plot

```
A swarm plot is like a strip plot, but points are adjusted so they don't overlap
It looks like a combination of the violin and strip plots
sns.swarmplot(x='day',y='total_bill',data=tips_df)
```

```
You can stack a violin plot with a swarm
sns.violinplot(x='day',y='total_bill',data=tips_df)
sns.swarmplot(x='day',y='total_bill',data=tips_df, color='white')
```

```
<Axes: xlabel='day', ylabel='total_bill'>
```

<Figure size 640x480 with 1 Axes>

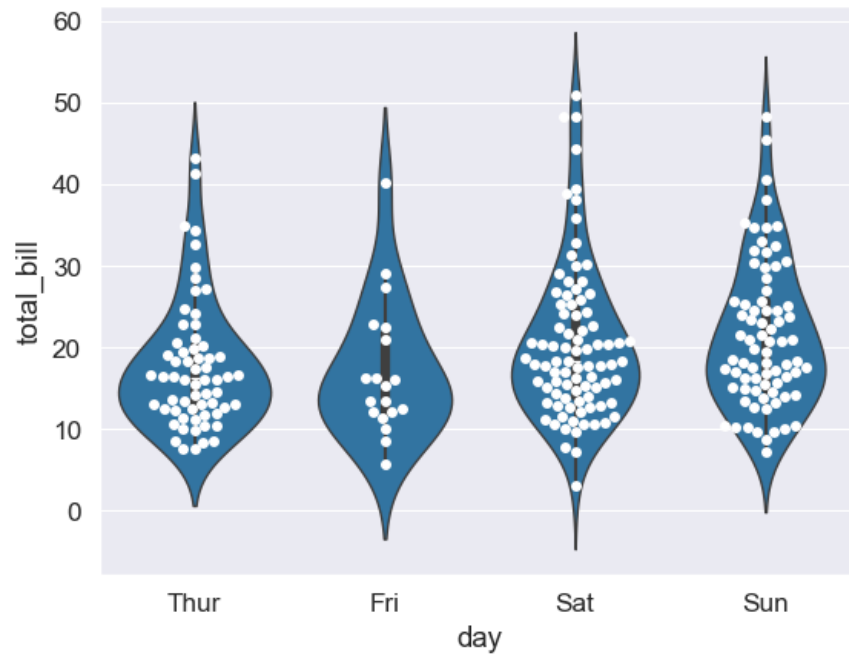


Figure 22.17: output

### 22.1.17 Palettes

```
plt.figure(figsize=(8,6))

sns.set_style('dark')

sns.set_context('talk')

You can use Matplotlib's color maps for color styling
https://matplotlib.org/3.3.1/tutorials/colors/colormaps.html
sns.stripplot(x='day', y='total_bill', data=tips_df, hue='sex',
 palette='seismic')

Add the optional legend with a location number (best: 0,
upper right: 1, upper left: 2, lower left: 3, lower right: 4,
```

```
https://matplotlib.org/3.3.1/api/_as_gen/matplotlib.pyplot.legend.html)
or supply a tuple of x & y from lower left
plt.legend(loc=0)
```

```
<matplotlib.legend.Legend at 0x38d338070>
```

```
<Figure size 800x600 with 1 Axes>
```

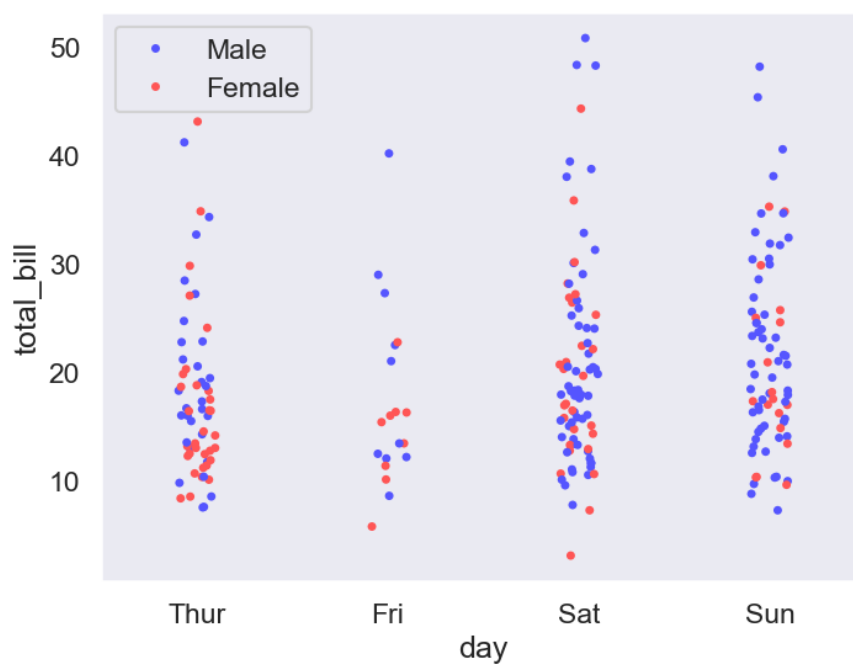


Figure 22.18: output

### 22.1.18 Matrix Plots

### 22.1.19 Heatmaps

```
plt.figure(figsize=(8,6))
sns.set_context('paper', font_scale=1.4)
To create a heatmap with data you must have data set up as a matrix where variables
are on the columns and rows
```

```
Correlation tells you how influential a variable is on the result
So we see that n previous accident is heavily correlated with accidents, while the
insurance premium is not
crash_mx = cs_df.corr()

Create the heatmap, add annotations and a color map
sns.heatmap(crash_mx, annot=True, cmap='Blues')
```

```
<Axes: >
```

```
<Figure size 800x600 with 2 Axes>
```

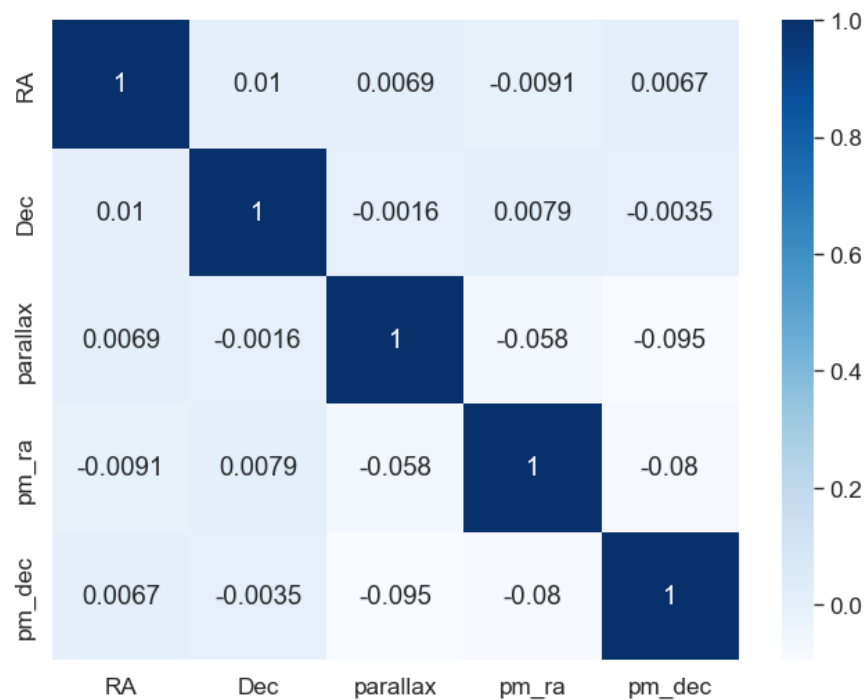


Figure 22.19: output

```
plt.figure(figsize=(8,6))
sns.set_context('paper', font_scale=1.4)

We can create a matrix with an index of month, columns representing years
and the number of passengers for each
```

```
We see that flights have increased over time and that most people travel in
July and August
flights = sns.load_dataset("flights")
flights = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/flights.csv")
flights = flights.pivot_table(index='month', columns='year', values='passengers')
You can separate data with lines
sns.heatmap(flights, cmap='Blues', linecolor='white', linewidth=1)
```

```
<Axes: xlabel='year', ylabel='month'>
```

```
<Figure size 800x600 with 2 Axes>
```

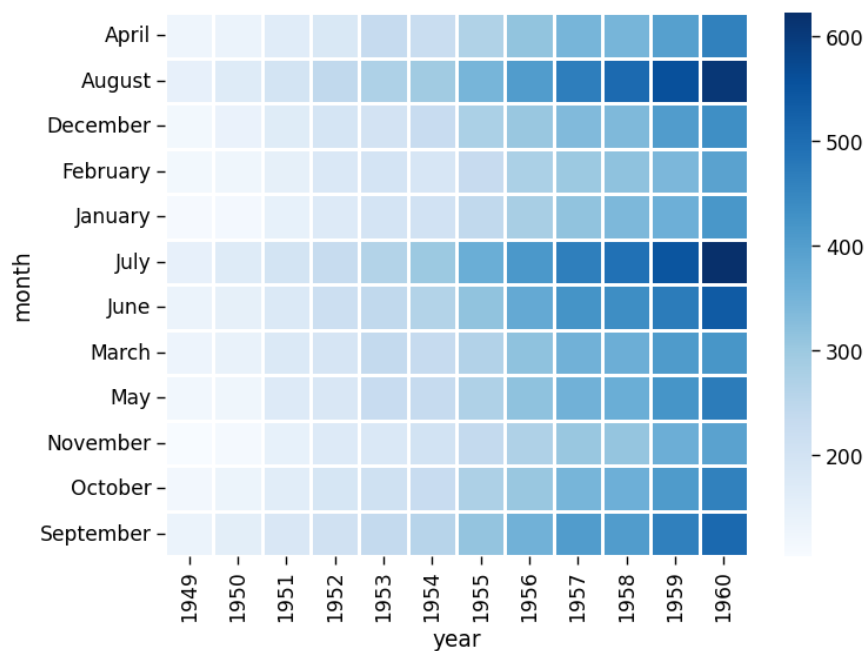


Figure 22.20: output

### 22.1.20 Cluster Map

```
plt.figure(figsize=(8,6))
sns.set_context('paper', font_scale=1.4)
```

```

A Cluster map is a hierarchically clustered heatmap
The distance between points is calculated, the closest are joined, and this
continues for the next closest (It compares columns / rows of the heatmap)
This is data on iris flowers with data on petal lengths
iris = sns.load_dataset("iris")
Return values for species
species = iris.pop("species")
sns.clustermap(iris)

With our flights data we can see that years have been reoriented to place
like data closer together
You can see clusters of data for July & August for the years 59 & 60
standard_scale normalizes the data to focus on the clustering
sns.clustermap(flights, cmap="Blues", standard_scale=1)

```

```
<seaborn.matrix.ClusterGrid at 0x11a028cd0>
```

```
<Figure size 800x600 with 0 Axes>
```

```
<Figure size 1000x1000 with 4 Axes>
```

### 22.1.21 PairGrid

```

plt.figure(figsize=(8,6))
sns.set_context('paper', font_scale=1.4)

You can create a grid of different plots with complete control over what is displayed
Create the empty grid system using the provided data
Colorize based on species
iris_g = sns.PairGrid(iris, hue="species")

Put a scatter plot across the upper, lower and diagonal
iris_g.map(plt.scatter)

Put a histogram on the diagonal
iris_g.map_diag(plt.hist)
And a scatter plot every place else
iris_g.map_offdiag(plt.scatter)

Have different plots in upper, lower and diagonal
iris_g.map_upper(plt.scatter)
iris_g.map_lower(sns.kdeplot)

```

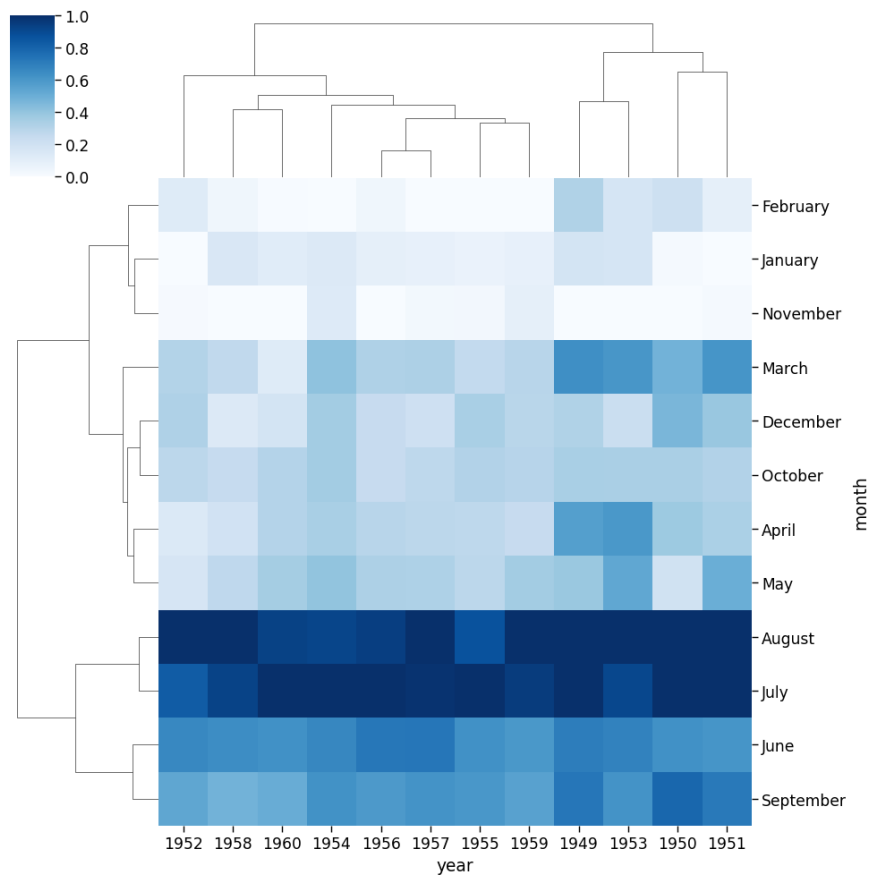


Figure 22.21: output

```
You can define variables for x & y for a custom grid
iris_g = sns.PairGrid(iris, hue="species",
 x_vars=["sepal_length", "sepal_width"],
 y_vars=["petal_length", "petal_width"])

iris_g.map(plt.scatter)

Add a legend last
iris_g.add_legend()
```

```
<seaborn.axisgrid.PairGrid at 0x11a07e770>
```

```
<Figure size 800x600 with 0 Axes>
```

```
<Figure size 645.6x500 with 4 Axes>
```

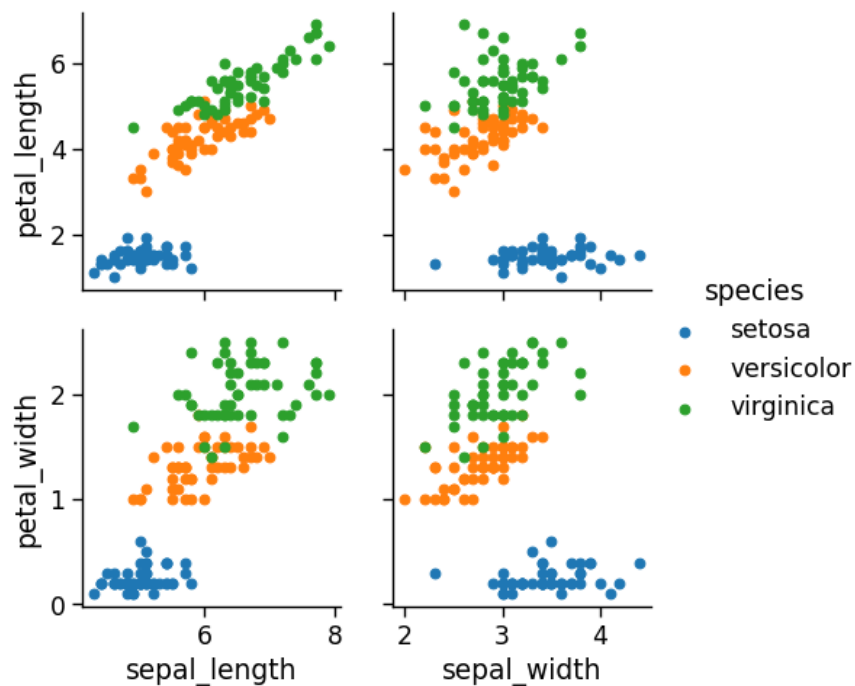


Figure 22.22: output



### 22.1.22 Facet Grid

```
Can also print multiple plots in a grid in which you define columns & rows
Get histogram for smokers and non with total bill for lunch & dinner
tips_fg = sns.FacetGrid(tips_df, col='time', row='smoker')

You can pass in attributes for the histogram
tips_fg.map(plt.hist, "total_bill", bins=8)

Create a scatter plot with data on total bill & tip (You need to parameters)
tips_fg.map(plt.scatter, "total_bill", "tip")

We can assign variables to different colors and increase size of grid
Aspect is 1.3 x the size of height
You can change the order of the columns
Define the palette used
tips_fg = sns.FacetGrid(tips_df, col='time', hue='smoker', height=4, aspect=1.3,
col_order=['Dinner', 'Lunch'], palette='Set1')
tips_fg.map(plt.scatter, "total_bill", "tip", edgecolor='w')

Define size, linewidth and assign a color of white to markers
kws = dict(s=50, linewidth=.5, edgecolor="w")
Define that we want to assign different markers to smokers and non
tips_fg = sns.FacetGrid(tips_df, col='sex', hue='smoker', height=4, aspect=1.3,
hue_order=['Yes', 'No'],
hue_kws=dict(marker=['^', 'v']))
tips_fg.map(plt.scatter, "total_bill", "tip", **kws)
tips_fg.add_legend()

This dataframe provides scores for different students based on the level
of attention they could provide during testing
att_df = sns.load_dataset("attention")
Put each person in their own plot with 5 per line and plot their scores
att_fg = sns.FacetGrid(att_df, col='subject', col_wrap=5, height=1.5)
att_fg.map(plt.plot, 'solutions', 'score', marker='.')
```

```
<seaborn.axisgrid.FacetGrid at 0x119fc1fc0>
```

```
<Figure size 750x600 with 20 Axes>
```

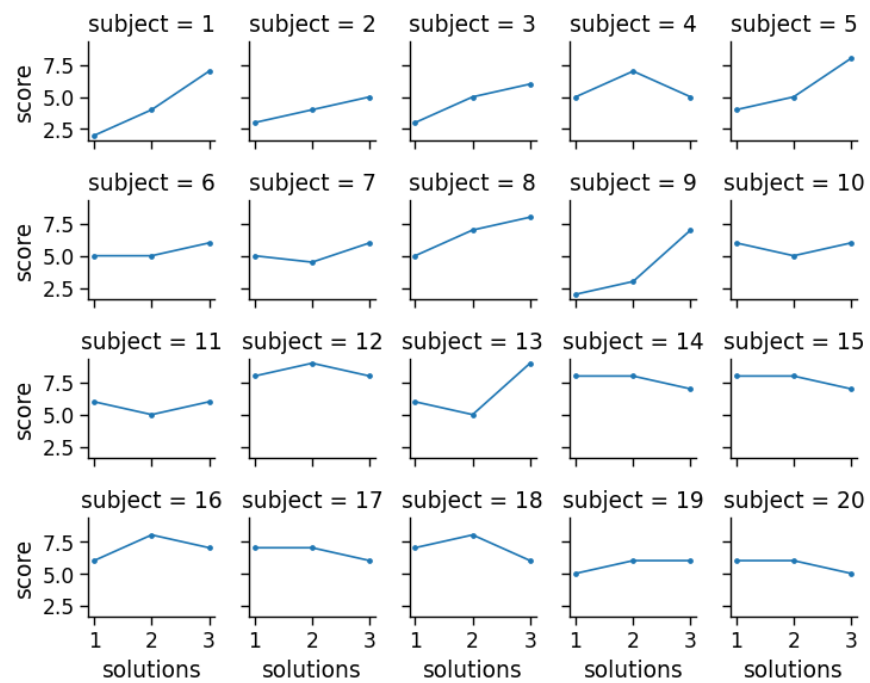


Figure 22.23: output

### 22.1.23 Regression Plots

```
lmplot combines regression plots with facet grid
tips_df = sns.load_dataset('tips')
tips_df.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
plt.figure(figsize=(8,6))
sns.set_context('paper', font_scale=1.4)
```

```
plt.figure(figsize=(8,6))
```

```
We can plot a regression plot studying whether total bill effects the tip
hue is used to show separation based off of categorical data
We see that males tend to tip slightly more
Define different markers for men and women
You can effect the scatter plot by passing in a dictionary for styling of markers
sns.lmplot(x='total_bill', y='tip', hue='sex', data=tips_df, markers=['o', '^'],
 scatter_kws={'s': 100, 'linewidths': 0.5, 'edgecolor': 'w'})
```

```
<seaborn.axisgrid.FacetGrid at 0x11d2e1360>
```

```
<Figure size 800x600 with 0 Axes>
```

```
<Figure size 800x600 with 0 Axes>
```

```
<Figure size 623.475x500 with 1 Axes>
```

```
You can separate the data into separate columns for day data
sns.lmplot(x='total_bill', y='tip', col='sex', row='time', data=tips_df)
tips_df.head()
```

```
Makes the fonts more readable
sns.set_context('poster', font_scale=1.4)
```

```
sns.lmplot(x='total_bill', y='tip', data=tips_df, col='day', hue='sex',
 height=8, aspect=0.6)
```

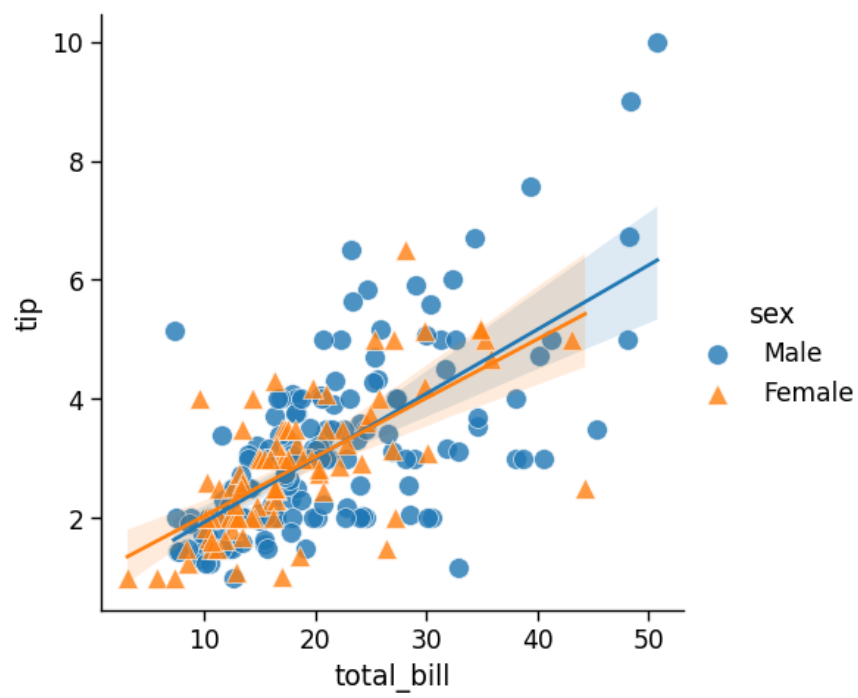


Figure 22.24: output

```
<seaborn.axisgrid.FacetGrid at 0x11d351ae0>
```

```
<Figure size 2228.5x800 with 4 Axes>
```

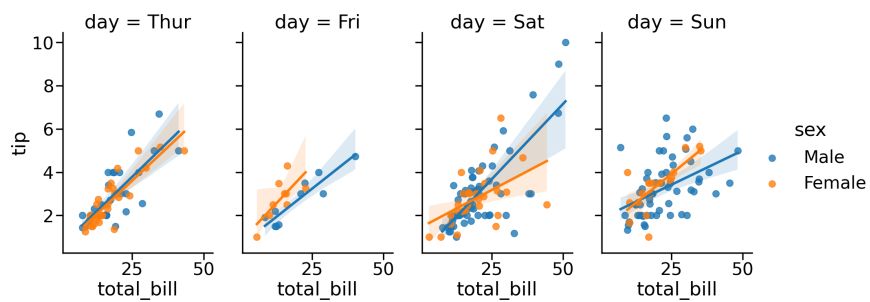


Figure 22.25: output



## Chapter 23

# astropy Constants (SI Units)

Introduction to the astropy library

### ▼ Get Started



astropy is a comprehensive, community-developed Python library designed to support professional-grade astronomical research and education. It serves as the foundation of the modern astronomical Python ecosystem, offering a coherent suite of tools that address the diverse computational needs of observational and theoretical astronomy.

astropy was created in response to the growing need for standardized, well-maintained, and interoperable software within the astronomical community. Before astropy, researchers often relied on fragmented tools, legacy code, and ad hoc scripts, which complicated collaboration, reproducibility, and long-term maintenance. Today, astropy has become the de facto standard for Python-based astronomical computing and is widely adopted in both academic and institutional settings.

At its core, astropy is built on principles of:

Scientific accuracy: rigorously tested and validated tools for real-world applications

Reproducibility: enabling transparent workflows through unit tracking, metadata, and open standards

Interoperability: seamless integration with other scientific libraries (e.g., NumPy, SciPy, Matplotlib)

Modularity: individual subpackages designed for specific domains within astronomy

Community governance: an open model maintained by astronomers, for astronomers

The astropy library includes submodules for:

Physical units and quantities (`astropy.units`)

Physical and astronomical constants (`astropy.constants`)

Celestial coordinate transformations (`astropy.coordinates`)

Time and calendar systems (`astropy.time`)

FITS and tabular data I/O (`astropy.io`, `astropy.table`)

World Coordinate System handling (`astropy.wcs`)

Analytical modeling and fitting (`astropy.modeling`)

Cosmological calculations (`astropy.cosmology`)

Image visualization tools (`astropy.visualization`)

These components are tightly integrated and extensively documented, making astropy not only a powerful research toolkit but also an ideal platform for teaching astronomy and data analysis.

**Note** If you are running this jupyter notebook from Colab, then run the next cell by pressing **SHIFT+ENTER** to install the required packages for this notebook. Otherwise, skip the next cell.

```
!pip install -q astroquery
```

## 23.1 Units, Quantities, and Constants in astropy

Astronomical computations typically involve working with physical quantities expressed in various specialized units. Unlike standard SI units commonly used in physics, astronomers frequently employ historical and practical units such as



**parsecs (pc)**, **astronomical units (AU)**, **solar masses (M<sub>⊙</sub>)**, **solar luminosities (L<sub>⊙</sub>)**, and flux units like **Jansky (Jy)**.

astropy provides three closely related components to simplify working with these diverse units and maintain scientific rigor:

### 23.1.1 1. Units (`astropy.units`)

Units represent physical measurement scales for example, parsecs, kilometers per second, or solar masses. astropy simplifies unit management by allowing easy conversions and ensuring dimensional consistency during calculations.

### 23.1.2 2. Quantities

A quantity combines a numerical value with its associated unit for instance, 10 pc or 1.5 M<sub>⊙</sub>. astropy's quantity objects automatically handle unit tracking, ensuring consistency and minimizing calculation errors from manual conversions.

### 23.1.3 3. Constants (`astropy.constants`)

Constants represent fundamental physical parameters, such as the gravitational constant (**G**), speed of light (**c**), and solar mass (**M<sub>sun</sub>**). astropy provides these constants pre-defined with precise numerical values and units, facilitating accurate and reliable astronomical computations.

Together, **Units**, **Quantities**, and **Constants** form a coherent framework in astropy, enabling astronomers to perform accurate, clear, and reproducible calculations. By embedding unit handling directly into computations, astropy promotes best practices in scientific programming, enhancing both clarity and reliability in astronomical analyses.

```
from astropy import units as u

Define simple quantities with units
distance = 1000 * u.km # Distance in kilometers
velocity = 200 * u.km / u.s # Velocity in kilometers per second
```

In this cell, we import the `astropy.units` module under the alias `u`, which provides access to a wide range of physical and astronomical units.

We then define two basic physical quantities:

- `distance = 1000 * u.km`: a distance of 10 kilometers.
- `velocity = 200 * u.km / u.s`: a velocity of 220 kilometers per second.

Each of these is an `astropy.units.Quantity` object, which stores both the numerical value and the physical unit. These objects can be used in calculations with automatic unit tracking and conversion.

```
print(type(distance))
print(distance)
print(velocity)
```

```
<class 'astropy.units.quantity.Quantity'>
1000.0 km
200.0 km / s
```

```
print(f"distance: {distance:.2f}")
print(distance.unit)
print(distance.value)
print(type(distance.value))
```

```
distance: 1000.00 km
km
1000.0
<class 'numpy.float64'>
```

In this cell, we printed several aspects of an `astropy.units.Quantity` object to explore its structure and its connection to NumPy:

- `print(f"distance: {distance:.2f}")`  
Displays the quantity with its value rounded to two decimal places, including the unit (e.g., 10.00 km).
- `print(distance.unit)`  
Prints the unit component of the quantity (here: km).
- `print(distance.value)`  
Prints just the numerical value, separate from the unit (here: 10.0).
- `print(type(distance.value))`  
Shows the type of the `.value` attribute. For a simple scalar, this is usually a NumPy float type (e.g., `<class 'numpy.float64'>`).

### Conclusion:

An `astropy.units.Quantity` object stores both the value and the unit, and its `.value` attribute is a NumPy type. This tight integration means you can use astropy quantities seamlessly with NumPy functions, combining the convenience of unit handling with the efficiency and flexibility of numerical array operations.

## 23.2 astropy Units List

Reference: [https://docs.astropy.org/en/stable/units/ref\\_api.html#module-astropy.units](https://docs.astropy.org/en/stable/units/ref_api.html#module-astropy.units)

### 23.2.1 Length Units

Unit	Description	Code Usage
<code>m</code>	meter	<code>u.m</code>
<code>cm</code>	centimeter	<code>u.cm</code>
<code>mm</code>	millimeter	<code>u.mm</code>
<code>µm</code>	micrometer	<code>u.µm</code>
<code>nm</code>	nanometer	<code>u.nm</code>
<code>km</code>	kilometer	<code>u.km</code>
<code>au</code>	astronomical unit	<code>u.au</code>
<code>pc</code>	parsec	<code>u.pc</code>
<code>lyr</code>	light-year	<code>u.lyr</code>
<code>R_sun</code>	solar radius	<code>u.R_sun</code>
<code>R_earth</code>	earth radius	<code>u.R_earth</code>
<code>R_jup</code>	jupiter radius	<code>u.R_jup</code>
<code>R_moon</code>	moon radius	<code>u.R_moon</code>

### 23.2.2 Time Units

Unit	Description	Code Usage
<code>s</code>	second	<code>u.s</code>
<code>ms</code>	millisecond	<code>u.ms</code>
<code>µs</code>	microsecond	<code>u.µs</code>
<code>ns</code>	nanosecond	<code>u.ns</code>
<code>min</code>	minute	<code>u.min</code>
<code>h</code>	hour	<code>u.h</code>
<code>day</code>	day	<code>u.day</code>
<code>yr</code>	Julian year	<code>u.yr</code>
<code>Gyr</code>	gigayear	<code>u.Gyr</code>

### 23.2.3 Mass Units

Unit	Description	Code Usage
<b>kg</b>	kilogram	<code>u.kg</code>
<b>g</b>	gram	<code>u.g</code>
<b>mg</b>	milligram	<code>u.mg</code>
<b>µg</b>	microgram	<code>u.ug</code>
<b>u</b>	atomic mass unit	<code>u.u</code>
<b>M_sun</b>	solar mass	<code>u.M_sun</code>
<b>M_earth</b>	earth mass	<code>u.M_earth</code>
<b>M_jup</b>	jupiter mass	<code>u.M_jup</code>
<b>M_moon</b>	moon mass	<code>u.M_moon</code>

### 23.2.4 Temperature Units

Unit	Description	Code Usage
<b>K</b>	kelvin	<code>u.K</code>
<b>deg_C</b>	degree Celsius	<code>u.deg_C</code>
<b>deg_F</b>	degree Fahrenheit	<code>u.deg_F</code>

### 23.2.5 Energy and Power Units

Unit	Description	Code Usage
<b>J</b>	joule	<code>u.J</code>
<b>erg</b>	erg	<code>u.erg</code>
<b>eV</b>	electron volt	<code>u.eV</code>
<b>keV</b>	kiloelectron volt	<code>u.keV</code>
<b>MeV</b>	megaelectron volt	<code>u.MeV</code>
<b>GeV</b>	gigaelectron volt	<code>u.GeV</code>
<b>W</b>	watt	<code>u.W</code>
<b>L_sun</b>	solar luminosity	<code>u.L_sun</code>

### 23.2.6 Velocity and Acceleration

Unit	Description	Code Usage
<b>m/s</b>	meters per second	<code>u.m / u.s</code>
<b>km/s</b>	kilometers per second	<code>u.km / u.s</code>
<b>cm/s</b>	centimeters per second	<code>u.cm / u.s</code>
<b>m/s<sup>2</sup></b>	meters per second squared	<code>u.m / u.s**2</code>

**23.2.7 Volume and Area**

Unit	Description	Code Usage
$\text{m}^2$	square meter	<code>u.m**2</code>
$\text{cm}^2$	square centimeter	<code>u.cm**2</code>
$\text{km}^2$	square kilometer	<code>u.km**2</code>
$\text{AU}^2$	square astronomical unit	<code>u.au**2</code>
$\text{m}^3$	cubic meter	<code>u.m**3</code>
$\text{cm}^3$	cubic centimeter	<code>u.cm**3</code>
L	liter	<code>u.L</code>

**23.2.8 Frequency Units**

Unit	Description	Code Usage
Hz	hertz	<code>u.Hz</code>
kHz	kilohertz	<code>u.kHz</code>
MHz	megahertz	<code>u.MHz</code>
GHz	gigahertz	<code>u.GHz</code>
THz	terahertz	<code>u.THz</code>

**23.2.9 Electric Units**

Unit	Description	Code Usage
C	coulomb	<code>u.C</code>
V	volt	<code>u.V</code>
A	ampere	<code>u.A</code>
ohm	ohm	<code>u.ohm</code>
S	siemens	<code>u.S</code>
F	farad	<code>u.F</code>
H	henry	<code>u.H</code>
T	tesla	<code>u.T</code>
Wb	weber	<code>u.Wb</code>

**23.2.10 Radiation and Flux**

Unit	Description	Code Usage
Jy	jansky	<code>u.Jy</code>

Unit	Description	Code Usage
photon	photon (dimensionless)	<code>u.photon</code>
$\text{W}/(\text{m}^2 \text{ Hz sr})$	spectral radiance (SI)	<code>u.W / (u.m**2 * u.Hz * u.sr)</code>
$\text{erg}/(\text{s cm}^2)$	flux in CGS	<code>u.erg / (u.s * u.cm**2)</code>

### 23.2.11 Angles and Solid Angles

Unit	Description	Code Usage
deg	degree	<code>u.deg</code>
rad	radian	<code>u.rad</code>
arcmin	arcminute	<code>u.arcmin</code>
arcsec	arcsecond	<code>u.arcsec</code>
sr	steradian	<code>u.sr</code>

### 23.2.12 Miscellaneous and Dimensionless

Unit	Description	Code Usage
dimensionless_unscaled	pure number	<code>u.dimensionless_unscaled</code>
dex	logarithmic unit	<code>u.dex</code>
%	percent	<code>u.percent</code>
ppm	parts per million	<code>u.ppm</code>
count	dimensionless count	<code>u.count</code>

```
distance_pc = u.Quantity(5, u.pc) #Just another way to define Quantity("Unit + Value")
print(type(distance_pc)) # Yes! Again Quantity class!
print(f"distance [pc]: {distance_pc:.2f}")
```

```
<class 'astropy.units.quantity.Quantity'>
distance [pc]: 5.00 pc
```

```
speed_of_light = u.Quantity(1, u.lyr/u.year)
print(f"Speed of light [?]: {speed_of_light:.2f}")
print(f"Speed of light [km/s]: {speed_of_light.to(u.km/u.s):.2f}") #Magic!
```

```
Speed of light [?]: 1.00 lyr / yr
Speed of light [km/s]: 299792.46 km / s
```

### 23.2.13 Converting to Different Units

One of the most important features of `astropy.units.Quantity` objects is the ability to convert between compatible units using the `.to()` method.

A `Quantity` object represents a numerical value associated with a physical unit. If you need to express that value in another unit of the same physical dimension (for example, meters to kilometers, seconds to hours, or joules to electron-volts), the `.to()` method provides a simple and robust way to perform the conversion.

#### 23.2.13.1 Basic Syntax:

```
converted_quantity = original_quantity.to(new_unit)
```

#### 23.2.13.2 Notes:

- The target unit (`new_unit`) must be **dimensionally compatible** with the original unit.
- If the target unit is not compatible (e.g., converting time to length), `astropy` will raise a `UnitsError`.

#### 23.2.13.3 Example:

Suppose you define a distance of `1.5 * u.parsec`. You can convert it to light-years using:

```
(1.5 * u.pc).to(u.lyr)
```

This returns a new `Quantity` object representing the same physical value expressed in light-years.

The `.to()` method ensures that your code remains physically meaningful and unit-consistent throughout your calculations.

```
print(f"distance [pc]: {distance_pc:.2f}")
print(f"distance [mpc]: {distance_pc.to(u.mpc):.2f}") #It's not in our list but pc with "mega" or
print(f"distance [lyr]: {distance_pc.to(u.lyr):.2f}")
print(f"distance [AU]: {distance_pc.to(u.au):.2f}")
print(f"distance [R_sun]: {distance_pc.to(u.R_sun):.2f}")
print(f"distance [km]: {distance_pc.to(u.km):.2f}")
print(f"distance [mm]: {distance_pc.to(u.mm):.2f}") #How about nm? Okay...
print(f"distance [pm]: {distance_pc.to(u.pm):.2f}") #YES! even picometer (pm) - metric unit of l
```

```

distance [pc]: 5.00 pc
distance [mpc]: 5000.00 mpc
distance [lyr]: 16.31 lyr
distance [AU]: 1031324.03 AU
distance [R_sun]: 221767829.63 solRad
distance [km]: 154283879074568.38 km
distance [mm]: 154283879074568339456.00 mm
distance [pm]: 154283879074568357719543119872.00 pm

```

### 23.2.14 Prefixes list

Source: [https://docs.astropy.org/en/stable/units/ref\\_api.html#module-astropy.units](https://docs.astropy.org/en/stable/units/ref_api.html#module-astropy.units)

```
distance_pc.to(u.s) #UnitsError
```

```

UnitConversionError Traceback (most recent call last)
Cell In[9], line 1
----> 1 distance_pc.to(u.s) #UnitsError

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astropy/units/quantity.py:9
 939 unit = Unit(unit)
 940 if copy:
 941 # Avoid using to_value to ensure that we make a copy. We also
 942 # don't want to slow down this method (esp. the scalar case).
--> 943 value = self._to_value(unit, equivalencies)
 944 else:
 945 # to_value only copies if necessary
 946 value = self.to_value(unit, equivalencies)

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astropy/units/quantity.py:8
 893 equivalencies = self._equivalencies
 894 if not self.dtype.names or isinstance(self.unit, StructuredUnit):
 895 # Standard path, let unit to do work.
--> 896 return self.unit.to(
 897 unit, self.view(np.ndarray), equivalencies=equivalencies
 898)
 900 else:
 901 # The .to() method of a simple unit cannot convert a structured
 902 # dtype, so we work around it, by recursing.
 903 # TODO: deprecate this?
 904 # Convert simple to Structured on initialization?
 905 result = np.empty_like(self.view(np.ndarray))

```



```

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astropy/units/core.py:1228, in UnitBase
 1226 return UNITY
 1227 else:
-> 1228 return self.get_converter(Unit(other), equivalencies)(value)

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astropy/units/core.py:1157, in UnitBase
 1154 else:
 1155 return lambda v: b(converter(v))
-> 1157 raise exc

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astropy/units/core.py:1140, in UnitBase
 1138 # if that doesn't work, maybe we can do it with equivalencies?
 1139 try:
-> 1140 return self._apply_equivalencies(
 1141 self, other, self._normalize_equivalencies(equivalencies)
 1142)
 1143 except UnitsError as exc:
 1144 # Last hope: maybe other knows how to do it?
 1145 # We assume the equivalencies have the unit itself as first item.
 1146 # TODO: maybe better for other to have a `_back_converter` method?
 1147 if hasattr(other, "equivalencies"):

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astropy/units/core.py:1091, in UnitBase
 1088 unit_str = get_err_str(unit)
 1089 other_str = get_err_str(other)
-> 1091 raise UnitConversionError(f"{unit_str} and {other_str} are not convertible")

UnitConversionError: 'pc' (length) and 's' (time) are not convertible

print(f"Speed of light [SI]: {speed_of_light.si:.2f}") #If you want some standarts
print(f"Speed of light [CGS]: {speed_of_light.cgs:.2f}")
print(f"Speed of light [custom value mm/ns]: {speed_of_light.to_value(u.mm/u.ns):.2f}") #Maybe you

Speed of light [SI]: 299792458.00 m / s
Speed of light [CGS]: 29979245800.00 cm / s
Speed of light [custom value mm/ns]: 299.79

```

## 23.3 Constants

Now that we know how to define physical quantities with explicit units using `astropy.units`, the next logical step in writing physically meaningful code is to incorporate commonly used constants—such as the gravitational constant, the speed of light, or the mass of the Sun—directly into our calculations.

astropy simplifies this process through the dedicated subpackage `astropy.constants`, which provides a rich collection of physical and astronomical constants, all implemented as fully unit-aware objects.

The constants in this module are not just hard-coded numbers; they are instances of `astropy.constants.Constant`, which means each one includes: - a precise numerical value (as defined by CODATA or the IAU), - a unit (compatible with the `astropy.units` system), - a description, and - support for seamless integration with other astropy subpackages.

This design ensures dimensional consistency and scientific correctness when constants are used in expressions, conversions, or calculations.

### 23.3.1 Why Use `astropy.constants`?

Using the `astropy.constants` module offers several clear advantages: - **Accurate:** All constants are based on the most recent and reliable scientific data. - **Built-in Units:** Each constant is unit-aware and works seamlessly with quantities, unit conversions, and equations. - **Convenient:** Constants are available by name—no need to memorize values or perform manual conversions. - **Consistent:** Shared use of constants across notebooks, projects, and collaborators avoids mismatched assumptions. - **Readable:** Code becomes more expressive and self-documenting when constants are used by name.

### 23.3.2 What Types of Constants Are Included?

The module includes constants in several broad categories:

- **Physical constants:** Gravitational constant, Planck constant, speed of light, Boltzmann constant, elementary charge, and more.
- **Astronomical constants:** Solar mass, radius, and luminosity; parameters for Earth, Jupiter, and the Moon; astronomical unit; parsec; light-year, etc.
- **Radiative and atomic constants:** Wien's displacement constant, Rydberg constant, Thomson cross-section, Stefan-Boltzmann constant, and others.

These constants are essential for research and computations in astrophysics, cosmology, planetary science, and observational astronomy.

---

In the next cell, you will find a complete table listing the most important constants available in `astropy.constants`, along with their values and units.

```
from astropy.constants import c #First way to import
print(c)
```

```
Name = Speed of light in vacuum
Value = 299792458.0
Uncertainty = 0.0
Unit = m / s
Reference = CODATA 2018
```

```
from astropy import constants as const #Second way to import
print(const.k_B)
```

```
Name = Boltzmann constant
Value = 1.380649e-23
Uncertainty = 0.0
Unit = J / K
Reference = CODATA 2018
```

```
print(c.to(u.pc/u.year)) #We can convert constants as other Quantities
print(const.c.cgs)
```

```
from IPython.display import Math, display
display(Math(f"{c:latex}")) #Some IPython and jupyter magic!
```

```
0.30660139378555057 pc / yr
29979245800.0 cm / s
```

```
<IPython.core.display.Math object>
```

## 23.4 Example: Luminosity of a star-like object using the Stefan–Boltzmann law

This example demonstrates how `astropy.units` and `astropy.constants` work seamlessly together in realistic astrophysical computations.

We calculate the **total luminosity**  $L$  of a hot, Jupiter-sized exoplanet using the **Stefan–Boltzmann law**:

$$L = 4\pi R^2 \sigma T^4$$

Here we define the planet's: - **radius** as 1.6 times the radius of Jupiter, and - **effective temperature** as 1800 K.

We then use the built-in **Stefan–Boltzmann constant** from `astropy.constants` and compute the result with full unit support.

This example highlights: - How to use physical constants and quantities with units in actual formulas, - How units propagate through calculations automatically, - And how to **compare the result** to another unit-aware constant, such as the Sun’s luminosity `L_sun`.

The final result is given in **watts** and as a **fraction of the solar luminosity**.

```
from astropy.constants import sigma_sb, L_sun, R_jup
import numpy as np

radius = 1.6 * R_jup # Define exoplanet radius
temperature = 1800 * u.K # Define effective temperature

luminosity = 4 * np.pi * radius**2 * sigma_sb * temperature**4 # Compute luminosity

print(f"Luminosity: {luminosity:.3e}")
print(f"As a fraction of solar luminosity: {(luminosity / L_sun).decompose():.4f}")
```

Luminosity: 9.787e+22 W  
As a fraction of solar luminosity: 0.0003

### 23.4.1 Wait! What does `.decompose()` do?

In the final line of the calculation, we compare the luminosity of our object to the luminosity of the Sun:

```
(luminosity / L_sun).decompose()
```

This operation divides the computed luminosity (in watts) by the solar luminosity constant `L_sun`. The result is a **dimensionless Quantity**, because both values have the same physical unit.

However, the resulting object still retains unit information internally. Calling `.decompose()` simplifies the unit expression and **returns a pure number**, explicitly removing unit tags and expressing the result in terms of base SI units (or a dimensionless value, in this case).

This is especially useful when: - You want a scalar value for plotting or formatting, - You’re comparing dimensionless ratios, - You want to confirm that the units cancel properly.

The output tells us **how many times more (or less)** luminous our object is compared to the Sun.

```
g = 980 * u.cm / u.s**2
g_decomposed = g.decompose()
print(g_decomposed) # meters instead of cm
```

```
9.8 m / s2
```

### 23.4.2 Be careful when using `.decompose()` on Quantities with Units!

The `.decompose()` method in `astropy.units` simplifies a quantity by expressing its unit in terms of **base SI units** (or any system you specify). It does **not** remove units, but rewrites them in their most fundamental form.

This is different from `.to()` which converts to a specified target unit — `.decompose()` converts to **base units** automatically.

#### 23.4.2.1 Example

Suppose we define a quantity with compound units, such as surface gravity

Here's what happened: -  $980 \text{ cm} / \text{s}^2$  is a valid unit, - `.decompose()` rewrote it as  $9.8 \text{ m} / \text{s}^2$ , since meters are the SI base unit for length.

This is useful when: - You want to check the **fundamental dimensional form** of a unit expression, - You're debugging complex formulas and want to ensure consistency, - You're comparing two quantities that may have **different-looking but equivalent units**.

#### 23.4.2.2 Note

`.decompose()` will preserve the numeric value adjusted to match the new unit, and it will never return a unitless value **unless** the original expression is truly dimensionless.

If you want to extract a scalar number (i.e., remove units entirely), you can use `.value` after decomposing.

```
unit = u.s**-1
alternative_units = unit.compose()
print(alternative_units) #Lots of possible units
```

```
[Unit("Hz"), Unit("Bq"), Unit("2.7027e-11 Ci")]
```

### 23.4.3 Example: Interpreting `1 / s` with `.compose()`

The `.compose()` method allows us to rewrite a unit in terms of **known standard combinations** from physical sciences, making it easier to **understand the context** of a computation.

Let's take the unit `1 / second`, which occurs in many physical situations (frequencies, decay rates, oscillations). While `1/s` is technically complete, `.compose()` helps us uncover the **physical meanings** it may represent.

#### 23.4.3.1 What This Tells Us:

- Hz (hertz) is used for frequency — cycles per second.
- Bq (becquerel) is used for radioactive decay — disintegrations per second.
- Ci (curie), though not SI, is a legacy unit for activity (with a scaling factor).

These are **dimensionally identical**, but semantically distinct — and `.compose()` surfaces them all.

#### 23.4.3.2 Summary

- `.compose()` helps **reveal context** by mapping abstract units like `1/s` to physical interpretations.
- This is particularly useful in education, debugging, or reporting results in specific domains (e.g., nuclear physics vs. signal processing).
- The method respects known units in the `astropy` system and returns all valid options.

You can further restrict the search using `system='si'`, `system='cgs'`, or a custom system to control the output.

## 23.5 Working with `astropy.units` and NumPy Arrays

One of the powerful features of `astropy.units` is that it integrates smoothly with NumPy arrays, allowing you to perform **vectorized calculations** with **unit-aware data**.

This is especially useful in astronomy, where data often comes in arrays (e.g., lists of distances, velocities, fluxes). With `Quantity` objects, you can attach physical units directly to arrays, and all subsequent computations will correctly propagate units.

### 23.5.1 Extending the Stefan–Boltzmann Law to Stellar Populations

In the previous example, we used the Stefan–Boltzmann law to calculate the luminosity of a single star based on its radius and effective temperature. However, in many astrophysical contexts — such as stellar population synthesis, HR diagram analysis, or galaxy modeling — we often deal with **large arrays of stars**.

Now, we extend the same physical principle to compute the luminosities of **multiple stars simultaneously**, each with its own temperature.

This demonstrates how: - Units are preserved through element-wise operations, - You can use `.to()` to convert the entire array of results, - And unit consistency is automatically enforced — reducing the risk of mistakes.

```
radii = 1 * u.R_sun # stellar radii
temperatures = np.linspace(3000, 30000, 300) * u.K # effective temperatures
luminosities = 4 * np.pi * radii**2 * const.sigma_sb * temperatures**4 # Stefan-Boltzmann law
print(luminosities.to(u.L_sun)[0:10]) #Convert all array elements and show 10 elements
```

```
[0.07297577 0.08216688 0.09220002 0.10312509 0.11499347 0.12785794
 0.14177275 0.15679356 0.17297749 0.19038307] solLum
```

```
np.mean(luminosities.to(u.L_sun)) #works with numpy
```

```
<Quantity 162.84502705 solLum>
```

```
from matplotlib import pyplot as plt
from astropy.visualization import quantity_support
quantity_support()
```

```
plt.plot(temperatures, luminosities.to(u.L_sun))
```

```
[<matplotlib.lines.Line2D at 0x110998d30>]
```

```
<Figure size 640x480 with 1 Axes>
```

### 23.5.2 Plotting with Astropy Quantities Using `quantity_support()`

In this plot, we directly visualize the relationship between stellar **effective temperature** and **luminosity** using `astropy.units.Quantity` objects without stripping their units.

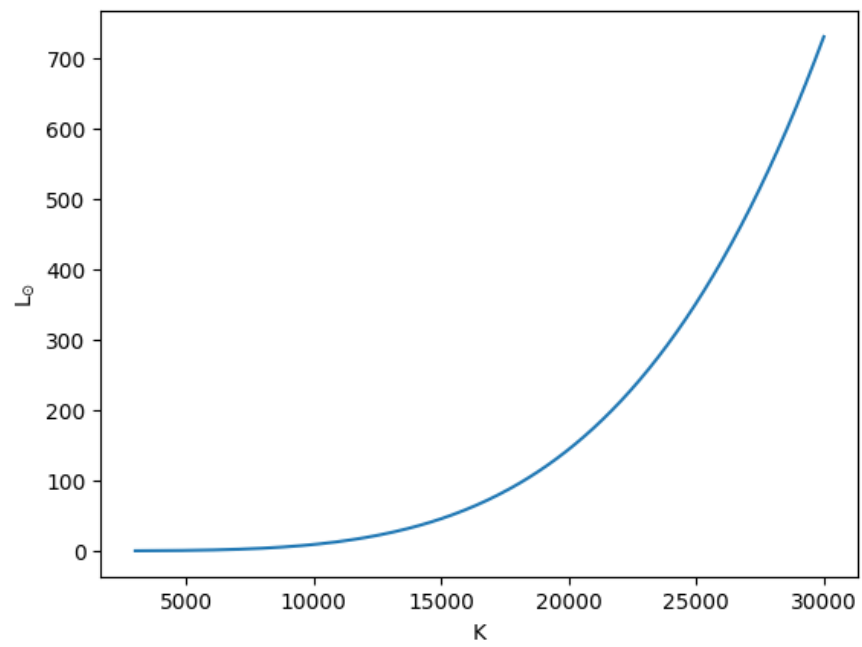


Figure 23.1: output



To make this possible, we activate `astropy.visualization.quantity_support()` — a utility that allows `matplotlib` to understand and display physical units on axes automatically. This means we can pass `Quantity` arrays (like `temperatures` and `luminosities`) directly to `plt.plot()` without manual conversion.

### 23.5.2.1 What happens:

- The x-axis (temperature) is in Kelvin and comes directly from the `Quantity` array `temperatures`.
- The y-axis (luminosity) is converted to **solar luminosities** using `.to(u.L_sun)`, still as a `Quantity`.
- Units are automatically displayed on the axes (e.g., [K] and [solLum]).
- The plot shows a steep, nonlinear rise in luminosity with increasing temperature, following the Stefan–Boltzmann law:

$$L = 4\pi R^2 \sigma T_{\text{eff}}^4$$

This kind of unit-aware plotting is especially helpful in scientific workflows, as it preserves **dimensional clarity**, reduces errors, and makes your graphs **self-documenting**.

## 23.6 Summary: astropy Units, Quantities, and Constants

This lesson introduced the foundational tools in `astropy` for working with physical units and constants in scientific and astrophysical calculations. The use of unit-aware computations ensures dimensional consistency, scientific clarity, and code reliability.

### 23.6.1 Concepts Covered

- **Astropy Units (`astropy.units`)**
  - Assigning physical units to scalar values and arrays.
  - Performing arithmetic operations with automatic unit propagation.
  - Converting between compatible units using `.to()`.
  - Simplifying units into base SI using `.decompose()`.
  - Expressing units in familiar compound forms using `.compose()`.
- **Physical Quantities (`Quantity`)**
  - Combining numerical values with units to create `Quantity` objects.

- Supporting NumPy-style vectorized operations on arrays of quantities.
- Enabling scientific plotting directly with units using `quantity_support()`.
- **Astropy Constants (`astropy.constants`)**
  - Accessing predefined physical and astronomical constants.
  - Using constants with units in physical equations.
  - Ensuring precision and dimensional correctness in calculations.
- **Scientific Visualization**
  - Plotting quantities directly with axis labels showing physical units.
  - Creating meaningful astrophysical plots such as luminosity-temperature diagram.

### 23.6.2 Additional features worth exploring

- **Equivalencies**  
Enable conversions between units that are not dimensionally identical but are physically related (e.g., frequency – wavelength – energy, temperature – energy). <https://docs.astropy.org/en/stable/units/equivalencies.html>
- **Custom Units**  
Define domain-specific or institutional units for specialized applications. [https://docs.astropy.org/en/stable/units/combining\\_and\\_defining.html](https://docs.astropy.org/en/stable/units/combining_and_defining.html)
- **Unit Formatting**  
Format units for LaTeX, Unicode, or plain text representations. <https://docs.astropy.org/en/stable/units/format.html>

## 23.7 Assignment: Determining Stellar Distances from Parallax

In this task, you will compute the distances to nearby stars based on their **measured parallaxes** using `astropy.units`.

### 23.7.1 Scientific Background

Stellar parallax is the apparent shift in a star’s position due to Earth’s orbit around the Sun. The distance  $d$  to a star in **parsecs** is inversely proportional to its parallax angle  $p$  in **arcseconds**:

$$d = \frac{1}{p}$$

### 23.7. ASSIGNMENT: DETERMINING STELLAR DISTANCES FROM PARALLAX287

Where: -  $d$  — distance to the star (in parsecs), -  $p$  — parallax angle (in arcseconds).

If the parallax is given in **milliarcseconds (mas)**, the formula becomes:

$$d = \frac{1000}{p_{\text{mas}}}$$

#### 23.7.2 Objective

- Convert observed parallaxes in **milliarcseconds** into distances in **parsecs**, **light-years**, and **AU**, using `astropy.units`.

---

#### 23.7.3 Given Data

Star	Parallax [mas]
Proxima Centauri	768.5
Alpha Centauri A	747.1
Barnard's Star	548.3
Wolf 359	419.1
Lalande 21185	392.6
Sirius A	379.2
Luyten's Star	264.0
Ross 154	336.0
Epsilon Eridani	310.9
Lacaille 9352	304.2

---

#### 23.7.4 Tasks

1. Convert parallax values (in milliarcseconds) to **Quantity** using `u.mas`.
2. Convert parallax to distance in **parsecs** using the inverse relationship.
3. Convert resulting distances to:
  - **light-years** using `.to(u.lyr)`
  - **astronomical units (AU)** using `.to(u.au)`

### 23.7.5 Hint

You can either use the `.value` attribute to extract the number from a `Quantity`. For a more elegant and unit-aware approach, you can apply an equivalency. To learn more, see: <https://docs.astropy.org/en/stable/units/equivalencies.html>

## 23.8 Part II: Interstellar travel time with Voyager-like speed

After computing distances to nearby stars from their parallaxes, estimate how long it would take a spacecraft traveling at Voyager 1's speed (approximately 17 km/s) to reach them.

### 23.8.1 Given

- Distance  $d$  to each star (already computed from parallax)
- Voyager 1 speed:

$$v = 17 \text{ km/s}$$

### 23.8.2 Formula

The time  $t$  required to travel distance  $d$  is given by:

$$t = \frac{d}{v}$$

### 23.8.3 Tasks

1. Use previously computed distances in **AU** or **parsecs** or even meters...
2. Convert Voyager's speed to an `astropy.Quantity`:

$$v = 17 \text{ km/s}$$

3. Compute  $t = d/v$  using astropy units.
4. Express travel times in:
  - **years**
  - **millennia**
5. Compare the results with the current age of humanity and the age of civilization.

## Chapter 24

# Summary: Core Concepts of `astropy.coordinates`

### 24.1 Introduction to `astropy.coordinates`

#### ▼ Get Started



Now that we have a solid understanding of `astropy.units` and how to represent physical quantities with consistent, meaningful units, we can move forward to a central topic in astronomical computing: **celestial coordinate systems**.

The `astropy.coordinates` subpackage provides a powerful and flexible framework for representing and transforming astronomical positions and motions. It allows astronomers to specify positions of objects in various coordinate systems, convert between them, and perform operations such as angular separations, projections, and velocity calculations.

This module is essential for tasks involving: - Sky positions (e.g., RA/Dec, Galactic coordinates) - Proper motions and radial velocities - Coordinate transformations (e.g., ICRS → Galactic → AltAz) - Observation planning and target tracking - Parsing, formatting, and aligning catalogs

Behind the scenes, `astropy.coordinates` is fully integrated with `astropy.units`, so all angles, distances, and velocities are unit-aware, ensuring physical and computational consistency.

In this section, we will focus on the fundamental functionality of the `astropy.coordinates` module — the tools most commonly used to define, interpret, and convert astronomical positions. Mastering these basic elements will be essential for working with astronomical databases and sky catalogs later, particularly when using tools such as `astroquery`.

**Note** If you are running this jupyter notebook from Colab, then run the next cell by pressing SHIFT+ENTER to install the required packages for this notebook. Otherwise, skip the next cell.

```
!pip install -q astropy
```

```
from astropy import units as u
from astropy.coordinates import SkyCoord
import numpy as np

coord = SkyCoord(ra=138.0108*u.degree, dec=-64.8625*u.degree, frame='icrs') #NGC 280
print(coord)
print(type(coord))
```

```
<SkyCoord (ICRS): (ra, dec) in deg
 (138.0108, -64.8625)>
<class 'astropy.coordinates.sky_coordinate.SkyCoord'>
```

In this cell, we create an object of the `SkyCoord` class from the `astropy.coordinates` module to represent the sky position of the globular cluster **NGC 2808**.

We provide: - **Right Ascension (RA)**: 138.0108 degrees  
 - **Declination (Dec)**: -64.8625 degrees  
 - **Coordinate frame**: 'icrs' (International Celestial Reference System), the standard celestial frame based on extragalactic reference points

By explicitly attaching the unit `u.degree` to each value, we ensure unit consistency and enable the coordinate object to interact properly with other tools in the `astropy` ecosystem.

The resulting `SkyCoord` object encapsulates both position and reference frame information, and serves as the foundation for coordinate transformations, distance associations, proper motion handling, and querying astronomical databases using tools like `astroquery`.

```
#Warning: astropy.coordinates can digest any coordinate system - keep your fingers away from the
coord = SkyCoord(138.0108, -64.8625, frame='icrs', unit='deg')
print(coord)
coord = SkyCoord('09h12m2.6s', '-64d51m45s', frame='icrs')
print(coord)
coord = SkyCoord('09 12 2.6 -64 51 45', unit=(u.hourangle, u.deg))
print(coord)
coord = SkyCoord('09:12:2.6 -64:51:45', unit=(u.hourangle, u.deg))
print(coord)
```

```
<SkyCoord (ICRS): (ra, dec) in deg
(138.0108, -64.8625)>
<SkyCoord (ICRS): (ra, dec) in deg
(138.01083333, -64.8625)>
<SkyCoord (ICRS): (ra, dec) in deg
(138.01083333, -64.8625)>
<SkyCoord (ICRS): (ra, dec) in deg
(138.01083333, -64.8625)>
```

```
print(coord.ra, coord.dec)
print(type(coord.ra))
print(type(coord.dec))
```

```
138d00m39s -64d51m45s
<class 'astropy.coordinates.angles.core.Longitude'>
<class 'astropy.coordinates.angles.core.Latitude'>
```

In this cell, we retrieve the stored **Right Ascension** and **Declination** from the `SkyCoord` object using the `.ra` and `.dec` attributes.

These attributes return `Quantity` objects from `astropy.units`, which means they retain both the numerical value and the unit (in this case, degrees). This allows for safe arithmetic, conversion to other units, and precise control over formatting.

By printing `type(coord.ra)`, we confirm that the output is not just a number but an instance of `astropy.units.Quantity`. This reinforces the fact that `SkyCoord` maintains full unit awareness internally.

```
print(coord.ra.hour, coord.dec.deg)
print(type(coord.ra.hour))
```

```
9.200722222222224 -64.8625
<class 'numpy.float64'>
```

In this cell, we extract the **Right Ascension** and **Declination** from the `SkyCoord` object, but in alternative representations:

- `.ra.hour` converts the right ascension from degrees to **hours**, since RA is traditionally measured in hours, minutes, and seconds in astronomy (1 hour = 15 degrees).
- `.dec.deg` simply returns the declination in decimal degrees, which is the default angular unit for declination.

Note that `.ra.hour` returns a plain Python float — it strips away the unit metadata and gives only the numerical value in hours. This can be useful for certain formatting or external tools, but lacks the unit safety of `Quantity` objects.

By printing `type(coord.ra.hour)`, we verify that the result is no longer a `Quantity` but a standard `float`.

```
print(coord.ra.hms)
print(coord.dec.dms)
```

```
hms_tuple(h=np.float64(9.0), m=np.float64(12.0), s=np.float64(2.600000000004883))
dms_tuple(d=np.float64(-64.0), m=np.float64(-51.0), s=np.float64(-44.99999999998977))
```

In this cell, we access the `.ra.hms` attribute of the `SkyCoord` object.

The `.hms` property returns the **hours**, **minutes**, and **seconds** representation of the Right Ascension (RA) as a named tuple:

```
hms = (hour, minute, second)
```

This format is especially useful in astronomical catalogs, telescope pointing systems, and human-readable reports where RA is traditionally expressed in sexagesimal units.

Unlike `.ra.hour`, which returns a single float in hours, `.ra.hms` provides a full breakdown for easy formatting or display:

- `coord.ra.hms.h`: hours
- `coord.ra.hms.m`: minutes
- `coord.ra.hms.s`: seconds

The resulting values are pure floats, not `Quantity` objects, so units are implied by context rather than enforced.



```
print(coord.ra.radian, coord.dec.radian)
```

```
2.4087434450878074 -1.132064186074822
```

In this cell, we access the `.radian` attribute of the Right Ascension and Declination stored in the `SkyCoord` object.

- `.ra.radian` returns the Right Ascension in **radians**
- `.dec.radian` returns the Declination in **radians**

Both values are returned as standard Python floats (not `Quantity` objects), and the unit is implicitly understood to be **radians**.

This representation is useful when working with trigonometric functions from libraries like NumPy or SciPy, which expect input angles in radians. However, because units are no longer attached, it's important to track units manually and avoid unit mismatches.

```
print(coord.to_string('decimal'))
print(coord.to_string('dms'))
print(coord.to_string('hmsdms'))
```

```
138.011 -64.8625
138d00m39s -64d51m45s
09h12m02.6s -64d51m45s
```

In this cell, we use the `.to_string()` method of the `SkyCoord` object to display the coordinates in various string formats.

The method formats the Right Ascension and Declination into human-readable strings using different notations:

- `coord.to_string('decimal')`  
Returns RA and Dec as decimal degrees, separated by a space.  
Example: '138.0108 -64.8625'
- `coord.to_string('dms')`  
Returns both RA and Dec in **degrees:minutes:seconds (DMS)** format.  
Example: '138d00m38.88s -64d51m45s'  
This is useful when reporting coordinates in formats commonly used in catalogs or observational logs.
- `coord.to_string('hmsdms')`  
Formats RA in **hours:minutes:seconds (HMS)** and Dec in **DMS**.  
Example: '09h12m2.60s -64d51m45s'  
This is the traditional astronomical format for sky coordinates.

These formatting options are especially useful when exporting results, generating catalogs, or matching coordinate formats from external datasets.

```
print(coord.galactic)
```

```
<SkyCoord (Galactic): (l, b) in deg
(282.19166541, -11.25257989)>
```

In this cell, we access the `.galactic` attribute of the `SkyCoord` object to obtain the position of the same object (NGC 2808) in the **Galactic coordinate system**.

The `.galactic` attribute returns a new `SkyCoord` object representing the same physical location on the sky, but in terms of:

- **Galactic Longitude (l)** — measured in degrees, increasing in the direction of the Galactic rotation
- **Galactic Latitude (b)** — measured in degrees, positive above the Galactic plane

This transformation is handled internally by `astropy`, using the built-in coordinate transformation graph. The result remains a fully unit-aware `SkyCoord` object and can be further queried, transformed, or formatted like any other coordinate.

Galactic coordinates are especially useful when studying the structure and components of the Milky Way.

```
NGC_2808 = SkyCoord.from_name("NGC 2808")
print(NGC_2808)
```

```
<SkyCoord (ICRS): (ra, dec) in deg
(138.01291667, -64.8635)>
```

In this cell, we use `SkyCoord.from_name()` to automatically retrieve the celestial coordinates of **NGC 2808** by querying the **SIMBAD** astronomical database.

This method sends a network request and resolves the name "NGC 2808" to its ICRS (RA/Dec) coordinates. The returned result is a standard `SkyCoord` object, ready for further use.

This feature is especially helpful when working with named astronomical objects, saving time and reducing errors compared to manually entering coordinates.

**Pro tip:** Even if you're banned from Google or can't load Wikipedia, you can still resolve star cluster positions like a pro! Thanks to `astropy` and SIMBAD!

```
ra_deg = np.array([6.023625, 13.18875, 78.528625, 81.046667, 138.0108,
 201.6975, 229.63875, 250.4225, 265.175833, 322.493958])
dec_deg = np.array([-72.081283, -26.582806, -40.046875, -24.524139, -64.8625,
 -47.479722, 2.081, 36.460278, -53.674694, 12.16625])

cluster_names = np.array([
 "NGC 104", # 47 Tucanae
 "NGC 288",
 "NGC 1851",
 "NGC 1904", # M79
 "NGC 2808",
 "NGC 5139", # Omega Centauri
 "NGC 5904", # M5
 "NGC 6205", # M13
 "NGC 6397",
 "NGC 7078" # M15
])

dist_kpc = np.array([
 4.5, 8.9, 12.1, 12.9, 9.6,
 5.2, 7.5, 7.7, 2.3, 10.4
]) #GAIA/DR3 distances

cluster_coords = SkyCoord(ra=ra_deg * u.deg, dec=dec_deg * u.deg, distance = dist_kpc*u.kpc, frame=frame)
print(cluster_coords) #RA and Dec in degrees for 10 globular clusters (J2000)
```

```
<SkyCoord (ICRS): (ra, dec, distance) in (deg, deg, kpc)
 [(6.023625, -72.081283, 4.5), (13.18875 , -26.582806, 8.9),
 (78.528625, -40.046875, 12.1), (81.046667, -24.524139, 12.9),
 (138.0108 , -64.8625 , 9.6), (201.6975 , -47.479722, 5.2),
 (229.63875 , 2.081 , 7.5), (250.4225 , 36.460278, 7.7),
 (265.175833, -53.674694, 2.3), (322.493958, 12.16625 , 10.4)]>
```

### 24.1.1 Using NumPy Arrays with SkyCoord

In this cell, we demonstrate how to create a `SkyCoord` object using `NumPy` arrays for multiple celestial objects.

We define: - An array of **Right Ascension** (`ra_deg`) values in degrees  
 - An array of **Declination** (`dec_deg`) values in degrees

- An array of **distances** (`dist_kpc`) in kiloparsecs
- A list of **names** for 10 well-known globular clusters

By passing these arrays directly into the `SkyCoord` constructor along with their appropriate units, we create a structured collection of coordinates. Each element includes full 3D spatial information: angular position (RA, Dec) and distance.

This approach allows efficient, vectorized operations on many coordinates at once and is ideal for working with astronomical catalogs and large datasets.

```
sep_3d = cluster_coords[0].separation_3d(cluster_coords[9])
sep_ang = cluster_coords[0].separation(cluster_coords[9])
print(f"3D distance between {cluster_names[0]} and {cluster_names[9]}: {sep_3d.to(u.kpc)}")
print(f"Angular separation {cluster_names[0]} and {cluster_names[9]}: {sep_ang.to(u.deg)}")
```

```
3D distance between NGC 104 and NGC 7078: 11.26 kpc
Angular separation NGC 104 and NGC 7078: 89.00 deg
```

### 24.1.2 Measuring 3D Distance Between Two Globular Clusters

In this cell, we compute the **true spatial separation** between two globular clusters using their full 3D coordinates.

Specifically, we call:

```
cluster_coords[0].separation_3d(cluster_coords[9])
```

This computes the **Euclidean distance in 3D space** between the first cluster (NGC 104) and the last cluster (NGC 7078) in our list.

Unlike angular separation (which only considers position on the sky), `.separation_3d()` uses both: - Angular coordinates (RA, Dec) - Radial distances (in kiloparsecs)

The result is returned as a `Quantity` with physical units (e.g., `kpc`), giving the **true line-of-sight separation** between the two objects in three-dimensional space.

### 24.1.3 Working with Angles in `astropy.coordinates`

In astronomy, angles are everywhere — from sky coordinates (RA/Dec, Alt/Az) to angular separations, field of view, proper motions, and more. To handle such angular quantities with both **unit safety** and **astronomical precision**, `astropy.coordinates` provides the `Angle` class.

The `Angle` object is a specialized form of `Quantity` designed specifically for angular values. It allows:

- Flexible creation from degrees, radians, hours, DMS, or HMS strings
- Easy conversion between angular formats
- Safe arithmetic (addition, subtraction, wrapping)
- High-precision comparison and transformations
- Use of custom wrapping and normalization (e.g. `wrap_at(180*u.deg)`)

Because `Angle` is fully unit-aware and interoperable with `SkyCoord`, it's ideal for precise and readable angular computations — whether you're rotating frames, computing angular distances, or converting between coordinate systems.

In short, the `Angle` class gives you both the **power of `astropy.units`** and the **expressiveness of astronomical angular conventions**, all in one object.

```
from astropy.coordinates import Angle
Angle('10.2345d') # String with 'd' abbreviation for degrees
Angle(['10.2345d', '-20d']) # Array of strings
Angle('1:2:30.43 degrees') # Sexagesimal degrees
Angle('1 2 0 hours') # Sexagesimal hours
Angle(np.arange(1., 8.), unit=u.deg) # Numpy array from 1..7 in degrees
Angle('1°2'3') # Unicode degree, arcmin and arcsec symbols
Angle('1°2'3 N') # Unicode degree, arcmin, arcsec symbols and direction
Angle('1d2m3.4s') # Degree, arcmin, arcsec.
Angle('1d2m3.4sS') # Degree, arcmin, arcsec, direction.
Angle('-1h2m3s') # Hour, minute, second
Angle('-1h2m3sW') # Hour, minute, second, direction
Angle(10.2345 * u.deg) # From a Quantity object in degrees
Angle(Angle(10.2345 * u.deg)) # From another Angle object
```

```
<Angle 10.2345 deg>
```

```
a1 = Angle("10d30m")
a2 = Angle("2d15m")
a3 = a1 + a2
a4 = a1 - a2
print(f"Sum: {a3}")
print(f"Difference: {a4}")
```

```
Sum: 12.75 deg
Difference: 8.25 deg
```

#### 24.1.4 Angle Arithmetic

**Angle** objects support standard arithmetic: - Addition and subtraction - Multiplication/division by scalars - Array operations

This is especially useful when calculating angular distances or adjusting positions.

## Chapter 25

# Introduction to astropy.table

```
!pip install -q astropy
```

```
from astropy.table import QTable
import astropy.units as u
import numpy as np

star_names = np.array(["Proxima Centauri", "Alpha Centauri A", "Barnard's Star", "Wolf 359", "Luy
distances = np.array([4.24, 4.37, 5.96, 7.86, 8.73]) * u.lyr
apparent_mags = np.array([11.13, -0.01, 9.54, 13.5, 12.6]) * u.mag
absolute_mags = np.array([15.5, 4.38, 13.2, 16.6, 15.0]) * u.mag
temperatures = np.array([3042, 5790, 3134, 2800, 2650]) * u.K
radii = np.array([0.14, 1.22, 0.18, 0.16, 0.12]) * u.R_sun
spectral_types = np.array(["M5.5Ve", "G2V", "M4.0Ve", "M6.0Ve", "M5.5Ve"])

stars_table = QTable(
 [star_names, distances, apparent_mags, absolute_mags, temperatures, radii, spectral_types],
 names=["Star", "Distance", "App_Mag", "Abs_Mag", "Temperature", "Radius", "Spectral_Type"], n
stars_table
```

```
<QTable length=5>
 Star Distance App_Mag Abs_Mag Temperature Radius Spectral_Type
 lyr mag mag K solRad
str16 float64 float64 float64 float64 float64 str6

```

Proxima Centauri	4.24	11.13	15.5	3042.0	0.14	M5.5Ve
Alpha Centauri A	4.37	-0.01	4.38	5790.0	1.22	G2V
Barnard's Star	5.96	9.54	13.2	3134.0	0.18	M4.0Ve
Wolf 359	7.86	13.5	16.6	2800.0	0.16	M6.0Ve
Luyten 726-8A	8.73	12.6	15.0	2650.0	0.12	M5.5Ve

### 25.0.1 Description of the Table Construction

In this cell, we construct a `QTable` using NumPy arrays for each column:

- The variable `star_names` holds the names of the five nearby stars (as strings).
- `distances` contains their distances from Earth in light-years, defined with units using `astropy.units`.
- `apparent_mags` and `absolute_mags` represent the stars' apparent and absolute magnitudes, respectively.
- `temperatures` stores the effective temperatures in kelvin.
- `radii` contains the stellar radii, each expressed in solar radii (`R_sun`).
- `spectral_types` holds the stars' spectral classifications as strings.

All arrays are passed into `QTable` as a list of columns along with their corresponding names. Since many columns are physical quantities, they retain unit-awareness, which allows for further physical calculations later on (e.g., computing luminosity).

```
print(stars_table)
```

Star	Distance lyr	App_Mag mag	Abs_Mag mag	Temperature K	Radius solRad	Spectral_Type
Proxima Centauri	4.24	11.13	15.5	3042.0	0.14	M5.5Ve
Alpha Centauri A	4.37	-0.01	4.38	5790.0	1.22	G2V
Barnard's Star	5.96	9.54	13.2	3134.0	0.18	M4.0Ve
Wolf 359	7.86	13.5	16.6	2800.0	0.16	M6.0Ve
Luyten 726-8A	8.73	12.6	15.0	2650.0	0.12	M5.5Ve

```
stars_table['Temperature'].info.format = '5.0f'
print(stars_table)
```

Star	Distance lyr	App_Mag mag	Abs_Mag mag	Temperature K	Radius solRad	Spectral_Type
Proxima Centauri	4.24	11.13	15.5	3042.0	0.14	M5.5Ve
Alpha Centauri A	4.37	-0.01	4.38	5790.0	1.22	G2V
Barnard's Star	5.96	9.54	13.2	3134.0	0.18	M4.0Ve
Wolf 359	7.86	13.5	16.6	2800.0	0.16	M6.0Ve
Luyten 726-8A	8.73	12.6	15.0	2650.0	0.12	M5.5Ve



Proxima Centauri	4.24	11.13	15.5	3042	0.14	M5.5Ve
Alpha Centauri A	4.37	-0.01	4.38	5790	1.22	G2V
Barnard's Star	5.96	9.54	13.2	3134	0.18	M4.0Ve
Wolf 359	7.86	13.5	16.6	2800	0.16	M6.0Ve
Luyten 726-8A	8.73	12.6	15.0	2650	0.12	M5.5Ve

```
stars_table.info
```

```
<QTable length=5>
 name dtype unit format class

 Star str16
Distance float64 lyr
App_Mag float64 mag
Abs_Mag float64 mag
Temperature float64 K 5.0f Quantity
 Radius float64 solRad
Spectral_Type str6
```

```
stars_table.colnames #Names of columns in table
```

```
['Star',
 'Distance',
 'App_Mag',
 'Abs_Mag',
 'Temperature',
 'Radius',
 'Spectral_Type']
```

```
stars_table.meta
```

```
{'name': 'stars_table'}
```

```
print(type(stars_table['Temperature'])) #class Quantity
```

```
<class 'astropy.units.quantity.Quantity'>
```

```
print(stars_table['Temperature']) #Column Temperature'
print(stars_table['Temperature'][0]) # Row 0 of column 'Temperature'
print(stars_table[0]['Temperature']) # Column 'Temperature' of row 0
```

```
[3042. 5790. 3134. 2800. 2650.] K
3042.0 K
3042.0 K
```

```
print(stars_table['Temperature'].value) #Knowledge about Quantity still useful here
print(type(stars_table['Temperature'].value))
```

```
[3042. 5790. 3134. 2800. 2650.]
<class 'numpy.ndarray'>
```

## 25.0.2 Accessing Data in a QTable

This example shows different ways to access data from a QTable:

- `stars_table['Temperature']`  
Accesses the entire **Temperature** column as a Quantity array with units.
- `stars_table['Temperature'][0]`  
Accesses the **first value** in the Temperature column — a single scalar quantity with units.
- `stars_table[0]['Temperature']`  
Accesses the **Temperature field** from the **first row**, which returns the same result as above.

Both `stars_table['col'][row]` and `stars_table[row]['col']` are valid, but the second form (`table[row]['col']`) is useful when accessing multiple fields from a single row as a record.

This flexibility allows convenient slicing and indexing for both **row-wise** and **column-wise** operations.

```
print(stars_table[0:2])
```

Star	Distance lyr	App_Mag mag	Abs_Mag mag	Temperature K	Radius solRad	Spectral_Type
Proxima Centauri	4.24	11.13	15.5	3042	0.14	M5.5Ve
Alpha Centauri A	4.37	-0.01	4.38	5790	1.22	G2V

```
print(stars_table['Temperature', 'Distance'])
```

Temperature	Distance
K	lyr
-----	-----
3042	4.24
5790	4.37
3134	5.96
2800	7.86
2650	8.73

```
print(stars_table['Temperature'].unit)
```

```
K
```

```
from astropy.constants import sigma_sb

R = stars_table['Radius']
T = stars_table['Temperature']

luminosity = 4 * np.pi * R**2 * sigma_sb * T**4
luminosities_Lsun = luminosity.to(u.L_sun)

stars_table['Luminosity'] = luminosities_Lsun #add new column to table
```

```
stars_table
```

<QTable length=5>					
Star	Distance	App_Mag	...	Spectral_Type	Luminosity
	lyr	mag	...		sollum
str16	float64	float64	...	str6	float64
-----	-----	-----	...	-----	-----
Proxima Centauri	4.24	11.13	...	M5.5Ve	0.0015121211520600123
Alpha Centauri A	4.37	-0.01	...	G2V	1.5070497682487838
Barnard's Star	5.96	9.54	...	M4.0Ve	0.0028160130105810885
Wolf 359	7.86	13.5	...	M6.0Ve	0.0014176393805220048
Luyten 726-8A	8.73	12.6	...	M5.5Ve	0.0006397932680773995

### 25.0.3 Calculating Stellar Luminosity and Adding It to the Table

To calculate the luminosity of stars, we use the **Stefan–Boltzmann law**:

$$L = 4 \pi R^2 T^4$$

Using `astropy.constants` and `astropy.units`, we ensure that all values carry correct physical units. This not only simplifies calculations but also eliminates unit mismatch errors.

After computing the luminosity, we convert the result to solar luminosities and store it in a new column named `Luminosity` in our `stars_table`.

This process illustrates how real astrophysical equations can be applied directly in Python using unit-aware data structures.

## 25.1 Summary of `astropy.table`

In this module, we explored basic capabilities of the `astropy.table` subpackage a flexible, unit-aware system for working with tabular astronomical data. We created `QTable` objects using NumPy arrays, computed derived quantities like stellar luminosity, and modified table structures using column operations.

Having established this foundation, you are now ready to work with tabular data in realistic scientific workflows, including the handling of **FITS tables** and the use of `astroquery` for retrieving remote datasets.

---

### 25.1.1 Recommended Topics for Further Study

#### 25.1.1.1 Modifying a Table

Learn how to:

- Add or remove rows
- Rename or reorder columns
- Update values within a table

Documentation: [Modifying Tables](#)

---

#### 25.1.1.2 Table Operations

Understand how to:

- Sort and filter rows
- Group data based on column values

- Stack and join multiple tables

Documentation: Table Operations

---

#### 25.1.1.3 Masking and Missing Values

Astronomical data frequently include missing or uncertain values. `astropy.table` supports:

- Masked values (e.g., NaNs or flagged entries)
- Propagation of missing data through operations
- Handling and filtering of incomplete datasets

Documentation: Masking and Missing Values

---

#### 25.1.2 Interoperability with pandas

`astropy.table` supports seamless conversion to and from `pandas.DataFrame`, enabling integration with the broader Python data science ecosystem.

Learn more:

Using Tables with pandas

---

#### 25.1.3 Saving and Loading Tables

You can easily save `Table` or `QTable` objects to a wide variety of file formats using the `write()` method and read them back using `read()`.

Supported formats include: - ASCII (e.g., CSV, TSV, fixed-width) - ECSV (Enhanced CSV with metadata and units) - FITS (Flexible Image Transport System) - HDF5 - VOTable - LaTeX and HTML for presentation

Read more: - Reading Tables - Writing Tables

---

### 25.1.4 Next Steps

Now that you are familiar with tables, coordinates, and units, you can begin exploring:

- Working with **FITS** files and binary tables
- Querying astronomical databases using **astroquery**
- Preparing tables for modeling, plotting, or catalog cross-matching

## Chapter 26

# CDS Services

```
!pip install -q astroquery ipyaladin mocpy
```

```
11.1/11.1 MB 39.3 MB/s eta 0:00:00
1.2/1.2 MB 15.0 MB/s eta 0:00:00
1.3/1.3 MB 15.8 MB/s eta 0:00:00
1.1/1.1 MB 18.8 MB/s eta 0:00:00
220.7/220.7 kB 8.5 MB/s eta 0:00:00
842.5/842.5 kB 9.3 MB/s eta 0:00:00
1.6/1.6 MB 15.6 MB/s eta 0:00:00
```

```
import numpy as np
import pandas as pd
from astropy.coordinates import Angle, SkyCoord
from astroquery.vizier import Vizier
from astroquery.cds import cds
import astropy.units as u
from ipyaladin import Aladin, Marker
from pathlib import Path
import pyvo
import mocpy
from astroquery.gaia import Gaia
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
WARNING: AstropyDeprecationWarning: The ``cds`` module has been moved to astroquery.mocserver, ar
WARNING:astroquery:AstropyDeprecationWarning: The ``cds`` module has been moved to astroquery.moc
```

Important: Gaia archive will be intermittently unavailable due to scheduled maintenance

```
from ipyaladin import __version__, __aladin_lite_version__
print("version:", __version__, "running Aladin Lite:", __aladin_lite_version__)
```

```
version: 0.6.0 running Aladin Lite: 3.6.5
```

```
aladin = Aladin(
 survey="CDS/P/DECaPS/DR1/color",
 show_coo_grid=True,
 target="galactic center",
 coo_frame="galactic",
 fov=40,
 height=600,
)
aladin
```

```
Aladin(coo_frame='galactic', survey='CDS/P/DECaPS/DR1/color')
```

```
aladin.target = "Antennae"
```

```
aladin = Aladin(fov=20, reticle_size=64, reticle_color="#ff89ff")
aladin
```

```
Aladin()
```

```
print('Hello there, the packages are imported and we are ready to go :)')
```

```
Hello there, the packages are imported and we are ready to go :)
```

## 26.1 Start Aladin Lite

We start this tutorial by initialising Aladin Lite. In the first line of code we tell Python that the variable `aladin` is now assigned to an Aladin Lite widget. This widget is supposed to **center** on the Antennae galaxies, set a **field of view** of 0.7deg and show colour images from the DSS **survey**. Typing `aladin` in the second line of code, tells the notebook to display the widget.

As with any Aladin Lite implementation, you can interact with this widget: - to zoom in and out place you mouse pointer on top of the image and scroll. -



with you can select other image surveys and manage the current view. - if you like to look at another target, you can use the search field to get there.

These interactions can also be steered by changing properties of the variable `aladin`. If for example you wanted to focus on M101 instead of the Antennae galaxies, do:

```
aladin.target = 'M101'
```

## 26.2 Find and download tables from Vizier

Besides using Aladin we can also query Vizier from within this Jupyter notebook. To do so we use the `Vizier` subpackage from the `astroquery` package. As we remember from before, the catalogue of peculiar galaxies was called “Arp’s peculiar galaxies” and was published by Webb. Hence, we ask `Vizier` to find all catalogues that have a match with the keywords ‘Arp Webb’ and write the result into the variable `catalog_list_arp` (first line of code). Then we tell Python to `print` out the query result in a readable way (second line of code).

```
catalog_list_arp = Vizier.find_catalogs('Arp')
for k, v in catalog_list_arp.items():
 print(k, ': ', v.description)
```

```
VII/74A : Atlas of Peculiar Galaxies (Arp 1966)
VII/192 : Arp's Peculiar Galaxies (Webb 1996)
```

Alternatively you can also search for the table with `pyvo` using the Table Access Protocol (TAP) to query Vizier. In this case, you first need to define the TAP endpoint of Vizier and then send a query written in ADQL (an SQL-like query language optimised for astronomy) to this endpoint. Finally we again print out the result of this query.

```
tap_vizier = pyvo.dal.TAPService('http://tapvizier.u-strasbg.fr/TAPVizier/tap')
query = """SELECT * FROM tap_schema.tables
 WHERE table_name LIKE '%arp%' """
catalog_list_arp = tap_vizier.search(query).to_table()
catalog_list_arp['table_name', 'description']
```

```
<Table length=33>
 table_name ...
 object ...

 VII/192/arplist ...
```

```

J/A+A/664/A53/harps03 ...
J/A+A/669/A39/sunharps ...
J/MNRAS/372/69/varphot ...
J/A+A/684/A22/harps ...
J/A+A/664/A53/harps15 ...
J/A+A/693/A297/harps15 ...
J/MNRAS/511/1043/harpsccf ...
J/A+A/693/A297/harps03 ...
... ..
J/A+A/666/A143/harps ...
VII/192/arpord ...
J/MNRAS/523/3069/harpsnr ...
J/A+A/666/A143/harps-a ...
J/A+A/612/A95/rvharps ...
J/A+A/686/A301/harps ...
J/other/RMxAA/40.209/varphot ...
J/ApJ/898/L11/harps ...
I/256/varpos ...
J/A+A/691/A298/harpsn ...

```

The catalogue that we are interested in today is “VII/192” and its description is “Arp’s Peculiar Galaxies (Webb 1996)”. In the TAP query we already see that there are two tables in this catalogue: VII/192/arp`list` and VII/192/arp`ord`. To be able to work with these data, let’s load them into this notebook.

Again we can use either `astroquery` or `pyvo`. Which one you chose is up to you and which one you like better. Attention **Spoiler:** over the week you’ll learn more about TAP and ADQL.

In order to get the full catalogue with `astroquery`, we first set the row limit to infinite (i.e. -1 in the notation of this package) and then ask `Vizier` to write the content of the catalogue into the variable `catalogs_arp` (second line):

```

Vizier.ROW_LIMIT = -1
catalogs_arp = Vizier.get_catalogs('VII/192')

```

Now let’s inspect, what we got:

```
print(catalogs_arp)
```

```

TableList with 2 tables:
 '0:VII/192/arpord' with 12 column(s) and 338 row(s)
 '1:VII/192/arpelist' with 13 column(s) and 592 row(s)

```

As we have seen in the first part of the tutorial, the catalogue “Arp’s Peculiar Galaxies (Webb 1996)” comes with two tables: arp`ord` and arp`list`. As you can

see from the printout `Vizier` has downloaded both of them. However, we are still only interested in the arplist (index 1 in the `TableList`). Therefore, we make Python write only the arplist table into a new variable `table_arplist` (first line) and then display the table (second line):

```
table_arplist = catalogs_arp[1]
table_arplist
```

```
<Table length=592>
 Arp Name VT u_VT dim1 ... RAJ2000 DEJ2000 Simbad NED
 mag arcmin ...
int16 str16 float32 str1 float32 ... str10 str9 str6 str3

 249 UGC 12891 16.2 -- ... 00 00 19.3 +22 59 26 Simbad NED
 249 UGC 12891 16.2 1.3 ... 00 00 21.6 +22 59 42 Simbad NED
 112 NGC 7805 13.3 1.2 ... 00 01 26.9 +31 26 02 Simbad NED
 112 NGC 7806 13.5 1.1 ... 00 01 30.2 +31 26 33 Simbad NED
 130 IC 5378 15.6 0.5 ... 00 02 37.7 +16 39 08 Simbad NED
 130 IC 5378 15.3 -- ... 00 02 37.8 +16 38 37 Simbad NED
 51 MGC-02-01-24 15.0 0.8 ... 00 06 16.8 -13 26 53 Simbad NED
 144 NGC 7828 14.4 0.9 ... 00 06 27.1 -13 24 58 Simbad NED
 144 NGC 7829 14.6 0.7 ... 00 06 29.0 -13 25 15 Simbad NED

 295 ARP 295 14.6 0.9 ... 23 42 04.1 -03 35 21 Simbad NED
 86 NGC 7752 14.3 0.8 ... 23 46 58.5 +29 27 32 Simbad NED
 86 NGC 7753 12.0 3.3 ... 23 47 04.7 +29 29 01 Simbad NED
 68 NGC 7757 12.7 2.5 ... 23 48 45.1 +04 10 24 Simbad NED
 323 NGC 7783 13.0 1.3 ... 23 54 10.0 +00 22 58 Simbad NED
 323 NGC 7783B 14.0 0.4 ... 23 54 12.0 +00 22 38 Simbad NED
 262 MCG+03-01-003 14.6 0.5 ... 23 56 43.8 +16 48 27 Simbad NED
 262 UGC 12856 13.2 2.4 ... 23 56 44.9 +16 48 45 Simbad NED
 50 IC 1520 14.0 0.5 ... 23 57 58.0 -14 01 48 Simbad NED
```

Nice, we got the data we want. Before we move on, let's have a look at how we can use `pyVO` to get the same table. Just for safety we define again the `VizieR` TAP endpoint. It is the same one as above, but if the cells of this notebook are not run one after another, this safety measure ensure we always query the right service.

Then we write a simple query telling `VizieR` that we want all the rows and all the columns from the table `VII/192/arplist`. Note that we have to encompass the table name with quotation marks due to the special character `/` in the table name.

```
tap_vizier = pyvo.dal.TAPService('http://tapvizier.u-strasbg.fr/TAPVizieR/tap')
query = """SELECT * FROM "VII/192/arplist" """
table_arplist = tap_vizier.search(query).to_table()
table_arplist
```

<Table length=592>

recno	Arp	Name	...	Uchart	RAJ2000	DEJ2000
			...		deg	deg
int32	int16	object	...	int16	float64	float64
112	333	NGC 1024	...	175	39.80083333333333	10.847222222222221
120	200	NGC 1134	...	175	43.42124999999999	13.015277777777776
119	190	UGC 02320	...	--	42.58291666666666	12.889444444444443
89	290	IC 0195	...	--	30.93583333333333	14.708611111111109
90	290	IC 0196	...	--	30.95833333333333	14.739722222222222
111	258	UGC 02140A	...	--	39.78874999999999	18.367499999999996
110	258	UGC 02140	...	--	39.77583333333333	18.382777777777775
109	258	Hickson 18C	...	--	39.77458333333333	18.388333333333332
108	258	Hickson 18D	...	--	39.76916666666666	18.393888888888885
...	...	...	...	...	...	...
464	261	ARP 261	...	--	222.38624999999999	-10.163055555555554
463	261	ARP 261	...	--	222.38624999999999	-10.156666666666665
549	14	NGC 7314	...	347	338.94041666666664	-26.050833333333333
547	93	NGC 7284	...	347	337.15	-24.844166666666663
548	93	NGC 7285	...	347	337.15833333333333	-24.840833333333333
546	226	NGC 7252	...	347	335.18666666666666	-24.678611111111106
541	325	ESO 601- G 018	...	--	331.60083333333333	-21.079166666666662
540	325	ESO 601- G 018	...	--	331.59666666666664	-21.077499999999997
539	325	ESO 601- G 018	...	--	331.5925	-21.072499999999998

There we go, we now know two nifty ways to get VizieR tables with Python. Obviously there are many other things you can do with this tools. For example if you set your TAP endpoint to <http://simbad.u-strasbg.fr:80/simbad/sim-tap> instead of the VizieR endpoint, you can query SIMBAD just like that.

Note that not all rows but only the first and last 10 rows for all columns are shown in the display. To finish off this little excursion, we now want to visualise the location of the entries of this table in our Aladin Lite widget. To do so, we tell Python to take the variable `aladin` (remember that this was the name of the widget) and add the table `table_arplist` to it:

```
aladin.add_table(table_arplist)
```

Now scroll back up to the Aladin Lite widget. You will find that the location of the sources in the arplist table are marked with coloured symbols. Again you

can zoom in and out to look at the different sources. If you click on one of the colour symbols, you will be able to see the corresponding row in the bottom of the Aladin Lite widget.

Note if you are using Jupyter Lab instead of single Jupyter notebooks, you can open two notebooks that share a kernel. Then one notebook could contain your working environment, where you get and work on your tables ect. The other notebook could contain the Aladin Lite widget, and just remain visible all the time. This way there would be no need to scroll up and down.

## 26.3 Bonus: MOCs and Python

As in the .pdf tutorial we can use MOCs to find out whether any of these galaxies has been observed both by SDSS and GALEX. For this task we use the `cds` module of `astroquery` and in the background the functionalities of `MOCpy` to get the intersection of the MOCs and to filter a table by a MOC. First let's query the MOC server for all things SDSS and all things GALEX.

```
info_sdss = cds.find_datasets(meta_data="ID=*SDSS*")
info_sdss[['ID']]
```

WARNING: AstropyDeprecationWarning: 'find\_datasets' is replaced by 'query\_region' which has a new

```
<Table length=14>
 ID
 str25

CDS/J/ApJ/749/10/SDSS-obs
 CDS/P/HLA/SDSSg
 CDS/P/HLA/SDSSr
 CDS/P/HLA/SDSSz
 CDS/P/HST/SDSSg
 CDS/P/HST/SDSSr
 CDS/P/HST/SDSSz
 CDS/P/SDSS9/color
 CDS/P/SDSS9/color-alt
 CDS/P/SDSS9/g
 CDS/P/SDSS9/i
 CDS/P/SDSS9/r
 CDS/P/SDSS9/u
 CDS/P/SDSS9/z
```

```
info_galex = cds.find_datasets(meta_data="ID=*GALEX*")
info_galex[['ID']]
```

WARNING: AstropyDeprecationWarning: 'find\_datasets' is replaced by 'query\_region' which

```
<Table length=6>
 ID
 str24

CDS/P/GALEXGR6/AIS/FUV
CDS/P/GALEXGR6/AIS/NUV
CDS/P/GALEXGR6/AIS/color
CDS/P/GALEXGR6_7/FUV
CDS/P/GALEXGR6_7/NUV
CDS/P/GALEXGR6_7/color
```

Now that we know what the relevant data sets are called, we can move on to download the corresponding MOCs:

### Querying Gaia Data with Astroquery

This Jupyter notebook demonstrates how to use astroquery to access data from the Gaia mission. We'll cover:

- Installing required packages
- Basic Gaia queries
- Cone searches
- Cross-matching with other catalogs
- Saving results
- Simple visualization

```
login = False

if login:
 Gaia.login(user='your_username', password='your_password')
else:
 print("Using anonymous access (query limits may apply)")
```

Using anonymous access (query limits may apply)

```
ngc1960 = SkyCoord.from_name('NGC 1960')
print(f"NGC 1960 coordinates: RA = {ngc1960.ra.degree:.4f}°, Dec = {ngc1960.dec.degree:.4f}°")
```

NGC 1960 coordinates: RA = 84.0840°, Dec = 34.1350°

```
Cone search parameters
radius = 0.5 * u.deg # Search radius (cluster is about 10' across)
max_mag = 18 # Maximum G magnitude

ADQL query for basic cluster members
query_basic = f"""
SELECT
source_id, ra, dec, parallax, pmra, pmdec,
phot_g_mean_mag, phot_bp_mean_mag, phot_rp_mean_mag,
radial_velocity,
astrometric_excess_noise,
ruwe
FROM gaiadr3.gaia_source
WHERE 1=CONTAINS(
 POINT('ICRS', ra, dec), CIRCLE('ICRS', {ngc1960.ra.degree}, {ngc1960.dec.degree}, {radius.val}
AND phot_g_mean_mag < {max_mag}
"""

Execute the query
job_basic = Gaia.launch_job(query_basic)
results_basic = job_basic.get_results()

print(f"Found {len(results_basic)} stars in the region")
results_basic[:5].show_in_notebook() # Display first 5 results
```

Found 2000 stars in the region

WARNING: AstropyDeprecationWarning: show\_in\_notebook() is deprecated as of 6.1 and to create interactive tables it is recommended to use dedicated tools like:

- <https://github.com/bloomberg/ipydatagrid>
- [https://docs.bokeh.org/en/latest/docs/user\\_guide/interaction/widgets.html#datatable](https://docs.bokeh.org/en/latest/docs/user_guide/interaction/widgets.html#datatable)
- <https://dash.plotly.com/datatable> [warnings]

<IPython.core.display.HTML object>

```
Basic query to get some Gaia data
query = """
SELECT TOP 2000
source_id, ra, dec, parallax, pmra, pmdec, phot_g_mean_mag, phot_bp_mean_mag, phot_rp_mean_mag, t
FROM gaiadr3.gaia_source
WHERE parallax > 10
ORDER BY phot_g_mean_mag
```

```

"""

job = Gaia.launch_job(query)
results = job.get_results()

Display the first few results
results[:5]

```

```

<Table length=5>
 source_id ra ... phot_rp_mean_mag bp_rp
 deg ... mag mag
 int64 float64 ... float32 float32

418551920284673408 10.12724197930297 ... 1.8435026 1.1434835
4357027756659697664 243.58621066034064 ... 1.9189217 1.5633104
4993479684438433792 6.57215550145885 ... 2.032224 1.2694821
4038055447778237312 274.4060904518451 ... 1.9154117 1.8272674
1279798794197267072 221.24648617076684 ... 2.1027422 1.1840065

```

```

results.write('m45_all.fits',format="fits", overwrite=True)

Save to CSV
results.write('m45_all.csv', format="csv", overwrite=True)

print("Saved results to FITS and CSV formats")

```

Saved results to FITS and CSV formats

```

%%html
<div style="text-align:center;">
<iframe src="https://gea.esac.esa.int/archive/" width="960" height="740"></iframe>
</div>

```

<IPython.core.display.HTML object>

```

#Now we can read the csv file into a pandas dataframe

all_stars = pd.read_csv('m45_all.csv') # I renamed my csv file to 'm5.csv' and put it

#Checking the top few rows of the data and the number of rows and columns
print("(Rows, Columns) =", all_stars.shape)
all_stars.head()

```



```
(Rows, Columns) = (2000, 10)
```

	source_id	ra	dec	parallax	pmra \
0	418551920284673408	10.127242	56.537189	14.090976	49.125646
1	4357027756659697664	243.586211	-3.694968	20.411292	-45.340019
2	4993479684438433792	6.572156	-42.307820	39.918258	176.268293
3	4038055447778237312	274.406090	-36.762429	23.885852	-131.804070
4	1279798794197267072	221.246486	27.074316	13.826673	-50.818256

	pmdec	phot_g_mean_mag	phot_bp_mean_mag	phot_rp_mean_mag	bp_rp
0	-31.595374	1.942524	2.986986	1.843503	1.143484
1	-144.432722	2.016425	3.482232	1.918922	1.563310
2	-398.872135	2.089978	3.301706	2.032224	1.269482
3	-166.308930	2.116495	3.742679	1.915412	1.827267
4	21.023521	2.183352	3.286749	2.102742	1.184006

```
plotting the skyplot
```

```
sns.set(rc={'figure.figsize':(8.7,6.27)})
```

```
skyplot = sns.scatterplot(x='ra', y='dec', data = all_stars)
#skyplot = sns.scatterplot(x='ra', y='dec', data = members)
skyplot.invert_xaxis()
plt.title('Skyplot of M45 data')
plt.show()
```

```
<Figure size 870x627 with 1 Axes>
```

```
sns.scatterplot(x = 'bp_rp', y='phot_g_mean_mag',palette='RdYlGn',
 data = all_stars)
plt.ylabel('Apparent Magnitude (G band)')
#plt.xlim(0,3)
#sns.scatterplot(hr.b_v+0.5, hr.V+10)

plt.gca().invert_yaxis()
```

```
/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_9992/533507744.py:1: UserWarning: Ign
sns.scatterplot(x = 'bp_rp', y='phot_g_mean_mag',palette='RdYlGn',
```

```
<Figure size 870x627 with 1 Axes>
```

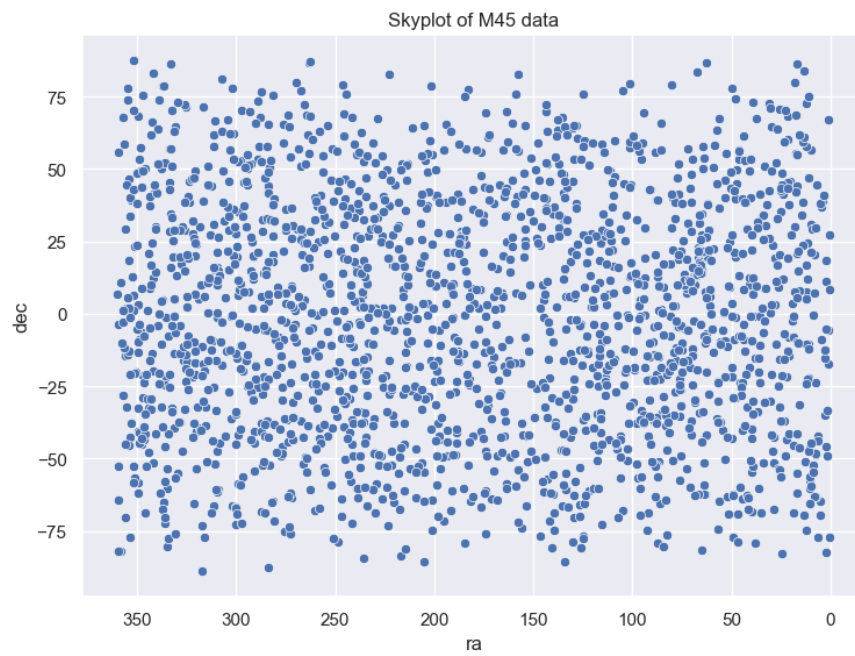


Figure 26.1: output

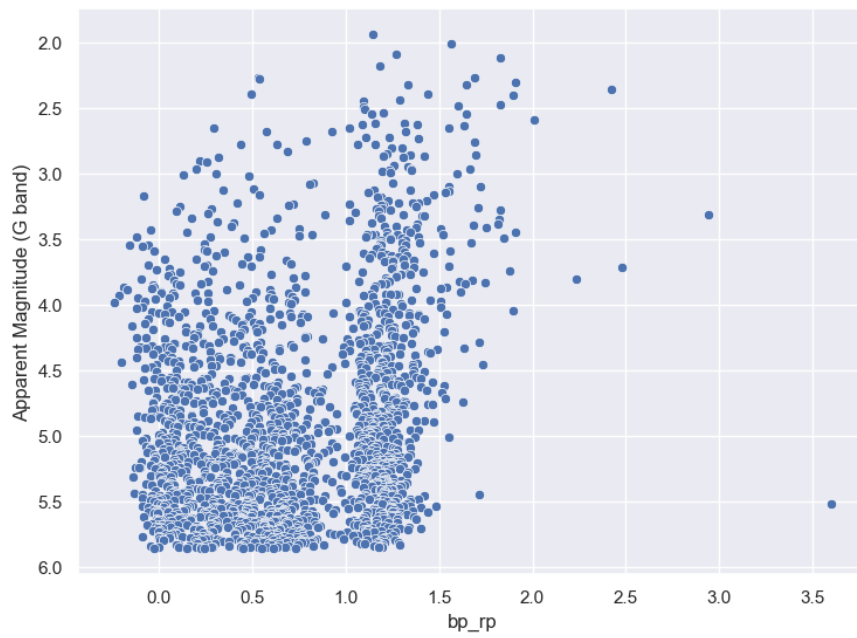


Figure 26.2: output

```
sns.histplot(x='parallax', data=all_stars,
 kde=True,color='blue')
#sns.histplot(x='parallax', data=rf_member, label='RF Member',
kde=True,color='Green')
plt.legend()
plt.xlim([-20,30])
plt.show()
```

```
/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_9992/2280469541.py:5: UserWarning:
plt.legend()
```

```
<Figure size 870x627 with 1 Axes>
```

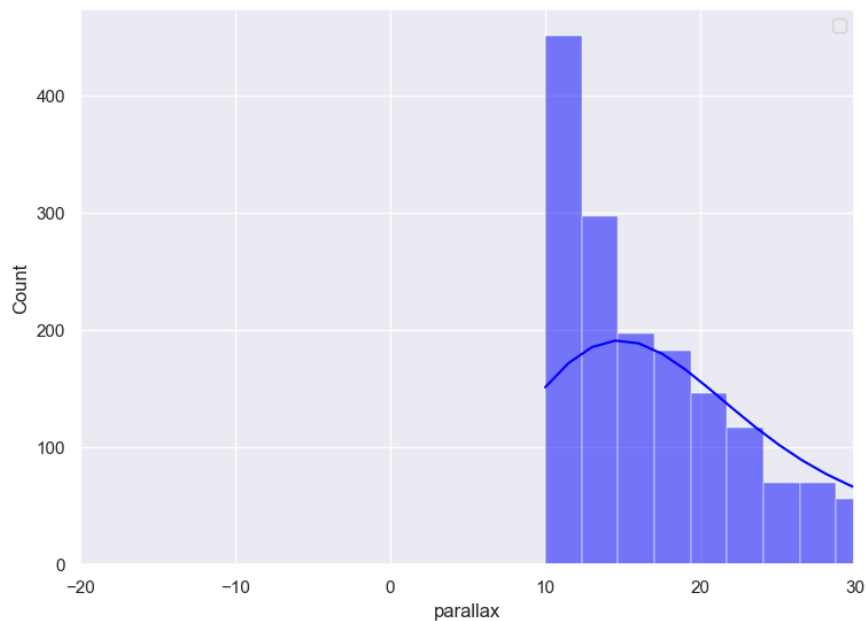


Figure 26.3: output

```
import seaborn as sns
sns.set(rc={'figure.figsize':(8.7,6.27)})

#sns.scatterplot(x='pmra', y='pmdec', data=features[features.labels > -1], hue='labels')
sns.scatterplot(x='pmra', y='pmdec', data=all_stars)
```

```
plt.xlim([-44,50])
plt.ylim([-60,30])
```

```
(-60.0, 30.0)
```

```
<Figure size 870x627 with 1 Axes>
```

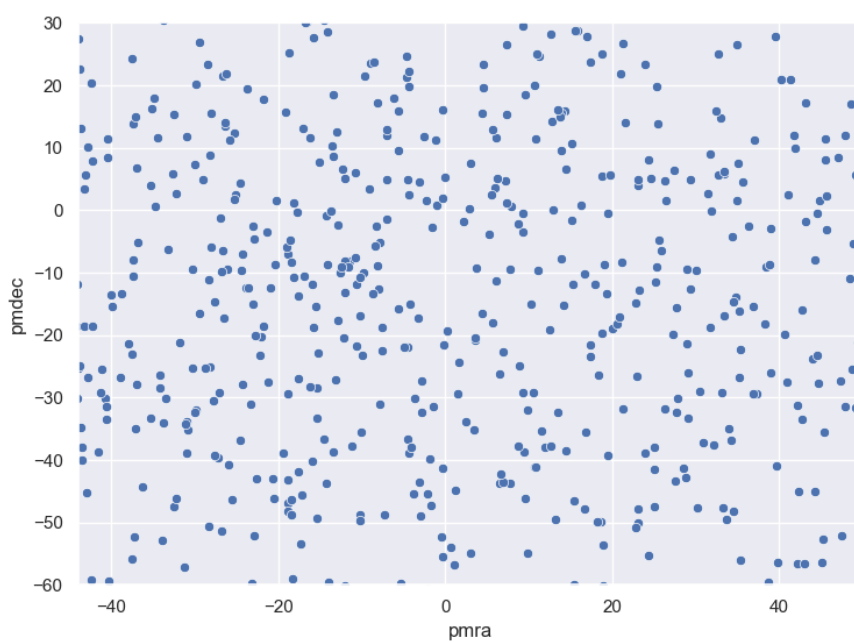


Figure 26.4: output

```
mask = (all_stars.parallax >= 6) & (all_stars.parallax <= 8.5) & (all_stars.pmra < 30) & (all_star
```

```
members = all_stars[mask]
```

```
sns.scatterplot(x = 'bp_rp', y='phot_g_mean_mag',palette='RdYlGn',
 data = all_stars)
sns.scatterplot(x = 'bp_rp', y='phot_g_mean_mag',palette='RdYlGn',
 data = members)
plt.ylabel('Apparent Magnitude (G band)')
#plt.xlim(0,3)
#sns.scatterplot(hr.b_v+0.5, hr.V+10)
```

```
plt.gca().invert_yaxis()
```

```
/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_9992/3127956097.py:1: UserWarning:
 sns.scatterplot(x = 'bp_rp', y='phot_g_mean_mag',palette='RdYlGn',
/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_9992/3127956097.py:3: UserWarning:
 sns.scatterplot(x = 'bp_rp', y='phot_g_mean_mag',palette='RdYlGn',
```

```
<Figure size 870x627 with 1 Axes>
```

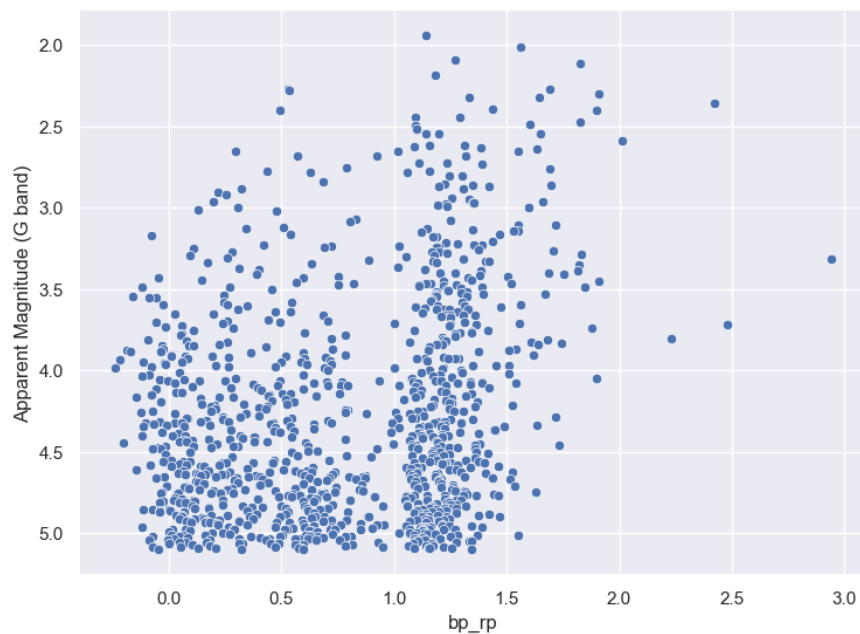


Figure 26.5: output

```
sns.scatterplot(x = 'bp_rp', y='phot_g_mean_mag',palette='RdYlGn',
 data = members)
plt.ylabel('Apparent Magnitude (G band)')
#plt.xlim(0,3)
#sns.scatterplot(hr.b_v+0.5, hr.V+10)

plt.gca().invert_yaxis()
```

```
/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_9992/3329751562.py:1: UserWarning: Igr
sns.scatterplot(x = 'bp_rp', y='phot_g_mean_mag',palette='RdYlGn',
```

```
<Figure size 870x627 with 1 Axes>
```

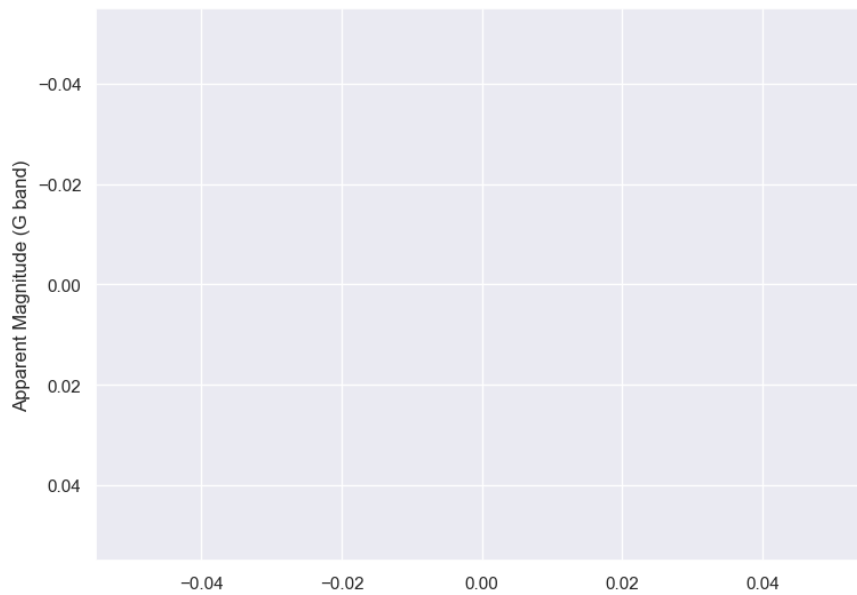


Figure 26.6: output





## Chapter 27

# Introduction to Image Analysis and Processing

```
!pip install -q astropy astroquery
```

```
import numpy as np
import matplotlib.pyplot as plt
from astropy.io import fits
import astropy.units as u
from astropy.visualization import make_lupton_rgb
```

```
To improve Aesthetics of plots
import matplotlib
matplotlib.rcParams['figure.figsize'] = (10,10)
matplotlib.rcParams['font.size'] = 15
```



## Chapter 28

# FITS

Flexible Image Transport System (FITS) is an open standard defining a digital file format useful for storage, transmission and processing of data: formatted as multi-dimensional arrays (for example a 2D image), or tables. FITS is the most commonly used digital file format in astronomy.

```
hdu = fits.open('data/dss2ir.fits')

If you are using Google Colab, uncomment the next line:

#hdu = fits.open("https://raw.githubusercontent.com/Danselem/brics_astro/main/Week4/data/dss2ir.j
hdu
```

```
[<astropy.io.fits.hdu.image.PrimaryHDU object at 0x1061721d0>]
```

HDU list class. This is the top-level FITS object. When a FITS file is opened, a HDUList object is returned.

Learn more about the object by running the cell below.

```
help(hdu)
```

```
Help on HDUList in module astropy.io.fits.hdu.hdulist object:
```

```
class HDUList(builtins.list, astropy.io.fits.verify._Verify)
| HDUList(hdus=[], file=None)
|
| HDU list class. This is the top-level FITS object. When a FITS
| file is opened, a `HDUList` object is returned.
|
```

```

| Method resolution order:
| HDUList
| builtins.list
| astropy.io.fits.verify._Verify
| builtins.object
|
| Methods defined here:
|
| __contains__(self, item)
| Returns `True` if ``item`` is an ``HDU`` in ``self`` or a valid
| extension specification (e.g., integer extension number, extension
| name, or a tuple of extension name and an extension version)
| of a ``HDU`` in ``self``.
|
| __copy__(self)
| Return a shallow copy of an HDUList.
|
| Returns
| -----
| copy : `HDUList`
| A shallow copy of this `HDUList` object.
|
| __deepcopy__(self, memo=None)
|
| __delitem__(self, key)
| Delete an HDU from the `HDUList`, indexed by number or name.
|
| __enter__(self)
| # Support the 'with' statement
|
| __exit__(self, type, value, traceback)
|
| __getitem__(self, key)
| Get an HDU from the `HDUList`, indexed by number or name.
|
| __init__(self, hdus=[], file=None)
| Construct a `HDUList` object.
|
| Parameters
| -----
| hdus : BaseHDU or sequence thereof, optional
| The HDU object(s) to comprise the `HDUList`. Should be
| instances of HDU classes like `ImageHDU` or `BinTableHDU`.
|
| file : file-like, bytes, optional

```

```

| The opened physical file associated with the `HDUList`
| or a bytes object containing the contents of the FITS
| file.
|
| __iter__(self)
| Implement iter(self).
|
| __len__(self)
| Return len(self).
|
| __repr__(self)
| Return repr(self).
|
| __setitem__(self, key, hdu)
| Set an HDU to the `HDUList`, indexed by number or name.
|
| append(self, hdu)
| Append a new HDU to the `HDUList`.
|
| Parameters
| -----
| hdu : BaseHDU
| HDU to add to the `HDUList`.
|
| close(self, output_verify='exception', verbose=False, closed=True)
| Close the associated FITS file and memmap object, if any.
|
| Parameters
| -----
| output_verify : str
| Output verification option. Must be one of ``"fix"`,
| ``"silentfix"`, ``"ignore"`, ``"warn"`, or
| ``"exception"`. May also be any combination of ``"fix"` or
| ``"silentfix" with ``"+ignore"`, ``+warn`, or ``+exception"
| (e.g. ``"fix+warn"`). See :ref:`astropy:verify` for more info.
|
| verbose : bool
| When `True`, print out verbose messages.
|
| closed : bool
| When `True`, close the underlying file object.
|
| copy = __copy__(self)
|
| fileinfo(self, index)

```

```

| Returns a dictionary detailing information about the locations
| of the indexed HDU within any associated file. The values are
| only valid after a read or write of the associated file with
| no intervening changes to the `HDUList`.
|
| Parameters
| -----
| index : int
| Index of HDU for which info is to be returned.
|
| Returns
| -----
| fileinfo : dict or None
|
| The dictionary details information about the locations of
| the indexed HDU within an associated file. Returns `None`
| when the HDU is not associated with a file.
|
| Dictionary contents:
|
| =====
| Key Value
| =====
| file File object associated with the HDU
| filename Name of associated file object
| filemode Mode in which the file was opened (readonly,
| update, append, denywrite, ostream)
| resized Flag that when `True` indicates that the data has been
| resized since the last read/write so the returned values
| may not be valid.
| hdrLoc Starting byte location of header in file
| datLoc Starting byte location of data block in file
| datSpan Data size including padding
| =====
|
| filename(self)
| Return the file name associated with the HDUList object if one exists.
| Otherwise returns None.
|
| Returns
| -----
| filename : str
| A string containing the file name associated with the HDUList
| object if an association exists. Otherwise returns None.
|

```

```

| flush(self, output_verify='fix', verbose=False)
| Force a write of the `HDUList` back to the file (for append and
| update modes only).
|
| Parameters
| -----
| output_verify : str
| Output verification option. Must be one of ``"fix"`` ,
| ``"silentfix"`` , ``"ignore"`` , ``"warn"`` , or
| ``"exception"`` . May also be any combination of ``"fix"`` or
| ``"silentfix"`` with ``"+ignore"`` , ``+warn`` , or ``+exception"
| (e.g. ``"fix+warn"``) . See :ref:`astropy:verify` for more info.
|
| verbose : bool
| When `True` , print verbose messages
|
| index_of(self, key)
| Get the index of an HDU from the `HDUList` .
|
| Parameters
| -----
| key : int, str, tuple of (string, int) or BaseHDU
| The key identifying the HDU. If ``key`` is a tuple, it is of the
| form ``(name, ver)`` where ``ver`` is an ``EXTVER`` value that must
| match the HDU being searched for.
|
| If the key is ambiguous (e.g. there are multiple 'SCI' extensions)
| the first match is returned. For a more precise match use the
| ``(name, ver)`` pair.
|
| If even the ``(name, ver)`` pair is ambiguous (it shouldn't be
| but it's not impossible) the numeric index must be used to index
| the duplicate HDU.
|
| When ``key`` is an HDU object, this function returns the
| index of that HDU object in the ``HDUList`` .
|
| Returns
| -----
| index : int
| The index of the HDU in the `HDUList` .
|
| Raises
| -----
| ValueError

```

```

| If ``key`` is an HDU object and it is not found in the ``HDUList``.
| KeyError
| If an HDU specified by the ``key`` that is an extension number,
| extension name, or a tuple of extension name and version is not
| found in the ``HDUList``.
|
| info(self, output=None)
| Summarize the info of the HDUs in this `HDUList`.
|
| Note that this function prints its results to the console---it
| does not return a value.
|
| Parameters
| -----
| output : file-like or bool, optional
| A file-like object to write the output to. If `False`, does not
| output to a file and instead returns a list of tuples representing
| the HDU info. Writes to ``sys.stdout`` by default.
|
| insert(self, index, hdu)
| Insert an HDU into the `HDUList` at the given ``index``.
|
| Parameters
| -----
| index : int
| Index before which to insert the new HDU.
|
| hdu : BaseHDU
| The HDU object to insert
|
| pop(self, index=-1)
| Remove an item from the list and return it.
|
| Parameters
| -----
| index : int, str, tuple of (string, int), optional
| An integer value of ``index`` indicates the position from which
| ``pop()`` removes and returns an HDU. A string value or a tuple
| of ``(string, int)`` functions as a key for identifying the
| HDU to be removed and returned. If ``key`` is a tuple, it is
| of the form ``(key, ver)`` where ``ver`` is an ``EXTVER``
| value that must match the HDU being searched for.
|
| If the key is ambiguous (e.g. there are multiple 'SCI' extensions)
| the first match is returned. For a more precise match use the

```



```

| `` (name, ver) `` pair.
|
| If even the `` (name, ver) `` pair is ambiguous the numeric index
| must be used to index the duplicate HDU.
|
| Returns
| -----
| hdu : BaseHDU
| The HDU object at position indicated by ``index`` or having name
| and version specified by ``index``.
|
| readall(self)
| Read data of all HDUs into memory.
|
| update_extend(self)
| Make sure that if the primary header needs the keyword ``EXTEND`` that
| it has it and it is correct.
|
| writeto(self, fileobj, output_verify='exception', overwrite=False, checksum=False)
| Write the `HDUList` to a new file.
|
| Parameters
| -----
| fileobj : str, file-like or `pathlib.Path`
| File to write to. If a file object, must be opened in a
| writeable mode.
|
| output_verify : str
| Output verification option. Must be one of ``"fix"`` ,
| ``"silentfix"`` , ``"ignore"`` , ``"warn"`` , or
| ``"exception"`` . May also be any combination of ``"fix"`` or
| ``"silentfix"`` with ``"+ignore"`` , ``"+warn"`` , or ``"+exception"``
| (e.g. ``"fix+warn"``). See :ref:`astropy:verify` for more info.
|
| overwrite : bool, optional
| If ``True`` , overwrite the output file if it exists. Raises an
| ``OSError`` if ``False`` and the output file exists. Default is
| ``False`` .
|
| checksum : bool
| When `True` adds both ``DATASUM`` and ``CHECKSUM`` cards
| to the headers of all HDU's written to the file.
|
| Notes
| -----

```

```

| gzip, zip and bzip2 compression algorithms are natively supported.
| Compression mode is determined from the filename extension
| ('.gz', '.zip' or '.bz2' respectively). It is also possible to pass a
| compressed file object, e.g. `gzip.GzipFile`.
|
| -----
| Class methods defined here:
|
| fromfile(fileobj, mode=None, memmap=None, save_backup=False, cache=True, lazy_load=True)
| Creates an `HDUList` instance from a file-like object.
|
| The actual implementation of ``fitsopen()`, and generally shouldn't
| be used directly. Use :func:`open` instead (and see its
| documentation for details of the parameters accepted by this method).
|
| fromstring(data, **kwargs) from builtins.type
| Creates an `HDUList` instance from a string or other in-memory data
| buffer containing an entire FITS file. Similar to
| :meth:`HDUList.fromfile`, but does not accept the mode or memmap
| arguments, as they are only relevant to reading from a file on disk.
|
| This is useful for interfacing with other libraries such as CFITSIO,
| and may also be useful for streaming applications.
|
| Parameters
| -----
| data : str, buffer-like, etc.
| A string or other memory buffer containing an entire FITS file.
| Buffer-like objects include :class:`~bytes`, :class:`~bytearray`,
| :class:`~memoryview`, and :class:`~numpy.ndarray`.
| It should be noted that if that memory is read-only (such as a
| Python string) the returned :class:`HDUList`'s data portions will
| also be read-only.
| **kwargs : dict
| Optional keyword arguments. See
| :func:`astropy.io.fits.open` for details.
|
| Returns
| -----
| hdul : HDUList
| An :class:`HDUList` object representing the in-memory FITS file.
|
| -----
| Data descriptors defined here:

```

```
| __dict__
| dictionary for instance variables (if defined)
|
| __weakref__
| list of weak references to the object (if defined)
|
| -----
```

```
| Methods inherited from builtins.list:
```

```
| __add__(self, value, /)
| Return self+value.
|
| __eq__(self, value, /)
| Return self==value.
|
| __ge__(self, value, /)
| Return self>=value.
|
| __getattr__(self, name, /)
| Return getattr(self, name).
|
| __gt__(self, value, /)
| Return self>value.
|
| __iadd__(self, value, /)
| Implement self+=value.
|
| __imul__(self, value, /)
| Implement self*=value.
|
| __le__(self, value, /)
| Return self<=value.
|
| __lt__(self, value, /)
| Return self<value.
|
| __mul__(self, value, /)
| Return self*value.
|
| __ne__(self, value, /)
| Return self!=value.
|
| __reversed__(self, /)
| Return a reverse iterator over the list.
|
```

```

| __rmul__(self, value, /)
| Return value*self.
|
| __sizeof__(self, /)
| Return the size of the list in memory, in bytes.
|
| clear(self, /)
| Remove all items from list.
|
| count(self, value, /)
| Return number of occurrences of value.
|
| extend(self, iterable, /)
| Extend list by appending elements from the iterable.
|
| index(self, value, start=0, stop=9223372036854775807, /)
| Return first index of value.
|
| Raises ValueError if the value is not present.
|
| remove(self, value, /)
| Remove first occurrence of value.
|
| Raises ValueError if the value is not present.
|
| reverse(self, /)
| Reverse *IN PLACE*.
|
| sort(self, /, *, key=None, reverse=False)
| Sort the list in ascending order and return None.
|
| The sort is in-place (i.e. the list itself is modified) and stable (i.e. the
| order of two equal elements is maintained).
|
| If a key function is given, apply it once to each list item and sort them,
| ascending or descending, according to their function values.
|
| The reverse flag can be set to sort in descending order.
|
| -----
| Class methods inherited from builtins.list:
|
| __class_getitem__(...) from builtins.type
| See PEP 585
|

```

```

| -----
| Static methods inherited from builtins.list:
|
| __new__(*args, **kwargs) from builtins.type
| Create and return a new object. See help(type) for accurate signature.
|
| -----
| Data and other attributes inherited from builtins.list:
|
| __hash__ = None
|
| -----
| Methods inherited from astropy.io.fits.verify._Verify:
|
| run_option(self, option='warn', err_text='', fix_text='Fixed.', fix=None, fixable=True)
| Execute the verification with selected option.
|
| verify(self, option='warn')
| Verify all values in the instance.
|
| Parameters
| -----
| option : str
| Output verification option. Must be one of ``"fix"`` ,
| ``"silentfix"`` , ``"ignore"`` , ``"warn"`` , or
| ``"exception"`` . May also be any combination of ``"fix"`` or
| ``"silentfix"`` with ``"+ignore"`` , ``"+warn"`` , or ``"+exception"``
| (e.g. ``"fix+warn"``) . See :ref:`astropy:verify` for more info.

```

Every Header Data Unit (HDU) normally has two components: header and data. In astropy these two components are accessed through the two attributes of the HDU, `hdu.header` and `hdu.data`.

FITS image headers can contain information about one or more scientific coordinate systems that are overlaid on the image itself. Images contain an implicit Cartesian coordinate system that describes the location of each pixel in the image, but scientific uses usually require working in ‘world’ coordinates, for example the celestial coordinate system.

`hdu[0].header`

```

SIMPLE = T / Written by SkyView Wed Jun 30 06:16:55 EDT 2021
BITPIX = -32 / 4 byte floating point
NAXIS = 2 / Two dimensional image
NAXIS1 = 600 / Width of image

```

```

NAXIS2 = 600 / Height of image
CRVAL1 = 202.46957500000002 / Reference longitude
CRVAL2 = 47.19525800000001 / Reference latitude
RADESYS = 'FK5 ' / Coordinate system
EQUINOX = 2000.0 / Epoch of the equinox
CTYPE1 = 'RA---TAN' / Coordinates -- projection
CTYPE2 = 'DEC--TAN' / Coordinates -- projection
CRPIX1 = 300.5 / X reference pixel
CRPIX2 = 300.5 / Y reference pixel
CDELT1 = -0.00050 / X scale
CDELT2 = 0.00050 / Y scale
COMMENT
COMMENT SkyView Survey metadata
COMMENT
COMMENT Provenance: Data taken by ROE, AAO, and CalTech, Compression
COMMENT and distribution by Space Telescope Science Inst
COMMENT itute.
COMMENT Copyright: There are multiple copyright holders depending up
COMMENT on the source plate or plates. See full copyright
COMMENT notices The coverage link below describes the
COMMENT coverage for each element of the surveys when the original
COMMENT plate used is not included in the SkyView files..
COMMENT Regime: Infrared
COMMENT NSurvey: 1
COMMENT Frequency: 353 THz
COMMENT Bandpass: 327-422 THz
COMMENT Coverage: All-sky
COMMENT PixelScale: 1"
COMMENT PixelUnits: Pixel values are given as scaled densities
COMMENT Resolution: Depends on plate. Typically 2-3".
COMMENT Coordinates: Equatorial
COMMENT Projection: Schmidt
COMMENT Equinox: 2000
COMMENT Epoch: 1987-2002
COMMENT Reference: Some information on the DSS2 is given in McLea
COMMENT n, 2000, The Second Generation Guide Star Catalog.
COMMENT Characteristics of the DSS surveys are summarized in
COMMENT this document.
COMMENT
COMMENT Survey specific cards
COMMENT
SURVEY = 'DSS2 IR '
HISTORY
HISTORY Settings used in processing:
HISTORY

```

```

HISTORY cache = /data/cache/surveys/
HISTORY coordinates = J2000.0
HISTORY deedger = skyview.process.Deedger
HISTORY descriptionxslt = cgifiles/description.xsl
HISTORY dft_cache = ./skycache/
HISTORY dummy
HISTORY equinox = 2000
HISTORY fileprefix = https://skyview.gsfc.nasa.gov/surveys/dss2/
HISTORY finalpostprocessor = skyview.ij.IJProcessor,skyview.data.BoxSmoother,sky
HISTORY float = on
HISTORY footertemplate = cgifiles/skyfooter.html
HISTORY gallerydir = /userimages
HISTORY galleryxslt = cgifiles/gallerymultipage.xsl
HISTORY headertemplate = cgifiles/skyheader.html
HISTORY htmlwriter = skyview.request.HTMLWriter
HISTORY imagefactory = skyview.survey.DSSImageFactory
HISTORY imagesize = 6
HISTORY imscale = 0.3
HISTORY localurl = https://skyview.gsfc.nasa.gov/surveys,/skyview/htdocs/surveys
HISTORY lut = colortables/b-w-linear.bin
HISTORY lutcbarpath = ../images/colorbars/
HISTORY mosaicker = skyview.process.Mosaicker
HISTORY name = 2nd Digitized Sky Survey-Near Infrared
HISTORY noexit =
HISTORY nullimagedir = ../images/nodata
HISTORY nwindow = on
HISTORY output = ../../temp space/fits/skv23139671872862_1
HISTORY outputroot = ../../temp space/fits
HISTORY pixels = 600
HISTORY position = m 51
HISTORY postprocessor = skyview.ij.IJProcessor,skyview.request.RGBWriter
HISTORY projection = Tan
HISTORY quicklook = jpeg
HISTORY requested_coords = 202.46957500000002, 47.19525800000001
HISTORY reqxpos = 202.46957500000002
HISTORY reqypos = 47.19525800000001
HISTORY resolver = SIMBAD-NED
HISTORY rgb =
HISTORY rgbtemplate = cgifiles/skyrgb.html
HISTORY rgbwriter = skyview.request.RGBWriter
HISTORY sampler = Default
HISTORY savebysurvey
HISTORY scale = 0.0002777777
HISTORY scaling = Log
HISTORY settingsupdaters = BatchCompatibility,SettingsFixer,skyview.request.Toas

```

```

HISTORY shortname = DSS2IR, DSS2 IR
HISTORY sia_header = ./sia.header
HISTORY siabase = https://skyview.gsfc.nasa.gov/cgi-bin/images?
HISTORY siaimageruntime = 300000
HISTORY size = 0.3,0.3
HISTORY survey = DSS2 IR,DSS2 Red,DSS2 Blue
HISTORY surveyfinder = skyview.survey.XMLSurveyFinder
HISTORY surveymanifest = surveys/survey.manifest
HISTORY surveyregimes = Radio,Millimeter,Infrared,Optical,Ultraviolet,X-ray,Gamm
HISTORY surveysheader = cgifiles/survey.header
HISTORY surveytemplate = cgifiles/skysurvey.html
HISTORY ui_imblue = DSS2 Blue
HISTORY ui_imgree = DSS2 Red
HISTORY ui_imredd = DSS2 IR
HISTORY url.heasarcbase = https://heasarc.gsfc.nasa.gov/xamin/vo/cone?showoffset
HISTORY url.ned = http://nedwww.ipac.caltech.edu/cgi-bin/nph-NEDobjsearch?search
HISTORY url.simbad = http://simbad.u-strasbg.fr/simbad-conesearch.pl?
HISTORY url.vizierbase = http://vizier.u-strasbg.fr/viz-bin/votable/-dtd/-A?-out
HISTORY urllocalhelp = https://skyview.gsfc.nasa.gov/help/help.html
HISTORY version = 3.5.5
HISTORY webrootpath = /skyview/htdocs
HISTORY xmlurlprefix = https://skyview.gsfc.nasa.gov/current/jar/
HISTORY
HISTORY Map generated at: Wed Jun 30 06:16:55 EDT 2021
HISTORY
HISTORY Resampler used: NNSampler
HISTORY
HISTORY Image mosaicking usingskyview.process.Mosaicker
HISTORY
HISTORY Used image:https://skyview.gsfc.nasa.gov/surveys/dss2/xi/xi220
HISTORY Pixels:78315
HISTORY Used image:https://skyview.gsfc.nasa.gov/surveys/dss2/xi/xi270
HISTORY Pixels:281685
HISTORY
HISTORY
HISTORY Edge adjustments applied (skyview.geometry.DeedgerList
HISTORY
HISTORY Image xi220 offset by -853.0
HISTORY

```

```

data = hdu[0].data
type(data)

```



```
numpy.ndarray
```

**Exercise 1.1**

Write code to display the image data in the variable 'data'. The plot should have following features

1. Grayscale colormap
2. Title : 'M 51 Galaxy'
3. Origin : 'lower'



## Chapter 29

# Scaling or Stretch function

Scaling is used to change the visual appearance of an image, to alter the quantity of information stored in a scene representation, or as a low-level preprocessor in multi-stage image processing chain which operates on features of a particular scale. You choose which “stretch function” or representation to use

A logarithmic representation of the pixel values tends to suppress the bright parts of the image, i.e. the stars, and to enhance the fainter part, e.g. nebulosity. This can be desirable if the ‘faint stuff’ needs ‘a boost’, but a logarithmic stretch function can also reduce the contrast in an image, producing a lower dynamic range

Here’s an example of logarithmic scaling

```
log = np.log10(data)
plt.imshow(log)
plt.title('Sqrt scaled M 51 Galaxy')
```

```
Text(0.5, 1.0, 'Sqrt scaled M 51 Galaxy')
```

```
<Figure size 1000x1000 with 1 Axes>
```

### Exercise 1.2.a

Write a function which takes 2D numpy array and scaling function name as input and returns scaled image. The function should have following scaling functions

1. Linear.
2. Square root.

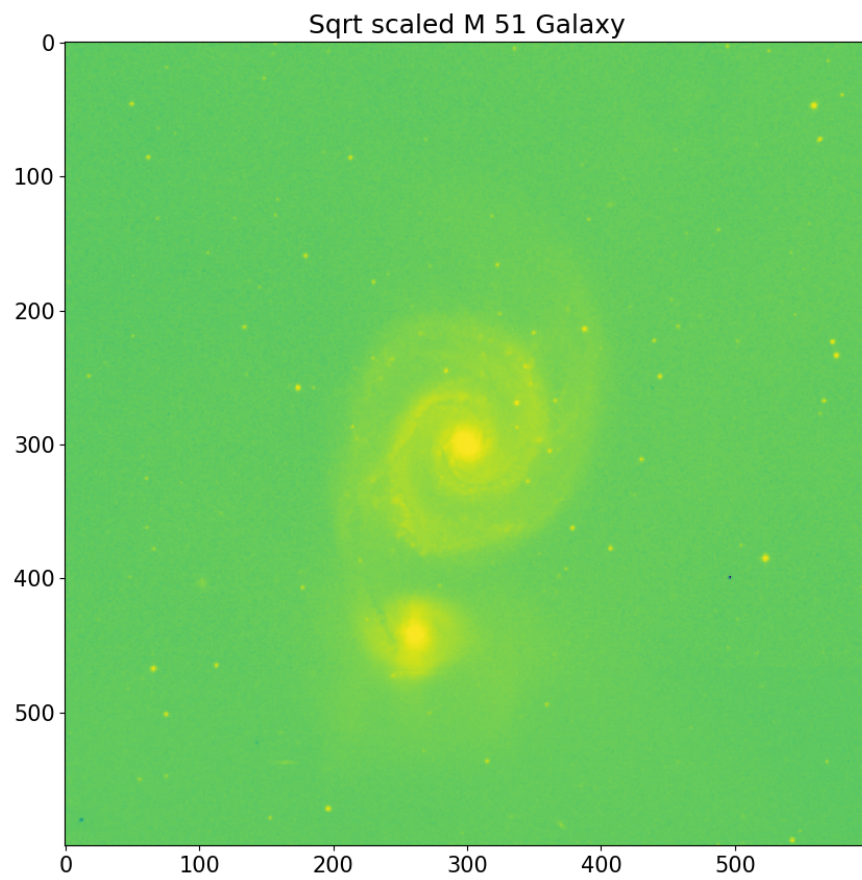


Figure 29.1: output

3. Logarithmic [Base 10]. ( Remember  $\log 0$  is indefinite, use  $\log(1 + \text{data})$  )
4. asinh

**Exercise 1.2.b**

Write a function which takes 2D array and normalization method as input and returns normalized array. The function should have following normalization method

1. Normalize data to the range  $[0,1]$ . Hint : Use min and max [Learn more](#)
2. Normalize data using mean and standard deviation



## Chapter 30

# RGB Composite image

The RGB colour model relates very closely to the way we perceive colour with the r, g and b receptors in our retinas. RGB uses additive colour mixing and is the basic colour model used in television or any other medium that projects colour with light. It is the basic colour model used in computers and for web graphics.

Figure 30.1: rgb\_model.png

Images of astronomical objects are usually taken with electronic detectors such as a CCD (Charge Coupled Device). Similar detectors are found in normal digital cameras. Telescope images are nearly always greyscale, but nevertheless contain some colour information. An astronomical image may be taken through a colour filter. Different detectors and telescopes also usually have different sensitivities to different wavelengths.

Filters can either be broad-band (Wide) or narrow-band (Narrow). A broad-band filter lets a wide range of colours through, for instance the entire green or red area of the spectrum. A narrow-band filter typically only lets a small wavelength span through, thus effectively restricting the transmitted radiation to that coming from a given atomic transition, allowing astronomers to investigate individual atomic processes in the object.

### 30.1 Sloan Digital Sky Survey

The Sloan Digital Sky Survey or SDSS is a major multi-spectral imaging and spectroscopic redshift survey using a dedicated 2.5-m wide-angle optical telescope at Apache Point Observatory in New Mexico, United States. [Learn more](#)

Figure 30.2: SDSS\_telescope\_new.jpg

The SDSS camera has five filters, which together span the optical window. Each filter images a section of sky nearly, but not exactly, simultaneously (each filter is separated by 71.72 seconds). The filters always observe in the same time sequence: r, i, u, z and then g. A mnemonic for remembering the order is “robert is under ze gunn.”

Figure 30.3: plot\_sdss\_filters\_11 (1).png



## Chapter 31

# Skyview

In the next exercise we will be using Skyview module from astroquery package to acquire r, g, and i filter images and use it to make RGB images. First let's install Astroquery package. (Astroquery Documentation)

```
!pip install astroquery --quiet
```

```
11.1/11.1 MB 100.3 MB/s eta 0:00:00
1.1/1.1 MB 50.4 MB/s eta 0:00:00
```

```
from astroquery.skyview import SkyView
```

The SkyView service offers a cutout service for a number of imaging surveys. It is a Virtual Observatory on the Net (at [skyview.gsfc.nasa.gov](http://skyview.gsfc.nasa.gov)) generating images of any part of the sky at wavelengths in all regimes from Radio to Gamma-Ray.

```
SkyView.list_surveys()
```

```
{'Allbands:GOODS/HDF/CDF': ['GOODS: Chandra ACIS HB',
 'GOODS: Chandra ACIS FB',
 'GOODS: Chandra ACIS SB',
 'GOODS: VLT VIMOS U',
 'GOODS: VLT VIMOS R',
 'GOODS: HST ACS B',
 'GOODS: HST ACS V',
 'GOODS: HST ACS I',
 'GOODS: HST ACS Z',
 'Hawaii HDF U',
 'Hawaii HDF B',
```

```

'Hawaii HDF V0201',
'Hawaii HDF V0401',
'Hawaii HDF R',
'Hawaii HDF I',
'Hawaii HDF z',
'Hawaii HDF HK',
'GOODS: HST NICMOS',
'GOODS: VLT ISAAC J',
'GOODS: VLT ISAAC H',
'GOODS: VLT ISAAC Ks',
'HUDF: VLT ISAAC Ks',
'GOODS: Spitzer IRAC 3.6',
'GOODS: Spitzer IRAC 4.5',
'GOODS: Spitzer IRAC 5.8',
'GOODS: Spitzer IRAC 8.0',
'GOODS: Spitzer MIPS 24',
'GOODS: Herschel 100',
'GOODS: Herschel 160',
'GOODS: Herschel 250',
'GOODS: Herschel 350',
'GOODS: Herschel 500',
'CDFS: LESS',
'GOODS: VLA North'],
'Allbands:HiPS': ['UltraVista-H',
 'UltraVista-J',
 'UltraVista-Ks',
 'UltraVista-NB118',
 'UltraVista-Y',
 'CFHTLS-W-u',
 'CFHTLS-W-g',
 'CFHTLS-W-r',
 'CFHTLS-W-i',
 'CFHTLS-W-z',
 'CFHTLS-D-u',
 'CFHTLS-D-g',
 'CFHTLS-D-r',
 'CFHTLS-D-i',
 'CFHTLS-D-z'],
'GammaRay': ['Fermi 5',
 'Fermi 4',
 'Fermi 3',
 'Fermi 2',
 'Fermi 1',
 'EGRET (3D)',
 'EGRET <100 MeV',

```

```

 'EGRET >100 MeV',
 'COMPTEL'],
'HardX-ray': ['INT GAL 17-35 Flux',
 'INT GAL 17-60 Flux',
 'INT GAL 35-80 Flux',
 'INTEGRAL/SPI GC',
 'GRANAT/SIGMA',
 'RXTE Allsky 3-8keV Flux',
 'RXTE Allsky 3-20keV Flux',
 'RXTE Allsky 8-20keV Flux'],
'IR:2MASS class=': ['2MASS-J', '2MASS-H', '2MASS-K'],
'IR:AKARI class=': ['AKARI N60', 'AKARI WIDE-S', 'AKARI WIDE-L', 'AKARI N160'],
'IR:IRAS': ['IRIS 12',
 'IRIS 25',
 'IRIS 60',
 'IRIS 100',
 'SFD100m',
 'SFD Dust Map',
 'IRAS 12 micron',
 'IRAS 25 micron',
 'IRAS 60 micron',
 'IRAS 100 micron'],
'IR:Planck': ['Planck 857 I',
 'Planck 545 I',
 'Planck 353 I',
 'Planck 353 Q',
 'Planck 353 U',
 'Planck 353 PI',
 'Planck 353 PA',
 'Planck 353 PI/I',
 'Planck 217 I',
 'Planck 217 Q',
 'Planck 217 U',
 'Planck 217 PI',
 'Planck 217 PA',
 'Planck 217 PI/I',
 'Planck 143 I',
 'Planck 143 Q',
 'Planck 143 U',
 'Planck 143 PI',
 'Planck 143 PA',
 'Planck 143 PI/I',
 'Planck 100 I',
 'Planck 100 Q',
 'Planck 100 U',

```

```

 'Planck 100 PI',
 'Planck 100 PA',
 'Planck 100 PI/I',
 'Planck 070 I',
 'Planck 070 Q',
 'Planck 070 U',
 'Planck 070 PI',
 'Planck 070 PA',
 'Planck 070 PI/I',
 'Planck 044 I',
 'Planck 044 Q',
 'Planck 044 U',
 'Planck 044 PI',
 'Planck 044 PA',
 'Planck 044 PI/I',
 'Planck 030 I',
 'Planck 030 Q',
 'Planck 030 U',
 'Planck 030 PI',
 'Planck 030 PA',
 'Planck 030 PI/I'],
'IR:UKIDSS class=': ['UKIDSS-Y',
 'UKIDSS-J',
 'UKIDSS-H',
 'UKIDSS-K',
 'UKIDSS-1-OS1'],
'IR:WISE class=': ['WISE 3.4', 'WISE 4.6', 'WISE 12', 'WISE 22'],
'IR:WMAP&COBE': ['WMAP ILC',
 'WMAP Ka',
 'WMAP K',
 'WMAP Q',
 'WMAP V',
 'WMAP W',
 'COBE DIRBE/AAM',
 'COBE DIRBE/ZSMA'],
'Optical:DSS class=': ['DSS',
 'DSS1 Blue',
 'DSS1 Red',
 'DSS2 Red',
 'DSS2 Blue',
 'DSS2 IR'],
'Optical:SDSS class=': ['SDSSg', 'SDSSi', 'SDSSr', 'SDSSu', 'SDSSz'],
'OtherOptical': ['TESS',
 'Mellinger Red',
 'Mellinger Green',

```

```

 'Mellinger Blue',
 'H-Alpha Comp',
 'SHASSA H',
 'SHASSA CC',
 'SHASSA C',
 'SHASSA Sm'],
'ROSATDiffuse class=': ['RASS Background 1',
 'RASS Background 2',
 'RASS Background 3',
 'RASS Background 4',
 'RASS Background 5',
 'RASS Background 6',
 'RASS Background 7'],
'ROSATw/sources class=': ['RASS-Cnt Soft',
 'RASS-Cnt Hard',
 'RASS-Cnt Broad',
 'PSPC 2.0 Deg-Int',
 'PSPC 1.0 Deg-Int',
 'PSPC 0.6 Deg-Int',
 'HRI'],
'Radio:GHz': ['CO',
 'GB6 (4850MHz)',
 'VLA FIRST (1.4 GHz)',
 'NVSS',
 'Stripe82VLA',
 '1420MHz (Bonn)',
 'HI4PI',
 'EBHIS',
 'nH'],
'Radio:GLEAM class=': ['GLEAM 72-103 MHz',
 'GLEAM 103-134 MHz',
 'GLEAM 139-170 MHz',
 'GLEAM 170-231 MHz'],
'Radio:MHz class=': ['SUMSS 843 MHz',
 '0408MHz',
 'WENSS',
 'TGSS ADR1',
 'VLSSr',
 '0035MHz',
 '0022MHz'],
'SoftX-ray class=': ['SwiftXRTCnt',
 'SwiftXRTEExp',
 'SwiftXRTInt',
 'HEAO 1 A-2'],
'SwiftUVOT class=': ['UVOT WHITE Intensity',

```

```

 'UVOT V Intensity',
 'UVOT B Intensity',
 'UVOT U Intensity',
 'UVOT UVW1 Intensity',
 'UVOT UVM2 Intensity',
 'UVOT UVW2 Intensity'],
'UV': ['GALEX Near UV',
 'GALEX Far UV',
 'ROSAT WFC F1',
 'ROSAT WFC F2',
 'EUVE 83 A',
 'EUVE 171 A',
 'EUVE 405 A',
 'EUVE 555 A'],
'X-ray:SwiftBAT': ['BAT SNR 14-195',
 'BAT SNR 14-20',
 'BAT SNR 20-24',
 'BAT SNR 24-35',
 'BAT SNR 35-50',
 'BAT SNR 50-75',
 'BAT SNR 75-100',
 'BAT SNR 100-150',
 'BAT SNR 150-195']}

```

```

size = 400
paths = SkyView.get_images(position='M 63', # radius=size*u.arcsec,pixels=512,
 survey=['SDSSg','SDSSr','SDSSi'])

```

```

TimeoutError Traceback (most recent call last)
File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/urllib3/connectionpool.py:516:
 533 try:
--> 534 response = conn.getresponse()
 535 except (BaseSSLError, OSError) as e:

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/urllib3/connection.py:516, in
 515 # Get the response from http.client.HTTPConnection
--> 516 httplib_response = super().getresponse()
 518 try:

File /Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/http/client.py:1374:
 1374 try:
-> 1375 response.begin()
 1376 except ConnectionError:

```

```
File /Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/http/client.py:318, in HTTPConnection.getresponse()
 317 while True:
--> 318 version, status, reason = self._read_status()
 319 if status != CONTINUE:
```

```
File /Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/http/client.py:279, in HTTPConnection._read_status(self)
 278 def _read_status(self):
--> 279 line = str(self.fp.readline(_MAXLINE + 1), "iso-8859-1")
 280 if len(line) > _MAXLINE:
```

```
File /Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/socket.py:705, in SocketIO.recv_into
 704 try:
--> 705 return self._sock.recv_into(b)
 706 except timeout:
```

TimeoutError: [Errno 60] Operation timed out

The above exception was the direct cause of the following exception:

```
ReadTimeoutError Traceback (most recent call last)
File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/requests/adapters.py:667, in HTTPAdapter.send
 666 try:
--> 667 resp = conn.urlopen(
 668 method=request.method,
 669 url=url,
 670 body=request.body,
 671 headers=request.headers,
 672 redirect=False,
 673 assert_same_host=False,
 674 preload_content=False,
 675 decode_content=False,
 676 retries=self.max_retries,
 677 timeout=timeout,
 678 chunked=chunked,
 679)
 681 except (ProtocolError, OSError) as err:
```

```
File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/urllib3/connectionpool.py:841, in HTTPConnectionPool._new_conn
 839 new_e = ProtocolError("Connection aborted.", new_e)
--> 841 retries = retries.increment(
 842 method, url, error=new_e, _pool=self, _stacktrace=sys.exc_info()[2]
 843)
 844 retries.sleep()
```

```
File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/urllib3/util/retry.py:474, in Retry.raise_error
 473 # Raise the error
```

```

 473 if read is False or method is None or not self._is_method_retryable(method):
--> 474 raise reraise(type(error), error, _stacktrace)
 475 elif read is not None:

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/urllib3/util/util.py:39, in
 38 raise value.with_traceback(tb)
--> 39 raise value
 40 finally:

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/urllib3/connectionpool.py:786, in
 786 # Make the request on the HTTPConnection object
--> 787 response = self._make_request(
 788 conn,
 789 method,
 790 url,
 791 timeout=timeout_obj,
 792 body=body,
 793 headers=headers,
 794 chunked=chunked,
 795 retries=retries,
 796 response_conn=response_conn,
 797 preload_content=preload_content,
 798 decode_content=decode_content,
 799 **response_kw,
 800)
 802 # Everything went great!

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/urllib3/connectionpool.py:535, in
 535 except (BaseSSLError, OSError) as e:
--> 536 self._raise_timeout(err=e, url=url, timeout_value=read_timeout)
 537 raise

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/urllib3/connectionpool.py:366, in
 366 if isinstance(err, SocketTimeout):
--> 367 raise ReadTimeoutError(
 368 self, url, f"Read timed out. (read timeout={timeout_value})"
 369) from err
 371 # See the above comment about EAGAIN in Python 3.

ReadTimeoutError: HTTPConnectionPool(host='skyview.gsfc.nasa.gov', port=80): Read timeo

During handling of the above exception, another exception occurred:

ReadTimeout Traceback (most recent call last)
Cell In[12], line 2

```



```

1 size = 400
----> 2 paths = SkyView.get_images(position='M 63',
3 survey=['SDSSg','SDSSr','SDSSI'])

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astroquery/skyview/core.py:196, in Sky
88 def get_images(self, position, survey, *, coordinates=None, projection=None,
89 pixels=None, scaling=None, sampler=None, resolver=None,
90 deedger=None, radius=None, height=None, width=None, cache=True,
91 show_progress=True):
92 """
93 Query the SkyView service, download the FITS file that will be
94 found and return a generator over the local paths to the
95 (...)
194
195 """
--> 196 readable_objects = self.get_images_async(position, survey, coordinates=coordinates,
197 projection=projection, pixels=pixels, scalar
198 sampler=sampler, resolver=resolver, deedger=
199 radius=radius, height=height, width=width,
200 cache=cache, show_progress=show_progress)
201 return [obj.get_fits() for obj in readable_objects]

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astroquery/skyview/core.py:214, in Sky
203 @prepend_docstr_nosections(get_images.__doc__)
204 def get_images_async(self, position, survey, *, coordinates=None,
205 projection=None, pixels=None, scaling=None,
206 sampler=None, resolver=None, deedger=None,
207 radius=None, height=None, width=None,
208 cache=True, show_progress=True):
209 """
210 Returns
211 -----
212 A list of context-managers that yield readable file-like objects
213 """
--> 214 image_urls = self.get_image_list(position, survey, coordinates=coordinates,
215 projection=projection, pixels=pixels, scaling=scalin
216 resolver=resolver, deedger=deedger, radius=radius,
217 height=height, width=width, cache=cache)
218 return [commons.FileContainer(url, encoding='binary',
219 show_progress=show_progress)
220 for url in image_urls]

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astroquery/skyview/core.py:266, in Sky
252 size_deg = None
254 input = {

```

```

255 'Position': parse_coordinates(position),
256 'survey': survey,
257 (...)
258 'size': size_deg,
259 'pixels': pixels}
--> 260 response = self._submit_form(input, cache=cache)
261 urls = self._parse_response(response)
262 return urls

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astroquery/skyview/core.py:82, in
82 def _submit_form(self, input=None, cache=True):
83 url, payload = self._generate_payload(input=input)
--> 84 response = self._request('GET', url, params=payload, cache=cache)
85 response.raise_for_status()
86 return response

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astroquery/query.py:379, in
377 response = query.from_cache(self.cache_location, cache_conf.cache_timeout)
378 if not response:
--> 379 response = query.request(self._session,
380 self.cache_location,
381 stream=stream,
382 auth=auth,
383 allow_redirects=allow_redirects,
384 verify=verify,
385 json=json)
386 to_cache(response, query.request_file(self.cache_location))
387 self._last_query = query

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/astroquery/query.py:77, in
74 def request(self, session, cache_location=None, stream=False,
75 auth=None, verify=True, allow_redirects=True,
76 json=None):
--> 77 return session.request(self.method, self.url, params=self.params,
78 data=self.data, headers=self.headers,
79 files=self.files, timeout=self.timeout,
80 stream=stream, auth=auth, verify=verify,
81 allow_redirects=allow_redirects,
82 json=json)

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/requests/sessions.py:589, in
584 send_kwargs = {
585 "timeout": timeout,
586 "allow_redirects": allow_redirects,
587 }
```

```

588 send_kwargs.update(settings)
--> 589 resp = self.send(prepare, **send_kwargs)
591 return resp

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/requests/sessions.py:703, in Session.send
700 start = preferred_clock()
702 # Send the request
--> 703 r = adapter.send(request, **kwargs)
705 # Total elapsed time of the request (approximately)
706 elapsed = preferred_clock() - start

File ~/Data/brics_astro/.venv/lib/python3.10/site-packages/requests/adapters.py:713, in HTTPAdapter.send
711 raise SSLError(e, request=request)
712 elif isinstance(e, ReadTimeoutError):
--> 713 raise ReadTimeout(e, request=request)
714 elif isinstance(e, _InvalidHeader):
715 raise InvalidHeader(e, request=request)

ReadTimeout: HTTPConnectionPool(host='skyview.gsfc.nasa.gov', port=80): Read timed out. (read timeout=30s)

```

Here position = 'M 63' refers to the name of the source. M 63 stands for 63rd source in the Messier Catalog. For specifying a source in the sky you can either use their catalog names, for example : M 51, NGC 3690, M87 etc or use RA and Dec of the source. RA and Dec are coordinates of the Equatorial Coordinate system similar to latitude and longitude geographical coordinate system.

radius = size\*u.arcsec gives angular size of the source in arc seconds.

pixels = 512 sets the number of pixels in the output image.

survey = ['SDSSg','SDSSr','SDSSi'] returns data for g,r and i band of SDSS

We notice that SkyView.get\_images returns a HDUList object

```
paths[0][0].header # First object in the list
```

### Exercise 1.3

Write code to store the g, r, and i filter data in variable B, G, and R respectively. It's convention to assign shortest wavelength to blue and longest wavelength to red.

```

R = paths[2][0].data # Example
G =
B =

```

**Exercise 1.4.a**

Write code for applying following operations on filter data

Step 1. linear scale

Step 2. Normalize to [0,1]

For each filter data and stack the 3 filters to create a 3D array of shape ( 512,512,3). Hint : Use np.stack

```
Use the functions you defined earlier
R_norm =
G_norm =
B_norm =
RGB = # Stacked 3D array

RGB.shape
```

```
plt.imshow(RGB)
```

For displaying 3D array as RGB image using matplotlib, the data of each 2D filter is expected to be in the range [0,1], thus we normalized the data to [0,1]

**Exercise 1.4.b**

Step 1. arcsinh scale

Step 2. Normalize to [0,1]

For each filter data and stack the 3 filters to create a 3D array of shape ( 512,512,3 ). Hint : Use np.stack

```
R_norm =
G_norm =
B_norm =
RGB = # Stacked 3D array
plt.imshow(RGB)
```

## Chapter 32

# RGB Image using astropy

Astropy.visualization has a function `make_lupton_rgb` which returns a Red/Green/Blue color image from up to 3 images using an asinh stretch. The input images can be int or float, and in any range or bit-depth. [Learn more](#)



## Chapter 33

# Puzzle I

### Step 1

Write code to open the FITS file named 'puzzle\_1.fits' and store the data in a variable (say RGB1 )and display the RGB image

### Step 2

Write code to make RGB image of 'SN 1054' with 'log' scaling, and radius equal to 9 arcminutes and 512 pixels using surveys 'DSS2 IR', 'DSS2 Red' and 'DSS2 Blue'. Let the RGB image be stored in a variable named RGB2

Make a new 3D array which stores the difference between RGB2 and RGB1. Plot the G band data of this new array.

```
RGB_diff =
```

```
G_diff =
```

Save the plot for submission and later reference





## Chapter 34

# Saving FITS image

Saving the final step as a fits file with name 'puzzle\_1\_solution.fits'

```
hdu = fits.PrimaryHDU(G_diff)
hdul = fits.HDUList(hdu)
hdul.writeto('puzzle_1_solution.fits', overwrite = True)
```



## Chapter 35

# Time Python

### 35.1 The Concept of Time in Python

#### ▼ Get Started



In addition to specialized astronomical timekeeping with libraries like `Astropy`, Python has built-in ways to handle everyday dates and times. This is useful for things like marking when a script was run, logging observation events in a simple way, or calculating durations in human-understandable units.

Python's standard library includes modules like `datetime` and `time` for this purpose. The `datetime` module is particularly powerful for working with calendar dates and times together.

While these built-in tools don't handle the complex astronomical timescales (like BJD) or relativistic corrections that `astropy.time` provides, they are excellent for general-purpose time tracking and calculation.

*# Example 1: Getting the Current Time in Python*

```
The 'datetime' module is commonly used for dates and times
import datetime
```

```
Get the current date and time
current_moment = datetime.datetime.now()

print(f"Current local time: {current_moment}")

You can access parts of the datetime object
print(f"Current year: {current_moment.year}")
print(f"Current month: {current_moment.month}")
print(f"Current day: {current_moment.day}")
print(f"Current hour: {current_moment.hour}")
print(f"Current minute: {current_moment.minute}")
```

```
Current local time: 2025-06-10 20:04:31.154310
Current year: 2025
Current month: 6
Current day: 10
Current hour: 20
Current minute: 4
```

#### *# Example 2: Creating a Specific Date and Time*

```
You can create a specific date and time object
For example, the date of the first detection of a gravitational wave event (GW150914).
event_date = datetime.datetime(2015, 9, 14, 9, 50, 45) # Year, Month, Day, Hour, Minute

print(f"Specific event time: {event_date}")

You can also create just a date or just a time object
just_a_date = datetime.date(2023, 11, 15)
print(f"Just a date: {just_a_date}")
```

```
Specific event time: 2015-09-14 09:50:45
Just a date: 2023-11-15
```

#### *# Example 3: Calculating the Difference Between Two Times*

```
You can subtract one datetime object from another to get a time difference
time1 = datetime.datetime(2023, 11, 15, 10, 0, 0) # First observation
time2 = datetime.datetime(2023, 11, 15, 10, 30, 0) # Second observation

Subtracting gives a timedelta object
duration = time2 - time1

print(f"Time difference (duration): {duration}")
```

```
print(f"Duration in minutes: {duration.total_seconds() / 60}")

Check if a time falls within a range (less common for beginners, but useful)
start_window = datetime.datetime(2023, 11, 15, 9, 0, 0)
end_window = datetime.datetime(2023, 11, 15, 11, 0, 0)

print(f"Is {time1} within the window? {start_window <= time1 <= end_window}")
```

```
Time difference (duration): 0:30:00
Duration in minutes: 30.0
Is 2023-11-15 10:00:00 within the window? True
```

### 35.1.1 Converting Columns to Datetime

Often, when you load data from a file or create a DataFrame from other sources, columns containing dates or times are treated as simple text strings or numbers. To leverage Pandas' time-aware functionalities, you need to convert these columns into a special datetime format that Pandas recognizes, typically `datetime64[ns]`. The key function for this conversion is `pd.to_datetime()`.

*# Example 1: Converting a Column to Datetime*

```
import pandas as pd

Create a simple DataFrame with date/time information as strings
event_data = {
 'Event_ID': [101, 102, 103, 104],
 'Timestamp_String': ['2023-01-05 09:00', '2023-01-06 10:30', '2023-01-07 11:00', '2023-01-08 14:15'],
 'Value': [15, 22, 18, 25]
}
events_df = pd.DataFrame(event_data)

print("Original DataFrame:")
events_df
```

Original DataFrame:

	Event_ID	Timestamp_String	Value
0	101	2023-01-05 09:00	15
1	102	2023-01-06 10:30	22
2	103	2023-01-07 11:00	18
3	104	2023-01-08 14:15	25

```
print("\nData Types Before Conversion:")
events_df.dtypes # 'Timestamp_String' is likely 'object' (string)
```

Data Types Before Conversion:

```
Event_ID int64
Timestamp_String object
Value int64
dtype: object
```

```
Use pd.to_datetime to convert the 'Timestamp_String' column
events_df['Timestamp_String'] = pd.to_datetime(events_df['Timestamp_String'])

print("\nDataFrame After Conversion:")
events_df
```

DataFrame After Conversion:

	Event_ID	Timestamp_String	Value
0	101	2023-01-05 09:00:00	15
1	102	2023-01-06 10:30:00	22
2	103	2023-01-07 11:00:00	18
3	104	2023-01-08 14:15:00	25

```
print("\nData Types After Conversion:")
events_df.dtypes # 'Timestamp_String' is now 'datetime64[ns]'
```

Data Types After Conversion:

```
Event_ID int64
Timestamp_String datetime64[ns]
Value int64
dtype: object
```

### 35.1.2 Filtering DataFrames by Time

Once you have a column with the `datetime64[ns]` data type, you can easily filter your DataFrame to select rows based on specific dates, times, or time ranges. Pandas allows you to use comparison operators (`>=`, `<=`, `==`) and even slice using date strings directly on a datetime column (or index).

*# Example 2: Filtering a DataFrame by Time Ranges*

```
Using the DataFrame from Example 1, with 'Timestamp_String' converted
print("DataFrame for Filtering:")
events_df
```

DataFrame for Filtering:

	Event_ID	Timestamp_String	Value
0	101	2023-01-05 09:00:00	15
1	102	2023-01-06 10:30:00	22
2	103	2023-01-07 11:00:00	18
3	104	2023-01-08 14:15:00	25

*# Filter for events on or after a specific date*

```
filter_start_date = '2023-01-06'
events_after_date = events_df[events_df['Timestamp_String'] >= filter_start_date]
print(f"\nEvents on or after {filter_start_date}:")
events_after_date
```

Events on or after 2023-01-06:

	Event_ID	Timestamp_String	Value
1	102	2023-01-06 10:30:00	22
2	103	2023-01-07 11:00:00	18
3	104	2023-01-08 14:15:00	25

*# Filter for events between two dates*

```
start_range = '2023-01-06'
end_range = '2023-01-07 23:59:59' # Include the whole day of the 7th
events_in_range = events_df[(events_df['Timestamp_String'] >= start_range) & (events_df['Timestamp_String'] < end_range)]
print(f"\nEvents between {start_range} and {end_range} (inclusive):")
events_in_range
```

Events between 2023-01-06 and 2023-01-07 23:59:59 (inclusive):

	Event_ID	Timestamp_String	Value
1	102	2023-01-06 10:30:00	22
2	103	2023-01-07 11:00:00	18

### 35.1.3 Using Time as the DataFrame Index

For data that is inherently sequential in time (like time series data), it is often very convenient to set the column containing the time information as the

DataFrame's index. This allows for powerful and concise time-based slicing, lookups, and operations.

*# Example 3: Setting Time as DataFrame Index*

```
Using the DataFrame from Example 1, with 'Timestamp_String' as datetime
print("DataFrame Before Setting Index:")
events_df[['Event_ID', 'Timestamp_String', 'Value']]
```

DataFrame Before Setting Index:

	Event_ID	Timestamp_String	Value
0	101	2023-01-05 09:00:00	15
1	102	2023-01-06 10:30:00	22
2	103	2023-01-07 11:00:00	18
3	104	2023-01-08 14:15:00	25

```
Set the 'Timestamp_String' column as the index
drop=True removes the column from the DataFrame after setting it as index
events_df_indexed = events_df.set_index('Timestamp_String', drop=True)

print("\nDataFrame with Timestamp_String as Index:")
events_df_indexed
```

DataFrame with Timestamp\_String as Index:

	Event_ID	Value
Timestamp_String		
2023-01-05 09:00:00	101	15
2023-01-06 10:30:00	102	22
2023-01-07 11:00:00	103	18
2023-01-08 14:15:00	104	25

```
You can now slice the DataFrame using time strings directly on the index
Slicing on a datetime index is often inclusive of the end bound
slice_by_date_string = events_df_indexed['2023-01-06':'2023-01-07'] # Slice from 2023-01-06 to 2023-01-07
print(f"\nSlice using index ['2023-01-06':'2023-01-07']:")
slice_by_date_string
```

Slice using index ['2023-01-06':'2023-01-07']:



	Event_ID	Value
Timestamp_String		
2023-01-06 10:30:00	102	22
2023-01-07 11:00:00	103	18

```
Selecting data for a specific day
select_day_data = events_df_indexed.loc['2023-01-08']
print(f"\nSelect data for '2023-01-08' using index.loc:")
select_day_data # Note: .loc is generally preferred for label-based indexing
```

Select data for '2023-01-08' using index.loc:

	Event_ID	Value
Timestamp_String		
2023-01-08 14:15:00	104	25



## Chapter 36

# The Time Domain in Astronomy

### 36.1 The Concept of Time in Astronomical Observations

Understanding how astronomical objects change requires accurately recording when observations are made. While a standard clock on Earth gives us a convenient measure of local time, precise astronomical comparisons, especially in **Time Domain Astronomy**, necessitate a more rigorous approach.

The complexity arises because our telescopes are located on a moving platform: the Earth. The Earth rotates on its axis and orbits the Sun. These motions affect the exact moment light from a distant source arrives at the telescope compared to when it would arrive at a fixed, inertial point (like the center of mass of the Solar System, the barycenter). Relativistic effects, though often small, can also play a role for high-precision work.

To overcome this, astronomers use standard astronomical timescales and coordinate systems. These scales provide a uniform reference, allowing observations taken at different locations or significantly different times to be compared consistently. For example, **Barycentric Julian Date (BJD)** is a widely used timescale that corrects the observation time to the moment light would have arrived at the Solar System barycenter. Using a standard scale ensures that observed variations (like the dip during an exoplanet transit) are physically real events occurring at a consistent time relative to the source, rather than artifacts of the observer's motion.

Handling these various timescales and formats (like Julian Date, Modified Julian Date, or precise calendar representations) accurately in computational analysis

is crucial. In Python, the `astropy.time` module is the standard and recommended library specifically designed for this purpose, providing tools to represent, convert, and perform arithmetic with astronomical time coordinates.

## 36.2 Handling Astronomical Time with Astropy

Accurately representing and manipulating time is fundamental in time domain astronomy. Observations from different telescopes, locations, or epochs need to be compared in a consistent framework. Simple numerical counts (like “days since the first observation”) are not sufficient for precise astronomical work due to complexities like Earth’s motion, rotation, and relativistic effects.

The `astropy.time` module provides a powerful and flexible way to handle astronomical time coordinates. The core object is `astropy.time.Time`. It allows you to:

- Create time objects from various standard formats (e.g., ISO 8601 strings, Julian Date, Modified Julian Date).
- Specify the appropriate **time scale** (e.g., UTC, Barycentric Julian Date - BJD, Terrestrial Time - TT).
- Convert between different time scales and formats.
- Perform arithmetic operations with time objects.
- Prepare time data for accurate plotting on a time axis.

Let’s look at a few examples of how to create and manipulate Astropy Time Objects First, let’s import the modules.

```
import numpy as np
from astropy.time import Time
import astropy.units as u # Import units for time arithmetic
import matplotlib.pyplot as plt
import pandas as pd
```

### 36.2.1 Creating Time Objects

Let’s look at the different time formats. All `astropy` time formats can be found at <https://docs.astropy.org/en/stable/time/index.html#time-format>.

#### 36.2.1.1 1. From a standard calendar date/time string (ISO 8601 format)

Specify the time scale, e.g., `utc` for Coordinated Universal Time

```
observation_time_utc = Time('2023-11-15 12:00:00', scale='utc')
print(f"Time from ISO string (UTC): {observation_time_utc.iso}")
```

```
Time from ISO string (UTC): 2023-11-15 12:00:00.000
```

### 36.2.1.2 2. From Julian Date (JD)

Julian Date or JD is a continuous count of days since noon Universal Time on Jan 1, 4713 BC. See <https://www.aavso.org/about-jd> for more details on JD.

```
time_jd_number = 2460265.0 # Corresponds to 2023-11-16 00:00:00 UTC
obs_time_jd = Time(time_jd_number, format='jd', scale='utc')
print(f"Time from Julian Date (UTC): {obs_time_jd.jd}")
```

```
Time from Julian Date (UTC): 2460265.0
```

### 36.2.1.3 3. From Modified Julian Date (MJD)

MJD is days since Nov 17, 1858, common in astronomical archives. For more on MJD, see <https://core2.gsfc.nasa.gov/time/>

```
time_mjd_number = 60264.5 # Corresponds to 2023-11-15 00:00:00 UTC
obs_time_mjd = Time(time_mjd_number, format='mjd', scale='utc')
print(f"Time from MJD number (UTC): {obs_time_mjd.mjd}")
```

```
Time from MJD number (UTC): 60264.5
```

## 36.2.2 Converting Between Formats

Let's convert our UTC Time object to other formats

```
print(f"\n{observation_time_utc.iso} in MJD: {observation_time_utc.mjd}")
print(f"{observation_time_utc.iso} in JD: {observation_time_utc.jd}")
```

```
2023-11-15 12:00:00.000 in MJD: 60263.5
2023-11-15 12:00:00.000 in JD: 2460264.0
```

```
--- Performing Time Arithmetic ---
```

```
Add 1 day and 6 hours to the observation time
time_plus_day_hours = observation_time_utc + 1*u.day + 6*u.hour
print(f"\nTime + 1 day 6 hours: {time_plus_day_hours.iso}")
```

```
Time + 1 day 6 hours: 2023-11-16 18:00:00.000
```

```
Calculate the difference between two times (result is an Astropy TimeDelta object)
time_difference = Time('2023-11-16 00:00:00') - observation_time_utc
print(f"Time difference: {time_difference}")
print(f"Time difference in hours: {time_difference.to(u.hour)}")
```

```
Time difference: 0.5
Time difference in hours: 12.0 h
```

```
Astropy Time can handle arrays of times
mjd_array = np.array([60264.1, 60264.2, 60264.3]) # Sequence of MJD values
time_array_mjd = Time(mjd_array, format='mjd', scale='utc')
print(f"\nAstropy Time array from MJD:\n{time_array_mjd}")
```

```
Astropy Time array from MJD:
[60264.1 60264.2 60264.3]
```

```
Accessing individual times from an array
print(f"Second time in the array (MJD {time_array_mjd[1].mjd}): {time_array_mjd[1].iso}")
```

```
Second time in the array (MJD 60264.2): 2023-11-16 04:48:00.000
```

## 36.3 The Light Curve

The light curve is the fundamental tool for analyzing time series data in astronomy. It is a two-dimensional plot of an object's measured brightness as a function of time. The morphology of a light curve—its shape, amplitude, period, and duration of features—reveals the physical processes driving the observed variability.

- **X-axis (Time):** Represents the time of each observation. For precise analysis, this is often a standard astronomical time coordinate like Barycentric Julian Date (BJD).
- **Y-axis (Brightness):** Represents the measured brightness, typically as **Flux** (a linear measure of energy) or **Magnitude** (a logarithmic, inverted scale).

### 36.3.1 The Importance of a Standard Time Reference: BJD

Astronomical observations are made from Earth, a platform that is both rotating and orbiting the Sun. This motion introduces variations in the light-travel time from a celestial source to the telescope. To remove this effect and accurately compare data from different epochs or observatories, astronomers convert observation times to a common, stable reference frame.

The most common reference is the Solar System Barycenter (the system's center of mass). Times corrected to this frame are given as **Barycentric Julian Date (BJD)**. Using BJD ensures that any observed timing variations are intrinsic to the astronomical source, not an artifact of the observer's motion. The `astropy.time` module is the standard tool for performing this conversion.

## 36.4 Common Light Curve Examples

### 36.4.1 OGLE-2019-BLG-1180

Below is the OGLE/KMTNet light curve for a planetary microlensing event OGLE-2019-BLG-1180. You can read more about the event in arXiv or the IOP publication. The event was observed with OGLE and KMTNet in optical I band magnitude and the time is in HJD.

```
from astropy.table import Table
from astroquery.vizier import Vizier

vizier = Vizier()

vizier.ROW_LIMIT = -1
ogle = vizier.get_catalogs("J/AJ/166/253/fig1")[0]

ogle[:5]
```

```
<Table length=5>
 Inst HJD Imag e_Imag
 str6 float64 float32 float32

KMTA02 8534.23049 19.001 0.175
KMTA02 8536.25367 19.351 0.179
KMTA02 8536.26887 19.180 0.164
```

```
KMTA02 8539.26076 18.844 0.116
KMTA02 8540.25279 18.862 0.115
```

```
jd_offset = 2450000
times = Time(ogle['HJD'] + jd_offset, format='jd', scale='utc')
x_vals = times.jd - jd_offset

2. Now plot Imag vs time with error bars

fig, ax = plt.subplots(figsize=(8,4))

ax.errorbar(x_vals, ogle['Imag'], yerr=ogle['e_Imag'], color='maroon', fmt='o', marker:
ax.invert_yaxis() # Magnitudes get brighter going down
ax.set(xlabel='heliocentric Julian Date - 2450000 (d)', ylabel='I band (mag)')
ax.set_title('Light Curve')
Disable scientific notation on x-axis
ax.ticklabel_format(style='plain', axis='x')

Rotate x-axis labels for neatness
plt.setp(ax.get_xticklabels(), rotation=45, ha='right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

<Figure size 800x400 with 1 Axes>

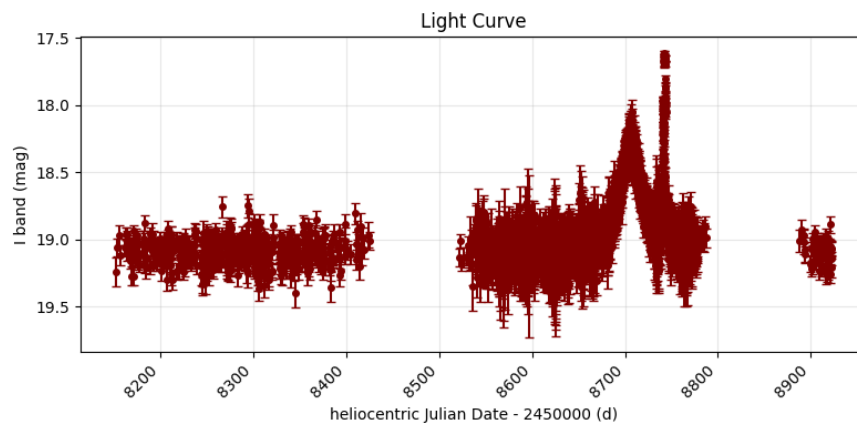


Figure 36.1: output



The light curve shows a transient event which spans from days 8650 to 8750 with a significant rise and fall and a long duration with 1 magnitude or more higher than the quiescent level.

## 36.5 Sgr A\*

The first detection from the Spitzer Space Telescope of 4.5  $\mu$ m variability from Sgr A\*, the emitting source associated with the Milky Way's central black hole. See the paper here <https://iopscience.iop.org/article/10.1088/0004-637X/793/2/120> or in arXiv.

```
sgr = vizier.get_catalogs("J/ApJ/793/120/table1")[0]

sgr[:5]
```

```
<Table length=5>
 MJD SgrA* Ref
 d Jy Jy
 float64 float64 float64

56636.1808736 0.00210 0.00036
56636.1809705 0.00404 0.00228
56636.1810678 0.00393 0.00001
56636.1811650 0.00294 0.00021
56636.1812623 0.00233 0.00096
```

```
mjd_offset = 56630
sgr_time = Time(sgr['MJD'], format='mjd', scale='utc')

x_vals = sgr_time.mjd - mjd_offset

2. Now plot Imag vs time with error bars

fig, ax = plt.subplots(figsize=(8,4))

ax.scatter(x_vals, sgr['SgrA*'], color='maroon', s=4)

ax.set(xlabel='Modified Julian Date - 56630 (d)', ylabel=r'4.5μm flux (Jy)')
ax.set_title('Light Curve of Sagittarius A')
Disable scientific notation on x-axis
ax.ticklabel_format(style='plain', axis='x')

Rotate x-axis labels for neatness
```

```
plt.setp(ax.get_xticklabels(), rotation=45, ha='right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

<Figure size 800x400 with 1 Axes>

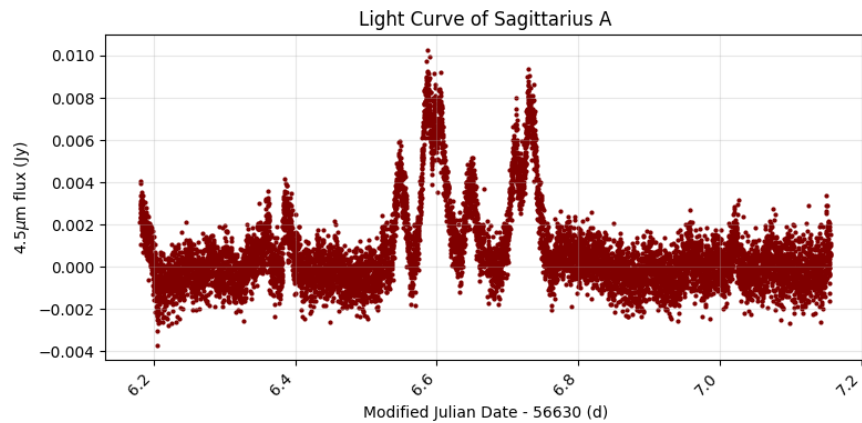


Figure 36.2: output

The light curve shows a function of the infra-red flux over time. A significant variability is noticed between MJD 56636.5 - 56636.8 days.

## 36.6 Radio light curve

Variability in the radio part of the electromagnetic spectrum typically originates from physical processes that are very different from the thermal (heat-based) emission we see from stars in the optical. The primary mechanisms are:

- **Synchrotron Emission:** Generated by relativistic electrons spiraling in strong magnetic fields. This is common in the jets launched by Active Galactic Nuclei (AGN), in the remnants of supernovae, and in stellar flares.
- **Coherent Emission:** A process where particles emit radiation in phase with each other, producing intensely bright, focused beams. This is the mechanism responsible for the radio emission from pulsars.

Radio brightness is measured in terms of Flux Density, with the standard unit being the Jansky (Jy), where  $1 \text{ Jy} = 10^{-26} \text{ W m}^{-2} \text{ Hz}^{-1}$ .

Some examples of objects studied with radio light curves include:

- Pulsars: Rapidly rotating neutron stars that emit beams of radio waves. If the beam sweeps across our line of sight, we observe incredibly regular, sharp pulses. The light curve consists of a series of narrow, periodic spikes against a noisy background.
- Active Galactic Nuclei (AGN): The central regions of galaxies hosting supermassive black holes. Their radio light curves often show slower, aperiodic “flickering” or large outbursts over months to years as material interacts with the accretion disk and jets.
- Fast Radio Bursts (FRBs): Mysterious, extremely powerful, and very brief (millisecond-duration) bursts of radio waves from extragalactic sources. Their light curve is a single, sharp spike.

Below is an example of a radio light curve from arXiv of the source (J1402+5347) showing extreme (~50%) and variations on a timescale of just 6.5 minutes observed in L band (1.4 GHz).

```
rad = vizier.get_catalogs("J/A+A/641/L4/02-03-20")[0]

rad[:5]

rad_time = Time(rad['JD'], format='jd', scale='utc')

times = Time(ogle['HJD'] + jd_offset, format='jd', scale='utc')
rad_time = rad_time.jd - jd_offset

fig, ax = plt.subplots(figsize=(8,4))

ax.scatter(rad_time, rad['Flux'], color='maroon', s=4,)

ax.set(xlabel='Julian Date - 2450000 (d)', ylabel=r'Flux density (Jy)')
ax.set_title('Light Curve of J1402+5347 observed on 02-03-20 at 1.4GHz')
Disable scientific notation on x-axis
ax.ticklabel_format(style='plain', axis='x')

Rotate x-axis labels for neatness
plt.setp(ax.get_xticklabels(), rotation=45, ha='right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

The radio source showed multiple variation across the observed timescale.

### 36.6.1 RL Leo

An example is RL Leo, a variable star at a distance of 4731 pc. The lightcurve shown below is from ASAS-SN Catalog of Variable Stars.

```
rleo = pd.read_csv("ASASSN-V J092730.63+110739.6.csv")
rleo = rleo[rleo['mag_err']<1]
rleo.head()

rleo.describe()

rl_time = Time(rleo['hjd'], format='jd', scale='utc')

rl_time = rl_time.jd - jd_offset

fig, ax = plt.subplots(figsize=(8,4))

ax.scatter(rl_time, rleo['mag'], color='maroon',s=4,)
ax.errorbar(rl_time, rleo['mag'], yerr=rleo['mag_err'], color='maroon', fmt='o', marke

ax.set(xlabel='Julian Date - 2450000 (d)', ylabel=r'V magnitude (mag)')
ax.invert_yaxis() # Magnitudes get brighter going down

ax.set_title('Light Curve of LR Leo')
Disable scientific notation on x-axis
ax.ticklabel_format(style='plain', axis='x')

Rotate x-axis labels for neatness
plt.setp(ax.get_xticklabels(), rotation=45, ha='right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

## 36.7 Summary

This notebook provided an introduction to working with astronomical time series data in Python, a fundamental skill in Time Domain Astronomy. You learned:

- Time series data captures the variability of celestial objects, including **transients**.
- **Light curves** are plots of brightness (quantified as **flux** or **magnitude**) versus time.

- Accurate time handling is crucial and is best managed using the **Astropy Time** module.
- Astropy **Time** objects can be created from various formats, converted between **time scales**, and used directly for plotting in Matplotlib.
- Variability patterns differ across the electromagnetic spectrum (Optical, IR, UV, X-ray, Radio), reflecting distinct physical processes.

Working with time series data is essential for understanding the dynamic universe, from pulsating stars to black hole flares and supernovae. This introduction provides a solid foundation for further exploration into time series analysis techniques. The next notebook will demonstrate how to query lightcurve data from TESS (Transiting Exoplanet Survey Satellite) and how to find an orbital period from a light curve.

## 36.8 Additional Resources

Time and Dates in Astronomy: <https://docs.astropy.org/en/stable/time/index.html>



## Chapter 37

# Time-series Analysis for Astronomical Data





## Chapter 38

# 1. What is Time-Series Data?

Time-series data is a sequence of data points collected or recorded at regular or known time intervals. Each data point represents a measurement taken at a specific moment in time. In astronomy, time-series data is extremely important. Astronomers often use it to study how the brightness, position, or other properties of celestial objects (like stars, planets, and galaxies) change over time.

The most common type of time-series data in astronomy is related to brightness, which is also called luminosity or magnitude. For example:

- **Variable Stars** Some stars naturally get brighter and dimmer over time. By recording their brightness every night, scientists get time-series data that shows a repeating pattern. This helps identify the type of variable star and how it works.
- **Exoplanets** When a planet moves in front of its star (as seen from Earth), it blocks a small part of the star's light. This creates a tiny dip in the light curve. By analyzing this time-series data, astronomers can discover new exoplanets.
- **Pulsars** Pulsars are spinning neutron stars that emit regular pulses of radiation, like a cosmic lighthouse. These pulses arrive at very regular intervals, so astronomers use time-series data to track their timing and speed.
- **Supernovae** When a star explodes, it suddenly becomes very bright, then slowly fades. By recording its brightness daily, astronomers create a light curve that shows the explosion's timeline. Astronomers record these brightness levels again and again over time, creating a time-series dataset.

In this lecture, we will exploit TESS data with ‘lightkurve’ to identify periodic signals in a variable star.

**Note:** If you are running this jupyter notebook from Colab, then run the next cell by pressing SHIFT+ENTER to install the required packages for this notebook. Otherwise, skip the next cell.

```
!pip install -q astroquery lightkurve
```

```
import numpy as np
import matplotlib.pyplot as plt
from astropy.timeseries import LombScargle
import lightkurve as lk
import os
```

## Chapter 39

# 2. What Data Products are Available?

TESS data products are stored on the Mikulski Archive for Space Telescopes (MAST) in two main forms:

1. Light curve products: tables containing the measured flux at each observation time.
2. Target pixel file products: stacks of images with the pixel-level observation at each observation time.

Lightkurve allows you to query and download each of these data products.



## Chapter 40

### 3. Searching for Light Curves

Lightcurve uses Astroquery to search for data products. Astroquery allows searches based on a target's coordinates, catalog ID number, or name.

This is passed into the search function using the target keyword, and all valid inputs for identifying a target include: - The TESS Input Catalog (TIC) identifier as an integer, for example, "464570167" - A coordinate string in decimal format, for example, "285.67942179 +50.24130576." - A coordinate string in sexagesimal format, for example, "19:02:43.1 +50:14:28.7."

You can also specify which mission you would like to retrieve data from using the mission keyword, which takes "Kepler," "K2," or "TESS." By default, all available missions will be returned.

We will start with the case of searching for a TESS target using its TESS Input Catalog (TIC) ID number. Below, we search for TIC 346783960, commonly known as Beta Persei, which is an eclipsing binary, using the `search_lightcurve` function.

```
search_result = lk.search_lightcurve(target='Beta Persei', mission='TESS', author='SPOC')
search_result
```

SearchResult containing 4 data products.

#	mission	year	author	exptime s	target_name	distance arcsec
0	TESS Sector 18	2019	SPOC	120	346783960	0.0
1	TESS Sector 58	2022	SPOC	120	346783960	0.0

2	TESS Sector 85	2024	SPOC	20	346783960	0.0
3	TESS Sector 85	2024	SPOC	120	346783960	0.0

`'search_lightcurve'` returns a `SearchResult` table, which contains information about the data products available to download. This search result tells us that TIC 346783960 was observed in TESS Sectors 18, 58, and 85. The **author** keyword helps refine the results by selecting data from a specific pipeline (e.g., SPOC), which provides 20 s and 120 s cadence data.

You can also search for available data using your target's coordinates instead of its name.

```
search_result = lk.search_lightcurve(target='03 08 10.13 +40 57 20.33', radius=0.0, au
search_result
```

SearchResult containing 4 data products.

#	mission	year	author	exptime s	target_name	distance arcsec
0	TESS Sector 18	2019	SPOC	120	346783960	0.0
1	TESS Sector 58	2022	SPOC	120	346783960	0.0
2	TESS Sector 85	2024	SPOC	20	346783960	0.0
3	TESS Sector 85	2024	SPOC	120	346783960	0.0

You can select an individual entry in this search result by indexing the search result.

```
search_result[0]
```

SearchResult containing 1 data products.

#	mission	year	author	exptime s	target_name	distance arcsec
0	TESS Sector 18	2019	SPOC	120	346783960	0.0

You can also narrow down the list of observations when you make the search using the mission-specific keywords, such as **sector**.

Let's perform the search for TIC 346783960 again, this time specifying that we only want data from TESS Sector 18.

```
search_result_18 = lk.search_lightcurve(target='Beta Persei', mission='TESS', author='SPOC', sector=18)
search_result_18
```

SearchResult containing 1 data products.

#	mission	year	author	exptime s	target_name	distance arcsec
0	TESS Sector 18	2019	SPOC	120	346783960	0.0

You can change the value of the ‘sector’ keyword to select different sectors.

## 40.1 3.1 Downloading a single light curve

A light curve can be downloaded by calling `.download()`.

```
lc = search_result_18.download()
lc
```

```
<TessLightCurve length=16447 LABEL="TIC 346783960" SECTOR=18 AUTHOR=SPOC FLUX_ORIGIN=pdcsap_flux>
```

time	flux	...	pos_corr1	pos_corr2
	electron / s	...	pix	pix
Time	float32	...	float32	float32
1790.661752048892	1.5847028e+07	...	-3.3331234e-02	-9.5643811e-02
1790.6631409781724	1.5852229e+07	...	-2.6948692e-02	-1.1051010e-01
1790.6645299069883	1.5860948e+07	...	-2.8188746e-02	-1.0195550e-01
1790.6659188358046	1.5884782e+07	...	-2.9503489e-02	-9.5719874e-02
1790.6673077646203	1.5884525e+07	...	-2.7450016e-02	-1.1521547e-01
1790.6686966934355	1.5886712e+07	...	-3.1409565e-02	-1.0103273e-01
1790.6700856222503	1.5897793e+07	...	-3.2533020e-02	-1.0061166e-01
1790.671474551066	1.5907906e+07	...	-2.9720914e-02	-1.0519896e-01
1790.672863479882	1.5916688e+07	...	-2.9785568e-02	-1.0134164e-01
...	...	...	...	...
1815.0243521604523	...	...	...	...
1815.0257410301285	...	...	...	...
1815.0271298993396	...	...	...	...
1815.028518769017	...	...	...	...
1815.0299076386939	...	...	...	...
1815.031296507904	...	...	...	...
1815.0326853775807	...	...	...	...
1815.0340742472574	...	...	...	...
1815.0354631164685	...	...	...	...

This returns a single `TessLightCurve` object, which is shown above in the form of an astropy table.

The 'lc' files contain the flux time series derived from the calibrated 2-minute and 20-second target pixels. These files contain several time series, including the simple aperture photometry, a detrended light curve, position vectors, and quality flags. The aperture photometry flux series is known as the `SAP_FLUX`, while the flux series that has the common instrumental systematics removed is known as the `PDCSAP_FLUX`. The `PDCSAP_FLUX` light curves also correct for the amount of flux captured by the photometric aperture and crowding from known nearby stars.

You can read more about the TESS data products here: <https://outerspace.stsci.edu/display/TESS/TESS+Archive+Manual>

We can examine and plot the light curve now.

```
lc_clean = lc.remove_nans().remove_outliers() #To remove nan values and outliers
plt.plot(lc_clean.time.value, lc_clean.pdcsap_flux.value)
plt.xlabel("Time (BTJD)")
plt.ylabel("Flux")
plt.show()
```

<Figure size 640x480 with 1 Axes>

The period of Beta Persei is approximately 2.867 days (or 2 days, 20 hours, 49 minutes).

This means that every 2.867 days, it undergoes a brightness dip as its eclipsing binary components pass in front of each other from our line of sight. It is one of the most famous eclipsing binaries, with a well-documented light curve showing periodic dimming due to stellar occultation.

```
Convert to pandas DataFrame
df = lc_clean.to_pandas()
df['time'] = lc_clean.time.value # .value gets rid of units/astropy Quantity
columns_to_save = ['time', 'sap_flux', 'sap_flux_err', 'pdcsap_flux', 'pdcsap_flux_err']
df1 = df[columns_to_save]

Create directory if it doesn't exist
output_dir = "data"
os.makedirs(output_dir, exist_ok=True)

Save to file inside the directory
output_path = os.path.join(output_dir, "lc_sec18.txt")
df1.to_csv(output_path, sep='\t', index=False)
```

You can save any light curve file to your local machine in this manner.



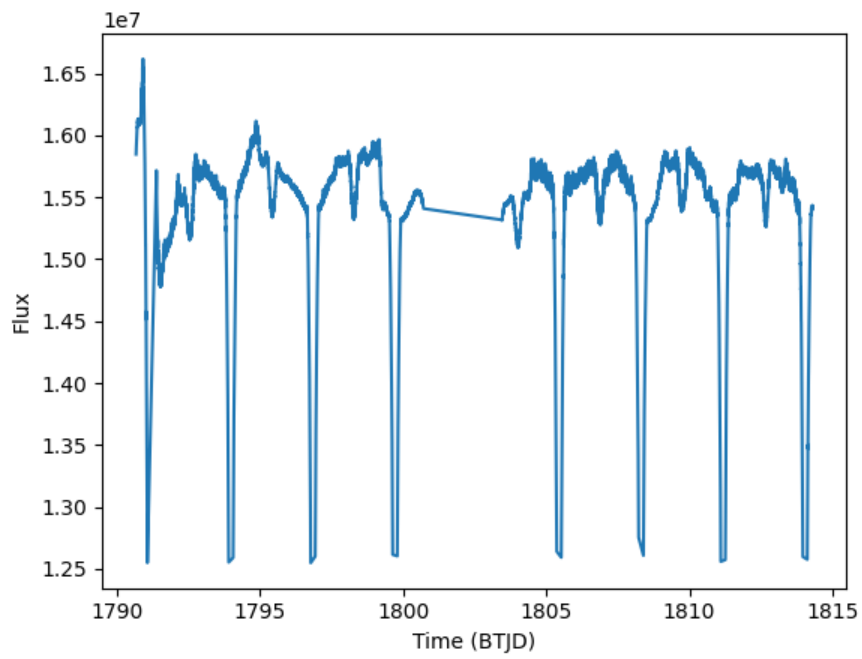


Figure 40.1: output



## Chapter 41

# 4. Finding periodicity in the light curve

When astronomers collect data over time, like how the brightness of a star changes, they often want to know if there's a pattern or cycle in the data. For example, does the star become brighter and dimmer in a regular way, like a periodic cycle?

To find these patterns, astronomers use a method called a periodogram, which is a commonly used statistical tool that helps us “listen” for repeating rhythms in the data. One special kind of periodogram is called the Lomb-Scargle (LS) periodogram (after Lomb[1], and Scargle

2

). This method is very popular in astronomy, especially when the data is not evenly spaced in time.

In an ideal world, astronomers would collect data at regular intervals — for example, every hour, or every night at the same time. But in the real world, especially in astronomy, this is practically impossible.

There are many reasons why data might not be evenly spaced: - The weather might be bad for telescope operation, for example, clouds, rain, high winds, etc. - The object might only be visible during certain seasons. - The telescope might be busy or offline. - It might be too expensive or difficult to observe constantly.

As a result, the data might be collected at irregular times. This makes it difficult to find patterns using standard tools like the Fourier Transform, which works best with evenly spaced data.

### 41.0.1 How Does the LS periodogram Work?

Input: - A dataset containing: - Times when data was collected (not equally spaced). - The values recorded at those times (e.g., brightness of a star).

Processing: - It checks many different frequencies (like testing many possible rhythms) and calculates how well each one fits your data.

Output: - It gives us another dataset with: - Frequency (or period). - Power (strength of the signal).

- The taller the peak, the stronger the evidence for a repeating pattern at that frequency.

Below, we use the in-built Python package ‘LombScargle’ to calculate the periodogram for our data.

```
ls = LombScargle(lc_clean.time.value, lc_clean.pdcsap_flux.value, lc_clean.pdcsap_flux,
freq, power = ls.autopower(nyquist_factor=1)
```

The Nyquist factor 1 means we would like to explore the periodogram to be calculated up to the Nyquist frequency, which is the maximum frequency you can capture correctly, and it’s exactly half of the sampling rate (cadence between two data points).

```
#period = 1 / freq
plt.plot(freq, power, color='blue', lw=1.0)
plt.xlim(0.1,1)
plt.ylim()
plt.xlabel("Frequency (day $^{-1}$)")
plt.ylabel("Power")
plt.title("Lomb-Scargle Periodogram")
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
best_freq = freq[np.argmax(power)]
best_period = 1/best_freq
print(f"Best frequency: {best_freq:.4f} day $^{-1}$, Best period: {best_period:.4f} days")
```

Best frequency: 0.3511 day  $^{-1}$ , Best period: 2.8486 days

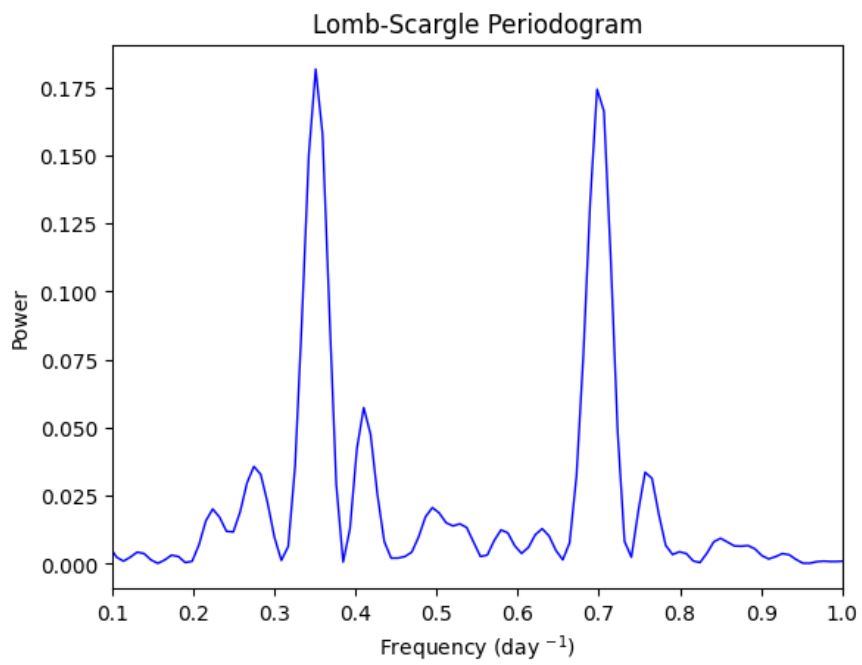


Figure 41.1: output



## Chapter 42

### 5. References

1. Lomb, N.R. Least-squares frequency analysis of unequally spaced data. Ap&SS 39 pp. 447-462 (1976)
2. Scargle, J. D. Studies in astronomical time series analysis. II - Statistical aspects of spectral analysis of unevenly spaced data. ApJ 1:263 pp. 835-853 (1982)





# Chapter 43

## 6. Exercise

1. Generate light curves of different sectors. Do you see any difference?
2. Generate periodograms of different light curves. What period did you derive?
3. What frequency is at  $\sim 0.70 \text{ day}^{-1}$ ?

### 43.1 Additional Resources

Getting started with Time series with Astropy: <https://docs.astropy.org/en/stable/timeseries/index.html>

Lomb Scargle periodograms: <https://docs.astropy.org/en/stable/timeseries/lombscargle.html>

Lomb-Scargle Periodograms for Multiband Data: <https://docs.astropy.org/en/stable/timeseries/lombscarglemb.html>

Lightcurve Tutorials: <https://lightcurve.github.io/lightcurve/tutorials/index.html>

TESS and Kepler Data: <https://lightcurve.github.io/lightcurve/tutorials/1-getting-started/searching-for-data-products.html>

Altaipony Flare Lightcurve Tutorial: <https://altaipony.readthedocs.io/en/latest/tutorials/index.html>

Stingray: <https://docs.stingray.science/en/stable/index.html>

High Energy time analysis with Gammapy: [https://docs.gammapy.org/dev/tutorials/analysis-time/light\\_curve\\_flare.html](https://docs.gammapy.org/dev/tutorials/analysis-time/light_curve_flare.html)



## Chapter 44

# Machine Learning for Astronomical Data Analysis: A Conceptual Introduction

### 44.1 Types of Machine Learning

Machine Learning is broadly categorized into two main types based on the kind of data used for training.

#### 44.1.1 1. Supervised Learning

In supervised learning, the algorithm learns from data that is already **labeled** with the correct outcome. The goal is to create a model that can predict the label for new, unlabeled data.

- **Analogy:** You teach a child to identify animals by showing them pictures, each with a label (“this is a cat,” “this is a dog”). After seeing enough examples, the child can identify new pictures of cats and dogs.

Supervised learning is primarily used for two types of tasks:

- **Classification:** The goal is to predict a discrete category. The output is a class label.

- **Question:** *What kind of object is this?*
- **Examples:** “Is this a star, a galaxy, or a quasar?”, “Is this supernova a Type Ia or a Type II?”
- **Regression:** The goal is to predict a continuous numerical value.
  - **Question:** *What is the numerical value of this property?*
  - **Examples:** “What is this galaxy’s redshift?”, “What is this star’s temperature?”

### 44.1.2 2. Unsupervised Learning

In unsupervised learning, the algorithm works with data that has **no labels**. The goal is to find hidden structures, patterns, or groupings within the data itself.

- **Analogy:** You give a child a box of mixed Lego bricks and ask them to sort them into piles. Without any prior instruction, they might group the bricks by color, by shape, or by size. The algorithm discovers the structure on its own.

Unsupervised learning is often used for:

- **Clustering:** The goal is to group similar data points together.
  - **Question:** *Are there natural groups in my data?*
  - **Examples:** “Find groups of stars with similar motions in the Gaia dataset (which might be a star cluster),” “Group galaxies based on their observed properties.”
- **Dimensionality Reduction:** The goal is to simplify a dataset by reducing the number of input features while retaining the most important information.
  - **Question:** *Can I simplify my data without losing key information?*
  - **Example:** “Take 100 different measurements of a galaxy and find the 3 most important underlying variables that describe most of the variation.”
- **Anomaly Detection:** The goal is to identify data points that are significantly different from the rest of the dataset.
  - **Question:** *What in my data is unusual or unexpected?*
  - **Example:** “Scan millions of star light curves and flag the ones that show a variability pattern never seen before.”

## 44.2 Common Use Cases in Astronomy

Here is a summary of how different ML tasks are applied to specific astronomical problems.

ML Task	Astronomical Problem	Example Question
<b>Classification</b>	<b>Object Identification</b> in images.	“Is the object in this image a star or a galaxy?”
	<b>Supernova Typing</b> from light curves.	“Based on its light curve, is this supernova a Type Ia, Ib, or II?”
	<b>Galaxy Morphology</b> classification.	“Is this galaxy a Spiral, Elliptical, or Irregular type?”
<b>Regression</b>	<b>Photometric Redshift Estimation.</b>	“Given the brightness of this galaxy in 5 different filters, what is its redshift?”
	<b>Stellar Parameter Estimation</b> from spectra.	“From this star’s spectrum, what are its temperature and surface gravity?”
<b>Clustering</b>	<b>Identifying Star Clusters and Associations</b> in large surveys like Gaia.	“Find groups of stars that are moving together through space.”
	<b>Discovering Galaxy Groups</b> based on shared properties.	“Are there distinct populations of galaxies in my dataset based on their color and size?”
<b>Anomaly Detection</b>	<b>Searching for Novel Transients</b> in real-time data streams from ZTF or LSST.	“Flag any light curve that doesn’t match known variable star or supernova models.”

## 44.3 Summary and Next Steps

This notebook provided a high-level conceptual overview of Machine Learning and its critical role in modern astronomy.

- **ML automates analysis** of massive datasets, enabling discoveries that are infeasible through manual inspection.

- **Supervised Learning** (Classification, Regression) uses labeled data to make predictions.
- **Unsupervised Learning** (Clustering, Anomaly Detection) uses unlabeled data to discover hidden patterns.

This is just the beginning. The next steps in learning ML involve hands-on practice with real data. The most popular Python libraries for these tasks are:

- **scikit-learn**: The go-to library for traditional machine learning algorithms (classification, regression, clustering).
- **TensorFlow** and **PyTorch**: Powerful libraries for deep learning, often used for more complex image and sequence analysis.

By understanding these core concepts, you are now better prepared to explore these tools and apply them to your own astronomical research questions.

## Chapter 45

# Introduction to Scikit-Learn

### 45.1 Loading an example dataset

scikit-learn comes with a few standard datasets, for instance the iris and digits datasets for classification and the boston house prices dataset for regression. Visit <https://scikit-learn.org/stable/datasets.html> to see all datasets available in scikit-learn.

```
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier
from sklearn import svm
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_ma
from sklearn.model_selection import train_test_split
import joblib
```

```
iris = datasets.load_iris()
digits = datasets.load_digits()
```

A dataset is a dictionary-like object that holds all the data and some metadata about the data. This data is stored in the `.data` member, which is a `n_samples, n_features` array. In the case of supervised problem, explanatory variables are stored in the `.target` member. More details on the different datasets can be found in the dedicated section.

For instance, in the case of the digits dataset, `digits.data` gives access to the features that can be used to classify the digits samples:

```
digits.data
```

```
array([[0., 0., 5., ..., 0., 0., 0.],
 [0., 0., 0., ..., 10., 0., 0.],
 [0., 0., 0., ..., 16., 9., 0.],
 ...,
 [0., 0., 1., ..., 6., 0., 0.],
 [0., 0., 2., ..., 12., 0., 0.],
 [0., 0., 10., ..., 12., 1., 0.]], shape=(1797, 64))
```

and `digits.target` gives the ground truth for the digit dataset, that is the number corresponding to each digit image that we are trying to learn:

```
digits.target
```

```
array([0, 1, 2, ..., 8, 9, 8], shape=(1797,))
```

```
import matplotlib.pyplot as plt

#Display the first digit
plt.figure(1, figsize=(6, 6))
plt.imshow(digits.images[0], cmap=plt.cm.plasma)
plt.show()
```

```
<Figure size 600x600 with 1 Axes>
```

## 45.2 Shape of the data arrays

The data is always a 2D array, `n_samples`, `n_features`, although the original data may have had a different shape. In the case of the digits, each original sample is an image of shape 8, 8 and can be accessed using:

```
digits.images[0]
```

```
array([[0., 0., 5., 13., 9., 1., 0., 0.],
 [0., 0., 13., 15., 10., 15., 5., 0.],
 [0., 3., 15., 2., 0., 11., 8., 0.],
 [0., 4., 12., 0., 0., 8., 8., 0.],
 [0., 5., 8., 0., 0., 9., 8., 0.],
 [0., 4., 11., 0., 1., 12., 7., 0.],
 [0., 2., 14., 5., 10., 12., 0., 0.],
 [0., 0., 6., 13., 10., 0., 0., 0.]])
```



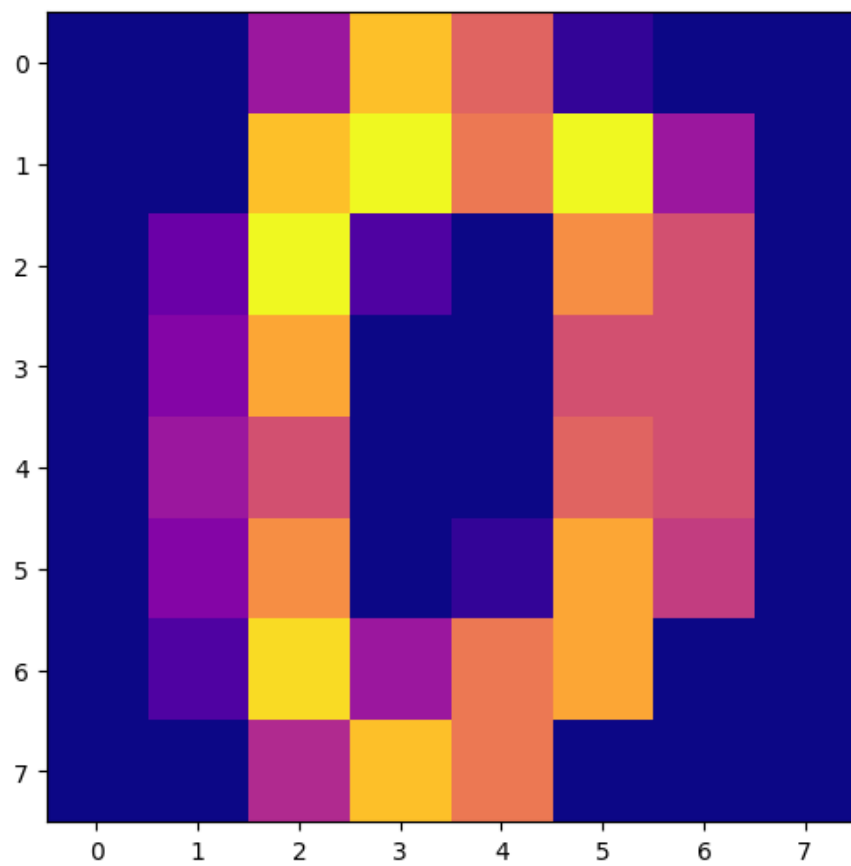


Figure 45.1: output

### 45.3 Learning and Predicting

For the digits dataset, the task is to predict the value of a hand-written digit from an image. There are samples of 10 possible classes on which you can fit an estimator to predict the labels corresponding to new data.

In scikit-learn, an estimator is just a plain Python class that implements the methods `fit(X, Y)` and `predict(T)`.

An example of estimator is `sklearn.svm.SVC` that implements Support Vector Classification. For more on SVC, see the documentation at <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>. From the documentation, you can see there are many parameters that could be defined when calling the class. The constructor of an estimator takes as arguments the parameters of the model, but for the time being, we will consider the estimator as a black box:

```
clf = svm.SVC(gamma=0.001, C=100.)
```

```
clf.fit(digits.data[:-1], digits.target[:-1])
```

```
SVC(C=100.0, gamma=0.001)
```

```
clf.predict(digits.data[-1].reshape(1, -1))
```

```
array([8])
```

```
import matplotlib.pyplot as plt
```

```
#Display the first digit
```

```
plt.figure(1, figsize=(6, 6))
```

```
plt.imshow(digits.images[2], cmap=plt.cm.plasma)
```

```
plt.show()
```

```
<Figure size 600x600 with 1 Axes>
```

```
The features (sepal length/width, petal length/width) are in the .data attribute
```

```
X = iris.data
```

```
X
```

```
array([[5.1, 3.5, 1.4, 0.2],
 [4.9, 3. , 1.4, 0.2],
```

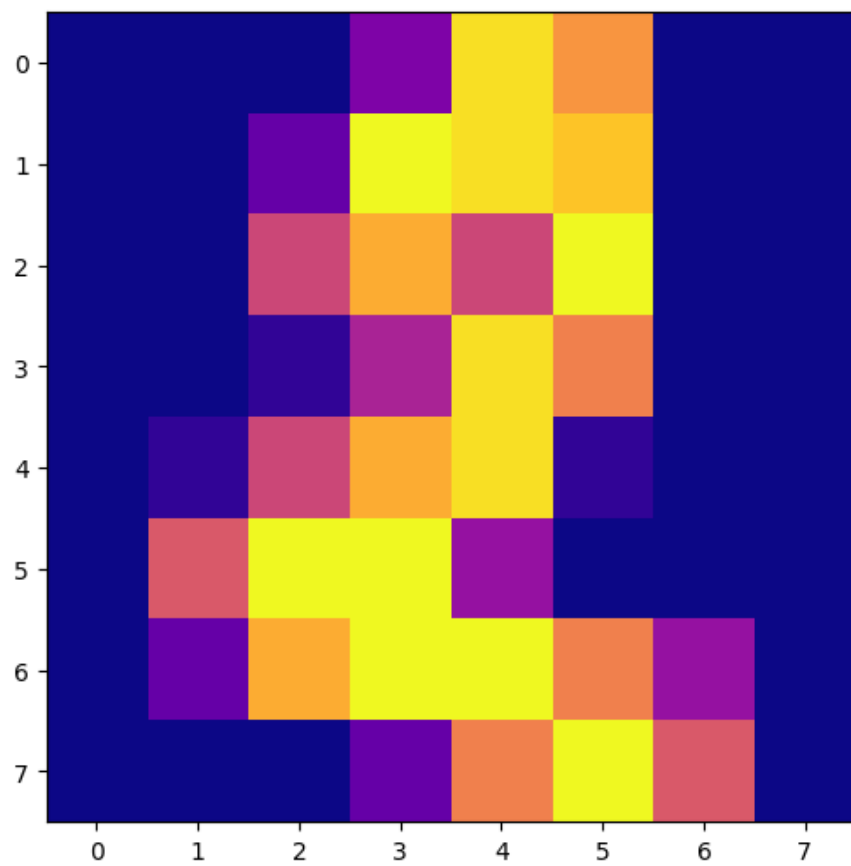


Figure 45.2: output

```
[4.7, 3.2, 1.3, 0.2],
[4.6, 3.1, 1.5, 0.2],
[5. , 3.6, 1.4, 0.2],
[5.4, 3.9, 1.7, 0.4],
[4.6, 3.4, 1.4, 0.3],
[5. , 3.4, 1.5, 0.2],
[4.4, 2.9, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.1],
[5.4, 3.7, 1.5, 0.2],
[4.8, 3.4, 1.6, 0.2],
[4.8, 3. , 1.4, 0.1],
[4.3, 3. , 1.1, 0.1],
[5.8, 4. , 1.2, 0.2],
[5.7, 4.4, 1.5, 0.4],
[5.4, 3.9, 1.3, 0.4],
[5.1, 3.5, 1.4, 0.3],
[5.7, 3.8, 1.7, 0.3],
[5.1, 3.8, 1.5, 0.3],
[5.4, 3.4, 1.7, 0.2],
[5.1, 3.7, 1.5, 0.4],
[4.6, 3.6, 1. , 0.2],
[5.1, 3.3, 1.7, 0.5],
[4.8, 3.4, 1.9, 0.2],
[5. , 3. , 1.6, 0.2],
[5. , 3.4, 1.6, 0.4],
[5.2, 3.5, 1.5, 0.2],
[5.2, 3.4, 1.4, 0.2],
[4.7, 3.2, 1.6, 0.2],
[4.8, 3.1, 1.6, 0.2],
[5.4, 3.4, 1.5, 0.4],
[5.2, 4.1, 1.5, 0.1],
[5.5, 4.2, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.2],
[5. , 3.2, 1.2, 0.2],
[5.5, 3.5, 1.3, 0.2],
[4.9, 3.6, 1.4, 0.1],
[4.4, 3. , 1.3, 0.2],
[5.1, 3.4, 1.5, 0.2],
[5. , 3.5, 1.3, 0.3],
[4.5, 2.3, 1.3, 0.3],
[4.4, 3.2, 1.3, 0.2],
[5. , 3.5, 1.6, 0.6],
[5.1, 3.8, 1.9, 0.4],
[4.8, 3. , 1.4, 0.3],
[5.1, 3.8, 1.6, 0.2],
```

```
[4.6, 3.2, 1.4, 0.2],
[5.3, 3.7, 1.5, 0.2],
[5. , 3.3, 1.4, 0.2],
[7. , 3.2, 4.7, 1.4],
[6.4, 3.2, 4.5, 1.5],
[6.9, 3.1, 4.9, 1.5],
[5.5, 2.3, 4. , 1.3],
[6.5, 2.8, 4.6, 1.5],
[5.7, 2.8, 4.5, 1.3],
[6.3, 3.3, 4.7, 1.6],
[4.9, 2.4, 3.3, 1.],
[6.6, 2.9, 4.6, 1.3],
[5.2, 2.7, 3.9, 1.4],
[5. , 2. , 3.5, 1.],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1.],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1.],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
[6.1, 2.8, 4. , 1.3],
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1.],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1.],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],
```

```
[5.8, 2.6, 4. , 1.2],
[5. , 2.3, 3.3, 1.],
[5.6, 2.7, 4.2, 1.3],
[5.7, 3. , 4.2, 1.2],
[5.7, 2.9, 4.2, 1.3],
[6.2, 2.9, 4.3, 1.3],
[5.1, 2.5, 3. , 1.1],
[5.7, 2.8, 4.1, 1.3],
[6.3, 3.3, 6. , 2.5],
[5.8, 2.7, 5.1, 1.9],
[7.1, 3. , 5.9, 2.1],
[6.3, 2.9, 5.6, 1.8],
[6.5, 3. , 5.8, 2.2],
[7.6, 3. , 6.6, 2.1],
[4.9, 2.5, 4.5, 1.7],
[7.3, 2.9, 6.3, 1.8],
[6.7, 2.5, 5.8, 1.8],
[7.2, 3.6, 6.1, 2.5],
[6.5, 3.2, 5.1, 2.],
[6.4, 2.7, 5.3, 1.9],
[6.8, 3. , 5.5, 2.1],
[5.7, 2.5, 5. , 2.],
[5.8, 2.8, 5.1, 2.4],
[6.4, 3.2, 5.3, 2.3],
[6.5, 3. , 5.5, 1.8],
[7.7, 3.8, 6.7, 2.2],
[7.7, 2.6, 6.9, 2.3],
[6. , 2.2, 5. , 1.5],
[6.9, 3.2, 5.7, 2.3],
[5.6, 2.8, 4.9, 2.],
[7.7, 2.8, 6.7, 2.],
[6.3, 2.7, 4.9, 1.8],
[6.7, 3.3, 5.7, 2.1],
[7.2, 3.2, 6. , 1.8],
[6.2, 2.8, 4.8, 1.8],
[6.1, 3. , 4.9, 1.8],
[6.4, 2.8, 5.6, 2.1],
[7.2, 3. , 5.8, 1.6],
[7.4, 2.8, 6.1, 1.9],
[7.9, 3.8, 6.4, 2.],
[6.4, 2.8, 5.6, 2.2],
[6.3, 2.8, 5.1, 1.5],
[6.1, 2.6, 5.6, 1.4],
[7.7, 3. , 6.1, 2.3],
[6.3, 3.4, 5.6, 2.4],
```

```

 [6.4, 3.1, 5.5, 1.8],
 [6. , 3. , 4.8, 1.8],
 [6.9, 3.1, 5.4, 2.1],
 [6.7, 3.1, 5.6, 2.4],
 [6.9, 3.1, 5.1, 2.3],
 [5.8, 2.7, 5.1, 1.9],
 [6.8, 3.2, 5.9, 2.3],
 [6.7, 3.3, 5.7, 2.5],
 [6.7, 3. , 5.2, 2.3],
 [6.3, 2.5, 5. , 1.9],
 [6.5, 3. , 5.2, 2.],
 [6.2, 3.4, 5.4, 2.3],
 [5.9, 3. , 5.1, 1.8]])

```

```

The labels (the species of flower) are in the .target attribute
y = iris.target

```

```

y

```

```

array([0, 0,
 0,
 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1,
 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2,
 2,
 2, 2])

```

```

print(f"Original dataset shape: {X.shape}")
print(f"Original labels shape: {y.shape}")
print(f"Feature names: {iris.feature_names}")
print(f"Target names (species): {iris.target_names}")

```

```

Original dataset shape: (150, 4)

```

```

Original labels shape: (150,)

```

```

Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

```

```

Target names (species): ['setosa' 'versicolor' 'virginica']

```

```

--- Split the Data ---

```

```

X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42,
)

```

The `train_test_split` function shuffles and splits the data. `test_size=0.2` means 20% of the data will be used for testing. `random_state=42` is a seed

for the random number generator, ensuring the split is reproducible. Anyone who runs this code with the same `random_state` will get the exact same split.

```
Check the shapes of the resulting sets
print(f"Shape of X_train (features for training): {X_train.shape}") # Should be 80% of
print(f"Shape of X_test (features for testing): {X_test.shape}") # Should be 20% of
print(f"Shape of y_train (labels for training): {y_train.shape}")
print(f"Shape of y_test (labels for testing): {y_test.shape}")
```

```
Shape of X_train (features for training): (120, 4)
Shape of X_test (features for testing): (30, 4)
Shape of y_train (labels for training): (120,)
Shape of y_test (labels for testing): (30,)
```

Let's look at the example of using K Nearest Neighbour classifier. See <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html> for more details.

We will tweak the parameters from the default ones.

```
Initialize a model
knn = KNeighborsClassifier(n_neighbors=3, weights='distance', algorithm='kd_tree', p=1)
knn
```

```
KNeighborsClassifier(algorithm='kd_tree', n_neighbors=3, p=1,
 weights='distance')
```

```
knn.fit(X_train, y_train)
```

```
KNeighborsClassifier(algorithm='kd_tree', n_neighbors=3, p=1,
 weights='distance')
```

```
Make predictions
y_pred = knn.predict(X_test)
```

```
Evaluate performance
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy on the Test Set: {accuracy:.4f}")
```

```
Model Accuracy on the Test Set: 1.0000
```

Here, we have an accuracy of 100%. This is not usually the case for most machine learning projects. Anyways, we are dealing with a toy dataset and do not expect any difference. Let's look at other ways of evaluating the models



```
For multi-class problems, precision/recall/f1 need an 'average' method
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
print(f"Model Precision (weighted): {precision:.4f}")
print(f"Model Recall (weighted): {recall:.4f}")
print(f"Model F1 Score (weighted): {f1:.4f}")
print("\nConfusion Matrix:")
print(conf_matrix)
```

```
Model Precision (weighted): 1.0000
Model Recall (weighted): 1.0000
Model F1 Score (weighted): 1.0000
```

```
Confusion Matrix:
[[10 0 0]
 [0 9 0]
 [0 0 11]]
```

## 45.4 Saving the model

It is possible to save a model in the scikit by using Python frameworks like joblib and pickle

```
Save the trained model to a file
model_filename = 'knn_iris_model.joblib'
joblib.dump(knn, model_filename)
print(f"\nModel saved to {model_filename}")
```

```
Model saved to knn_iris_model.joblib
```

```
Load the model from the file
knn2 = joblib.load(model_filename)
print(f"Model loaded from {model_filename}")

You can now use the loaded model to make new predictions
Example: Predict for the first 5 samples in the test set using the loaded model
knn_predictions = knn2.predict(X_test[:5])
print(f"\nPredictions from loaded model for first 5 test samples: {knn_predictions}")
print(f"Actual labels for first 5 test samples: {y_test[:5]}")
```

```
Model loaded from knn_iris_model.joblib
```

```
Predictions from loaded model for first 5 test samples: [1 0 2 1 1]
```

```
Actual labels for first 5 test samples: [1 0 2 1 1]
```

## Chapter 46

# Regression: Photometric Redshifts of Galaxies

```
!mkdir -p data
!wget -P data https://raw.githubusercontent.com/Danselem/brics_astro/main/Week6/data/sdss_photoz.

import numpy as np
import pandas as pd
import os

Paths
input_path = 'data/sdss_photoz.npy'
output_path = 'data/sdss_photoz_clean.csv'

Load structured array
data = np.load(input_path)

Convert magnitude fields to float64
u = data['u'].astype('float64')
g = data['g'].astype('float64')
r = data['r'].astype('float64')
i = data['i'].astype('float64')
z = data['z'].astype('float64')

Step 1: Remove non-finite values
finite_mask = (
 np.isfinite(u) &
 np.isfinite(g) &
 np.isfinite(r) &
```

```

 np.isfinite(i) &
 np.isfinite(z)
)

 # Step 2: Filter for valid SDSS magnitude range
 valid_range_mask = (
 (u > 10) & (u < 40) &
 (g > 10) & (g < 40) &
 (r > 10) & (r < 40) &
 (i > 10) & (i < 40) &
 (z > 10) & (z < 40)
)

 # Step 3: Remove unwanted specClass values
 excluded_spec_classes = [60, 61, 62, 63]
 spec_mask = ~np.isin(data['specClass'], excluded_spec_classes)

 # Combine all masks
 final_mask = finite_mask & valid_range_mask & spec_mask

 # Apply the mask
 data_clean = data[final_mask]

 # Convert to DataFrame
 df = pd.DataFrame({
 'u': data_clean['u'],
 'g': data_clean['g'],
 'r': data_clean['r'],
 'i': data_clean['i'],
 'z': data_clean['z'],
 'specClass': data_clean['specClass'],
 'redshift': data_clean['redshift'],
 'redshifterr': data_clean['redshifterr']
 })

 df = df[(df['redshift'] > 0) & (df['redshift'] < 2.5)]

 # Save to CSV
 os.makedirs(os.path.dirname(output_path), exist_ok=True)
 df.to_csv(output_path, index=False)

```

```

/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_3994/4202856265.py:13: Runt:

```

```

 u = data['u'].astype('float64')

```

```

/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_3994/4202856265.py:14: Runt:

```

```

 g = data['g'].astype('float64')

```

```

/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_3994/4202856265.py:15: RuntimeWarning:
 r = data['r'].astype('float64')
/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_3994/4202856265.py:16: RuntimeWarning:
 i = data['i'].astype('float64')
/var/folders/v0/v_qnst7n3hn5fdcjh69y2qvm0000gn/T/ipykernel_3994/4202856265.py:17: RuntimeWarning:
 z = data['z'].astype('float64')

```

Let's perform some data transformation

```

df['u-g'] = df['u'] - df['g']
df['g-r'] = df['g'] - df['r']
df['r-i'] = df['r'] - df['i']
df['i-z'] = df['i'] - df['z']

```

```
df.head()
```

	u	g	r	i	z	specClass	redshift	\
0	17.451954	16.137806	15.548249	15.042027	14.882285	2	0.086991	
1	18.127691	16.766823	16.083586	15.647685	15.394728	2	0.081496	
2	18.656502	17.544571	17.042847	16.730713	16.515930	2	0.046715	
3	19.226503	18.123615	17.409952	16.892635	16.665926	2	0.092328	
4	18.729752	17.535135	17.159952	16.877113	16.829420	2	0.046642	

	redshifterr	u-g	g-r	r-i	i-z
0	0.000108	1.314148	0.589557	0.506222	0.159742
1	0.000097	1.360868	0.683237	0.435901	0.252957
2	0.000078	1.111931	0.501724	0.312134	0.214783
3	0.000079	1.102888	0.713663	0.517317	0.226709
4	0.000097	1.194616	0.375183	0.282839	0.047693

```
df['specClass'].value_counts()
```

```

specClass
2 11341
3 1906
4 34
Name: count, dtype: int64

```

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

```

```

Set Seaborn theme
sns.set(style="whitegrid")

```

```

Create subplots
fig, axes = plt.subplots(3, 1, figsize=(12, 14))
fig.suptitle('SDSS Photometry Distributions', fontsize=18, y=1)

1. Magnitudes
mags = ['u', 'g', 'r', 'i', 'z']
for mag in mags:
 sns.kdeplot(df[mag], label=mag, ax=axes[0], fill=True, alpha=0.5)
axes[0].set_title('Magnitude Distributions')
axes[0].set_xlabel('Magnitude')
axes[0].legend()

2. Colors
colors = ['u-g', 'g-r', 'r-i', 'i-z']
for color in colors:
 sns.kdeplot(df[color], label=color, ax=axes[1], fill=True, alpha=0.5)
axes[1].set_title('Color Index Distributions')
axes[1].set_xlabel('Color')
axes[1].legend()

3. Redshift
sns.histplot(df['redshift'], bins=100, kde=True, ax=axes[2], color='purple')
axes[2].set_title('Redshift Distribution')
axes[2].set_xlabel('Redshift')

plt.tight_layout()
plt.show()

```

<Figure size 1200x1400 with 3 Axes>

Now let's define the data into features and target then split the data,

```

from sklearn.model_selection import train_test_split

Define feature matrix X and target vector y
features = ['u-g', 'g-r', 'r-i', 'i-z']
target = 'redshift'

X = df[features]
y = df[target]

Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42
)

```

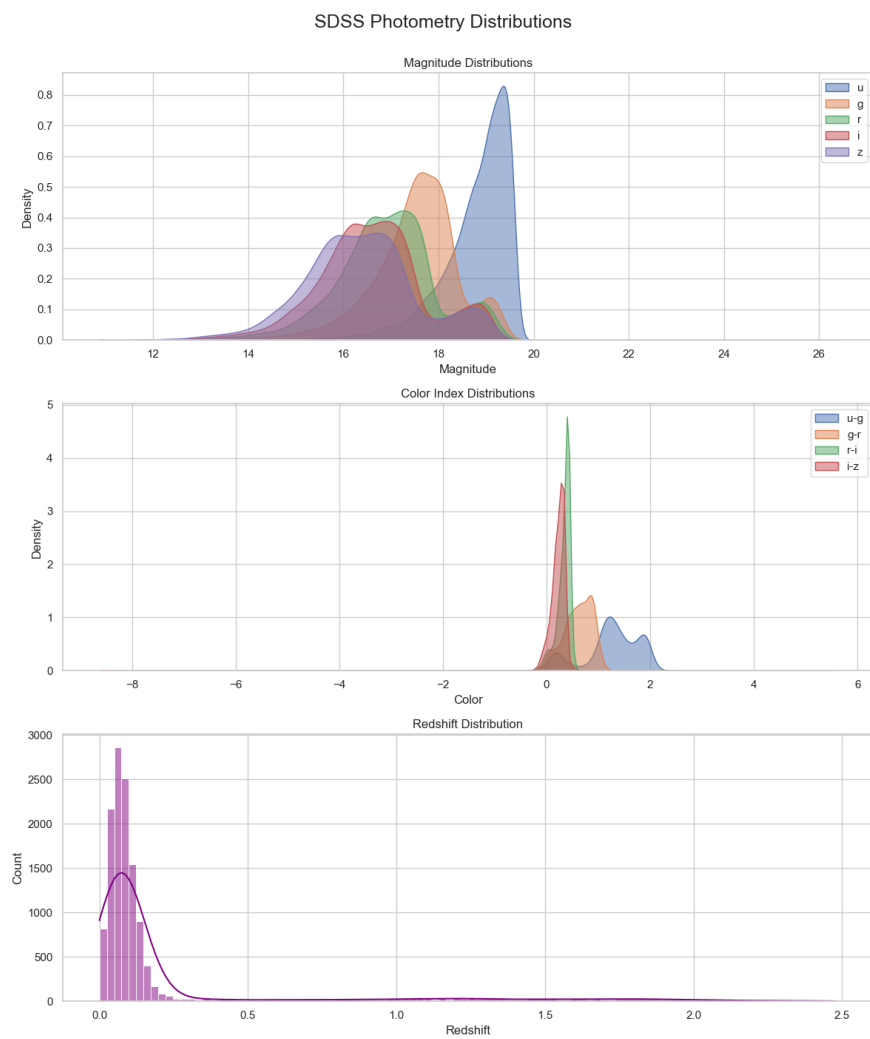


Figure 46.1: output

```
)

print("Training set size:", X_train.shape)
print("Test set size:", X_test.shape)
```

```
Training set size: (10624, 4)
Test set size: (2657, 4)
```

Next step is to define the evaluation metrics

## 46.1 Regression Evaluation

When evaluating the performance of a regression model (such as predicting galaxy redshift from colors), several metrics can be used to quantify the error between predicted and actual values. Below are common metrics used in regression.

---

### 46.1.1 1. Mean Absolute Error (MAE)

- **Formula:**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Objective:**

Measures the average absolute difference between predicted values ( $\hat{y}_i$ ) and true values ( $y_i$ ). MAE is simple and interpretable; it tells us, on average, how wrong our predictions are.

---

### 46.1.2 2. Mean Squared Error (MSE)

- **Formula:**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



- **Objective:**  
Penalizes larger errors more than MAE by squaring the difference. This makes it sensitive to outliers.
- 

### 46.1.3 3. Root Mean Squared Error (RMSE)

- **Formula:**

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- **Objective:**  
Provides error in the same units as the target variable (e.g., redshift). Like MSE, it emphasizes larger errors.
- 

### 46.1.4 4. $R^2$ Score (Coefficient of Determination)

- **Formula:**

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where  $\bar{y}$  is the mean of the true values.

- **Objective:**  
Indicates how well the model explains the variance in the data.
    - ( $R^2 = 1$ ): perfect prediction
    - ( $R^2 = 0$ ): model does no better than the mean
    - ( $R^2 < 0$ ): model is worse than predicting the mean
- 

More info can be found here: [https://scikit-learn.org/0.15/modules/model\\_evaluation.html#regression-metrics](https://scikit-learn.org/0.15/modules/model_evaluation.html#regression-metrics)

```

from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

Regression models
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR

Helper function to evaluate and plot model performance
def evaluate_model(model, name):
 model.fit(X_train, y_train)
 y_pred = model.predict(X_test)

 mse = mean_squared_error(y_test, y_pred)
 rmse = np.sqrt(mse)
 mae = mean_absolute_error(y_test, y_pred)
 r2 = r2_score(y_test, y_pred)

 print(f"\n{name}")
 print("-" * len(name))
 print(f"MAE: {mae:.4f}")
 print(f"RMSE: {rmse:.4f}")
 print(f"R2 Score: {r2:.4f}")

Predicted vs Actual
plt.figure(figsize=(6, 4))
plt.scatter(y_test, y_pred, s=5, alpha=0.5)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel('True Redshift')
plt.ylabel('Predicted Redshift')
plt.title(f'{name} - Predicted vs Actual')
plt.grid(True)
plt.tight_layout()
plt.show()

Residuals
residuals = y_test - y_pred
plt.figure(figsize=(6, 4))
plt.hist(residuals, bins=50, color='gray', edgecolor='black')
plt.axvline(0, color='red', linestyle='--')
plt.title(f'{name} - Residual Distribution')
plt.xlabel('Residual (y_true - y_pred)')
plt.ylabel('Frequency')

```

```
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
1. Linear Regression
evaluate_model(LinearRegression(), "Linear Regression")

2. Lasso Regression (L1)
evaluate_model(Lasso(alpha=0.01), "Lasso Regression (L1)")

3. Ridge Regression (L2)
evaluate_model(Ridge(alpha=1.0), "Ridge Regression (L2)")

4. K-Nearest Neighbors
evaluate_model(KNeighborsRegressor(n_neighbors=5), "K-Nearest Neighbors (k=5)")

5. Decision Tree Regressor
evaluate_model(DecisionTreeRegressor(max_depth=10, random_state=42), "Decision Tree Regressor")

6. Random Forest Regressor
evaluate_model(RandomForestRegressor(n_estimators=100, random_state=42), "Random Forest Regressor")

7. Support Vector Regressor (SVR)
evaluate_model(SVR(kernel='rbf', C=1.0, epsilon=0.01), "Support Vector Regressor")
```

```
Linear Regression

MAE: 0.2350
RMSE: 0.3351
R2 Score: 0.4500
```

<Figure size 600x400 with 1 Axes>

<Figure size 600x400 with 1 Axes>

```
Lasso Regression (L1)

MAE: 0.2334
RMSE: 0.3361
R2 Score: 0.4468
```

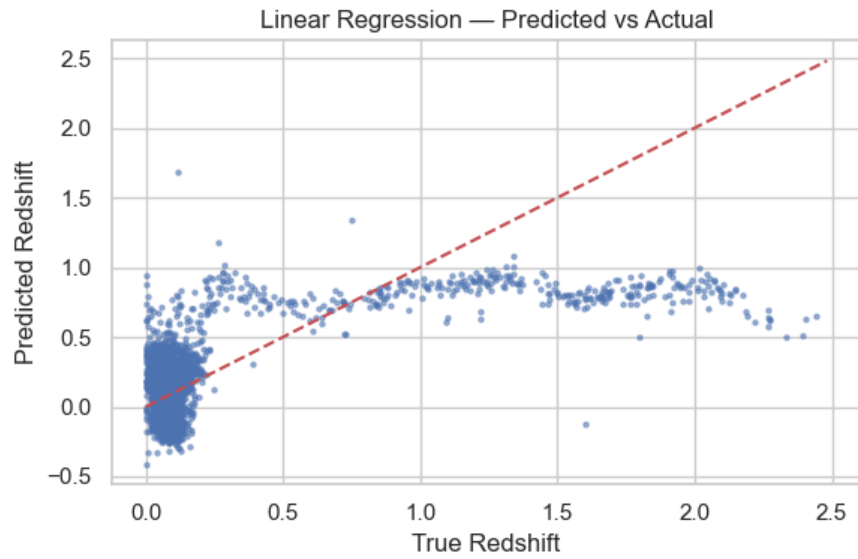


Figure 46.2: output

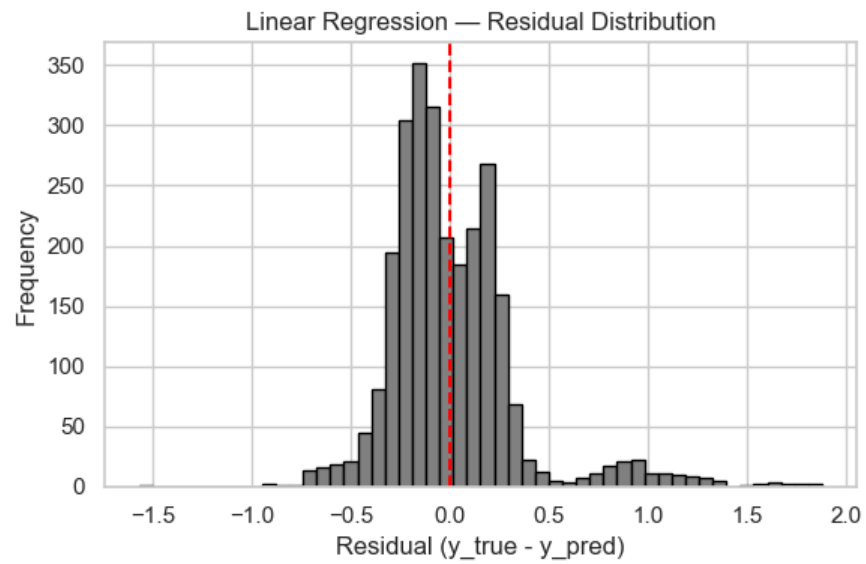


Figure 46.3: output

<Figure size 600x400 with 1 Axes>

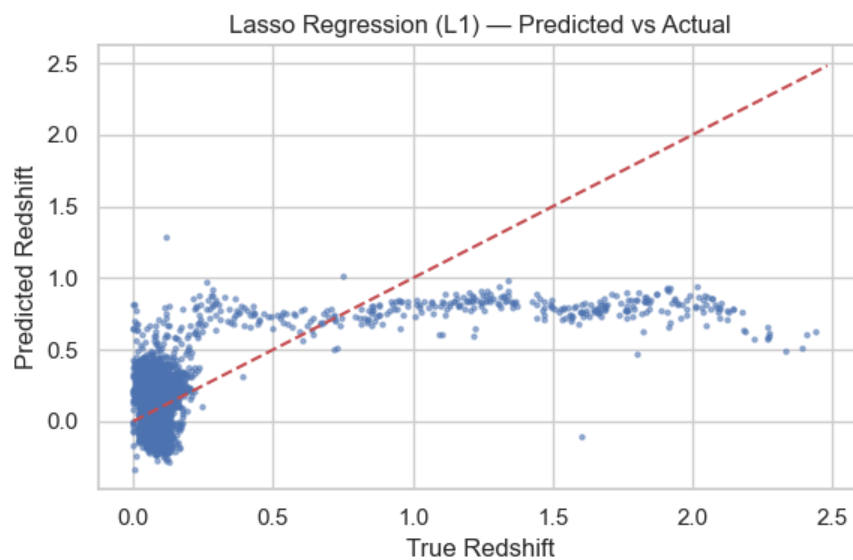


Figure 46.4: output

<Figure size 600x400 with 1 Axes>

Ridge Regression (L2)

-----  
MAE: 0.2350  
RMSE: 0.3351  
R<sup>2</sup> Score: 0.4500

<Figure size 600x400 with 1 Axes>

<Figure size 600x400 with 1 Axes>

K-Nearest Neighbors (k=5)

-----  
MAE: 0.0554  
RMSE: 0.1741  
R<sup>2</sup> Score: 0.8515

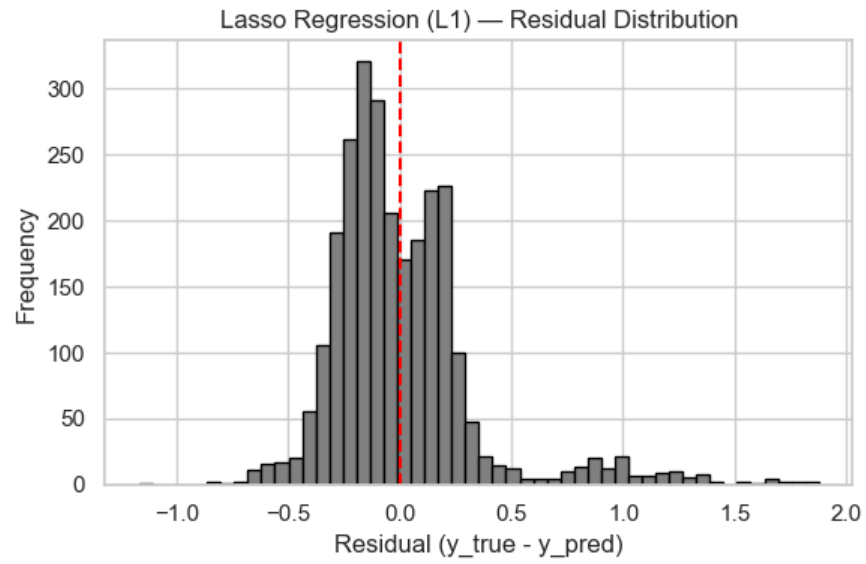


Figure 46.5: output

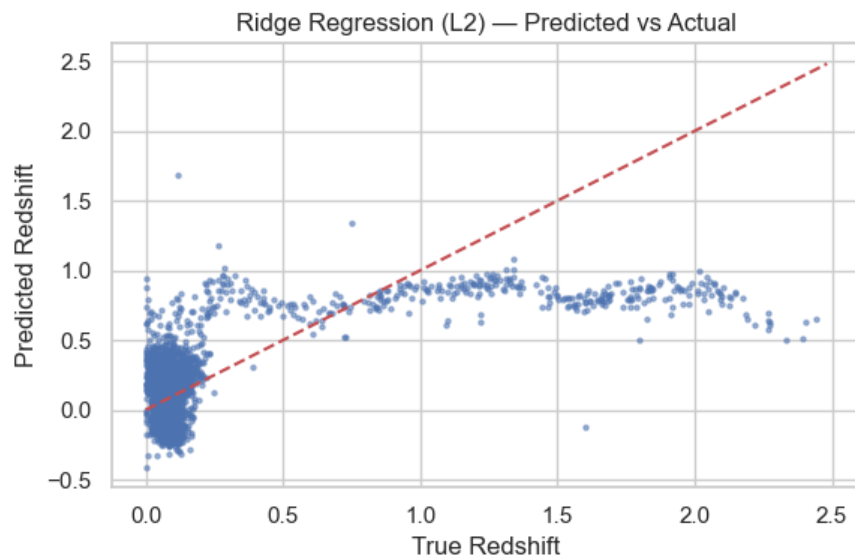


Figure 46.6: output

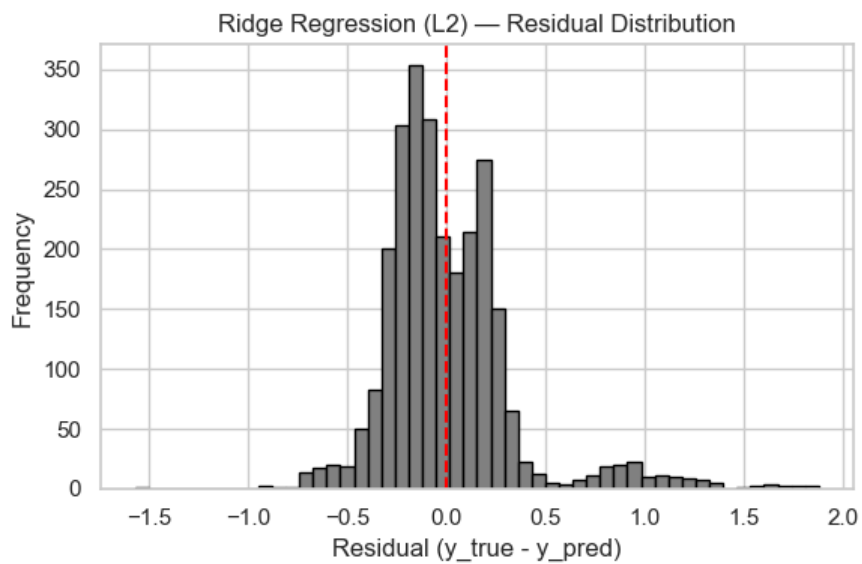


Figure 46.7: output

<Figure size 600x400 with 1 Axes>

<Figure size 600x400 with 1 Axes>

Decision Tree Regressor

-----

MAE: 0.0664

RMSE: 0.2241

R<sup>2</sup> Score: 0.7540

<Figure size 600x400 with 1 Axes>

<Figure size 600x400 with 1 Axes>

Random Forest Regressor

-----

MAE: 0.0578

RMSE: 0.1787

R<sup>2</sup> Score: 0.8435

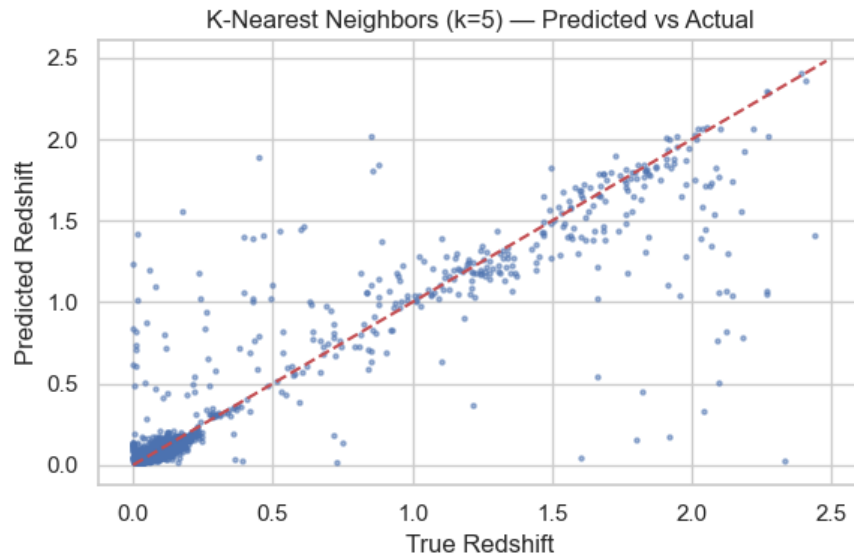


Figure 46.8: output

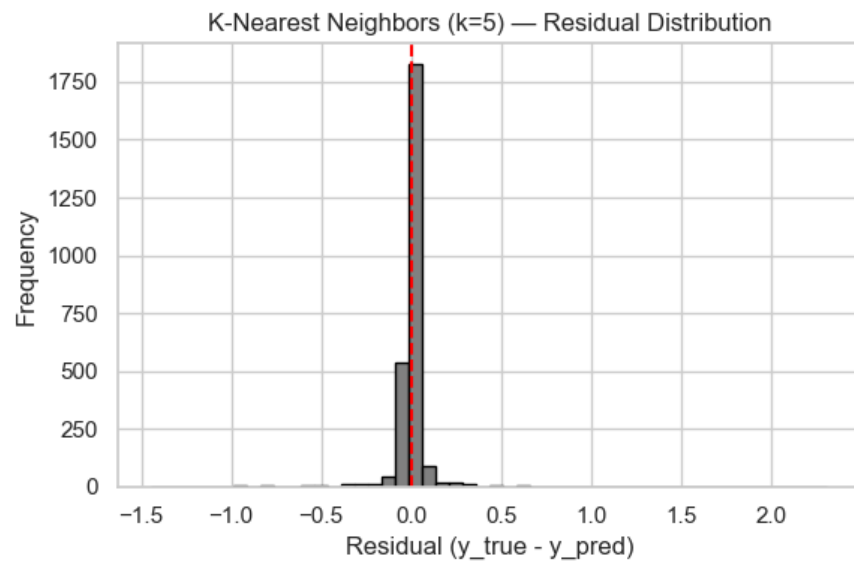


Figure 46.9: output



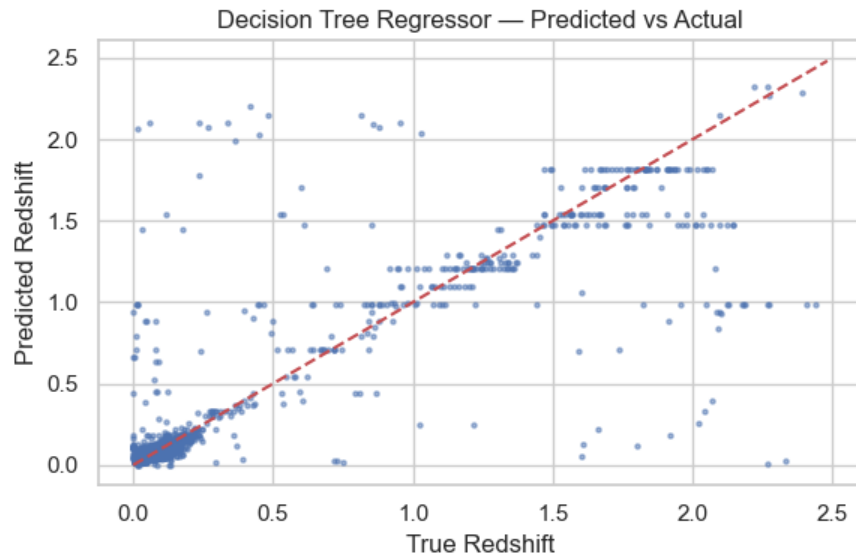


Figure 46.10: output

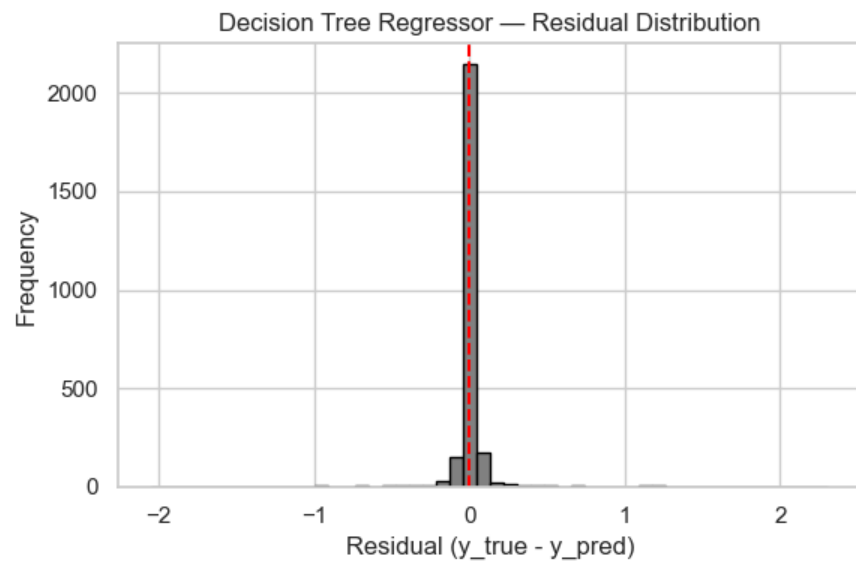


Figure 46.11: output

<Figure size 600x400 with 1 Axes>

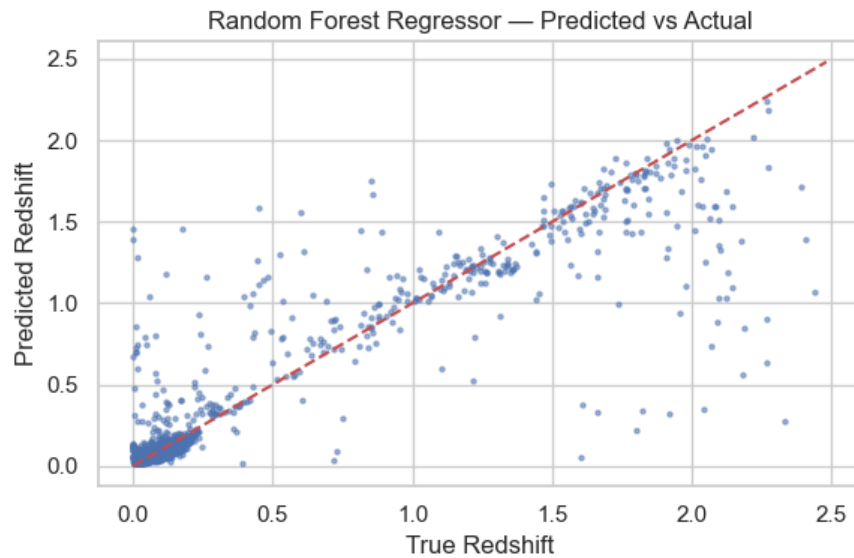


Figure 46.12: output

<Figure size 600x400 with 1 Axes>

```
Support Vector Regressor

MAE: 0.0724
RMSE: 0.2128
R2 Score: 0.7781
```

<Figure size 600x400 with 1 Axes>

<Figure size 600x400 with 1 Axes>

We can see the Linear Regression models performed poorly compared to KNN, SVM, Decision Trees and Random Forest.

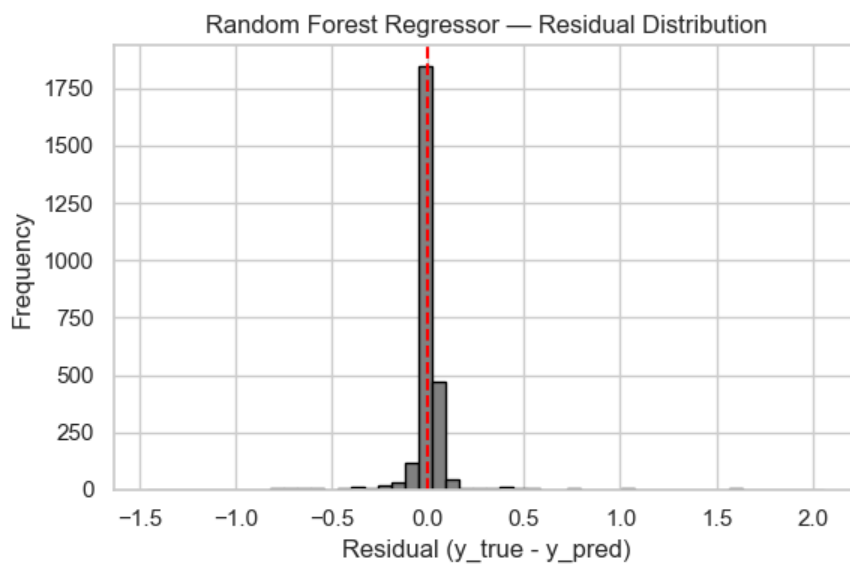


Figure 46.13: output

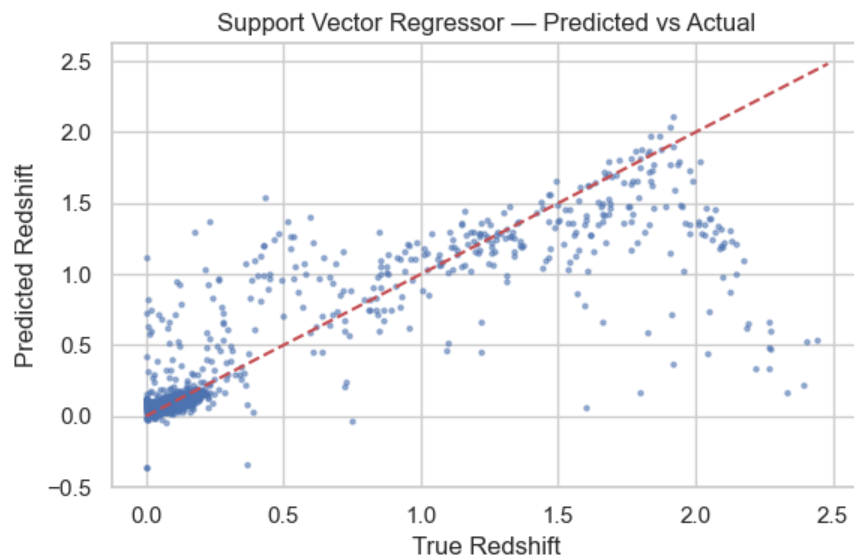


Figure 46.14: output

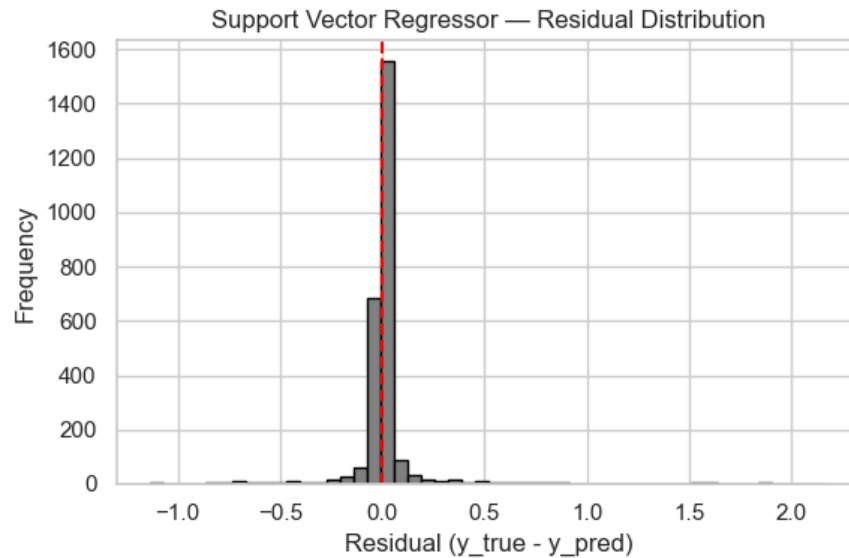


Figure 46.15: output

## 46.2 Feature Scaling

Feature scaling is a preprocessing step in many machine learning tasks. It ensures that features contribute equally to the model by transforming them into a comparable scale. This is especially important for algorithms that compute distances or assume normally distributed data.

---

### 46.2.1 Why Feature Scaling is Important

Many machine learning algorithms perform better or converge faster when features are on a similar scale. Without scaling:

- Models may become biased toward features with larger magnitudes.
  - Optimization algorithms may take longer to converge.
  - Models that rely on distances (e.g., k-NN, SVM) may produce suboptimal results.
-

## 46.2.2 Common Feature Scaling Techniques

### 46.2.2.1 1. Min-Max Scaling (Normalization)

**Formula:**

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

**Range:**  $[0, 1]$  or any custom range.

**Use when:** - You want to preserve the shape of the original distribution. - The algorithm is sensitive to the scale (e.g., neural networks, k-NN, SVM).

---

### 46.2.2.2 2. Standardization (Z-score Normalization)

**Formula:**

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

Where: -  $(\mu)$  is the mean of the feature. -  $(\sigma)$  is the standard deviation.

**Range:** Unbounded (mean = 0, standard deviation = 1)

**Use when:** - Features have Gaussian-like distributions. - The algorithm assumes normally distributed features (e.g., logistic regression, linear regression, SVM, PCA).

---

### 46.2.2.3 3. Robust Scaling

**Formula:**

$$x_{\text{scaled}} = \frac{x - \text{median}(x)}{\text{IQR}(x)}$$

Where IQR is the interquartile range  $(Q_3 - Q_1)$ .

**Use when:** - The dataset contains many outliers. - You want a robust alternative to standardization.

---

#### 46.2.2.4 4. MaxAbs Scaling

**Formula:**

$$x_{\text{scaled}} = \frac{x}{\max(|x|)}$$

**Use when:** - Features are already centered around zero. - You want to preserve sparsity in sparse datasets (e.g., text data).

---

### 46.2.3 When to Scale Features

Feature scaling is especially important for:

- Distance-based models: k-Nearest Neighbors (k-NN), Support Vector Machines (SVM)
- Regularized models: Ridge, Lasso
- Gradient-based methods: Logistic Regression, Neural Networks
- Dimensionality reduction: PCA, t-SNE

**Not usually required for:** - Tree-based models: Decision Trees, Random Forests, Gradient Boosted Trees

---

For more on feature scaling, see <https://scikit-learn.org/stable/modules/preprocessing.html>.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

Plot scaled features distribution
scaled_df = pd.DataFrame(X_train_scaled, columns=['u-g', 'g-r', 'r-i', 'i-z'])
plt.figure(figsize=(10, 6))
sns.histplot(data=scaled_df.melt(), x='value', hue='variable', bins=50, element='step')
plt.title("Distribution of Scaled Color Features")
plt.xlabel("Scaled Value")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```

<Figure size 1000x600 with 1 Axes>

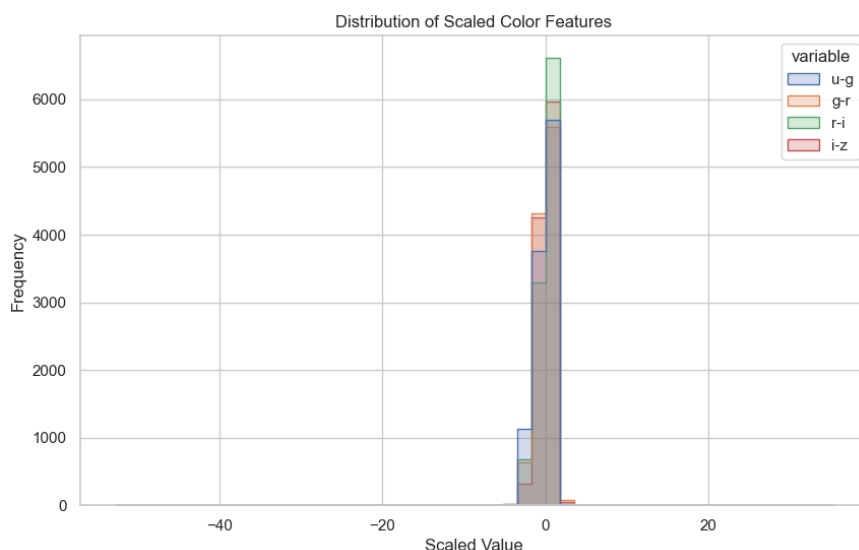


Figure 46.16: output

```
Evaluation function
def evaluate_model(model, name):
 model.fit(X_train_scaled, y_train)
 preds = model.predict(X_test_scaled)
 mse = mean_squared_error(y_test, preds)
 r2 = r2_score(y_test, preds)
 print(f"{name}:")
 print(f" Mean Squared Error: {mse:.4f}")
 print(f" R2 Score: {r2:.4f}")
 print("-" * 40)

Train models
evaluate_model(LinearRegression(), "Linear Regression")
evaluate_model(Ridge(alpha=1.0, random_state=42), "Ridge Regression (L2)")
evaluate_model(Lasso(alpha=0.01, random_state=42), "Lasso Regression (L1)")
evaluate_model(KNeighborsRegressor(n_neighbors=5), "K-Nearest Neighbors")
evaluate_model(DecisionTreeRegressor(max_depth=10, random_state=42), "Decision Tree Regressor")
evaluate_model(RandomForestRegressor(n_estimators=100, random_state=42), "Random Forest Regressor")
evaluate_model(SVR(kernel='rbf', C=1.0, epsilon=0.01), "Support Vector Regressor")
```

```
Linear Regression:
 Mean Squared Error: 0.1123
 R2 Score: 0.4500

```

```
Ridge Regression (L2):
 Mean Squared Error: 0.1123
 R2 Score: 0.4500

```

```
Lasso Regression (L1):
 Mean Squared Error: 0.1126
 R2 Score: 0.4484

```

```
K-Nearest Neighbors:
 Mean Squared Error: 0.0326
 R2 Score: 0.8403

```

```
Decision Tree Regressor:
 Mean Squared Error: 0.0502
 R2 Score: 0.7540

```

```
Random Forest Regressor:
 Mean Squared Error: 0.0319
 R2 Score: 0.8435

```

```
Support Vector Regressor:
 Mean Squared Error: 0.0367
 R2 Score: 0.8201

```

By standard scaling the galaxy colors, we noticed an improvement with Support Vector Regressor. However, there is no improvement for the other models.

In the next notebook, we will talk about tuning a model to improve the model prediction capabilities.

## 46.3 Summary

In this notebook, we introduce a simple astronomy dataset application for machine learning. You performed series of regression algorithms and evaluated your model performance on unseen dataset.



## 46.4 Additional Resources

Intro to machine learning: <https://www.kaggle.com/learn/intro-to-machine-learning>

Google Machine Learning Crash Course: <https://developers.google.com/machine-learning/crash-course>

Regression Evaluation Metrics: <https://www.kaggle.com/code/marcinrutecki/regression-models-evaluation-metrics>



## Chapter 47

# Hyperparameter Tuning: Optimizing Model Performance

```
import the modules

import time
import numpy as np
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import (
 mean_squared_error,
 mean_absolute_error,
 r2_score,
)
import matplotlib.pyplot as plt
```

**Note:** If you are running this jupyter notebook from Colab, then run the next cell by pressing SHIFT+ENTER to install the required packages for this notebook. Otherwise, skip the next cell.

```
!mkdir -p data
!wget -P data https://raw.githubusercontent.com/Danselem/brics_astro/main/Week6/data/sdss_photoz.
```

```

import numpy as np
import pandas as pd
import os

Paths
input_path = 'data/sdss_photoz.npy'
output_path = 'data/sdss_photoz_clean.csv'

Load structured array
data = np.load(input_path)

Convert magnitude fields to float64
u = data['u'].astype('float64')
g = data['g'].astype('float64')
r = data['r'].astype('float64')
i = data['i'].astype('float64')
z = data['z'].astype('float64')

Step 1: Remove non-finite values
finite_mask = (
 np.isfinite(u) &
 np.isfinite(g) &
 np.isfinite(r) &
 np.isfinite(i) &
 np.isfinite(z)
)

Step 2: Filter for valid SDSS magnitude range
valid_range_mask = (
 (u > 10) & (u < 40) &
 (g > 10) & (g < 40) &
 (r > 10) & (r < 40) &
 (i > 10) & (i < 40) &
 (z > 10) & (z < 40)
)

Step 3: Remove unwanted specClass values
excluded_spec_classes = [60, 61, 62, 63]
spec_mask = ~np.isin(data['specClass'], excluded_spec_classes)

Combine all masks
final_mask = finite_mask & valid_range_mask & spec_mask

Apply the mask
data_clean = data[final_mask]

```

```

Convert to DataFrame
df = pd.DataFrame({
 'u': data_clean['u'],
 'g': data_clean['g'],
 'r': data_clean['r'],
 'i': data_clean['i'],
 'z': data_clean['z'],
 'specClass': data_clean['specClass'],
 'redshift': data_clean['redshift'],
 'redshifterr': data_clean['redshifterr']
})

df = df[(df['redshift']>0) & (df['redshift']< 2.5)]

Save to CSV
os.makedirs(os.path.dirname(output_path), exist_ok=True)
df.to_csv(output_path, index=False)

Load the preprocessed data
df = pd.read_csv("data/sdss_photoz_clean.csv")

df['u-g'] = df['u'] - df['g']
df['g-r'] = df['g'] - df['r']
df['r-i'] = df['r'] - df['i']
df['i-z'] = df['i'] - df['z']

Features: color indices; Target: redshift
X = df[['u-g', 'g-r', 'r-i', 'i-z']].values
y = df['redshift'].values

Train-test split
X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42
)

```

## 47.1 Cross-Validation with KFold and Grid Search

### 47.1.1 What is Cross-Validation?

Cross-validation is a model validation technique used to assess how a predictive model generalizes to an independent dataset. It helps detect issues like

overfitting or underfitting by splitting the data into training and validation sets multiple times in a systematic way.

---

### 47.1.2 KFold Cross-Validation

**KFold** (K-Folds cross-validation) divides the dataset into  $K$  equal-sized folds. The model is trained on  $K-1$  folds and validated on the remaining fold. This process is repeated  $K$  times so that each fold serves as a validation set once.

**How it works:**

- Split data into  $K$  subsets (folds)
- Train the model on  $K-1$  folds
- Validate on the remaining fold
- Repeat the process for all  $K$  folds
- Average the performance metrics across all folds

**Advantages:**

- More robust estimate of model performance
  - Reduces variance compared to a single train-test split
  - Ensures that every data point is used for validation
- 

### 47.1.3 Grid Search with Cross-Validation

**GridSearchCV** performs an exhaustive search over a specified parameter grid by:

1. Trying all possible combinations of hyperparameters.
2. Performing cross-validation (e.g., using **KFold**) for each combination.
3. Selecting the combination that yields the best cross-validated performance.

**Key Parameters:**

- **param\_grid**: A dictionary defining which parameters and values to try.
- **cv**: The cross-validation strategy (e.g., **KFold(n\_splits=5)**).
- **scoring**: A string or dictionary of scoring metrics (e.g., **'r2'**, **'neg\_mean\_squared\_error'**).

- **refit**: Which scoring metric to use for choosing the best model.

---

For more on cross validation, check out the sklearn documentation: [https://scikit-learn.org/stable/modules/cross\\_validation.html](https://scikit-learn.org/stable/modules/cross_validation.html)

Let's try out an example for KNNRegressor, See <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsRegressor.html> for more details

```
Intialise the KNN regressor class
knn = KNeighborsRegressor()

Define the hyperparameter grid
param_grid = {
 'n_neighbors': list(range(1, 51, 2)), # odd values from 1 to 49
 'weights': ['uniform', 'distance'],
 'p': [1, 2], # 1: Manhattan, 2: Euclidean
 'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute']
}

Cross-validation strategy (reproducible splits)
cv = KFold(n_splits=20, shuffle=True, random_state=42)

Multiple scoring metrics
scoring = {
 'neg_mse': 'neg_mean_squared_error',
 'neg_mae': 'neg_mean_absolute_error',
 'r2': 'r2'
}

Grid search with multiple scorers
grid_search = GridSearchCV(
 estimator=knn,
 param_grid=param_grid,
 scoring=scoring,
 refit='r2', # refit based on R2 score
 cv=cv,
 n_jobs=-1,
 verbose=1
)

grid_search
```

```

GridSearchCV(cv=KFold(n_splits=20, random_state=42, shuffle=True),
 estimator=KNeighborsRegressor(), n_jobs=-1,
 param_grid={'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
 'n_neighbors': [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21,
 23, 25, 27, 29, 31, 33, 35, 37, 39, 41,
 43, 45, 47, 49],
 'p': [1, 2], 'weights': ['uniform', 'distance']}},
 refit='r2',
 scoring={'neg_mae': 'neg_mean_absolute_error',
 'neg_mse': 'neg_mean_squared_error', 'r2': 'r2'},
 verbose=1)

```

```

Start timing
start_time = time.time()

Fit the model
grid_search.fit(X_train, y_train)

End timing
end_time = time.time()
elapsed_time = end_time - start_time
print(f"\nGrid search took {elapsed_time:.2f} seconds.")

```

Fitting 20 folds for each of 400 candidates, totalling 8000 fits

Grid search took 24.98 seconds.

```

Best model
best_knn = grid_search.best_estimator_
print("Best Parameters:")
print(grid_search.best_params_)

Predict on test set
y_pred = best_knn.predict(X_test)

Evaluation metrics
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nTest Set Evaluation:")
print(f"R2 Score: {r2:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")

```



```
Plot: True vs Predicted Redshifts
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.3, edgecolors='k')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], 'r--')
plt.xlabel(r"True Redshift (z)")
plt.ylabel(r"Predicted Redshift (z)")
plt.title("KNN Regression: True vs Predicted Redshift")
plt.grid(True)
plt.tight_layout()
plt.show()
```

Best Parameters:

```
{'algorithm': 'brute', 'n_neighbors': 13, 'p': 2, 'weights': 'distance'}
```

Test Set Evaluation:

R<sup>2</sup> Score: 0.8559

Mean Squared Error (MSE): 0.0294

Mean Absolute Error (MAE): 0.0555

<Figure size 800x600 with 1 Axes>

```
Get best parameters from grid search
knn_params = grid_search.best_params_

Create a new RandomForestRegressor with the best params and the same random_state
knn_model = KNeighborsRegressor(**knn_params,)

Retrain the model on the full training data
knn_model.fit(X_train, y_train)

Predict on test set
y_pred = knn_model.predict(X_test)

Evaluate predictions
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

print("Evaluation after retraining with best parameters:")
print(f"R2 Score: {r2_score(y_test, y_pred):.4f}")
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")
print(f"Mean Absolute Error: {mean_absolute_error(y_test, y_pred):.4f}")
```

Evaluation after retraining with best parameters:

R<sup>2</sup> Score: 0.8559

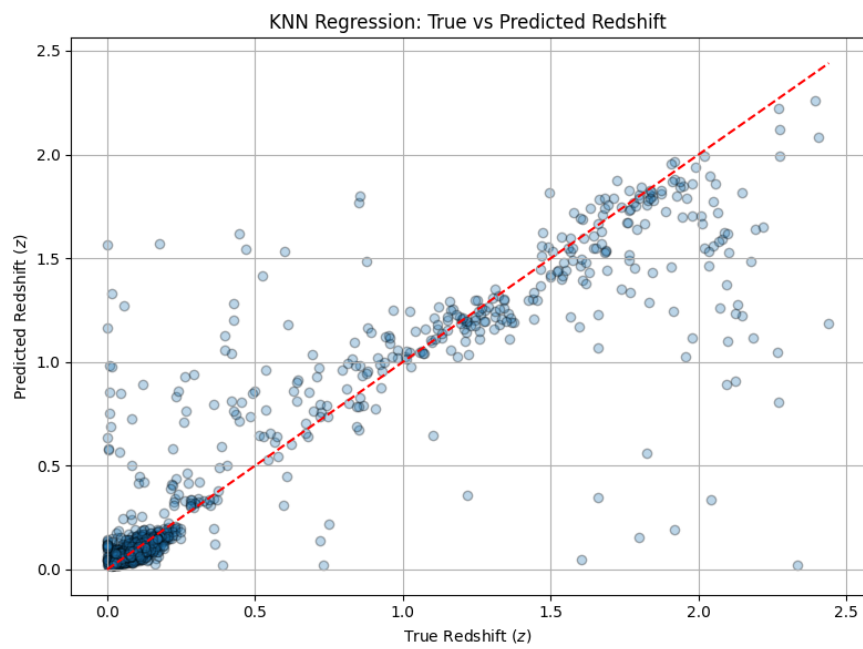


Figure 47.1: output

```
Mean Squared Error: 0.0294
Mean Absolute Error: 0.0555
```

```
Save the KNN trained model to disk
knn_model_filename = "knn_model.pkl"
joblib.dump(knn_model, knn_model_filename)
```

```
['knn_model.pkl']
```

## 47.2 Random Forest

Let's try for Random Forest Regressor, see <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html> for more details.

```
Set up Random Forest Regressor
rf = RandomForestRegressor(random_state=42)

Hyperparameter grid
param_grid = {
 'n_estimators': [50, 100, 200],
 'max_depth': [None, 10, 20, 30],
 'min_samples_split': [2, 5, 10],
 'min_samples_leaf': [1, 2, 4],
 'max_features': ['auto', 'sqrt', 'log2'],
}

Cross-validation strategy (reproducible splits)
cv = KFold(n_splits=3, shuffle=True, random_state=42)

Grid search with multiple scoring metrics
scoring = {
 'neg_mse': 'neg_mean_squared_error',
 'neg_mae': 'neg_mean_absolute_error',
 'r2': 'r2'
}

grid_search = GridSearchCV(
 estimator=rf,
 param_grid=param_grid,
 scoring=scoring,
 refit='r2', # refit based on R2 score
 cv=cv,
```

```

 n_jobs=4,
 verbose=1
)

 grid_search

```

```

GridSearchCV(cv=KFold(n_splits=3, random_state=42, shuffle=True),
 estimator=RandomForestRegressor(random_state=42), n_jobs=4,
 param_grid={'max_depth': [None, 10, 20, 30],
 'max_features': ['auto', 'sqrt', 'log2'],
 'min_samples_leaf': [1, 2, 4],
 'min_samples_split': [2, 5, 10],
 'n_estimators': [50, 100, 200]},
 refit='r2',
 scoring={'neg_mae': 'neg_mean_absolute_error',
 'neg_mse': 'neg_mean_squared_error', 'r2': 'r2'},
 verbose=1)

```

```

Start timing
start_time = time.time()

Fit the model
grid_search.fit(X_train, y_train)

End timing
end_time = time.time()
elapsed_time = end_time - start_time
print(f"\nGrid search took {elapsed_time:.2f} seconds.")

```

Fitting 3 folds for each of 324 candidates, totalling 972 fits

/Users/user1/Data/brics\_astro/.venv/lib/python3.10/site-packages/sklearn/model\_selection/\_search.py:105: UserWarning: 324 fits failed out of a total of 972.

The score on these train-test partitions for these parameters will be set to nan.  
If these failures are not expected, you can try to debug them by setting error\_score='raise'.

Below are more details about the failures:

-----

324 fits failed with the following error:

Traceback (most recent call last):

```

File "/Users/user1/Data/brics_astro/.venv/lib/python3.10/site-packages/sklearn/model_selection/_search.py", line 105, in _fit
 estimator.fit(X_train, y_train, **fit_params)
File "/Users/user1/Data/brics_astro/.venv/lib/python3.10/site-packages/sklearn/base.py", line 118, in _validate_params
 self._validate_params()

```

```

File "/Users/user1/Data/brics_astro/.venv/lib/python3.10/site-packages/sklearn/base.py", line 4
 validate_parameter_constraints(
File "/Users/user1/Data/brics_astro/.venv/lib/python3.10/site-packages/sklearn/utils/_param_val
 raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' parameter of RandomFore

 warnings.warn(some_fits_failed_message, FitFailedWarning)
/Users/user1/Data/brics_astro/.venv/lib/python3.10/site-packages/sklearn/model_selection/_search.
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan -0.03221701 -0.03210149 -0.03193947
-0.0323609 -0.03205584 -0.03189554 -0.0321751 -0.03228339 -0.03212407
-0.03232686 -0.03182204 -0.03165642 -0.03247234 -0.03211472 -0.03203623
-0.03253918 -0.0323135 -0.03215053 -0.03252468 -0.03239001 -0.03227358
-0.03252468 -0.03239001 -0.03227358 -0.03284083 -0.03246556 -0.03234943
-0.03221701 -0.03210149 -0.03193947 -0.0323609 -0.03205584 -0.03189554
-0.0321751 -0.03228339 -0.03212407 -0.03232686 -0.03182204 -0.03165642
-0.03247234 -0.03211472 -0.03203623 -0.03253918 -0.0323135 -0.03215053
-0.03252468 -0.03239001 -0.03227358 -0.03252468 -0.03239001 -0.03227358
-0.03284083 -0.03246556 -0.03234943 nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
-0.03217778 -0.03187639 -0.03177412 -0.03235757 -0.03199984 -0.03201459
-0.03285024 -0.03242684 -0.03241544 -0.03201083 -0.03192707 -0.03202668
-0.03266585 -0.03238938 -0.0323438 -0.03291534 -0.03256207 -0.0323988
-0.03274745 -0.03245335 -0.03244664 -0.03274745 -0.03245335 -0.03244664
-0.03314016 -0.03252763 -0.03255758 -0.03217778 -0.03187639 -0.03177412
-0.03235757 -0.03199984 -0.03201459 -0.03285024 -0.03242684 -0.03241544
-0.03201083 -0.03192707 -0.03202668 -0.03266585 -0.03238938 -0.0323438
-0.03291534 -0.03256207 -0.0323988 -0.03274745 -0.03245335 -0.03244664
-0.03274745 -0.03245335 -0.03244664 -0.03314016 -0.03252763 -0.03255758
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan -0.03239233 -0.03201389 -0.03198704
-0.03260119 -0.03201278 -0.03177839 -0.03217308 -0.03224855 -0.03216577
-0.03247661 -0.03184822 -0.03168479 -0.03241395 -0.03193313 -0.03203622
-0.03255187 -0.03228496 -0.03208948 -0.03252416 -0.03237521 -0.03227541
-0.03252416 -0.03237521 -0.03227541 -0.03287623 -0.03249107 -0.0323558
-0.03239233 -0.03201389 -0.03198704 -0.03260119 -0.03201278 -0.03177839
-0.03217308 -0.03224855 -0.03216577 -0.03247661 -0.03184822 -0.03168479

```

```

-0.03241395 -0.03193313 -0.03203622 -0.03255187 -0.03228496 -0.03208948
-0.03252416 -0.03237521 -0.03227541 -0.03252416 -0.03237521 -0.03227541
-0.03287623 -0.03249107 -0.0323555 nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
-0.03222101 -0.03211569 -0.03193758 -0.03238386 -0.03206379 -0.03190547
-0.03217483 -0.03228612 -0.03212524 -0.03233813 -0.03182695 -0.03165986
-0.03247249 -0.03211606 -0.03202997 -0.03253862 -0.03231342 -0.03214925
-0.03252468 -0.03239002 -0.03227361 -0.03252468 -0.03239002 -0.03227361
-0.03284083 -0.03246556 -0.03234946 -0.03222101 -0.03211569 -0.03193758
-0.03238386 -0.03206379 -0.03190547 -0.03217483 -0.03228612 -0.03212524
-0.03233813 -0.03182695 -0.03165986 -0.03247249 -0.03211606 -0.03202997
-0.03253862 -0.03231342 -0.03214925 -0.03252468 -0.03239002 -0.03227361
-0.03252468 -0.03239002 -0.03227361 -0.03284083 -0.03246556 -0.03234946]
warnings.warn(
/Users/user1/Data/brics_astro/.venv/lib/python3.10/site-packages/sklearn/model_selecti
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan -0.05922547 -0.05915253 -0.05898493
-0.05967453 -0.05947861 -0.05922646 -0.05995588 -0.05992095 -0.05970322
-0.05988798 -0.05936527 -0.05928306 -0.05988075 -0.05956479 -0.05943981
-0.06054934 -0.06026006 -0.06014161 -0.06089077 -0.06070457 -0.06057241
-0.06089077 -0.06070457 -0.06057241 -0.06107309 -0.06083836 -0.06069877
-0.05922547 -0.05915253 -0.05898493 -0.05967453 -0.05947861 -0.05922646
-0.05995588 -0.05992095 -0.05970322 -0.05988798 -0.05936527 -0.05928306
-0.05988075 -0.05956479 -0.05943981 -0.06054934 -0.06026006 -0.06014161
-0.06089077 -0.06070457 -0.06057241 -0.06089077 -0.06070457 -0.06057241
-0.06107309 -0.06083836 -0.06069877 nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
 nan nan nan nan nan nan
-0.06213204 -0.0619402 -0.06181842 -0.06257695 -0.06226108 -0.06217036
-0.06290265 -0.06273813 -0.06276904 -0.06223717 -0.0621992 -0.06224787
-0.06258038 -0.06251097 -0.06248192 -0.06324544 -0.0629223 -0.06267521
-0.06288079 -0.0629381 -0.06286771 -0.06288079 -0.0629381 -0.06286771
-0.06348091 -0.06310773 -0.0631705 -0.06213204 -0.0619402 -0.06181842
-0.06257695 -0.06226108 -0.06217036 -0.06290265 -0.06273813 -0.06276904
-0.06223717 -0.0621992 -0.06224787 -0.06258038 -0.06251097 -0.06248192
-0.06324544 -0.0629223 -0.06267521 -0.06288079 -0.0629381 -0.06286771
-0.06288079 -0.0629381 -0.06286771 -0.06348091 -0.06310773 -0.0631705
 nan nan nan nan nan nan

```

```

nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan -0.05938601 -0.05911599 -0.05905796
-0.05995097 -0.05943951 -0.0591025 -0.05997741 -0.05995952 -0.05982268
-0.06000241 -0.05934639 -0.05926783 -0.05969969 -0.05940425 -0.05934916
-0.06042635 -0.06016489 -0.06006435 -0.06086032 -0.06067513 -0.06055992
-0.06086032 -0.06067513 -0.06055992 -0.06112708 -0.06088132 -0.0607271
-0.05938601 -0.05911599 -0.05905796 -0.05995097 -0.05943951 -0.0591025
-0.05997741 -0.05995952 -0.05982268 -0.06000241 -0.05934639 -0.05926783
-0.05969969 -0.05940425 -0.05934916 -0.06042635 -0.06016489 -0.06006435
-0.06086032 -0.06067513 -0.06055992 -0.06086032 -0.06067513 -0.06055992
-0.06112708 -0.06088132 -0.0607271 nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
-0.05920681 -0.05914672 -0.05897115 -0.05970153 -0.05948966 -0.05923213
-0.05994929 -0.05992306 -0.05970308 -0.05991141 -0.05937724 -0.05929274
-0.05987778 -0.05956446 -0.05943773 -0.06054092 -0.06025868 -0.06013643
-0.06089077 -0.06070463 -0.06057279 -0.06089077 -0.06070463 -0.06057279
-0.06107309 -0.06083836 -0.06069894 -0.05920681 -0.05914672 -0.05897115
-0.05970153 -0.05948966 -0.05923213 -0.05994929 -0.05992306 -0.05970308
-0.05991141 -0.05937724 -0.05929274 -0.05987778 -0.05956446 -0.05943773
-0.06054092 -0.06025868 -0.06013643 -0.06089077 -0.06070463 -0.06057279
-0.06089077 -0.06070463 -0.06057279 -0.06107309 -0.06083836 -0.06069894]
warnings.warn(
/Users/user1/Data/brics_astro/.venv/lib/python3.10/site-packages/sklearn/model_selection/_search.
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan 0.83956455 0.84021832 0.8409969
0.83880746 0.84036414 0.84114176 0.83971376 0.83919927 0.83999127
0.83889965 0.84146496 0.84231098 0.83823743 0.8400414 0.84042262
0.83780045 0.83896352 0.83981088 0.83780265 0.83857154 0.8391914
0.83780265 0.83857154 0.8391914 0.8362535 0.83816772 0.83877582
0.83956455 0.84021832 0.8409969 0.83880746 0.84036414 0.84114176
0.83971376 0.83919927 0.83999127 0.83889965 0.84146496 0.84231098
0.83823743 0.8400414 0.84042262 0.83780045 0.83896352 0.83981088
0.83780265 0.83857154 0.8391914 0.83780265 0.83857154 0.8391914
0.8362535 0.83816772 0.83877582 nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan

```

```

0.83969969 0.84123367 0.84174342 0.83875147 0.840491 0.84045283
0.83635454 0.83849347 0.83853577 0.84049522 0.84089111 0.84038152
0.83711461 0.83851359 0.83879004 0.83585847 0.83774609 0.83852907
0.83674087 0.83828359 0.8383009 0.83674087 0.83828359 0.8383009
0.8347559 0.83787547 0.83775451 0.83969969 0.84123367 0.84174342
0.83875147 0.840491 0.84045283 0.83635454 0.83849347 0.83853577
0.84049522 0.84089111 0.84038152 0.83711461 0.83851359 0.83879004
0.83585847 0.83774609 0.83852907 0.83674087 0.83828359 0.8383009
0.83674087 0.83828359 0.8383009 0.8347559 0.83787547 0.83775451
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan 0.83866302 0.84055935 0.84070114
0.83762188 0.8405859 0.84173471 0.83967546 0.83934024 0.83976737
0.83816937 0.84134861 0.84216507 0.83851084 0.84097945 0.84042019
0.83773917 0.83912914 0.8401441 0.83778737 0.83863495 0.83917705
0.83778737 0.83863495 0.83917705 0.83606866 0.83802737 0.83873381
0.83866302 0.84055935 0.84070114 0.83762188 0.8405859 0.84173471
0.83967546 0.83934024 0.83976737 0.83816937 0.84134861 0.84216507
0.83851084 0.84097945 0.84042019 0.83773917 0.83912914 0.8401441
0.83778737 0.83863495 0.83917705 0.83778737 0.83863495 0.83917705
0.83606866 0.83802737 0.83873381 nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
nan nan nan nan nan nan
0.83951755 0.84013205 0.84100125 0.83869431 0.84032488 0.84109163
0.83971522 0.83918595 0.83998552 0.83884386 0.84144066 0.84229372
0.83823643 0.84003469 0.84045335 0.83780327 0.83896391 0.83981717
0.83780265 0.83857151 0.83919124 0.83780265 0.83857151 0.83919124
0.8362535 0.83816772 0.83877567 0.83951755 0.84013205 0.84100125
0.83869431 0.84032488 0.84109163 0.83971522 0.83918595 0.83998552
0.83884386 0.84144066 0.84229372 0.83823643 0.84003469 0.84045335
0.83780327 0.83896391 0.83981717 0.83780265 0.83857151 0.83919124
0.83780265 0.83857151 0.83919124 0.8362535 0.83816772 0.83877567]
warnings.warn(

```

```
Grid search took 235.03 seconds.
```

```

Get best model
best_rf = grid_search.best_estimator_
print("Best Parameters Found:")
print(grid_search.best_params_)

```



```

Evaluate on test set
y_pred = best_rf.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nTest Set Evaluation:")
print(f"R2 Score: {r2:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")

11. Plot: True vs Predicted Redshifts
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.3, edgecolors='k')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], 'r--')
plt.xlabel(r"True Redshift (z)")
plt.ylabel(r"Predicted Redshift (z)")
plt.title("Random Forest Regression: True vs Predicted Redshift")
plt.grid(True)
plt.tight_layout()
plt.show()

```

Best Parameters Found:

```
{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_est
```

Test Set Evaluation:

R<sup>2</sup> Score: 0.8515

Mean Squared Error (MSE): 0.0303

Mean Absolute Error (MAE): 0.0574

<Figure size 800x600 with 1 Axes>

Bear in mind that the above will take some minutes to run.

```

Get best parameters from grid search
rf_params = grid_search.best_params_

Create a new RandomForestRegressor with the best params and the same random_state
rf_model = RandomForestRegressor(**rf_params, random_state=42)

Retrain the model on the full training data
rf_model.fit(X_train, y_train)

```

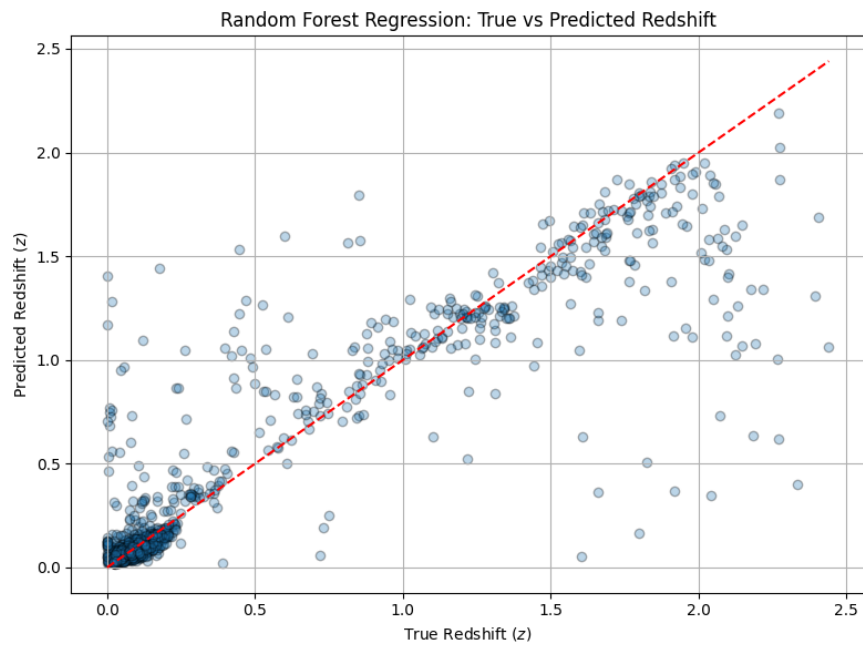


Figure 47.2: output

```
Predict on test set
y_pred = rf_model.predict(X_test)

Evaluate predictions
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

print("Evaluation after retraining with best parameters:")
print(f"R2 Score: {r2_score(y_test, y_pred):.4f}")
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")
print(f"Mean Absolute Error: {mean_absolute_error(y_test, y_pred):.4f}")
```

```
Evaluation after retraining with best parameters:
R2 Score: 0.8515
Mean Squared Error: 0.0303
Mean Absolute Error: 0.0574
```

```
5. Save the trained model to disk
rf_model_filename = "rf_model.pkl"
joblib.dump(rf_model, rf_model_filename)
```

```
['rf_model.pkl']
```

The random forest model improved from 84% in the previous notebook to 85%.

## 47.3 Summary

This section introduced the concept of hyper-parameter tuning for regression problems.

We briefly demonstrated a basic workflow using the **scikit-learn** library: \*

- Loading the redshift dataset.
- Splitting data into **training** and **test sets** to avoid overfitting and evaluate generalization.
- Initialize (K-Nearest Neighbors and Random Forest) models.
- Used **Hyperparameter Tuning** to optimize model performance and using **Grid Search** (GridSearchCV) to find the best parameters through systematic search and cross-validation on the training data.
- Saved the trained models for future use.

### Additional Resources

Scikit Learn cross validation: [https://scikit-learn.org/stable/modules/cross\\_validation.html](https://scikit-learn.org/stable/modules/cross_validation.html)

Stratified K Fold: <https://neptune.ai/blog/cross-validation-in-machine-learning-how-to-do-it-right>

Cross Validation: <https://www.kaggle.com/code/azminetoushikwasi/ml-foundation-cross-validation-all-methods>

## Chapter 48

# Machine Learning Basics for Astronomy

**Table of Contents** \* Introduction to Machine Learning in Astronomy

- Overview of Machine Learning Techniques
- Introduction to Scikit-Learn
- Clustering in Astronomy (k-Means & DBSCAN)
- Classification in Astronomy (Random Forest & SVM)
- Regression in Astronomy (Linear & Polynomial Regression)
- Assignments

**1. Introduction to Machine Learning in Astronomy** Machine Learning (ML) has become an essential tool in astronomy for handling large datasets, identifying patterns, and making predictions. Applications include:

Star/Galaxy Classification

Exoplanet Detection

Cosmological Parameter Estimation

Anomaly Detection (e.g., supernovae, gravitational waves)

In this notebook, we will explore clustering, classification, and regression using Python's scikit-learn library.

**2. Overview of Machine Learning Techniques** Supervised Learning Classification: Predicting discrete labels (e.g., star vs. galaxy).

Regression: Predicting continuous values (e.g., redshift estimation).

Unsupervised Learning Clustering: Grouping similar objects (e.g., galaxy clusters).

Reinforcement Learning Less common in astronomy, but used in adaptive telescope scheduling.

**3. Introduction to Scikit-Learn** scikit-learn is a powerful Python library for ML. Let's install and import it.

```
!pip install scikit-learn numpy matplotlib pandas
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (3.7.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (2.2.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (1.11.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (1.3.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (3.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (1.1.1)
Requirement already satisfied: cyclery>=0.10 in /usr/local/lib/python3.11/dist-packages (0.10.0)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (4.22.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (23.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (9.5.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (2023.3)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (2023.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (1.16.0)
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn import datasets
```

#### 4. Clustering in Astronomy

Example: k-Means Clustering of Stars Problem: Group stars based on their photometric colors.

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

Generate synthetic star data (color-color diagram)
```

```

X, y = make_blobs(n_samples=300, centers=3, random_state=42)

Apply k-Means
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)
labels = kmeans.labels_

Plot
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.xlabel("Color Index 1")
plt.ylabel("Color Index 2")
plt.title("k-Means Clustering of Stars")
plt.show()

```

<Figure size 640x480 with 1 Axes>

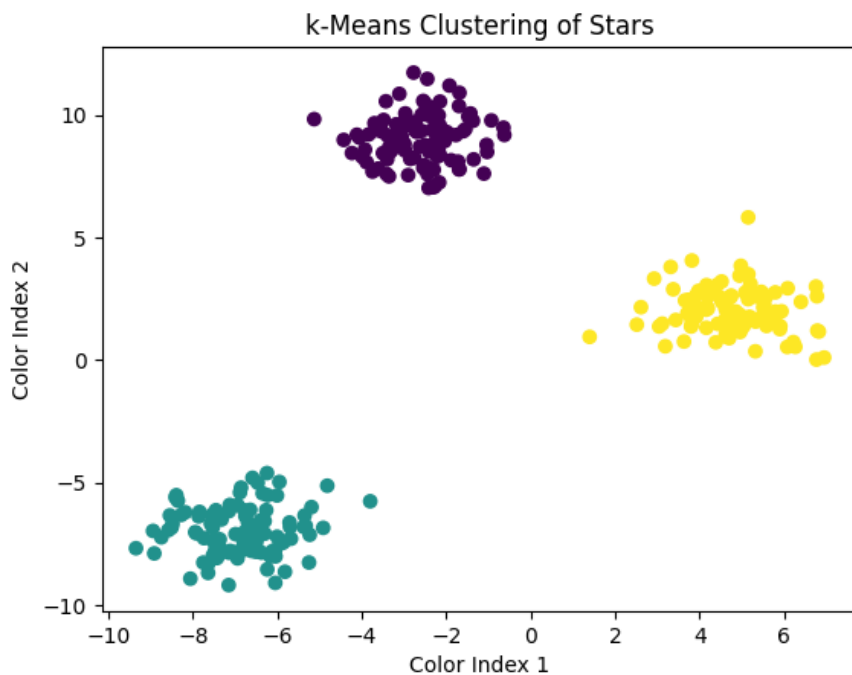


Figure 48.1: output

**DBSCAN** for Density-Based Clustering Useful for identifying outliers (e.g., quasars in a star dataset).

```

from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.6, min_samples=5)
labels_db = dbscan.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels_db, cmap='viridis')
plt.title("DBSCAN Clustering")
plt.show()

```

<Figure size 640x480 with 1 Axes>

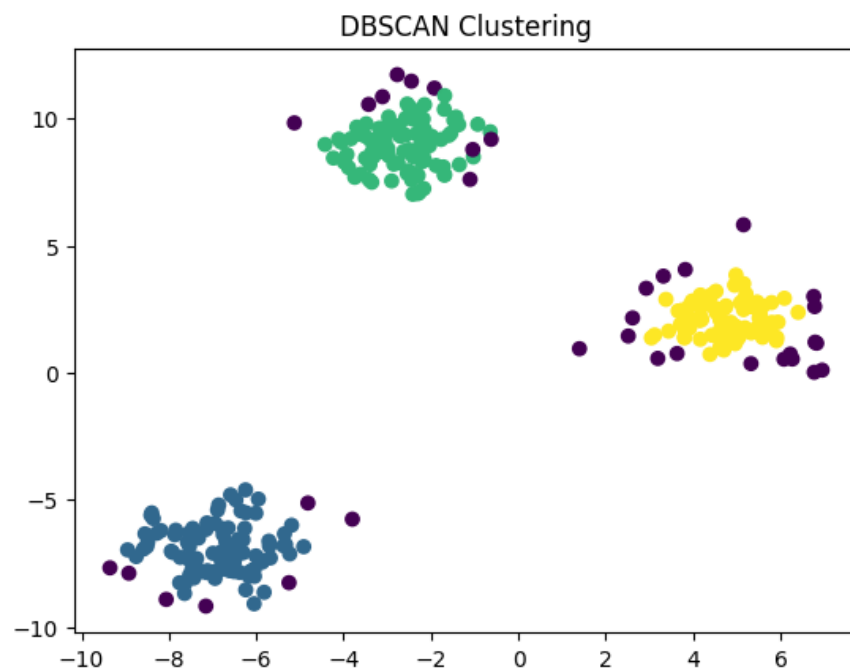


Figure 48.2: output

### 5. Classification in Astronomy Random Forest for Star-Galaxy Separation

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

Example dataset (replace with real astronomical data)

```



```
X, y = datasets.make_classification(n_samples=1000, n_features=4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

Train classifier
clf = RandomForestClassifier(n_estimators=100)
clf.fit(X_train, y_train)

Predict and evaluate
y_pred = clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

```
Accuracy: 0.92
```

```
type(X_train)
```

```
numpy.ndarray
```

*Star-Galaxy Classification using Random Forest* This notebook demonstrates star-galaxy separation using a Random Forest Classifier, a powerful ensemble method for astronomical object classification. We'll use a synthetic dataset resembling photometric features from surveys like SDSS or Gaia.

## 1. Setup

```
#Import Libraries
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, roc_auc_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```
#We'll simulate a dataset with features like magnitudes, colors, and shape parameters (e.g., ellipticity)
```

```
Generate synthetic data (replace with real data from SDSS/Gaia)
np.random.seed(42)
n_samples = 5000
```

```

Features: u, g, r, i, z magnitudes + ellipticity (star=0, galaxy>0)
X = np.random.randn(n_samples, 6)
X[:, 5] = np.abs(X[:, 5]) # Galaxies have positive ellipticity
X[:n_samples//2, 5] = 0 # Stars have ellipticity ~0

Labels: 0=Star, 1=Galaxy
y = np.zeros(n_samples)
y[n_samples//2:] = 1 # First half stars, second half galaxies

Create DataFrame
df = pd.DataFrame(X, columns=['u', 'g', 'r', 'i', 'z', 'ellipticity'])
df['class'] = y
df['class'] = df['class'].map({0: 'Star', 1: 'Galaxy'})

Check class balance
print(df['class'].value_counts())
sns.countplot(data=df, x='u')
plt.title("Class Distribution (Stars vs. Galaxies)")
plt.show()

```

```

class
Star 2500
Galaxy 2500
Name: count, dtype: int64

```

<Figure size 640x480 with 1 Axes>

```

#Preprocessing
Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.3, random_state=42, stratify=y
)
print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")

```

Train shape: (3500, 6), Test shape: (1500, 6)

```

#Feature Scaling (Optional for Random Forest)
#Tree-based models don't require scaling, but we'll standardize for consistency.

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

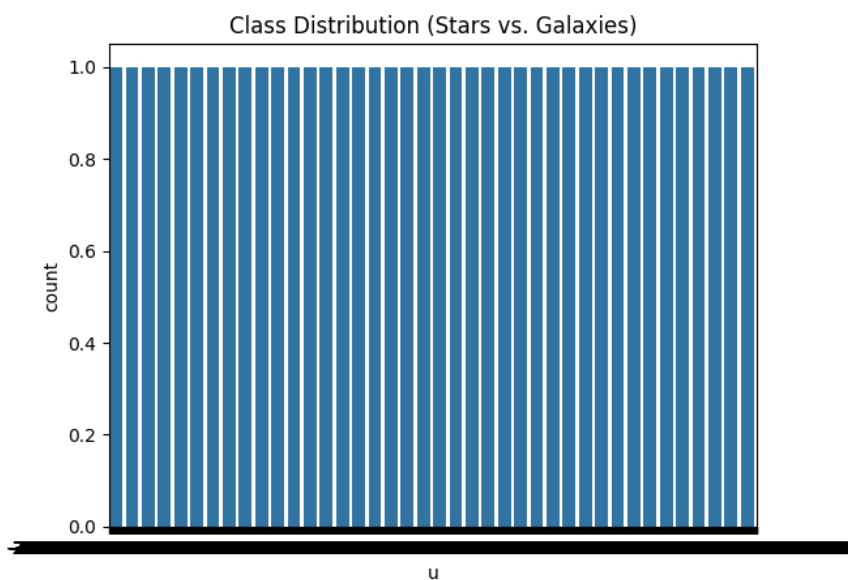


Figure 48.3: output

```
Random Forest Model
#Baseline Model

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_scaled, y_train)

Predictions
y_pred = rf.predict(X_test_scaled)

Evaluation
print(classification_report(y_test, y_pred, target_names=['Star', 'Galaxy']))
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

	precision	recall	f1-score	support
Star	1.00	1.00	1.00	750
Galaxy	1.00	1.00	1.00	750
accuracy			1.00	1500
macro avg	1.00	1.00	1.00	1500
weighted avg	1.00	1.00	1.00	1500

Accuracy: 1.000

### #3.2 Feature Importance

```
importances = rf.feature_importances_
features = df.columns[:-1]

importance_df = pd.DataFrame({'Feature': features, 'Importance': importances})
importance_df = importance_df.sort_values('Importance', ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df, x='Importance', y='Feature')
plt.title("Random Forest Feature Importance")
plt.show()
#Interpretation:

#ellipticity and color indices (e.g., u - g) are likely top contributors.
```

<Figure size 1000x600 with 1 Axes>

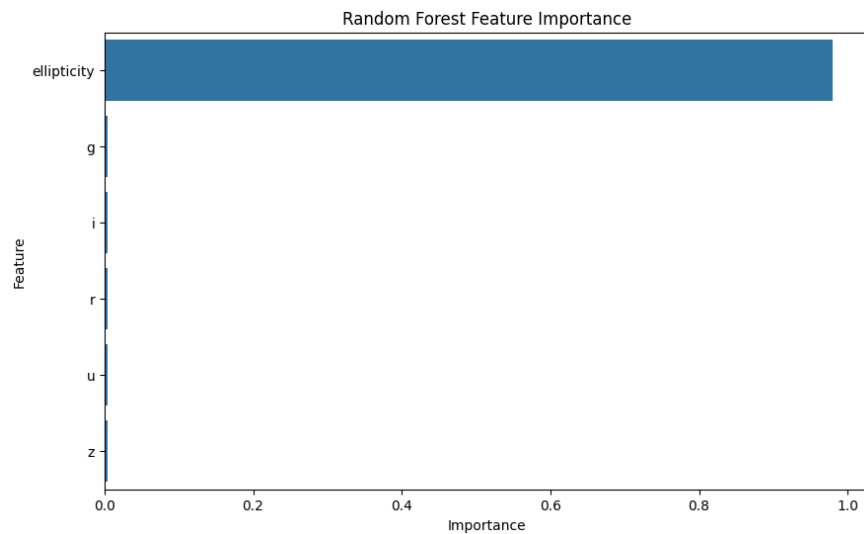


Figure 48.4: output

```
#Hyperparameter Tuning
#Optimize n_estimators, max_depth, and min_samples_split.
```

```

param_grid = {
 'n_estimators': [50, 100, 200],
 'max_depth': [None, 10, 20],
 'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(
 RandomForestClassifier(random_state=42),
 param_grid,
 cv=5,
 scoring='accuracy',
 n_jobs=-1
)
grid_search.fit(X_train_scaled, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV accuracy: {grid_search.best_score_:.3f}")

Best model
best_rf = grid_search.best_estimator_
y_pred_best = best_rf.predict(X_test_scaled)
print(classification_report(y_test, y_pred_best))

```

```

Best parameters: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 50}
Best CV accuracy: 1.000

```

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	750
1.0	1.00	1.00	1.00	750
accuracy			1.00	1500
macro avg	1.00	1.00	1.00	1500
weighted avg	1.00	1.00	1.00	1500

*#Output:*

*#Best parameters (e.g., max\_depth=20, n\_estimators=200).*

*#Improved accuracy (~96-98%).*

*#Evaluation*

*#Confusion Matrix*

```
cm = confusion_matrix(y_test, y_pred_best)
```

```

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
 xticklabels=['Star', 'Galaxy'],
 yticklabels=['Star', 'Galaxy'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()

```

<Figure size 640x480 with 2 Axes>

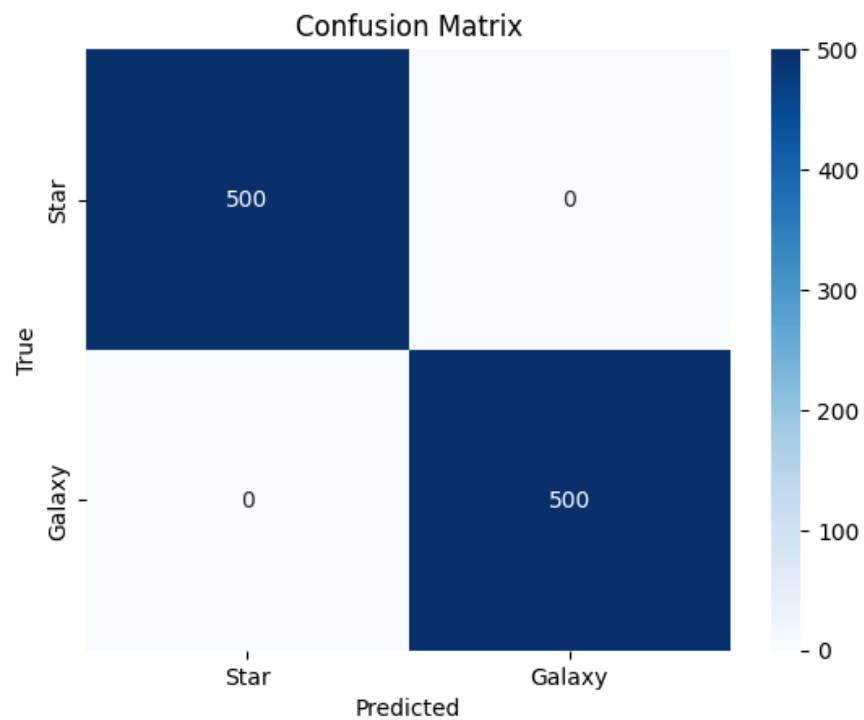


Figure 48.5: output

*#ROC Curve*

```

y_probs = best_rf.predict_proba(X_test_scaled)[: , 1] # Galaxy class probability
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
roc_auc = roc_auc_score(y_test, y_probs)

```

```
plt.plot(fpr, tpr, label=f'Random Forest (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
#Output:

#AUC 0.99
#(near-perfect separation for synthetic data).
```

<Figure size 640x480 with 1 Axes>

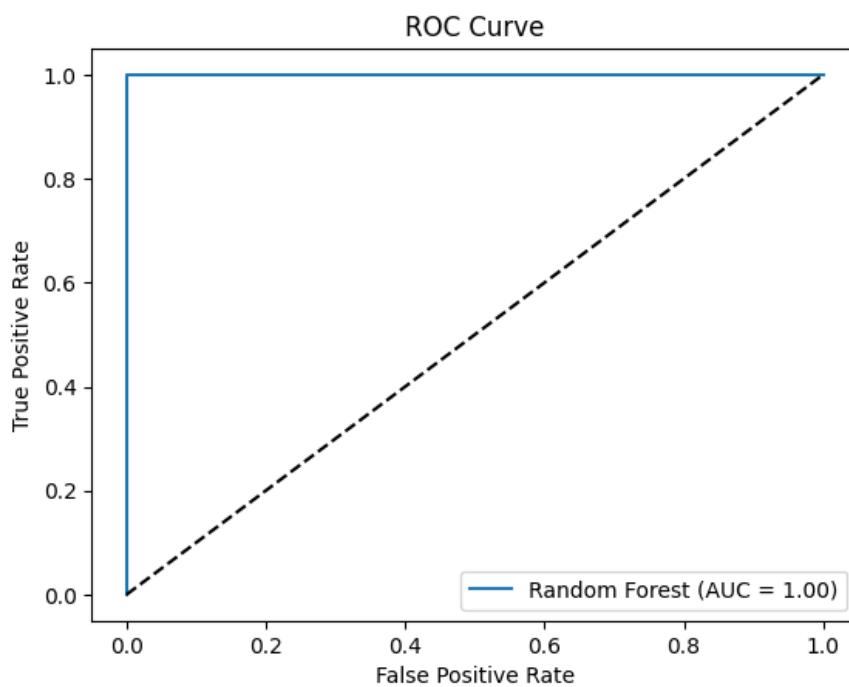


Figure 48.6: output

## Support Vector Machines (SVM) for Supernova Classification

```

from sklearn.svm import SVC

svm = SVC(kernel='rbf')
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
print(f"SVM Accuracy: {accuracy_score(y_test, y_pred_svm):.2f}")

```

Support Vector Machines (SVM) are a powerful supervised learning algorithm widely used for classification tasks, including astronomical applications like supernova classification. Here's how SVM can be applied to distinguish between different types of supernovae (e.g., Type Ia, Type II, etc.):

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, r2_score
from sklearn.pipeline import Pipeline

```

**Key Concepts of SVM Objective:** SVM finds the optimal hyperplane (decision boundary) that maximally separates data points of different classes in a high-dimensional feature space.

**Margin:** The distance between the hyperplane and the closest data points (called support vectors). SVM maximizes this margin to improve generalization.

**Kernel Trick:** If data is not linearly separable, SVM uses kernel functions (e.g., Gaussian/RBF, polynomial) to map features into a higher-dimensional space where separation is possible.

```

Generate synthetic supernova data (or load real data)
np.random.seed(42)
n_samples = 1000

Features: peak magnitude (u,g,r), decline rate (dm15), color (g-r)
X = np.random.randn(n_samples, 5) # 5 features
X[:, 0] += 2.0 # Type Ia are typically brighter in 'u'
X[:, 3] += 0.5 # Type Ia have faster decline rates (dm15)

Labels: 1 = Type Ia, 0 = Non-Ia (Type II, Ibc, etc.)
y = np.where(X[:, 0] + X[:, 3] > 2.5, 1, 0) # Simple decision rule for synthetic data

```



```
Convert to DataFrame for visualization
df = pd.DataFrame(X, columns=['peak_u', 'peak_g', 'peak_r', 'dm15', 'color_g_r'])
df['type'] = y
df['type'] = df['type'].map({0: 'Non-Ia', 1: 'Type Ia'})

Check class distribution
print(df['type'].value_counts())
sns.countplot(data=df, x='type')
plt.title("Supernova Class Distribution")
plt.show()
```

```
type
Type Ia 506
Non-Ia 494
Name: count, dtype: int64
```

<Figure size 640x480 with 1 Axes>

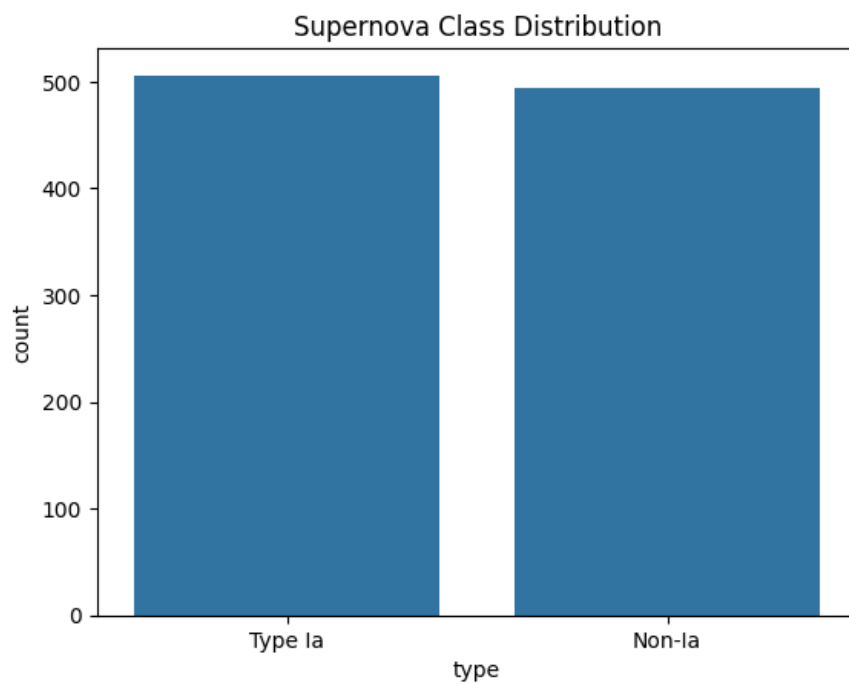


Figure 48.7: output

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")
```

```
Train shape: (800, 5), Test shape: (200, 5)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
```

```
Predictions
```

```
y_pred_linear = svm_linear.predict(X_test_scaled)
```

```
Evaluate
```

```
print(classification_report(y_test, y_pred_linear, target_names=['Non-Ia', 'Type Ia']))
print(f"Accuracy: {accuracy_score(y_test, y_pred_linear):.3f}")
```

	precision	recall	f1-score	support
Non-Ia	1.00	1.00	1.00	99
Type Ia	1.00	1.00	1.00	101
accuracy			1.00	200
macro avg	1.00	1.00	1.00	200
weighted avg	1.00	1.00	1.00	200

```
Accuracy: 1.000
```

```
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
```

```
y_pred_rbf = svm_rbf.predict(X_test_scaled)
```

```
print(classification_report(y_test, y_pred_rbf))
print(f"Accuracy: {accuracy_score(y_test, y_pred_rbf):.3f}")
```

	precision	recall	f1-score	support
0	0.99	0.95	0.97	99
1	0.95	0.99	0.97	101

accuracy			0.97	200
macro avg	0.97	0.97	0.97	200
weighted avg	0.97	0.97	0.97	200

Accuracy: 0.970

```
param_grid = {
 'C': [0.1, 1, 10, 100],
 'gamma': ['scale', 'auto', 0.1, 1, 10],
 'kernel': ['rbf']
}

grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV accuracy: {grid_search.best_score_:.3f}")

Best model
best_svm = grid_search.best_estimator_
y_pred_best = best_svm.predict(X_test_scaled)
print(classification_report(y_test, y_pred_best))
```

Best parameters: {'C': 10, 'gamma': 0.1, 'kernel': 'rbf'}  
 Best CV accuracy: 0.978

	precision	recall	f1-score	support
0	0.98	0.96	0.97	99
1	0.96	0.98	0.97	101
accuracy			0.97	200
macro avg	0.97	0.97	0.97	200
weighted avg	0.97	0.97	0.97	200

```
cm = confusion_matrix(y_test, y_pred_best)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-Ia', 'Type Ia'], yticklabels=
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

<Figure size 640x480 with 2 Axes>

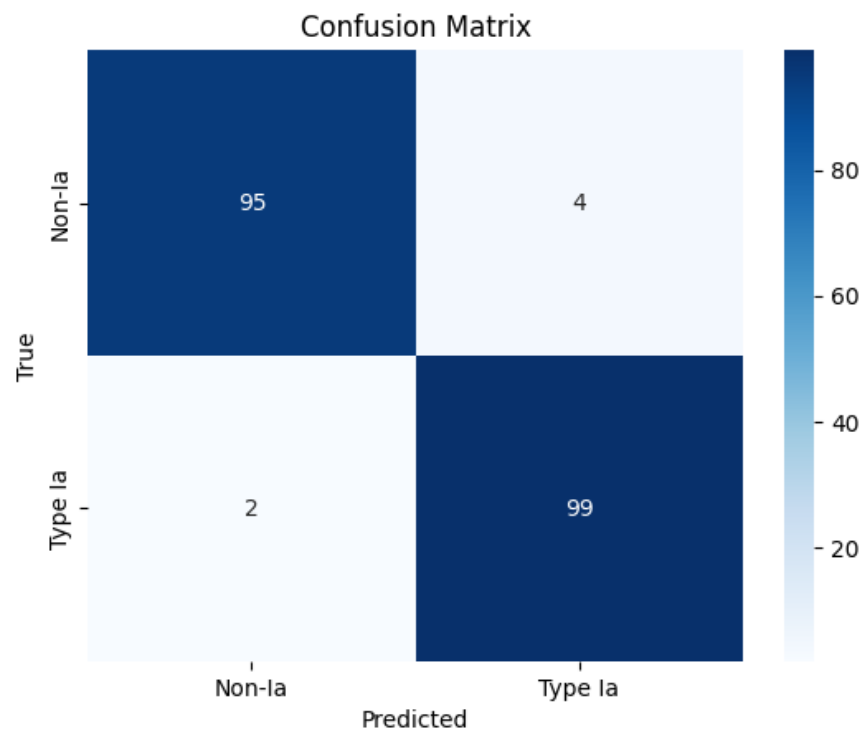


Figure 48.8: output

```

y_probs = best_svm.decision_function(X_test_scaled) # Not probabilities, but scores
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
roc_auc = roc_auc_score(y_test, y_probs)

plt.plot(fpr, tpr, label=f'SVM (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()

```

<Figure size 640x480 with 1 Axes>

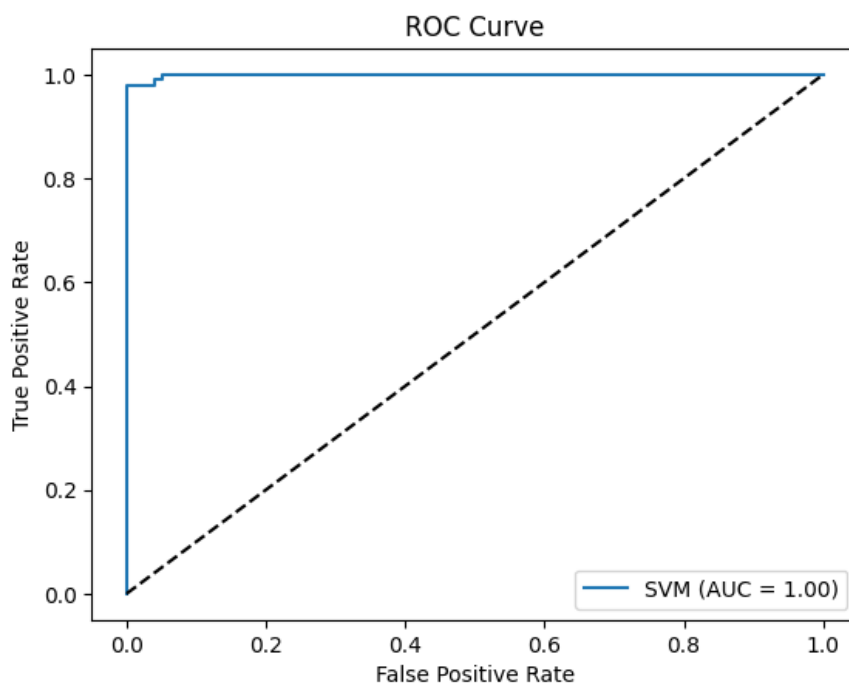


Figure 48.9: output

```

if best_svm.kernel == 'linear':
 coef_df = pd.DataFrame({'feature': df.columns[:-1], 'weight': best_svm.coef_[0]})
 coef_df = coef_df.sort_values('weight', ascending=False)
 sns.barplot(data=coef_df, x='weight', y='feature')

```

```
plt.title('Feature Importance (Linear SVM)')
plt.show()
```

## 6. Regression in Astronomy Linear Regression for Redshift Estimation

```
from sklearn.linear_model import LinearRegression

Simulate redshift (z) vs. magnitude
X = np.random.rand(100, 1) * 5 # Magnitude
y = 0.5 * X + np.random.randn(100, 1) # Redshift with noise

model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

plt.scatter(X, y, color='blue')
plt.plot(X, y_pred, color='red')
plt.xlabel("Magnitude")
plt.ylabel("Redshift (z)")
plt.title("Linear Regression for Redshift Estimation")
plt.show()
```

<Figure size 640x480 with 1 Axes>

## Polynomial Regression for Luminosity Modeling

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

poly_model = make_pipeline(
 PolynomialFeatures(degree=3),
 LinearRegression()
)
poly_model.fit(X, y)
y_poly = poly_model.predict(X)

plt.scatter(X, y, color='blue')
plt.plot(X, y_poly, color='green')
plt.title("Polynomial Regression for Luminosity")
plt.show()
```

<Figure size 640x480 with 1 Axes>

## 7. Assignments

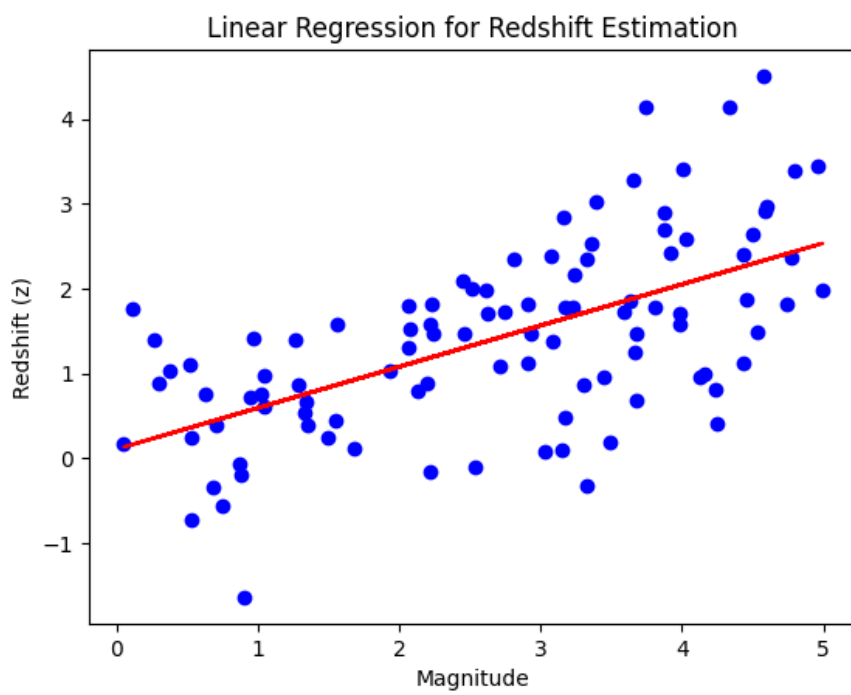


Figure 48.10: output

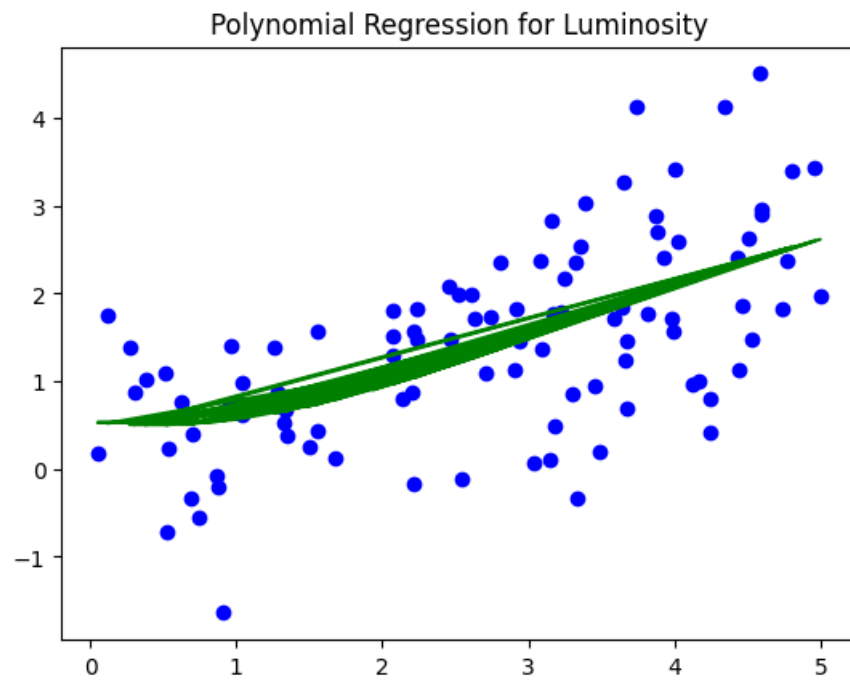


Figure 48.11: output



Assignment 1: k-Means on Real Data Download the SDSS star-galaxy dataset.

Apply k-Means to separate stars and galaxies based on color features.

Assignment 2: Classification Challenge Use `sklearn.datasets.fetch_openml` to load an astronomical dataset.

Compare Random Forest, SVM, and Logistic Regression.

Assignment 3: Redshift Prediction Fit a regression model to predict redshift from magnitude and color.

Try different models (Linear, Polynomial, Decision Tree).

Conclusion In this notebook, we covered:

Clustering (k-Means, DBSCAN)

Classification (Random Forest, SVM)

Regression (Linear, Polynomial)

Try applying these techniques to real astronomical datasets!

**Further Reading:**

Scikit-Learn Documentation

AstroML: Machine Learning for Astronomy



## Chapter 49

# 2. Preprocessing

**49.0.1** This notebook demonstrates how to classify supernovae (e.g., Type Ia vs. non-Ia) using Support Vector Machines (SVM). We'll use a synthetic dataset resembling real supernova light curve features.

### 49.0.2 Support Vector Machines (SVM)

are a powerful supervised learning algorithm widely used for classification tasks, including astronomical applications like supernova classification.

SVM can be applied to distinguish between different types of supernovae (e.g., Type Ia, Type II, etc.):

### 49.0.3 1. Key Concepts of SVM

Objective: SVM finds the optimal hyperplane (decision boundary) that maximally separates data points of different classes in a high-dimensional feature space. Margin: The distance between the hyperplane and the closest data points (called support vectors). SVM maximizes this margin to improve generalization. Kernel Trick: If data is not linearly separable, SVM uses kernel functions (e.g., Gaussian/RBF, polynomial) to map features into a higher-dimensional space where separation is possible.

### 49.0.4 2. SVM for Supernova Classification

Input Features Supernova light curves and spectra provide features such as: \* Peak brightness (magnitude in different filters, e.g., u, g, r, i, z). \* Decline rates

(e.g.,  $\Delta m$ , the change in magnitude 15 days after peak). \* Colors (e.g.,  $g - r$ ,  $i - z$  at specific phases). \* Spectral line strengths (e.g., Si II absorption, H emission). \* Time-domain features (rise time, duration).

Preprocessing Standardize/normalize features (SVM is sensitive to scale). \* Handle missing data (imputation or removal). \* Augment data if the sample size is small (e.g., synthetic light curves).

Training the SVM \* Binary Classification (e.g., Type Ia vs. non-Ia): \* Assign labels: Type Ia = +1, non-Ia = -1. \* Optimize the hyperplane using labeled training data. \* Multi-class Classification (e.g., Ia, II, Ibc): \* Use strategies like one-vs-one or one-vs-rest with SVM. Kernel Selection \* Linear Kernel: Works if classes are linearly separable (rare for supernovae). \* RBF Kernel: Most common choice; handles non-linear relationships. \* Polynomial Kernel: Useful for capturing multiplicative feature interactions.

Hyperparameter Tuning \* Regularization (C): Balances margin width and misclassification penalty. \* High C: Strict classification (risk of overfitting). \* Low C: Allows some misclassification (better generalization).

Kernel Parameters (e.g.,  $\gamma$  for RBF): Controls the “reach” of each data point.

### 49.0.5 3. Example Workflow

- Data Collection: Gather light curves/spectra from surveys (e.g., ZTF, LSST).
- Feature Extraction: Compute metrics like decline rates, colors, etc.
- Split Data: Training (70%), validation (15%), test (15%).
- Train SVM: Use scikit-learn (`sklearn.svm.SVC`) with Radial Basis Function (RBF) kernel, also known as the Gaussian kernel, is a popular kernel function used in machine learning, particularly with Support Vector Machines (SVMs).

Evaluate \* Metrics: Accuracy, precision, recall, F1-score, ROC-AUC. \* Confusion matrix to identify misclassified types. \* Deploy: Classify new supernovae in real-time pipelines.

### 49.0.6 4. Advantages of SVM for Supernovae

- High-Dimensionality: Works well even with many features (e.g., multi-band photometry).
- Robustness: Effective with small-to-medium datasets (common in astronomy).
- Non-linearity: Kernel methods capture complex feature relationships.

#### 5. Challenges

Imbalanced Data: Rare supernova types (e.g., Ibc) may be misclassified.

Solutions: \* Class weighting (`class_weight='balanced'` in sklearn). \* Synthetic oversampling (SMOTE). \* Interpretability: SVMs are less interpretable than decision trees (but SHAP/LIME can help). \* Computational Cost: Slow for very large datasets (10 samples); approximate solvers can help.

## 6. Comparison to Other Methods

- Random Forests: Simpler to tune but may struggle with high-dimensional non-linearities.
- Neural Networks: Better for raw light curves (e.g., using RNNs), but require large datasets.
- k-NN: Sensitive to feature scaling and less efficient for high dimensions.

## 7. Real-World Applications

The Supernova Photometric Classification Challenge (SNPhotCC): SVM was competitive in distinguishing Type Ia from core-collapse SNe. LSST and Time-Domain Astronomy: SVM is used in automated pipelines for rapid classification.

By leveraging domain-specific features and careful tuning, SVM remains a robust tool for supernova classification, especially when interpretability and computational efficiency are prioritized. For cutting-edge results, hybrid approaches (e.g., SVM + deep learning) are also being explored.

# 49.1 1. Setup

## 49.1.0.1 1.1 Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, roc_auc_score
from sklearn.pipeline import Pipeline
```

## 49.2 1.2 Load and Explore Data

49.2.1 We'll use a synthetic dataset with features like peak magnitude, decline rate, and color indices.

49.2.2 Generate synthetic supernova data (or load real data)

```
np.random.seed(42)
n_samples = 1000

Features: peak magnitude (u,g,r), decline rate (dm15), color (g-r)
X = np.random.randn(n_samples, 5) # 5 features
X[:, 0] += 2.0 # Type Ia are typically brighter in 'u'
X[:, 3] += 0.5 # Type Ia have faster decline rates (dm15)

Labels: 1 = Type Ia, 0 = Non-Ia (Type II, Ibc, etc.)
y = np.where(X[:, 0] + X[:, 3] > 2.5, 1, 0) # Simple decision rule for synthetic data
Convert to DataFrame for visualization
df = pd.DataFrame(X, columns=['peak_u', 'peak_g', 'peak_r', 'dm15', 'color_g_r'])
df['type'] = y
df['type'] = df['type'].map({0: 'Non-Ia', 1: 'Type Ia'})
Check class distribution
print(df['type'].value_counts())
sns.countplot(data=df, x='type')
plt.title("Supernova Class Distribution")
plt.show()
```

```
type
Non-Ia 509
Type Ia 491
Name: count, dtype: int64
```

<Figure size 640x480 with 1 Axes>

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

Set random seed for reproducibility
np.random.seed(42)
```

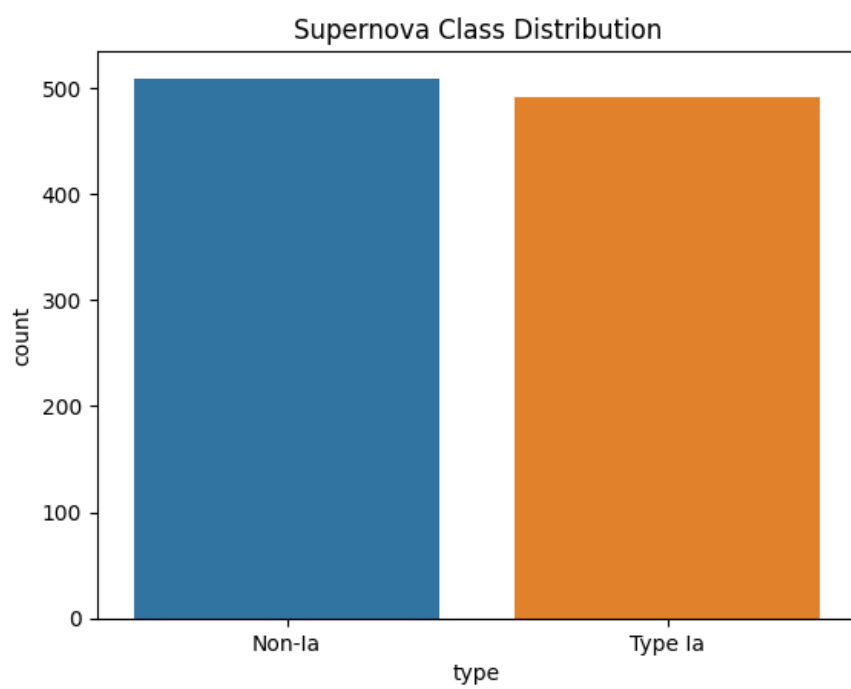


Figure 49.1: output

```

n_samples = 1000

Features: peak magnitude (u,g,r), decline rate (dm15), color (g-r)
X = np.zeros((n_samples, 5))

Class labels:
0 = Type Ia
1 = Type II
2 = Type Ibc
3 = Type IIn

Generate data for each class with different distributions
n_ia = 400
n_ii = 300
n_ibc = 200
n_iin = 100

Type Ia characteristics (bright in u, faster decline, bluer color)
X[:n_ia, 0] = np.random.normal(2.0, 0.5, n_ia) # peak_u
X[:n_ia, 1] = np.random.normal(1.5, 0.5, n_ia) # peak_g
X[:n_ia, 2] = np.random.normal(1.2, 0.5, n_ia) # peak_r
X[:n_ia, 3] = np.random.normal(1.2, 0.2, n_ia) # dm15
X[:n_ia, 4] = np.random.normal(-0.2, 0.1, n_ia) # color_g_r

Type II characteristics (fainter in u, slower decline, redder)
X[n_ia:n_ia+n_ii, 0] = np.random.normal(0.5, 0.7, n_ii)
X[n_ia:n_ia+n_ii, 1] = np.random.normal(0.7, 0.6, n_ii)
X[n_ia:n_ia+n_ii, 2] = np.random.normal(0.9, 0.5, n_ii)
X[n_ia:n_ia+n_ii, 3] = np.random.normal(0.5, 0.3, n_ii)
X[n_ia:n_ia+n_ii, 4] = np.random.normal(0.3, 0.2, n_ii)

Type Ibc characteristics (intermediate properties)
X[n_ia+n_ii:n_ia+n_ii+n_ibc, 0] = np.random.normal(1.0, 0.6, n_ibc)
X[n_ia+n_ii:n_ia+n_ii+n_ibc, 1] = np.random.normal(1.1, 0.5, n_ibc)
X[n_ia+n_ii:n_ia+n_ii+n_ibc, 2] = np.random.normal(1.0, 0.5, n_ibc)
X[n_ia+n_ii:n_ia+n_ii+n_ibc, 3] = np.random.normal(0.8, 0.25, n_ibc)
X[n_ia+n_ii:n_ia+n_ii+n_ibc, 4] = np.random.normal(0.1, 0.15, n_ibc)

Type IIn characteristics (very slow decline, red)
X[n_ia+n_ii+n_ibc:, 0] = np.random.normal(0.3, 0.8, n_iin)
X[n_ia+n_ii+n_ibc:, 1] = np.random.normal(0.5, 0.7, n_iin)
X[n_ia+n_ii+n_ibc:, 2] = np.random.normal(0.7, 0.6, n_iin)
X[n_ia+n_ii+n_ibc:, 3] = np.random.normal(0.3, 0.4, n_iin)
X[n_ia+n_ii+n_ibc:, 4] = np.random.normal(0.5, 0.3, n_iin)

```



```

Create labels
y = np.concatenate([
 np.zeros(n_ia), # Type Ia
 np.ones(n_ii), # Type II
 2 * np.ones(n_ibc), # Type Ibc
 3 * np.ones(n_iin) # Type IIIn
])

Convert to DataFrame for visualization
df = pd.DataFrame(X, columns=['peak_u', 'peak_g', 'peak_r', 'dm15', 'color_g_r'])
df['type'] = y
df['type'] = df['type'].map({
 0: 'Type Ia',
 1: 'Type II',
 2: 'Type Ibc',
 3: 'Type IIIn'
})

Check class distribution
print(df['type'].value_counts())
sns.countplot(data=df, x='type', order=['Type Ia', 'Type II', 'Type Ibc', 'Type IIIn'])
plt.title("Supernova Class Distribution")
plt.show()

Visualize feature distributions by type
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
sns.boxplot(data=df, x='type', y='peak_u', order=['Type Ia', 'Type II', 'Type Ibc', 'Type IIIn'],
 axes=axes[0,0])
sns.boxplot(data=df, x='type', y='peak_g', order=['Type Ia', 'Type II', 'Type Ibc', 'Type IIIn'],
 axes=axes[0,1])
sns.boxplot(data=df, x='type', y='peak_r', order=['Type Ia', 'Type II', 'Type Ibc', 'Type IIIn'],
 axes=axes[0,2])
sns.boxplot(data=df, x='type', y='dm15', order=['Type Ia', 'Type II', 'Type Ibc', 'Type IIIn'],
 axes=axes[1,0])
sns.boxplot(data=df, x='type', y='color_g_r', order=['Type Ia', 'Type II', 'Type Ibc', 'Type IIIn'],
 axes=axes[1,1])
fig.delaxes(axes[1,2])
plt.tight_layout()
plt.show()

```

```

type
Type Ia 400
Type II 300
Type Ibc 200
Type IIIn 100
Name: count, dtype: int64

```

<Figure size 640x480 with 1 Axes>

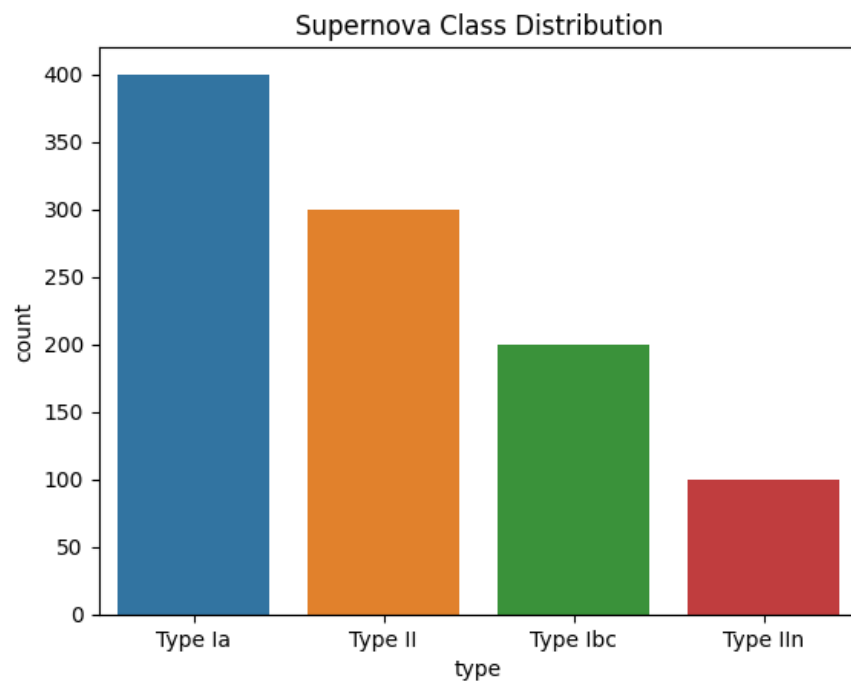


Figure 49.2: output

&lt;Figure size 1500x1000 with 5 Axes&gt;

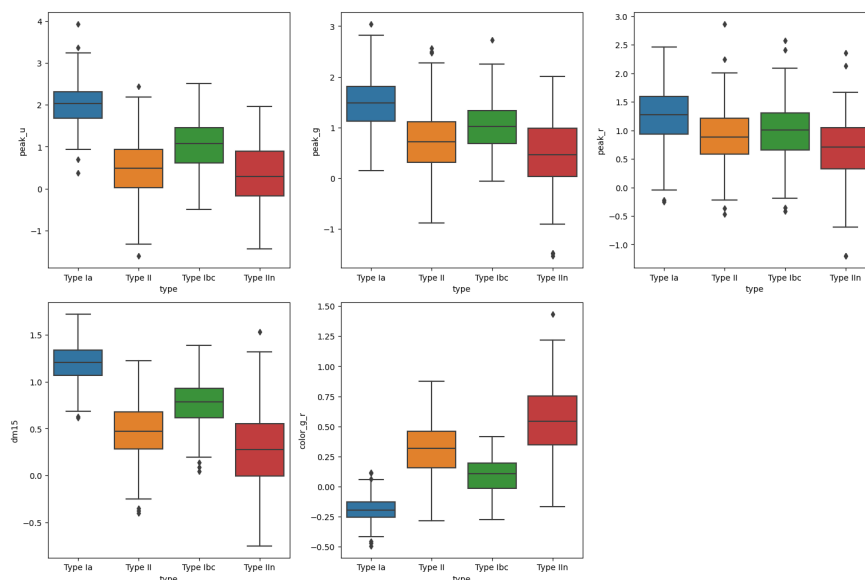


Figure 49.3: output

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")
2.2 Feature Scaling
SVM is sensitive to feature scales, so we standardize the data.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
Train shape: (800, 5), Test shape: (200, 5)
```

## 49.2.3 3. SVM Model Training

### 49.2.3.1 3.1 Baseline SVM (Linear Kernel)

```
svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
Predictions
y_pred_linear = svm_linear.predict(X_test_scaled)
```

```

Evaluate
print(classification_report(y_test, y_pred_linear, target_names=['Non-Ia', 'Type Ia']))
print(f"Accuracy: {accuracy_score(y_test, y_pred_linear):.3f}")
Output:
Precision, recall, F1-score for each class.
Accuracy (~85-90% for synthetic data).
3.2 SVM with RBF Kernel (Non-linear Decision Boundary)

```

	precision	recall	f1-score	support
Non-Ia	0.99	1.00	1.00	102
Type Ia	1.00	0.99	0.99	98
accuracy			0.99	200
macro avg	1.00	0.99	0.99	200
weighted avg	1.00	0.99	0.99	200

Accuracy: 0.995

```

svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
y_pred_rbf = svm_rbf.predict(X_test_scaled)
print(classification_report(y_test, y_pred_rbf))
print(f"Accuracy: {accuracy_score(y_test, y_pred_rbf):.3f}")

Output:
Potentially better performance if data is non-linear.
4. Hyperparameter Tuning (Grid Search)
Optimize C (regularization) and gamma (RBF kernel influence).

```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	102
1	0.98	0.98	0.98	98
accuracy			0.98	200
macro avg	0.98	0.98	0.98	200
weighted avg	0.98	0.98	0.98	200

Accuracy: 0.980

```

param_grid = {
 'C': [0.1, 1, 10, 100],
 'gamma': ['scale', 'auto', 0.1, 1, 10],

```

```

 'kernel': ['rbf']
}

grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV accuracy: {grid_search.best_score_:.3f}")

Best model
best_svm = grid_search.best_estimator_
y_pred_best = best_svm.predict(X_test_scaled)
print(classification_report(y_test, y_pred_best))

Output:
Best hyperparameters (e.g., C=10, gamma=0.1).
Improved test accuracy (~90-95%).
5. Evaluation Metrics
5.1 Confusion Matrix

```

```

Best parameters: {'C': 0.1, 'gamma': 0.1, 'kernel': 'rbf'}
Best CV accuracy: 0.969

```

	precision	recall	f1-score	support
0	0.97	0.96	0.97	102
1	0.96	0.97	0.96	98
accuracy			0.96	200
macro avg	0.96	0.97	0.96	200
weighted avg	0.97	0.96	0.97	200

```

cm = confusion_matrix(y_test, y_pred_best)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-Ia', 'Type Ia'], yticklabels=
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()

```

<Figure size 640x480 with 2 Axes>

*# 5.2 ROC Curve (for Probability Estimates)*

```

y_probs = best_svm.decision_function(X_test_scaled) # Not probabilities, but scores
fpr, tpr, thresholds = roc_curve(y_test, y_probs)

```

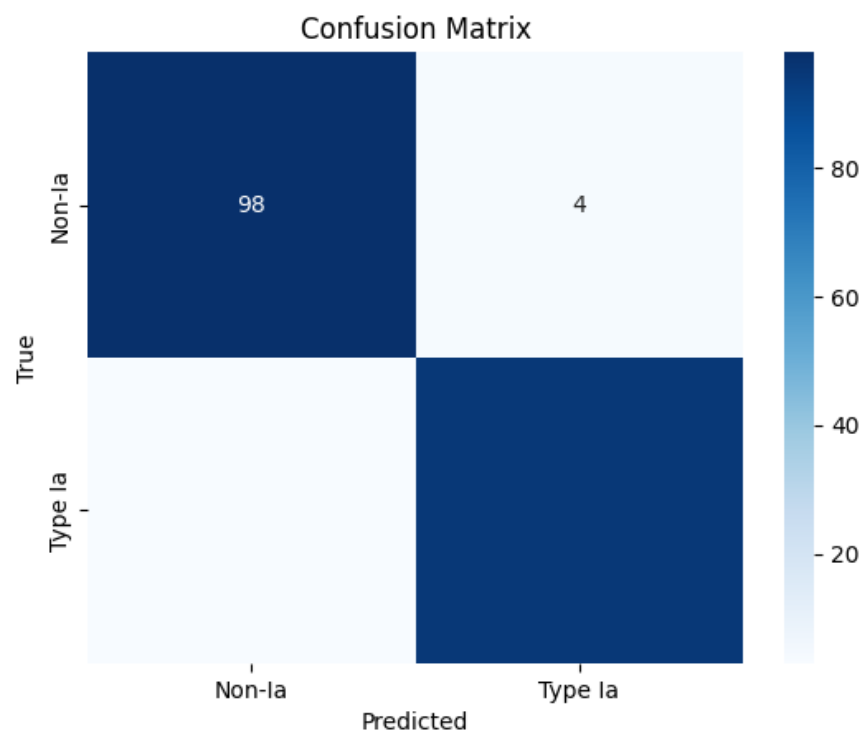


Figure 49.4: output

```
roc_auc = roc_auc_score(y_test, y_probs)
plt.plot(fpr, tpr, label=f'SVM (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()

Output:
AUC score (e.g., 0.95) indicating strong separability.
```

<Figure size 640x480 with 1 Axes>

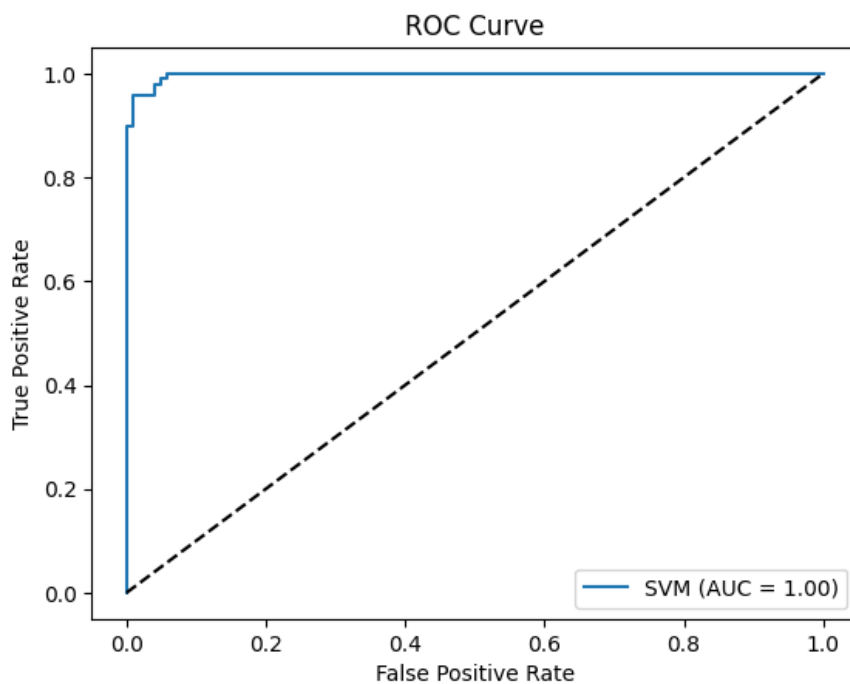


Figure 49.5: output

#### 49.2.4 6. Feature Importance (Linear SVM Coefficients)

```
if best_svm.kernel == 'linear':
 coef_df = pd.DataFrame({'feature': df.columns[:-1], 'weight': best_svm.coef_[0]})
 coef_df = coef_df.sort_values('weight', ascending=False)
 sns.barplot(data=coef_df, x='weight', y='feature')
 plt.title('Feature Importance (Linear SVM)')
 plt.show()
```



## Chapter 50

# Vizier Data Query

```
from astroquery.vizier import Vizier
from astropy.table import vstack

Vizier.ROW_LIMIT = 1000
Vizier.TIMEOUT = 120

cols = ['objID', 'RA_ICRS', 'DE_ICRS', 'class', 'umag',
 'gmag', 'rmag', 'imag', 'zmag', 'spCl', 'subCl', 'zph', 'MJD']

v = Vizier(columns=cols, catalog="V/154/sdss16", row_limit=150000)

Get objects by spectral class
galaxies = v.query_constraints(spCl="GALAXY")[0]
qsos = v.query_constraints(spCl="QSO")[0]
stars = v.query_constraints(spCl="STAR")[0]

Stack results
sds = vstack([galaxies, qsos, stars])
```

```
WARNING: MergeConflictWarning: Cannot merge meta key 'null' types <class 'float'> and <class 'flo
```

```
sds = vstack([galaxies, qsos, stars])
```

```
sds
```

```
<Table length=450000>
 objID RA_ICRS DE_ICRS ... subCl zph MJD
```

int64	deg float64	deg float64	...	...	str12	float32	d float64
-----	-----	-----	...	...	-----	-----	-----
1237663784217084463	44.991889	0.040779	...			0.3671	52963.2664
1237663784217084230	44.997801	0.063080	...	STARBURST		0.2365	52963.2664
1237663784217084376	44.955188	0.054225	...			0.3736	52963.2664
1237663784217084216	44.961076	0.077095	...	STARBURST		0.1349	52963.2664
1237663784217084075	44.931939	0.077110	...	STARBURST		0.1043	52963.2664
1237663784217084117	44.980259	0.093354	...			0.6221	52963.2664
1237663784217084424	44.975252	0.127604	...			0.4493	52963.2664
1237663784217149584	45.074930	0.110856	...	STARBURST		0.0534	52963.2668
1237663784217149588	45.090614	0.136589	...	STARFORMING		0.0927	52963.2668
...	...	...	...	...	...	...	...
1237654381983170598	155.742436	58.910619	...	K1		0.6326	51935.2739
1237653613189333263	155.550247	58.896320	...	M6		--	51875.3846
1237655369818505878	155.534143	58.946723	...	G4V		--	52047.1532
1237655369818636896	156.097029	59.085468	...	M3III		--	52047.1540
1237654381982908473	154.668480	58.592408	...	A4V		--	51935.2722
1237654381982908781	154.677939	58.649928	...	CV		--	51935.2722
1237655369818374163	154.818572	58.795788	...	F3/F5V		--	52047.1523
1237655369818439745	155.096163	58.812224	...	K5Ve		--	52047.1528
1237655369818439859	155.079645	58.858260	...	M7		--	52047.1528

```
sds.write('data/sdssdr16-class.csv', format='csv', overwrite=True)
```

## Chapter 51

# Classification: Learning Labels of Astronomical Sources

```
!mkdir -p data
!wget -P data https://raw.githubusercontent.com/Danselem/brics_astro/main/Week7/data/sdss_photoz.
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, LabelEncoder, label_binarize
from sklearn.model_selection import cross_val_score, cross_val_predict, train_test_split
from sklearn.metrics import (
 accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, roc_curve, auc,
 confusion_matrix, classification_report, RocCurveDisplay, ConfusionMatrixDisplay
)

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
```

```
spec_data = pd.read_csv('data/sdssdr16-class.csv')
```

```
spec_data.head()
```

	objID	RA_ICRS	DE_ICRS	class	umag	gmag	rmag	\
0	1237663784217084463	44.991889	0.040779	3	23.202	21.731	20.136	
1	1237663784217084230	44.997801	0.063080	3	21.739	20.905	20.276	
2	1237663784217084376	44.955188	0.054225	3	21.714	20.851	19.734	
3	1237663784217084216	44.961076	0.077095	3	21.143	20.027	19.504	
4	1237663784217084075	44.931939	0.077110	3	19.888	18.975	18.607	

	imag	zmag	spCl	subCl	zph	MJD
0	19.498	18.987	GALAXY	NaN	0.3671	52963.2664
1	19.973	19.640	GALAXY	STARBURST	0.2365	52963.2664
2	19.423	19.272	GALAXY	NaN	0.3736	52963.2664
3	19.059	19.022	GALAXY	STARBURST	0.1349	52963.2664
4	18.392	18.359	GALAXY	STARBURST	0.1043	52963.2664

```
spec_data.shape
```

```
(450000, 13)
```

```
spec_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 450000 entries, 0 to 449999
Data columns (total 13 columns):
Column Non-Null Count Dtype
--- -
0 objID 450000 non-null int64
1 RA_ICRS 450000 non-null float64
2 DE_ICRS 450000 non-null float64
3 class 450000 non-null int64
4 umag 450000 non-null float64
5 gmag 450000 non-null float64
6 rmag 450000 non-null float64
7 imag 449999 non-null float64
8 zmag 450000 non-null float64
9 spCl 450000 non-null object
10 subCl 263711 non-null object
11 zph 162516 non-null float64
12 MJD 450000 non-null float64
```

```
dtypes: float64(9), int64(2), object(2)
memory usage: 44.6+ MB
```

```
spec_data= spec_data[spec_data['zph'].notna()]
```

```
spec_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 162516 entries, 0 to 449991
Data columns (total 13 columns):
Column Non-Null Count Dtype
--- -
0 objID 162516 non-null int64
1 RA_ICRS 162516 non-null float64
2 DE_ICRS 162516 non-null float64
3 class 162516 non-null int64
4 umag 162516 non-null float64
5 gmag 162516 non-null float64
6 rmag 162516 non-null float64
7 imag 162516 non-null float64
8 zmag 162516 non-null float64
9 spCl 162516 non-null object
10 subCl 27049 non-null object
11 zph 162516 non-null float64
12 MJD 162516 non-null float64
dtypes: float64(9), int64(2), object(2)
memory usage: 17.4+ MB
```

```
spec_data["spCl"].value_counts()
```

```
spCl
GALAXY 140323
QSO 13439
STAR 8754
Name: count, dtype: int64
```

```
sns.displot(spec_data["spCl"])
plt.show()
```

```
<Figure size 500x500 with 1 Axes>
```

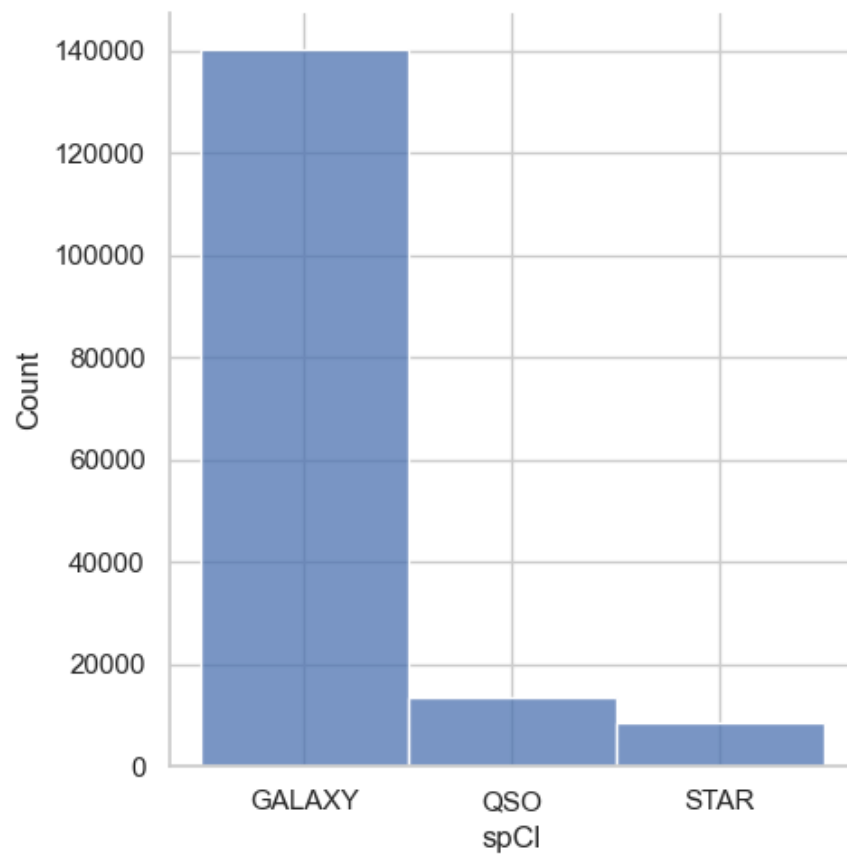


Figure 51.1: output

```
spec_data['u-g'] = spec_data['umag'] - spec_data['gmag']
spec_data['g-r'] = spec_data['gmag'] - spec_data['rmag']
spec_data['r-i'] = spec_data['rmag'] - spec_data['imag']
spec_data['i-z'] = spec_data['imag'] - spec_data['zmag']
```

```
spec_data.head()
```

	objID	RA_ICRS	DE_ICRS	class	umag	gmag	rmag	\
0	1237663784217084463	44.991889	0.040779	3	23.202	21.731	20.136	
1	1237663784217084230	44.997801	0.063080	3	21.739	20.905	20.276	
2	1237663784217084376	44.955188	0.054225	3	21.714	20.851	19.734	
3	1237663784217084216	44.961076	0.077095	3	21.143	20.027	19.504	
4	1237663784217084075	44.931939	0.077110	3	19.888	18.975	18.607	

	imag	zmag	spCl	subCl	zph	MJD	u-g	g-r	r-i	\
0	19.498	18.987	GALAXY	NaN	0.3671	52963.2664	1.471	1.595	0.638	
1	19.973	19.640	GALAXY	STARBURST	0.2365	52963.2664	0.834	0.629	0.303	
2	19.423	19.272	GALAXY	NaN	0.3736	52963.2664	0.863	1.117	0.311	
3	19.059	19.022	GALAXY	STARBURST	0.1349	52963.2664	1.116	0.523	0.445	
4	18.392	18.359	GALAXY	STARBURST	0.1043	52963.2664	0.913	0.368	0.215	

	i-z
0	0.511
1	0.333
2	0.151
3	0.037
4	0.033

```
label_encoder = LabelEncoder()
spec_data['label'] = label_encoder.fit_transform(spec_data['spCl'])

print("Class Labels:", dict(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_))))
```

```
Class Labels: {'GALAXY': np.int64(0), 'QSO': np.int64(1), 'STAR': np.int64(2)}
```

```
spec_data['label'] = spec_data['label'].astype('Int64')
spec_data.describe()
```

	objID	RA_ICRS	DE_ICRS	class	umag	\
count	1.625160e+05	162516.000000	162516.000000	162516.0	162516.000000	
mean	1.237670e+18	92.379428	19.879098	3.0	22.898095	
std	8.059387e+12	57.331574	10.623694	0.0	2.175848	

min	1.237646e+18	5.304001	0.040779	3.0	10.542000
25%	1.237665e+18	26.729008	12.229291	3.0	21.642000
50%	1.237668e+18	130.348461	18.527169	3.0	23.067000
75%	1.237679e+18	141.386179	27.694916	3.0	24.439000
max	1.237681e+18	179.996562	72.357177	3.0	29.899000

	gmag	rmag	imag	zmag	\
count	162516.000000	162516.000000	162516.000000	162516.000000	
mean	21.217975	19.867106	19.080326	18.653949	
std	2.011545	1.842298	1.665427	1.638262	
min	12.637000	11.066000	10.704000	11.307000	
25%	19.832000	18.395000	17.814000	17.447000	
50%	21.857000	20.346000	19.410000	18.932000	
75%	22.628000	21.256000	20.164000	19.628000	
max	28.927000	30.334000	27.763000	27.587000	

	zph	MJD	u-g	g-r	\
count	162516.000000	162516.000000	162516.000000	162516.000000	
mean	0.431925	53737.625343	1.680120	1.350869	
std	0.217994	979.526571	1.315391	0.596457	
min	0.000700	51078.414400	-9.476000	-7.747000	
25%	0.247500	53081.306100	0.826000	0.933000	
50%	0.477700	53500.182100	1.646000	1.421000	
75%	0.583400	54770.396800	2.320000	1.745000	
max	1.000000	55152.319500	12.480000	11.019000	

	r-i	i-z	label
count	162516.000000	162516.000000	162516.0
mean	0.786779	0.426377	0.190424
std	0.368562	0.309471	0.511757
min	-12.301000	-6.251000	0.0
25%	0.514000	0.325000	0.0
50%	0.789000	0.412000	0.0
75%	1.038000	0.548000	0.0
max	8.878000	10.754000	2.0

```
spec_data.label.value_counts()
```

```
label
0 140323
1 13439
2 8754
Name: count, dtype: Int64
```



```

Set Seaborn theme
sns.set(style="whitegrid")

Create subplots
fig, axes = plt.subplots(3, 1, figsize=(12, 14))
fig.suptitle('SDSS Photometry Distributions', fontsize=18, y=1)

1. Magnitudes
mags = ['umag', 'gmag', 'rmag', 'imag', 'zmag']
for mag in mags:
 sns.kdeplot(spec_data[mag], label=mag, ax=axes[0], fill=True, alpha=0.5)
axes[0].set_title('Magnitude Distributions')
axes[0].set_xlabel('Magnitude')
axes[0].legend()

2. Colors
colors = ['u-g', 'g-r', 'r-i', 'i-z']
for color in colors:
 sns.kdeplot(spec_data[color], label=color, ax=axes[1], fill=True, alpha=0.5)
axes[1].set_title('Color Index Distributions')
axes[1].set_xlabel('Color')
axes[1].legend()

3. Redshift
sns.histplot(spec_data['zph'], bins=100, kde=True, ax=axes[2], color='purple')
axes[2].set_title('Redshift Distribution')
axes[2].set_xlabel('Redshift')

plt.tight_layout()
plt.show()

```

<Figure size 1200x1400 with 3 Axes>

```

features= ['umag', 'gmag', 'rmag', 'imag', 'zmag', 'u-g', 'g-r', 'r-i', 'i-z', 'zph']
label = 'label'

```

```

plt.figure(figsize=(10, 10))
sns.heatmap(spec_data[features].corr(), annot = True, fmt = ".2f", linewidths = .5, cmap='plasma')
plt.show()

```

<Figure size 1000x1000 with 2 Axes>

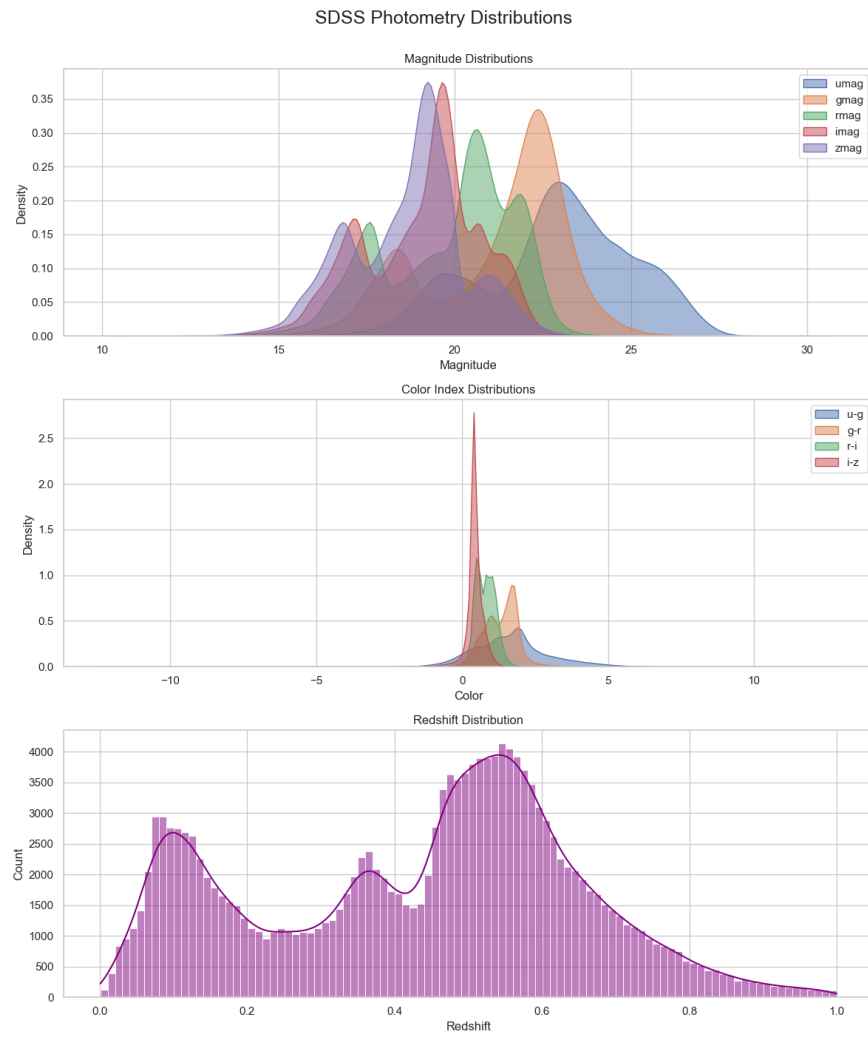


Figure 51.2: output

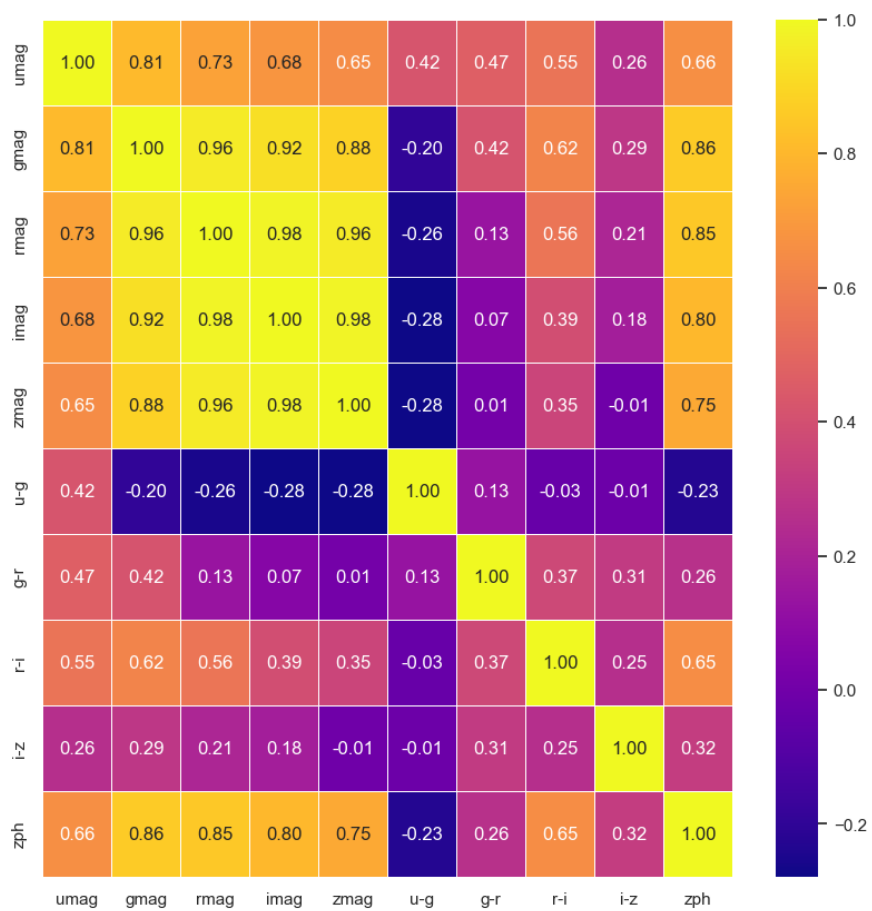


Figure 51.3: output

```
X = spec_data[features]
y = spec_data[label]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, stratify=y,
```

```
X_train
```

	umag	gmag	rmag	imag	zmag	u-g	g-r	r-i	i-z	\
128605	22.502	22.075	20.475	19.481	18.873	0.427	1.600	0.994	0.608	
29000	25.714	22.227	22.076	21.144	21.090	3.487	0.151	0.932	0.054	
398002	21.876	19.178	17.735	17.141	16.776	2.698	1.443	0.594	0.365	
51715	22.927	20.316	18.632	17.965	17.553	2.611	1.684	0.667	0.412	
15796	23.577	22.545	20.493	19.547	19.026	1.032	2.052	0.946	0.521	
...	...	...	...	...	...	...	...	...	...	
431355	26.231	25.194	22.580	21.570	19.950	1.037	2.614	1.010	1.620	
18080	25.338	22.649	20.950	19.794	19.464	2.689	1.699	1.156	0.330	
26100	22.151	21.877	21.861	21.337	22.042	0.274	0.016	0.524	-0.705	
922	20.002	18.743	18.074	17.677	17.412	1.259	0.669	0.397	0.265	
132262	25.746	23.952	22.185	22.013	21.511	1.794	1.767	0.172	0.502	

	zph
128605	0.5481
29000	0.3067
398002	0.2316
51715	0.3129
15796	0.5149
...	...
431355	0.4836
18080	0.5717
26100	0.2669
922	0.1059
132262	0.2985

```
[130012 rows x 10 columns]
```

```
def evaluate_pipeline(model, name, X, y, feature_names=None, cv=5):
 print(f"\n{name}")
 print("-" * len(name))

 # Build pipeline
 pipeline = Pipeline([
 ('scaler', StandardScaler()),
 ('clf', model)
])
```

```

Cross-validated predictions and probabilities
y_pred = cross_val_predict(pipeline, X, y, cv=cv, method='predict')
pipeline.fit(X, y)

print(f"Accuracy (CV): {accuracy_score(y, y_pred):.4f}")
print(f"Precision: {precision_score(y, y_pred, average='weighted'):.4f}")
print(f"Recall: {recall_score(y, y_pred, average='weighted'):.4f}")
print(f"F1 Score: {f1_score(y, y_pred, average='weighted'):.4f}")

print("\nClassification Report:")
print(classification_report(y, y_pred))

Confusion matrix
cm = confusion_matrix(y, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title(f'{name} - Confusion Matrix')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

Feature importances for tree-based models
if hasattr(model, "feature_importances_") and feature_names is not None:
 importances = model.feature_importances_
 indices = np.argsort(importances)[::-1]
 plt.figure(figsize=(8, 5))
 sns.barplot(x=importances[indices], y=np.array(feature_names)[indices])
 plt.title(f'{name} - Feature Importance')
 plt.xlabel("Importance")
 plt.tight_layout()
 plt.show()

Multiclass ROC AUC
if hasattr(model, "predict_proba"):
 y_proba = cross_val_predict(pipeline, X, y, cv=cv, method="predict_proba")
 class_labels = np.unique(y)
 y_bin = label_binarize(y, classes=class_labels)

 roc_auc = roc_auc_score(y_bin, y_proba, average="weighted", multi_class="ovr")
 print(f"Multiclass ROC AUC (OvR, weighted): {roc_auc:.4f}")

Plot ROC curves for each class
fpr = dict()
tpr = dict()

```

```

roc_auc_dict = dict()

plt.figure(figsize=(8, 6))
for i, label in enumerate(class_labels):
 fpr[i], tpr[i], _ = roc_curve(y_bin[:, i], y_proba[:, i])
 roc_auc_dict[i] = auc(fpr[i], tpr[i])
 plt.plot(fpr[i], tpr[i], label=f"Class {label} (AUC = {roc_auc_dict[i]:.2f}")

plt.plot([0, 1], [0, 1], 'k--')
plt.title(f'{name} - Multiclass ROC Curve')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.show()

```

```

classifiers = [
 (RandomForestClassifier(random_state=42), "Random Forest"),
 (LogisticRegression(max_iter=1000), "Logistic Regression"),
 (DecisionTreeClassifier(random_state=42), "Decision Tree"),
 (KNeighborsClassifier(), "KNN Classifier"),
 (SVC(probability=True), "Support Vector Classifier")
]

for model, name in classifiers:
 evaluate_pipeline(model, name, X, y, feature_names=X.columns)

```

Random Forest

-----

Accuracy (CV): 0.8619

Precision: 0.8342

Recall: 0.8619

F1 Score: 0.8409

Classification Report:

	precision	recall	f1-score	support
0.0	0.89	0.96	0.92	140323
1.0	0.35	0.18	0.23	13439
2.0	0.69	0.32	0.44	8754
accuracy			0.86	162516
macro avg	0.64	0.49	0.53	162516

```
weighted avg 0.83 0.86 0.84 162516
```

```
<Figure size 600x500 with 2 Axes>
```

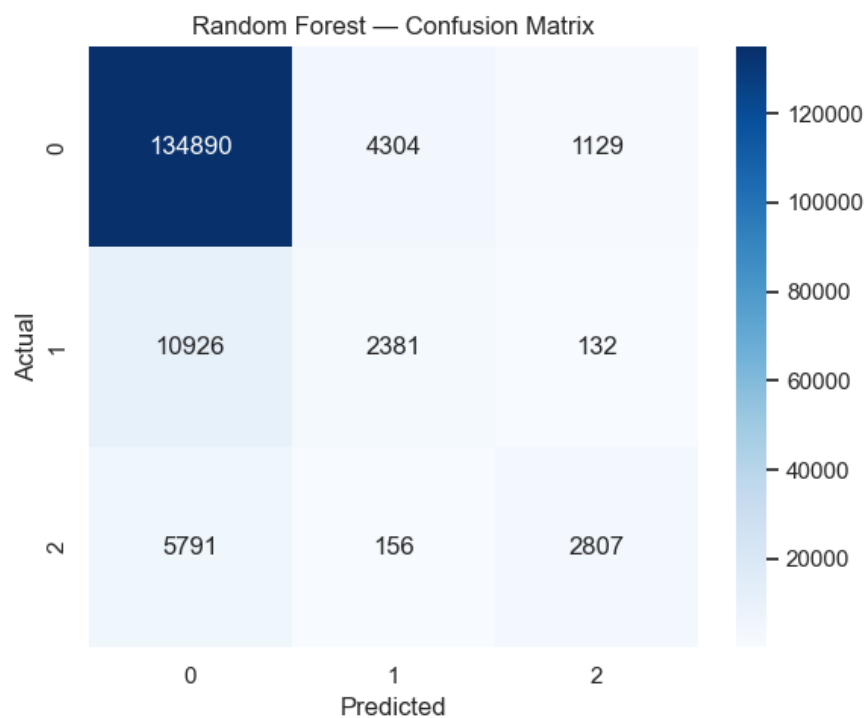


Figure 51.4: output

```
<Figure size 800x500 with 1 Axes>
```

```
Multiclass ROC AUC (OvR, weighted): 0.8236
```

```
<Figure size 800x600 with 1 Axes>
```

```
Logistic Regression
```

```

Accuracy (CV): 0.8480
Precision: 0.7766
```

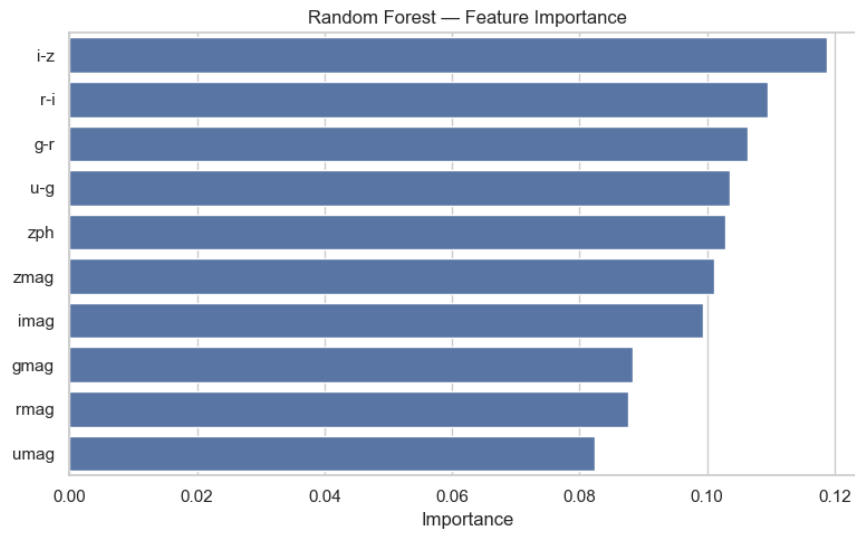


Figure 51.5: output

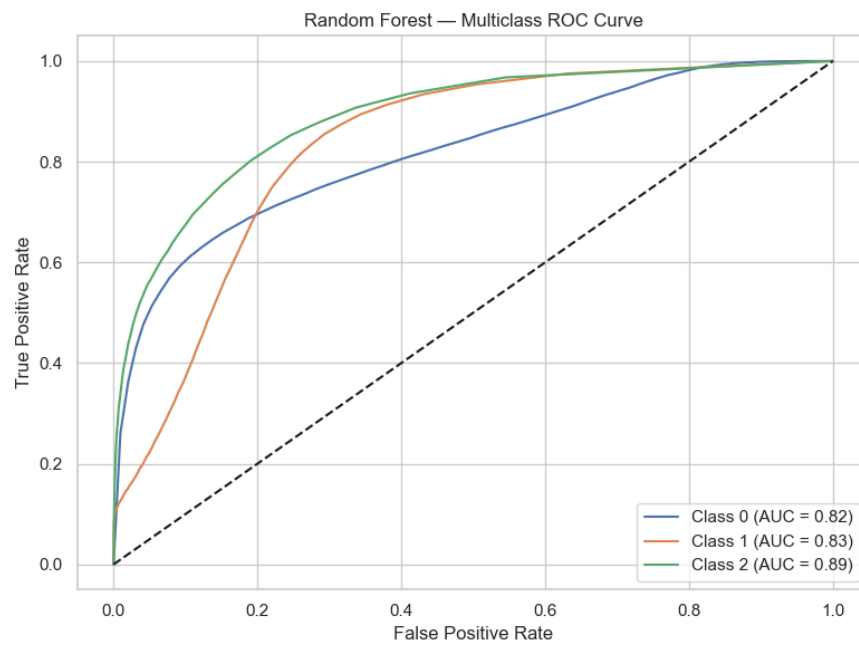


Figure 51.6: output



Recall: 0.8480  
F1 Score: 0.7983

Classification Report:

	precision	recall	f1-score	support
0.0	0.86	0.98	0.92	140323
1.0	0.12	0.03	0.05	13439
2.0	0.38	0.02	0.04	8754
accuracy			0.85	162516
macro avg	0.45	0.34	0.33	162516
weighted avg	0.78	0.85	0.80	162516

<Figure size 600x500 with 2 Axes>

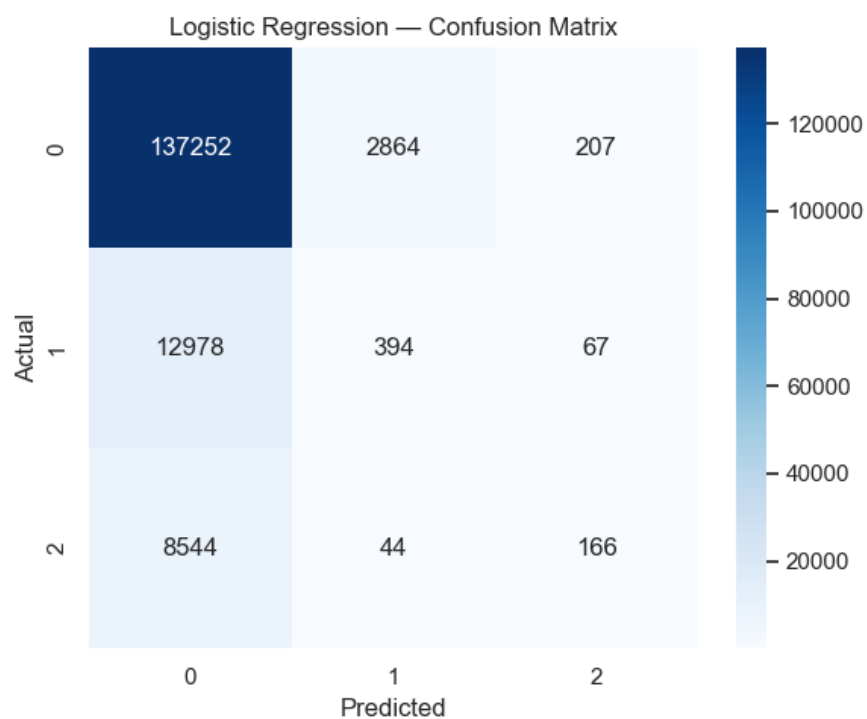


Figure 51.7: output

Multiclass ROC AUC (OvR, weighted): 0.7142

<Figure size 800x600 with 1 Axes>

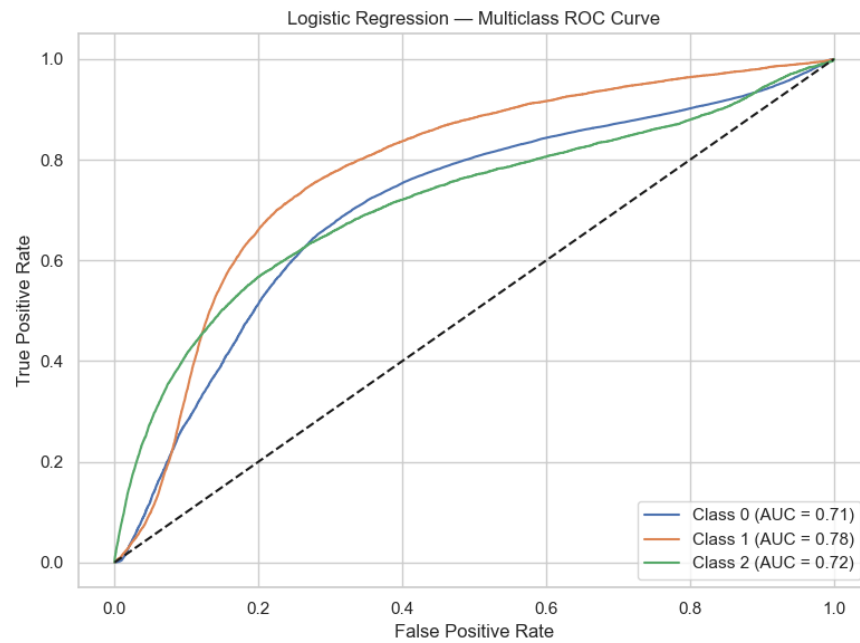


Figure 51.8: output

#### Decision Tree

Accuracy (CV): 0.8001

Precision: 0.8151

Recall: 0.8001

F1 Score: 0.8072

#### Classification Report:

	precision	recall	f1-score	support
0.0	0.90	0.88	0.89	140323
1.0	0.25	0.31	0.28	13439
2.0	0.33	0.36	0.34	8754
accuracy			0.80	162516

macro avg	0.49	0.51	0.50	162516
weighted avg	0.82	0.80	0.81	162516

<Figure size 600x500 with 2 Axes>

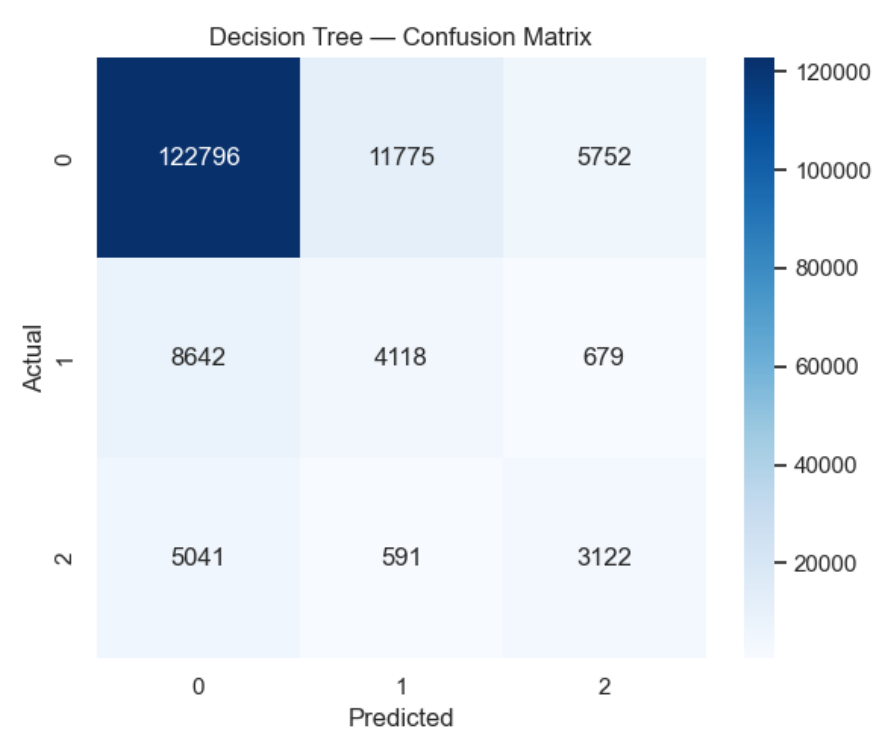


Figure 51.9: output

<Figure size 800x500 with 1 Axes>

Multiclass ROC AUC (OvR, weighted): 0.6293

<Figure size 800x600 with 1 Axes>

KNN Classifier  
-----  
Accuracy (CV): 0.8525

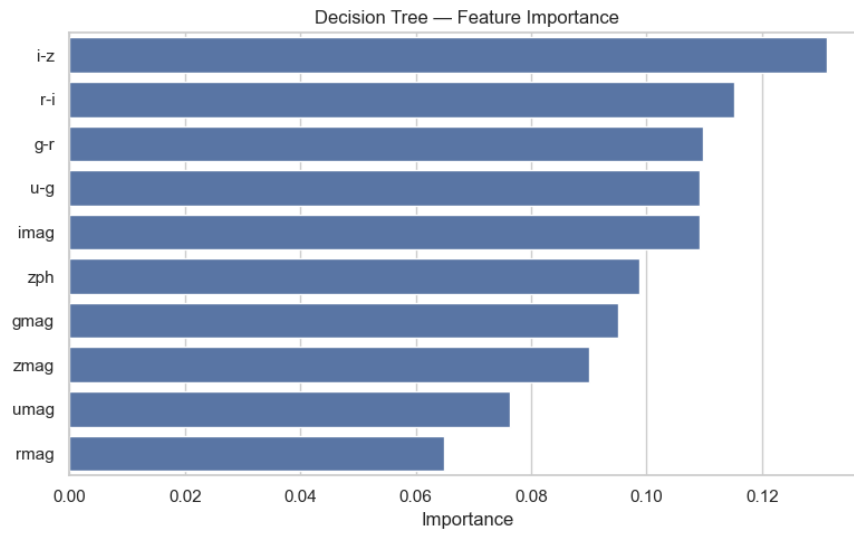


Figure 51.10: output

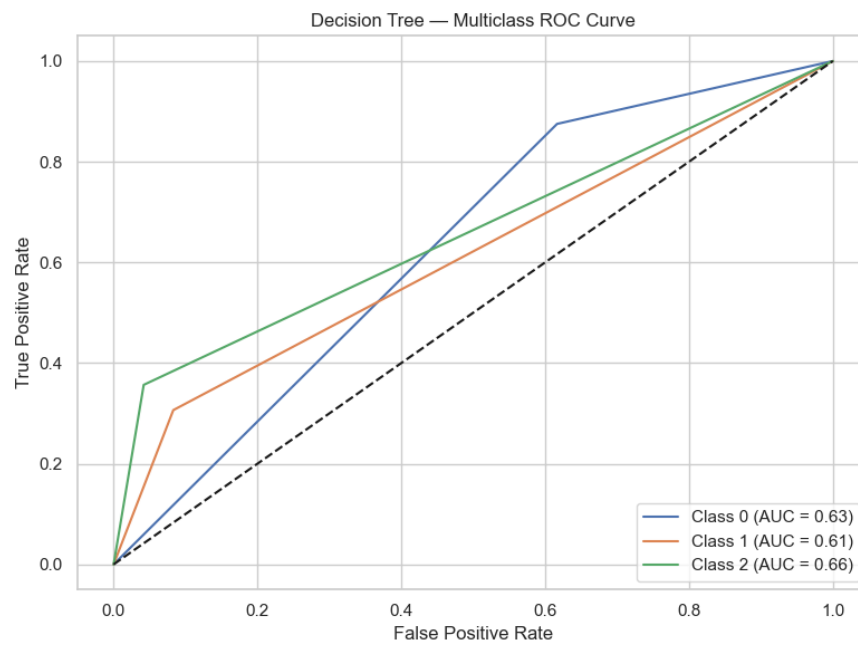


Figure 51.11: output

```
Precision: 0.8283
Recall: 0.8525
F1 Score: 0.8350
```

#### Classification Report:

	precision	recall	f1-score	support
0.0	0.89	0.95	0.92	140323
1.0	0.31	0.20	0.25	13439
2.0	0.64	0.29	0.40	8754
accuracy			0.85	162516
macro avg	0.61	0.48	0.52	162516
weighted avg	0.83	0.85	0.84	162516

<Figure size 600x500 with 2 Axes>

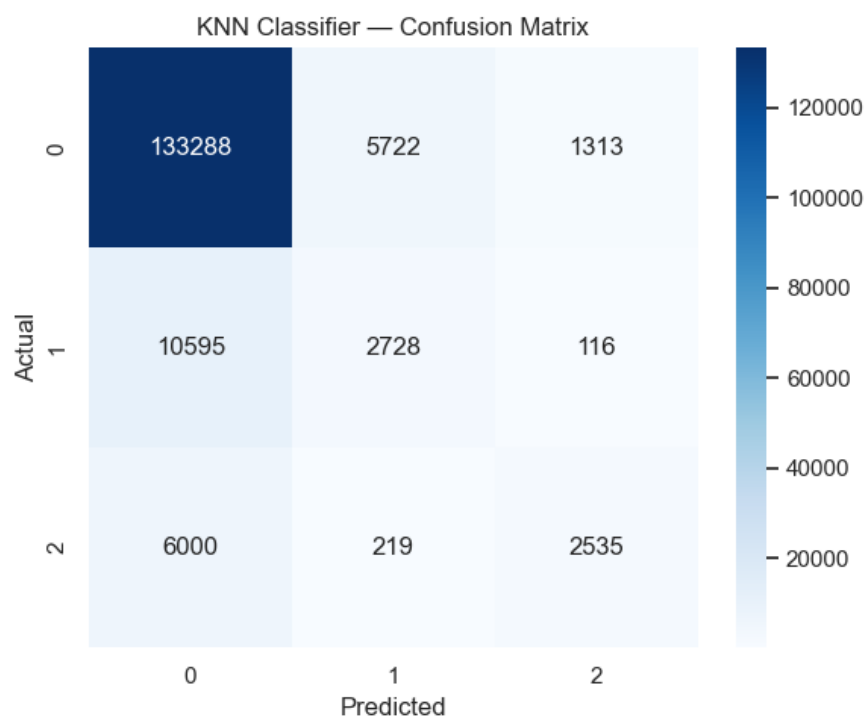


Figure 51.12: output

Multiclass ROC AUC (OvR, weighted): 0.7397

<Figure size 800x600 with 1 Axes>

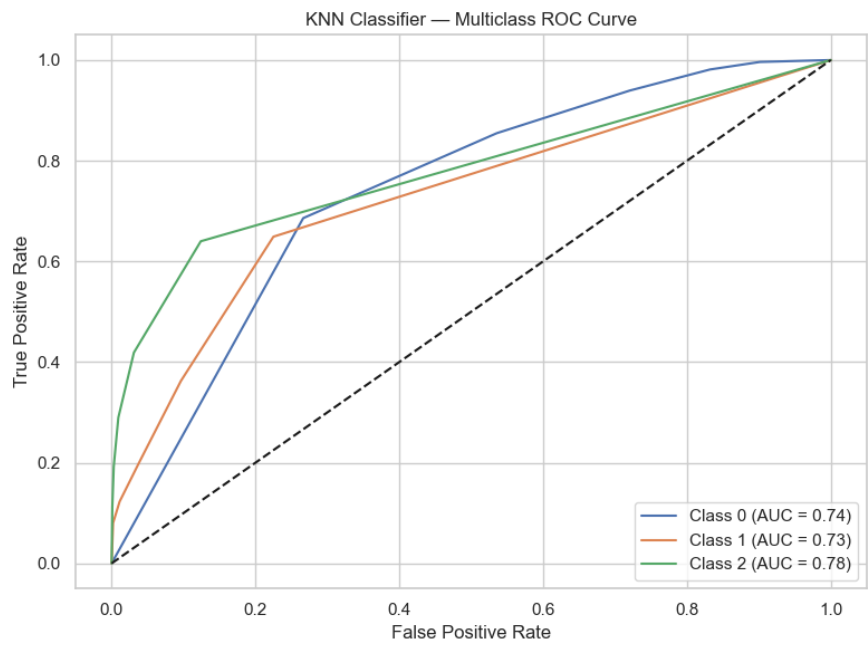


Figure 51.13: output

Support Vector Classifier

## Chapter 52

# Astroml Dimensionality Reduction

### 52.1 Dimensionality reduction

#### ▼ Get Started



#### 52.1.1 The curse of dimensionality

Fitting and overfitting get worse with ‘curse of dimensionality’ Bellman 1961

Think about a hypersphere. Its volume is given by

$$V_D(r) = \frac{2r^D \pi^{D/2}}{D \Gamma(D/2)}$$

where  $\Gamma(z)$  is the complete gamma function,  $D$  is the dimension, and  $r$  the radius of the sphere.

If you populated a hypercube of size  $2r$  how much data would be enclosed by the hypersphere - as  $D$  increases the fractional volume enclosed by the hypersphere goes to 0!

For example: the SDSS comprises a sample of 357 million sources. - each source has 448 measured attributes - selecting just 30 (e.g., magnitude, size..) and normalizing the data range  $-1$  to  $1$

probability of having one of the 357 million sources reside within a unit hypersphere 1 in  $1.4 \times 10^5$ .

### 52.1.2 Principal Component Analysis

Points are correlated along a particular direction which doesn't align with the initial choice of axes. \* we should rotate our axes to align with this correlation.  
\* rotation preserves the relative ordering of data

Choose rotation to maximize the ability to discriminate between the data points  
\* first axis, or principal component, is direction of maximal variance \* second principal component is orthogonal to the first component and maximizes the residual variance \* ...

In the following example a distribution of points drawn from a bivariate Gaussian and centered on the origin of  $x$  and  $y$ . PCA defines a rotation such that the new axes ( $x'$  and  $y'$ ) are aligned along the directions of maximal variance (the principal components) with zero covariance. This is equivalent to minimizing the square of the perpendicular distances between the points and the principal components.

```
import numpy as np
from matplotlib import pyplot as plt
from matplotlib.patches import Ellipse
from matplotlib import ticker

np.random.seed(42)
r = 0.9

sigma1 = 0.25
sigma2 = 0.08
rotation = np.pi / 6
s = np.sin(rotation)
c = np.cos(rotation)

X = np.random.normal(0, [sigma1, sigma2], size=(100, 2)).T
R = np.array([[c, -s],
 [s, c]])
X = np.dot(R, X)
```



```

#-----
Plot the diagram
fig = plt.figure(figsize=(5, 5), facecolor='w')
ax = plt.axes((0, 0, 1, 1), xticks=[], yticks=[], frameon=False)

draw axes
ax.annotate(r'x', (-r, 0), (r, 0),
 ha='center', va='center',
 arrowprops=dict(arrowstyle='<->', color='k', lw=1))
ax.annotate(r'y', (0, -r), (0, r),
 ha='center', va='center',
 arrowprops=dict(arrowstyle='<->', color='k', lw=1))

draw rotated axes
ax.annotate(r'$x^\prime$', (-r * c, -r * s), (r * c, r * s),
 ha='center', va='center',
 arrowprops=dict(color='k', arrowstyle='<->', lw=1))
ax.annotate(r'$y^\prime$', (r * s, -r * c), (-r * s, r * c),
 ha='center', va='center',
 arrowprops=dict(color='k', arrowstyle='<->', lw=1))

scatter points
ax.scatter(X[0], X[1], s=25, lw=0, c='k', zorder=2)

draw lines
vnorm = np.array([s, -c])
for v in (X.T):
 d = np.dot(v, vnorm)
 v1 = v - d * vnorm
 ax.plot([v[0], v1[0]], [v[1], v1[1]], '-k')

draw ellipses
for sigma in (1, 2, 3):
 ax.add_patch(Ellipse((0, 0), 2 * sigma * sigma1, 2 * sigma * sigma2,
 rotation * 180. / np.pi,
 ec='k', fc='gray', alpha=0.2, zorder=1))

ax.set_xlim(-1, 1)
ax.set_ylim(-1, 1)

plt.show()

```

<Figure size 360x360 with 1 Axes>

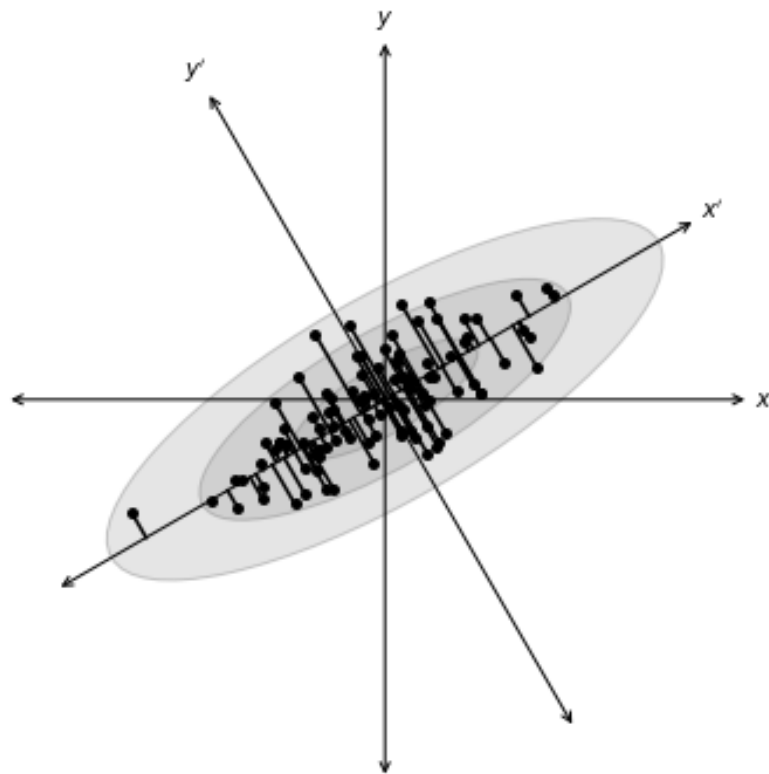


Figure 52.1: output

**52.1.2.1 Derivation of principal component analyses**

Set of data  $X$ :  $N$  observations by  $K$  measurements

Center data by subtracting the mean

The covariance is

$$C_X = \frac{1}{N-1} X^T X,$$

$N-1$  as the sample covariance matrix.

We want a projection,  $R$ , aligned with the directions of maximal variance ( $Y = XR$ ) with covariance

$$C_Y = R^T X^T X R = R^T C_X R$$

Derive principal component by maximizing its variance (using Lagrange multipliers and constraint)

$$\phi(r_1, \lambda_1) = r_1^T C_X r_1 - \lambda_1 (r_1^T r_1 - 1)$$

derivative of  $\phi(r_1, \lambda)$  with respect to  $r_1$  set to 0

$$C_X r_1 - \lambda_1 r_1 = 0$$

$\lambda_1$  is the root of the equation  $\det(C_X - \lambda_1 \mathbf{I}) = 0$  and the largest eigenvalue

$$\lambda_1 = r_1^T C_X r_1$$

Other principal components derived by applying additional constraint that components are uncorrelated (e.g.,  $r_2^T C_X r_1 = 0$ ).

**52.1.2.2 Singular value decomposition (SVD)**

Common approach is eigenvalue decomposition of the covariance or correlation matrix, or singular value decomposition (SVD) of the data matrix

$$U \Sigma V^T = \frac{1}{\sqrt{N-1}} X$$

columns of  $U$  are *left-singular vectors*

columns of  $V$  are the *right-singular vectors*

The columns of  $U$  and  $V$  form orthonormal bases ( $U^T U = V^T V = I$ )  
Covariance matrix is

$$C_X = \left[ \frac{1}{\sqrt{N-1}} X \right]^T \left[ \frac{1}{\sqrt{N-1}} X \right] = V \Sigma U^T U \Sigma V^T = V \Sigma^2 V^T.$$

right singular vectors  $V$  are the principal components so principal from the SVD of  $X$  dont need  $C_X$ .

### 52.1.2.3 Application of PCA

In the following example we have 100 data points in 3 dimensions,  $X$ , and  $R$  as the projection matrix. To compute the PCA components (4 in our case) we use PCA from `scikit-learn`.

```
from sklearn.decomposition import PCA

X = np.random.normal(size=(100, 3))
R = np.random.random((3, 10))
X = np.dot(X, R)

pca = PCA(n_components=4)
pca.fit(X)
comp = pca.transform(X)

mean = pca.mean_
components = pca.components_
var = pca.explained_variance_
```

For our astronomy use case, we are using the SDSS spectroscopic dataset. The SDSS spectra come from galaxies at a range of redshifts, and have sections of unreliable or missing data due to sky absorption, cosmic rays, bad detector pixels, or other effects. AstroML provides a set of spectra which have been moved to rest frame, corrected for masking using an iterative PCA reconstruction technique, and resampled to 1000 common wavelength bins.

The spectra can be downloaded using `fetch_sdss_corrected_spectra()`. In the following example we plot 15 of these spectra:

```
import numpy as np
from matplotlib import pyplot as plt
from astroML.datasets import sdss_corrected_spectra

#-----
```

```

Use pre-computed PCA to reconstruct spectra
data = sdss_corrected_spectra.fetch_sdss_corrected_spectra()
spectra_raw = data['spectra']
spectra_corr = sdss_corrected_spectra.reconstruct_spectra(data)
wavelengths = sdss_corrected_spectra.compute_wavelengths(data)

#-----
select random spectra
np.random.seed(5)
nrows = 5
ncols = 3
ind = np.random.randint(spectra_corr.shape[0], size=nrows * ncols)
spec_sample_raw = spectra_raw[ind]
spec_sample_corr = spectra_corr[ind]

```

We show the SDSS downloaded spectra with black, and our corrected spectra with blue.

```

fig = plt.figure(figsize=(10, 8))

fig.subplots_adjust(left=0.05, right=0.95, wspace=0.05,
 bottom=0.1, top=0.95, hspace=0.05)

for i in range(ncols):
 for j in range(nrows):
 ax = fig.add_subplot(nrows, ncols, ncols * j + 1 + i)
 ax.plot(wavelengths, spec_sample_raw[ncols * j + i], '-k', lw=1)
 ax.plot(wavelengths, spec_sample_corr[ncols * j + i], '-k', lw=1, c='blue')
 ax.set_xlim(3100, 7999)

 ax.yaxis.set_major_formatter(plt.NullFormatter())
 ax.xaxis.set_major_locator(plt.MultipleLocator(1000))
 if j < nrows - 1:
 ax.xaxis.set_major_formatter(plt.NullFormatter())
 else:
 plt.xlabel(r'wavelength $(\AA)$')

 ylim = ax.get_ylim()
 dy = 0.05 * (ylim[1] - ylim[0])
 ax.set_ylim(ylim[0] - dy, ylim[1] + dy)

plt.show()

```

<Figure size 720x576 with 15 Axes>

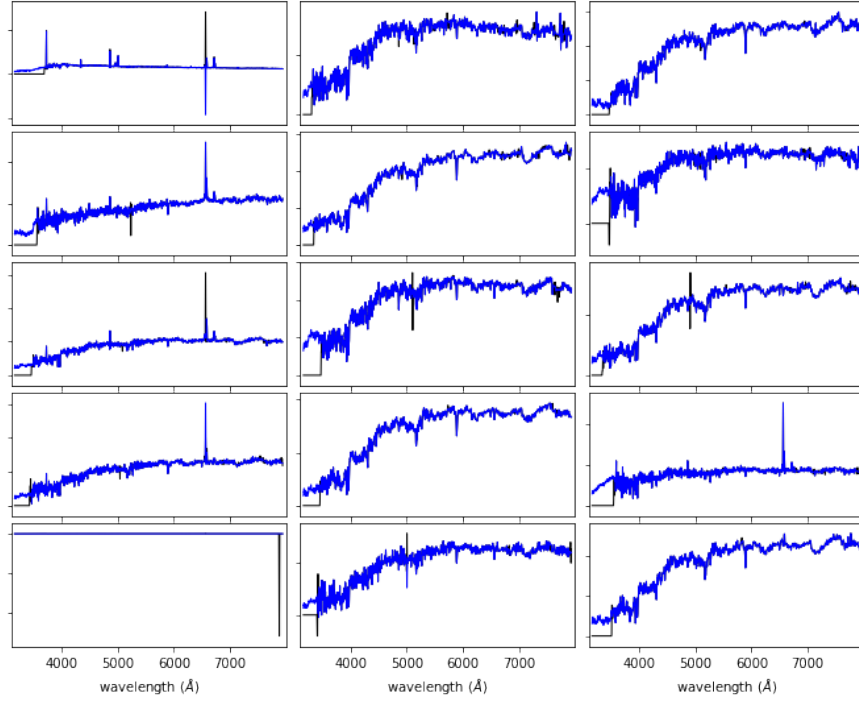


Figure 52.2: output

### 52.1.3 PCA Reconstruction of a spectrum

Reconstruction of spectrum,  $x(k)$ , from the eigenvectors,  $e_i(k)$

$$x_i(k) = \mu(k) + \sum_j^R \theta_{ij} e_j(k),$$

Truncating this expansion (i.e.,  $r < R$ )

$$x_i(k) = \mu(k) + \sum_i^{r < R} \theta_i e_i(k),$$

- eigenvectors ordered by their associated eigenvalues

- eigenvalues reflect variance within each eigenvector (sum of the eigenvalues is total variance of the system).
- project a each spectrum onto these first few eigenspectra is a compression of the data

This is the sense in which PCA gives for dimensionality reduction.

In the following example, the input spectrum is shown in gray, and the partial reconstruction for progressively more terms is shown in black. The top panel shows only the mean of the set of spectra. By the time 20 PCA components are added, the reconstruction is very close to the input, as indicated by the expected total variance of 94%.

```
#-----
Compute PCA components

Eigenvalues can be computed using PCA as in the commented code below:

#from sklearn.decomposition import PCA
#pca = PCA()
#pca.fit(spectra_corr)
#evals = pca.explained_variance_ratio_
#evals_cs = evals.cumsum()

because the spectra have been reconstructed from masked values, this
is not exactly correct in this case: we'll use the values computed
in the file compute_sdss_pca.py
evals = data['evals'] ** 2
evals_cs = evals.cumsum()
evals_cs /= evals_cs[-1]
evecs = data['evecs']
spec_mean = spectra_corr.mean(0)

#-----
Find the coefficients of a particular spectrum
spec = spectra_corr[1]
coeff = np.dot(evecs, spec - spec_mean)

#-----
Plot the sequence of reconstructions
fig = plt.figure(figsize=(5, 5))
fig.subplots_adjust(hspace=0, top=0.95, bottom=0.1, left=0.12, right=0.93)

for i, n in enumerate([0, 4, 8, 20]):
 ax = fig.add_subplot(411 + i)
 ax.plot(wavelengths, spec, '-', c='gray')
```

```

ax.plot(wavelengths, spec_mean + np.dot(coeff[:n], evecs[:n]), '-k')

if i < 3:
 ax.xaxis.set_major_formatter(plt.NullFormatter())

ax.set_ylim(-2, 21)
ax.set_ylabel('flux')

if n == 0:
 text = "mean"
elif n == 1:
 text = "mean + 1 component\n"
 text += r"$(\sigma^2_{\text{tot}} = %.2f)$" % evals_cs[n - 1]
else:
 text = "mean + %i components\n" % n
 text += r"$(\sigma^2_{\text{tot}} = %.2f)$" % evals_cs[n - 1]

ax.text(0.02, 0.93, text, ha='left', va='top', transform=ax.transAxes)

fig.axes[-1].set_xlabel(r"${\rm wavelength} \ (\AA)$')
plt.show()

```

<Figure size 360x360 with 4 Axes>

### 52.1.4 Eigenvalues

The eigenvalues for the PCA decomposition of the SDSS spectra described in the previous section. The top panel shows the decrease in eigenvalue as a function of the number of eigenvectors, with a break in the distribution at ten eigenvectors. The lower panel shows the cumulative sum of eigenvalues normalized to unity. 94% of the variance in the SDSS spectra can be captured using the first ten eigenvectors.

```

fig = plt.figure(figsize=(10, 7.5))
fig.subplots_adjust(hspace=0.05, bottom=0.12)

ax = fig.add_subplot(211, xscale='log', yscale='log')
ax.grid()
ax.plot(evals, c='k')
ax.set_ylabel('Normalized Eigenvalues')
ax.xaxis.set_major_formatter(plt.NullFormatter())
ax.set_ylim(5E-4, 100)

```



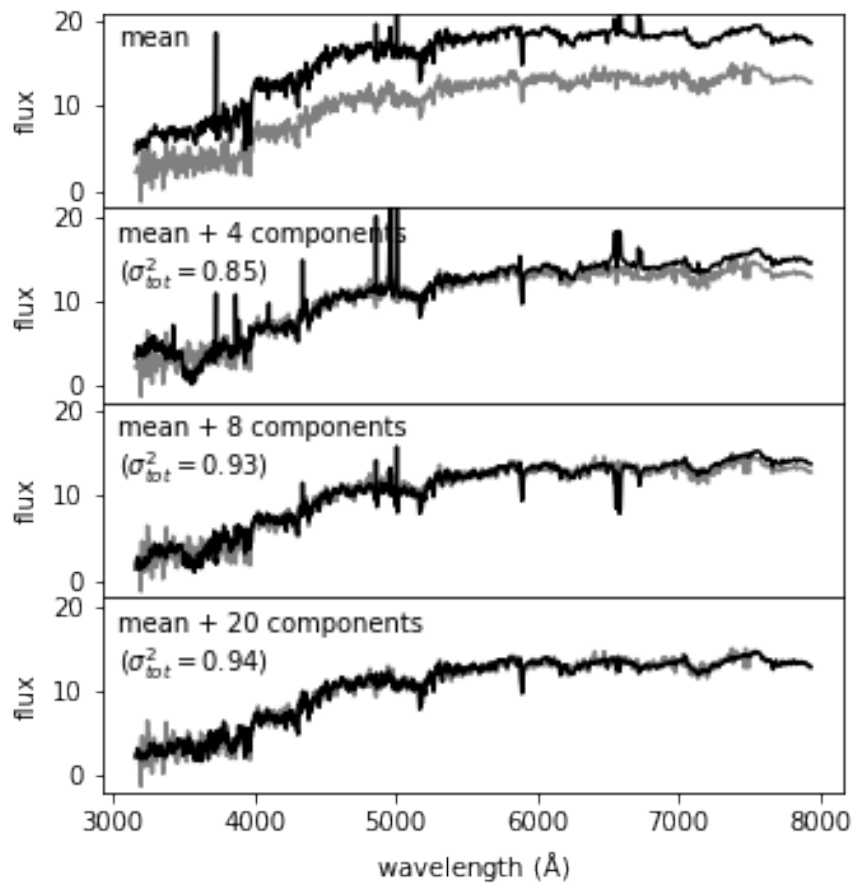


Figure 52.3: output

```

ax = fig.add_subplot(212, xscale='log')
ax.grid()
ax.semilogx(evals_cs, color='k')
ax.set_xlabel('Eigenvalue Number')
ax.set_ylabel('Cumulative Eigenvalues')
ax.set_ylim(0.65, 1.00)

plt.show()

```

<Figure size 720x540 with 2 Axes>

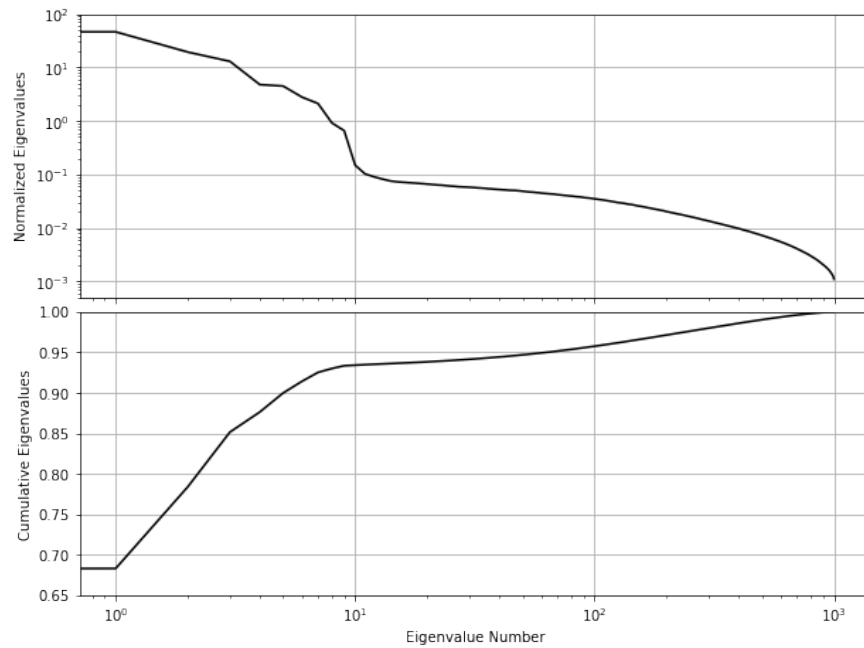


Figure 52.4: output

### 52.1.5 PCA with missing data

Observed spectrum,  $x^o$  is the true spectrum,  $x$  plus a wavelength-dependent weight,  $w$ .

Weight is zero where data are missing and  $1/\sigma^2$  for rest

Minimizing the quadratic deviation between  $x^o$  truncated reconstruction,  $\sum_i \theta_i e_i$  and solving for  $\theta_i$  gives

$$\sum_k \theta_i w(k) e_i(k) e_j(k) = \sum_k w(k) x^o(k) e_j(k),$$

If  $M_{ij} = \sum_k w(k) e_i(k) e_j(k)$  and  $F_i = \sum_k w(k) x^o(k) e_i(k)$  then

$$\theta_i = \sum_j M_{ij}^{-1} F_j,$$

- $F_j$  are coefficients derived from gappy data
- $M_{ij}^{-1}$  shows how correlated eigenvectors are over the missing regions.

An estimate of the uncertainty on the reconstruction coefficients is given by

$$\text{Cov}(\theta_i, \theta_j) = M_{ij}^{-1}.$$

Accuracy of this reconstruction will depend on the distribution of the gaps within the data vector.

The principal component vectors defined for the SDSS spectra can be used to interpolate across or reconstruct missing data. Examples of three masked spectral regions are shown comparing the reconstruction of the input spectrum (black line) using the mean and the first ten eigenspectra (blue line). The gray bands represent the masked region of the spectrum.

```
evecs = data['evecs']
mu = data['mu']
norms = data['norms']
mask = data['mask']

#-----
plot the results
i_plot = ((wavelengths > 5750) & (wavelengths < 6350))
wavelength = wavelengths[i_plot]

specnums = [20, 8, 9]
subplots = [311, 312, 313]

fig = plt.figure(figsize=(10, 12.5))
fig.subplots_adjust(left=0.09, bottom=0.08, hspace=0, right=0.92, top=0.95)

for subplot, i in zip(subplots, specnums):
 ax = fig.add_subplot(subplot)

 # compute eigen-coefficients
```

```

spectra_i_centered = spectra_corr[i] / norms[i] - mu
coeffs = np.dot(spectra_i_centered, evecs.T)

blank out masked regions
spectra_i = spectra_corr[i]
mask_i = mask[i]
spectra_i[mask_i] = np.nan

plot the raw masked spectrum
ax.plot(wavelength, spectra_i[i_plot], '-', color='k',
 label='True spectrum', lw=1.5)

plot two levels of reconstruction
for nev in [10]:
 if nev == 0:
 label = 'mean'
 else:
 label = 'reconstruction\n(nev=%i)' % nev
 spectra_i_recons = norms[i] * (mu + np.dot(coeffs[:nev], evecs[:nev]))
 ax.plot(wavelength, spectra_i_recons[i_plot], label=label, color='grey')

plot shaded background in masked region
ylim = ax.get_ylim()
mask_shade = ylim[0] + mask[i][i_plot].astype(float) * ylim[1]
plt.fill(np.concatenate([wavelength[:1], wavelength, wavelength[-1:]]),
 np.concatenate([ylim[0], mask_shade, [ylim[0]]]),
 lw=0, fc='k', alpha=0.2)

ax.set_xlim(wavelength[0], wavelength[-1])
ax.set_ylim(ylim)
ax.yaxis.set_major_formatter(ticker.NullFormatter())

if subplot == 311:
 ax.legend(loc=1)

ax.set_xlabel('λ (\AA)')
ax.set_ylabel('normalized flux')

plt.show()

```

<Figure size 720x900 with 3 Axes>

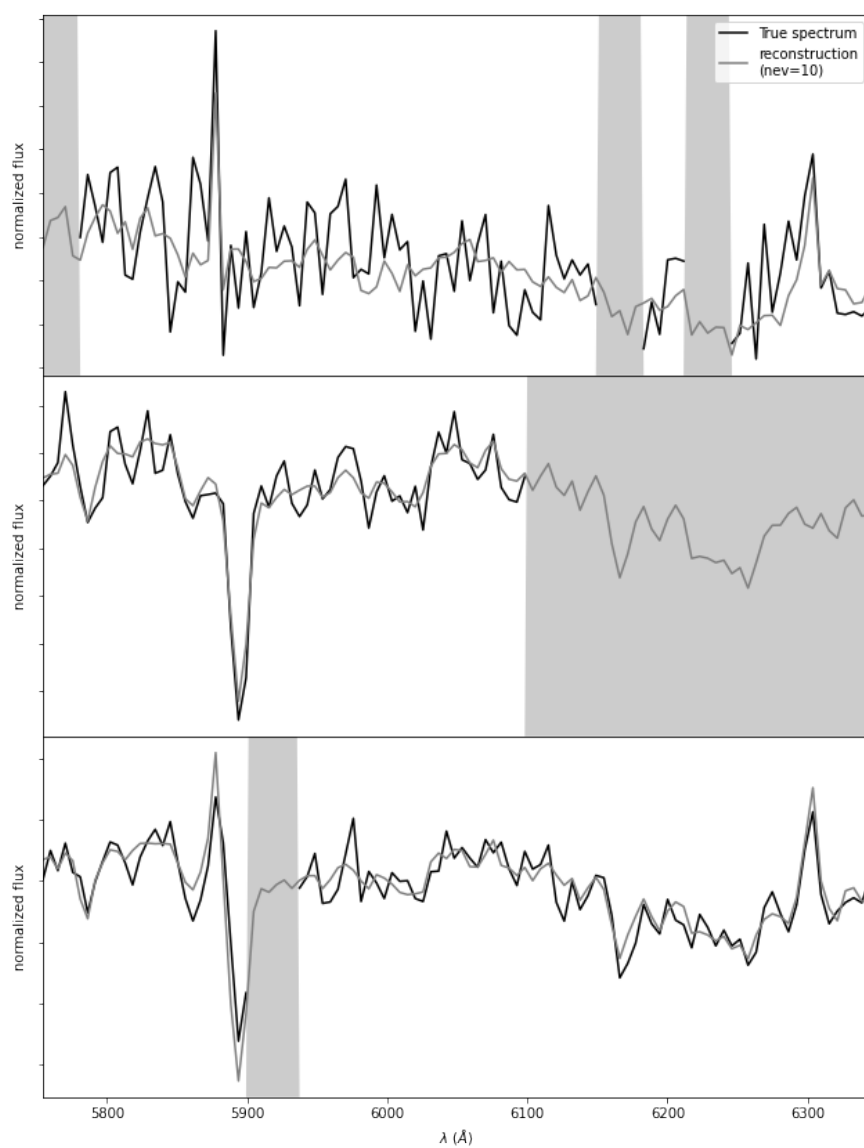


Figure 52.5: output

## 52.1.6 Comparing PCA, NMF and ICA

### 52.1.6.1 Nonnegative Matrix Factorization

Eigenvectors are defined relative to mean data vector. Principal components that can be positive or negative but for many physical systems we know data are a linear sum of positive components (e.g. galaxy spectrum is a linear sum of stellar components)

Nonnegative matrix factorization (NMF) applies positivity constraint

```
from sklearn.decomposition import NMF

X = np.random.random((100, 3)) # 100 points in 3 dims, all positive
nmf = NMF(n_components=3) # setting n_components is optional
nmf.fit(X)
proj = nmf.transform(X) # project to 3 dimensions

comp = nmf.components_ # 3 x 10 array of components
err = nmf.reconstruction_err_ # how well 3 components captures data
```

```
/Users/bsipocz/.pyenv/versions/3.9.1/lib/python3.9/site-packages/sklearn/decomposition,
warnings.warn(("The 'init' value, when 'init=None' and "
```

### 52.1.6.2 Independent component analysis

```
from sklearn.decomposition import FastICA

X = np.random.normal(size=(100, 2)) # 100 pts in 2 dims
R = np.random.random((2, 5)) # mixing matrix
X = np.dot(X, R) # X is now 2D data in 5D space
ica = FastICA(2) # fit two components

sources = ica.fit_transform(X)
proj = ica.transform(X) # 100 x 2 projection of data
comp = ica.components_ # the 2 x 5 matrix of indep. components
```

### 52.1.6.3 Comparing PCA, NMF and ICA

A comparison of the decomposition of SDSS spectra using PCA (left panel), ICA (middle panel) and NMF (right panel). The rank of the component increases from top to bottom. For the ICA and PCA the first component is the mean spectrum (NMF does not require mean subtraction). All of these techniques

isolate a common set of spectral features (identifying features associated with the continuum and line emission). The ordering of the spectral components is technique dependent.

```
from sklearn.decomposition import NMF, FastICA, PCA

data = sdss_corrected_spectra.fetch_sdss_corrected_spectra()
spectra = sdss_corrected_spectra.reconstruct_spectra(data)
wavelengths = sdss_corrected_spectra.compute_wavelengths(data)

#-----
Compute PCA, ICA, and NMF components
def compute_PCA_ICA_NMF(n_components=5):
 spec_mean = spectra.mean(0)

 # PCA: use randomized PCA for speed
 pca = PCA(n_components - 1, random_state=0, svd_solver='randomized')
 pca.fit(spectra)
 pca_comp = np.vstack([spec_mean,
 pca.components_])

 # ICA treats sequential observations as related. Because of this, we need
 # to fit with the transpose of the spectra
 ica = FastICA(n_components - 1, random_state=0)
 ica.fit(spectra.T)
 ica_comp = np.vstack([spec_mean,
 ica.transform(spectra.T).T])

 # NMF requires all elements of the input to be greater than zero
 spectra[spectra < 0] = 0
 nmf = NMF(n_components, random_state=0)
 nmf.fit(spectra)
 nmf_comp = nmf.components_

 return pca_comp, ica_comp, nmf_comp

n_components = 5
decompositions = compute_PCA_ICA_NMF(n_components)

#-----
Plot the results
fig = plt.figure(figsize=(10, 8))
fig.subplots_adjust(left=0.05, right=0.95, wspace=0.05,
 bottom=0.1, top=0.95, hspace=0.05)

titles = ['PCA components', 'ICA components', 'NMF components']
```

```

for i, comp in enumerate(decompositions):
 for j in range(n_components):
 ax = fig.add_subplot(n_components, 3, 3 * j + 1 + i)

 ax.yaxis.set_major_formatter(plt.NullFormatter())
 ax.xaxis.set_major_locator(plt.MultipleLocator(1000))
 if j < n_components - 1:
 ax.xaxis.set_major_formatter(plt.NullFormatter())
 else:
 ax.xaxis.set_major_locator(
 plt.FixedLocator(list(range(3000, 7999, 1000))))
 ax.set_xlabel(r'wavelength $\{\rm (\AA)}$')

 ax.plot(wavelengths, comp[j], '-k', lw=1)

 # plot zero line
 xlim = [3000, 8000]
 ax.plot(xlim, [0, 0], '-', c='gray', lw=1)

 if j == 0:
 ax.set_title(titles[i])

 if titles[i].startswith('PCA') or titles[i].startswith('ICA'):
 if j == 0:
 label = 'mean'
 else:
 label = 'component %i' % j
 else:
 label = 'component %i' % (j + 1)

 ax.text(0.03, 0.94, label, transform=ax.transAxes,
 ha='left', va='top')

 for l in ax.get_xticklines() + ax.get_yticklines():
 l.set_markersize(2)

 # adjust y limits
 ylim = plt.ylim()
 dy = 0.05 * (ylim[1] - ylim[0])

 ax.set_ylim(ylim[0] - dy, ylim[1] + 4 * dy)
 ax.set_xlim(xlim)

plt.show()

```



```

/Users/bsipocz/.pyenv/versions/3.9.1/lib/python3.9/site-packages/sklearn/decomposition/_nmf.py:31
warnings.warn(("The 'init' value, when 'init=None' and "
/Users/bsipocz/.pyenv/versions/3.9.1/lib/python3.9/site-packages/sklearn/decomposition/_nmf.py:10
warnings.warn("Maximum number of iterations %d reached. Increase it to"

```

<Figure size 720x576 with 15 Axes>

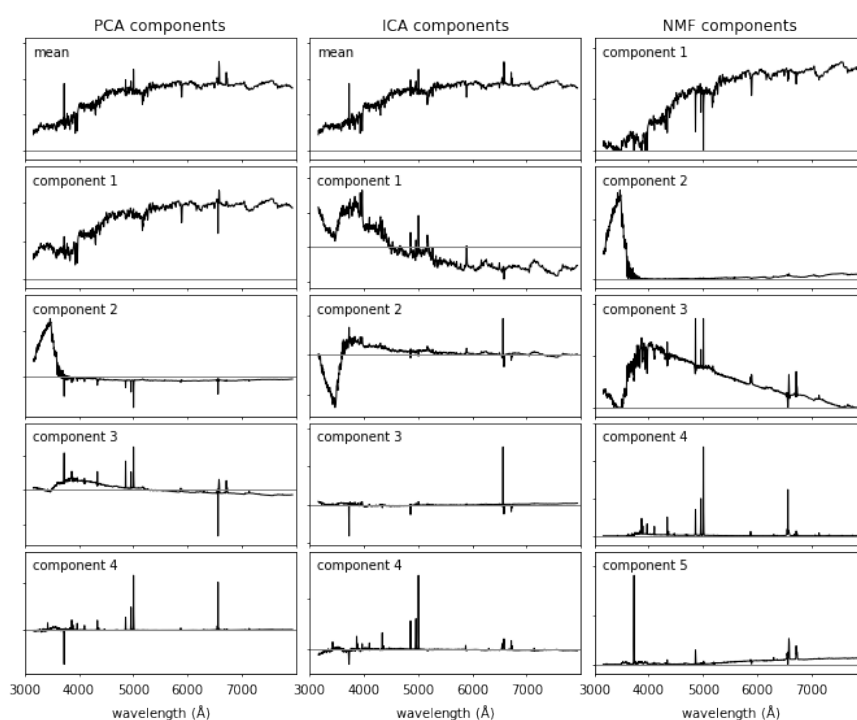


Figure 52.6: output

#### 52.1.6.4 Summary of practical properties of PCA, NMF, and ICA

The following table is a simple summary of the trade-offs along our axes of accuracy, interpretability, simplicity, and speed in dimension reduction methods, expressed in terms of high (H), medium (M), and low (L) categories.

Method	Accuracy	Interpretability	Simplicity	Speed
Principal component analysis	H	H	H	H
Nonnegative matrix factorization	H	H	M	M
Independent component analysis	M	M	L	L

## Chapter 53

# Deep Learning: Classifying Astronomical Images

Deep learning, an extension of the neural networks that were popularized in the 1990s. The concepts are inspired by the structure and function of the brain. A neuron in the brain is a core computational unit that takes a series of inputs from branched extensions of the neuron called dendrites, operates on these inputs, and generates an output that is transmitted along an axon to one or more other neurons. In the context of a neural network a neuron,  $j$ , takes a set of inputs,  $x_i$ , applies a, typically non-linear, function to these inputs and generates an output value. Networks are then created by connecting multiple neurons or layers of neurons to one another.

*Diagram omitted in the PDF build because the source image file is not present in this repository.*

If we consider the simplified network inputs are passed to the neurons in the network. Each input is weighted by a value,  $w_{ij}$  and the sum of these weighted inputs is operated on by a response or activation function  $f(\theta)$ , which transform the input signal so that it varies between 0 and 1 through the application of a non-linear response. The output from any neuron is then given by,

$$a_j = f \left( \sum_i w_{ij} x_i + b_j \right)$$

where  $b_j$  is a bias term which determines the input level at which the neuron becomes activated.

We refer to the neurons between the inputs and the output layers as the hidden layers. If the neurons from one layer connect to all neurons in a subsequent layer we call this a fully connected layer. When the outputs from the neurons

only connect to subsequent layers (i.e. the graph is acyclic) we refer to this as a feed-forward network – this is the most common structure for a neural network used in classification.

The final layer in the network is the output layer. As with the hidden layer, an activation function,  $g(\theta)$ , in the output layer acts on the weighted sum of its inputs. In this figure we have a single output node but there can be multiple outputs. For our example network the output from the final neuron,  $y_k$ , would be given by

$$y_k = g \left( \sum_j w_{jk} a_j + b_k \right) = g \left( \sum_j w_{jk} f \left( \sum_i w_{ij} x_i + b_j \right) + b_k \right)$$

**Training of the network is simply the learning of the weights and bias values**

```
to reduce the deprecation warnings
import warnings
warnings.filterwarnings("ignore")
```

```
#check we have a GPU available
from tensorflow.python.client import device_lib
device_lib.list_local_devices()
```

```
INFO:tensorflow:Enabling eager execution
INFO:tensorflow:Enabling v2 tensorshape
INFO:tensorflow:Enabling resource variables
INFO:tensorflow:Enabling tensor equality
INFO:tensorflow:Enabling control flow v2
```

```
[name: "/device:CPU:0"
 device_type: "CPU"
 memory_limit: 268435456
 locality {
 }
 incarnation: 17111024148454857511]
```

```
import numpy as np
import matplotlib.pyplot as plt
from astroML.utils import split_samples
%matplotlib inline
```

### 53.0.1 Loading some visualization and helper functions

These functions `normalize_image`, `plot_image_array`, `plot_confusion_matrix`, `plot_model_history` will be used to visualize the data and the outputs of the neural networks as a function of the type and complexity of the network

```
np.set_printoptions(precision=2)

def normalize_image(image):
 '''Rescale the contrast in an image based on the noise (used for displays and the CNN)'''
 sigmaG_coeff = 0.7413
 image = image.reshape(21,21)

 per25,per50,per75 = np.percentile(image,[25,50,75])
 sigmaG = sigmaG_coeff * (per75 - per25)
 # sigma clip image, remove background, and normalize to unity
 image[image<(per50-2*sigmaG)] = per50-2*sigmaG
 image -= np.min(image)
 image /= np.sum(image)

 return image

def plot_image_array(images, nrows=2, ncols=5, figsize=[8,4], nx=21, ny=21, title='', subtitle=False,
 class_true=None, classes=None):
 '''Plot an array of images'''
 fig, ax = plt.subplots(nrows=nrows,ncols=ncols,figsize=figsize)
 fig.subplots_adjust(hspace=0, left=0.07, right=0.95, wspace=0.05, bottom=0.15)
 for indx in np.arange(nrows*ncols):
 i = int(indx/ncols)
 j = indx%ncols
 if (i == 0):
 ax[i][j].xaxis.set_major_formatter(plt.NullFormatter())
 if (j != 0):
 ax[i][j].yaxis.set_major_formatter(plt.NullFormatter())

 ax[i][j].imshow(images[indx].reshape(nx,ny), cmap='gray')
 if (subtitle == True):
 ax[i][j].set_title('True Class: %i, Predicted Class: %i\n Prob Class 1 %e ' %
 (np.argmax(class_true[indx]), np.argmax(classes[indx]), classes[indx,1]))

 fig.suptitle(title)
 ax[0][0].set_ylabel('y')
 ax[nrows-1][int(ncols/2)].set_xlabel('x')

from sklearn.metrics import confusion_matrix
from sklearn.utils.multiclass import unique_labels
```

```

def plot_confusion_matrix(y_true, y_pred,
 normalize=False,
 title=None,
 cmap=plt.cm.Blues):
 """
 From scikit-learn: plots a confusion matrix.
 Normalization can be applied by setting `normalize=True`.
 """
 if not title:
 if normalize:
 title = 'Normalized confusion matrix'
 else:
 title = 'Confusion matrix, without normalization'

 # Compute confusion matrix
 cm = confusion_matrix(y_true, y_pred)

 if normalize:
 cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

 fig, ax = plt.subplots()
 im = ax.imshow(cm, interpolation='nearest', cmap=cmap)
 ax.figure.colorbar(im, ax=ax)

 ax.set(xticks=np.arange(cm.shape[1]),
 yticks=np.arange(cm.shape[0]),
 # ... and label them with the respective list entries
 title=title,
 ylabel='True label',
 xlabel='Predicted label')

 # Rotate the tick labels and set their alignment.
 plt.setp(ax.get_xticklabels(), rotation=45, ha="right",
 rotation_mode="anchor")

 #fixes "squishing of plot"
 plt.ylim([1.5, -.5])

 # Loop over data dimensions and create text annotations.
 fmt = '.2f' if normalize else 'd'
 thresh = cm.max() / 2.
 for i in range(cm.shape[0]):
 for j in range(cm.shape[1]):
 ax.text(j, i, format(cm[i, j], fmt),
 ha="center", va="center",

```

```

 color="white" if cm[i, j] > thresh else "black")
fig.tight_layout()

def plot_model_history(history):
 '''Plot the training and validation history for a TensorFlow network'''

 # Extract loss and accuracy
 loss = history.history['loss']
 val_loss = history.history['val_loss']
 acc = history.history['accuracy']
 val_acc = history.history['val_accuracy']

 fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,5))
 ax[0].plot(np.arange(n_epochs), loss, label='Training Loss')
 ax[0].plot(np.arange(n_epochs), val_loss, label='Validation Loss')
 ax[0].set_title('Loss Curves')
 ax[0].legend()
 ax[0].set_xlabel('Epoch')
 ax[0].set_ylabel('Loss')

 ax[1].plot(np.arange(n_epochs), acc, label='Training Accuracy')
 ax[1].plot(np.arange(n_epochs), val_acc, label='Validation Accuracy')
 ax[1].set_title('Accuracy Curves')
 ax[1].legend()
 ax[1].set_xlabel('Epoch')
 ax[1].set_ylabel('Accuracy')

```

### 53.0.2 Load the training samples and create test and validation data sets

The data we are using is taken from a survey for NEOs by Lori Allen and collaborators using DECam on the Blanco 4m Telescope at CTIO. The data comprise a stack of images taken over a period of 5 nights. Within these images we search for slowly moving sources (TNOs) along potential orbital trajectories. Given these trajectories we coadd the images. Our goal is to determine whether there is a point source within the coadded images. The training sample includes images of simulated TNOs (true positives; stamps\_sources.npz) and random trajectories where there is no known source (false positives; stamps\_noise.npz). The true positives range in signal-to-noise from 100 to 3

```

Brute force direct downloads source and noise images to circumvent size limitations
for google drive internal virus scan. Download may take some time.

import os

```

```

import requests

files = {'sources': (os.path.join('data', 'stamps_noise.npy'), '1UT2BCf-IDUEpvTmcU4bq6i'),
 'noise': (os.path.join('data', 'stamps_sources.npy'), '1cZaMCA0z_nPX6GB_meLGoi')}

for name, file_id in files.values():
 if not os.path.exists(name):
 print(f"Downloading file {name}.")

 os.makedirs(os.path.dirname(name), exist_ok=True)
 url = f"https://docs.google.com/uc?export=download&id={file_id}&confirm=t"
 response = requests.post(url)
 with open(name, 'wb') as file:
 file.write(response.content)
 print(f"File {name} is downloaded")

sources = np.load(files['sources'][0])
noise = np.load(files['noise'][0])

```

```

Downloading file data/stamps_noise.npy.
File data/stamps_noise.npy is downloaded
Downloading file data/stamps_sources.npy.
File data/stamps_sources.npy is downloaded

```

```

normalizing images

point_source_stamps = []
for image in sources:
 point_source_stamps.append(normalize_image(image))

no_point_source_stamps = []
for image in noise:
 no_point_source_stamps.append(normalize_image(image))

plot sample of images
plot_image_array(no_point_source_stamps, title='false positives')
plot_image_array(point_source_stamps, title='true positives')

```

```
<Figure size 576x288 with 10 Axes>
```

```
<Figure size 576x288 with 10 Axes>
```



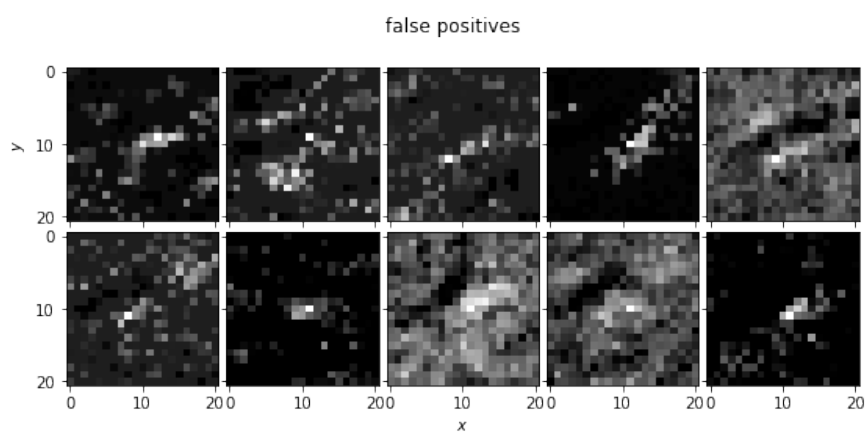


Figure 53.1: output

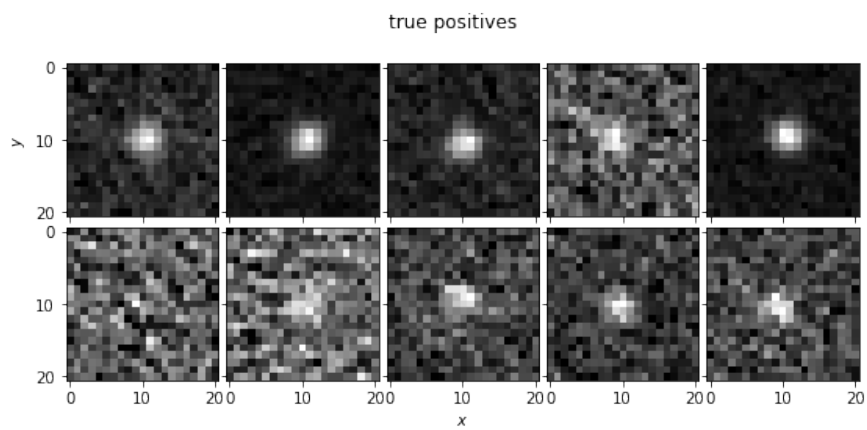


Figure 53.2: output

### 53.0.3 Create a training, validation, and test sample

We will use astroML's `split_samples` to do this `split_samples(input_stamps, stamp_class, [0.7,0.1,0.2])` breaks the data in to random selections with appropriate fractions of sources

```
from tensorflow.keras.utils import to_categorical
def reshape_arrays(data, labels):
 '''reshape arrays for Keras'''
 data = data.reshape(-1,21, 21, 1)
 labels = to_categorical(labels)
 return data,labels

combine the false positives and true positives
input_stamps = np.vstack([no_point_source_stamps, point_source_stamps])
stamp_class = np.zeros(len(no_point_source_stamps) + len(point_source_stamps))
stamp_class[len(no_point_source_stamps):] = 1 # 0 for noise, 1 for a star

split the samples into training, validation and test data sets
(data_train, data_val, data_test), (class_train, class_val, class_test) = split_samples(

data_train, class_train = reshape_arrays(data_train, class_train)
data_val, class_val = reshape_arrays(data_val, class_val)
data_test, class_test = reshape_arrays(data_test, class_test)

print ('Number of samples in the training ({}); test ({}); and validation ({}). data set
data_val
data_test
```

```
Number of samples in the training (76666); test (21905); and validation (10952) data set
```

## 53.1 Neural Network Frameworks

The development and release of open source deep learning libraries has made the use of deep neural networks accessible to a wide range of fields. Currently there are two common packages PyTorch (<https://pytorch.org>) and Tensorflow (<https://www.tensorflow.org>). Either code base can be utilized for the figures and problems in this book (and generally they have the same functionality).

### 53.1.1 TensorFlow:

Tensorflow is the more established code base with a large community and a large number of tutorials (<https://www.tensorflow.org/tutorials>) and online courses.

Its functionality is more developed than PyTorch with tools to visualize and inspect a network (e.g., see TensorBoard). On the other hand, the learning curve for PyTorch is generally considered to be easier than that for Tensorflow with PyTorch having a more natural object oriented interface for people used to writing Python code.

### 53.1.2 PyTorch:

The primary difference between TensorFlow and PyTorch is that the networks (or graphs) that TensorFlow generates are static while the networks for PyTorch are dynamic (see TensorFlow Fold for dynamic graphs). This means that with PyTorch one can modify and adjust the network on-the-fly (e.g., making it easier to adjust for changes in the input dimensionality or number of input nodes within a network). This feature and the object-oriented design of PyTorch often results in fewer lines of code to achieve the same solution when compared to Tensorflow.

### 53.1.3 Keras:

Keras is a high-level API written on top of TensorFlow (and its precursor Theano). It is written in Python and provides a simple and intuitive interface when building neural networks. It is currently released as part of TensorFlow.

**What should you choose?** Both frameworks are continuously evolving. The choice of deep learning library will likely come down to which one you find better fits your style of programming and learning. For this tutorial we will use Keras as it has an intuitive implementation of the graphical or network models.

### 53.1.4 Building a network:

Let's start by defining what we need for the network. We will start with Keras and

- create a sequential model (this means we add layers one-by-one as we see in our introductory figure)
- add a dense (fully connected) layer with 30 neurons
  - **input\_shape** describes the dimensionality of the *input data* to this first hidden layer
  - **activation** describes the activation function for the neurons (in this case we will be using 'relu'; rectified linear unit)
- add a second dense (fully connected) layer with 30 neurons

- flatten the output of the second layer into a single vector so we can use `categorical_crossentropy` as we are assuming that our classes are “one-hot encoding” (i.e. [1,0] or [0,1])
- add an output layer using “softmax” (this means the activation values for each class sum to 1 so they can be treated like probabilities) with 2 nodes (for our example we could have used a single output)

### 53.1.5 Training the network

Training a neural network is conceptually simple. Given a labelled set of data and a loss function, we need to optimize the weights and biases within the network by minimizing the loss. A solution for training large networks uses backpropagation to efficiently estimate the gradient of the loss function with respect to the weights and biases.

## 53.2 Creating a Neural Network with Keras

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv2D, Flatten, Activation
from tensorflow.keras.utils import to_categorical

def simple(input_shape=(21, 21, 1), n_classes: int = 2):

 model = tf.keras.models.Sequential(name='simple')

 # input: 21x21 images with 1 channel -> (21, 21, 1) tensors.
 model.add(tf.keras.layers.Dense(30, input_shape=input_shape, activation='relu', name='fc_1'))

 model.add(tf.keras.layers.Dense(30, activation='relu', name='fc_2'))

 # output layer
 model.add(tf.keras.layers.Flatten())
 activation = 'sigmoid' if n_classes == 1 else 'softmax'
 model.add(tf.keras.layers.Dense(n_classes, activation=activation, name='fc_out'))

 return model

#output the structure of the model
simple_model = simple()
simple_model.summary()
```

Model: "simple"

Layer (type)	Output Shape	Param #
fc_1 (Dense)	(None, 21, 21, 30)	60
fc_2 (Dense)	(None, 21, 21, 30)	930
flatten (Flatten)	(None, 13230)	0
fc_out (Dense)	(None, 2)	26462
Total params: 27,452		
Trainable params: 27,452		
Non-trainable params: 0		

```
n_epochs=1
batch_size=1
simple_model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])
simple_model_history = simple_model.fit(data_train, class_train, epochs=n_epochs, batch_size=batch_size,
 validation_data=(data_val, class_val), shuffle=True)
```

```
76666/76666 [=====] - 66s 857us/step - loss: 0.6549 - accuracy: 0.5892 -
```

### 53.2.1 Exercise

**Mini-batch:** Optimization of the weights uses a standard gradient descent technique. If the loss function can be expressed in terms of a sum over subsets of the training data (e.g., as is the case for the L2 norm) the training can be undertaken either for the dataset as a whole, for subsets of the data (batch learning), or for individual entries (on-line or stochastic learning). *Batch gradient descent* looks at all points in the data and calculates the average gradients before updating the weights in the model. *Stochastic gradient descent* takes a single point and calculates the gradients and then updates the model (and then repeats). *Mini-batch gradient descent* takes a subset of the training data and calculates the average gradients and updates the model (and then repeats over all mini-batches).

```
n_epochs=20
batch_size=
simple_model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])
simple_model_history = simple_model.fit(data_train, class_train, epochs=n_epochs, batch_size=batch_size,
 validation_data=(data_val, class_val), shuffle=True)
```

### 53.2.2 Performance of the network

```
use the network to predict class values
classes = simple_model.predict(data_test)
print (classes)
```

```
[[0.41 0.59]
 [0.41 0.59]
 [0.41 0.59]
 ...
 [0.41 0.59]
 [0.41 0.59]
 [0.41 0.59]]
```

```
plot the confusion matrix
plot_confusion_matrix(np.argmax(class_test,axis=1), np.argmax(classes,axis=1), normali:
 title='Normalized confusion matrix')
```

```
<Figure size 432x288 with 2 Axes>
```

```
plot the training history of the network
plot_model_history(simple_model_history)
```

```
<Figure size 720x360 with 2 Axes>
```

### 53.2.3 Batch normalization

Our first optimization over the vanilla NN. Batch normalization scales the activations from a layer (note we normalized the input data) to have zero mean and unit variance. In reality, the two parameters gamma (for the standard deviation) and beta (for the mean) are learned by the network and the activations multiplied/added by these parameters. Batch normalization provides a degree of regularization and allows for faster learning rates as the outputs are constrained to 0-1 (i.e. you don't get large excursions in the weights of subsequent layers in a network that need to be reoptimized/trained).

The normalization is applied to mini-batches of training data (as opposed to using the full training sample) - add a batch normalization layer:

```
model.add(tf.keras.layers.BatchNormalization(axis = 3, name =
'bn_1'))
```

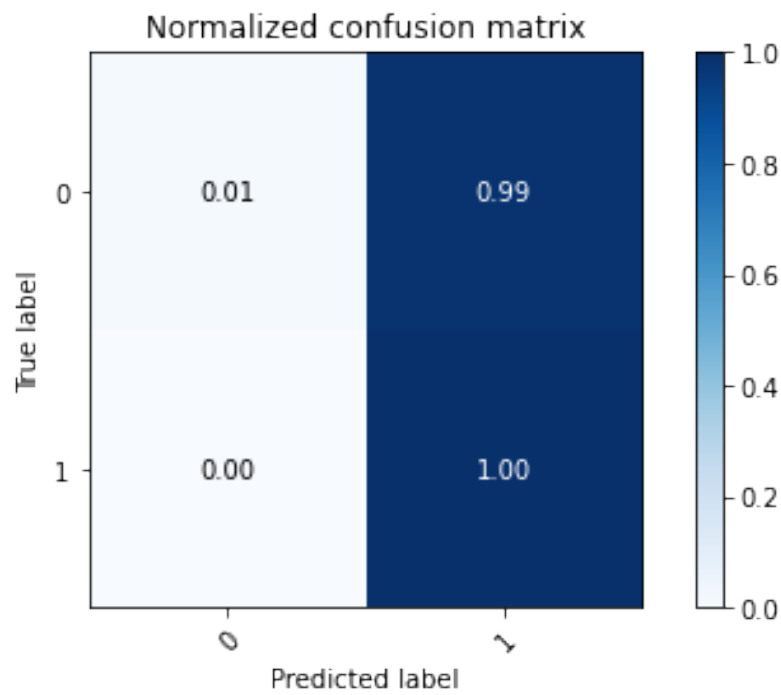


Figure 53.3: output

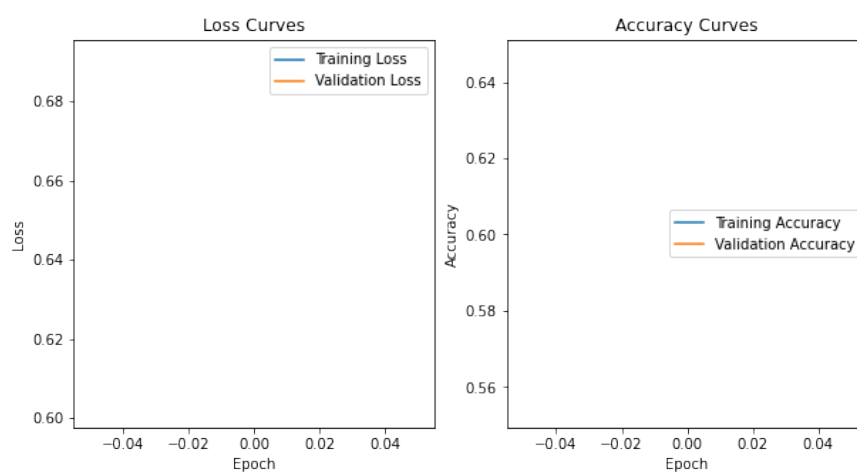


Figure 53.4: output

```

def simpleBN(input_shape=(21, 21, 1), n_classes: int = 2):

 model = tf.keras.models.Sequential(name='simple')

 # input: 21x21 images with 1 channel -> (21, 21, 1) tensors.
 model.add(tf.keras.layers.Dense(30, input_shape=input_shape, activation='relu', name='fc_in'))

 model.add(tf.keras.layers.Dense(30, activation='relu', name='fc_2'))

 # output layer
 model.add(tf.keras.layers.Flatten())
 activation = 'sigmoid' if n_classes == 1 else 'softmax'
 model.add(tf.keras.layers.Dense(n_classes, activation=activation, name='fc_out'))

 return model

n_epochs=20
simple_model = simpleBN()
simple_model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])
simple_model_history = simple_model.fit(data_train, class_train, epochs=n_epochs, batch_size=32,
 validation_data=(data_val, class_val), shuffle=True)
classes = simple_model.predict(data_test)

```

```

Epoch 1/20
300/300 [=====] - 12s 39ms/step - loss: 0.6903 - accuracy: 0.0000
Epoch 2/20
300/300 [=====] - 13s 44ms/step - loss: 0.6887 - accuracy: 0.0000
Epoch 3/20
300/300 [=====] - 12s 41ms/step - loss: 0.6883 - accuracy: 0.0000
Epoch 4/20
300/300 [=====] - 10s 35ms/step - loss: 0.6889 - accuracy: 0.0000
Epoch 5/20
300/300 [=====] - 12s 39ms/step - loss: 0.6881 - accuracy: 0.0000
Epoch 6/20
300/300 [=====] - 12s 38ms/step - loss: 0.6887 - accuracy: 0.0000
Epoch 7/20
300/300 [=====] - 12s 39ms/step - loss: 0.6881 - accuracy: 0.0000
Epoch 8/20
300/300 [=====] - 12s 40ms/step - loss: 0.6883 - accuracy: 0.0000
Epoch 9/20
300/300 [=====] - 12s 39ms/step - loss: 0.6878 - accuracy: 0.0000
Epoch 10/20
300/300 [=====] - 12s 40ms/step - loss: 0.6879 - accuracy: 0.0000
Epoch 11/20
300/300 [=====] - 12s 40ms/step - loss: 0.6878 - accuracy: 0.0000

```



```

Epoch 12/20
300/300 [=====] - 11s 36ms/step - loss: 0.6879 - accuracy: 0.5477 - val_
Epoch 13/20
300/300 [=====] - 13s 42ms/step - loss: 0.6882 - accuracy: 0.5456 - val_
Epoch 14/20
300/300 [=====] - 12s 40ms/step - loss: 0.6874 - accuracy: 0.5495 - val_
Epoch 15/20
300/300 [=====] - 12s 39ms/step - loss: 0.6871 - accuracy: 0.5511 - val_
Epoch 16/20
300/300 [=====] - 12s 39ms/step - loss: 0.6870 - accuracy: 0.5505 - val_
Epoch 17/20
300/300 [=====] - 12s 40ms/step - loss: 0.6881 - accuracy: 0.5440 - val_
Epoch 18/20
300/300 [=====] - 12s 41ms/step - loss: 0.6874 - accuracy: 0.5475 - val_
Epoch 19/20
300/300 [=====] - 12s 39ms/step - loss: 0.6873 - accuracy: 0.5477 - val_
Epoch 20/20
300/300 [=====] - 12s 39ms/step - loss: 0.6869 - accuracy: 0.5493 - val_

Plot normalized confusion matrix
plot_confusion_matrix(np.argmax(class_test,axis=1), np.argmax(classes,axis=1), normalize=True,
 title='Normalized confusion matrix')

```

<Figure size 432x288 with 2 Axes>

```
plot_model_history(simple_model_history)
```

<Figure size 720x360 with 2 Axes>

### 53.2.4 Convolutional Networks

Convolutional Neural Networks or CNNs are networks designed to work with images or with any regularly sampled dataset. CNNs reduce the complexity of the network by requiring that neurons only respond to inputs from a subset of an image (the receptive field). This mimics the operation of the visual cortex where neurons only respond to a small part of the field-of-view.

There are four principal components to a CNN: - a convolutional layer, - a *non-linear activation function*, - a pooling or downsampling operation, and - a *fully connected layer for classification*

Dependent on the complexity of the network or structure of the data, these components can occur singularly or chained together in multiple sequences.

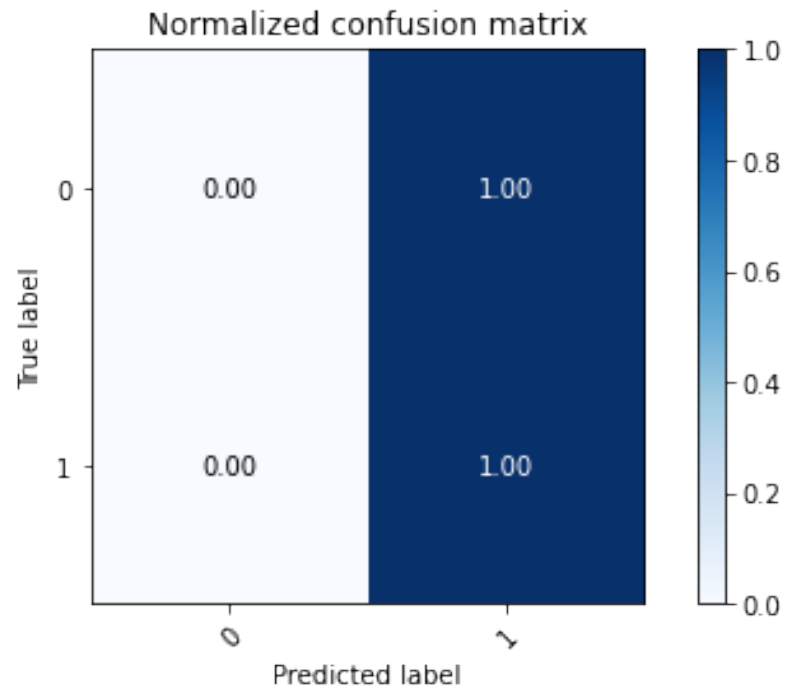


Figure 53.5: output

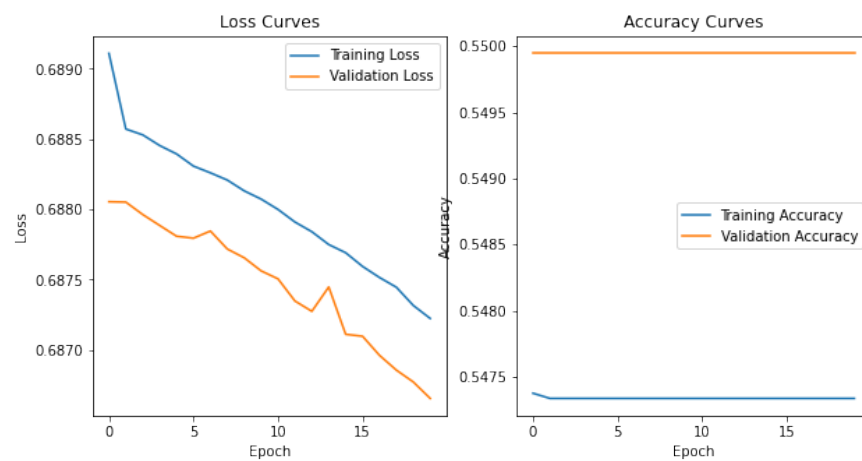


Figure 53.6: output

*CNN architecture diagram omitted in the PDF build because the source image file is not present in this repository.*

**Convolution** in a CNN refers to the convolution of the input data  $I(x, y)$  with a kernel  $K(x, y)$  which will produce a feature map  $F(x, y)$

$$F(x, y) = K(x, y) * I(x, y) = \sum_{x_0} \sum_{y_0} I(x - x_0, y - y_0) K(x_0, y_0). \quad (53.1)$$

The kernel only responds to pixels within its receptive field (i.e., the size of the kernel), reducing the computational complexity of the resulting network. The kernels in the convolution are described by a depth (the number of kernels,  $K$ , applied to the image), and a stride (how many pixels a kernel shifts at each step in the convolution; typically one). Given an  $N \times M$  image, the result of the convolution step is to transform a single image into a data cube of feature maps with a dimension  $N \times M \times K$ .

Once **learned** the kernels within the convolutional layer can appear as physically intuitive operations on the images such as edge detection filters.

As with traditional neural networks, a non-linear activation function is applied to the individual pixels in the resulting feature maps.

The **pooling** in the CNN downsamples or subsamples the feature maps. Pooling summarizes values within a region of interest (e.g., a 2x2 pixel window). The summary can be the average pixel value but more commonly the maximum pixel value is preserved (Max Pooling) in the downsampling. This pooling of the feature maps reduces the size of the resulting network and makes the network less sensitive to small translations or distortions between images.

The final layer of a CNN is the classification layer which maps the output of the CNN to a set of labels. This is typically a fully connected layer where each output of the final pooling layer connects to all neurons in the classification layer.

### 53.2.5 CNNs: Increasing the complexity of the architecture with VGG6

Let's start with a simple network architecture from Oxford's Visual Geometry Group. VGG6 is a very simple network that performs well in traditional image classification competitions (e.g. those using the ImageNet data)

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv2D, Flatten
from tensorflow.keras.utils import to_categorical
```

```

def vgg6(input_shape=(21, 21, 1), n_classes: int = 2):
 """
 VGG6
 :param input_shape:
 :param n_classes:
 :return:
 """

 model = tf.keras.models.Sequential(name='VGG6')
 # input: 21x21 images with 1 channel -> (21, 21, 1) tensors.
 # this applies 16 convolution filters of size 3x3 each.
 model.add(tf.keras.layers.Conv2D(16, (3, 3), activation='relu', input_shape=input_shape))
 model.add(tf.keras.layers.Conv2D(16, (3, 3), activation='relu', name='conv2'))
 model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2)))
 model.add(tf.keras.layers.Dropout(0.25))

 model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu', name='conv3'))
 model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu', name='conv4'))
 model.add(tf.keras.layers.BatchNormalization(axis = 3, name = 'bn_2'))
 model.add(tf.keras.layers.MaxPooling2D(pool_size=(4, 4)))
 model.add(tf.keras.layers.Dropout(0.25))

 model.add(tf.keras.layers.Flatten())

 model.add(tf.keras.layers.Dense(256, activation='relu', name='fc_1'))
 model.add(tf.keras.layers.Dropout(0.5))
 # output layer
 activation = 'sigmoid' if n_classes == 1 else 'softmax'
 model.add(tf.keras.layers.Dense(n_classes, activation=activation, name='fc_out'))

 return model

```

### 53.2.6 The use of dropout layers

As we increase the complexity of the network we run into the issue of overfitting the data (as seen in many of the astroML examples). The dropout layer `model.add(tf.keras.layers.Dropout(0.5))` at each training epoch randomly sets a neuron to 0 with a probability of 0.5. There is debate over whether the dropout layer should come before or after an activation layer but a recommended rule of thumb is that it should come after the activation layer for activation functions other than relu.

```

vgg_model = vgg6()
vgg_model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])

```

```

n_epochs=15
vgg_model.summary()
vgg_model_history = vgg_model.fit(data_train, class_train, epochs=n_epochs, batch_size=1024, validation_data=(data_val, class_val), shuffle=True)
classes = vgg_model.predict(data_test)

```

Model: "VGG6"

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 19, 19, 16)	160
conv2 (Conv2D)	(None, 17, 17, 16)	2320
max_pooling2d (MaxPooling2D)	(None, 8, 8, 16)	0
dropout (Dropout)	(None, 8, 8, 16)	0
conv3 (Conv2D)	(None, 6, 6, 32)	4640
conv4 (Conv2D)	(None, 4, 4, 32)	9248
bn_2 (BatchNormalization)	(None, 4, 4, 32)	128
max_pooling2d_1 (MaxPooling2D)	(None, 1, 1, 32)	0
dropout_1 (Dropout)	(None, 1, 1, 32)	0
flatten_2 (Flatten)	(None, 32)	0
fc_1 (Dense)	(None, 256)	8448
dropout_2 (Dropout)	(None, 256)	0
fc_out (Dense)	(None, 2)	514

```

Total params: 25,458
Trainable params: 25,394
Non-trainable params: 64

```

Epoch 1/15

75/75 - 13s - loss: 0.6890 - accuracy: 0.5519 - val\_loss: 0.6904 - val\_accuracy: 0.5499

Epoch 2/15

75/75 - 13s - loss: 0.6869 - accuracy: 0.5557 - val\_loss: 0.6892 - val\_accuracy: 0.5499

Epoch 3/15

```

75/75 - 13s - loss: 0.6845 - accuracy: 0.5594 - val_loss: 0.6881 - val_accuracy: 0.5494
Epoch 4/15
75/75 - 13s - loss: 0.6807 - accuracy: 0.5685 - val_loss: 0.6866 - val_accuracy: 0.5494
Epoch 5/15
75/75 - 13s - loss: 0.6779 - accuracy: 0.5772 - val_loss: 0.6850 - val_accuracy: 0.5494
Epoch 6/15
75/75 - 13s - loss: 0.6747 - accuracy: 0.5922 - val_loss: 0.6831 - val_accuracy: 0.5500
Epoch 7/15
75/75 - 13s - loss: 0.6714 - accuracy: 0.6051 - val_loss: 0.6798 - val_accuracy: 0.5500
Epoch 8/15
75/75 - 13s - loss: 0.6675 - accuracy: 0.6169 - val_loss: 0.6759 - val_accuracy: 0.5520
Epoch 9/15
75/75 - 13s - loss: 0.6628 - accuracy: 0.6290 - val_loss: 0.6693 - val_accuracy: 0.5610
Epoch 10/15
75/75 - 13s - loss: 0.6579 - accuracy: 0.6362 - val_loss: 0.7803 - val_accuracy: 0.4500
Epoch 11/15
75/75 - 13s - loss: 0.6523 - accuracy: 0.6394 - val_loss: 0.7370 - val_accuracy: 0.5500
Epoch 12/15
75/75 - 13s - loss: 0.6451 - accuracy: 0.6452 - val_loss: 0.7329 - val_accuracy: 0.5500
Epoch 13/15
75/75 - 13s - loss: 0.6398 - accuracy: 0.6427 - val_loss: 0.7394 - val_accuracy: 0.5530
Epoch 14/15
75/75 - 13s - loss: 0.6337 - accuracy: 0.6444 - val_loss: 1.0248 - val_accuracy: 0.5494
Epoch 15/15
75/75 - 13s - loss: 0.6285 - accuracy: 0.6464 - val_loss: 0.6993 - val_accuracy: 0.5500

```

```

plot the confusion matrix
plot_confusion_matrix(np.argmax(class_test,axis=1), np.argmax(classes,axis=1), normalize=True,
 title='Normalized confusion matrix')

```

<Figure size 432x288 with 2 Axes>

```

plot the training history
plot_model_history(vgg_model_history)

```

<Figure size 720x360 with 2 Axes>

```

plot example classifications
plot_image_array(data_val, figsize=[16,10], subtitle=True, classes=classes, class_true=

```

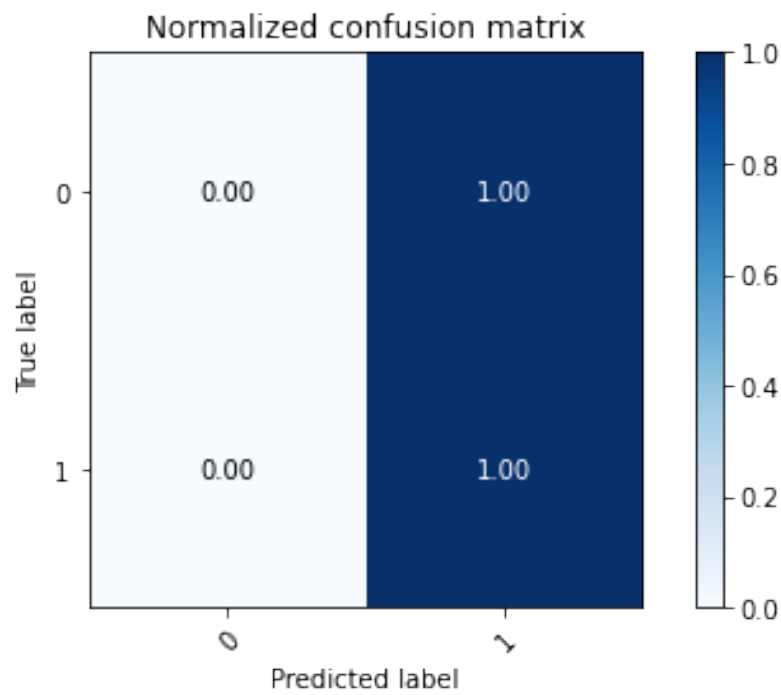


Figure 53.7: output

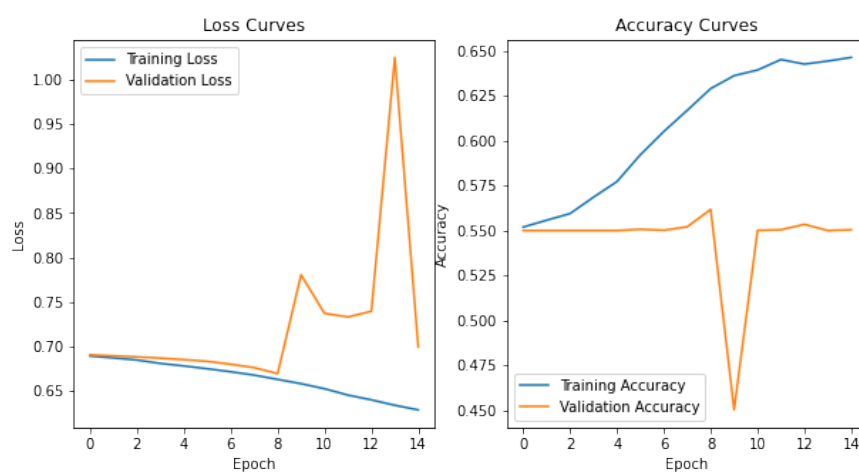


Figure 53.8: output

<Figure size 1152x720 with 10 Axes>

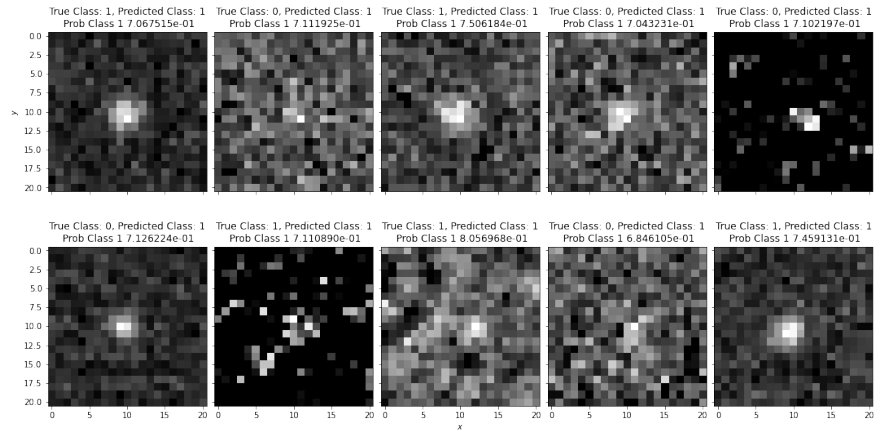


Figure 53.9: output

### 53.2.7 Interpreting networks: how many layers and how many neurons?

The number of layers, number of neurons in a layer, and the connectivity of these layers is typically described as the network architecture.

Approaches to defining a network architecture become more trial and error than applying an underlying set of principles. For a starting point, however, there are relatively few problems that benefit significantly from more than two layers and we recommend starting with a single layer when training an initial network and using cross-validation to determine when additional layers lead result in the data being overfit.

As with the number of layers, the number of neurons within a layer drives the computational cost (and requiring progressively larger training sets to avoid overfitting of the data). There are many proposals for rules of thumb for defining a network architecture: - the number of neurons should lie between the number of inputs and output nodes - the number of neurons should be equal to the number of outputs plus 2/3rd the number input nodes. - the number of neurons in the hidden layer should be less than twice the size of the input layers

### 53.2.8 Interpreting networks: where is a network looking

Occulsion maps, saliency maps, class activation maps are all techniques for expressing which pixels contribute to classification. These are attempts to reduce



the “black box” nature of the networks. The simplest of these is the occlusion map where we part of an image and calculate the probability of it belonging to a class. If the probability decreases the occluded part of the image is assumed to be important. If there is no change in probability the occluded pixels are not assumed to be important. A simple implementation of this is shown here.

```

model = vgg_model
image_number = 11

kernel_size=5
input_stamp = data_test[image_number].reshape(21,21)
i = 0
j=0
heatmap = []
keras_stamps = []
for j in range(22-kernel_size):
 for i in range(22-kernel_size):
 img = np.copy(input_stamp)
 img[i:i+kernel_size,j:j+kernel_size] = 0
 img = normalize_image(img)
 keras_stamps.append(img)
keras_stamps = np.array(keras_stamps).reshape([-1,21,21,1])
probs = 1. - model.predict(keras_stamps)
heatmap = probs[:,1].reshape(22-kernel_size,22-kernel_size)

def transparent_cmap(cmap, N=255):
 "Copy colormap and set alpha values"
 mycmap = cmap
 mycmap._init()
 mycmap._lut[:, -1] = np.linspace(0, 0.8, N+4)
 return mycmap

pad heatmap to same size as original image
heatmap = np.pad(heatmap, pad_width=np.int(kernel_size/2), mode='minimum')

use the base cmap to create transparent overlay
mycmap = transparent_cmap(plt.cm.Red)
fig, ax = plt.subplots(nrows=1,ncols=1)
ax.imshow(data_test[image_number].reshape(21,21), cmap='gray')
ax.imshow(np.array(heatmap), alpha=0.5, cmap=mycmap)

```

```
<matplotlib.image.AxesImage at 0x196d13f10>
```

```
<Figure size 432x288 with 1 Axes>
```

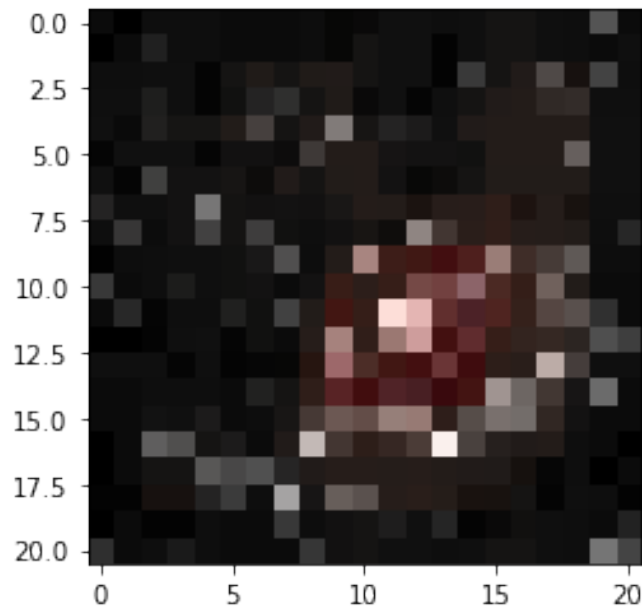


Figure 53.10: output

### 53.2.9 Exercise for the reader: more complicated architectures: resnet50

```
Based on https://github.com/priya-dwivedi/Deep-Learning/blob/master/resnet_keras/Res
import tensorflow as tf
from tensorflow.keras.layers import Input, Add, Dense, Activation, ZeroPadding2D, BatchNormalization
from tensorflow.keras.initializers import glorot_uniform
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.utils import to_categorical
import tensorflow.keras.backend as K
K.set_image_data_format('channels_last')
def identity_block(X, f, filters, stage, block):
 """
 Implementation of the identity block as defined in Figure 3

 Arguments:
 X -- input tensor of shape (m, n_H_prev, n_W_prev, n_C_prev)
 f -- integer, specifying the shape of the middle CONV's window for the main path
 filters -- python list of integers, defining the number of filters in the CONV layer
 stage -- integer, used to name the layers, depending on their position in the network
 """
```

```

 block -- string/character, used to name the layers, depending on their position in the network

Returns:
X -- output of the identity block, tensor of shape (n_H, n_W, n_C)
"""

defining name basis
conv_name_base = 'res' + str(stage) + block + '_branch'
bn_name_base = 'bn' + str(stage) + block + '_branch'

Retrieve Filters
F1, F2, F3 = filters

Save the input value. You'll need this later to add back to the main path.
X_shortcut = X

First component of main path
X = Conv2D(filters = F1, kernel_size = (1, 1), strides = (1,1), padding = 'valid', name = conv_name_base + '1a')(X)
X = BatchNormalization(axis = 3, name = bn_name_base + '2a')(X)
X = Activation('relu')(X)

Second component of main path (3 lines)
X = Conv2D(filters = F2, kernel_size = (f, f), strides = (1,1), padding = 'same', name = conv_name_base + '1b')(X)
X = BatchNormalization(axis = 3, name = bn_name_base + '2b')(X)
X = Activation('relu')(X)

Third component of main path (2 lines)
X = Conv2D(filters = F3, kernel_size = (1, 1), strides = (1,1), padding = 'valid', name = conv_name_base + '1c')(X)
X = BatchNormalization(axis = 3, name = bn_name_base + '2c')(X)

Final step: Add shortcut value to main path, and pass it through a RELU activation (2 lines)
X = Add()([X, X_shortcut])
X = Activation('relu')(X)

return X

def convolutional_block(X, f, filters, stage, block, s = 2):
 """
 Implementation of the convolutional block as defined in Figure 4

 Arguments:
 X -- input tensor of shape (m, n_H_prev, n_W_prev, n_C_prev)
 f -- integer, specifying the shape of the middle CONV's window for the main path
 filters -- python list of integers, defining the number of filters in the CONV layers of the block
 """

```

```

stage -- integer, used to name the layers, depending on their position in the netw
block -- string/character, used to name the layers, depending on their position in
s -- Integer, specifying the stride to be used

Returns:
X -- output of the convolutional block, tensor of shape (n_H, n_W, n_C)
"""

defining name basis
conv_name_base = 'res' + str(stage) + block + '_branch'
bn_name_base = 'bn' + str(stage) + block + '_branch'

Retrieve Filters
F1, F2, F3 = filters

Save the input value
X_shortcut = X

MAIN PATH
First component of main path
X = Conv2D(F1, (1, 1), strides = (s,s), name = conv_name_base + '2a', kernel_initi:
X = BatchNormalization(axis = 3, name = bn_name_base + '2a')(X)
X = Activation('relu')(X)

Second component of main path (3 lines)
X = Conv2D(filters = F2, kernel_size = (f, f), strides = (1,1), padding = 'same', 1
X = BatchNormalization(axis = 3, name = bn_name_base + '2b')(X)
X = Activation('relu')(X)

Third component of main path (2 lines)
X = Conv2D(filters = F3, kernel_size = (1, 1), strides = (1,1), padding = 'valid',
X = BatchNormalization(axis = 3, name = bn_name_base + '2c')(X)

SHORTCUT PATH ##### (2 lines)
X_shortcut = Conv2D(filters = F3, kernel_size = (1, 1), strides = (s,s), padding =
 kernel_initializer = glorot_uniform(seed=0))(X_shortcut)
X_shortcut = BatchNormalization(axis = 3, name = bn_name_base + '1')(X_shortcut)

Final step: Add shortcut value to main path, and pass it through a RELU activati
X = Add()(X, X_shortcut)
X = Activation('relu')(X)

```

```

return X

def ResNet50(input_shape=(21, 21, 1), classes=2):
 """
 Implementation of the popular ResNet50 the following architecture:
 CONV2D -> BATCHNORM -> RELU -> MAXPOOL -> CONVBLOCK -> IDBLOCK*2 -> CONVBLOCK -> IDBLOCK*3
 -> CONVBLOCK -> IDBLOCK*5 -> CONVBLOCK -> IDBLOCK*2 -> AVGPPOOL -> TOPLAYER

 Arguments:
 input_shape -- shape of the images of the dataset
 classes -- integer, number of classes

 Returns:
 model -- a Model() instance in Keras
 """

 # Define the input as a tensor with shape input_shape
 X_input = Input(input_shape)

 # Zero-Padding
 X = ZeroPadding2D((3, 3))(X_input)

 # Stage 1
 X = Conv2D(64, (7, 7), strides=(2, 2), name='conv1', kernel_initializer=glorot_uniform(seed=0))(X)
 X = BatchNormalization(axis=3, name='bn_conv1')(X)
 X = Activation('relu')(X)
 X = MaxPooling2D((3, 3), strides=(2, 2))(X)

 # Stage 2
 X = convolutional_block(X, f=3, filters=[64, 64, 256], stage=2, block='a', s=1)
 X = identity_block(X, 3, [64, 64, 256], stage=2, block='b')
 X = identity_block(X, 3, [64, 64, 256], stage=2, block='c')

 ### START CODE HERE ###

 # Stage 3 (4 lines)
 X = convolutional_block(X, f = 3, filters = [128, 128, 512], stage = 3, block='a', s = 2)
 X = identity_block(X, 3, [128, 128, 512], stage=3, block='b')
 X = identity_block(X, 3, [128, 128, 512], stage=3, block='c')
 X = identity_block(X, 3, [128, 128, 512], stage=3, block='d')

 # Stage 4 (6 lines)
 X = convolutional_block(X, f = 3, filters = [256, 256, 1024], stage = 4, block='a', s = 2)
 X = identity_block(X, 3, [256, 256, 1024], stage=4, block='b')
 X = identity_block(X, 3, [256, 256, 1024], stage=4, block='c')

```

```

X = identity_block(X, 3, [256, 256, 1024], stage=4, block='d')
X = identity_block(X, 3, [256, 256, 1024], stage=4, block='e')
X = identity_block(X, 3, [256, 256, 1024], stage=4, block='f')

Stage 5 (3 lines)
X = convolutional_block(X, f = 3, filters = [512, 512, 2048], stage = 5, block='a')
X = identity_block(X, 3, [512, 512, 2048], stage=5, block='b')
X = identity_block(X, 3, [512, 512, 2048], stage=5, block='c')

AVGPOOL (1 line). Use "X = AveragePooling2D(...)(X)"
#X = AveragePooling2D((2,2), name="avg_pool")(X)

END CODE HERE

output layer
X = Flatten()(X)
X = Dense(classes, activation='softmax', name='fc' + str(classes), kernel_initializer='he_normal')(X)

Create model
model = Model(inputs = X_input, outputs = X, name='ResNet50')

return model

```

```

resnet50_model = ResNet50()
resnet50_model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'],
n_epochs=5)
resnet_model_history = resnet50_model.fit(data_train, class_train, epochs=n_epochs, batch_size=batch_size,
validation_data=(data_val, class_val), shuffle=True)

```

## Chapter 54

# Principal Component Analysis (PCA) - BRICS Tutorial

### 1. Introduction

Principal Component Analysis (PCA) is a powerful dimensionality reduction technique widely used in astronomy to:

Identify patterns in high-dimensional datasets (e.g., spectra, images, catalogs).

Remove noise or instrumental effects.

Visualize complex data in lower dimensions.

In this tutorial, we will apply PCA to an example dataset of galaxy spectra.

### 2. Setup

First, let's import the necessary Python libraries.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from astropy.utils.data import download_file
from astropy.io import fits
```

### 3. Loading Astronomical Data

We'll use galaxy spectra from the Sloan Digital Sky Survey (SDSS) as an example.

```
Download an example SDSS spectrum FITS file
sdss_url = "https://dr12.sdss.org/sas/dr12/sdss/spectro/redux/26/spectra/1323/spec-1323.fits"
file_path = download_file(sdss_url, cache=True, show_progress=True)

Open the FITS file and extract wavelength and flux
with fits.open(file_path) as hdul:
 flux = hdul[1].data['flux'] # Flux array
 wavelength = 10 ** hdul[1].data['loglam'] # Wavelength in Angstroms
```

### 3. Loading Astronomical Data

We'll use galaxy spectra from the Sloan Digital Sky Survey (SDSS) as an example.



## Chapter 55

# Download an example SDSS spectrum FITS file

```
sdss_url = "https://dr12.sdss.org/sas/dr12/sdss/spectro/redux/26/spectra/
1323/spec-1323-52797-0012.fits" file_path = download_file(sdss_url,
cache=True, show_progress=True)
```



## Chapter 56

# Open the FITS file and extract wavelength and flux

```
with fits.open(file_path) as hdul: flux = hdul[1].data['flux'] # Flux array wave-
length = 10 ** hdul[1].data['loglam'] # Wavelength in Angstroms
```

### 4. Preprocessing the Data

PCA requires standardized data (mean=0, variance=1).

```
Standardize the flux (important for PCA)
scaler = StandardScaler()
flux_scaled = scaler.fit_transform(flux.reshape(-1, 1)).flatten()

Plot the original and scaled spectrum
plt.figure(figsize=(10, 5))
plt.plot(wavelength, flux, label="Original Spectrum", alpha=0.7)
plt.plot(wavelength, flux_scaled, label="Scaled Spectrum", alpha=0.7)
plt.xlabel("Wavelength (Å)")
plt.ylabel("Flux")
plt.legend()
plt.title("Galaxy Spectrum (Original vs. Scaled)")
plt.show()
```

<Figure size 1000x500 with 1 Axes>

### 5. Applying PCA

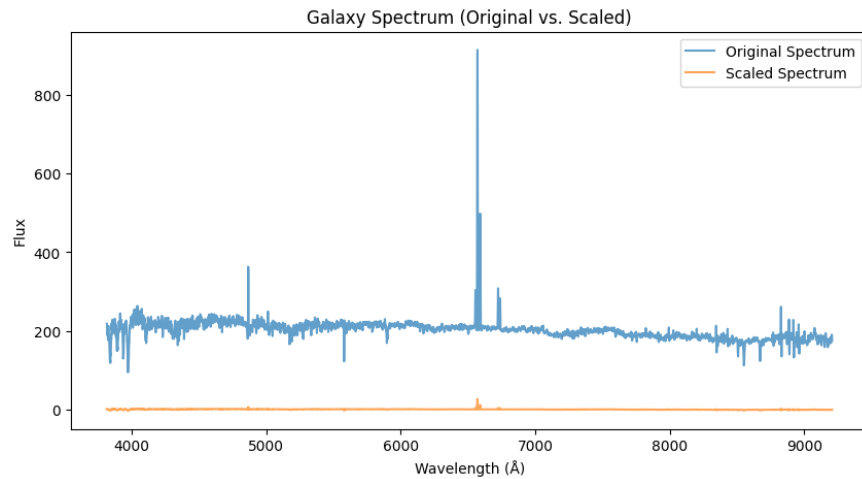


Figure 56.1: output

Now, let's apply PCA to a set of multiple spectra (for simplicity, we'll simulate a small dataset).

```
Simulate a dataset of 10 spectra with some noise and variations
np.random.seed(42)
n_spectra = 10
spectra = np.zeros((n_spectra, len(flux)))
for i in range(n_spectra):
 spectra[i] = flux + np.random.normal(0, 0.5, len(flux)) # Add noise

Standardize the dataset
spectra_scaled = StandardScaler().fit_transform(spectra)

Apply PCA
pca = PCA(n_components=2) # Keep the top 3 principal components
pca_result = pca.fit_transform(spectra_scaled)

Explained variance ratio
print("Explained variance ratio:", pca.explained_variance_ratio_)
```

```
Explained variance ratio: [0.12198097 0.11690865]
```

## 6. Visualizing PCA Results

### 6.1 PCA Components (Eigenspectra)

```
plt.figure(figsize=(10, 5))
for i in range(2):
 plt.plot(wavelength, pca.components_[i], label=f"PC {i+1}")
plt.xlabel("Wavelength (Å)")
plt.ylabel("Component Weight")
plt.legend()
plt.title("Principal Components (Eigenspectra)")
plt.show()
```

<Figure size 1000x500 with 1 Axes>

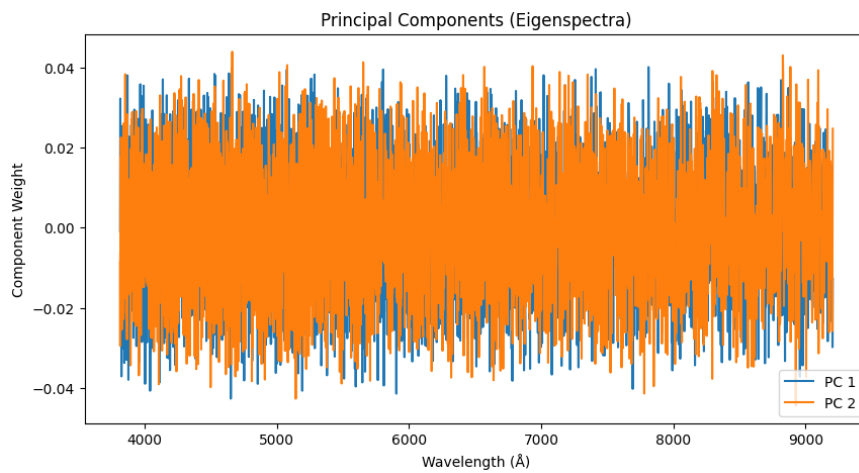


Figure 56.2: output

## 6.2 Projection of Data in PCA Space

```
plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], c=range(n_spectra), cmap='viridis')
plt.colorbar(label="Spectrum Index")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Galaxy Spectra in PCA Space")
plt.show()
```

<Figure size 800x600 with 2 Axes>

## 7. Interpretation

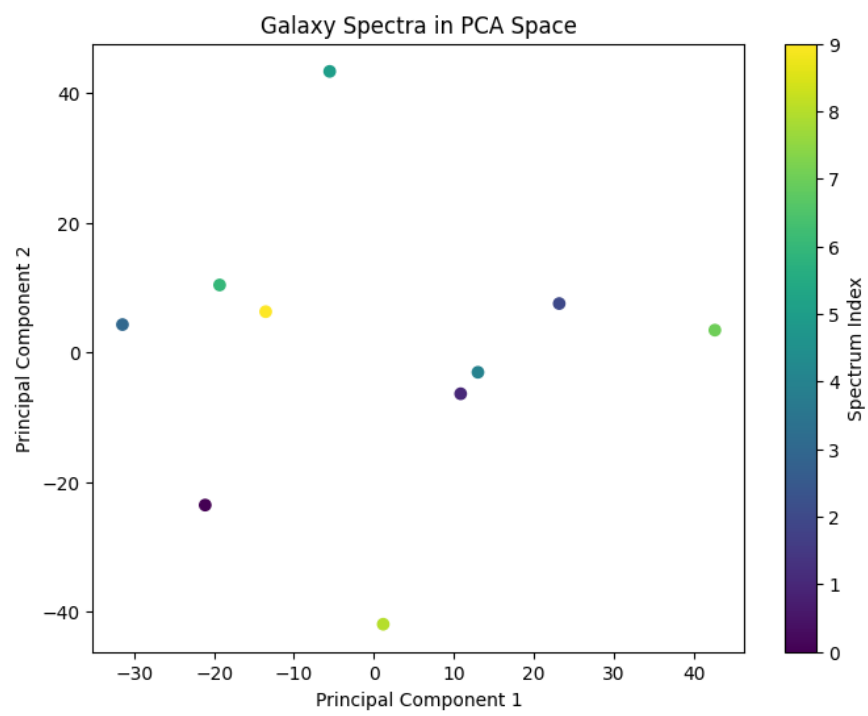


Figure 56.3: output

PC1 often captures the dominant continuum shape.

PC2 and PC3 may correspond to emission/absorption features.

The scatter plot helps identify outliers or clusters in the dataset.

## 8. Applications in Astronomy

PCA is used in:

Spectral Classification (e.g., stars, galaxies, quasars).

Noise Reduction (e.g., removing sky lines).

Feature Extraction (e.g., identifying important spectral lines).

## 9. Exercises for Practice

Try on Real Data: Download more SDSS spectra and apply PCA.

Reconstruction: Reconstruct spectra using only the top 2 PCs.

Other Datasets: Apply PCA to photometric data (e.g., galaxy colors).

```
Example: Reconstructing a spectrum using 2 PCs
pca_2d = PCA(n_components=2)
reconstructed = pca_2d.inverse_transform(pca_2d.fit_transform(spectra_scaled))

plt.plot(wavelength, spectra_scaled[0], label="Original")
plt.plot(wavelength, reconstructed[0], label="Reconstructed (2 PCs)")
plt.legend()
plt.xlabel("Wavelength (Å)")
plt.ylabel("Flux")
plt.title("Spectrum Reconstruction using PCA")
plt.show()
```

<Figure size 640x480 with 1 Axes>

## 10. Conclusion

PCA is a valuable tool for astronomers working with high-dimensional data.

This tutorial covered:

Standardizing data.

Computing principal components.

Visualizing results.

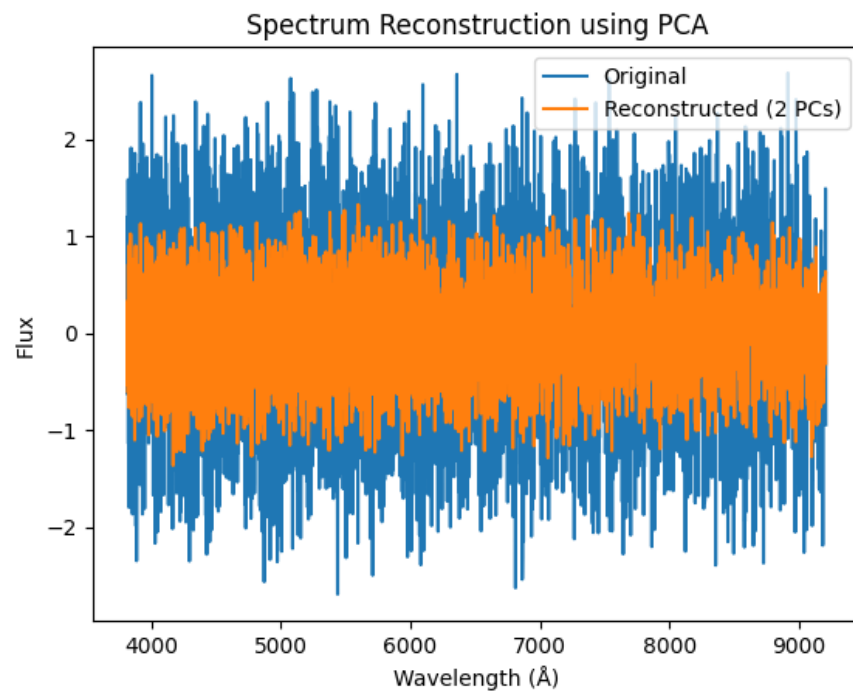


Figure 56.4: output



Try applying PCA to your own datasets!

Further Reading:

[Scikit-learn PCA Documentation](#)

[AstroML: PCA for Spectral Analysis](#)



## Chapter 57

# Capstone Project Guidelines – BRICS Astronomy/IDIA Data Analytics

### 57.1 Objective

The capstone project is an opportunity for participants to apply their data analytics skills to a research or real-world problem, ideally using astronomy-related data but open to other domains. Projects should demonstrate proficiency in data exploration, visualization, modeling (if applicable), and interpretation of results.

---

### 57.2 Project Requirements

#### 57.2.1 1. Project Topic

- Should involve a dataset that allows for meaningful analysis or insights.
- Can be astronomy-focused or from another domain such as climate, health, or economics.
- Participants are encouraged to select a topic of personal or regional relevance within the BRICS context.

### 57.2.2 2. Submission Format

- Submit a single **Google Colab notebook** containing all code, explanations, visualizations, and conclusions.
- Ensure the notebook is well-commented and can be followed by someone with no prior context.
- Share your final notebook as a **view-only link** (no editing permissions).

### 57.2.3 3. Notebook Structure

- Title & Objective
  - Dataset Description
  - Data Cleaning/Preprocessing
  - Exploratory Data Analysis (EDA)
  - Modeling/Analysis (if applicable)
  - Results & Interpretation
  - Conclusion & Future Work
  - References/Data Sources
- 

## 57.3 Timeline

- You will have **2–3 weeks** to complete the project.
  - Final deadline: Week 8
- 

## 57.4 Suggested Data Sources

### 57.4.1 Astronomy & Space Science

- ESA Gaia Archive
- NASA Exoplanet Archive
- Astroquery (Python)
- VizieR

### 57.4.2 Other Open Datasets

- Kaggle Datasets

- World Bank Open Data
  - Our World in Data
  - Google Dataset Search
  - UCI Machine Learning Repository
- 

## 57.5 Grading Rubric (100 points total)

Component	Points	Description
<b>Problem Definition</b>	10	Clearly defined research question and objectives.
<b>Dataset Selection &amp; Justification</b>	10	Relevance and appropriateness of chosen dataset.
<b>Data Cleaning &amp; Preprocessing</b>	15	Evidence of thoughtful preparation of data.
<b>Exploratory Data Analysis (EDA)</b>	15	Quality and depth of visualizations and insights.
<b>Modeling / Analysis</b>	20	Appropriate use of analytical or ML methods (if applicable).
<b>Interpretation &amp; Results</b>	15	Clarity and significance of insights drawn.
<b>Code Quality &amp; Reproducibility</b>	10	Well-commented, organized, and reproducible notebook.
<b>Presentation &amp; Structure</b>	5	Overall readability and flow of the notebook.

**Bonus:** Up to **5 bonus points** may be awarded for creativity, novelty, or interdisciplinary relevance.

## 57.6 Submission

- Submit your **view-only Colab notebook link** via the course platform.
  - Ensure the notebook **runs from top to bottom** without errors.
- 

If you have questions or need help selecting a dataset, feel free to ask during the live sessions or in the course Slack forum.